Практический анализ пакетов. Использование Wireshark для решения реальных сетевых задач. 3-е издание

Русский перевод практического руководства Криса Сандерса по анализу сетевых пакетов и диагностике реальных сетевых проблем с помощью Wireshark.

Больше крутых материалов в моём телеграм-канале @grogrigs

Благодарности

Я хотел бы выразить искреннюю признательность людям, которые поддерживали меня и работу над этой книгой.

Эллен, спасибо за твою безусловную любовь и за то, что бесчисленными ночами ты терпела, как я стучу по клавиатуре в постели, пока ты пытаешься уснуть.

Мама, даже после смерти твой пример доброты продолжает вдохновлять меня. Папа, от тебя я узнал, что такое тяжёлый труд, и без этого ничего бы не получилось.

Джейсон Смит, ты мне как брат, и я не могу в достаточной мере отблагодарить тебя за то, что ты всегда готов выслушать мои идеи.

Что касается моих бывших и нынешних коллег, мне очень повезло окружить себя людьми, которые сделали меня умнее и лучше. Я никак не смогу назвать всех, но хочу искренне поблагодарить Дастина, Алека, Мартина, Патрика, Криса, Майка и Грейди за ежедневную поддержку и за то, что они воплощают смысл служащего лидерства.

Спасибо Тайлеру Регули, который выступил главным техническим редактором. Иногда я совершаю глупые ошибки, а благодаря тебе выгляжу менее глупо. Спасибо также Дэвиду Вону за дополнительную проверку, Джеффу Карреллу за помощь в редактировании материалов об IPv6, Брэду Дункану за файл захвата, использованный в главе о безопасности, и команде QA Cafe за лицензию CloudShark, с помощью которой я организовал захваты пакетов для книги.

Разумеется, я также должен поблагодарить Джеральда Комбса и команду разработчиков Wireshark. Именно преданность Джеральда и сотен других разработчиков делает Wireshark такой прекрасной платформой анализа. Без их усилий положение в информационных технологиях и сетевой безопасности было бы значительно хуже.

Наконец, спасибо Биллу, Серене, Анне, Джен, Аманде, Элисон и остальным сотрудникам No Starch Press за усердие при редактировании и выпуске всех трёх изданий *Practical Packet Analysis*.

Введение

Третье издание *Practical Packet Analysis* создавалось и редактировалось на протяжении полутора лет, с конца 2015 до начала 2017 года, примерно через шесть лет после выхода второго издания и через десять лет после публикации оригинала. Книга содержит значительный объём нового материала, совершенно новые файлы захвата и сценарии, а также целиком новую главу об анализе пакетов в командной строке с помощью TShark и tcpdump. Если вам понравились первые два издания, понравится и это. Оно написано в том же тоне и даёт простые, понятные объяснения. Если вы не решались взяться за предыдущие издания, потому что в них не было самых свежих сведений о сетях или обновлениях Wireshark, вам стоит прочитать это: здесь расширены материалы о новых сетевых протоколах и обновлены сведения о Wireshark 2.x.

Почему эта книга?

Возможно, вы спрашиваете себя, почему стоит купить именно эту, а не какую-либо другую книгу об анализе пакетов. Ответ заключён в названии: *Практический анализ пакетов*. Посмотрим правде в глаза: ничто не заменит опыт реальной работы, а максимально приблизиться к нему в книге позволяют практические примеры с настоящими сценариями.

Первая половина книги даёт знания, необходимые для понимания анализа пакетов и Wireshark. Вторая целиком посвящена практическим случаям, которые легко могут встретиться при повседневном управлении сетью.

Кем бы вы ни были - сетевым техником, сетевым администратором, директором по информационным технологиям, специалистом по настольным системам или даже аналитиком сетевой безопасности, - понимание и применение описанных здесь методов анализа пакетов принесёт вам большую пользу.

Понятия и подход

В целом я очень спокойный человек, поэтому и объяснять понятия стараюсь непринуждённо. Это относится и к языку книги. В техническом жаргоне легко потеряться, но я изо всех сил старался сохранять максимально неформальный стиль. Все термины и понятия определены ясно и без лишней воды. В конце концов, я родом из великого штата Кентукки, поэтому стараюсь свести большие слова к минимуму. Но вам придётся простить некоторые деревенские выражения, встречающиеся в тексте.

Первые несколько глав неотделимы от понимания остальной книги, поэтому сначала обязательно освойте понятия на этих страницах. Вторая половина носит исключительно практический характер. Возможно, вы не увидите на работе точно таких сценариев, но сможете применить преподанные ими идеи в ситуациях, с которыми столкнётесь.

Ниже приведён краткий обзор содержания книги.

Глава 1. Анализ пакетов и основы сетей

Что такое анализ пакетов? Как он работает? Как его выполнять? В этой главе рассматриваются основы сетевого обмена и анализа пакетов.

Глава 2. Подключение к линии

В этой главе рассматриваются различные способы размещения анализатора пакетов в сети.

Глава 3. Введение в Wireshark

Здесь мы рассмотрим основы Wireshark: где его взять, как использовать, что он делает, чем хорош и всё остальное. В этом издании добавлено новое обсуждение настройки Wireshark с помощью профилей конфигурации.

Глава 4. Работа с захваченными пакетами

После запуска Wireshark вы захотите узнать, как взаимодействовать с захваченными пакетами. Здесь вы освоите основы, включая новые, более подробные разделы о прослеживании потоков пакетов и разрешении имён.

Глава 5. Расширенные возможности Wireshark

Научившись ползать, пора переходить к бегу. Эта глава углубляется в расширенные возможности Wireshark и заглядывает под капот, показывая некоторые менее очевидные операции. Сюда входят новые, более подробные разделы о прослеживании потоков пакетов и разрешении имён.

Глава 6. Анализ пакетов в командной строке

Wireshark прекрасен, но иногда приходится покинуть уют графического интерфейса и взаимодействовать с пакетом в командной строке. Эта новая глава показывает, как пользоваться TShark и tcpdump - двумя лучшими инструментами командной строки для анализа пакетов.

Глава 7. Протоколы сетевого уровня

В этой главе на примерах ARP, IPv4, IPv6 и ICMP показано, как распространённый обмен сетевого уровня выглядит на уровне пакетов. Чтобы устранять неполадки этих протоколов в реальных сценариях, сначала нужно понять их работу.

Глава 8. Протоколы транспортного уровня

Поднимаясь по стеку, эта глава обсуждает два самых распространённых транспортных протокола: TCP и UDP. Большинство исследуемых пакетов использует один из них, поэтому важно понимать, как они выглядят на уровне пакетов и чем отличаются.

Глава 9. Распространённые протоколы верхнего уровня

Продолжая обзор протоколов, эта глава показывает, как на уровне пакетов выглядят четыре самых распространённых протокола сетевого обмена верхнего уровня: HTTP, DNS, DHCP и SMTP.

Глава 10. Базовые реальные сценарии

Эта глава содержит разбор распространённого трафика и первый набор сценариев из реальной жизни. Каждый сценарий представлен в удобном формате: проблема, анализ и решение. В базовых сценариях участвует лишь несколько компьютеров и требуется ограниченный анализ - ровно столько, чтобы слегка намочить ноги.

Глава 11. Борьба с медленной сетью

Самые частые проблемы, о которых слышат сетевые специалисты, обычно связаны с низкой производительностью сети. Эта глава посвящена решению таких проблем.

Глава 12. Анализ пакетов для обеспечения безопасности

Сетевая безопасность - самая острая тема в области информационных технологий. Глава 12 показывает несколько сценариев решения связанных с безопасностью проблем посредством анализа пакетов.

Глава 13. Анализ беспроводных пакетов

Эта глава служит введением в анализ беспроводных пакетов. В ней обсуждаются различия между беспроводным и проводным анализом и приводятся примеры трафика беспроводной сети.

Приложение A. Дальнейшее чтение

Первое приложение предлагает другие справочные инструменты и сайты, которые могут пригодиться при дальнейшем применении изученных методов анализа пакетов.

Приложение B. Навигация по пакетам

Если вы хотите глубже разобраться в интерпретации отдельных пакетов, второе приложение даёт обзор хранения сведений пакета в двоичном виде и преобразования двоичных значений в шестнадцатеричную запись. Затем оно показывает, как с помощью схем пакетов разбирать пакеты, представленные в шестнадцатеричном виде. Это полезно, если вы собираетесь много анализировать пользовательские протоколы или применять инструменты командной строки.

Как пользоваться книгой

Я предполагал два способа использования книги:

• Как учебник. Читайте главу за главой, уделяя особое внимание реальным сценариям в поздних главах, чтобы получить представление об анализе пакетов.
• Как справочник. Некоторые функции Wireshark используются редко, поэтому можно забыть, как они работают. *Practical Packet Analysis* полезно держать на полке, чтобы при необходимости быстро освежить способ применения конкретной функции. Выполняя анализ пакетов по работе, вы можете обращаться к представленным здесь уникальным таблицам, схемам и методологиям.

О файлах захвата из примеров

Все использованные в книге файлы захвата доступны на странице книги No Starch Press: https://www.nostarch.com/packetanalysis3/. Чтобы получить от книги максимальную пользу, загрузите их и открывайте при прохождении примеров.

Rural Technology Fund

Я не мог написать введение, не упомянув лучшее, к чему привела *Practical Packet Analysis*. Вскоре после выхода первого издания я основал некоммерческую организацию Rural Technology Fund (RTF), имеющую в США статус 501(c)(3).

У учащихся из сельской местности, даже получающих отличные оценки, часто меньше возможностей познакомиться с технологиями, чем у их городских и пригородных сверстников. Основанный в 2008 году RTF воплощает одну из моих самых больших мечтаний. Он стремится сократить цифровой разрыв между сельскими сообществами и городскими и пригородными районами. Для этого RTF проводит целевые стипендиальные программы, участвует в жизни сообществ, жертвует учебные технологические ресурсы классам, а также в целом продвигает и защищает технологии в сельских районах и областях с высоким уровнем бедности.

В 2016 году RTF смог передать ресурсы для технологического образования более чем 10 000 учащихся сельских районов и областей с высоким уровнем бедности в США. Я рад сообщить, что все авторские доходы от этой книги направляются непосредственно RTF на поддержку этих целей. Подробнее о Rural Technology Fund и способах внести вклад можно узнать на сайте http://www.ruraltechfund.org/ или в Twitter: `@RuralTechFund`.

Как со мной связаться

Я всегда рад отзывам людей, которые читают мои работы. Если вы хотите связаться со мной по любой причине, отправляйте все вопросы, комментарии, угрозы и предложения руки и сердца непосредственно на `chris@chrissanders.org`. Я также регулярно пишу в блоге http://www.chrissanders.org/; в Twitter меня можно найти как `@chrissanders88`.

Глава 1. Анализ пакетов и основы сетей

Каждый день в компьютерной сети может случиться миллион разных неприятностей - от простого заражения шпионской программой до сложной ошибки в конфигурации маршрутизатора, - и немедленно устранить любую проблему невозможно. Самое большее, на что мы можем рассчитывать, - во всеоружии встретить такие ситуации, обладая необходимыми знаниями и инструментами.

Чтобы по-настоящему понять проблемы сети, нужно спуститься на уровень пакетов. Все сетевые проблемы берут начало именно там: на этом уровне даже самые привлекательные внешне приложения способны обнаружить отвратительное внутреннее устройство, а протоколы, которые кажутся заслуживающими доверия, могут оказаться вредоносными. Здесь от нас ничто не скрыто. Ничто не заслонено вводящими в заблуждение структурами меню, эффектной графикой или ненадёжными сотрудниками - настоящих секретов здесь нет (есть только зашифрованные). Чем больше мы умеем делать на уровне пакетов, тем лучше можем управлять своей сетью и решать проблемы. Таков мир анализа пакетов.

Эта книга сразу и без колебаний погружает вас в этот мир. На примерах реальных ситуаций вы научитесь разбираться с медленным сетевым обменом, выявлять узкие места приложений и даже выслеживать злоумышленников. Завершив чтение, вы сможете применять методы анализа пакетов, которые помогут решать даже самые трудные проблемы в вашей собственной сети.

В этой главе мы начнём с основ, сосредоточившись на сетевом обмене. Изложенный здесь материал даст вам инструменты, необходимые для изучения различных ситуаций.

Анализ пакетов и анализаторы пакетов

Анализ пакетов, который часто называют перехватом пакетов или анализом протоколов, - это процесс захвата и интерпретации передаваемых по сети данных в реальном времени, позволяющий лучше понять происходящее в сети. Обычно анализ пакетов выполняют с помощью анализатора пакетов - инструмента для захвата необработанных сетевых данных, проходящих по линии связи.

Анализ пакетов позволяет:

• понять характеристики сети;
• узнать, кто находится в сети;
• определить, кто или что использует доступную пропускную способность;
• выявить периоды пиковой нагрузки на сеть;
• обнаружить вредоносную активность;
• найти небезопасные и чрезмерно раздутые приложения.

Существуют различные программы для перехвата пакетов - как бесплатные, так и коммерческие. Каждая создавалась для своих целей. Среди популярных анализаторов пакетов можно назвать tcpdump, OmniPeek и Wireshark (в этой книге мы будем главным образом пользоваться Wireshark). OmniPeek и Wireshark имеют графический пользовательский интерфейс (GUI), тогда как tcpdump представляет собой программу командной строки.

Выбор анализатора пакетов

При выборе анализатора пакетов следует учитывать ряд факторов, перечисленных далее.

Поддерживаемые протоколы. Все анализаторы пакетов способны интерпретировать различные протоколы. Большинство понимает распространённые сетевые протоколы (например, IPv4 и ICMP), транспортные протоколы (например, TCP и UDP) и даже протоколы прикладного уровня (например, DNS и HTTP). Однако в них может отсутствовать поддержка нестандартных, новых или более сложных протоколов (например, IPv6, SMBv2 и SIP). Выбирая анализатор, убедитесь, что он поддерживает протоколы, которыми вы собираетесь пользоваться.

Удобство для пользователя. Оцените компоновку интерфейса анализатора пакетов, простоту установки и общий порядок работы. Выбранная программа должна соответствовать уровню вашей подготовки. Если опыта анализа пакетов у вас совсем немного, возможно, стоит избегать более сложных анализаторов командной строки, таких как tcpdump. С другой стороны, опытному специалисту по анализу пакетов сложная программа может оказаться полезнее. Набравшись опыта, вы, возможно, обнаружите, что в некоторых ситуациях удобно сочетать несколько программ перехвата пакетов.

Стоимость. Анализаторы пакетов хороши тем, что среди них есть много бесплатных программ, не уступающих коммерческим продуктам. Самое заметное различие между коммерческими продуктами и их бесплатными аналогами состоит в средствах подготовки отчётов. В коммерческие продукты обычно входит тот или иной эффектный модуль формирования отчётов, тогда как бесплатные приложения либо вовсе лишены такой возможности, либо предлагают лишь весьма ограниченные средства отчётности.

Поддержка программы. Даже освоив основы работы с программой перехвата, вы время от времени можете нуждаться в поддержке, чтобы решать новые проблемы по мере их появления. Оценивая доступную поддержку, ищите документацию для разработчиков, открытые форумы и списки рассылки. У бесплатных программ перехвата пакетов, таких как Wireshark, может не быть официальной коммерческой поддержки, однако сообщества пользователей и участников разработки часто предоставляют активные доски обсуждений, вики-сайты и блоги, помогающие получить от анализатора пакетов больше пользы.

Доступ к исходному коду. Некоторые анализаторы пакетов распространяются как программы с открытым исходным кодом. Это означает, что вы можете просматривать исходный код программы, а в некоторых случаях даже предлагать и вносить в него изменения. Такая возможность может быть привлекательной, если анализатор нужен вам для очень специфического или сложного варианта использования. Большинство коммерческих приложений доступа к исходному коду не предоставляет.

Поддержка операционных систем. К сожалению, не всякий анализатор пакетов поддерживает любую операционную систему. Выберите инструмент, который будет работать во всех операционных системах, требующих вашей поддержки. Если вы консультант, вам может понадобиться захватывать и анализировать пакеты в самых разных операционных системах, а значит, понадобится инструмент, работающий в большинстве из них. Помните также, что иногда пакеты захватывают на одной машине, а просматривают на другой. Различия между операционными системами могут вынудить вас применять на каждом устройстве отдельное приложение.

Как работают анализаторы пакетов

Перехват пакетов требует совместной работы программного и аппаратного обеспечения. Этот процесс можно разделить на три этапа:

1. **Сбор.** Сначала анализатор пакетов получает с линии связи необработанные двоичные данные. Обычно для этого выбранный сетевой интерфейс переводится в неразборчивый режим. В таком режиме сетевая плата может прослушивать весь трафик сегмента сети, а не только адресованный ей.
2. **Преобразование.** Затем захваченные двоичные данные преобразуются в удобочитаемый вид. На этом возможности большинства сложных анализаторов пакетов командной строки заканчиваются. На данном этапе сетевые данные допускают лишь самую общую интерпретацию, а основная часть анализа остаётся пользователю.

3. **Анализ.** Наконец, анализатор пакетов исследует захваченные и преобразованные данные. На основании извлечённой информации он определяет протокол захваченных сетевых данных и приступает к анализу характерных особенностей этого протокола.

Как взаимодействуют компьютеры

Чтобы в полной мере понимать анализ пакетов, нужно точно знать, как компьютеры взаимодействуют друг с другом. В этом разделе мы рассмотрим основы сетевых протоколов, модель взаимодействия открытых систем (Open Systems Interconnection, OSI), кадры сетевых данных и аппаратное обеспечение, которое поддерживает всё перечисленное.

Протоколы

Современные сети состоят из разнообразных систем, работающих на множестве разных платформ. Для взаимодействия между системами используется набор общих языков, называемых протоколами. К распространённым протоколам относятся протокол управления передачей (Transmission Control Protocol, TCP), интернет-протокол (Internet Protocol, IP), протокол разрешения адресов (Address Resolution Protocol, ARP) и протокол динамической конфигурации узла (Dynamic Host Configuration Protocol, DHCP). Логическая группа совместно работающих протоколов называется стеком протоколов.

Полезно представить, что протоколы подобны правилам, управляющим человеческим языком. В каждом языке есть правила спряжения глаголов, приветствия людей и даже правильного выражения благодарности. Протоколы действуют во многом так же: они позволяют определить, как следует маршрутизировать пакеты, устанавливать соединение и подтверждать получение данных.

В зависимости от назначения протокол может быть чрезвычайно простым или очень сложным. Хотя разные протоколы способны значительно отличаться друг от друга, многие из них решают следующие вопросы.

Установление соединения. Кто устанавливает соединение - клиент или сервер? Какой информацией необходимо обменяться до начала взаимодействия?

Согласование характеристик соединения. Шифруется ли обмен по этому протоколу? Как ключи шифрования передаются между взаимодействующими узлами?

Форматирование данных. Как организованы содержащиеся в пакете данные? В каком порядке их обрабатывают принимающие устройства?

Обнаружение и исправление ошибок. Что происходит, если пакет слишком долго идёт к месту назначения? Как клиент восстанавливает работу, если в течение короткого времени не может установить связь с сервером?

Завершение соединения. Как один узел сообщает другому об окончании взаимодействия? Какую заключительную информацию необходимо передать для корректного завершения обмена?

Семиуровневая модель OSI

В соответствии с эталонной моделью OSI, принятой в отрасли, протоколы разделяются по выполняемым функциям. Эта иерархическая модель, состоящая из семи отдельных уровней, очень полезна для понимания сетевого взаимодействия. На рисунке 1-1 справа показаны уровни модели OSI, а слева - правильные названия данных на каждом из этих уровней. Верхний, прикладной уровень представляет программы, используемые для доступа к сетевым ресурсам. Нижний уровень - физический; через него проходят сетевые данные. Протоколы каждого уровня работают совместно, обеспечивая правильную обработку данных протоколами непосредственно вышележащего и нижележащего уровней.

> **Примечание.** Первоначально модель OSI была опубликована Международной организацией по стандартизации (International Organization for Standardization, ISO) в 1983 году в документе ISO 7498. Модель OSI является всего лишь рекомендованным отраслевым стандартом. Разработчики протоколов не обязаны в точности ему следовать. Более того, OSI - не единственная сетевая модель: например, некоторые предпочитают модель Министерства обороны США (Department of Defense, DoD), также известную как модель TCP/IP.

Каждый уровень модели OSI выполняет определённую функцию.

Прикладной уровень (уровень 7). Самый верхний уровень модели OSI предоставляет пользователям средства доступа к сетевым ресурсам. Как правило, это единственный уровень, видимый конечным пользователям, поскольку он предоставляет интерфейс, лежащий в основе всей их работы с сетью.

Уровень представления (уровень 6). Этот уровень преобразует полученные данные в формат, который может прочитать прикладной уровень. Выполняемые здесь кодирование и декодирование данных зависят от протокола прикладного уровня, который отправляет или получает данные. Кроме того, уровень представления обрабатывает несколько видов шифрования и дешифрования, применяемых для защиты данных.

Сеансовый уровень (уровень 5). Этот уровень управляет диалогом, или сеансом, между двумя компьютерами. Он устанавливает соединение между всеми взаимодействующими устройствами, управляет им и завершает его. Сеансовый уровень также определяет, является ли соединение дуплексным (двусторонним) или полудуплексным (односторонним), и отвечает за корректное закрытие соединения между узлами вместо его резкого обрыва.

Транспортный уровень (уровень 4). Основное назначение транспортного уровня - предоставлять нижележащим уровням услуги надёжной передачи данных. Благодаря управлению потоком, сегментации и обратной сборке сегментов, а также контролю ошибок транспортный уровень обеспечивает безошибочную доставку данных из одной точки в другую.

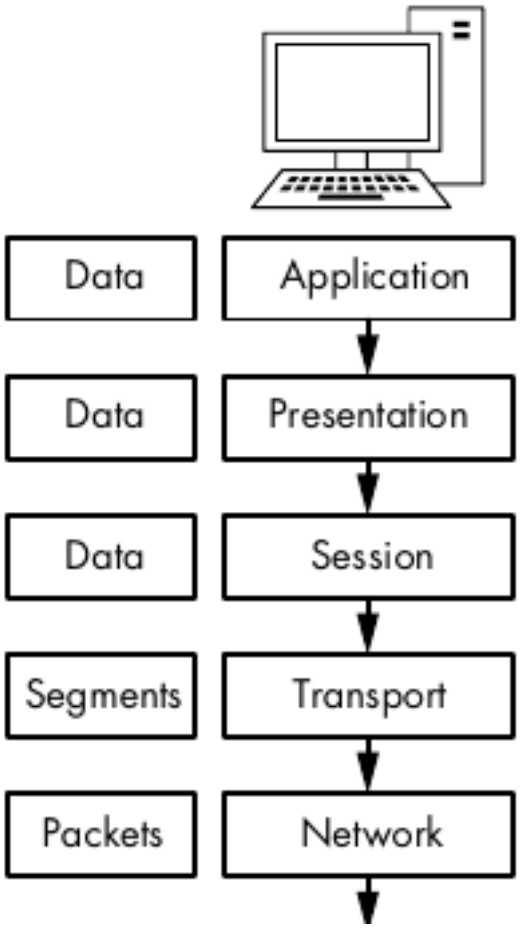


Рисунок 1-1. Иерархическое представление семи уровней модели OSI

Поскольку обеспечение надёжной передачи данных может быть чрезвычайно трудоёмким, в модели OSI ему посвящён целый уровень. Транспортный уровень использует как протоколы с установлением соединения, так и протоколы без установления соединения. На этом уровне работают некоторые межсетевые экраны и прокси-серверы.

Сетевой уровень (уровень 3). Этот уровень, один из самых сложных в модели OSI, отвечает за маршрутизацию данных между физическими сетями. Он обеспечивает логическую адресацию сетевых узлов (например, посредством IP-адресов). Кроме того, он разбивает потоки данных на более мелкие фрагменты и в некоторых случаях обнаруживает ошибки. На этом уровне работают маршрутизаторы.

Канальный уровень (уровень 2). Этот уровень предоставляет средства передачи данных по физической сети. Его основное назначение - обеспечить схему адресации, позволяющую идентифицировать физические устройства (например, по MAC-адресам). Мосты и коммутаторы являются физическими устройствами, работающими на канальном уровне.

Физический уровень (уровень 1). Находящийся внизу модели OSI уровень представляет физическую среду, по которой передаются сетевые данные. Он определяет физические и электрические характеристики всего используемого оборудования, включая напряжения, концентраторы, сетевые адаптеры, повторители и спецификации кабелей. Физический уровень устанавливает и завершает соединения, предоставляет средства совместного использования ресурсов связи и преобразует цифровые сигналы в аналоговые и обратно.

> **Примечание.** Распространённая английская мнемоническая фраза для запоминания уровней модели OSI - *Please Do Not Throw Sausage Pizza Away*. Первая буква каждого слова соответствует очередному уровню модели OSI, начиная с первого.

В таблице 1-1 перечислены некоторые наиболее распространённые протоколы каждого уровня модели OSI.

Таблица 1-1. Типичные протоколы на каждом уровне модели OSI

Уровень	Протоколы
Прикладной	HTTP, SMTP, FTP, Telnet
Представления	ASCII, MPEG, JPEG, MIDI
Сеансовый	NetBIOS, SAP, SDP, NWLink
Транспортный	TCP, UDP, SPX
Сетевой	IP, IPX
Канальный	Ethernet, Token Ring, FDDI, AppleTalk
Физический	проводные, беспроводные

Хотя модель OSI представляет собой всего лишь рекомендуемый стандарт, её следует знать наизусть, поскольку она даёт удобный словарь для осмысления и описания сетевых проблем. По мере чтения книги вы увидите, как неполадки маршрутизаторов быстро становятся «проблемами уровня 3», а неполадки программ легко распознаются как «проблемы уровня 7».

> **Примечание.** Однажды коллега рассказал мне о пользователе, который жаловался, что не может получить доступ к сетевому ресурсу. Причиной оказался неверно введённый пользователем пароль. Коллега назвал это проблемой уровня 8. Уровень 8 - неофициальный пользовательский уровень. Этот термин часто употребляют те, кто живёт на уровне пакетов.

Прохождение данных через модель OSI

Передача данных по сети начинается на прикладном уровне отправляющей системы. Данные последовательно спускаются через семь уровней модели OSI до физического уровня, после чего физический уровень отправляющей системы передаёт их принимающей системе. Принимающая система получает данные на физическом уровне, а затем они поднимаются по её уровням до расположенного наверху прикладного уровня.

Каждый уровень модели OSI способен взаимодействовать только с уровнями, находящимися непосредственно над ним и под ним. Например, уровень 2 может отправлять и получать данные только от уровней 1 и 3.

Ни одна из услуг, предоставляемых различными протоколами на определённом уровне OSI, не дублируется. Например, если протокол одного уровня предоставляет определённую услугу, никакой протокол другого уровня не будет предоставлять ту же услугу. У протоколов разных уровней могут быть функции со схожими целями, но работать они будут несколько по-разному.

Протоколы соответствующих уровней отправляющего и принимающего устройств дополняют друг друга. Например, если протокол уровня 7 отправляющего устройства отвечает за форматирование передаваемых данных, то соответствующий протокол уровня 7 принимающего устройства должен уметь читать эти отформатированные данные.

На рисунке 1-2 модель OSI представлена графически применительно к двум взаимодействующим устройствам. Можно увидеть, как на одном устройстве обмен идёт сверху вниз, а после достижения второго устройства направление меняется на обратное.

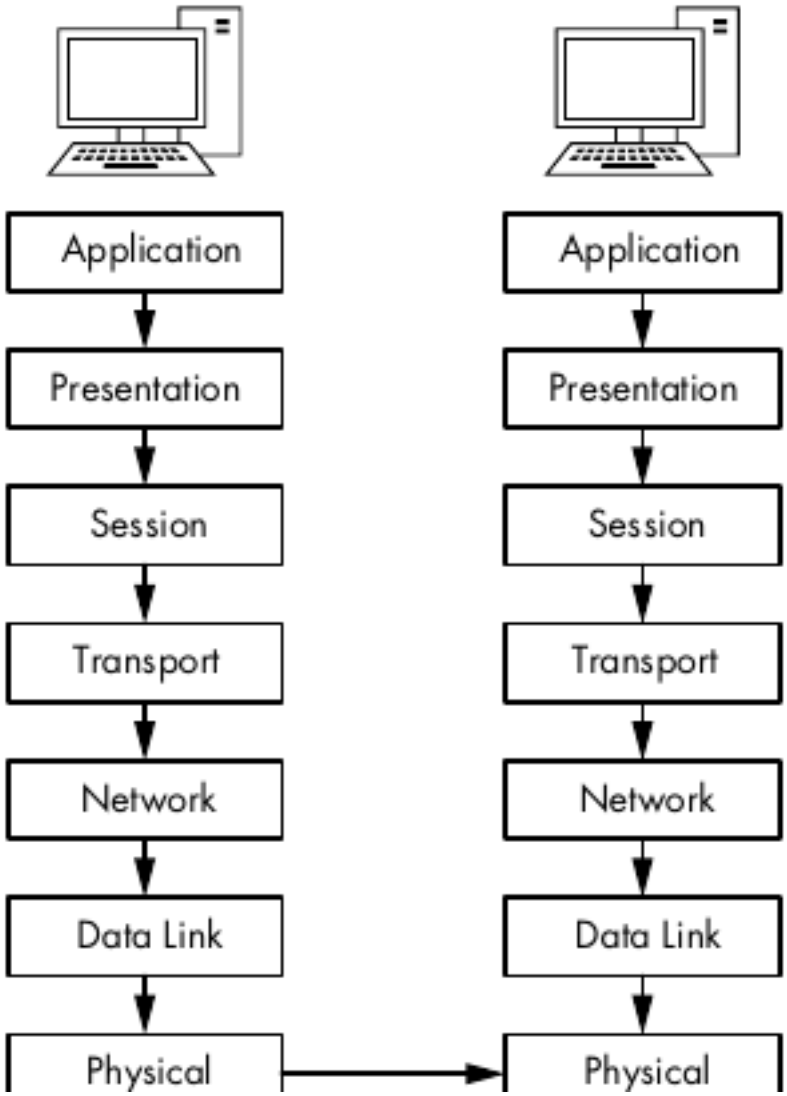


Рисунок 1-2. Протоколы, работающие на одном и том же уровне отправляющей и принимающей систем

Инкапсуляция данных

Протоколы разных уровней модели OSI передают данные друг другу с помощью инкапсуляции данных. Каждый уровень стека отвечает за добавление к передаваемым данным заголовка или концевика - дополнительных битов информации, позволяющих уровням взаимодействовать. Например, получив данные от сеансового уровня, транспортный уровень добавляет к ним собственную информацию заголовка, прежде чем передать их сетевому уровню.

В процессе инкапсуляции создаётся протокольный блок данных (protocol data unit, PDU), включающий отправляемые данные и всю добавленную к ним информацию заголовков и концевиков. По мере продвижения данных вниз по модели OSI и добавления заголовков и концевиков различными протоколами PDU изменяется и увеличивается. Окончательную форму PDU принимает на физическом уровне, после чего отправляется устройству назначения. На

принимающем устройстве заголовки и концевики протоколов удаляются из PDU по мере подъёма данных через уровни OSI в порядке, обратном порядку их добавления. Когда PDU достигает верхнего уровня модели OSI, от него остаются только исходные данные прикладного уровня.

> **Примечание.** В модели OSI для упакованных данных на каждом уровне используются особые термины. На физическом уровне находятся биты, на канальном - кадры, на сетевом - пакеты, а на транспортном - сегменты. Для трёх верхних уровней применяется просто термин «данные». На практике эта номенклатура используется редко, поэтому обычно мы будем называть пакетом полный или частичный PDU, содержащий информацию заголовков и концевиков нескольких или многих уровней модели OSI.

Чтобы показать, как работает инкапсуляция данных, рассмотрим упрощённый практический пример построения, передачи и получения пакета в контексте модели OSI. Помните, что аналитики редко говорят о сеансовом уровне или уровне представления, поэтому в этом примере (как и в остальной части книги) они отсутствуют.

В рассматриваемой ситуации мы пытаемся открыть google.com. Сначала необходимо сформировать пакет запроса, который передаётся от исходного клиентского компьютера конечному серверному компьютеру. Предполагается, что сеанс обмена по TCP/IP уже установлен. На рисунке 1-3 показан процесс инкапсуляции данных в этом примере.

Мы начинаем на клиентском компьютере с прикладного уровня. Поскольку мы открываем веб-сайт, используется протокол прикладного уровня HTTP; протокол HTTP выдаст команду загрузить файл `index.html` с сайта google.com.

> **Примечание.** На практике браузер сначала запросит корень документов веб-сайта, обозначаемый косой чертой (`/`). Получив этот запрос, веб-сервер перенаправит браузер к файлу, который настроен выдавать в ответ на запрос корня документов. Обычно это что-то вроде `index.html` или `index.php`. Подробнее мы рассмотрим это в главе 9 при обсуждении HTTP.

После того как протокол прикладного уровня отправил команду, наша задача - доставить пакет к месту назначения. Данные пакета передаются вниз по стеку OSI, к транспортному уровню. HTTP - протокол прикладного уровня, использующий TCP (или работающий поверх него), поэтому TCP выступает протоколом транспортного уровня, обеспечивающим надёжную доставку пакета. Формируется заголовок TCP, который добавляется к PDU, как показано на транспортном уровне рисунка 1-3. В заголовок TCP входят порядковые номера и другие добавляемые к пакету данные, обеспечивающие его правильную доставку.

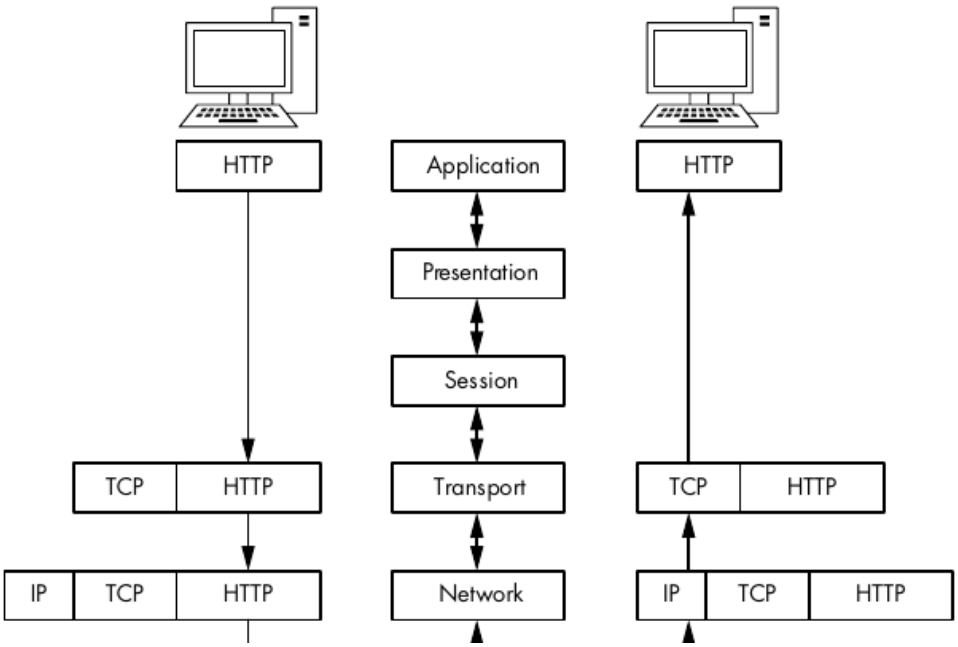


Рисунок 1-3. Графическое представление инкапсуляции данных между клиентом и сервером

> **Примечание.** Мы часто говорим, что один протокол «работает поверх» другого или «переносится» другим протоколом, поскольку модель OSI устроена сверху вниз. Такой протокол прикладного уровня, как HTTP, предоставляет определённую услугу и полагается на TCP в обеспечении её надёжной доставки. Обе эти службы зависят от протокола IP на сетевом уровне, который адресует и доставляет их данные. Таким образом, HTTP работает поверх TCP, а TCP - поверх IP.

Выполнив свою работу, TCP передаёт пакет протоколу IP - протоколу уровня 3, отвечающему за логическую адресацию пакета. IP создаёт содержащий логическую адресную информацию заголовок, добавляет его к PDU и передаёт пакет протоколу Ethernet на канальном уровне. Физические адреса Ethernet хранятся в заголовке Ethernet. Теперь пакет полностью собран и передаётся физическому уровню, где отправляется по сети в виде нулей и единиц.

Готовый пакет проходит по кабельной системе сети и в конце концов достигает веб-сервера Google. Веб-сервер начинает читать пакет снизу вверх: сначала он считывает данные канального уровня, содержащие физическую адресную информацию Ethernet, по которой сетевая плата определяет, что пакет предназначен конкретному серверу. После обработки этой информации сведения уровня 2 удаляются, и начинается обработка сведений уровня 3. Адресная информация IP уровня 3 считывается, чтобы убедиться в правильной адресации пакета и в том, что он не фрагментирован. Затем эти данные также удаляются, чтобы можно было обработать следующий уровень.

Теперь считываются сведения TCP уровня 4, позволяющие проверить, что пакет пришёл в правильной последовательности. После этого заголовок уровня 4 удаляется, и остаются только данные прикладного уровня, которые можно передать приложению веб-сервера, обслуживающему сайт. В ответ на этот пакет клиента сервер должен отправить пакет подтверждения TCP, чтобы клиент узнал о получении своего запроса, а затем - файл `index.html`.

Все пакеты строятся и обрабатываются так, как описано в этом примере, независимо от используемых протоколов. В то же время помните: не каждый пакет в сети порождается протоколом прикладного уровня, поэтому вы будете встречать пакеты, содержащие сведения лишь протоколов уровня 2, 3 или 4.

Сетевое оборудование

Пора рассмотреть сетевое оборудование, которое и выполняет всю чёрную работу. Мы остановимся лишь на нескольких самых распространённых видах сетевого оборудования: концентраторах, коммутаторах и маршрутизаторах.

Концентраторы

Концентратор обычно представляет собой коробку с несколькими портами RJ-45, подобную концентратору NETGEAR на рисунке 1-4. Размеры концентраторов варьируются от совсем небольших устройств с четырьмя портами до крупных 48-портовых устройств, рассчитанных на установку в стойку в корпоративной среде.

Рисунок 1-4. Типичный четырёхпортовый концентратор Ethernet

Поскольку концентраторы способны порождать много ненужного сетевого трафика и работают только в полудуплексном режиме (они не могут одновременно отправлять и получать данные), в большинстве современных сетей и сетей с высокой плотностью устройств они обычно не используются; вместо них применяются коммутаторы, которые рассматриваются в следующем разделе. Тем не менее нужно понимать работу концентраторов, поскольку при анализе пакетов они очень важны для метода подключения через концентратор, описанного в главе 2.

Концентратор - всего лишь повторяющее устройство, работающее на физическом уровне модели OSI. Он принимает пакеты, поступившие на один порт, и передаёт (повторяет) их на все остальные порты устройства, а уже принимающее устройство решает, принять или отклонить каждый пакет. Например, если компьютер на порте 1 четырёхпортового концентратора должен отправить данные компьютеру на порте 2, концентратор передаёт эти пакеты на порты 2, 3 и 4. Клиенты, подключённые к портам 3 и 4, проверяют поле конечного адреса управления доступом к среде (Media Access Control, MAC) в заголовке Ethernet пакета, видят, что пакет предназначен не им, и отбрасывают его. На рисунке 1-5 приведён пример, в котором компьютер A передаёт данные компьютеру B. Когда компьютер A отправляет эти данные, их получают все компьютеры, подключённые к концентратору. Однако принимает данные только компьютер B, а остальные компьютеры их отбрасывают.

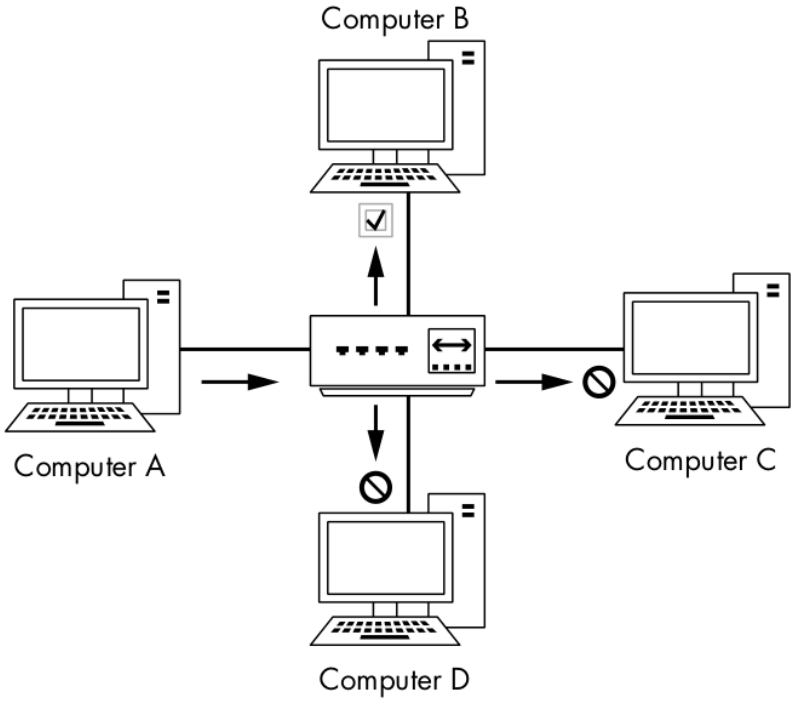


Рисунок 1-5. Поток трафика при передаче данных от компьютера A компьютеру B через концентратор

В качестве аналогии представьте, что вы отправили письмо с темой «Вниманию всех сотрудников отдела маркетинга» каждому сотруднику компании, а не только работающим в отделе маркетинга. Сотрудники отдела маркетинга видят, что письмо предназначено им, и открывают его. Остальные понимают, что оно предназначено не им, и удаляют. Легко увидеть, сколько ненужного трафика и напрасно потраченного времени породил бы такой способ общения, однако именно так действует концентратор.

Лучшая альтернатива концентраторам в рабочих сетях и сетях с высокой плотностью устройств - коммутаторы, то есть полнодуплексные устройства, способные синхронно отправлять и получать данные.

Коммутаторы

Подобно концентратору, коммутатор предназначен для повторения пакетов. Однако в отличие от концентратора он не рассылает данные на каждый порт, а отправляет их только тому компьютеру, которому они предназначены. Внешне коммутаторы похожи на концентраторы, как видно на рисунке 1-6.

Рисунок 1-6. Устанавливаемый в стойку 48-портовый коммутатор Ethernet

Некоторые представленные на рынке крупные коммутаторы, например устройства Cisco, управляются с помощью специализированных программ изготовителя или веб-интерфейсов. Такие коммутаторы обычно называют управляемыми. Управляемые коммутаторы предоставляют несколько полезных для управления сетью функций, в том числе позволяют включать и отключать отдельные порты, просматривать статистику портов, изменять конфигурацию и выполнять удалённую перезагрузку.

Кроме того, коммутаторы предлагают расширенные возможности обработки передаваемых пакетов. Чтобы напрямую взаимодействовать с определёнными устройствами, коммутаторы должны однозначно распознавать устройства по их MAC-адресам, а значит, они обязаны работать на канальном уровне модели OSI.

Коммутаторы хранят адрес уровня 2 каждого подключённого устройства в таблице CAM, играющей роль своеобразного регулировщика движения. При передаче пакета коммутатор считывает из него сведения заголовка уровня 2 и, сверяясь с таблицей CAM, определяет, на какой порт или порты отправить пакет. Коммутаторы передают пакеты только на определённые порты, тем самым значительно уменьшая сетевой трафик.

На рисунке 1-7 показано прохождение трафика через коммутатор. Компьютер A отправляет данные только предполагаемому получателю - компьютеру B. Одновременно в сети может происходить несколько обменов, но сведения передаются напрямую между коммутатором и предполагаемым получателем, а не между коммутатором и всеми подключёнными компьютерами.

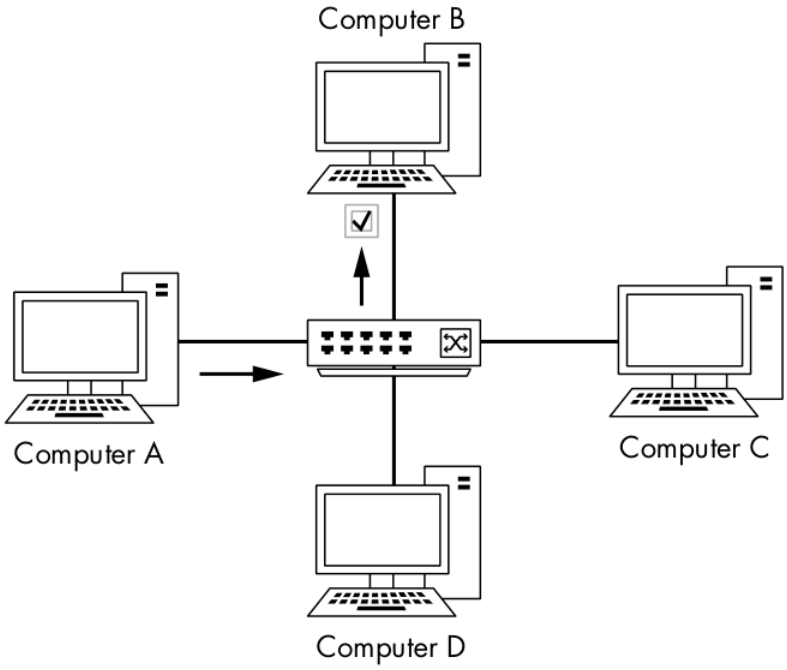


Рисунок 1-7. Поток трафика при передаче данных от компьютера A компьютеру B через коммутатор

Маршрутизаторы

Маршрутизатор - сложное сетевое устройство, функциональность которого находится на гораздо более высоком уровне, чем у коммутатора или концентратора. Маршрутизаторы бывают самых разных форм и размеров, однако у большинства устройств на передней панели имеется несколько светодиодных индикаторов, а на задней - несколько сетевых портов, число которых зависит от размера сети. На рисунке 1-8 приведён пример небольшого маршрутизатора.

Рисунок 1-8. Маршрутизатор Enterasys начального уровня, подходящий для небольшой или средней сети

Маршрутизаторы работают на уровне 3 модели OSI, где отвечают за пересылку пакетов между двумя или несколькими сетями. Процесс, посредством которого маршрутизаторы направляют

потоки трафика между сетями, называется маршрутизацией. Порядок маршрутизации пакетов разных типов в другие сети задают несколько видов протоколов маршрутизации. Для однозначной идентификации устройств в сети маршрутизаторы обычно используют адреса уровня 3, например IP-адреса.

Понятие маршрутизации удобно объяснить по аналогии с кварталом, в котором есть несколько улиц. Представьте, что дома с их адресами - это компьютеры, а каждая улица - сегмент сети. Эта аналогия показана на рисунке 1-9. Из своего дома вы можете без труда навестить соседей в других домах на той же улице, пройдя по прямой от собственной двери до их двери. Точно так же коммутатор обеспечивает взаимодействие всех компьютеров в одном сегменте сети.

Однако связь с соседом, живущим на другой улице, подобна связи с компьютером, находящимся в другом сегменте. Обратимся к рисунку 1-9 и предположим, что вы находитесь в доме 502 по Вайн-стрит и хотите попасть в дом 206 по Догвуд-лейн. Для этого нужно сначала повернуть на Оук-стрит, а затем - на Догвуд-лейн. Считайте эти повороты переходами между сегментами сети. Если устройство `192.168.0.3` должно связаться с устройством `192.168.0.54`, ему сначала придётся пройти через маршрутизатор, чтобы попасть в сеть `10.100.1.x`, а затем через маршрутизатор конечного сегмента сети, прежде чем оно достигнет этого сегмента.

Размер и количество маршрутизаторов в сети обычно зависят от размера и назначения сети. В личной сети или сети домашнего офиса может быть лишь небольшой маршрутизатор, расположенный на границе сети. В крупной корпоративной сети может быть несколько маршрутизаторов в разных подразделениях, соединённых с одним большим центральным маршрутизатором или коммутатором уровня 3 (усовершенствованным видом коммутатора со встроенными функциями маршрутизатора).

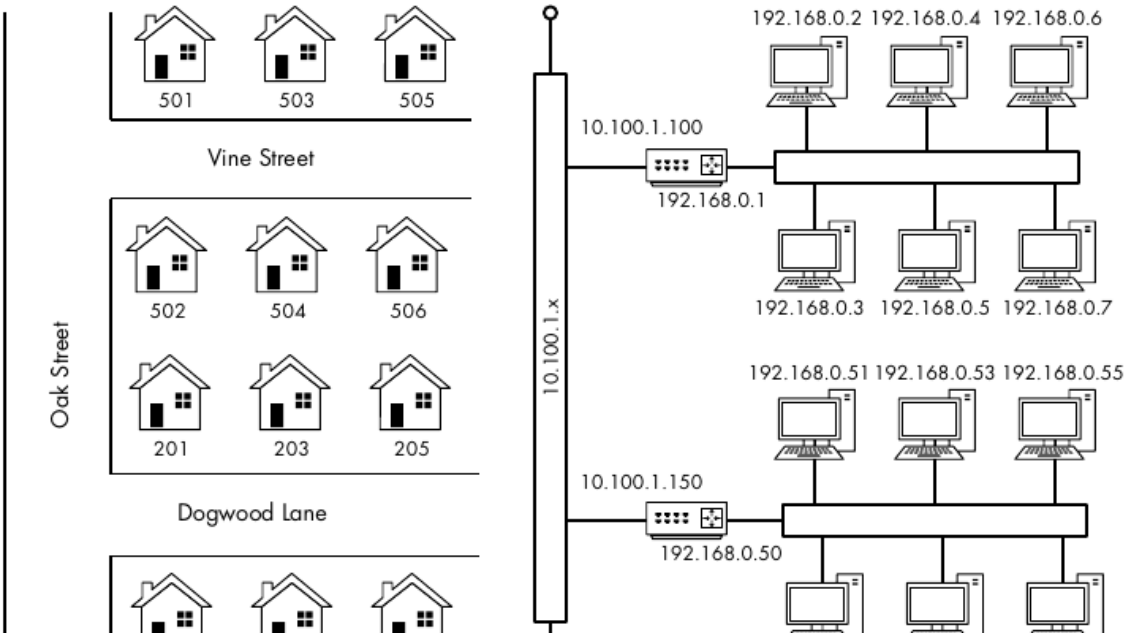


Рисунок 1-9. Сравнение маршрутизируемой сети с улицами квартала

Рассматривая всё больше сетевых схем, вы научитесь понимать, как данные проходят через различные точки. На рисунке 1-10 показано устройство весьма распространённого вида маршрутизируемой сети. В этом примере две отдельные сети соединены одним маршрутизатором. Если компьютер сети A хочет связаться с компьютером сети B, передаваемые данные должны пройти через маршрутизатор.

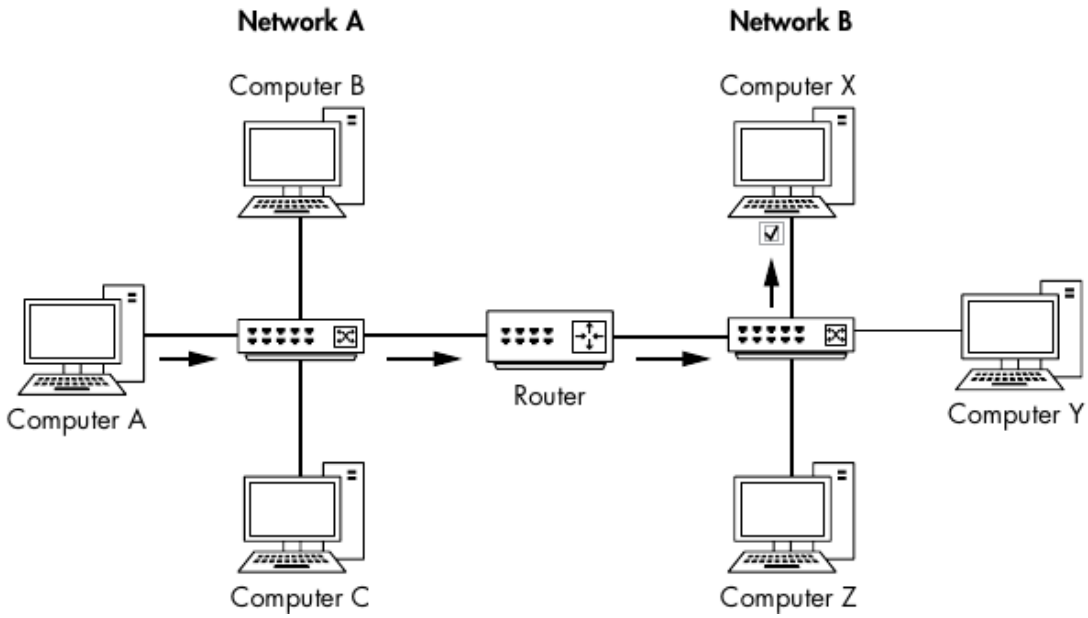


Рисунок 1-10. Поток трафика при передаче данных компьютером A из одной сети компьютеру X в другой сети через маршрутизатор

Классификация трафика

Сетевой трафик можно отнести к одному из трёх типов: широковещательному, многоадресному или одноадресному. У каждого типа есть особенность, определяющая, как сетевое оборудование обрабатывает относящиеся к нему пакеты.

Широковещательный трафик

Широковещательный пакет отправляется на все порты сегмента сети независимо от того, принадлежит ли конкретный порт концентратору или коммутатору.

Широковещательный трафик существует на уровнях 2 и 3. На уровне 2 MAC-адрес `ff:ff:ff:ff:ff:ff` зарезервирован как широковещательный, и любой отправленный на него трафик распространяется по всему сегменту сети. На уровне 3 также имеется определённый широковещательный адрес, но он зависит от используемого диапазона сетевых адресов.

Под широковещательный адрес резервируется наибольший возможный IP-адрес в диапазоне IP-сети. Например, если компьютер имеет адрес `192.168.0.20` и маску подсети `255.255.255.0`, широковещательным будет адрес `192.168.0.255` (подробнее об IP-адресации рассказано в главе 7).

Область, в пределах которой могут распространяться широковещательные пакеты, называется широковещательным доменом. Это сегмент сети, в котором любой компьютер может напрямую передавать данные другому компьютеру, не проходя через маршрутизатор. В крупных сетях с несколькими концентраторами или коммутаторами, соединёнными различными средами передачи, широковещательные пакеты от одного коммутатора достигают всех портов всех остальных коммутаторов сети, повторяясь от коммутатора к коммутатору. На рисунке 1-11 показаны два широковещательных домена в небольшой сети. Поскольку каждый широковещательный домен простирается до маршрутизатора, широковещательные пакеты циркулируют только в пределах

указанного широковещательного домена.

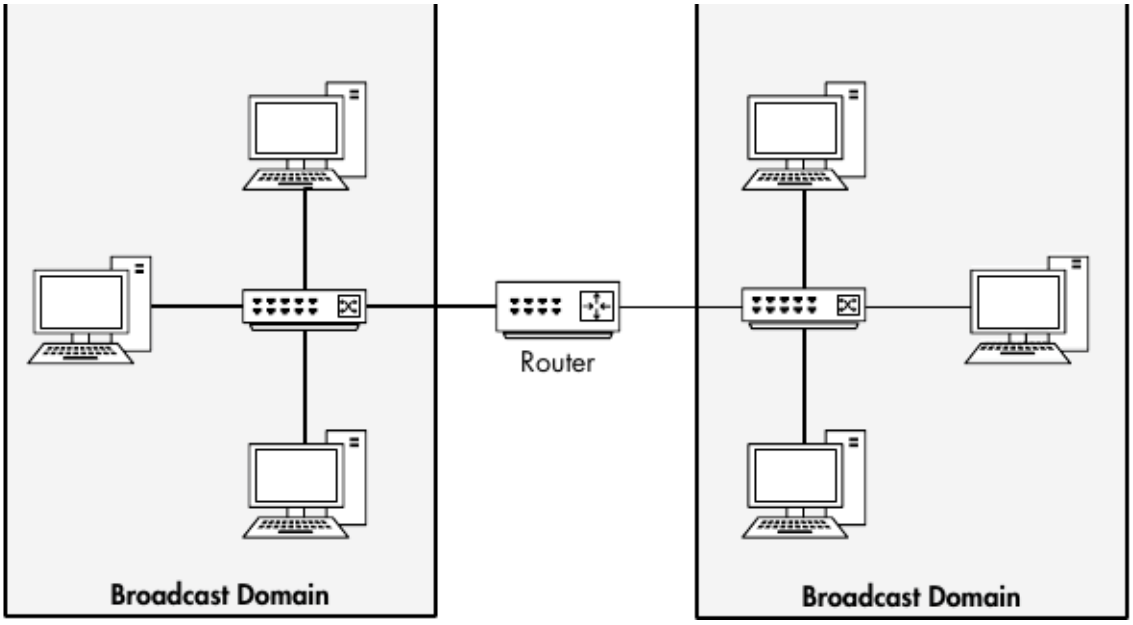


Рисунок 1-11. Широковещательный домен охватывает всё, что находится за текущим маршрутизируемым сегментом

Приведённая ранее аналогия с кварталом также хорошо объясняет работу широковещательных доменов. Широковещательный домен можно представить как улицу квартала, на которой все соседи сидят на крыльце. Если вы встанете на своё крыльцо и крикнете, люди на вашей улице вас услышат. Но если вы хотите поговорить с кем-то на другой улице, нужно найти способ обратиться непосредственно к этому человеку, а не вести широковещательную передачу (то есть кричать) со своего крыльца.

Многоадресный трафик

Многоадресная передача - способ одновременно отправить пакет от одного источника нескольким получателям. Её цель состоит в использовании как можно меньшей пропускной способности. Такой трафик оптимизируется благодаря тому, что по пути к получателям поток данных копируется меньшее число раз. Точная обработка многоадресного трафика в значительной степени зависит от её реализации в конкретных протоколах.

Основной способ реализации многоадресного трафика состоит в применении схемы адресации, объединяющей получателей пакетов в многоадресную группу. Именно так работает многоадресная передача IP. Такая схема адресации гарантирует, что пакеты не будут отправлены компьютерам, которым они не предназначены. Более того, для многоадресной передачи IP выделен целый диапазон адресов. Если вы видите IP-адрес от `224.0.0.0` до `239.255.255.255`, он, скорее всего, обслуживает многоадресный трафик, поскольку эти диапазоны зарезервированы именно для данной цели.

Одноадресный трафик

Одноадресный пакет передаётся напрямую от одного компьютера другому. Подробности одноадресной передачи зависят от использующего её протокола. Например, рассмотрим устройство, которое хочет связаться с веб-сервером. Это соединение между двумя участниками, поэтому обмен начинается с того, что клиентское устройство отправляет пакет только веб-серверу.

Заключение

В этой главе изложены основы сетей, которые необходимы как фундамент для анализа пакетов. Прежде чем приступать к поиску и устранению сетевых неполадок, следует понимать, что происходит на этом уровне сетевого взаимодействия. В главе 2 мы рассмотрим несколько способов захвата пакетов, которые требуется проанализировать.

Глава 2. Подключение к линии

Одно из ключевых решений для эффективного анализа пакетов состоит в том, где физически разместить анализатор, чтобы правильно захватывать данные. Аналитики пакетов часто называют размещение анализатора прослушиванием линии, подключением к сети или подключением к линии.

К сожалению, перехват пакетов не сводится к тому, чтобы подключить ноутбук к сетевому порту и захватить трафик. Более того, иногда разместить анализатор пакетов в сети труднее, чем собственно проанализировать пакеты. Размещение анализатора сложно потому, что устройства могут соединяться с помощью самого разного сетевого оборудования. На рисунке 2-1 показана типичная ситуация. Поскольку устройства современной сети - коммутаторы и маршрутизаторы - по-разному обрабатывают трафик, необходимо учитывать физическое устройство анализируемой сети.

Цель этой главы - помочь вам понять, как размещать анализатор пакетов в сетях с различной топологией. Но сначала выясним, благодаря чему мы способны видеть все пакеты, проходящие по линии, к которой подключились.

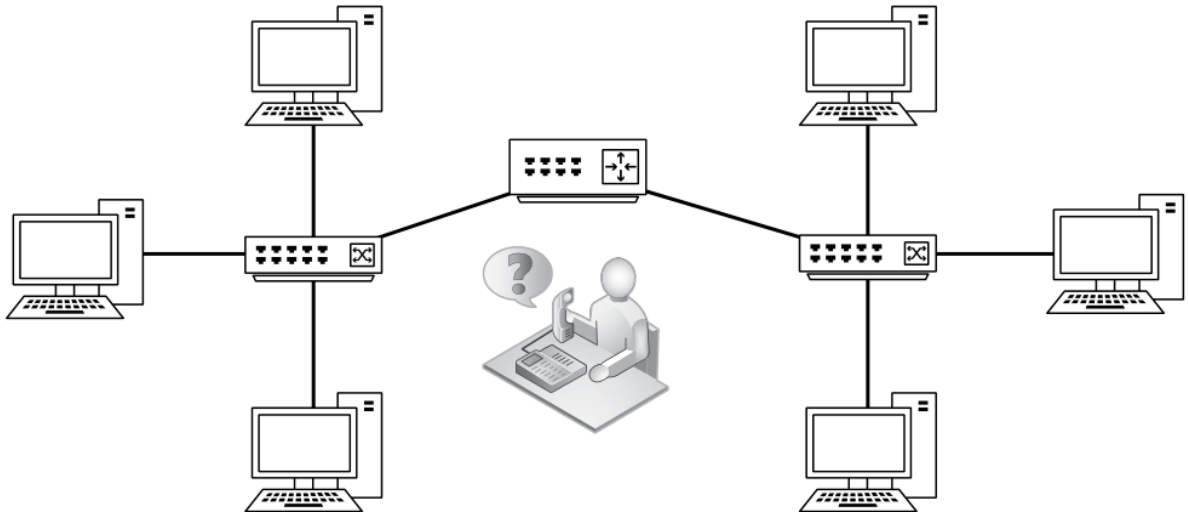


Рисунок 2-1. Когда в сети много соединений, разместить анализатор бывает трудно, а получить нужные данные - непросто

Жизнь в неразборчивом режиме

Прежде чем перехватывать пакеты в сети, необходимо иметь сетевую интерфейсную плату (network interface card, NIC), драйвер которой поддерживает неразборчивый режим. Именно неразборчивый режим позволяет NIC видеть все пакеты, проходящие по линии.

Как вы узнали из главы 1, при широковещательном сетевом обмене устройства часто получают пакеты, которые на самом деле им не предназначены. Например, протокол разрешения адресов (Address Resolution Protocol, ARP) - важнейшая составляющая любой сети, которую мы подробно изучим в главе 7, - используется для определения MAC-адреса, соответствующего определённому IP-адресу. Чтобы найти нужный MAC-адрес, устройство отправляет широковещательный пакет ARP каждому устройству своего широковещательного домена в надежде, что нужное устройство ответит.

Широковещательный домен - сегмент сети, в котором любой компьютер может напрямую передавать данные другому компьютеру, не проходя через маршрутизатор, - может включать несколько устройств, но отправленный широковещательный пакет ARP должен интересовать только правильное устройство-получатель в этом домене. Было бы ужасно неэффективно, если бы каждое устройство сети обрабатывало широковещательный пакет ARP. Вместо этого, если пакет не предназначен устройству и потому не представляет для него пользы, NIC устройства отбрасывает пакет, не передавая его центральному процессору для обработки.

Отбрасывание пакетов, не предназначенных принимающему узлу, повышает эффективность обработки, однако совсем не радует аналитиков пакетов. Аналитики обычно хотят захватывать каждый отправленный по линии пакет, чтобы не пропустить критически важную часть информации.

Гарантировать захват всего трафика позволяет неразборчивый режим NIC. Работая в таком режиме, NIC передаёт процессору узла каждый увиденный пакет независимо от его адресации. После того как пакет достигает центрального процессора, приложение для перехвата пакетов может получить его для анализа.

Большинство современных NIC поддерживает неразборчивый режим, а Wireshark включает драйвер libpcap/WinPcap, позволяющий переводить NIC в этот режим непосредственно из графического интерфейса Wireshark. (Подробнее о libpcap/WinPcap мы поговорим в главе 3.)

Для работы с этой книгой необходимы NIC и операционная система, поддерживающие неразборчивый режим. Единственный случай, когда перехватывать данные в неразборчивом режиме не требуется, - если вы хотите видеть только трафик, отправленный непосредственно на MAC-адрес интерфейса, с которого ведётся перехват.

> **Примечание.** Большинство операционных систем, включая Windows, не позволит использовать NIC в неразборчивом режиме без повышенных пользовательских привилегий. Если вы не можете законным путём получить такие привилегии в системе, скорее всего, вам вообще не следует перехватывать пакеты в данной сети.

Перехват в сетях с концентраторами

Перехват данных в сети с концентраторами - мечта любого аналитика пакетов. Как вы узнали из главы 1, отправленный через концентратор трафик проходит через каждый подключённый к нему порт. Поэтому для анализа трафика компьютера, подключённого к концентратору, достаточно подсоединить анализатор пакетов к свободному порту концентратора. Вы сможете видеть весь обмен с этим компьютером и от него, а также весь обмен между любыми другими устройствами, подключёнными к данному концентратору. Как показано на рисунке 2-2, когда анализатор подключён к сети на основе концентратора, окно видимости не имеет границ.

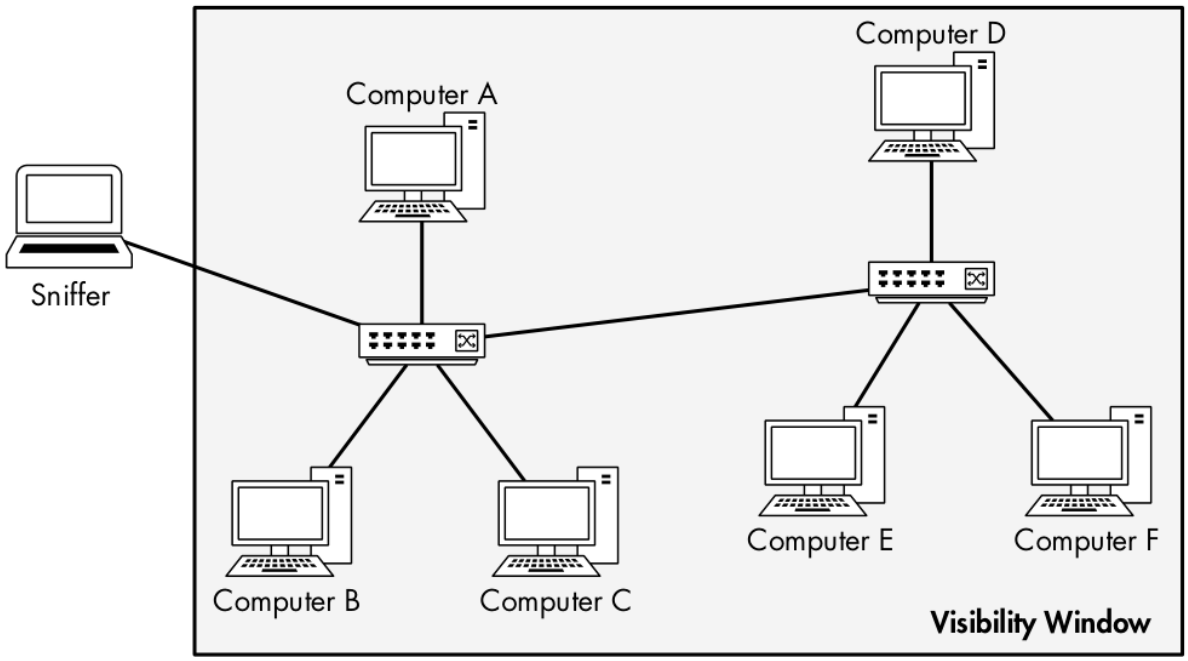


Рисунок 2-2. Перехват в сети на основе концентратора даёт неограниченное окно видимости

> **Примечание.** Окно видимости, показанное на различных схемах в этой книге, обозначает устройства сети, трафик которых можно видеть с помощью анализатора пакетов.

К несчастью для нас, сети на основе концентраторов редки из-за головной боли, которую они причиняют администраторам. Поскольку в каждый момент времени через концентратор может обмениваться данными только одно устройство, подключённому устройству приходится соперничать за пропускную способность со всеми остальными устройствами, пытающимися вести обмен. Когда два или более устройств передают данные одновременно, пакеты сталкиваются, как показано на рисунке 2-3. Это может привести к потере пакетов; чтобы компенсировать потерю, взаимодействующие устройства могут отправлять пакеты повторно, дополнительно перегружая сеть. По мере роста трафика и числа коллизий устройствам может понадобиться передавать пакет три или четыре раза, а производительность сети резко падает. Поэтому легко понять, почему в большинстве современных сетей любого размера используются коммутаторы. В наши дни концентраторы встречаются редко, но иногда их можно обнаружить в сетях, поддерживающих устаревшее оборудование или специализированные устройства, например в сетях промышленных систем управления (industrial control system, ICS).

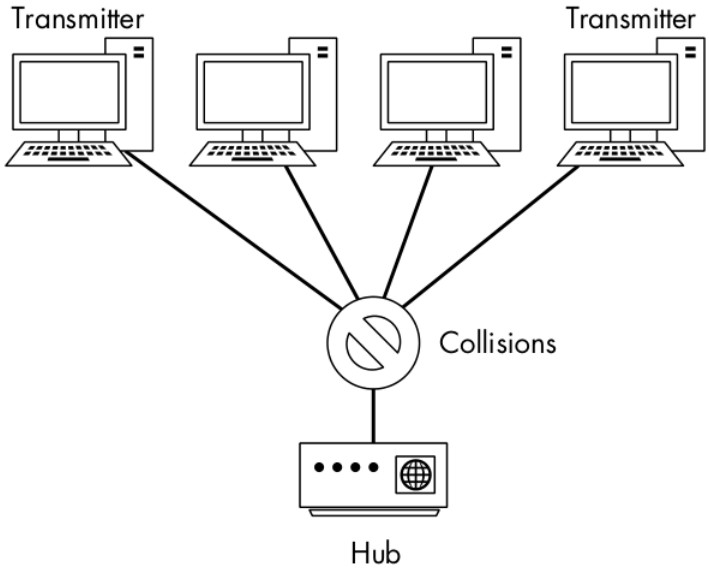


Рисунок 2-3. В сети на основе концентратора коллизии возникают, когда два или более устройств передают данные одновременно

Проще всего определить, используется ли в сети концентратор, заглянув в серверную или сетевой шкаф. На большинстве концентраторов прямо указано, что это за устройства. Если ничего больше не помогает, просто поищите в самом тёмном углу серверного шкафа сетевое оборудование, покрытое слоем пыли толщиной в несколько сантиметров.

Перехват в коммутируемой среде

Коммутаторы - самый распространённый вид соединительных устройств в современных сетях. Они обеспечивают эффективную передачу широковещательного, одноадресного и многоадресного трафика. Коммутаторы поддерживают полнодуплексный обмен, то есть машины могут одновременно отправлять и получать данные.

К сожалению для аналитиков пакетов, коммутаторы усложняют дело. Подключив анализатор к порту коммутатора, вы увидите только широковещательный трафик, а также трафик, который отправляет и получает устройство с установленным анализатором, как показано на рисунке 2-4. Чтобы захватить трафик целевого устройства в коммутируемой сети, необходимо выполнить дополнительное действие.

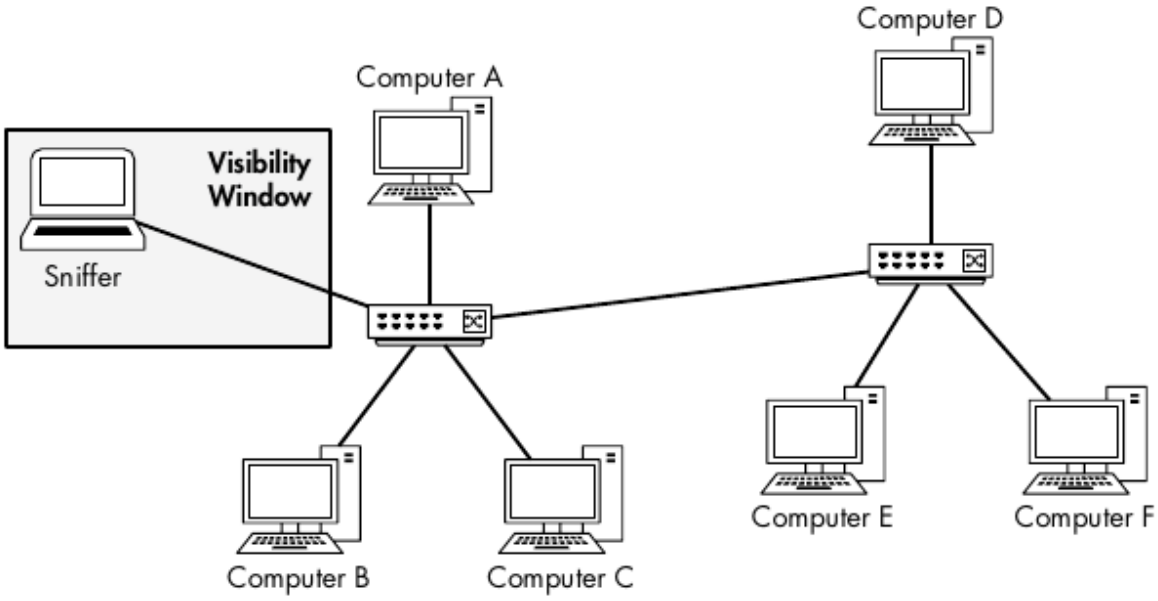


Рисунок 2-4. Окно видимости в коммутируемой сети ограничено портом, к которому вы подключены

Существует четыре основных способа захватить такой трафик: зеркалирование портов, подключение через концентратор, использование ответвителя и отравление кэша ARP.

Зеркалирование портов

Зеркалирование портов, также называемое дублированием портов, - пожалуй, самый простой способ захватить трафик целевого устройства в коммутируемой сети. При такой конфигурации необходимо иметь доступ к командной строке или веб-интерфейсу управления коммутатора, на котором находится целевой компьютер. Кроме того, коммутатор должен поддерживать зеркалирование портов и иметь свободный порт для подключения анализатора.

Чтобы включить зеркалирование, вы даёте коммутатору команду копировать весь трафик одного порта на другой. Например, чтобы захватывать весь трафик, отправляемый и принимаемый устройством на порте 3 коммутатора, можно подключить анализатор к порту 4 и зеркалировать порт 3 на порт 4. Зеркалирование портов показано на рисунке 2-5.

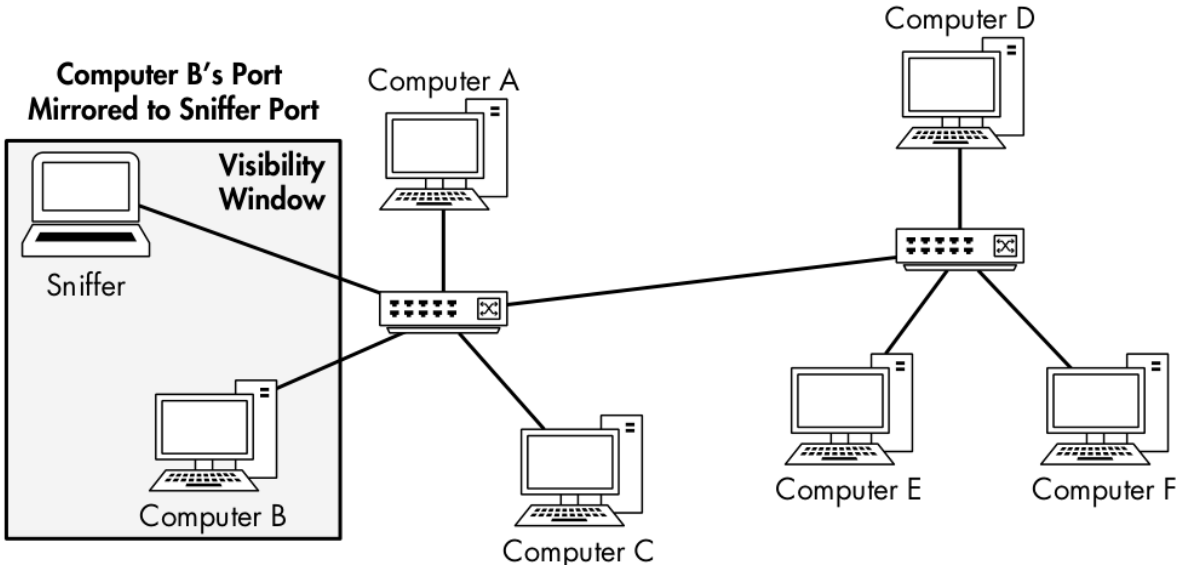


Рисунок 2-5. Зеркалирование портов позволяет расширить окно видимости в коммутируемой сети

Способ настройки зеркалирования зависит от изготовителя коммутатора. В большинстве корпоративных коммутаторов потребуется войти в интерфейс командной строки и настроить зеркалирование портов определённой командой. Примеры таких команд приведены в таблице 2-1.

Таблица 2-1. Команды для включения зеркалирования портов

Изготовитель	Команда
Cisco	`set span <source port> <destination port>`
Enterasys	`set port mirroring create <source port> <destination port>`
Nortel	`port-mirroring mode mirror-port <source port> monitor-port <destination port>`

> **Примечание.** Некоторые корпоративные коммутаторы предоставляют веб-интерфейс, в котором зеркалирование портов предлагается как один из вариантов, однако такие интерфейсы встречаются редко и не стандартизованы. Впрочем, если ваш коммутатор позволяет удобно настроить зеркалирование через графический интерфейс, непременно им воспользуйтесь. Кроме того, поддержка зеркалирования начинает появляться во всё большем числе коммутаторов для малых и домашних офисов (small office and home office, SOHO), и такие устройства обычно настраиваются через графический интерфейс.

При зеркалировании учитывайте пропускную способность зеркалируемых портов. Некоторые изготовители коммутаторов позволяют зеркалировать несколько портов на один; это может пригодиться при анализе обмена между двумя или несколькими устройствами на одном коммутаторе. Но давайте с помощью простых вычислений посмотрим, что способно произойти. Если в 24-портовом коммутаторе зеркалировать 23 полнодуплексных порта по 100 Мбит/с на один порт, к нему потенциально устремится поток 4600 Мбит/с. Это намного выше физического предела одного порта, поэтому при достижении трафиком определённого уровня возможны потеря пакетов или замедление сети. Иногда это называют переподпиской. Известны случаи, когда в подобных ситуациях коммутаторы полностью отбрасывали лишние пакеты или даже «приостанавливали» свои внутренние схемы, целиком прекращая обмен. Следите, чтобы при захвате не создавать таких проблем.

Зеркалирование портов может показаться привлекательным и недорогим решением для корпоративных сетей и ситуаций, в которых нужно постоянно наблюдать за определёнными сегментами, например при мониторинге сетевой безопасности. Однако обычно этот метод недостаточно надёжен для подобного применения. Особенно при высокой пропускной способности зеркалирование способно давать нестабильные результаты и вызывать труднообнаружимую потерю данных. В таких ситуациях рекомендуется применять ответвитель, который рассматривается в разделе «Использование ответвителя» на странице 24.

Подключение через концентратор

Ещё один способ захватить проходящий через целевое устройство трафик коммутируемой сети - подключиться через концентратор. При таком способе целевое устройство и систему анализа помещают в один сегмент сети, напрямую подключая оба к концентратору. Многие считают подключение через концентратор «жульничеством», но на самом деле это вполне допустимое решение, когда зеркалирование портов недоступно, а физический доступ к коммутатору целевого устройства имеется.

Для такого подключения нужны лишь концентратор и несколько сетевых кабелей. Собрав оборудование, соедините его следующим образом:

1. Найдите коммутатор, на котором находится целевое устройство, и отключите целевое устройство от сети.
2. Подключите сетевой кабель целевого устройства к концентратору.
3. Другим кабелем подключите анализатор к концентратору.
4. Чтобы подсоединить концентратор к сети, проложите сетевой кабель от него к сетевому коммутатору.

Теперь целевое устройство и анализатор находятся в одном широковещательном домене, а весь трафик целевого устройства будет передаваться широковещательно, благодаря чему анализатор сможет захватывать пакеты, как показано на рисунке 2-6.

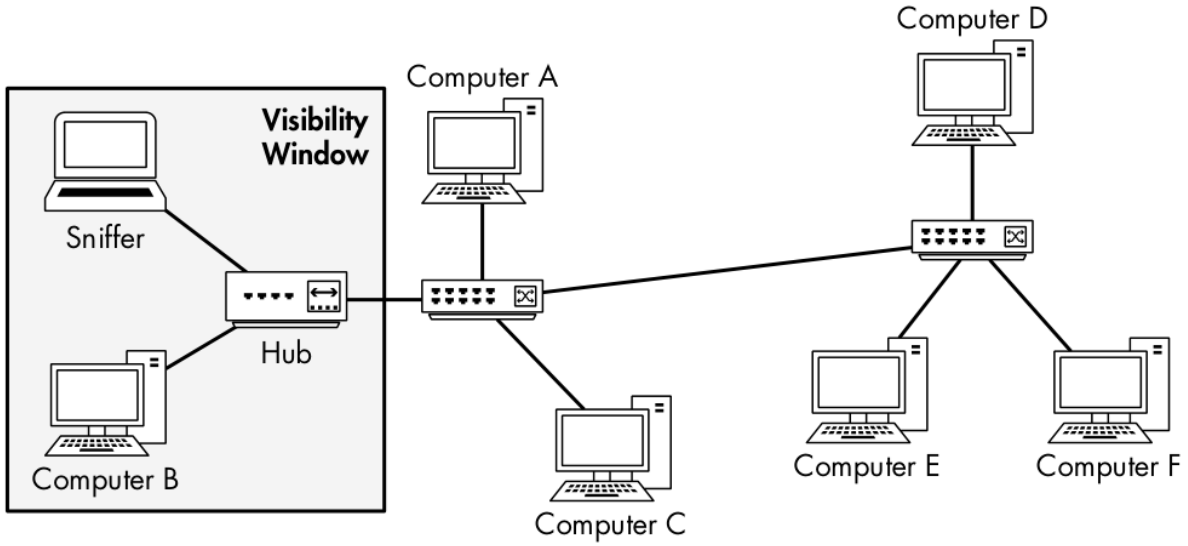


Рисунок 2-6. Подключение через концентратор изолирует целевое устройство и анализатор

В большинстве ситуаций подключение через концентратор переводит целевое устройство из полнодуплексного (двустороннего) режима в полудуплексный (односторонний). Это не самый изящный способ захвата пакетов, но иногда он остаётся единственным вариантом, если коммутатор не поддерживает зеркалирование портов. Помните, однако, что концентратору также

потребуется питание, источник которого бывает трудно найти.

> **Примечание.** Напомним, что обычно вежливо предупредить пользователя устройства, которое вы собираетесь отключить, особенно если этот пользователь - генеральный директор компании!

Использование ответвителя

Всем известна поговорка: «Зачем есть курицу, если можно съесть бифштекс?» (Или, если вы с Юга: «Зачем есть ливерный хлеб, если можно съесть жареную колбасу?») Тот же выбор возникает между подключением через концентратор и использованием ответвителя.

Сетевой ответвитель - аппаратное устройство, которое можно установить между двумя точками кабельной системы для захвата проходящих между ними пакетов. Как и при подключении через концентратор, в сеть помещается оборудование, позволяющее захватывать нужные пакеты. Разница в том, что вместо концентратора применяется специализированное аппаратное устройство, предназначенное для анализа сети.

Существует два основных вида сетевых ответвителей: агрегирующие и неагрегирующие. Устройства обоих типов устанавливаются между двумя устройствами для перехвата обмена. Главное различие состоит в том, что неагрегирующий ответвитель имеет четыре порта, как показано на рисунке 2-7, и для двунаправленного наблюдения за трафиком требует отдельных интерфейсов, тогда как у агрегирующего ответвителя лишь три порта и для двунаправленного наблюдения ему достаточно одного интерфейса.

Рисунок 2-7. Неагрегирующий ответвитель Barracuda

Как найти «настоящий» концентратор

При подключении через концентратор убедитесь, что используете настоящий концентратор, а не коммутатор под неверной этикеткой. Некоторые поставщики сетевого оборудования имеют дурную привычку рекламировать и продавать как «концентратор» устройство, которое на деле работает как коммутатор начального уровня. Если вы пользуетесь не проверенным и испытанным концентратором, то увидите только собственный трафик, но не трафик целевого устройства.

Найдя устройство, которое кажется концентратором, обязательно проверьте его. Лучший способ установить, настоящий ли это концентратор, - подключить к нему пару компьютеров и посмотреть, может ли один из них перехватывать обмен другого с различными устройствами сети, например с ещё одним компьютером или принтером. Если может, находку стоит оставить себе!

Поскольку концентраторы безнадёжно устарели, массово их почти не выпускают. Купить настоящий концентратор в обычном магазине практически невозможно, поэтому при поисках придётся проявить изобретательность. Отличным источником часто становятся распродажи излишков оборудования, которые проводит местный школьный округ. Прежде чем выбрасывать лишнее имущество, государственные школы обязаны попытаться продать его на аукционе, а старого оборудования у них нередко предостаточно. Я видел, как люди уходили с таких аукционов с несколькими концентраторами, заплатив меньше, чем за тарелку белой фасоли с кукурузным хлебом. Другим хорошим источником может быть eBay, но будьте осторожны: там вы можете столкнуться с той же проблемой неверно обозначенных коммутаторов.

Ответвители также обычно требуют подключения питания, хотя некоторые модели оснащены аккумуляторами, позволяющими недолго перехватывать пакеты.

Агрегирующие ответвители

Агрегирующий ответвитель проще всего в использовании. У него всего один физический порт наблюдения для перехвата двунаправленного трафика.

Чтобы с помощью агрегирующего ответвителя захватить весь входящий и исходящий трафик одного компьютера, подключённого к коммутатору, выполните следующие действия:

1. Отключите компьютер от коммутатора.
2. Подключите один конец сетевого кабеля к компьютеру, а другой - к входному порту ответвителя (`in`).
3. Подключите один конец другого сетевого кабеля к выходному порту ответвителя (`out`), а другой - к сетевому коммутатору.
4. Подключите один конец последнего кабеля к порту наблюдения ответвителя (`monitor`), а другой - к компьютеру, который выполняет роль анализатора.

Агрегирующий ответвитель следует подключить, как показано на рисунке 2-8. После этого анализатор должен захватывать весь входящий и исходящий трафик компьютера, подключённого к ответвителю.

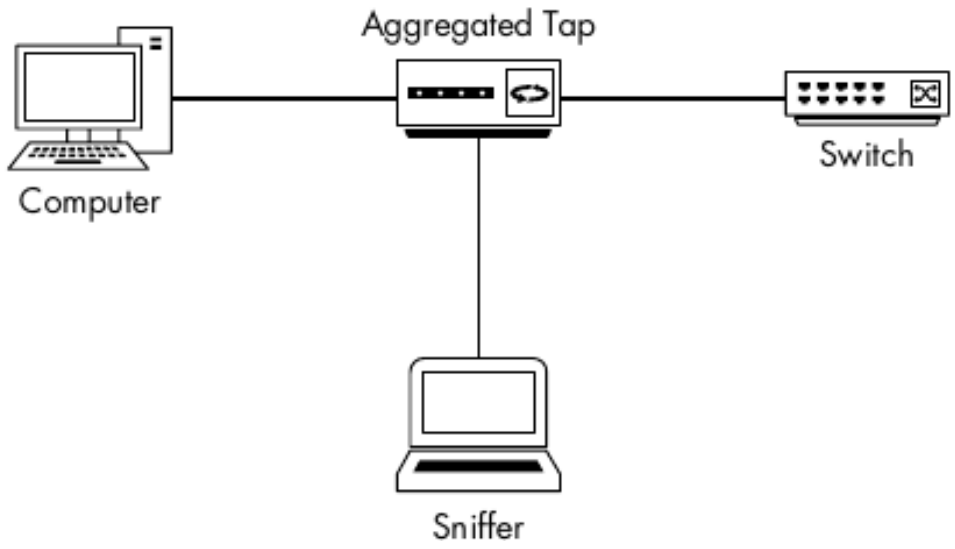


Рисунок 2-8. Использование агрегирующего ответвителя для перехвата сетевого трафика

Неагрегирующие ответвители

Неагрегирующий ответвитель немного сложнее агрегирующего, однако даёт чуть больше гибкости при захвате трафика. Вместо единственного порта наблюдения, через который можно прослушивать двунаправленный обмен, у неагрегирующего устройства есть два порта наблюдения. Один используется для перехвата трафика в одном направлении (от подключённого к ответвителю компьютера), а другой - для перехвата трафика в обратном направлении (к подключённому компьютеру).

Чтобы захватить весь входящий и исходящий трафик одного компьютера, подключённого к коммутатору, выполните следующие действия:

1. Отключите компьютер от коммутатора.
2. Подключите один конец сетевого кабеля к компьютеру, а другой - к входному порту ответвителя (`in`).
3. Подключите один конец другого сетевого кабеля к выходному порту ответвителя (`out`), а другой - к сетевому коммутатору.
4. Подключите один конец третьего сетевого кабеля к порту ответвителя `monitor A`, а другой - к одной из NIC компьютера, выполняющего роль анализатора.
5. Подключите один конец последнего кабеля к порту ответвителя `monitor B`, а другой - ко второй NIC компьютера, выполняющего роль анализатора.

Неагрегирующий ответвитель следует подключить, как показано на рисунке 2-9.

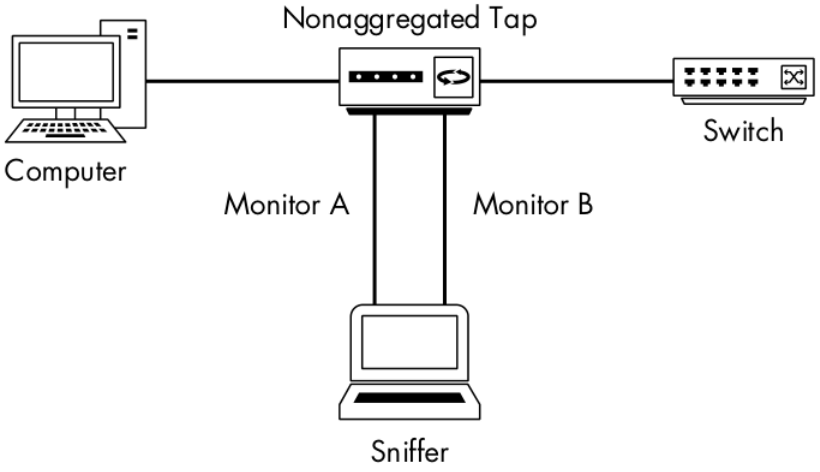


Рисунок 2-9. Использование неагрегирующего ответвителя для перехвата сетевого трафика

Из этих примеров может показаться, будто ответвитель позволяет наблюдать лишь за одним устройством, но при изобретательном выборе места установки с его помощью можно следить за множеством устройств. Например, если требуется наблюдать за всем обменом между целым сегментом сети и Интернетом, ответвитель можно поместить между коммутатором, к которому подключены все остальные устройства, и вышестоящим маршрутизатором сети. Размещение в таком узком месте сети позволит собирать нужный трафик. Эта стратегия часто применяется при мониторинге безопасности.

Выбор сетевого ответвителя

Какой ответвитель лучше? В большинстве ситуаций предпочтительны агрегирующие ответвители, потому что им нужно меньше кабелей и они не требуют двух NIC в компьютере-анализаторе. Однако для захвата большого объёма трафика или трафика только в одном направлении лучше выбрать неагрегирующий ответвитель.

Можно приобрести ответвители любого размера - от простых моделей Ethernet стоимостью около 150 долларов до корпоративных оптоволоконных устройств с шестизначной ценой. Я пользовался корпоративными ответвителями Ixia (прежде Net Optics), Dualcomm и Fluke Networks и остался ими очень доволен, но на рынке есть и множество других превосходных устройств. Если ответвитель применяется в корпоративной среде, убедитесь, что он поддерживает отказоустойчивое размыкание. Это означает, что даже при неисправности или отключении ответвителя пакеты продолжат проходить сквозь него, а сетевое соединение линии, к которой он подключён, не прервётся.

Отравление кэша ARP

Один из моих любимых способов подключиться к линии - отравление кэша ARP. В главе 7 мы подробно рассмотрим протокол ARP, но здесь необходимо дать краткое объяснение, чтобы вы поняли работу этого метода.

Процесс ARP

Вспомните из главы 1, что два основных вида адресации пакетов относятся к уровням 2 и 3 модели OSI. Адреса уровня 2, то есть MAC-адреса, используются совместно с применяемой вами системой адресации уровня 3. В соответствии с принятой в отрасли терминологией в этой книге я называю систему адресации уровня 3 системой IP-адресации.

Все устройства сети взаимодействуют на уровне 3, пользуясь IP-адресами. Поскольку коммутаторы работают на уровне 2 модели OSI, им известны только MAC-адреса уровня 2, поэтому устройства должны уметь включать эти сведения в создаваемые пакеты. Когда MAC-адрес неизвестен, его необходимо получить по известному IP-адресу уровня 3, чтобы трафик можно было переслать нужному устройству. Такое преобразование выполняет протокол уровня 2 ARP.

В сетях Ethernet процесс ARP начинается, когда один компьютер хочет связаться с другим. Передающий компьютер сначала проверяет свой кэш ARP: возможно, там уже есть MAC-адрес, связанный с IP-адресом компьютера назначения. Если его нет, компьютер отправляет запрос ARP на широковещательный адрес канального уровня `ff:ff:ff:ff:ff:ff`, который обсуждался в главе 1. Этот широковещательный пакет получает каждый компьютер соответствующего сегмента Ethernet. По существу, пакет спрашивает: «Какому IP-адресу принадлежит MAC-адрес `xx:xx:xx:xx:xx:xx`?»

Устройства, не имеющие IP-адреса компьютера назначения, просто отбрасывают запрос ARP. Целевая машина отвечает на пакет сообщением ARP со своим MAC-адресом. Теперь исходный передающий компьютер располагает адресной информацией канального уровня, необходимой для связи с удалённым компьютером, и сохраняет её в кэше ARP для быстрого извлечения.

Как работает отравление кэша ARP

Отравление кэша ARP, иногда называемое подменой ARP, - сложный способ подключения к линии в коммутируемой сети. Он основан на отправке коммутатору или маршрутизатору Ethernet сообщений ARP с поддельными MAC-адресами уровня 2 с целью перехватить трафик другого компьютера. Такая конфигурация показана на рисунке 2-10.

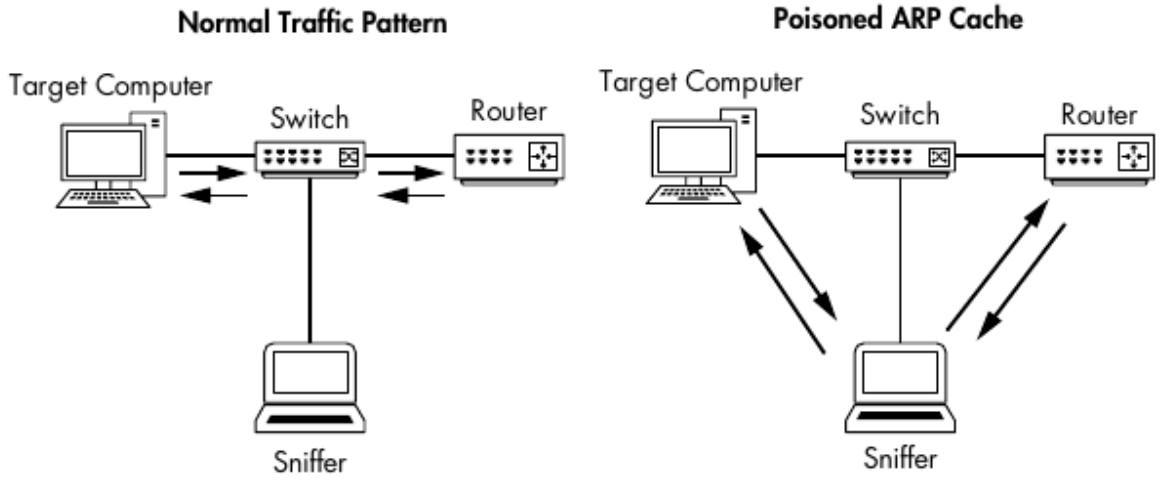


Рисунок 2-10. Отравление кэша ARP позволяет перехватывать трафик целевого компьютера

Злоумышленники часто применяют этот метод, чтобы отправлять клиентским системам пакеты с поддельными адресами и тем самым перехватывать определённый трафик или проводить атаки типа «отказ в обслуживании» (denial-of-service, DoS) против цели. Однако он может быть и

законным способом захватить пакеты целевой машины в коммутируемой сети.

Использование Cain & Abel

Первый шаг при попытке отравить кэш ARP - получить необходимые инструменты и собрать некоторые сведения. Для демонстрации мы воспользуемся популярным средством безопасности Cain & Abel с сайта oxid.it (oxid.it), которое работает в Windows. Скачайте и установите его, следуя указаниям на сайте.

> **Примечание.** Велика вероятность, что при попытке скачать Cain & Abel антивирусная программа или браузер пометит программу как вредоносную или как «хакерский инструмент». У этого средства много назначений, в том числе несколько злоумышленных. Для наших целей оно не представляет угрозы вашей системе.

Прежде чем пользоваться Cain & Abel, необходимо собрать определённые сведения, включая IP-адрес системы анализа, удалённой системы, трафик которой вы хотите захватить, и маршрутизатора, ниже которого находится удалённая система.

Впервые открыв Cain & Abel, вы увидите в верхней части окна ряд вкладок. (Отравление кэша ARP - лишь одна из возможностей Cain & Abel.) Для наших целей нужна вкладка Sniffer. После щелчка по ней должна появиться пустая таблица, показанная на рисунке 2-11.

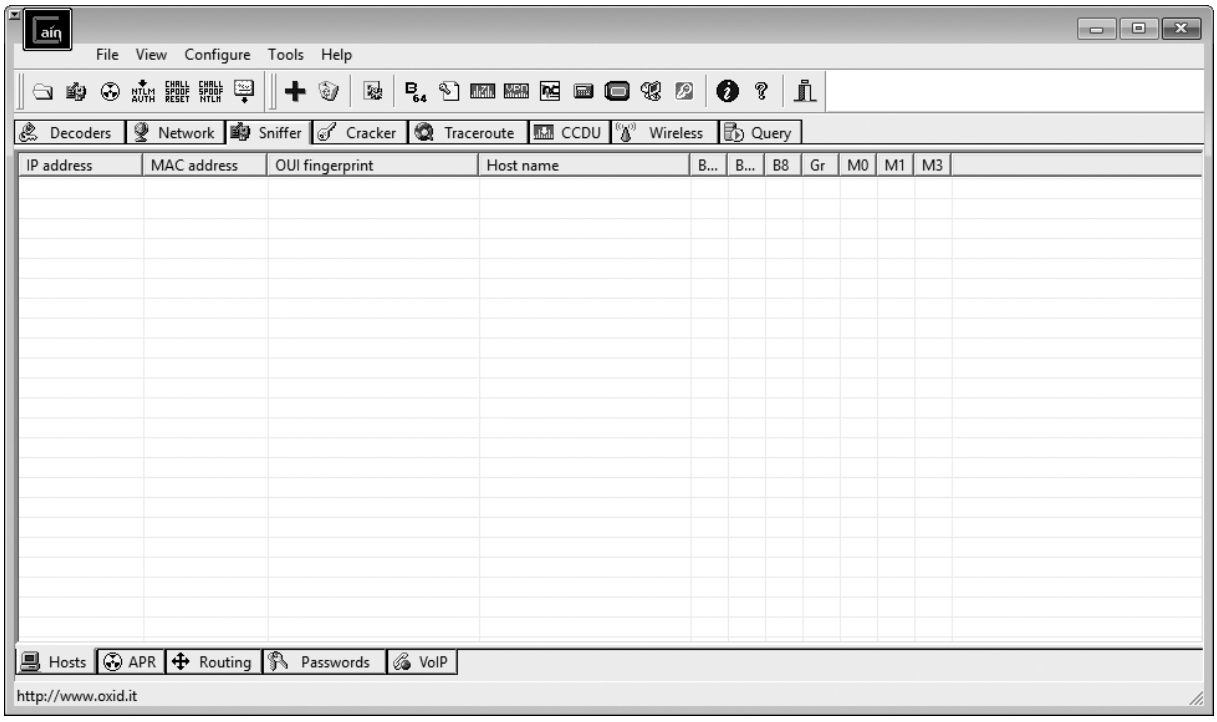


Рисунок 2-11. Вкладка Sniffer в главном окне Cain & Abel

Чтобы заполнить эту таблицу, нужно активировать встроенный анализатор программы и просканировать сеть в поисках узлов. Выполните следующие действия:

1. Щёлкните второй слева значок на панели инструментов, похожий на NIC.
2. Вам предложат выбрать интерфейс для перехвата. Выберите интерфейс, подключённый к сети, в которой вы собираетесь отравить кэш ARP. Если вы впервые пользуетесь Cain & Abel, выберите этот интерфейс и щёлкните OK. Если же вы уже выбирали интерфейс в Cain & Abel, выбор был сохранён и для выбора интерфейса понадобится ещё раз нажать значок NIC. (Убедитесь, что эта кнопка нажата: так включается встроенный анализатор Cain & Abel.)

3. Чтобы построить список доступных узлов сети, щёлкните кнопку со знаком «плюс» (+). Появится диалоговое окно MAC Address Scanner, показанное на рисунке 2-12. Должен быть выбран переключатель All hosts in my subnet (либо при необходимости можно задать диапазон адресов). Для продолжения щёлкните OK.

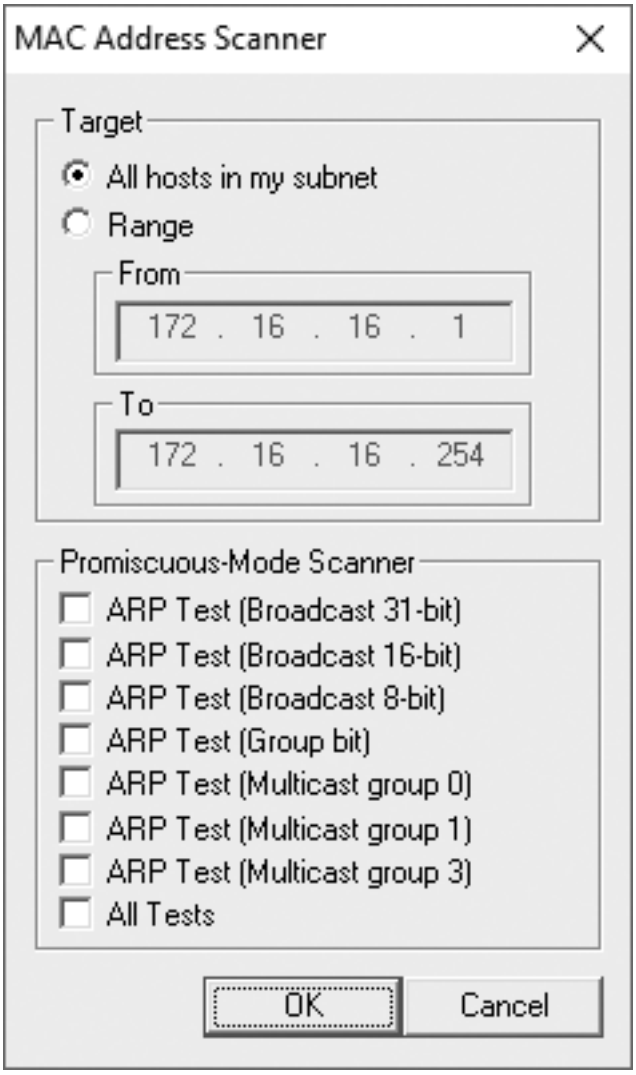


Рисунок 2-12. Сканирование MAC-адресов средством обнаружения сети Cain & Abel

Некоторые пользователи Windows 10 сообщают, что Cain & Abel не может определить IP-адрес сетевых интерфейсов, из-за чего завершить этот процесс невозможно. При такой проблеме во время настройки сетевых интерфейсов вы увидите для них IP-адрес `0.0.0.0`. Чтобы исправить положение, выполните следующие действия:

1. Если Cain & Abel открыт, закройте его.
2. В строке поиска на рабочем столе введите `ncpa.cpl`, чтобы открыть диалоговое окно Network Connections.
3. Щёлкните правой кнопкой мыши сетевой интерфейс, с которого будете перехватывать данные, и выберите Properties.
4. Дважды щёлкните Internet Protocol Version 4 (TCP/IPv4).
5. Щёлкните кнопку Advanced и выберите вкладку DNS.
6. Установите флажок Use this connection's DNS suffix in DNS registration.
7. Щёлкайте OK, чтобы закрыть открытые диалоговые окна, а затем снова запустите Cain & Abel.

Теперь таблица должна заполниться списком всех узлов подключённой сети вместе с их MAC-адресами, IP-адресами и сведениями об изготовителях. При настройке отравления кэша ARP

вы будете работать с этим списком.

В нижней части окна программы должен находиться набор вкладок, ведущих к другим окнам раздела Sniffer. Теперь, когда список узлов построен, вы будете работать на вкладке APR (ARP Poison Routing - маршрутизация с отравлением ARP). Перейдите в окно APR, щёлкнув эту вкладку.

В окне APR находятся две пустые таблицы. После завершения настройки верхняя будет показывать устройства, участвующие в отравлении кэша ARP, а нижняя - весь обмен между отравленными машинами.

Чтобы настроить отравление, выполните следующие действия:

1. Щёлкните пустую область в верхней части экрана. Затем нажмите кнопку со знаком «плюс» (+) на стандартной панели инструментов программы.
2. В появившемся окне будет две панели выбора. Слева расположен список всех доступных узлов сети. Если щёлкнуть IP-адрес целевого компьютера, справа появится список всех узлов сети, кроме IP-адреса целевой машины.
3. На правой панели щёлкните IP-адрес маршрутизатора, находящегося непосредственно выше целевой машины, как показано на рисунке 2-13, а затем щёлкните OK. Теперь IP-адреса обоих устройств должны появиться в верхней таблице главного окна приложения.
4. Чтобы завершить процесс, щёлкните жёлто-чёрный знак радиационной опасности на стандартной панели инструментов. Это включит функции отравления кэша ARP в Cain & Abel и позволит системе анализа стать посредником во всём обмене между целевой системой и её вышестоящим маршрутизатором.

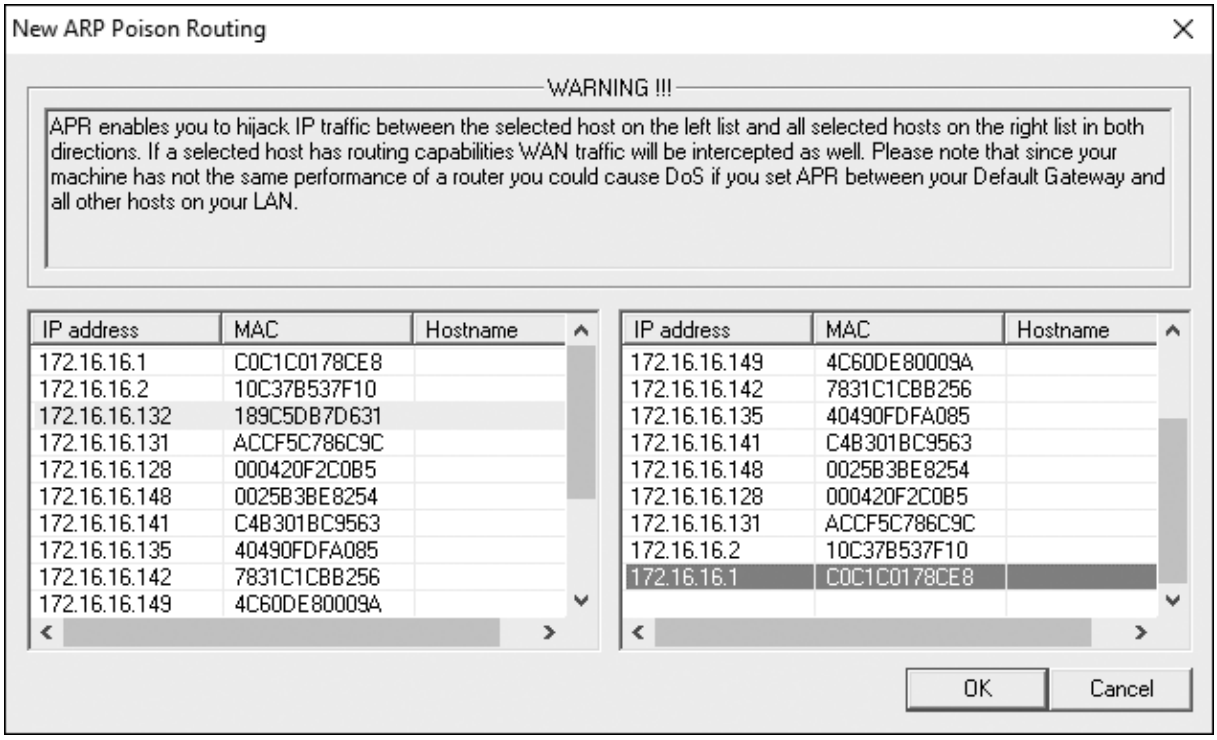


Рисунок 2-13. Выбор устройств, для которых требуется включить отравление кэша ARP

Теперь можно запустить анализатор пакетов и приступить к анализу. Закончив захват трафика, просто снова щёлкните жёлто-чёрный знак радиационной опасности, чтобы прекратить отравление кэша ARP.

Предостережение об отравлении кэша ARP

И последнее замечание об отравлении кэша ARP: необходимо очень хорошо понимать назначение систем, для которых вы выполняете этот процесс. Например, не применяйте такой метод, если целевое устройство чрезвычайно активно использует сеть, как файловый сервер с подключением 1 Гбит/с (особенно когда система анализа подключена лишь на скорости 100 Мбит/с).

Когда трафик перенаправляется показанным в примере способом, весь трафик, передаваемый и получаемый целевой системой, должен сначала пройти через систему анализа; таким образом, именно анализатор становится узким местом обмена. Подобное перенаправление способно оказать на анализируемую машину воздействие, подобное отказу в обслуживании, привести к ухудшению производительности сети и получению ошибочных данных анализа. Перегрузка трафиком также может нарушить ожидаемую работу обмена на основе SSL.

> **Примечание.** Избежать прохождения всего трафика через систему анализа можно с помощью функции асимметричной маршрутизации. Подробнее об этом методе рассказано в руководстве пользователя oxid.it: oxid.it.

Перехват в маршрутизируемой среде

Все способы подключения к линии в коммутируемой сети доступны и в маршрутизируемых сетях. Единственное важное соображение при работе в маршрутизируемой среде - правильное размещение анализатора при поиске неполадки, охватывающей несколько сегментов сети.

Как вы уже узнали, широковещательный домен устройства простирается до маршрутизатора, после чего трафик передаётся следующему вышестоящему маршрутизатору. Когда данные должны пройти через несколько маршрутизаторов, важно анализировать трафик со всех сторон каждого из них.

Например, представьте неполадку в сети, где несколько сегментов соединены несколькими маршрутизаторами. В этой сети каждый сегмент обменивается с вышестоящим сегментом данными для хранения и извлечения. На рисунке 2-14 показана проблема, которую мы пытаемся решить: нижестоящая подсеть, сеть D, не может связаться ни с одним устройством сети A.

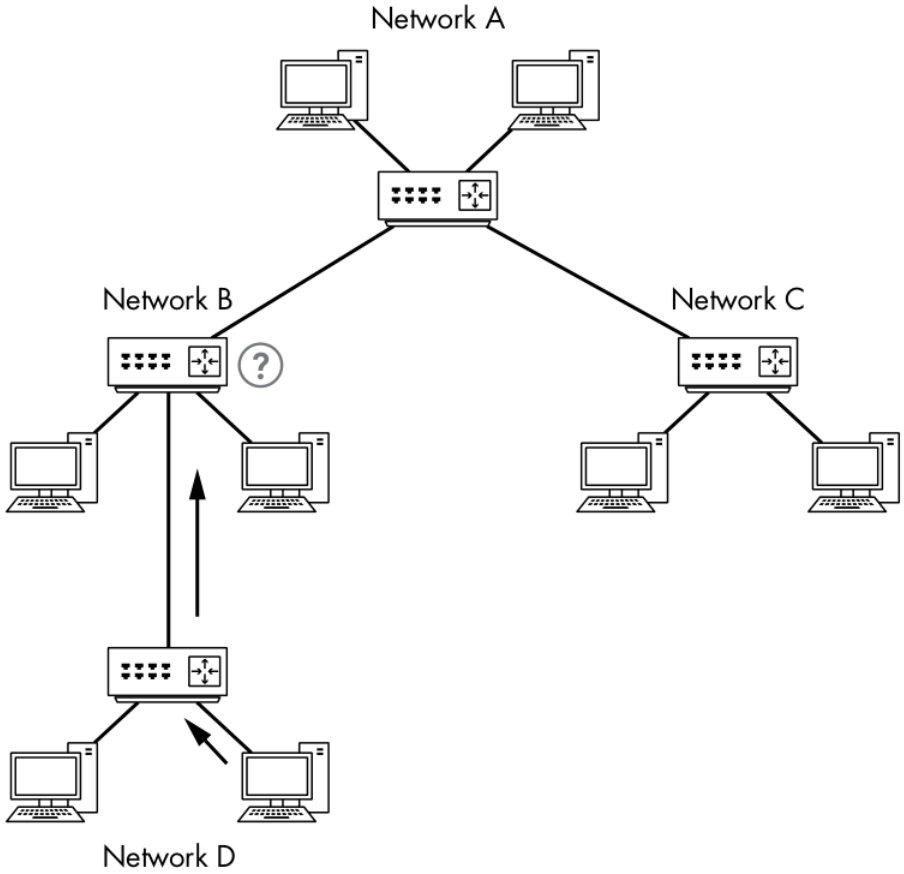


Рисунок 2-14. Компьютер сети D не может связаться с компьютерами сети A

Если перехватывать трафик устройства в сети D, которое не может связаться с устройствами других сетей, можно отчётливо увидеть передачу данных в другой сегмент, но не увидеть возвращающихся данных. Переместив анализатор и начав перехватывать трафик в следующем вышестоящем сегменте (сети B), вы получите более ясную картину происходящего. Здесь может обнаружиться, что маршрутизатор сети B отбрасывает трафик или направляет его неверно. В конце концов вы придёте к проблеме конфигурации маршрутизатора, исправление которой устранит более общую неполадку. Этот пример несколько обобщён, но его смысл состоит в следующем: при работе с несколькими маршрутизаторами и сегментами сети иногда приходится немного перемещать анализатор, чтобы увидеть картину целиком и точно определить источник проблемы.

Размещение анализатора на практике

Мы рассмотрели четыре способа захвата сетевого трафика в коммутируемой среде. Можно добавить пятый, просто установив приложение для перехвата пакетов непосредственно на устройство, трафик которого требуется захватить (метод прямой установки). При наличии пяти вариантов бывает трудно решить, какой из них подходит лучше всего. В таблице 2-2 приведены общие рекомендации для каждого способа.

Аналитикам следует действовать как можно незаметнее. В идеальном мире мы собираем нужные данные, не оставляя следов. Подобно тому как криминалисты не хотят загрязнять место преступления, мы не хотим загрязнять захватываемый сетевой трафик.

Карты сети

При обсуждении размещения в сети мы рассмотрели несколько сетевых карт. Карта, или схема, сети показывает все её технические ресурсы и связи между ними.

Нет лучшего способа определить место для анализатора пакетов, чем наглядно представить сеть. Если у вас есть карта сети, держите её под рукой: она станет ценным подспорьем при поиске неполадок и анализе. Возможно, стоит даже самостоятельно составить подробную карту собственной сети. Помните, что иногда половина успеха при поиске неполадки - убедиться, что вы собираете правильные данные.

Таблица 2-2. Рекомендации по перехвату пакетов в коммутируемой среде

Способ	Рекомендации
Зеркалирование портов	Не оставляет следов в сети и не создаёт дополнительных пакетов. Можно настроить, не отключая клиента от сети, что удобно при зеркалировании портов маршрутизатора или сервера. Расходует вычислительные ресурсы коммутатора и может давать нестабильные результаты при высокой пропускной способности.
Подключение через концентратор	Подходит, когда допустимо временно отключить узел от сети. Неэффективно, если требуется захватывать трафик нескольких узлов, поскольку вероятны коллизии и потеря пакетов. На современных узлах 100/1000 Мбит/с может приводить к потере пакетов, потому что большинство настоящих концентраторов рассчитано лишь на 10 Мбит/с.
Использование ответвителя	Идеально подходит, если допустимо временно отключить узел от сети. Единственный вариант, когда требуется перехватывать трафик оптоволоконного соединения. Предпочтительное решение для корпоративного захвата пакетов и непрерывного мониторинга, поскольку ответвители надёжны и способны масштабироваться до линий с высокой пропускной способностью. Поскольку ответвители специально созданы для этой задачи и соответствуют скоростям современных сетей, этот способ превосходит подключение через концентратор. Может быть дорогим, особенно при масштабном внедрении, и потому оказаться неприемлемым по стоимости.
Отравление кэша ARP	Считается очень грубым способом, поскольку требует внедрять в сеть пакеты, перенаправляющие трафик через анализатор. Если зеркалирование портов недоступно, позволяет быстро захватить трафик устройства, не отключая его от сети. Требует большой осторожности, чтобы не нарушить работу сети.

Способ	Рекомендации
Прямая установка	Обычно не рекомендуется: если на узле возникла неполадка, она может привести к отбрасыванию или изменению пакетов так, что они будут представлены неточно. NIC узла не требуется переводить в неразборчивый режим. Лучше всего подходит для испытательных сред, исследования и определения базовой производительности, а также изучения файлов захвата, созданных в другом месте.

Когда в последующих главах мы будем разбирать практические ситуации, то в каждом случае обсудим лучший способ захвата необходимых данных. Пока же выбрать оптимальный способ захвата трафика в конкретной ситуации вам поможет блок-схема на рисунке 2-15. Она учитывает различные факторы, начиная с того, где вы захватываете пакеты - дома или на работе. Помните, что это лишь общая справочная схема, которая не охватывает всех возможных ситуаций подключения к линии.

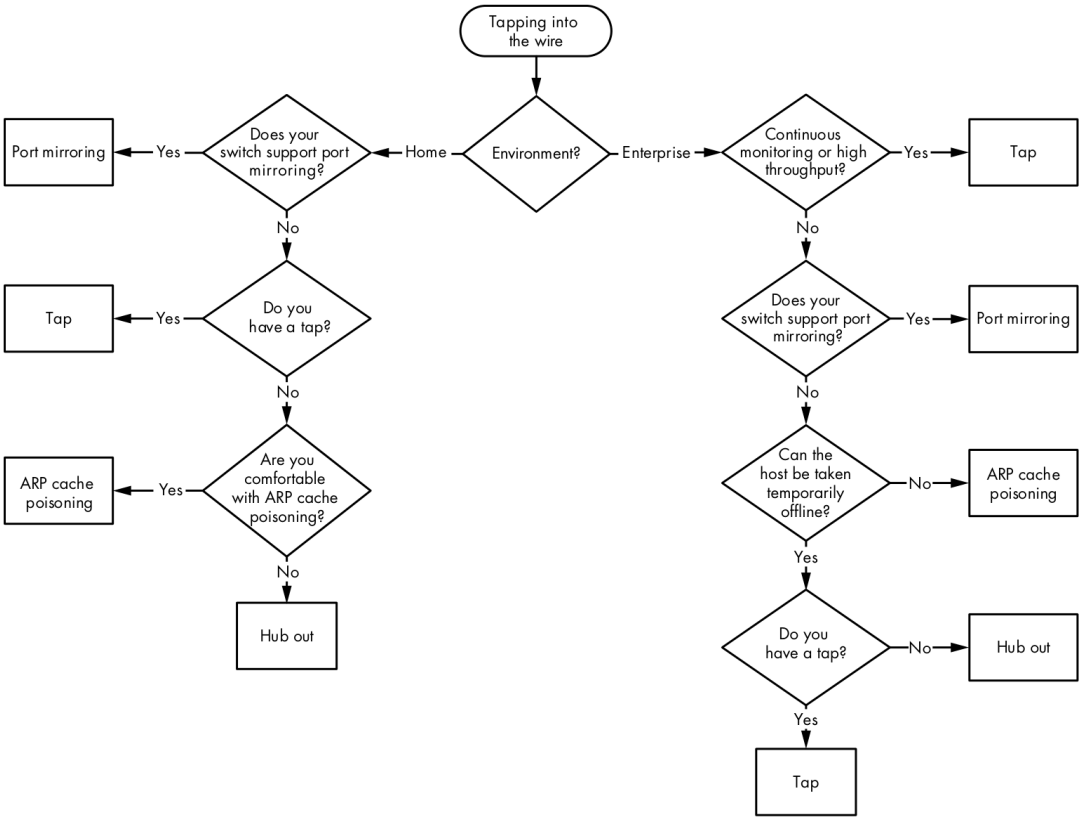


Рисунок 2-15. Схема, помогающая определить лучший способ подключения к линии

Глава 3. Введение в Wireshark

Как уже говорилось в главе 1, для анализа сети существует несколько приложений перехвата пакетов, но в этой книге мы сосредоточимся главным образом на Wireshark. Эта глава знакомит вас с Wireshark.

Краткая история Wireshark

У Wireshark весьма богатая история. Джеральд Комбс, выпускник факультета информатики Университета Миссури в Канзас-Сити, первоначально разработал программу по необходимости. Первая версия приложения Комбса под названием Ethereal была выпущена в 1998 году на условиях Стандартной общественной лицензии GNU (GNU Public License, GPL).

Через восемь лет после выпуска Ethereal Комбс ушёл с работы ради других возможностей профессионального роста. К сожалению, его тогдашнему работодателю принадлежали все права на товарные знаки Ethereal, и Комбсу не удалось заключить соглашение, которое позволило бы ему управлять брендом Ethereal. Поэтому в середине 2006 года Комбс и остальные участники команды разработки переименовали проект в Wireshark.

Популярность Wireshark стремительно выросла, и теперь его объединённая команда разработки насчитывает более 500 участников. Программа, оставшаяся под названием Ethereal, больше не развивается.

Преимущества Wireshark

Wireshark обладает рядом преимуществ, которые делают его привлекательным для повседневного применения. Он рассчитан и на начинающих, и на опытных аналитиков пакетов и предлагает множество возможностей для каждой из этих групп. Оценим Wireshark по критериям выбора средства перехвата пакетов, определённым в главе 1.

Поддерживаемые протоколы. Wireshark выделяется количеством поддерживаемых протоколов - на момент написания книги их было более 1000. Диапазон простирается от распространённых протоколов вроде IP и DHCP до более сложных закрытых протоколов, таких как DNP3 и BitTorrent. А поскольку Wireshark разрабатывается по модели открытого исходного кода, с каждым обновлением добавляется поддержка новых протоколов.

> **Примечание.** В маловероятном случае, если Wireshark не поддерживает нужный протокол, вы можете самостоятельно написать соответствующий код. Затем его можно отправить разработчикам Wireshark, чтобы они рассмотрели возможность включения в приложение. Требования к участию в разработке Wireshark описаны на странице wireshark.org.

Удобство для пользователя. Интерфейс Wireshark - один из самых понятных среди всех приложений перехвата пакетов. Это графический интерфейс с ясно подписанными контекстными меню и простой компоновкой. В нём также предусмотрено несколько функций для повышения удобства, например цветовое кодирование по протоколам и подробное графическое представление необработанных данных. В отличие от некоторых более сложных альтернатив, управляемых из командной строки, таких как tcpdump, графический интерфейс Wireshark доступен

тем, кто лишь начинает знакомиться с анализом пакетов.

Стоимость. Поскольку Wireshark имеет открытый исходный код и выпускается на условиях Стандартной общественной лицензии GNU (GPL), превзойти его цену невозможно: программа совершенно бесплатна. Wireshark можно скачать и использовать для любых целей - личных или коммерческих.

> **Примечание.** Хотя Wireshark бесплатен, некоторые люди по ошибке умудрялись за него заплатить. Если поискать анализаторы пакетов на eBay, вас может удивить количество желающих продать «профессиональную корпоративную лицензию» Wireshark по бросовой цене 39,95 доллара. Если вы всё же решите, что непременно хотите её купить, позвоните мне - заодно обсудим участок побережья в Кентукки, который я продаю!

Поддержка программы. Уровень поддержки способен определить успех или провал программного пакета. У свободно распространяемых программ вроде Wireshark может не быть официальной поддержки, поэтому сообщество открытого исходного кода часто полагается на помощь пользователей. К счастью для нас, сообщество Wireshark - одно из самых активных среди всех проектов с открытым исходным кодом. Сайт Wireshark содержит прямые ссылки на несколько видов поддержки, включая документацию в Интернете, вики-сайт, часто задаваемые вопросы и страницу подписки на список рассылки Wireshark, за которым следит большинство ведущих разработчиков программы. Платную поддержку Wireshark также предоставляет Riverbed Technology.

Доступ к исходному коду. Wireshark - программа с открытым исходным кодом, поэтому получить доступ к её коду можно в любое время. Это полезно при поиске неполадок приложения, изучении работы анализаторов протоколов и подготовке собственного вклада в проект.

Поддержка операционных систем. Wireshark поддерживает все основные современные операционные системы, включая Windows, платформы на основе Linux и OS X. Полный перечень поддерживаемых операционных систем можно найти на домашней странице Wireshark.

Установка Wireshark

Процесс установки Wireshark на удивление прост. Однако прежде чем устанавливать программу, убедитесь, что система отвечает следующим требованиям:

- любой современный 32-разрядный процессор x86 или 64-разрядный процессор;
- 400 Мбайт доступной оперативной памяти, а для крупных файлов захвата - больше;
- не менее 300 Мбайт свободного места в хранилище плюс место для файлов захвата;
- NIC с поддержкой неразборчивого режима;
- драйвер захвата WinPcap/libpcap.

Драйвер захвата WinPcap - реализация программного интерфейса приложения (API) pcap для захвата пакетов в Windows. Проще говоря, этот драйвер взаимодействует с операционной системой, чтобы захватывать необработанные данные пакетов, применять фильтры и переводить NIC в неразборчивый режим и обратно.

Хотя WinPcap можно скачать отдельно с сайта winpcap.org, обычно лучше устанавливать его из установочного пакета Wireshark, поскольку включённая туда версия WinPcap проверена на совместную работу с Wireshark.

Установка в Windows

Текущая версия Wireshark испытывается на поддержку версий Windows, срок расширенной поддержки которых ещё не закончился. На момент написания книги к ним относились Windows Vista, Windows 7, Windows 8, Windows 10, а также Windows Server 2003, 2008 и 2012. Wireshark часто работает и в других версиях Windows, например Windows XP, но официально они не поддерживаются.

Первый шаг при установке Wireshark в Windows - получить последнюю сборку установщика с официальной веб-страницы Wireshark wireshark.org. Перейдите на сайте в раздел Download Wireshark и выберите зеркало выпуска. Скачав пакет, выполните следующие действия:

1. Дважды щёлкните файл `.exe`, чтобы начать установку, а затем щёлкните Next в начальном окне.
2. Прочитайте лицензионное соглашение и, если согласны, щёлкните I Agree.
3. Выберите компоненты Wireshark, которые хотите установить, как показано на рисунке 3-1. Для наших целей можно принять значения по умолчанию, щёлкнув Next.

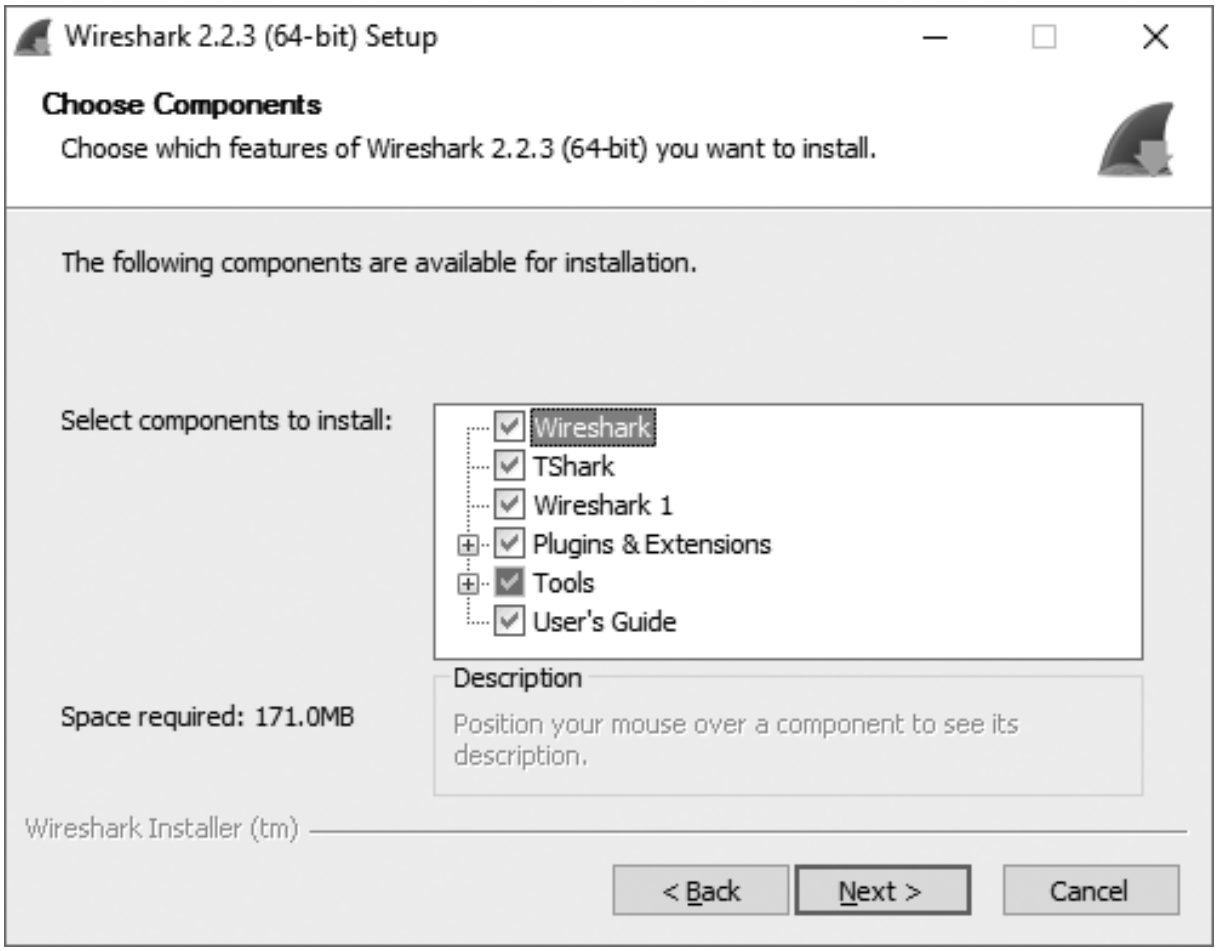


Рисунок 3-1. Выбор компонентов Wireshark для установки

4. В окне Additional Tasks щёлкните Next.
5. Выберите место установки Wireshark и щёлкните Next.
6. Когда в диалоговом окне появится вопрос об установке WinPcap, сначала убедитесь, что установлен флажок Install WinPcap, как показано на рисунке 3-2. Затем щёлкните Install. Должен начаться процесс установки.
7. Примерно на середине установки Wireshark должна запуститься установка WinPcap. Когда это произойдёт, щёлкните Next в начальном окне, прочитайте лицензионное соглашение и щёлкните I Agree.

8. Вам предложат установить USBPcap - служебную программу для сбора данных с устройств USB. При желании установите соответствующий флажок и щёлкните Next.

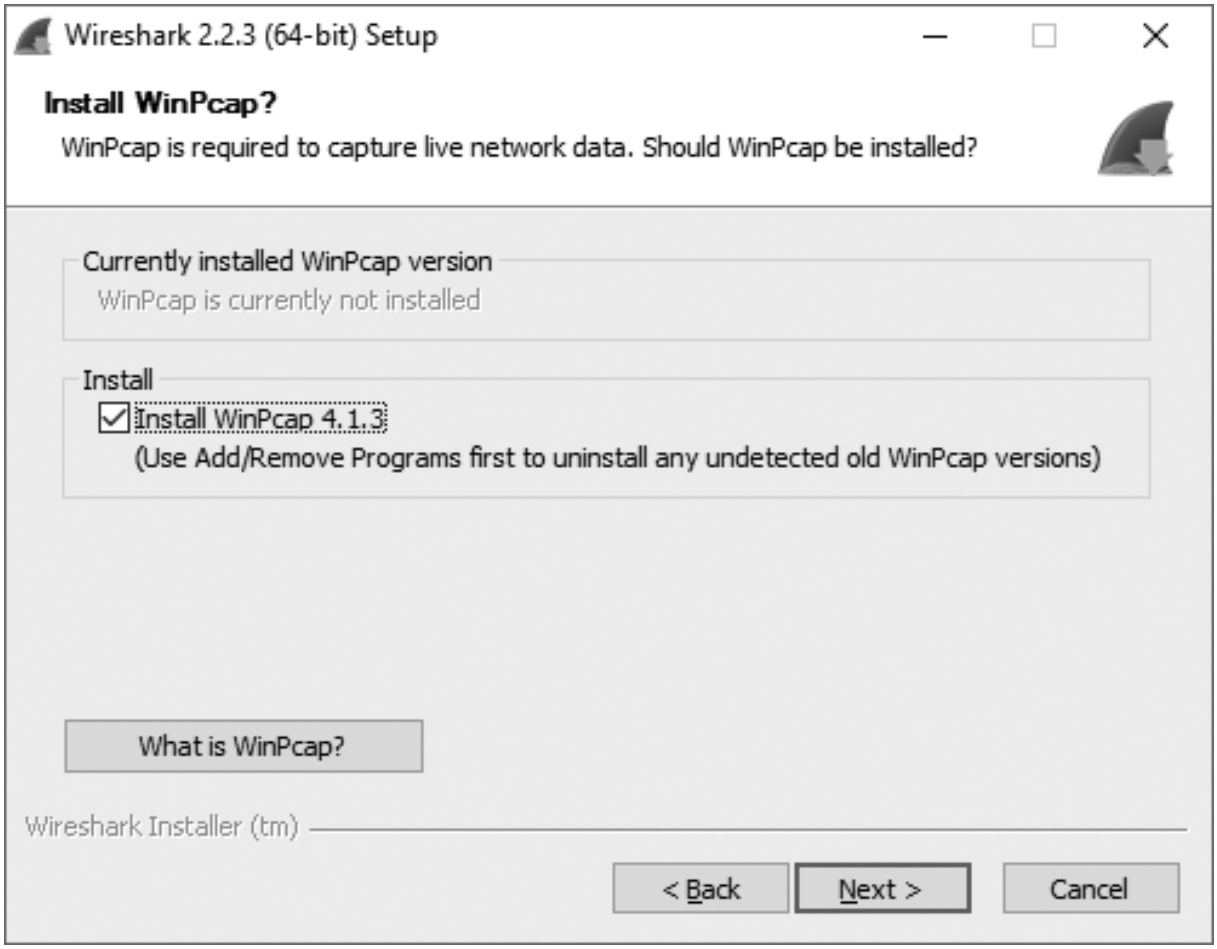


Рисунок 3-2. Выбор варианта установки драйвера WinPcap

9. На компьютер должны установиться WinPcap и, если вы его выбрали, USBPcap. После завершения этой установки щёлкните Finish.
10. Wireshark должен завершить свою установку. Когда это произойдёт, щёлкните Next.
11. В окне подтверждения установки щёлкните Finish.

Установка в Linux

Wireshark работает на большинстве современных платформ на основе Unix. Его можно установить с помощью выбранного диспетчера пакетов дистрибутива либо скачать и установить пакет, соответствующий вашему дистрибутиву. Описать процедуры установки для всех нереально, поэтому мы рассмотрим лишь несколько вариантов.

Как правило, для установки программ во всей системе требуются права пользователя root. Однако локальные программы, скомпилированные из исходного кода, обычно можно установить без прав root.

Системы на основе RPM

Если вы пользуетесь Red Hat Linux или основанным на нём дистрибутивом, например CentOS, скорее всего, в операционной системе по умолчанию установлен диспетчер пакетов Yum. В таком случае Wireshark можно быстро установить из репозитория программ дистрибутива. Для этого откройте окно консоли и введите следующую команду:

```
$ sudo yum install wireshark
```

Если понадобятся зависимости, вам предложат их установить. Если всё завершится успешно, Wireshark можно будет запускать из командной строки и открывать через графический интерфейс.

Системы на основе DEB

В большинство дистрибутивов на основе DEB, таких как Debian и Ubuntu, входит диспетчер пакетов APT, позволяющий установить Wireshark из репозитория программ операционной системы. Чтобы установить Wireshark этим средством, откройте окно консоли и введите:

```
$ sudo apt-get install wireshark wireshark-qt
```

Вам снова предложат установить все зависимости, необходимые для завершения установки.

Компиляция из исходного кода

Из-за изменений в архитектуре операционных систем и возможностях Wireshark инструкции по компиляции программы из исходного кода со временем могут меняться. Это одна из причин, по которым для установки рекомендуется пользоваться диспетчером пакетов операционной системы. Однако, если в вашем дистрибутиве Linux нет автоматизированного диспетчера пакетов или вам требуется особая установка, Wireshark можно установить вручную, скомпилировав из исходного кода. Для этого выполните следующие действия:

1. Скачайте пакет исходного кода с веб-страницы Wireshark.
2. Распакуйте архив следующей командой, подставив вместо имени файла название скачанного пакета:

```
$ tar -jxvf <file_name_here>.tar.bz2
```

3. Прежде чем настраивать и устанавливать Wireshark, может потребоваться несколько зависимостей в зависимости от выбранного варианта Linux. Например, для работы Wireshark в Ubuntu 14.04 необходимо установить несколько дополнительных пакетов. Их можно установить следующей командой (её нужно выполнить как пользователь уровня root либо предварить командой `sudo`):

```
$ sudo apt-get install pkg-config bison flex qt5-default libgtk-3-dev libpcap-dev qttools5-dev-tools
```

4. Установив необходимые компоненты, перейдите в каталог, куда были извлечены файлы Wireshark.
5. Настройте исходный код для правильной сборки в вашем дистрибутиве Linux командой `./configure`. Если вы хотите отклониться от параметров установки по умолчанию, их можно указать на этом этапе. При отсутствии каких-либо зависимостей вы, скорее всего, получите сообщение об ошибке. Прежде чем продолжать, необходимо установить и настроить эти зависимости. При успешной настройке должно появиться сообщение об успехе с выбранными параметрами, как показано на рисунке 3-3.

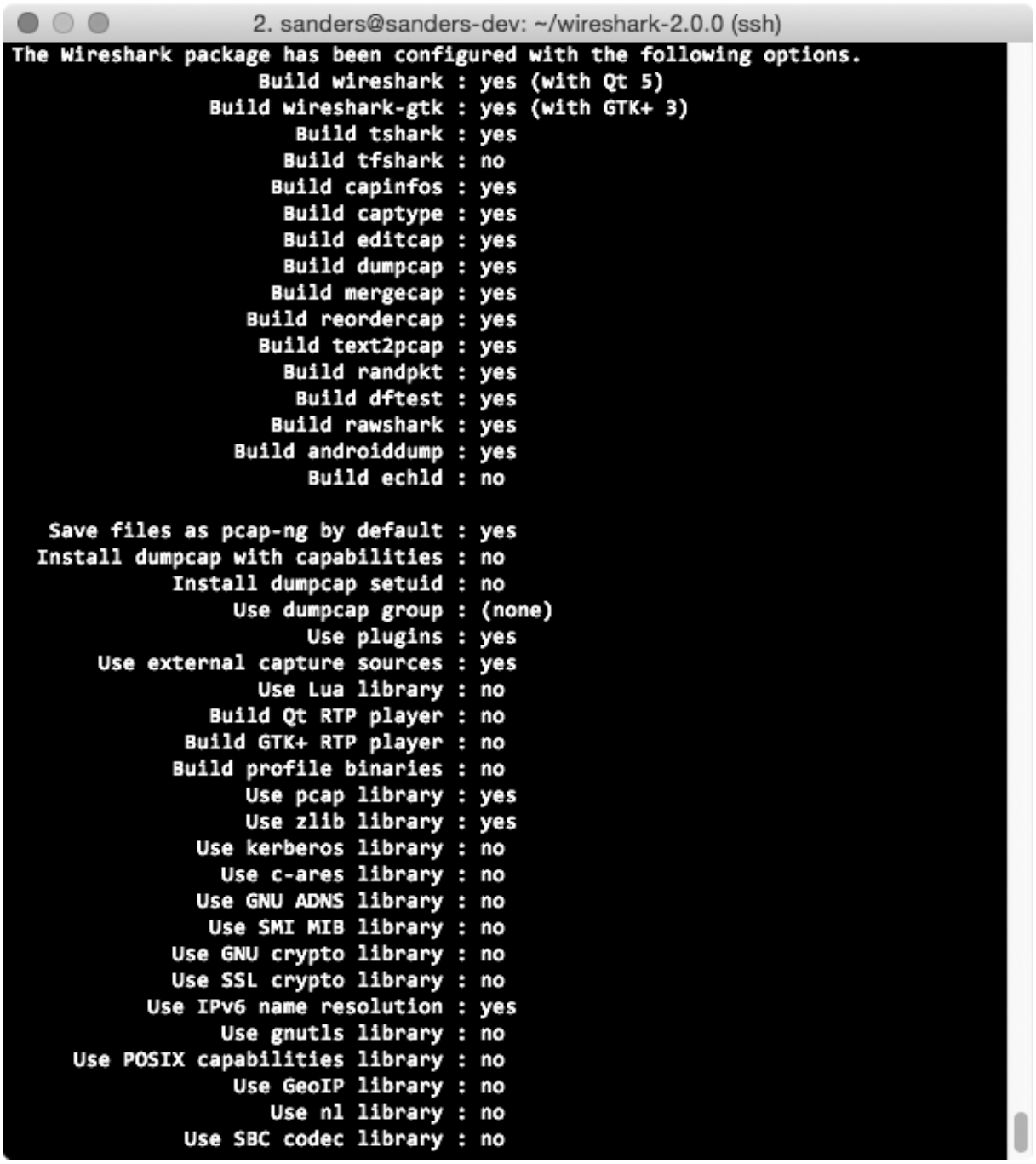


Рисунок 3-3. После успешного выполнения команды `./configure` *появляется сообщение с выбранными параметрами*

6. Введите команду `make`, чтобы собрать из исходного кода двоичный файл.
7. Запустите окончательную установку командой `sudo make install`.
8. Выполните `sudo /sbin/ldconfig`, чтобы завершить установку.

> **Примечание.** Если при выполнении этих действий возникнет ошибка, возможно, придётся установить дополнительный пакет.

Установка в OS X

Чтобы установить Wireshark в OS X, выполните следующие действия:

1. Скачайте пакет OS X с веб-страницы Wireshark.
2. Запустите программу установки и последовательно выполните её шаги. После принятия обязательного лицензионного соглашения с конечным пользователем можно будет выбрать место установки.
3. Завершите работу мастера установки.

Основы Wireshark

Успешно установив Wireshark в системе, можно начать знакомство с программой. Наконец-то вы открываете полностью работоспособный анализатор пакетов и видите... ровным счётом ничего!

Ладно, при первом запуске Wireshark выглядит не слишком интересно. Чтобы стало по-настоящему увлекательно, нужно получить какие-нибудь данные.

Ваш первый захват пакетов

Чтобы загрузить данные пакетов в Wireshark, вы выполните свой первый захват. Возможно, вы думаете: «Как же мне захватывать пакеты, если в сети ничего не сломано?»

Во-первых, в сети всегда что-нибудь сломано. Если вы мне не верите, отправьте всем пользователям сети письмо о том, что всё работает безупречно.

Во-вторых, для анализа пакетов вовсе не обязательно ждать поломки. На самом деле большинство аналитиков пакетов тратит больше времени на изучение исправного трафика, чем на трафик при поиске неполадок. В конце концов, чтобы эффективно искать неполадки сетевого трафика, нужна база для сравнения. Например, если вы надеетесь когда-нибудь решить проблему DHCP посредством анализа трафика, необходимо понимать, как выглядит поток исправного трафика DHCP.

В более широком смысле, чтобы находить аномалии в повседневной сетевой активности, нужно знать, как выглядит нормальная повседневная активность. Наблюдения за исправно работающей сетью становятся базовым уровнем, представляющим трафик в нормальном состоянии.

Итак, давайте захватим несколько пакетов!

1. Откройте Wireshark.
2. В главном раскрывающемся меню выберите Capture, затем Options. Должно появиться диалоговое окно со списком интерфейсов, которые можно использовать для захвата пакетов, и основными сведениями о каждом из них (рисунок 3-4). Обратите внимание на столбец Traffic: в нём линейный график показывает объём трафика, проходящего через интерфейс в данный момент. Пики линии означают, что пакеты действительно захватываются. Если захвата нет, линия будет ровной. Можно также развернуть каждый интерфейс стрелкой слева от него и увидеть связанную с ним адресную информацию, например MAC-адрес или IP-адрес.
3. Щёлкните интерфейс, который хотите использовать, и нажмите Start. Окно должно начать заполняться данными.
4. Подождите около минуты; когда будете готовы остановить захват и просмотреть данные, щёлкните Stop в раскрывающемся меню Capture.

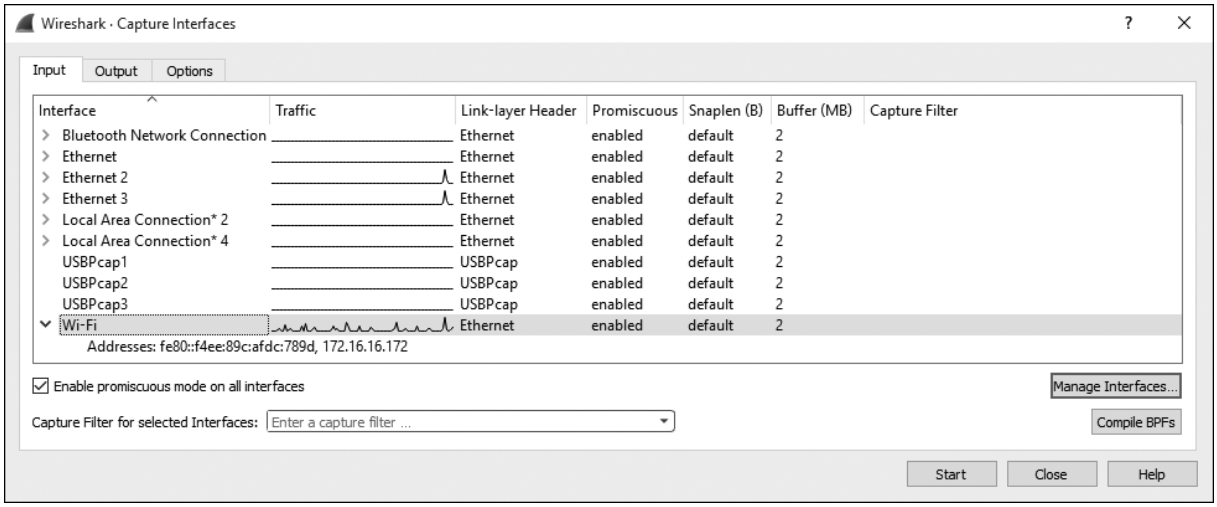


Рисунок 3-4. Выбор интерфейса для захвата пакетов

После выполнения этих действий и завершения захвата главное окно Wireshark должно ожить и наполниться данными. Возможно, их количество даже ошеломит вас, но всё быстро начнёт обретать смысл, когда мы по частям разберём главное окно Wireshark.

Главное окно Wireshark

Большую часть времени вы будете проводить в главном окне Wireshark. Здесь отображаются все захваченные пакеты и представляются в более понятном виде. Рассмотрим главное окно Wireshark, показанное на рисунке 3-5, на примере только что выполненного захвата.

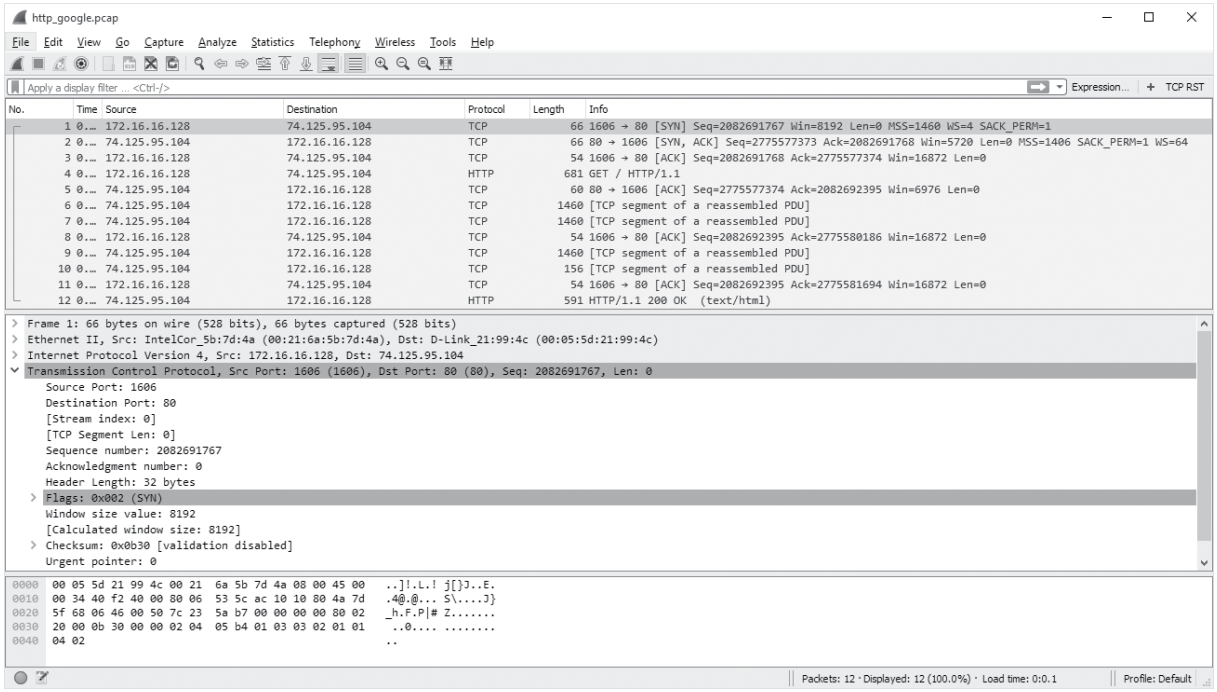


Рисунок 3-5. Главное окно Wireshark имеет трёхпанельную компоновку: Packet List, Packet Details и Packet Bytes

Три панели главного окна - сверху вниз Packet List, Packet Details и Packet Bytes - зависят друг от друга. Чтобы просмотреть подробности отдельного пакета на панели Packet Details, его сначала нужно выбрать на панели Packet List. При выборе части пакета на панели Packet Details панель Packet Bytes показывает байты, соответствующие выбранной части.

> **Примечание.** Обратите внимание, что на рисунке 3-5 в панели Packet List перечислено несколько разных протоколов. Наглядного разделения протоколов разных уровней нет, если не считать цветового кодирования; все пакеты показаны в том порядке, в каком получены с линии.

Вот что содержится на каждой панели.

Packet List. Верхняя панель показывает таблицу со всеми пакетами текущего файла захвата. Её столбцы содержат номер пакета, относительное время его захвата, источник и получателя пакета, протокол и некоторые общие сведения, найденные в пакете.

> **Примечание.** Говоря о трафике, я имею в виду все пакеты, показанные на панели Packet List. Говоря конкретно о трафике DNS, я подразумеваю пакеты протокола DNS на панели Packet List.

Packet Details. Средняя панель иерархически показывает сведения об одном пакете; её элементы можно сворачивать и разворачивать, чтобы видеть все собранные сведения об отдельном пакете.

Packet Bytes. Нижняя и, пожалуй, самая непонятная панель показывает пакет в необработанном исходном виде, то есть таким, каким он проходит по линии. Это необработанные сведения без каких-либо приятных и удобных добавлений, облегчающих понимание. Способы интерпретации данных такого вида обсуждаются в приложении B.

Настройки Wireshark

В Wireshark имеется несколько настроек, которые можно изменять в соответствии со своими потребностями. Чтобы открыть настройки, выберите Edit в главном раскрывающемся меню и щёлкните Preferences. Появится диалоговое окно Preferences с различными изменяемыми параметрами, показанное на рисунке 3-6.

Настройки Wireshark разделены на шесть основных разделов и дополнительный раздел Advanced.

Appearance. Эти настройки определяют представление данных в Wireshark. Большинство параметров можно выбирать согласно личным предпочтениям: сохранять ли положение окон, как компоновать три основные панели, где размещать полосу прокрутки и столбцы панели Packet List, какие шрифты использовать для захваченных данных, а также каковы цвета фона и переднего плана.

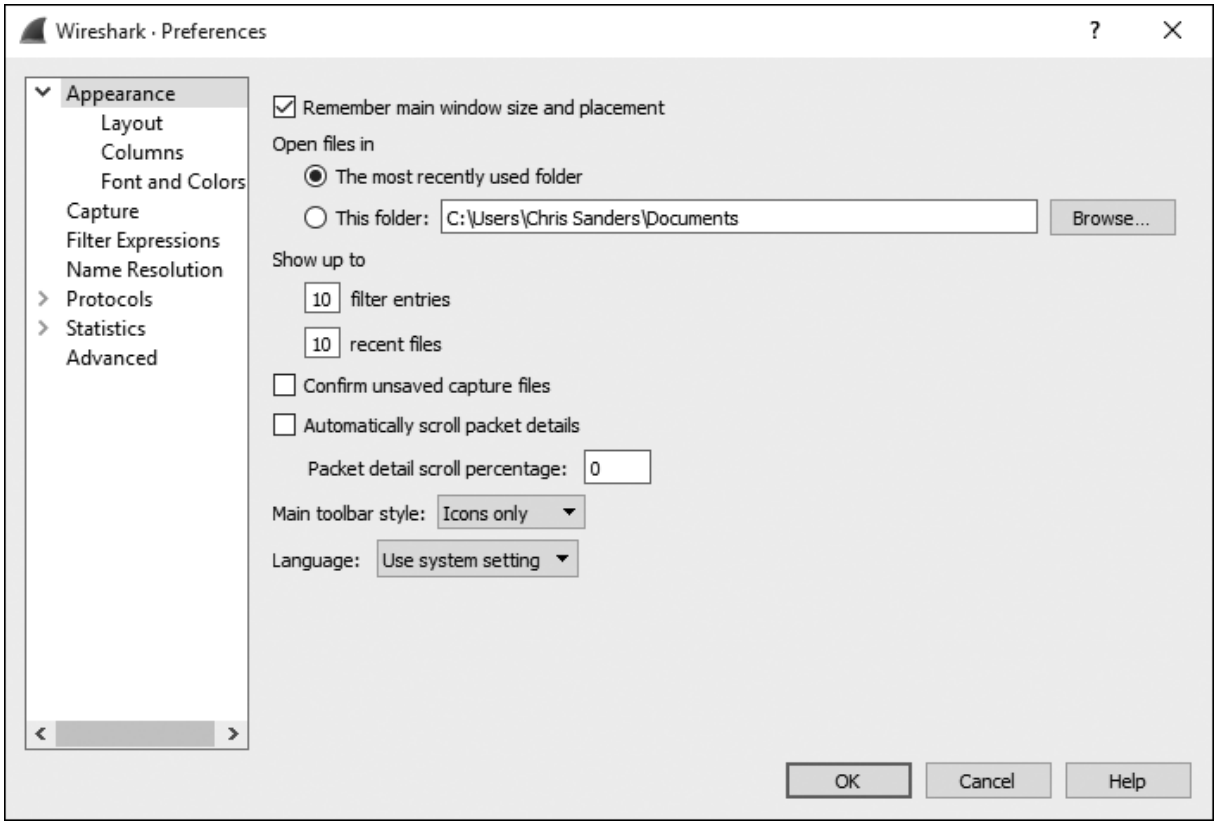


Рисунок 3-6. Wireshark можно настроить с помощью параметров диалогового окна Preferences

Capture. Эти настройки позволяют задавать параметры захвата пакетов, включая интерфейс захвата по умолчанию, использование неразборчивого режима по умолчанию и обновление панели Packet List в реальном времени.

Filter Expressions. Далее мы обсудим, как Wireshark позволяет фильтровать трафик по определённым критериям. Этот раздел диалогового окна Preferences предназначен для создания фильтров и управления ими.

Name Resolution. Здесь можно включать функции Wireshark, преобразующие адреса в более узнаваемые имена, в том числе разрешение имён MAC, сетевых и транспортных адресов, а также задавать максимальное число одновременно выполняемых запросов разрешения имён.

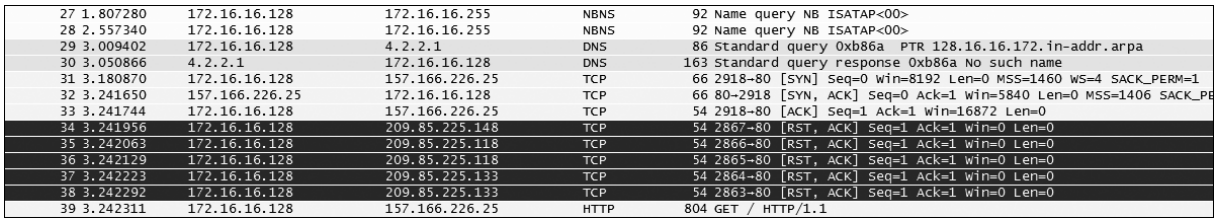
Protocols. В этом разделе можно изменять параметры захвата и отображения различных пакетов, которые способен декодировать Wireshark. Настраиваемые параметры есть не у каждого протокола, но у некоторых их несколько. Если у вас нет конкретной причины что-либо менять, лучше оставить значения по умолчанию.

Statistics. Этот раздел содержит несколько изменяемых параметров статистических функций Wireshark, которые подробнее рассматриваются в главе 5.

Advanced. Здесь находятся настройки, которые нельзя естественным образом отнести ни к одной из предыдущих категорий. Обычно их изменяют лишь опытные пользователи Wireshark.

Цветовое кодирование пакетов

Если вы хоть немного похожи на меня, вам нравятся блестящие предметы и красивые цвета. Тогда вы, вероятно, обрадовались, увидев все эти разные цвета на панели Packet List, как в примере на рисунке 3-7 (правда, если вы читаете печатную книгу, рисунок будет чёрно-белым, но смысл понятен). Может показаться, что цвета назначены пакетам случайно, однако это не так.



27	1.807280	172.16.16.128	172.16.16.255	NBNS	92 Name query NB ISATAP<00>
28	2.557340	172.16.16.128	172.16.16.255	NBNS	92 Name query NB ISATAP<00>
29	3.009402	172.16.16.128	4.2.2.1	DNS	86 Standard query 0xb86a PTR 128.16.16.172.in-addr.arpa
30	3.050866	4.2.2.1	172.16.16.128	DNS	163 Standard query response 0xb86a No such name
31	3.180870	172.16.16.128	157.166.226.25	TCP	66 2918→80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
32	3.241650	157.166.226.25	172.16.16.128	TCP	66 80→2918 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1406 SACK_PE
33	3.241744	172.16.16.128	157.166.226.25	TCP	54 2918→80 [ACK] Seq=1 Ack=1 Win=16872 Len=0
34	3.241956	172.16.16.128	209.85.225.148	TCP	54 2867→80 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
35	3.242063	172.16.16.128	209.85.225.118	TCP	54 2866→80 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
36	3.242129	172.16.16.128	209.85.225.118	TCP	54 2865→80 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
37	3.242223	172.16.16.128	209.85.225.133	TCP	54 2864→80 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
38	3.242292	172.16.16.128	209.85.225.133	TCP	54 2863→80 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
39	3.242311	172.16.16.128	157.166.226.25	HTTP	804 GET / HTTP/1.1

Рисунок 3-7. Цветовое кодирование Wireshark позволяет быстро распознавать протоколы

Каждый пакет отображается определённым цветом не просто так. Цвет может отражать протокол пакета и значения конкретных полей. Например, по умолчанию весь трафик UDP окрашен синим, а весь трафик HTTP - зелёным. Цветовое кодирование помогает быстро различать протоколы и не читать поле протокола на панели Packet List для каждого пакета. Вы обнаружите, что это значительно ускоряет просмотр крупных файлов захвата.

Узнать, какие цвета назначены каждому протоколу, легко в окне Coloring Rules, показанном на рисунке 3-8. Чтобы открыть его, выберите View в главном раскрывающемся меню и щёлкните Coloring Rules.

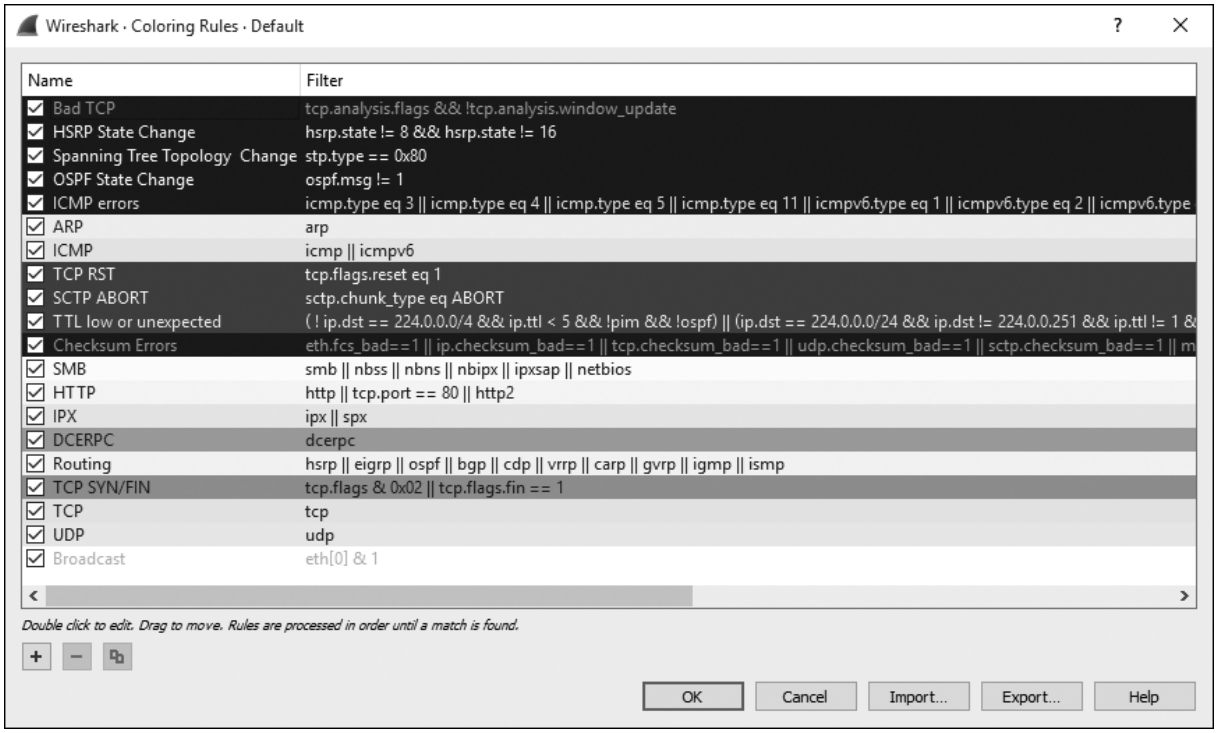


Рисунок 3-8. В окне Coloring Rules можно просматривать и изменять цвета пакетов

Правила раскрашивания основаны на фильтрах Wireshark, которые мы рассмотрим в главе 4. С помощью этих фильтров можно определять собственные правила раскрашивания и изменять существующие. Например, чтобы заменить цвет фона для трафика HTTP со стандартного зелёного на лавандовый, выполните следующие действия:

1. Откройте Wireshark и перейдите в окно Coloring Rules (View > Coloring Rules).
2. Найдите правило раскрашивания HTTP в списке и выберите его одним щелчком.
3. В нижней части экрана появятся цвета переднего плана и фона, как показано на рисунке 3-9.

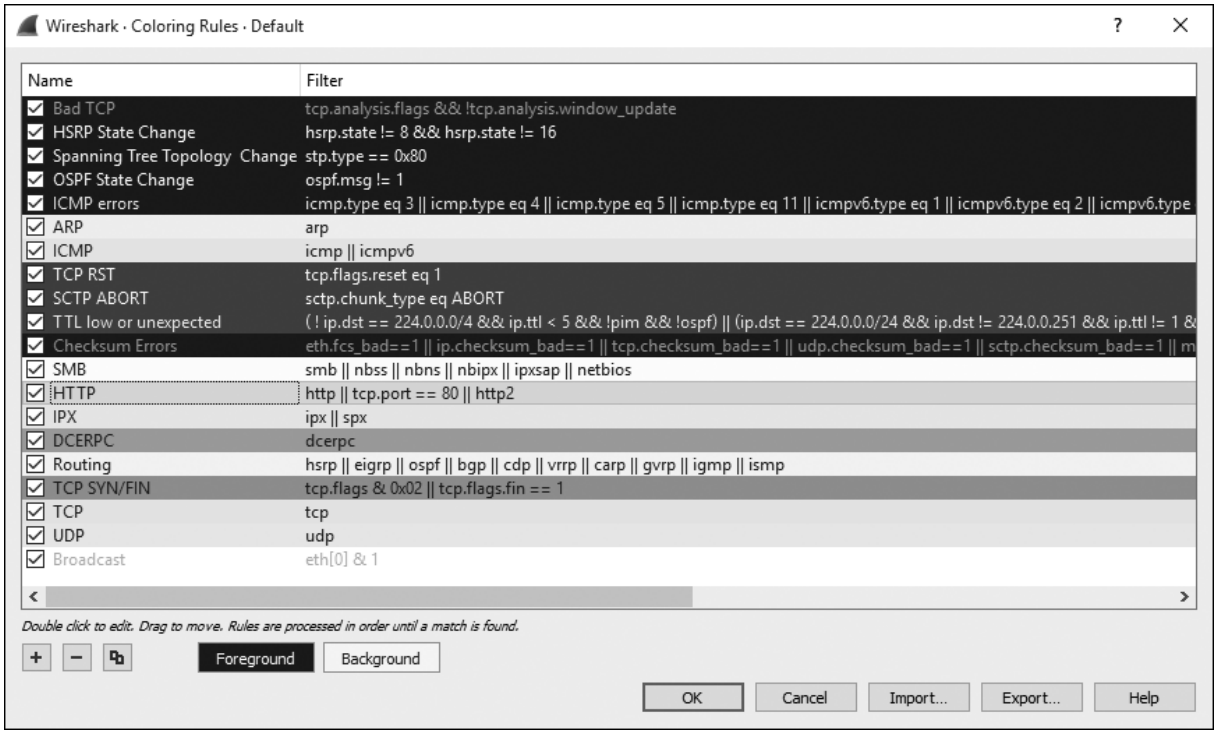


Рисунок 3-9. При изменении цветового фильтра можно задать и цвет переднего плана, и цвет фона

4. Щёлкните кнопку Background.
5. Выберите нужный цвет на цветовом круге и щёлкните OK.
6. Ещё раз щёлкните OK, чтобы принять изменения и вернуться в главное окно. После этого пользовательский интерфейс должен перезагрузиться и показать обновлённую цветовую схему.

Работая с Wireshark в своей сети, вы начнёте замечать, что с одними протоколами приходится иметь дело чаще, чем с другими. Вот тут пакеты с цветовым кодированием способны значительно облегчить жизнь. Например, если вы подозреваете, что в сети есть посторонний сервер DHCP, выдающий аренды IP-адресов, можно изменить правило раскрашивания протокола DHCP так, чтобы он показывался ярко-жёлтым или другим легко узнаваемым цветом. Тогда весь трафик DHCP будет гораздо быстрее бросаться в глаза, что сделает анализ пакетов эффективнее.

> **Примечание.** Не так давно во время выступления перед местной группой студентов я рассказывал о правилах раскрашивания Wireshark. Один студент с облегчением узнал, что эти правила можно изменять: он не различал некоторые цвета и с трудом распознавал определённые протоколы по стандартной раскраске. Таким образом, возможность изменять стандартные правила раскрашивания в некоторой степени повышает доступность программы.

Файлы конфигурации

Полезно понимать, где Wireshark хранит различные параметры конфигурации, на случай если когда-нибудь понадобится изменять эти файлы напрямую. Местоположение файлов конфигурации Wireshark можно узнать, выбрав Help в главном раскрывающемся меню, затем About Wireshark и вкладку Folders. Это окно показано на рисунке 3-10.

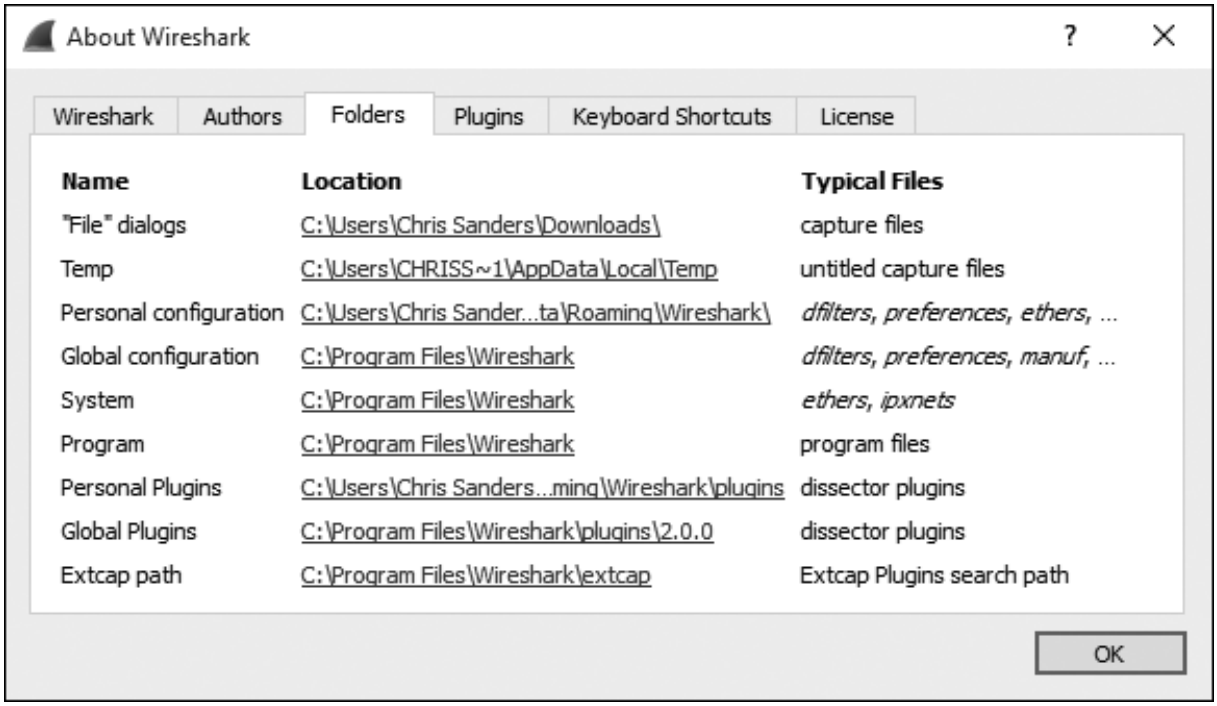


Рисунок 3-10. Поиск файлов конфигурации Wireshark

С точки зрения настройки Wireshark важнее всего два места: личный и глобальный каталоги конфигурации. Глобальный каталог содержит все стандартные настройки Wireshark, и именно там хранит настройки стандартный профиль. В личной папке конфигурации находятся изменённые настройки и профили, относящиеся только к вашей учётной записи. Каждый созданный профиль сохраняется в подкаталоге личной папки конфигурации под заданным вами именем.

Различие между глобальным и личным каталогами конфигурации важно, поскольку любые изменения глобальных файлов конфигурации затронут каждого пользователя Wireshark в системе.

Профили конфигурации

Познакомившись с настройками Wireshark, вы можете обнаружить, что иногда хотите пользоваться одним набором параметров, а затем быстро переключаться на другой для иной ситуации. Чтобы не заставлять вас каждый раз изменять параметры вручную, в Wireshark появились профили конфигурации, позволяющие сохранять наборы настроек.

Профиль конфигурации хранит следующее:

• настройки;
• фильтры захвата;
• фильтры отображения;
• правила раскрашивания;
• отключённые протоколы;
• принудительное декодирование;
• недавние настройки, например размеры панелей, параметры меню View и ширину столбцов;
• таблицы конкретных протоколов, например пользователей SNMP и пользовательские заголовки HTTP.

Чтобы просмотреть список профилей, щёлкните Edit в главном раскрывающемся меню и выберите Configuration Profiles. Можно также щёлкнуть правой кнопкой мыши раздел профилей в правом нижнем углу экрана и выбрать Manage Profiles. В окне Configuration Profiles вы увидите несколько

стандартных профилей Wireshark, в том числе Default, Bluetooth и Classic, показанные на рисунке 3-11. Latency Investigation - добавленный мной пользовательский профиль, показанный прямым шрифтом, тогда как глобальный и стандартный профили выделены курсивом.

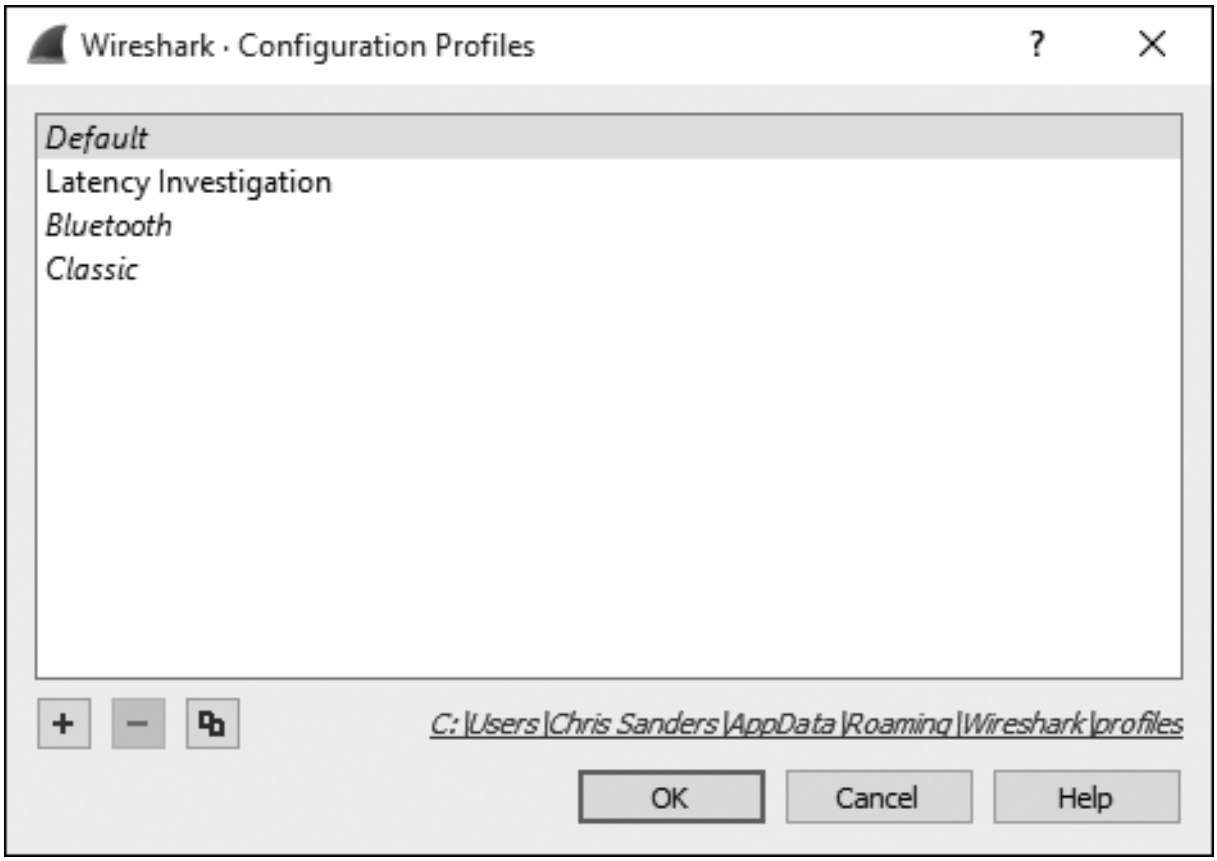


Рисунок 3-11. Просмотр профилей конфигурации

В окне Configuration Profiles можно создавать, копировать, удалять и применять профили конфигурации. Создать новый профиль очень просто:

1. Настройте Wireshark так, как хотите сохранить в профиле.
2. Перейдите в окно Configuration Profiles: щёлкните Edit в главном раскрывающемся меню и выберите Configuration Profiles.
3. Щёлкните кнопку со знаком «плюс» (+) и дайте профилю понятное имя.
4. Щёлкните OK.

Чтобы переключить профиль, можно открыть окно Configuration Profiles, щёлкнуть имя профиля и нажать OK. Быстрее сделать это можно, щёлкнув заголовок Profile в правом нижнем углу окна Wireshark и выбрав нужный профиль, как показано на рисунке 3-12.

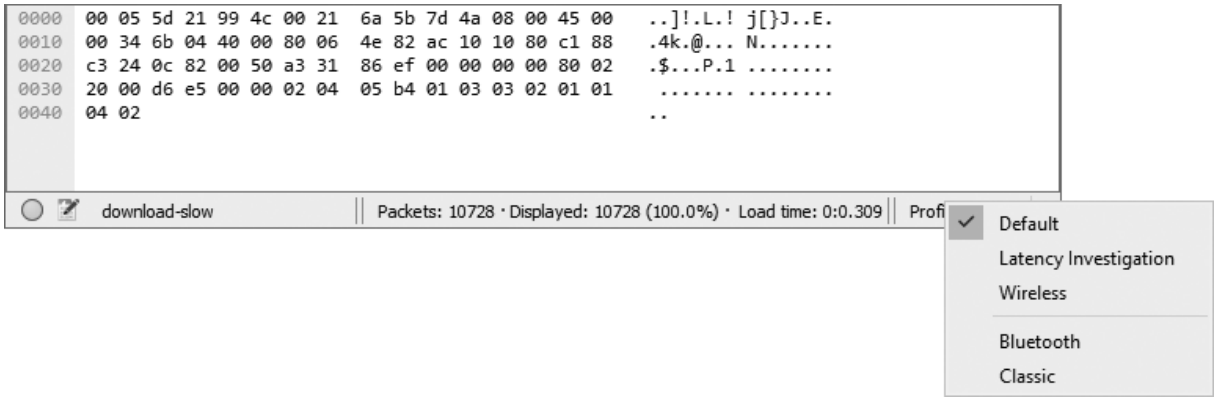


Рисунок 3-12. Быстрое переключение профилей через заголовок Profile

Одна из самых полезных особенностей профилей конфигурации состоит в том, что каждый профиль хранится в собственном каталоге с набором файлов конфигурации. Поэтому профили можно архивировать и передавать другим людям. Вкладка Folders, показанная на рисунке 3-10, содержит пути к личному и глобальному каталогам файлов конфигурации. Чтобы передать профиль пользователю другого компьютера, просто скопируйте папку с именем нужного профиля и вставьте её в тот же каталог соответствующего пользователя на другом компьютере.

При чтении этой книги может понадобиться создать несколько профилей верхнего уровня: для общего поиска неполадок, выявления источника сетевых задержек и расследования проблем безопасности. Не бойтесь пользоваться профилями щедро. Они действительно экономят время, когда требуется быстро включить или отключить несколько параметров. Я встречал людей, которые с большим успехом применяли десятки профилей для разных ситуаций.

Теперь Wireshark установлен и работает, а вы готовы заняться анализом пакетов. В главе 4 рассказывается, как работать с захваченными пакетами.

Глава 4. Работа с захваченными пакетами

Теперь, познакомившись с Wireshark, вы готовы приступить к захвату и анализу пакетов. В этой главе вы узнаете, как работать с файлами захвата, пакетами и форматами отображения времени. Мы также рассмотрим более сложные параметры захвата пакетов и погрузимся в мир фильтров.

Работа с файлами захвата

Вы обнаружите, что значительная часть анализа пакетов выполняется уже после захвата. Обычно вы проводите несколько захватов в разное время, сохраняете их, а затем анализируете все сразу. Поэтому Wireshark позволяет сохранять файлы захвата для последующего анализа. Несколько файлов захвата можно также объединить.

Сохранение и экспорт файлов захвата

Чтобы сохранить захват пакетов, выберите File > Save As. Должно появиться диалоговое окно Save file as, показанное на рисунке 4-1. Вам предложат указать место сохранения захвата пакетов и нужный формат файла. Если формат не задан, Wireshark использует стандартный формат `.pcapng`.

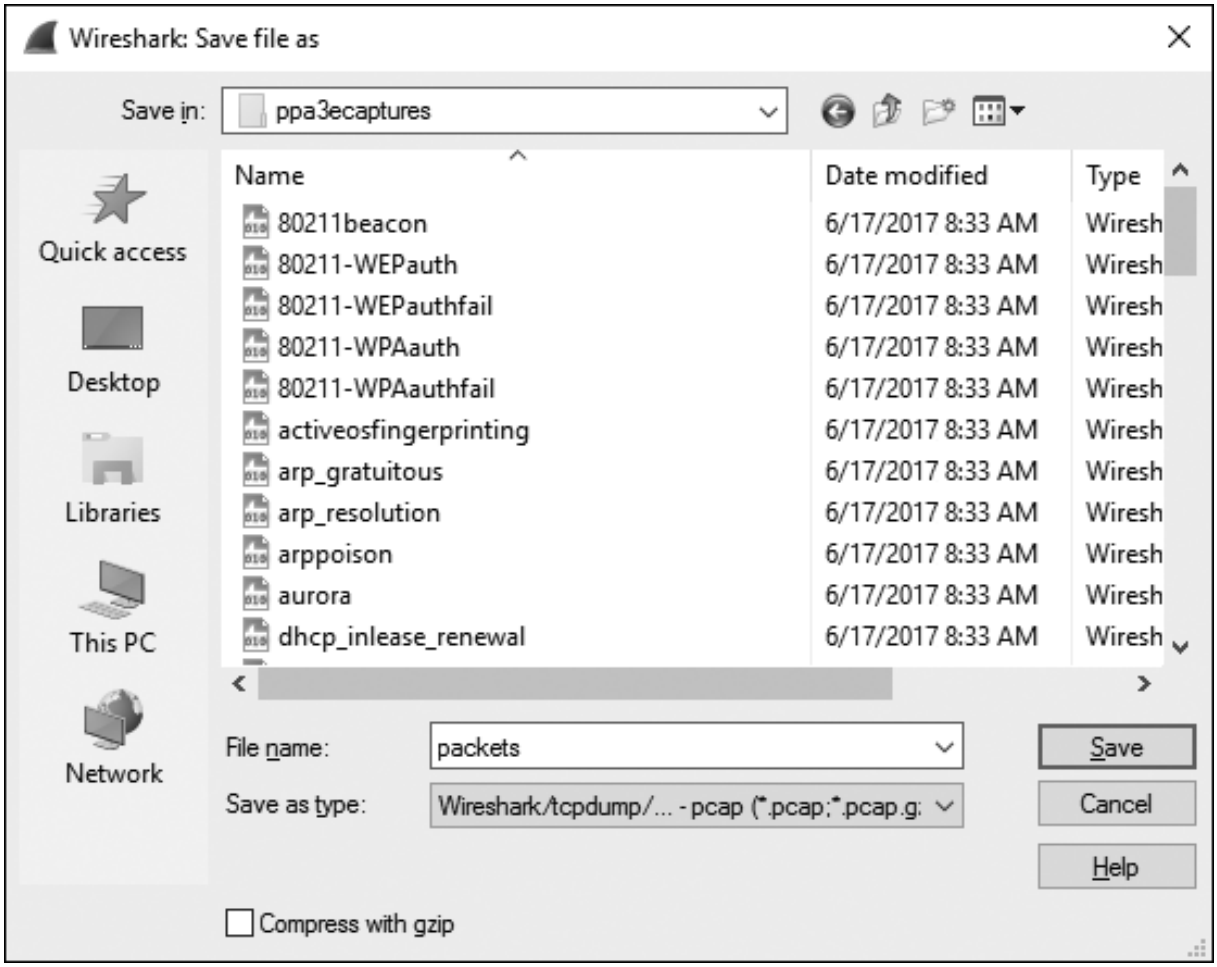


Рисунок 4-1. Диалоговое окно Save file as позволяет сохранять захваты пакетов

Во многих случаях требуется сохранить только часть пакетов из захвата. Для этого выберите File > Export Specified Packets. Появившееся диалоговое окно показано на рисунке 4-2. Это прекрасный способ уменьшить раздутые файлы захвата пакетов. Можно сохранить только пакеты из определённого диапазона номеров, отмеченные пакеты или пакеты, видимые в результате применения фильтра отображения (отмеченные пакеты и фильтры рассматриваются далее в этой главе).

Данные захвата Wireshark можно экспортировать в несколько форматов для просмотра другими средствами или импорта в другие инструменты анализа пакетов. Среди них простой текст, PostScript, значения, разделённые запятыми (CSV), и XML. Чтобы экспортировать захват пакетов в одном из этих форматов, выберите File > Export Packet Dissections, а затем - формат экспортируемого файла. Появится диалоговое окно Save As с параметрами выбранного формата.

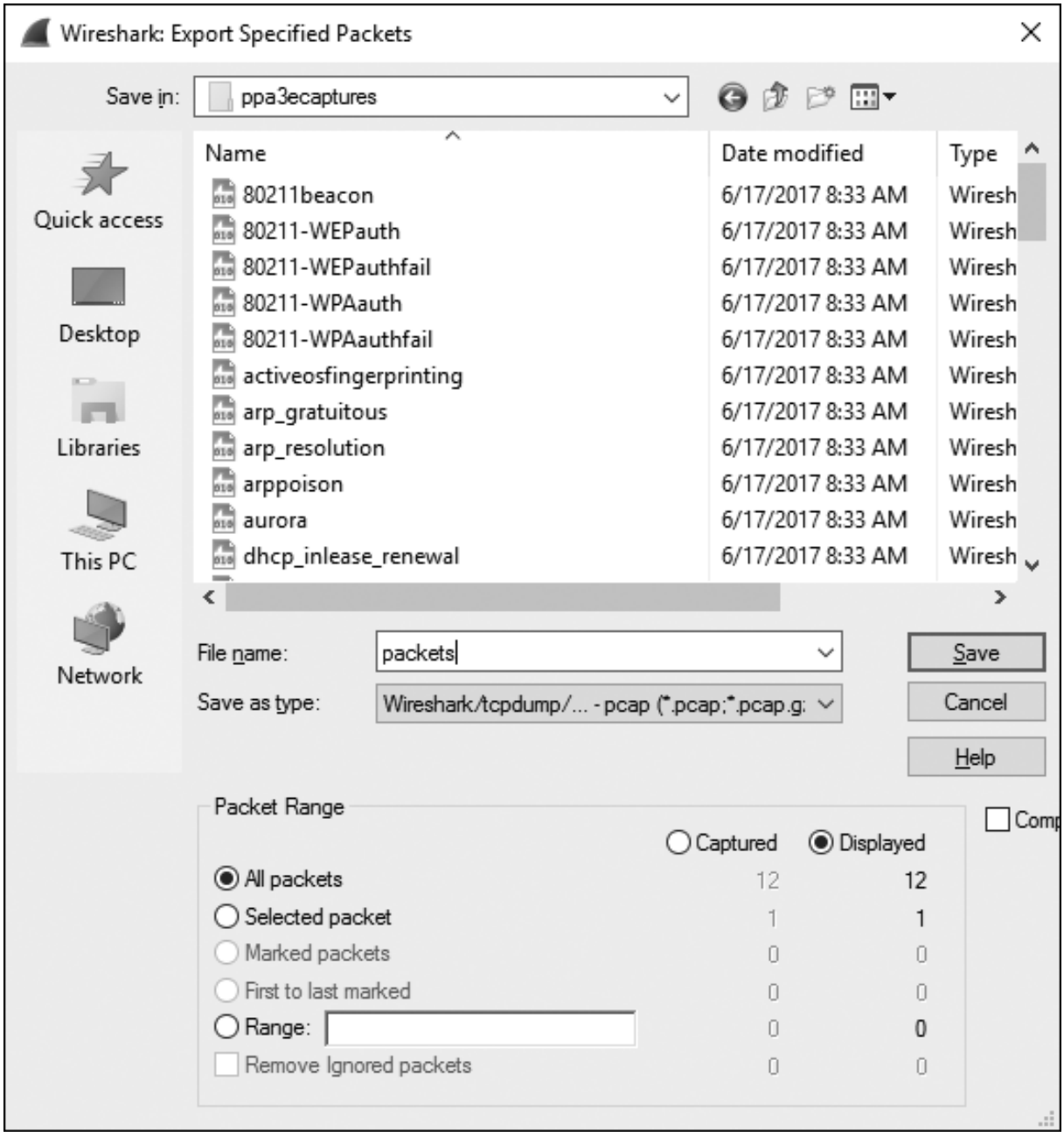


Рисунок 4-2. Диалоговое окно Export Specified Packets даёт более точный контроль над выбором сохраняемых пакетов

Объединение файлов захвата

Некоторые виды анализа требуют объединять несколько файлов захвата. Это часто делают при сравнении двух потоков данных или сведении потоков одного и того же трафика, захваченных по отдельности.

Чтобы объединить файлы захвата, откройте один из нужных файлов и выберите File > Merge: появится диалоговое окно Merge with capture file, показанное на рисунке 4-3. Выберите новый файл, который требуется объединить с уже открытым, а затем укажите способ объединения. Выбранный файл можно поместить перед открытым файлом, добавить после него либо объединить файлы в хронологическом порядке на основании временных меток.

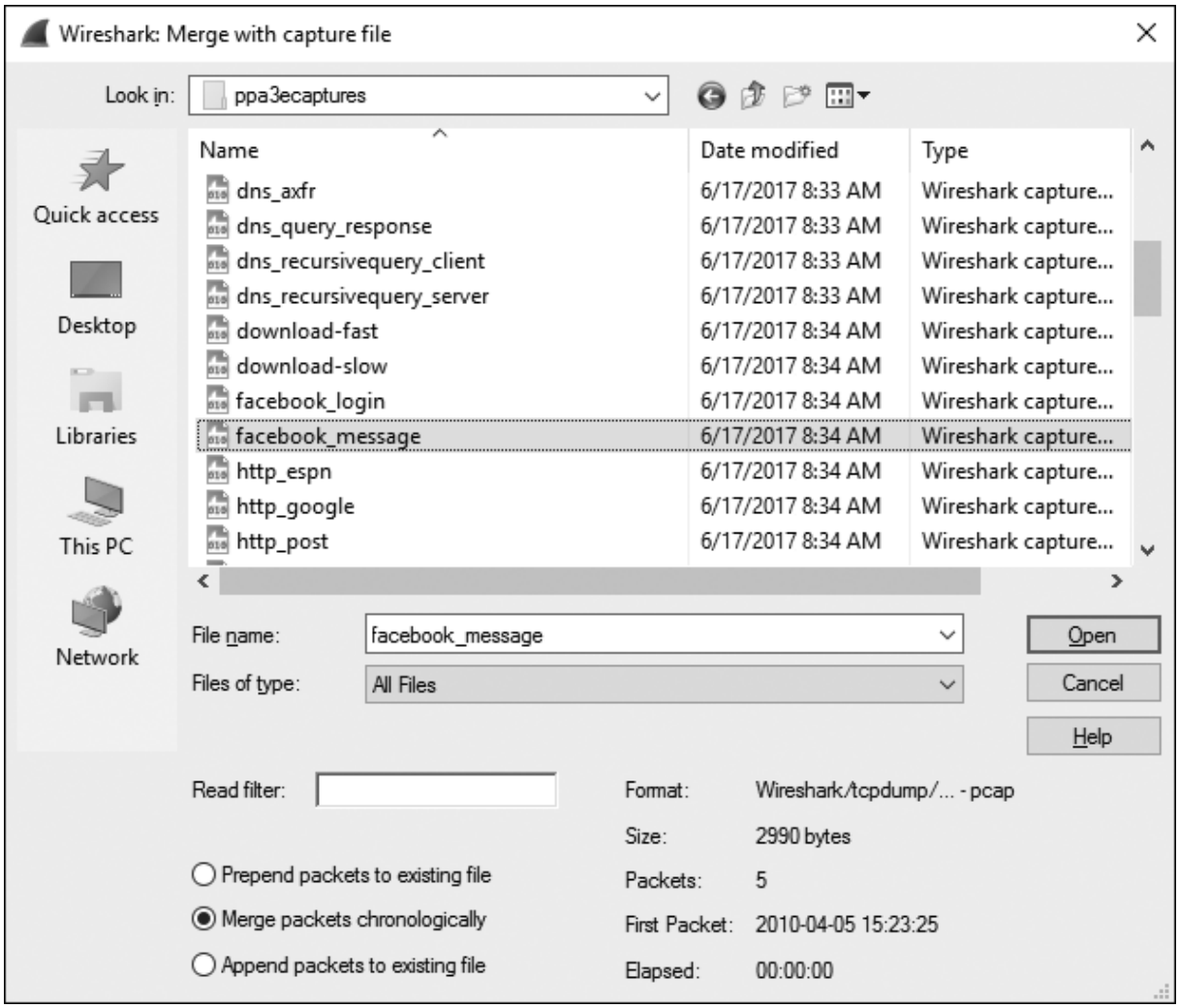


Рисунок 4-3. Диалоговое окно Merge with capture file позволяет объединить два файла захвата

Работа с пакетами

Рано или поздно вы столкнётесь с ситуацией, в которой пакетов очень много. Когда их число возрастёт до тысяч и даже миллионов, понадобится перемещаться по пакетам эффективнее. Для этого Wireshark позволяет находить и отмечать пакеты, соответствующие определённым критериям. Пакеты можно также печатать для удобства обращения к ним.

Поиск пакетов

Чтобы найти пакеты, соответствующие определённым критериям, нажмите `Ctrl+F` и откройте панель Find Packet, обведённую на рисунке 4-4. Эта панель должна появиться между панелью Filter и панелью Packet List.

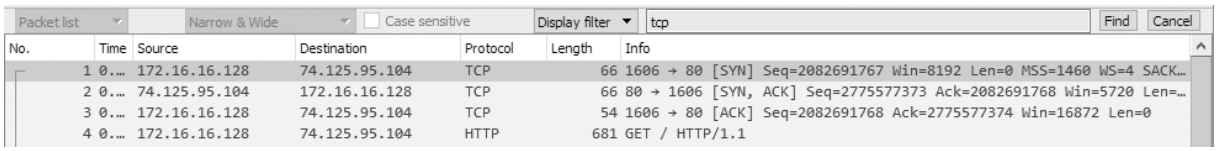


Рисунок 4-4. Поиск пакетов в Wireshark по заданным критериям - в данном случае пакетов, соответствующих выражению фильтра отображения `tcp`

На этой панели есть три варианта поиска пакетов:

• **Display filter** позволяет ввести фильтр в виде выражения, который найдёт только удовлетворяющие ему пакеты. Именно этот вариант используется на рисунке 4-4.
• **Hex value** ищет пакеты с указанным шестнадцатеричным значением.
• **String** ищет пакеты с указанной текстовой строкой. Можно задать панель, в которой выполняется поиск, и включить учёт регистра строки.

Примеры этих видов поиска приведены в таблице 4-1.

Таблица 4-1. Виды поиска пакетов

Вид поиска	Примеры

Display filter	`not ip`
	`ip.addr==192.168.0.1`
	`arp`

Hex value	`00ff`
	`ffff`
	`00ABB1f0`

String	`Workstation1`
	`UserB`
	`domain`

Выбрав вид поиска, введите критерий в текстовое поле и щёлкните Find, чтобы найти первый удовлетворяющий ему пакет. Для поиска следующего совпадающего пакета снова щёлкните Find или нажмите `Ctrl+N`; предыдущий совпадающий пакет находится сочетанием `Ctrl+B`.

Отметка пакетов

Найдя соответствующие критерию пакеты, можно отметить наиболее интересные. Например, это позволит сохранить только отмеченные пакеты. Кроме того, их легко находить по чёрному фону и белому тексту, как показано на рисунке 4-5.

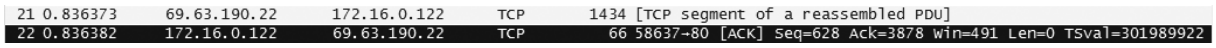


Рисунок 4-5. Отмеченный пакет выделяется на экране. В этом примере отмечен и показан более тёмным второй пакет

Чтобы отметить пакет, щёлкните его правой кнопкой мыши на панели Packet List и выберите во всплывающем меню Mark Packet либо выберите пакет на панели Packet List и нажмите `Ctrl+M`. Чтобы снять отметку, отключите её, снова нажав `Ctrl+M`. В одном захвате можно отметить сколько угодно пакетов. Для перехода вперёд и назад между отмеченными пакетами нажимайте

соответственно `Shift+Ctrl+N` и `Shift+Ctrl+B`.

Печать пакетов

Хотя основная часть анализа выполняется на экране компьютера, иногда захваченные данные нужно печатать. Я время от времени распечатываю пакеты и приклеиваю их к столу, чтобы во время другого анализа можно было быстро сверяться с их содержимым. Очень удобно также печатать пакеты в файл PDF, особенно при подготовке отчётов.

Чтобы распечатать захваченные пакеты, выберите в главном меню File > Print и откройте диалоговое окно Print, показанное на рисунке 4-6.

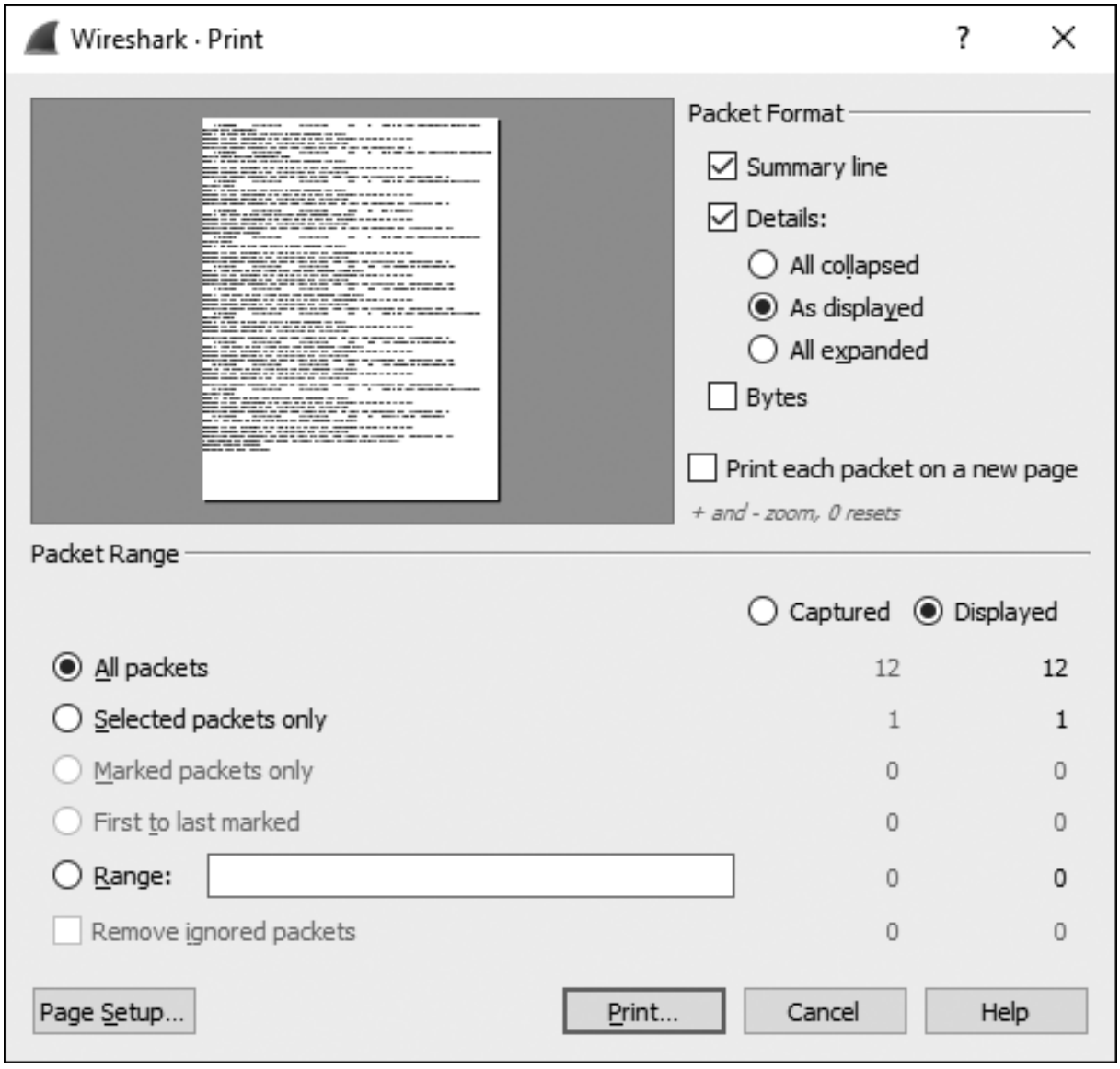


Рисунок 4-6. Диалоговое окно Print позволяет печатать указанные пакеты

Как и в диалоговом окне Export Specified Packets, можно напечатать определённый диапазон пакетов, только отмеченные пакеты или пакеты, показанные в результате применения фильтра. Кроме того, для каждого пакета можно выбрать печатаемый уровень подробности. Задав параметры, щёлкните Print.

Настройка форматов отображения времени и точек отсчёта

Время имеет решающее значение - особенно в анализе пакетов. Всё происходящее в сети чувствительно ко времени, и в файлах захвата вам часто придётся изучать тенденции и сетевые задержки. Wireshark предлагает несколько настраиваемых параметров, связанных со временем. В этом разделе мы рассмотрим форматы его отображения и точки отсчёта.

Форматы отображения времени

Каждый захваченный Wireshark пакет получает временную метку, которую назначает операционная система. Wireshark может показывать абсолютную временную метку, то есть точный момент захвата пакета, а также время относительно последнего захваченного пакета, начала и конца захвата.

Параметры отображения времени находятся в разделе View главного меню. Раздел Time Display Format, показанный на рисунке 4-7, позволяет настроить формат представления и точность отображаемого времени.

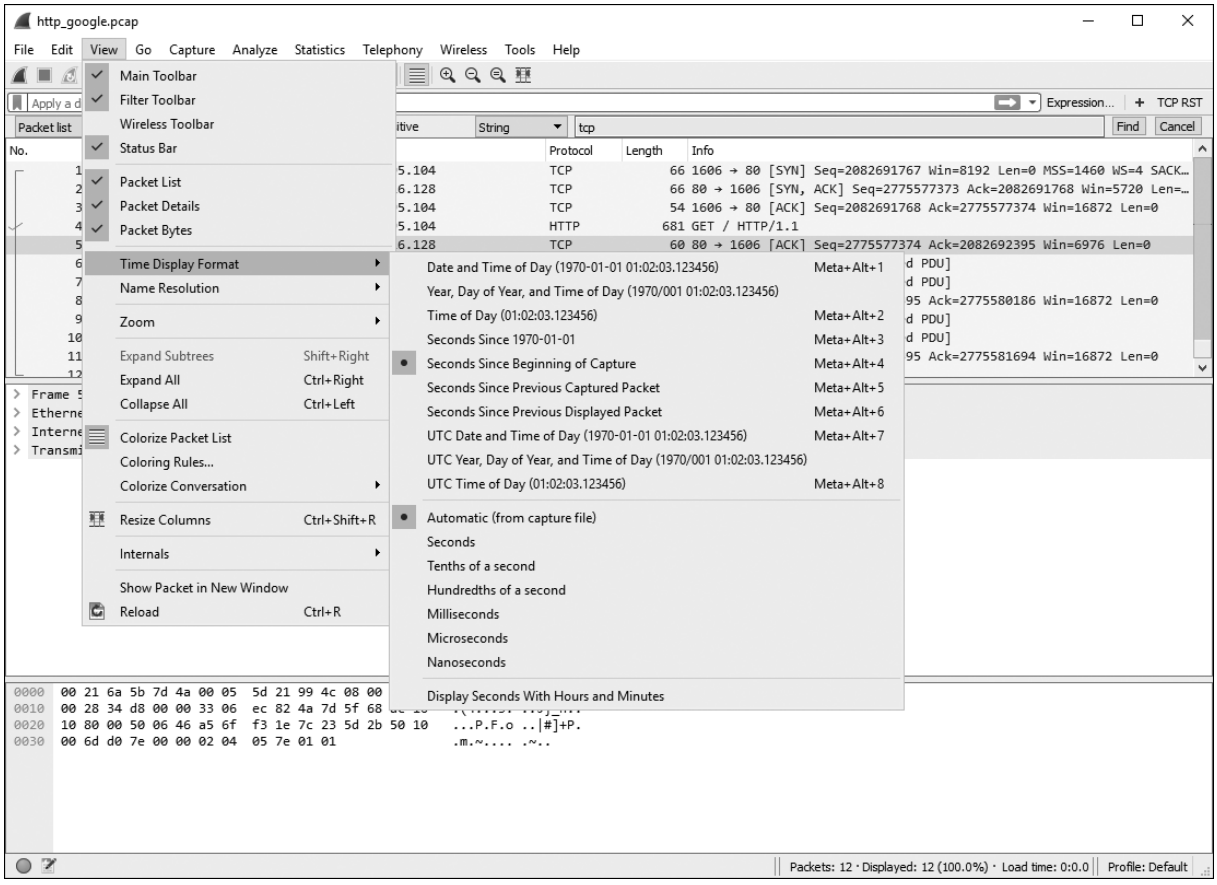


Рисунок 4-7. Доступно несколько форматов отображения времени

Среди форматов представления можно выбирать различные варианты отображения времени: дату и время суток, дату и время суток UTC, секунды с начала эпохи, секунды с начала захвата (стандартное значение), секунды с момента захвата предыдущего пакета и другие.

Параметры точности позволяют задать автоматическую точность отображения, при которой формат берётся из файла захвата, или выбрать её вручную: секунды, миллисекунды, микросекунды и так далее. Далее в книге мы будем изменять эти параметры, поэтому познакомьтесь с ними сейчас.

Примечание. Сравнивая данные пакетов с нескольких устройств, убедитесь, что устройства синхронизированы с одним источником времени, особенно при криминалистическом анализе или поиске неполадок. Для синхронизации сетевых устройств можно использовать протокол сетевого времени (Network Time Protocol, NTP). Если исследуемые пакеты получены от устройств из нескольких часовых поясов, во избежание путаницы при подготовке отчёта рассмотрите возможность анализировать их время в UTC, а не местное время.

Отсчёт времени от пакета

Отсчёт времени от пакета позволяет назначить определённый пакет точкой, относительно которой выполняются все последующие вычисления времени. Эта функция особенно удобна при изучении последовательности событий, которая запускается в некоторый момент, не совпадающий с началом файла захвата.

Чтобы назначить пакет точкой отсчёта времени, щёлкните опорный пакет правой кнопкой мыши на панели Packet List и выберите Set/Unset Time Reference. Для отключения точки отсчёта повторите то же действие. Можно также включать и отключать этот режим, выбрав нужный пакет на панели Packet List и нажав `Ctrl+T`.

Когда для пакета включена точка отсчёта, в столбце Time панели Packet List появляется `*REF*`, как показано на рисунке 4-8.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.030107	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406…
3	0.030182	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
4	*REF*	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/1.1
5	0.048778	74.125.95.104	172.16.16.128	TCP	60	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
6	0.070954	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]
7	0.071217	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]
8	0.071247	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082692395 Ack=2775580186 Win=16872 Len=0

Рисунок 4-8. Для пакета 4 включён режим точки отсчёта времени

Назначать пакет точкой отсчёта имеет смысл только тогда, когда формат отображения времени захвата настроен на показ времени относительно начала захвата. При любой другой настройке полезных результатов не получится; более того, возникнет набор времён, способный сильно запутать.

Сдвиг времени

Иногда могут встретиться пакеты из нескольких источников, не синхронизированных с одним источником времени. Особенно часто это бывает при изучении файлов захвата из двух мест, содержащих один и тот же поток данных. Хотя большинство администраторов стремится синхронизировать каждое устройство сети, между устройствами определённых типов нередко сохраняется расхождение в несколько секунд. Чтобы облегчить анализ в такой ситуации, Wireshark позволяет сдвигать временные метки пакетов.

Чтобы сдвинуть временную метку одного или нескольких пакетов, выберите Edit > Time Shift либо нажмите `Ctrl+Shift+T`. В открывшемся окне Time Shift можно задать интервал времени, на который сдвигается весь файл захвата, либо указать время для отдельных пакетов. В примере на рисунке 4-9 я решил сдвинуть временную метку каждого пакета захвата, прибавив к ней две минуты и пять секунд.

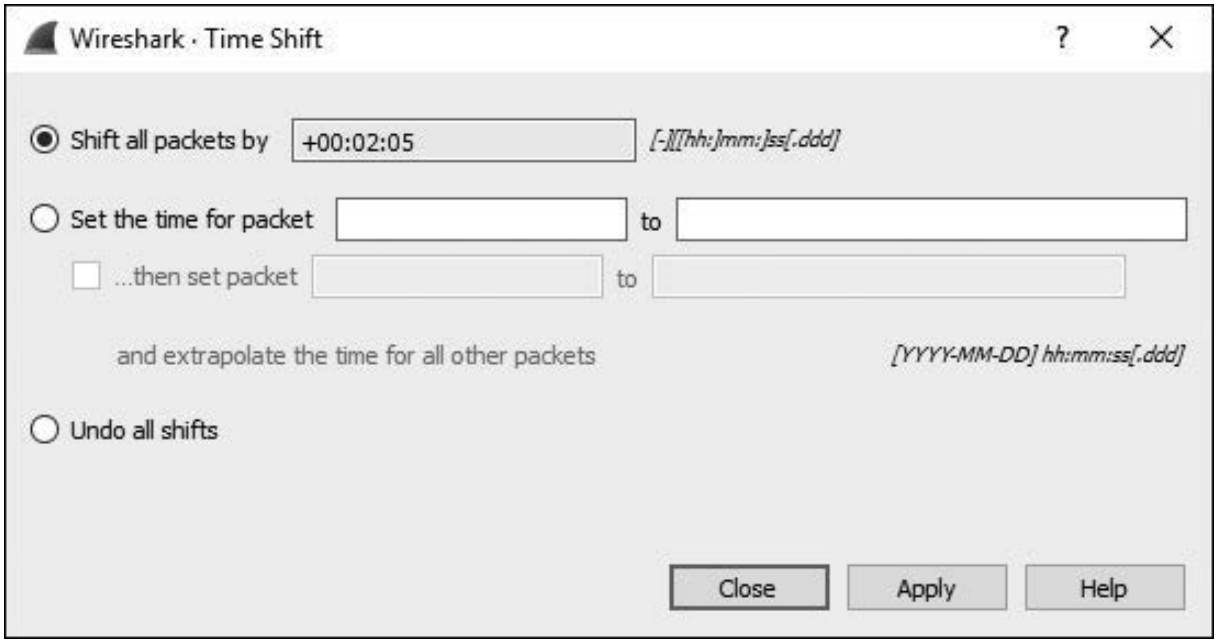


Рисунок 4-9. Диалоговое окно Time Shift

Настройка параметров захвата

В предыдущей главе при выполнении самого простого захвата пакетов мы уже видели диалоговое окно Capture Interfaces. В Wireshark имеется немало дополнительных параметров захвата, которых тогда мы не касались. Чтобы получить к ним доступ, выберите Capture > Options.

В диалоговом окне Capture Interfaces масса возможностей, призванных повысить гибкость захвата пакетов. Оно разделено на три вкладки: Input, Output и Options. Рассмотрим каждую отдельно.

Вкладка Input

Основное назначение вкладки Input (рисунок 4-10) - показывать все доступные для захвата пакетов интерфейсы и основные сведения о каждом. Сюда входят понятное имя интерфейса, предоставленное операционной системой, график трафика с пропускной способностью интерфейса и дополнительные параметры конфигурации, например состояние неразборчивого режима и размер буфера. В крайней правой части, не попавшей на рисунок, есть также столбец применённого фильтра захвата, который обсуждается в разделе «Фильтры захвата» на странице 65.

Большинство этих параметров можно щёлкать и изменять прямо на месте. Например, чтобы отключить неразборчивый режим интерфейса, щёлкните соответствующее поле и выберите в раскрывающемся меню вместо включённого состояния отключённое.

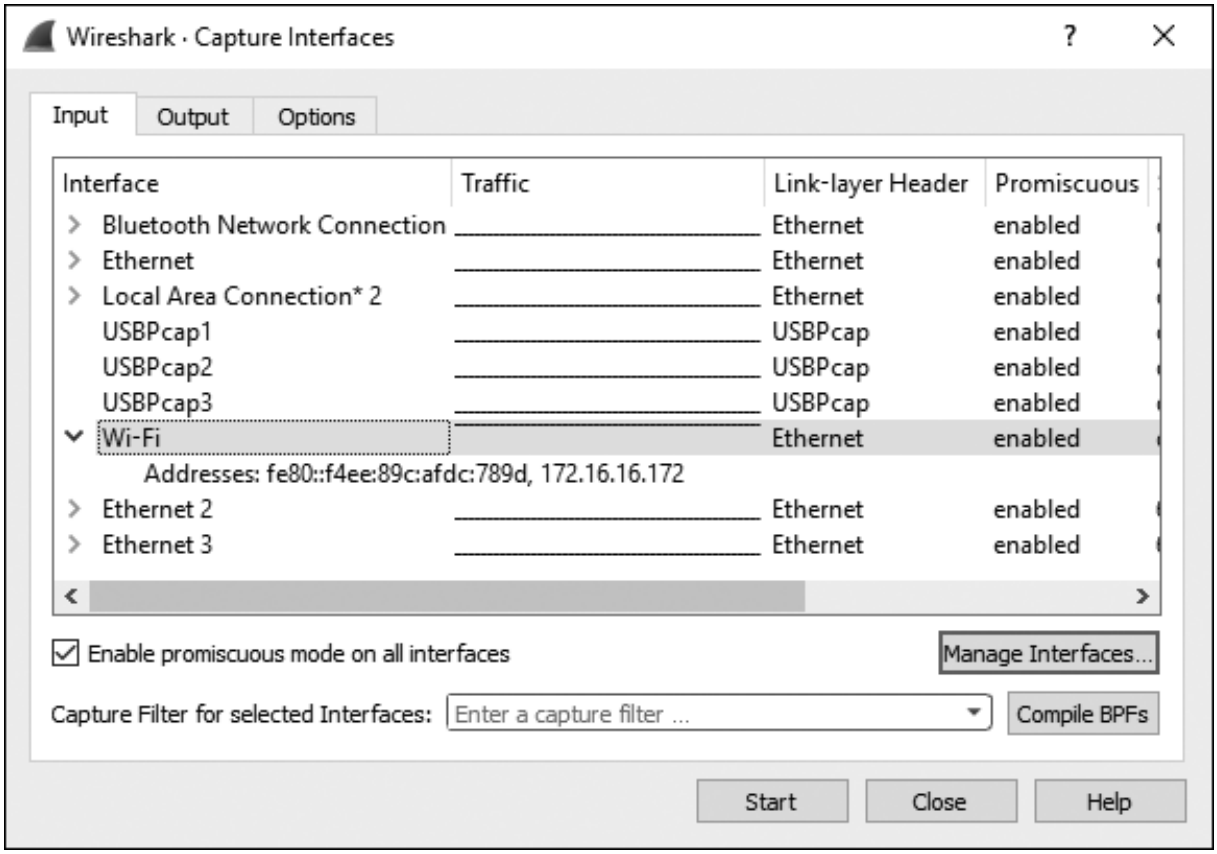


Рисунок 4-10. Вкладка параметров Input окна Capture Interfaces

Вкладка Output

Вкладка Output (рисунок 4-11) позволяет автоматически сохранять захваченные пакеты в файл вместо того, чтобы сначала захватить их, а затем сохранять файл. Это даёт больше гибкости в управлении сохранением пакетов. Можно сохранять их в одном файле или в наборе файлов и даже управлять числом создаваемых файлов с помощью кольцевого буфера, который мы вскоре рассмотрим. Чтобы включить этот вариант, введите полный путь и имя файла в текстовое поле File. Можно также выбрать каталог кнопкой Browse... и указать имя файла.

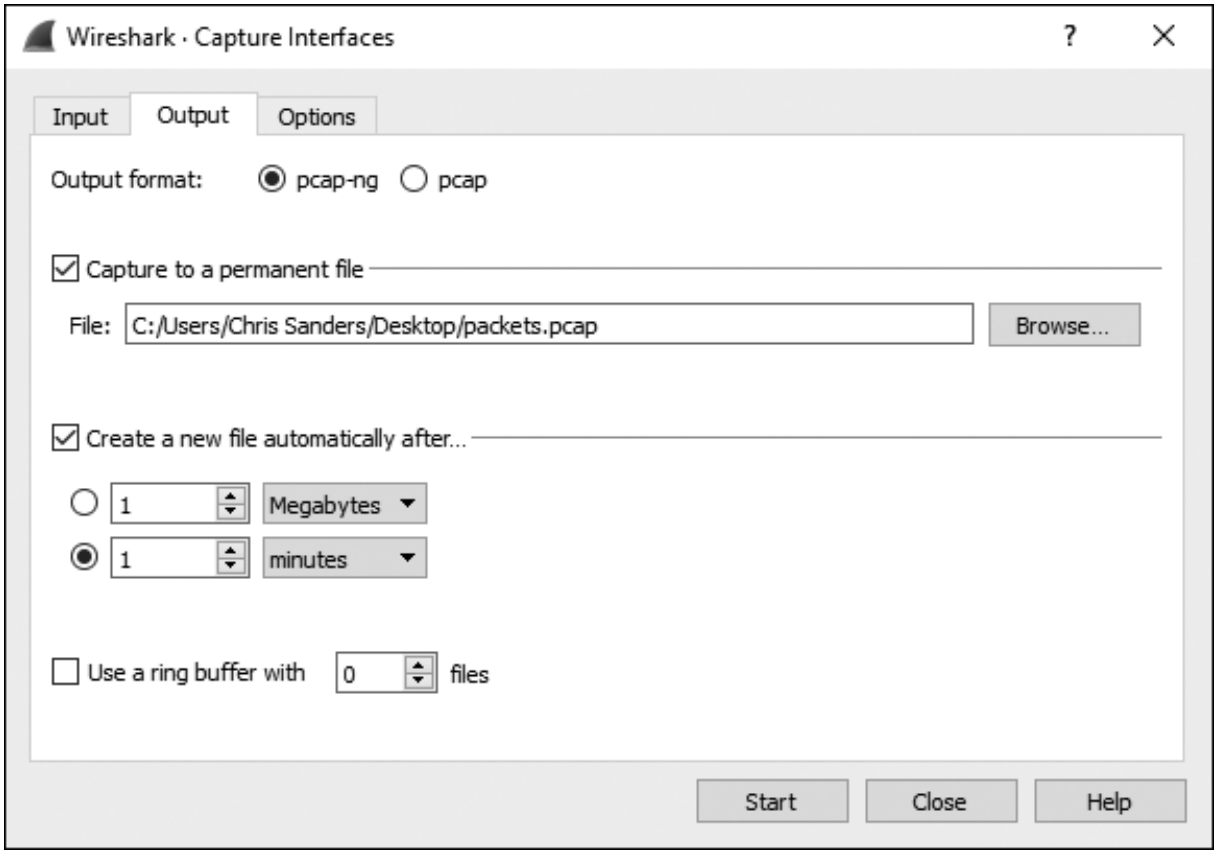


Рисунок 4-11. Вкладка параметров Output окна Capture Interfaces

При захвате большого объёма трафика или продолжительном захвате особенно полезны наборы файлов. Набор файлов - это группа из нескольких файлов, разделяемых при наступлении определённого условия. Чтобы сохранять данные в набор, установите флажок Create a new file automatically after....

Для управления сохранением в наборы Wireshark использует разные условия, основанные на размере файла или времени. Чтобы включить одно из них, выберите переключатель рядом с вариантом по размеру или времени, а затем задайте пороговое значение и единицу измерения. Например, можно создать новый файл после каждого захваченного 1 Мбайт трафика или, как показано на рисунке 4-12, после каждой минуты захвата.

Name	Date modified	Type	Size
intervalcapture_00001_20151009141804	10/9/2017 2:19 PM	File	172 KB
intervalcapture_00002_20151009141904	10/9/2017 2:20 PM	File	25 KB
intervalcapture_00003_20151009142004	10/9/2017 2:21 PM	File	3,621 KB
intervalcapture_00004_20151009142104	10/9/2017 2:22 PM	File	52 KB
intervalcapture_00005_20151009142204	10/9/2017 2:23 PM	File	47 KB
intervalcapture_00006_20151009142304	10/9/2017 2:24 PM	File	37 KB

Рисунок 4-12. Набор файлов, создаваемых Wireshark с интервалом в одну минуту

Параметр Use a ring buffer позволяет задать количество файлов в наборе, после которого Wireshark начнёт перезаписывать файлы. Хотя у термина «кольцевой буфер» несколько значений, для наших целей это, по существу, набор файлов, в котором после записи последнего допустимого файла при необходимости сохранить новые данные перезаписывается первый файл. Иными словами, устанавливается порядок записи «первым пришёл - первым ушёл» (first in, first out, FIFO). Можно

включить этот параметр и указать максимальное число файлов в цикле. Предположим, вы выбрали для захвата несколько файлов с созданием нового каждый час и задали размер кольцевого буфера 6. После создания шестого файла буфер вернётся к началу и вместо седьмого перезапишет первый. Благодаря этому на диске останется не более шести файлов, то есть в данном случае часов данных, но новые данные по-прежнему смогут записываться.

Наконец, на вкладке Output можно указать, использовать ли формат `.pcapng`. Если сохранённые пакеты планируется открывать средством, которое не умеет разбирать `.pcapng`, можно выбрать традиционный формат `.pcap`.

Вкладка Options

Вкладка Options содержит ряд других параметров захвата пакетов, в том числе варианты отображения, разрешения имён и завершения захвата, показанные на рисунке 4-13.

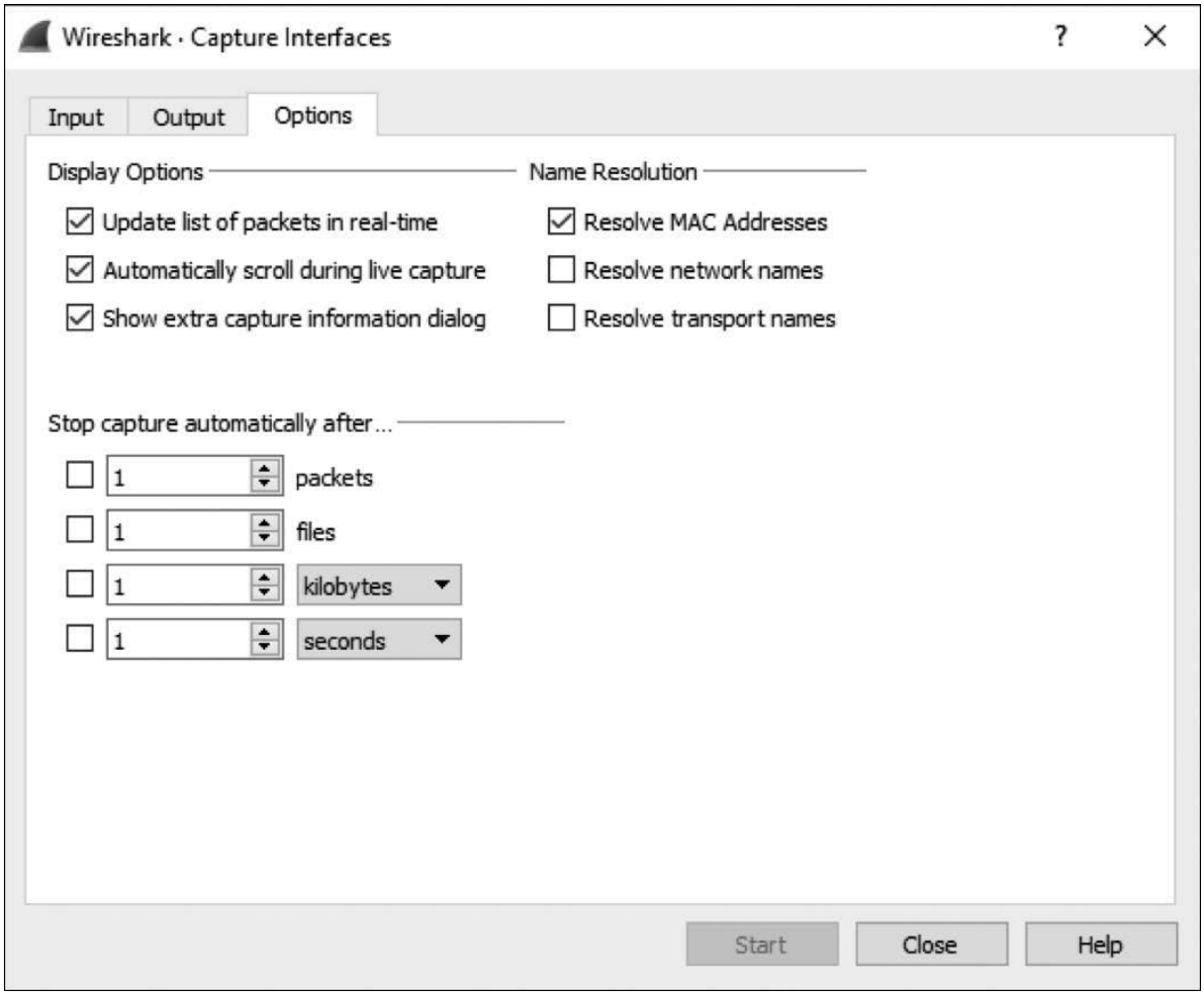


Рисунок 4-13. Вкладка Options окна Capture Interfaces

Параметры отображения

Раздел Display Options управляет показом пакетов во время захвата. Назначение параметра Update list of packets in real-time понятно из названия; его можно использовать вместе с Automatically scroll during live capture. Если включены оба параметра, на экране показываются все захваченные пакеты, причём самые свежие появляются немедленно.

> **Предупреждение.** Совместное использование Update list of packets in real-time и Automatically scroll during live capture может сильно нагружать процессор даже при захвате умеренного объёма данных. Если нет конкретной необходимости видеть пакеты в реальном времени, лучше отключить оба параметра.

Параметр Show extra capture information dialog позволяет включать или отключать небольшое окно, показывающее количество и долю захваченных пакетов с разбивкой по протоколам. Я предпочитаю показывать окно сведений о захвате, поскольку обычно не включаю прокрутку пакетов в реальном времени во время захвата.

Настройки разрешения имён

Параметры раздела Name Resolution позволяют включить автоматическое разрешение имён MAC (уровень 2), сетевых (уровень 3) и транспортных (уровень 4) адресов для захвата. Разрешение имён в целом, в том числе его недостатки, подробнее обсуждается в главе 5.

Параметры остановки захвата

Раздел Stop capture automatically after... позволяет остановить выполняющийся захват при наступлении определённых условий. Как и в случае наборов из нескольких файлов, остановку можно запускать по размеру файла и интервалу времени, а также по количеству пакетов. Эти параметры можно использовать вместе с вариантами нескольких файлов на вкладке Output.

Использование фильтров

Фильтры позволяют указать, какие пакеты будут доступны для анализа. Проще говоря, фильтр - это выражение, определяющее критерии включения или исключения пакетов. Если вы не хотите видеть какие-либо пакеты, можно написать фильтр, который от них избавится. Если вы хотите видеть только определённые пакеты, можно написать фильтр, показывающий исключительно их.

Wireshark предлагает два основных вида фильтров:

• **фильтры захвата** задаются во время захвата пакетов и захватывают лишь пакеты, которые указанное выражение предписывает включить или исключить;
• **фильтры отображения** применяются к уже существующему набору захваченных пакетов и скрывают ненужные либо показывают нужные пакеты в соответствии с заданным выражением.

Сначала рассмотрим фильтры захвата.

Фильтры захвата

Фильтры захвата применяются во время самого захвата и с самого начала ограничивают пакеты, передаваемые аналитику. Одна из главных причин использовать такой фильтр - производительность. Если вы знаете, что определённый вид трафика анализировать не нужно, его можно просто исключить фильтром и сэкономить вычислительные ресурсы, которые обычно затрачивались бы на захват этих пакетов.

Возможность создавать собственные фильтры захвата особенно полезна при работе с большими объёмами данных. Анализ ускоряется, если вы с самого начала рассматриваете только пакеты, относящиеся к исследуемой проблеме.

Предположим, например, что вы ищете неполадку службы на порте 262, но анализируемый сервер обслуживает несколько разных служб на множестве портов. Даже один лишь поиск и анализ трафика единственного порта потребует немалой работы. Чтобы захватить трафик только определённого порта, можно применить фильтр захвата. В диалоговом окне Capture Interfaces выполните следующие действия:

1. Выберите Capture > Options рядом с интерфейсом, на котором хотите захватывать пакеты. Откроется диалоговое окно Capture Interfaces.
2. Найдите нужный интерфейс и прокрутите до параметра Capture Filter в крайнем правом столбце.
3. Чтобы применить фильтр захвата, щёлкните этот столбец и введите выражение. Нам нужно показывать только входящий и исходящий трафик порта 262, поэтому введите `port 262`, как показано на рисунке 4-14. (Выражения подробнее обсуждаются в следующем разделе.) Ячейка должна стать зелёной, показывая, что введено допустимое выражение; недопустимое выражение окрасит её в красный цвет.

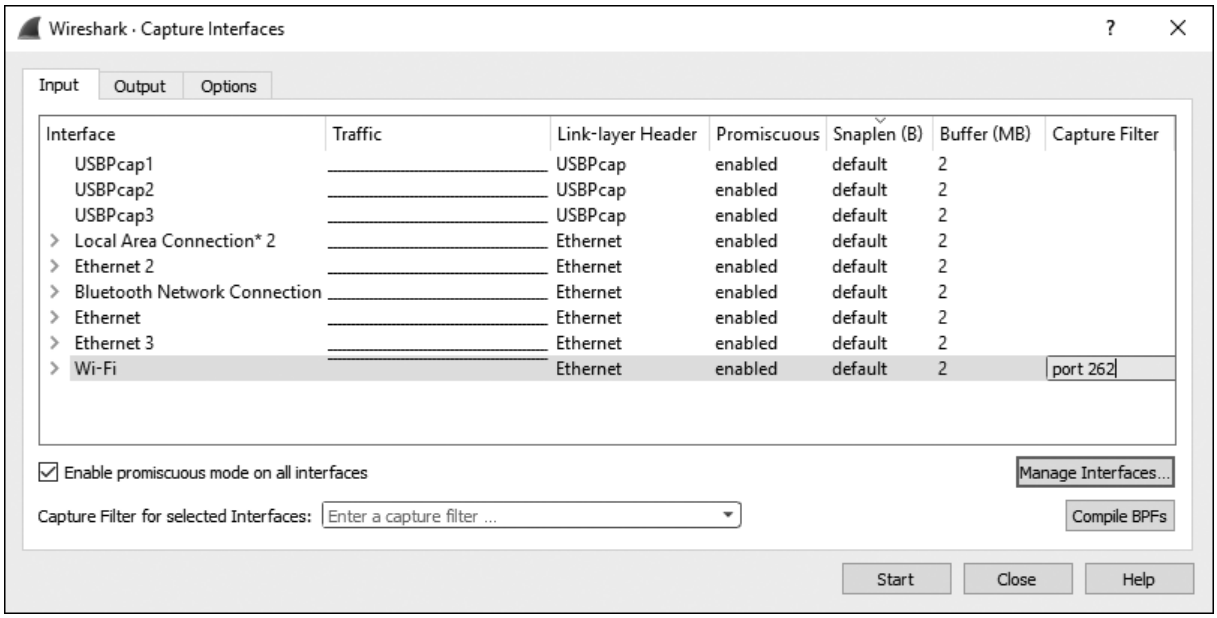


Рисунок 4-14. Создание фильтра захвата в диалоговом окне Capture Interfaces

4. Настроив фильтр, щёлкните Start, чтобы начать захват.

Теперь должен показываться только трафик порта 262, и эти данные можно анализировать эффективнее.

Синтаксис фильтров захвата/BPF

Фильтры захвата применяются библиотеками libpcap/WinPcap и используют синтаксис фильтра пакетов Беркли (Berkeley Packet Filter, BPF). Этот синтаксис распространён в нескольких приложениях перехвата пакетов, главным образом потому, что такие приложения обычно опираются на библиотеки libpcap/WinPcap, которые позволяют применять BPF. Знание синтаксиса BPF будет чрезвычайно важно по мере более глубокого погружения в сети на уровне пакетов.

Созданный в синтаксисе BPF фильтр называется выражением, и каждое выражение состоит из одного или нескольких примитивов. Примитив включает один или несколько квалификаторов, перечисленных в таблице 4-2, после которых следует идентификатор - имя или число, как показано на рисунке 4-15.

Таблица 4-2. Квалификаторы BPF

Квалификатор	Описание	Примеры
Type	Указывает, к чему относится имя или число идентификатора	`host`, `net`, `port`
Dir	Задаёт направление передачи к идентификатору или от него	`src`, `dst`
Proto	Ограничивает совпадение определённым протоколом	`ether`, `ip`, `tcp`, `udp`, `http`, `ftp`

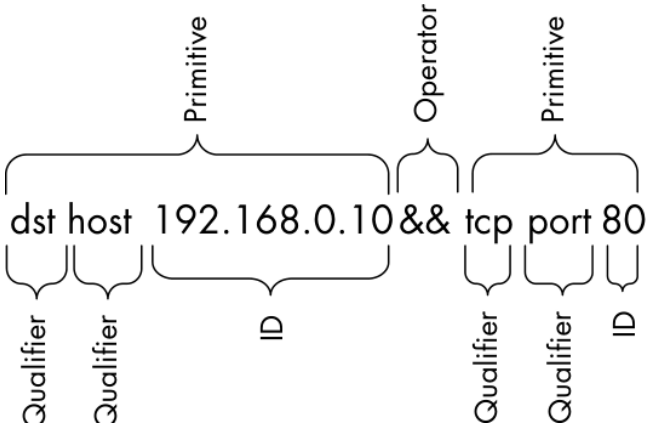


Рисунок 4-15. Пример фильтра захвата: `dst host 192.168.0.10 && tcp port 80`

Если учесть составные части выражения, квалификатор `dst host` вместе с идентификатором `192.168.0.10` образует примитив. Этот примитив сам по себе является выражением, которое захватывает лишь трафик с IP-адресом назначения `192.168.0.10`.

Примитивы можно объединять логическими операторами, создавая более сложные выражения. Доступны три логических оператора:

• оператор конъюнкции AND (`&&`);
• оператор дизъюнкции OR (`||`);
• оператор отрицания NOT (`!`).

Например, следующее выражение захватит только трафик с исходным IP-адресом `192.168.0.10` и исходным либо конечным портом 80:

```
src host 192.168.0.10 && port 80
```

Фильтры имён узлов и адресов

Большинство создаваемых фильтров будет сосредоточено на определённом сетевом устройстве или группе устройств. В зависимости от обстоятельств фильтрация может основываться на MAC-адресе, адресе IPv4, адресе IPv6 или имени узла DNS.

Допустим, вас интересует трафик определённого узла, взаимодействующего с сервером вашей сети. На сервере можно создать фильтр с квалификатором `host`, который захватит весь трафик, связанный с IPv4-адресом этого узла:

```
host 172.16.16.149
```

В сети IPv6 фильтрация выполняется по IPv6-адресу с тем же квалификатором `host`:

```
host 2001:db8:85a3::8a2e:370:7334
```

С помощью квалификатора `host` можно также фильтровать по имени узла:

```
host testserver2
```

Если вас беспокоит, что IP-адрес узла может измениться, можно фильтровать и по его MAC-адресу, добавив квалификатор протокола `ether`:

```
ether host 00-1a-a0-52-e2-a0
```

Квалификаторы направления передачи часто используются вместе с фильтрами, подобными предыдущим, чтобы захватывать трафик в зависимости от того, идёт ли он к узлу или от него. Например, для захвата только исходящего от определённого узла трафика добавьте квалификатор `src`:

```
src host 172.16.16.149
```

Чтобы захватить только данные, направленные узлу `172.16.16.149`, примените квалификатор `dst`:

```
dst host 172.16.16.149
```

Если в примитиве не используется квалификатор типа (`host`, `net` или `port`), предполагается квалификатор `host`. Поэтому следующее выражение без этого квалификатора равнозначно предыдущему:

```
dst 172.16.16.149
```

Фильтры портов

Кроме узлов, можно фильтровать по портам, используемым в каждом пакете. Фильтрация портов позволяет отбирать службы и приложения, которые используют известные служебные порты. Например, вот простой фильтр для захвата трафика только к порту 8080 или от него:

```
port 8080
```

Чтобы захватить весь трафик, кроме проходящего через порт 8080, подойдёт следующий фильтр:

```
!port 8080
```

Фильтры портов можно объединять с квалификаторами направления передачи. Например, чтобы захватить только трафик, направляемый веб-серверу, который прослушивает стандартный порт HTTP 80, примените квалификатор `dst`:

```
dst port 80
```

Фильтры протоколов

Фильтры протоколов позволяют отбирать пакеты по определённым протоколам. Они применяются для совпадения с протоколами не прикладного уровня, которые нельзя определить просто по использованию определённого порта. Поэтому, если требуется видеть только трафик ICMP, можно использовать следующий фильтр:

```
icmp
```

Чтобы видеть весь трафик, кроме IPv6, подойдёт такой фильтр:

```
!ip6
```

Фильтры полей протоколов

Одна из настоящих сильных сторон синтаксиса BPF - возможность исследовать каждый байт заголовка протокола и создавать по этим данным очень точные фильтры. Сложные фильтры из этого раздела позволяют извлечь определённое число байтов пакета, начиная с указанного места.

Предположим, например, что нужно фильтровать по полю типа заголовка ICMP. Поле типа находится в самом начале пакета, то есть имеет смещение 0. Чтобы указать исследуемое место в пакете, запишите смещение байта в квадратных скобках после квалификатора протокола - в данном примере `icmp[0]`. Эта конструкция возвращает однобайтовое целое значение, которое можно сравнивать. Например, чтобы получить лишь пакеты ICMP с сообщением о недоступности назначения (тип 3), в выражении фильтра применяется оператор равенства:

```
icmp[0] == 3
```

Чтобы исследовать только пакеты ICMP, представляющие эхо-запрос (тип 8) или эхо-ответ (тип 0), используйте два примитива с оператором OR:

```
icmp[0] == 8 || icmp[0] == 0
```

Эти фильтры прекрасно работают, но основываются лишь на одном байте сведений из заголовка пакета. В выражении фильтра можно также задать длину возвращаемых данных, добавив после номера смещения внутри квадратных скобок длину в байтах через двоеточие.

Допустим, требуется создать фильтр для захвата всех пакетов ICMP «назначение недоступно, узел недоступен», которые определяются типом 3 и кодом 1. Это однобайтовые поля, расположенные рядом со смещения 0 заголовка пакета. Создадим фильтр, который проверяет два байта данных начиная со смещения 0 заголовка и сравнивает их с шестнадцатеричным значением `0301` (тип 3, код 1):

```
icmp[0:2] == 0x0301
```

Распространённая задача - захватить только пакеты TCP с установленным флагом RST. Протокол TCP подробно рассматривается в главе 8. Пока достаточно знать, что флаги пакета TCP расположены по смещению 13. Это интересное поле: совокупное поле флагов занимает один байт, но каждый отдельный флаг определяется одним битом этого байта. Как подробнее объясняется в приложении B, каждый бит байта представляет определённое число по основанию 2. Бит, в котором хранится флаг, задаётся представляемым этим битом числом: первый бит представляет 1, второй - 2, третий - 4 и так далее. В пакете TCP одновременно может быть установлено несколько флагов. Поэтому эффективно фильтровать по единственному значению `tcp[13]` нельзя: установленному биту RST может соответствовать несколько значений.

Вместо этого нужно указать исследуемое место внутри байта, добавив одиночный амперсанд (&), а затем число, представляющее позицию флага. Флаг RST находится в бите, который представляет в

этом байте число 4, и значение этого бита 4 говорит о том, что флаг RST установлен. Фильтр выглядит так:

```
tcp[13] & 4 == 4
```

Чтобы увидеть все пакеты с установленным флагом PSH, который определяется разрядом с числом 8 в поле флагов TCP по смещению 13, используйте в фильтре эту позицию:

```
tcp[13] & 8 == 8
```

Примеры выражений фильтров захвата

Нередко успех или неудача анализа зависит от способности создавать фильтры, соответствующие текущей ситуации. В таблице 4-3 приведены распространённые фильтры захвата, которыми вы можете часто пользоваться.

Таблица 4-3. Распространённые фильтры захвата

Фильтр	Описание
`tcp[13] & 32 == 32`	Пакеты TCP с установленным флагом URG
`tcp[13] & 16 == 16`	Пакеты TCP с установленным флагом ACK
`tcp[13] & 8 == 8`	Пакеты TCP с установленным флагом PSH
`tcp[13] & 4 == 4`	Пакеты TCP с установленным флагом RST
`tcp[13] & 2 == 2`	Пакеты TCP с установленным флагом SYN
`tcp[13] & 1 == 1`	Пакеты TCP с установленным флагом FIN
`tcp[13] == 18`	Пакеты TCP SYN-ACK
`ether host 00:00:00:00:00:00`	Трафик к вашему MAC-адресу или от него
`!ether host 00:00:00:00:00:00`	Трафик не к вашему MAC-адресу и не от него
`broadcast`	Только широковещательный трафик
`icmp`	Трафик ICMP
`icmp[0:2] == 0x0301`	ICMP: назначение недоступно, узел недоступен
`ip`	Только трафик IPv4
`ip6`	Только трафик IPv6
`udp`	Только трафик UDP

Фильтры отображения

Фильтр отображения после применения к файлу захвата предписывает Wireshark показывать лишь совпадающие с ним пакеты. Его можно ввести в текстовое поле Filter над панелью Packet List.

Фильтры отображения используются чаще фильтров захвата, поскольку позволяют отбирать видимые данные пакетов, в действительности не исключая остальные данные из файла захвата. Если понадобится вернуться к исходному захвату, достаточно удалить выражение фильтра. Кроме того, благодаря обширной библиотеке анализаторов пакетов Wireshark эти фильтры намного мощнее.

Например, иногда фильтр отображения можно применить для удаления из файла захвата не относящегося к делу широковещательного трафика: скрыть широковещательные пакеты ARP на

панели Packet List, если они не связаны с исследуемой проблемой. Но поскольку эти пакеты ARP могут пригодиться позднее, лучше временно отфильтровать их, чем удалять.

Чтобы исключить все пакеты ARP из окна захвата, установите курсор в текстовое поле Filter над панелью Packet List и введите `!arp`, как показано на рисунке 4-16. Для удаления фильтра щёлкните кнопку X, а для сохранения на будущее - кнопку со знаком «плюс» (+).

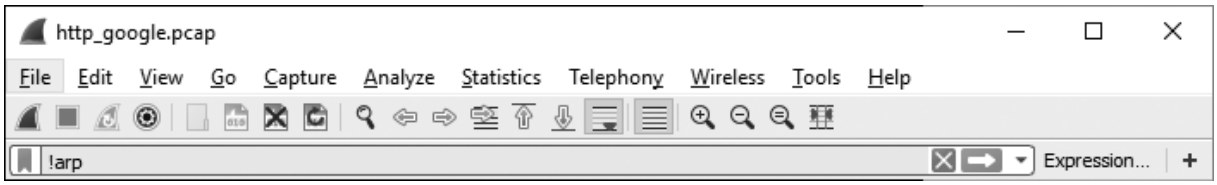


Рисунок 4-16. Создание фильтра отображения в текстовом поле Filter над панелью Packet List

Фильтры отображения можно применять двумя способами. Первый - вводить их напрямую с правильным синтаксисом, как в этом примере. Второй - последовательно строить фильтр в диалоговом окне Display Filter Expression; начинающим этот способ даётся легче. Исследуем оба, начав с более простого.

Диалоговое окно Display Filter Expression

Диалоговое окно Display Filter Expression, показанное на рисунке 4-17, упрощает начинающим пользователям Wireshark создание фильтров захвата и отображения. Чтобы открыть его, щёлкните кнопку Expression на панели Filter.

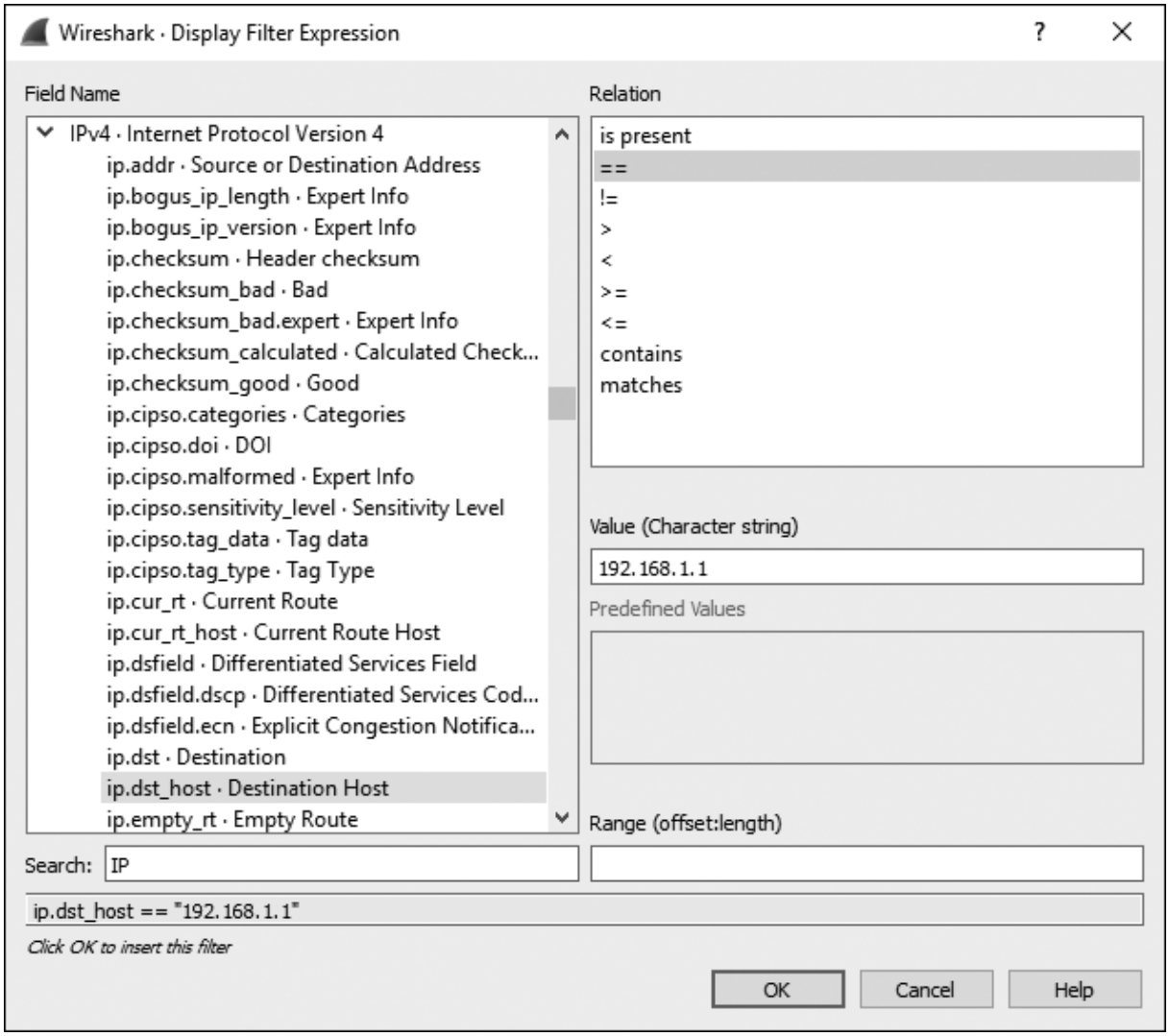


Рисунок 4-17. Диалоговое окно Display Filter Expression упрощает создание фильтров в Wireshark

В левой части окна перечислены все возможные поля протоколов, которые задают все возможные критерии фильтра. Для создания фильтра выполните следующие действия:

1. Чтобы увидеть поля критериев, связанные с протоколом, разверните его, щёлкнув стрелку рядом. Найдя критерий, на котором будет основан фильтр, щёлкните его.
2. Выберите отношение выбранного поля к заданному значению критерия: равно, больше, меньше и так далее.
3. Создайте выражение фильтра, указав значение критерия, которое будет сопоставляться выбранному полю. Значение можно определить самостоятельно либо выбрать из заранее заданных в Wireshark.
4. Полное выражение фильтра появится в нижней части окна. Закончив, щёлкните OK, чтобы вставить его на панель фильтра.

Диалоговое окно Display Filter Expression прекрасно подходит начинающим, однако, освоившись, вы обнаружите, что ручной ввод выражений значительно повышает эффективность. Структура синтаксиса выражений фильтра отображения проста, но чрезвычайно мощна.

Структура синтаксиса выражений фильтра

По мере освоения Wireshark ради экономии времени вам захочется вводить синтаксис фильтров отображения непосредственно в главном окне. К счастью, он подчиняется стандартной схеме, в которой легко ориентироваться. В большинстве случаев схема сосредоточена на протоколе и имеет формат `протокол.функция.подфункция`, как вы уже видели в диалоговом окне Display Filter Expression. Теперь рассмотрим несколько примеров.

Чаще всего фильтр захвата или отображения нужен, чтобы видеть пакеты лишь определённого протокола. Допустим, вы ищете проблему TCP и хотите видеть в файле захвата только трафик TCP. С этим справится простой фильтр `tcp`.

Теперь посмотрим с другой стороны. Представьте, что при поиске проблемы TCP вы часто пользовались служебной программой `ping` и тем самым создали много трафика ICMP. Удалить этот трафик ICMP из видимой части файла захвата можно выражением фильтра `!icmp`.

Операторы сравнения позволяют сопоставлять значения. Например, при поиске неполадок в сетях TCP/IP часто требуется просмотреть все пакеты, в которых упоминается определённый IP-адрес. Оператор равенства (==) позволяет создать фильтр, показывающий все пакеты с IP-адресом `192.168.0.1`:

```
ip.addr==192.168.0.1
```

Теперь предположим, что нужно видеть только пакеты размером не более 128 байт. Для этого можно применить оператор «меньше или равно» (<=):

```
frame.len<=128
```

Операторы сравнения Wireshark приведены в таблице 4-4.

Таблица 4-4. Операторы сравнения в выражениях фильтров Wireshark

Оператор	Описание
==	Равно
!=	Не равно
>	Больше
<	Меньше
>=	Больше или равно
<=	Меньше или равно

Логические операторы позволяют объединить несколько выражений фильтра в одну конструкцию, значительно повышая эффективность фильтров. Например, предположим, что вас интересуют только пакеты к двум IP-адресам. Оператор `or` позволяет создать одно выражение, которое покажет пакеты с любым из этих адресов:

```
ip.addr==192.168.0.1 or ip.addr==192.168.0.2
```

Логические операторы Wireshark перечислены в таблице 4-5.

Таблица 4-5. Логические операторы в выражениях фильтров Wireshark

Оператор	Описание
and	Оба условия должны быть истинны
or	Хотя бы одно условие должно быть истинно
xor	Должно быть истинно одно и только одно условие
not	Ни одно из условий не является истинным

Примеры выражений фильтров отображения

Хотя принципы создания выражений фильтров довольно просты, при построении новых фильтров для различных проблем потребуется использовать несколько специальных ключевых слов и операторов. В таблице 4-6 приведены некоторые фильтры отображения, которыми я пользуюсь чаще всего. Полный перечень находится в справочнике фильтров отображения Wireshark: wireshark.org.

Таблица 4-6. Распространённые фильтры отображения

Фильтр	Описание
`!tcp.port==3389`	Исключить трафик RDP
`tcp.flags.syn==1`	Пакеты TCP с установленным флагом SYN
`tcp.flags.reset==1`	Пакеты TCP с установленным флагом RST
`!arp`	Удалить трафик ARP из отображения
`http`	Весь трафик HTTP
`tcp.port==23 || tcp.port==21`	Трафик Telnet или FTP
`smtp || pop || imap`	Почтовый трафик SMTP, POP или IMAP

Сохранение фильтров

Начав создавать множество фильтров захвата и отображения, вы обнаружите, что некоторыми пользуетесь часто. К счастью, вводить их заново каждый раз не придётся: Wireshark позволяет сохранять фильтры для последующего применения. Чтобы сохранить собственный фильтр захвата, выполните следующие действия:

1. Выберите Capture > Capture Filters, чтобы открыть диалоговое окно Capture Filter.
2. Создайте новый фильтр кнопкой со знаком «плюс» (+) в левом нижнем углу окна.
3. Введите имя фильтра в поле Filter Name.
4. Введите само выражение фильтра в поле Filter String.
5. Щёлкните OK, чтобы сохранить выражение фильтра в списке.

Чтобы сохранить собственный фильтр отображения, выполните следующие действия:

1. Введите фильтр на панели Filter над панелью Packet List в главном окне и щёлкните кнопку с лентой в левой части панели.
2. Выберите Save this Filter: в отдельном диалоговом окне появится список сохранённых фильтров отображения. Там можно дать фильтру имя, а затем щёлкнуть OK, чтобы сохранить его (рисунок 4-18).

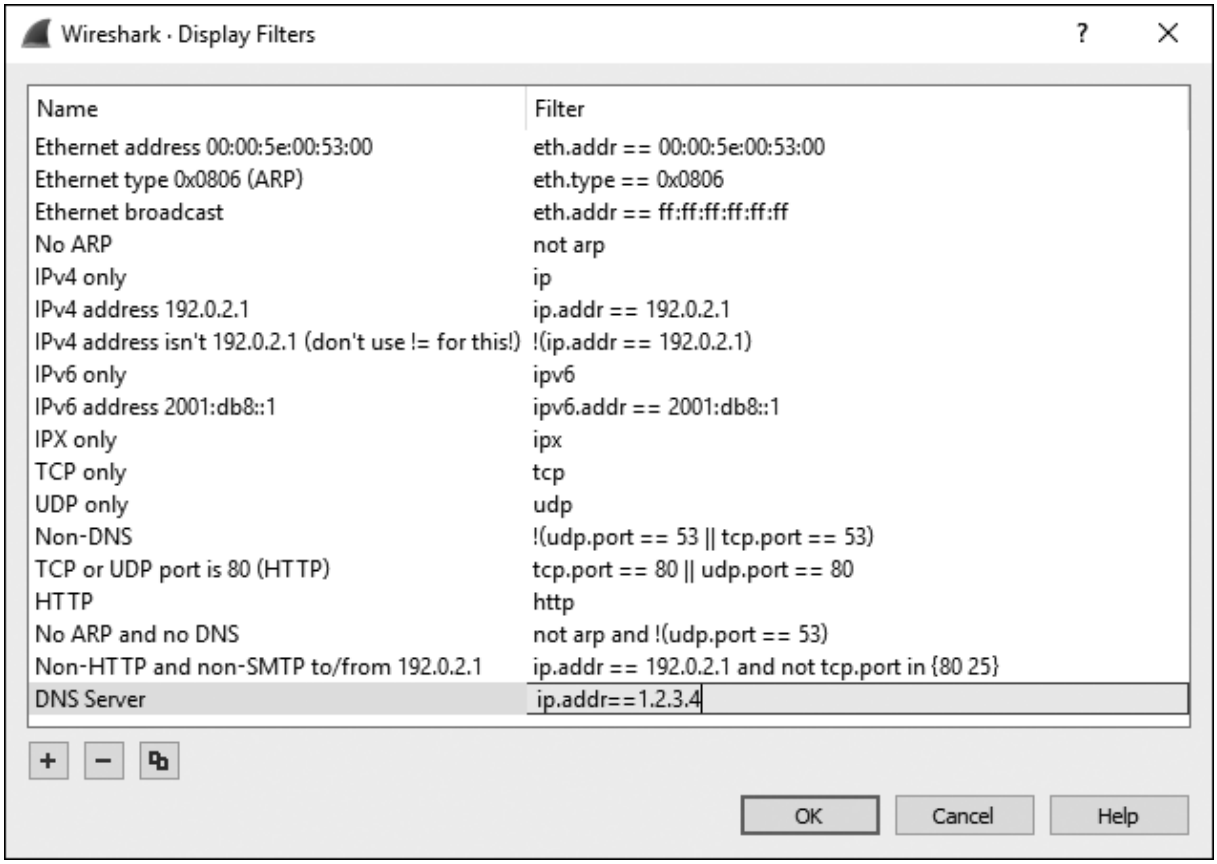


Рисунок 4-18. Фильтры отображения можно сохранять непосредственно с главной панели инструментов

Добавление фильтров отображения на панель инструментов

Если некоторые фильтры приходится часто включать и выключать, один из самых простых способов работать с ними - добавить переключатели фильтров на панель Filter непосредственно над панелью Packet List. Выполните следующие действия:

1. Введите фильтр на панели Filter над панелью Packet List в главном окне и щёлкните кнопку со знаком «плюс» (+) в правой части панели.
2. Под панелью Filter появится новая панель, где в поле Label можно задать имя фильтра (рисунок 4-19). Эта подпись будет представлять фильтр на панели инструментов. Введя значение, щёлкните OK, чтобы создать ярлык выражения на панели инструментов Filter.

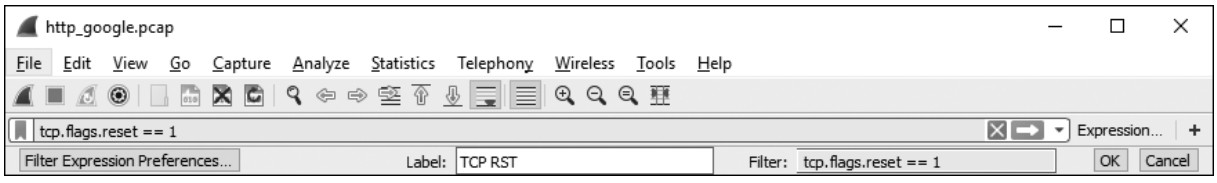


Рисунок 4-19. Добавление ярлыка выражения фильтра на панель инструментов Filter

Как видно на рисунке 4-20, мы создали ярлык фильтра, который быстро показывает все пакеты TCP с установленным флагом RST. Добавления на панель фильтрации сохраняются в профиле конфигурации, который обсуждался в главе 3, и потому служат мощным средством для повышения способности выявлять проблемы в захватах пакетов в различных ситуациях.

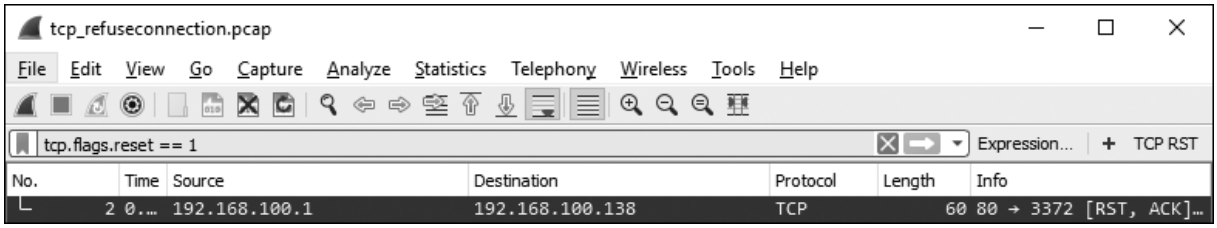


Рисунок 4-20. Фильтрация с помощью ярлыка на панели инструментов

В Wireshark имеется несколько встроенных фильтров, которые служат прекрасными примерами правильного вида фильтра. При создании собственных фильтров стоит пользоваться ими вместе со страницами справки Wireshark. Мы будем применять фильтры в примерах на протяжении всей книги.

Глава 5. Расширенные возможности Wireshark

Освоив основы Wireshark, следующим шагом следует углубиться в возможности анализа и построения графиков. В этой главе мы рассмотрим некоторые из этих мощных средств, включая окна Endpoints и Conversations, тонкости разрешения имён, разбор протоколов, интерпретацию потоков, построение графиков ввода-вывода и многое другое.

Эти функции, отличающие Wireshark как графический инструмент анализа, полезны на разных этапах процесса исследования. Прежде чем двигаться дальше, обязательно хотя бы попробуйте каждую из перечисленных здесь возможностей, потому что мы будем часто возвращаться к ним при разборе практических ситуаций на протяжении остальной части книги.

Конечные точки и сетевые обмены

Чтобы в сети происходил обмен, данные должны передаваться как минимум между двумя устройствами. Каждое устройство, которое отправляет или получает данные в сети, представляет собой то, что Wireshark называет конечной точкой. Взаимодействие двух конечных точек называется обменом. Wireshark описывает конечные точки и обмены по характеристикам взаимодействия, в частности по адресам, используемым в различных протоколах.

Конечные точки определяются несколькими адресами, назначенными на разных уровнях модели OSI. Например, на канальном уровне конечная точка имеет MAC-адрес - уникальный адрес, встроенный в устройство (хотя его можно изменить, и тогда требование уникальности потенциально нарушится). На сетевом уровне конечная точка, напротив, имеет IP-адрес, который может измениться в любой момент. В нескольких следующих главах мы обсудим, как используются адреса этих типов.

На рисунке 5-1 показаны два примера применения адресов для распознавания конечных точек в обменах. Обмен A состоит из двух конечных точек, взаимодействующих на канальном уровне MAC. Конечная точка A имеет MAC-адрес `00:ff:ac:ce:0b:de`, а конечная точка B - `00:ff:ac:e0:dc:0f`. Обмен B определяется двумя устройствами, взаимодействующими на сетевом уровне IP. IP-адрес конечной точки A - `192.168.1.25`, а адрес конечной точки B - `192.168.1.30`.

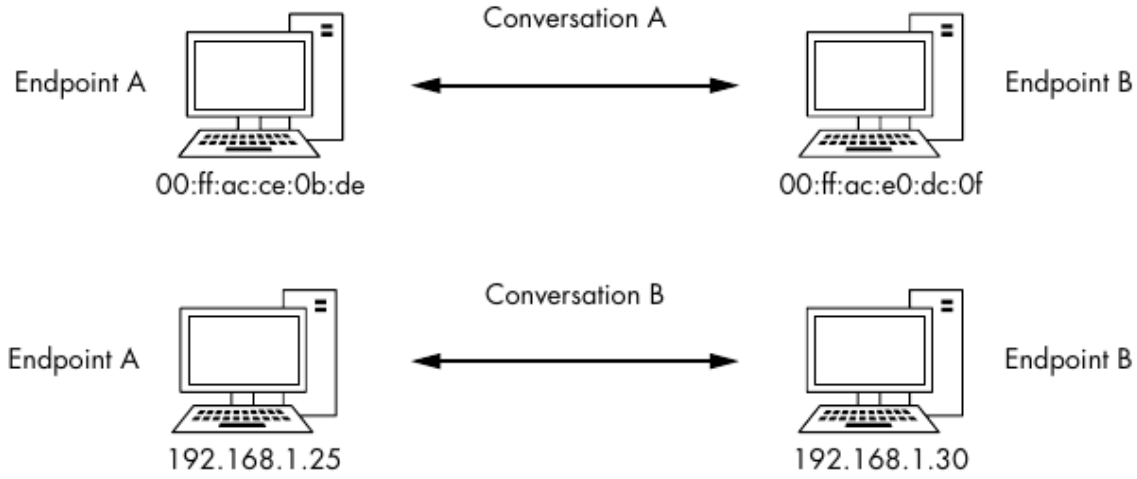


Рисунок 5-1. Конечные точки и обмены в сети

Посмотрим, как Wireshark предоставляет сведения о сетевом обмене в разрезе отдельных конечных точек и обменов.

Просмотр статистики конечных точек

При анализе трафика иногда удаётся определить, что проблема связана с конкретной конечной точкой сети. Например, откройте файл захвата `lotsofweb.pcapng`, а затем окно Endpoints в Wireshark (Statistics > Endpoints). В этом окне для каждой конечной точки показывается несколько полезных статистических показателей, как на рисунке 5-2, в том числе адрес, количество пакетов и число переданных и полученных байтов.

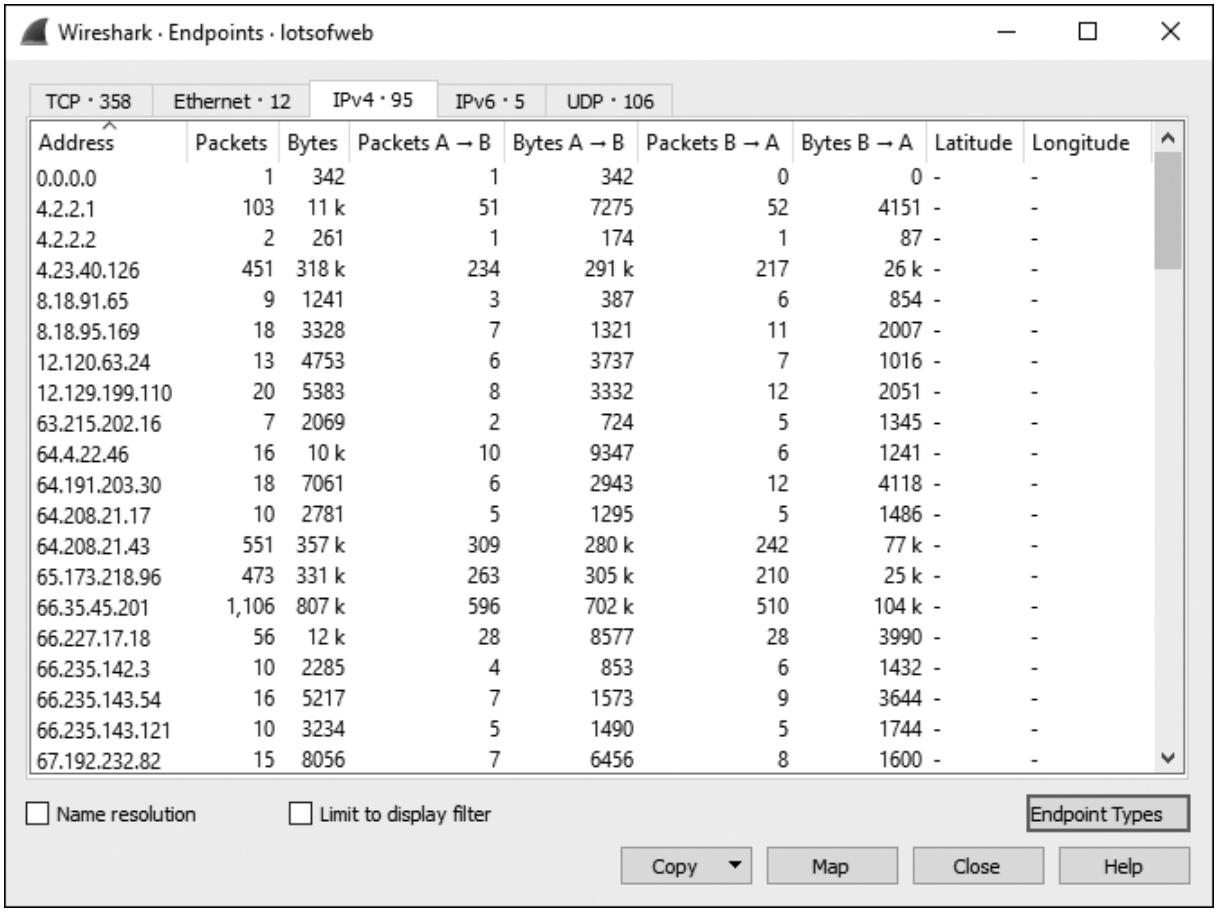


Рисунок 5-2. Окно Endpoints позволяет просматривать каждую конечную точку файла захвата

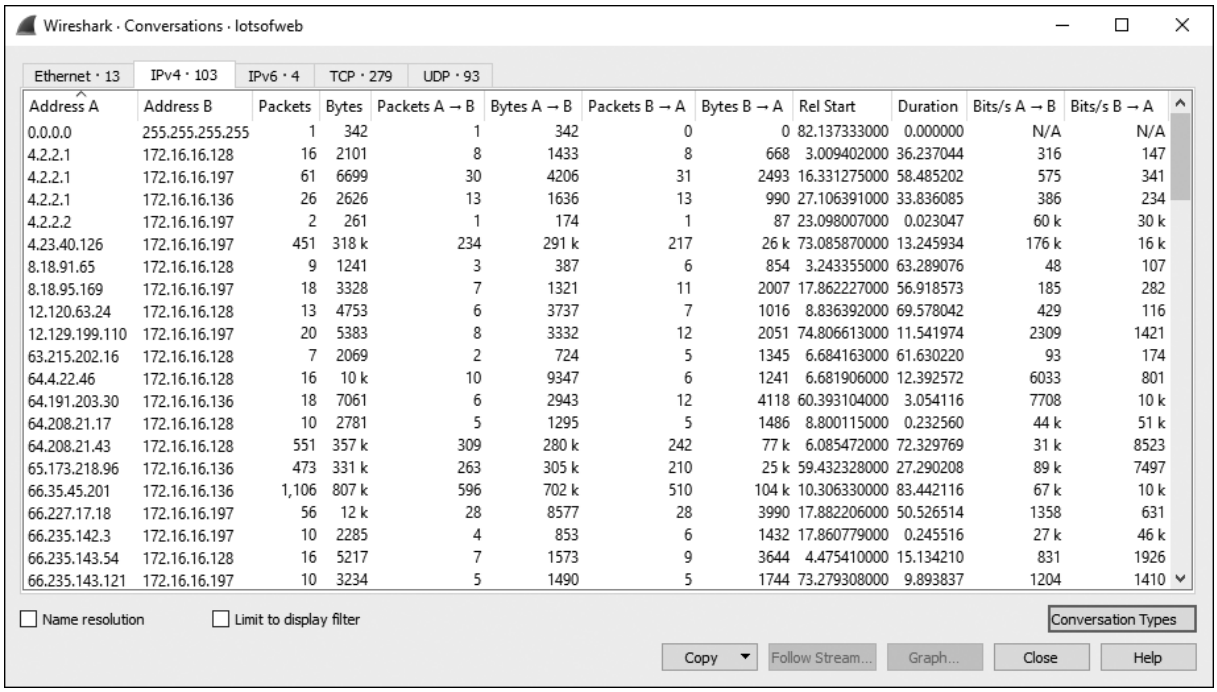
Вкладки в верхней части окна - TCP, Ethernet, IPv4, IPv6 и UDP - показывают количество конечных точек с разбивкой по протоколам. Чтобы отобразить конечные точки только определённого протокола, щёлкните соответствующую вкладку. Дополнительные вкладки фильтрации по протоколам можно добавить, щёлкнув поле Endpoint Types в правом нижнем углу экрана и выбрав нужный протокол. Чтобы просматривать адреса конечных точек с разрешением имён (см. раздел «Разрешение имён» на странице 84), установите флажок Name resolution. Если вы работаете с большим захватом и хотите отфильтровать показанные конечные точки, примените фильтр отображения в главном окне Wireshark и выберите Limit to display filter в окне Endpoints. Тогда окно покажет только конечные точки, совпадающие с фильтром отображения.

Ещё одна удобная функция окна Endpoints - фильтрация определённых пакетов для показа на панели Packet List. Так можно быстро перейти к пакетам отдельной конечной точки. Щёлкните конечную точку правой кнопкой мыши, чтобы выбрать доступные параметры фильтрации. В появившемся диалоговом окне можно показать или исключить пакеты, связанные с выбранной строкой. Параметр Colorize в этом окне позволяет непосредственно перенести адрес конечной точки в правило раскрашивания (правила раскрашивания обсуждались в главе 4). Таким способом

можно быстро выделить пакеты определённой конечной точки, чтобы во время анализа они сразу бросались в глаза.

Просмотр сетевых обменов

Не закрывая `lotsofweb.pcapng`, откройте окно Conversations в Wireshark командой Statistics > Conversations (рисунок 5-3), чтобы показать все обмены в файле захвата. Окно Conversations похоже на Endpoints, но в нём для представления обмена в каждой строке показаны два адреса, а также пакеты и байты, переданные каждому устройству и полученные от него. Столбец Address A содержит исходную конечную точку, а Address B - конечную.



Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
0.0.0.0	255.255.255.255	1	342	1	342	0	0	82.137333000	0.000000	N/A	N/A
4.2.2.1	172.16.16.128	16	2101	8	1433	8	668	3.009402000	36.237044	316	147
4.2.2.1	172.16.16.197	61	6699	30	4206	31	2493	16.331275000	58.485202	575	341
4.2.2.1	172.16.16.136	26	2626	13	1636	13	990	27.106391000	33.836085	386	234
4.2.2.2	172.16.16.197	2	261	1	174	1	87	23.098007000	0.023047	60 k	30 k
4.23.40.126	172.16.16.197	451	318 k	234	291 k	217	26 k	73.085870000	13.245934	176 k	16 k
8.18.91.65	172.16.16.128	9	1241	3	387	6	854	3.243355000	63.289076	48	107
8.18.95.169	172.16.16.197	18	3328	7	1321	11	2007	17.862227000	56.918573	185	282
12.120.63.24	172.16.16.128	13	4753	6	3737	7	1016	8.836392000	69.578042	429	116
12.129.199.110	172.16.16.197	20	5383	8	3332	12	2051	74.806613000	11.541974	2309	1421
63.215.202.16	172.16.16.128	7	2069	2	724	5	1345	6.684163000	61.630220	93	174
64.4.22.46	172.16.16.128	16	10 k	10	9347	6	1241	6.681906000	12.392572	6033	801
64.191.203.30	172.16.16.136	18	7061	6	2943	12	4118	60.393104000	3.054116	7708	10 k
64.208.21.17	172.16.16.128	10	2781	5	1295	5	1486	8.800115000	0.232560	44 k	51 k
64.208.21.43	172.16.16.128	551	357 k	309	280 k	242	77 k	6.085472000	72.329769	31 k	8523
65.173.218.96	172.16.16.136	473	331 k	263	305 k	210	25 k	59.432328000	27.290208	89 k	7497
66.35.45.201	172.16.16.136	1,106	807 k	596	702 k	510	104 k	10.306330000	83.442116	67 k	10 k
66.227.17.18	172.16.16.197	56	12 k	28	8577	28	3990	17.882206000	50.526514	1358	631
66.235.142.3	172.16.16.197	10	2285	4	853	6	1432	17.860779000	0.245516	27 k	46 k
66.235.143.54	172.16.16.128	16	5217	7	1573	9	3644	4.475410000	15.134210	831	1926
66.235.143.121	172.16.16.197	10	3234	5	1490	5	1744	73.279308000	9.893837	1204	1410

Рисунок 5-3. Окно Conversations позволяет подробно исследовать каждый обмен в файле захвата

Окно Conversations организовано по протоколам. Чтобы видеть обмены лишь определённого протокола, щёлкните одну из вкладок в верхней части окна, как в Endpoints, или добавьте другие виды протоколов кнопкой Conversation Types в правом нижнем углу. Подобно окну Endpoints, здесь можно использовать разрешение имён, ограничивать видимые обмены фильтром отображения и щёлкать определённый обмен правой кнопкой мыши, чтобы создавать фильтры на основе конкретных обменов. Такие фильтры полезны для подробного изучения интересных последовательностей взаимодействия.

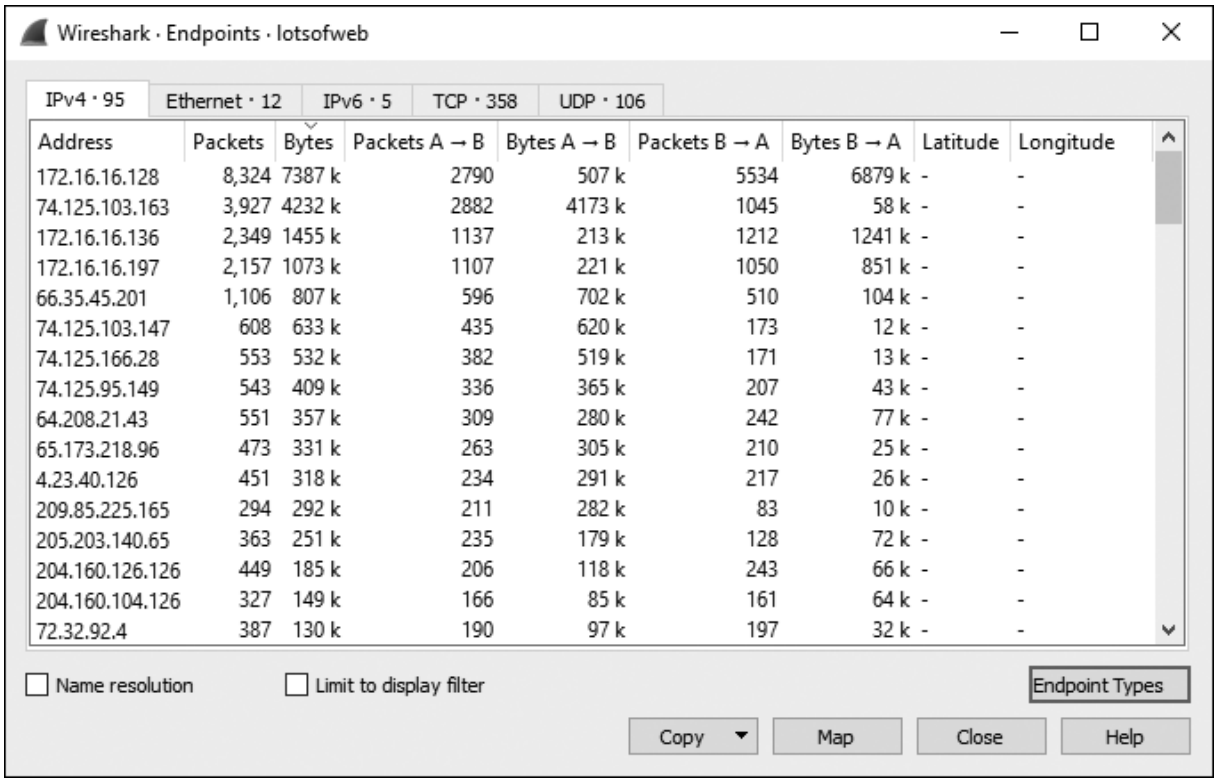
Выявление самых активных участников с помощью Endpoints и Conversations

Окна Endpoints и Conversations помогают при поиске сетевых неполадок, особенно когда требуется найти источник значительного объёма трафика в сети.

В качестве примера снова рассмотрим `lotsofweb.pcapng`. Как следует из имени, этот файл захвата содержит трафик HTTP, созданный несколькими клиентами, которые просматривали Интернет. На рисунке 5-4 показан список конечных точек этого файла, отсортированный по числу

байтов.

Обратите внимание: конечная точка, создавшая наибольший объём трафика в байтах, имеет адрес `172.16.16.128`. Это внутренний сетевой адрес (в главе 7 мы расскажем, как это определяется), а поскольку данное устройство отвечает за самый большой объём обмена в захвате, оно получает обозначение «самый активный участник».



Address	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Latitude	Longitude
172.16.16.128	8,324	7387 k	2790	507 k	5534	6879 k	-	-
74.125.103.163	3,927	4232 k	2882	4173 k	1045	58 k	-	-
172.16.16.136	2,349	1455 k	1137	213 k	1212	1241 k	-	-
172.16.16.197	2,157	1073 k	1107	221 k	1050	851 k	-	-
66.35.45.201	1,106	807 k	596	702 k	510	104 k	-	-
74.125.103.147	608	633 k	435	620 k	173	12 k	-	-
74.125.166.28	553	532 k	382	519 k	171	13 k	-	-
74.125.95.149	543	409 k	336	365 k	207	43 k	-	-
64.208.21.43	551	357 k	309	280 k	242	77 k	-	-
65.173.218.96	473	331 k	263	305 k	210	25 k	-	-
4.23.40.126	451	318 k	234	291 k	217	26 k	-	-
209.85.225.165	294	292 k	211	282 k	83	10 k	-	-
205.203.140.65	363	251 k	235	179 k	128	72 k	-	-
204.160.126.126	449	185 k	206	118 k	243	66 k	-	-
204.160.104.126	327	149 k	166	85 k	161	64 k	-	-
72.32.92.4	387	130 k	190	97 k	197	32 k	-	-

Рисунок 5-4. Окно Endpoints показывает, какие узлы обмениваются данными активнее всего

Второй по объёму трафика адрес - `74.125.103.163`, внешний адрес в Интернете. Встретив неизвестный внешний адрес, можно найти его зарегистрированного владельца в реестре WHOIS. В данном случае Американский реестр интернет-номеров (American Registry for Internet Numbers, ARIN) на странице whois.arin.net сообщает, что этот IP-адрес принадлежит Google, как видно на рисунке 5-5.

Network	
Net Range	74.125.0.0 - 74.125.255.255
CIDR	74.125.0.0/16
Name	GOOGLE
Handle	NET-74-125-0-0-1
Parent	NET74 (NET-74-0-0-0-0)
Net Type	Direct Allocation
Origin AS	
Organization	Google Inc. (GOGL)
Registration Date	2007-03-13
Last Updated	2012-02-24
Comments	
RESTful Link	https://whois.arin.net/rest/net/NET-74-125-0-0-1
See Also	Related organization's POC records.
See Also	Related delegations.

Рисунок 5-5. Результаты WHOIS для `74.125.103.163` *указывают на IP-адрес Google*

Определение владельца IP-адреса с помощью WHOIS

Назначением IP-адресов управляют разные организации в зависимости от географического региона. ARIN отвечает за назначение IP-адресов в США и некоторых соседних территориях, AfriNIC управляет адресами в Африке, RIPE - в Европе, а APNIC - в Азиатско-Тихоокеанском регионе. Обычно запрос WHOIS для IP-адреса выполняют на сайте реестра, отвечающего за этот адрес. Конечно, по одному взгляду на адрес вряд ли можно определить нужный региональный реестр. Такие сайты, как Robtex (robtex.com), выполняют сложную работу за вас: обращаются к правильному реестру и предоставляют результаты. Впрочем, если сначала запросить не тот реестр, он обычно укажет правильный.

Зная это, можно предположить, что либо `172.16.16.128` и `74.125.103.163` по отдельности много взаимодействуют с несколькими другими устройствами, либо обе конечные точки обмениваются данными друг с другом. На деле, как часто бывает с парой самых активных конечных точек, они взаимодействуют друг с другом. Чтобы подтвердить это, откройте окно Conversations, выберите вкладку IPv4 и отсортируйте список по байтам. Эти две конечные точки должны образовать обмен с наибольшим числом переданных байтов. Характер передачи указывает на крупную загрузку, поскольку количество байтов, отправленных внешним адресом A (`74.125.103.163`), намного больше количества байтов от внутреннего адреса B (`172.16.16.128`), как показано на рисунке 5-6.

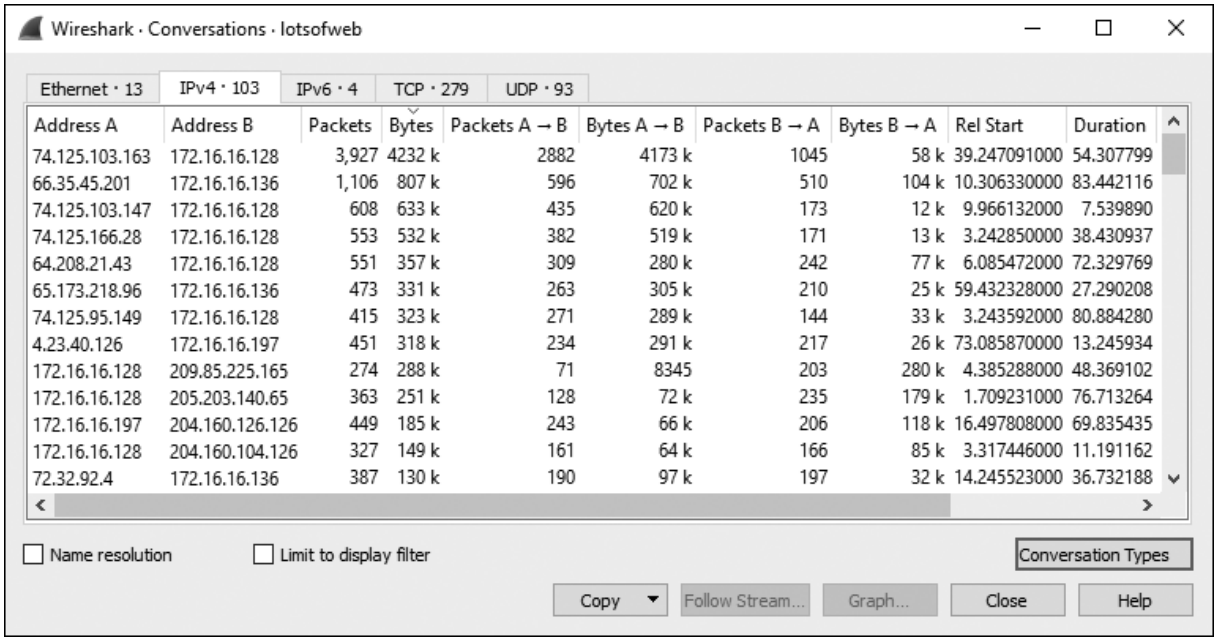


Рисунок 5-6. Окно Conversations подтверждает, что два самых активных участника взаимодействуют друг с другом

Этот обмен можно исследовать, применив следующий фильтр отображения:

```
ip.addr == 74.125.103.163 && ip.addr == 172.16.16.128
```

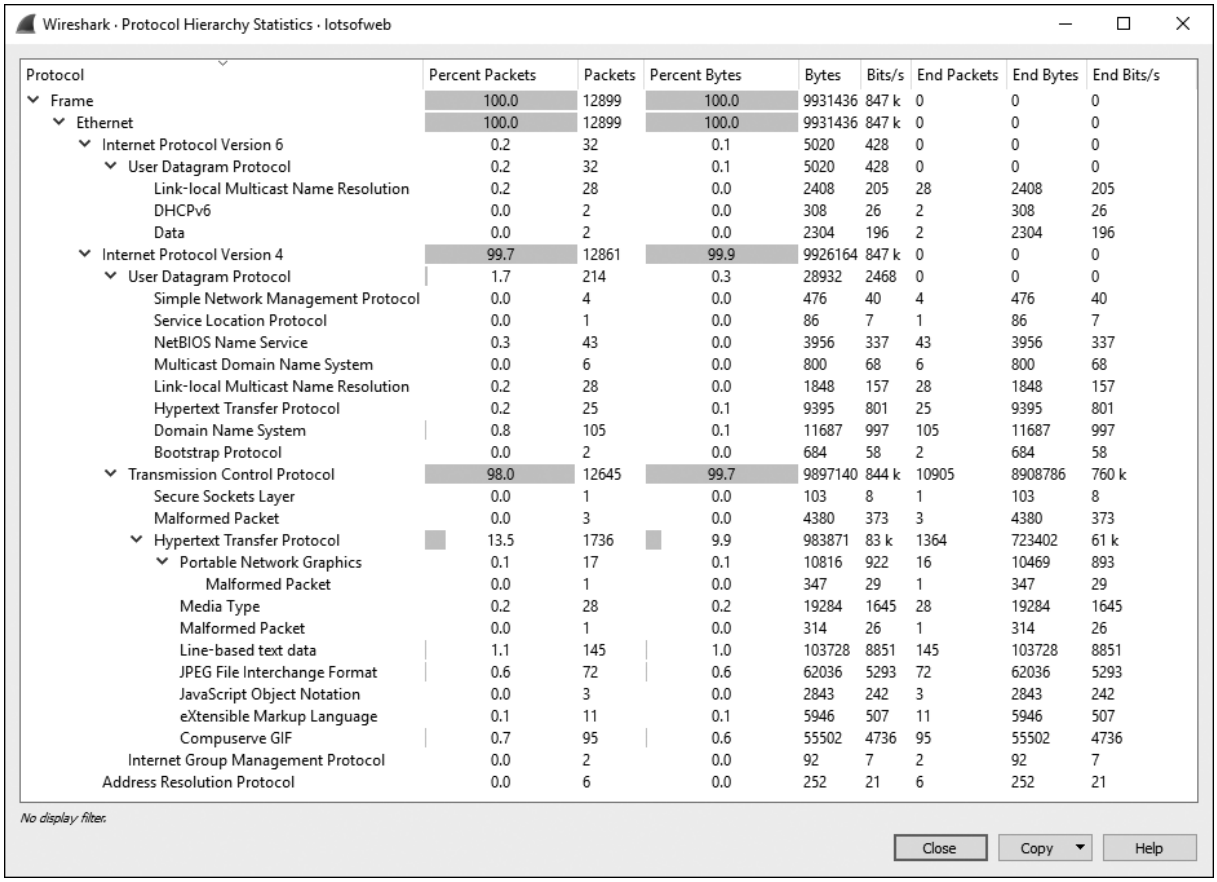
Если прокручивать список пакетов, в столбце Info окна Packet List обнаружится несколько запросов DNS к домену `youtube.com`. Это согласуется с выводом, что `74.125.103.163` принадлежит Google, поскольку Google владеет YouTube.

В остальных главах книги вы увидите применение окон Endpoints и Conversations в практических ситуациях.

Статистика иерархии протоколов

При работе с незнакомыми файлами захвата иногда требуется определить распределение трафика по протоколам. Иными словами, какую долю захвата составляют TCP, IP, DHCP и так далее? Вместо ручного подсчёта пакетов и сложения результатов эти сведения можно получить в окне Protocol Hierarchy Statistics программы Wireshark.

Например, не закрывая файл `lotsofweb.pcapng` и удалив все ранее применённые фильтры, выберите Statistics > Protocol Hierarchy и откройте окно Protocol Hierarchy Statistics, показанное на рисунке 5-7.



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	12899	100.0	9931436	847 k	0	0	0
Ethernet	100.0	12899	100.0	9931436	847 k	0	0	0
Internet Protocol Version 6	0.2	32	0.1	5020	428	0	0	0
User Datagram Protocol	0.2	32	0.1	5020	428	0	0	0
Link-local Multicast Name Resolution	0.2	28	0.0	2408	205	28	2408	205
DHCPv6	0.0	2	0.0	308	26	2	308	26
Data	0.0	2	0.0	2304	196	2	2304	196
Internet Protocol Version 4	99.7	12861	99.9	9926164	847 k	0	0	0
User Datagram Protocol	1.7	214	0.3	28932	2468	0	0	0
Simple Network Management Protocol	0.0	4	0.0	476	40	4	476	40
Service Location Protocol	0.0	1	0.0	86	7	1	86	7
NetBIOS Name Service	0.3	43	0.0	3956	337	43	3956	337
Multicast Domain Name System	0.0	6	0.0	800	68	6	800	68
Link-local Multicast Name Resolution	0.2	28	0.0	1848	157	28	1848	157
Hypertext Transfer Protocol	0.2	25	0.1	9395	801	25	9395	801
Domain Name System	0.8	105	0.1	11687	997	105	11687	997
Bootstrap Protocol	0.0	2	0.0	684	58	2	684	58
Transmission Control Protocol	98.0	12645	99.7	9897140	844 k	10905	8908786	760 k
Secure Sockets Layer	0.0	1	0.0	103	8	1	103	8
Malformed Packet	0.0	3	0.0	4380	373	3	4380	373
Hypertext Transfer Protocol	13.5	1736	9.9	983871	83 k	1364	723402	61 k
Portable Network Graphics	0.1	17	0.1	10816	922	16	10469	893
Malformed Packet	0.0	1	0.0	347	29	1	347	29
Media Type	0.2	28	0.2	19284	1645	28	19284	1645
Malformed Packet	0.0	1	0.0	314	26	1	314	26
Line-based text data	1.1	145	1.0	103728	8851	145	103728	8851
JPEG File Interchange Format	0.6	72	0.6	62036	5293	72	62036	5293
JavaScript Object Notation	0.0	3	0.0	2843	242	3	2843	242
eXtensible Markup Language	0.1	11	0.1	5946	507	11	5946	507
Compuserve GIF	0.7	95	0.6	55502	4736	95	55502	4736
Internet Group Management Protocol	0.0	2	0.0	92	7	2	92	7
Address Resolution Protocol	0.0	6	0.0	252	21	6	252	21

Рисунок 5-7. Окно Protocol Hierarchy Statistics показывает распределение трафика по протоколам

Окно Protocol Hierarchy Statistics даёт моментальный снимок видов активности в сети. На рисунке 5-7 трафик Ethernet составляет 100 процентов, IPv4 - 99,7 процента, TCP - 98 процентов, а HTTP от просмотра веб-страниц - 13,5 процента. Эти сведения прекрасно подходят для определения базового уровня сети, особенно когда вы уже мысленно представляете её обычный трафик. Например, если вам известно, что в норме 10 процентов сетевого трафика составляет ARP, а в недавнем захвате его доля равна 50 процентам, возможно, что-то не в порядке. Иногда интерес представляет само наличие протокола. Если ни одно устройство не настроено на применение протокола связующего дерева (Spanning Tree Protocol, STP), его появление в иерархии протоколов может означать, что какое-то устройство настроено неверно.

Со временем вы обнаружите, что окно Protocol Hierarchy Statistics позволяет составлять профиль пользователей и устройств сети, просто глядя на распределение используемых протоколов. Например, повышенная доля HTTP говорит о частом просмотре веб-страниц. Иногда конкретные устройства сети можно распознавать, исследуя трафик сегмента, принадлежащего определённому подразделению. Например, отдел информационных технологий может чаще применять административные протоколы вроде ICMP и SNMP, служба поддержки клиентов может создавать большой объём трафика SMTP, то есть электронной почты, а назойливый стажёр в углу - наводнять сеть трафиком World of Warcraft!

Разрешение имён

Сетевые данные передаются между конечными точками с помощью различных буквенно-цифровых систем адресации, значения которых часто слишком длинны или сложны для запоминания, например MAC-адрес `00:16:ce:6e:8b:24`, IPv4-адрес `192.168.47.122` или IPv6-адрес `2001:db8:a0b:12f0::1`. Разрешение имён, также называемое поиском имени, преобразует один идентифицирующий адрес в другой главным образом для того, чтобы адрес было проще запомнить. Например, `google.com` запомнить намного легче, чем `216.58.217.238`. Связывая понятные имена с загадочными адресами, мы упрощаем их запоминание и распознавание.

Включение разрешения имён

Для упрощения анализа Wireshark умеет применять разрешение имён при отображении данных пакетов. Чтобы его включить, выберите Edit > Preferences > Name Resolution. Это окно показано на рисунке 5-8. Ниже перечислены основные доступные варианты разрешения имён.

Resolve MAC addresses. С помощью протокола ARP пытается преобразовать MAC-адреса уровня 2, например `00:09:5b:01:02:03`, в адреса уровня 3, например `10.100.12.1`. Если такое преобразование не удаётся, Wireshark пытается выполнить его по файлу `ethers` в каталоге программы. Последнее средство Wireshark - преобразовать первые три байта MAC-адреса в указанное IEEE имя изготовителя устройства, например `Netgear_01:02:03`.

Resolve transport names. Пытается преобразовать номер порта в связанное с ним имя, например показать порт 80 как `http`. Это удобно, когда встречается необычный порт и вы не знаете, какая служба обычно с ним связана.

Resolve network (IP) addresses. Пытается преобразовать адрес уровня 3, например `192.168.1.50`, в удобочитаемое имя DNS, например `MarketingPC1.domain.com`. Это помогает определить назначение или владельца системы при условии, что у неё понятное имя.

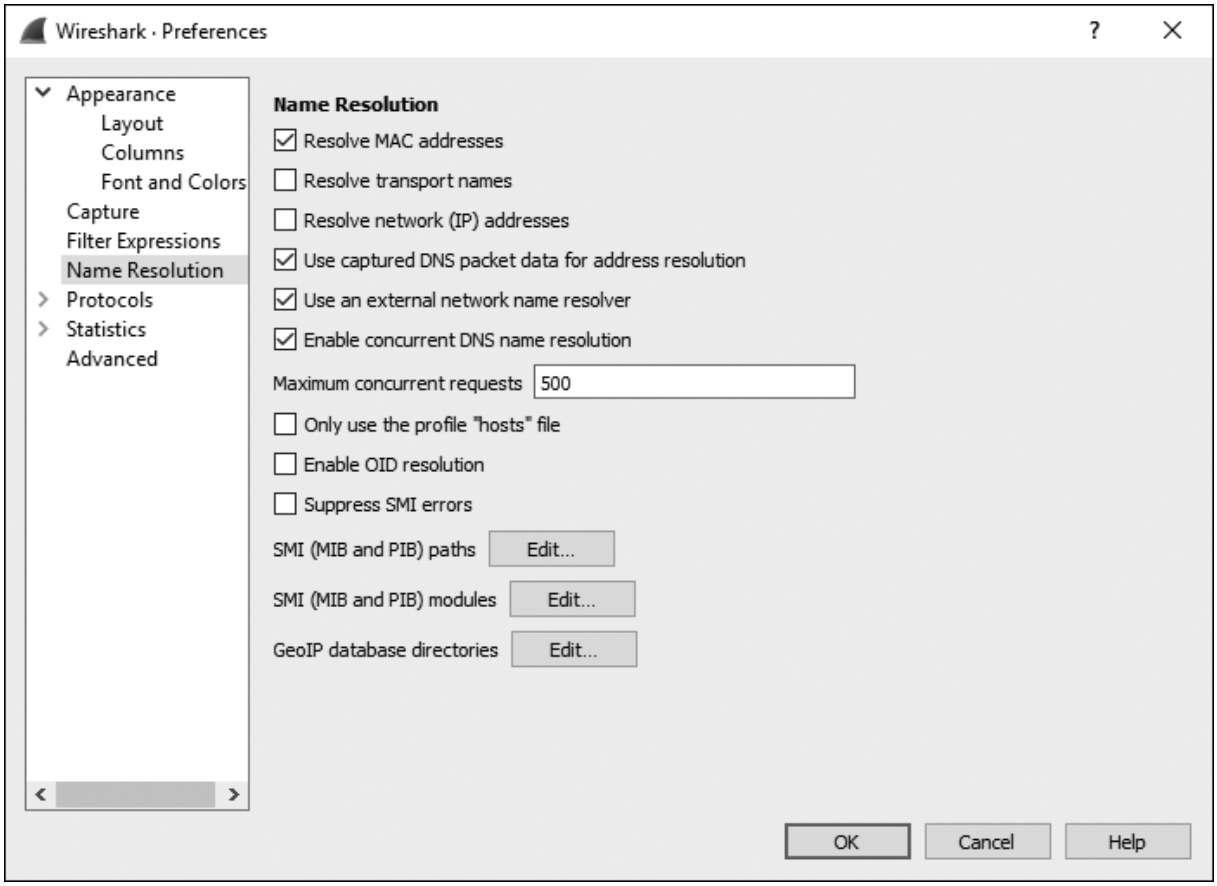


Рисунок 5-8. Включение разрешения имён в диалоговом окне Preferences. Среди первых трёх флажков, относящихся к видам разрешения имён, установлен только Resolve MAC addresses

Диалоговое окно настроек Name Resolution на рисунке 5-8 содержит ещё несколько полезных параметров.

Use captured DNS packet data for address resolution. Разбирает данные DNS из захваченных пакетов DNS, чтобы преобразовывать IP-адреса в имена DNS.

Use an external network name resolver. Позволяет Wireshark отправлять запросы серверу DNS, которым пользуется машина анализа, для преобразования IP-адресов в имена DNS. Это полезно, если вы хотите применять разрешение имён DNS, но в анализируемом захвате нет соответствующих пакетов DNS.

Maximum concurrent requests. Ограничивает число одновременно ожидающих ответа запросов DNS. Используйте этот параметр, если захват породит множество запросов DNS и вы опасаетесь занять слишком большую долю пропускной способности сети или сервера DNS.

Only use the profile “hosts” file. Ограничивает разрешение DNS файлом `hosts`, связанным с активным профилем Wireshark. Далее в этом разделе я расскажу, как пользоваться данным файлом.

Изменения, внесённые на экране Preferences, сохраняются после закрытия и повторного открытия Wireshark. Чтобы менять разрешение имён на ходу без постоянного сохранения, включайте и отключайте параметры через View > Name Resolution в главном раскрывающемся меню. Разрешение имён можно включать и выключать для физических, транспортных и сетевых адресов.

Различные средства разрешения имён способны сделать файлы захвата удобнее для чтения и в некоторых ситуациях значительно сэкономить время. Например, разрешение имён DNS помогает быстро выяснить имя компьютера, который вы пытаетесь определить как источник определённого пакета.

Возможные недостатки разрешения имён

С учётом преимуществ применение разрешения имён может показаться очевидным выбором, однако у него есть возможные недостатки. Во-первых, разрешение сетевых имён может завершиться неудачей, если нет доступного сервера DNS, способного предоставить имя, связанное с IP-адресом. Сведения разрешения имён не сохраняются в файле захвата, поэтому процесс приходится выполнять при каждом открытии файла. Если захватить пакеты в одной сети, а затем открыть захват в другой, система может не получить доступ к серверам DNS исходной сети, и разрешение имён не сработает.

Кроме того, разрешение имён требует дополнительных вычислительных ресурсов. При работе с очень большим файлом захвата от него, возможно, стоит отказаться ради экономии ресурсов системы. Если крупный захват загружается с трудом или Wireshark аварийно завершается, может помочь отключение разрешения имён.

Ещё одна проблема состоит в том, что зависимость разрешения сетевых имён от DNS способна породить нежелательные пакеты, которые замутнят файл захвата, когда трафик отправится серверам DNS для разрешения адресов. Более того, если анализируемый файл содержит вредоносные IP-адреса, попытка разрешить их может создать запросы к инфраструктуре под управлением злоумышленника. Тот способен понять, что вам известно о его действиях, и, возможно, выбрать вас новой целью. Чтобы уменьшить риск замутить файл пакетов или неосознанно связаться со злоумышленником, отключите Use an external network name resolver в диалоговом окне настроек Name Resolution.

Использование собственного файла hosts

Отслеживать трафик множества узлов в больших файлах захвата бывает утомительно, особенно когда внешнее разрешение узлов недоступно. Один из способов облегчить задачу - вручную присвоить системам метки по их IP-адресам с помощью файла Wireshark `hosts`, то есть текстового файла со списком соответствий IP-адресов и имён. Файл `hosts` позволяет назначить адресам в Wireshark имена для быстрого обращения. Эти имена будут показаны на панели Packet List.

Чтобы использовать файл `hosts`, выполните следующие действия:

1. Выберите Edit > Preferences > Name Resolution и установите флажок Only use the profile “hosts” file.
2. Создайте новый файл в Блокноте Windows или похожем текстовом редакторе. Каждая строка должна содержать одну запись с IP-адресом и именем, в которое он разрешается, как показано на рисунке 5-9. Выбранное справа имя будет появляться в окне списка пакетов всякий раз, когда Wireshark встречает указанный слева IP-адрес.

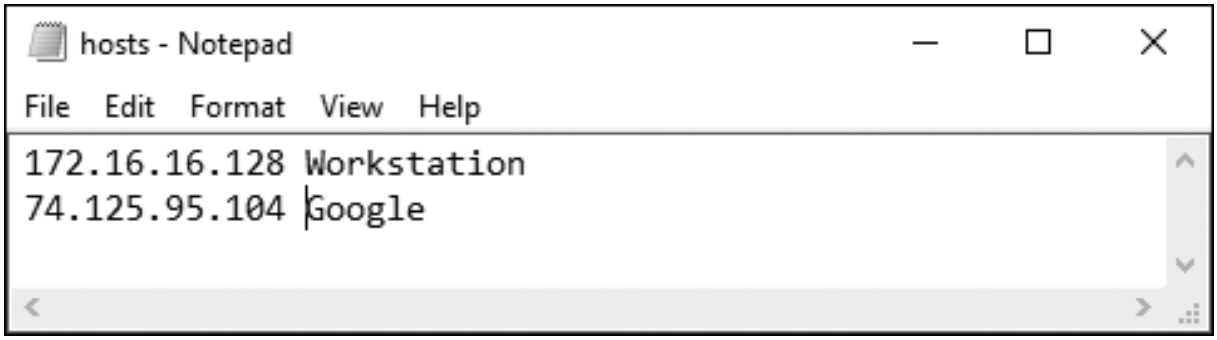


Рисунок 5-9. Создание файла `hosts` *Wireshark*

3. Сохраните файл как простой текст под именем `hosts` в соответствующем каталоге из списка ниже. Убедитесь, что у файла нет расширения!

• Windows: `<USERPROFILE>\Application Data\Wireshark\hosts`
• OS X: `/Users/<username>/.wireshark/hosts`
• Linux: `/home/<username>/.wireshark/hosts`

Теперь откройте захват: все IP-адреса из файла `hosts` должны разрешиться в указанные имена, как показано на рисунке 5-10. Вместо IP-адресов в столбцах Source и Destination окна списка пакетов появятся более содержательные имена.

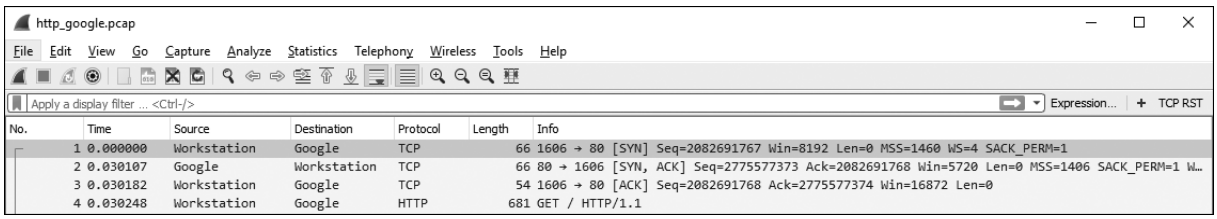


Рисунок 5-10. Разрешение имён из файла `hosts` *в Wireshark*

Такое применение файлов `hosts` способно значительно улучшить распознавание определённых узлов во время анализа. Работая в группе аналитиков, подумайте о том, чтобы предоставить сетевым специалистам общий файл `hosts` с известными ресурсами. Он поможет группе быстро распознавать системы со статическими адресами, например серверы и маршрутизаторы.

> **Примечание.** Если файл `hosts` не работает, убедитесь, что случайно не добавили к его имени расширение. Файл должен называться просто `hosts`.

Разрешение имён по запросу вручную

Wireshark также способен принудительно разрешать имена временно, по запросу. Для этого щёлкните пакет правой кнопкой мыши на панели Packet List и выберите Edit Resolved Name. Во всплывающем окне можно указать имя адреса, подобное метке. После закрытия файла захвата это разрешение исчезнет, поэтому данный способ позволяет быстро пометить адрес без постоянных изменений, которые позднее пришлось бы отменять. Я часто пользуюсь этим приёмом, потому что он немного проще ручного редактирования файла `hosts` для каждого просматриваемого захвата.

Разбор протоколов

Одна из главных сильных сторон Wireshark - поддержка анализа более тысячи протоколов. Такая возможность существует благодаря открытому исходному коду, предоставляющему инфраструктуру для создания анализаторов протоколов. Они позволяют Wireshark распознавать и декодировать протокол в различные поля для отображения в пользовательском интерфейсе. При интерпретации каждого пакета Wireshark согласованно применяет несколько анализаторов. Например, анализатор протокола ICMP позволяет Wireshark распознать, что пакет IP содержит данные ICMP, извлечь тип и код ICMP и отформатировать эти поля для показа в столбце Info панели Packet List.

Анализатор можно представить как переводчика между необработанными данными и программой Wireshark. Чтобы Wireshark поддерживал протокол, для него должен существовать анализатор (либо вы можете написать собственный).

Изменение анализатора

Wireshark применяет анализаторы для обнаружения отдельных протоколов и выбора способа отображения сетевых сведений. К сожалению, при выборе анализатора для пакета Wireshark не всегда принимает правильное решение. Особенно часто это происходит, когда сетевой протокол использует нестандартную конфигурацию, например нестандартный порт (который сетевые администраторы часто назначают в качестве меры безопасности, а сотрудники - пытаясь обойти ограничения доступа).

Если Wireshark неверно применил анализатор, его выбор можно переопределить. Например, откройте файл трассировки `wrongdissector.pcapng`. Он содержит множество обменов SSL между двумя компьютерами. SSL - протокол уровня защищённых сокетов (Secure Socket Layer), используемый для зашифрованного обмена между узлами. В обычных обстоятельствах просмотр трафика SSL в Wireshark мало что даёт из-за его зашифрованной природы. Однако здесь определённо что-то не так. Если просмотреть содержимое нескольких пакетов, щёлкая их и исследуя панель Packet Bytes, обнаружится открытый текст. Более того, в пакете 4 упоминается серверное приложение FTP FileZilla. Следующие несколько пакетов ясно показывают запрос и ответ и для имени пользователя, и для пароля.

Если бы это действительно был трафик SSL, прочитать содержащиеся в пакетах данные было бы невозможно, а уж все имена пользователей и пароли точно не передавались бы открытым текстом, как на рисунке 5-11. Судя по показанным сведениям, можно уверенно предположить, что это, вероятно, трафик FTP, а не SSL. Скорее всего, Wireshark интерпретирует его как SSL из-за использования порта 443, что видно в столбце Info, а порт 443 является стандартным для HTTPS (HTTP поверх SSL).

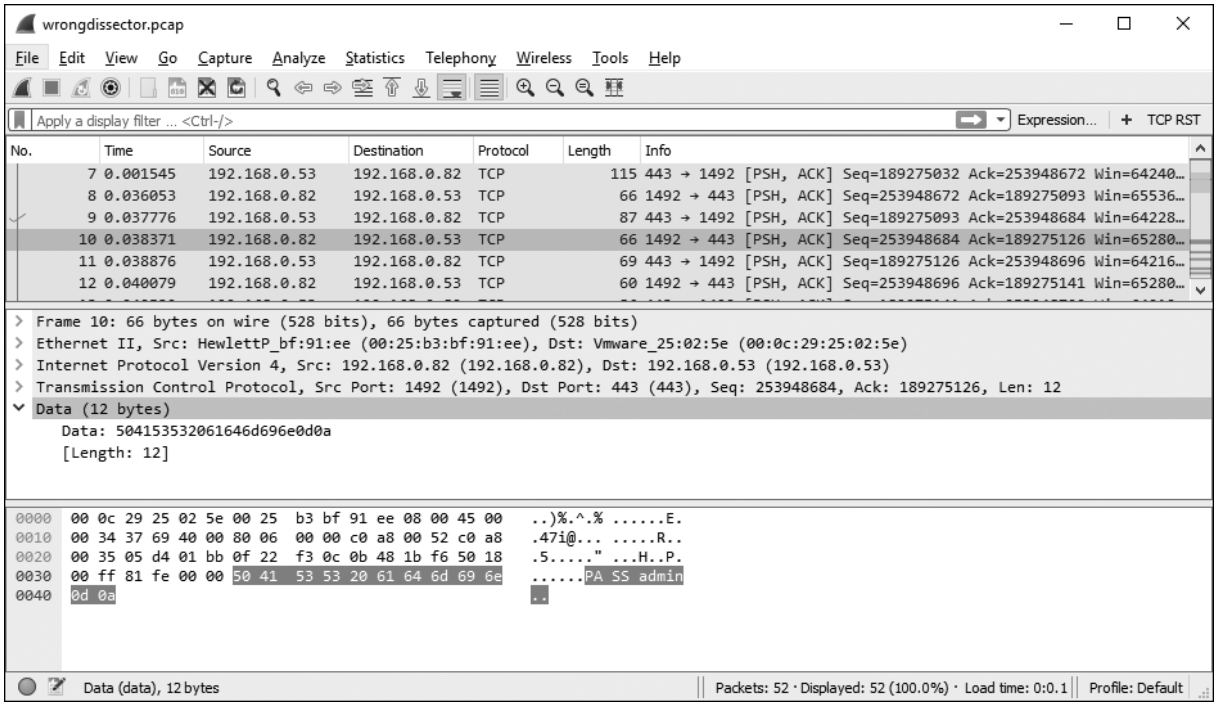


Рисунок 5-11. Имена пользователей и пароли открытым текстом? Это гораздо больше похоже на FTP, чем на SSL!

Чтобы исправить проблему, можно задать Wireshark принудительное декодирование этих пакетов анализатором протокола FTP. Выполните следующие действия:

1. Щёлкните правой кнопкой мыши пакет SSL, например пакет 30, в столбце Protocol и выберите Decode As, чтобы открыть новое диалоговое окно.
2. Предпишите Wireshark декодировать весь трафик TCP порта 443 как FTP: выберите TCP port в столбце Field, введите 443 в столбце Value и выберите FTP в раскрывающемся меню столбца Current, как показано на рисунке 5-12.

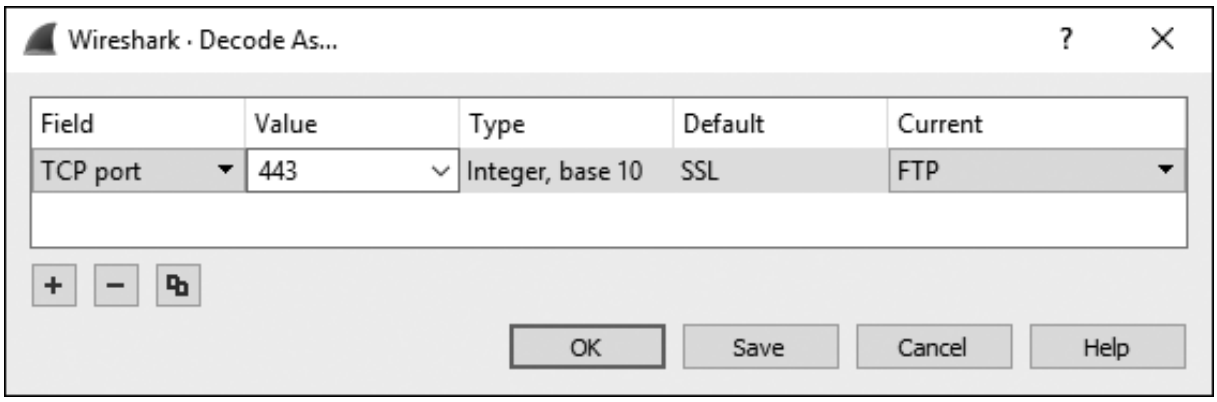


Рисунок 5-12. Диалоговое окно Decode As... позволяет создавать правила принудительного декодирования

3. Щёлкните OK, чтобы немедленно применить изменения к файлу захвата.

Данные будут декодированы как трафик FTP, и их можно будет анализировать на панели Packet List без глубокого изучения отдельных байтов (рисунок 5-13).

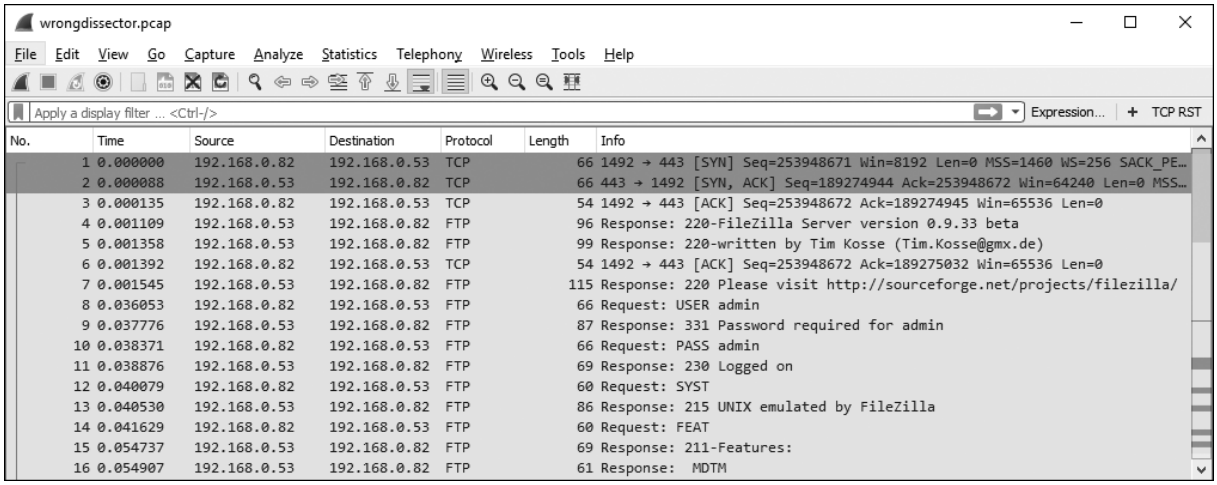


Рисунок 5-13. Просмотр правильно декодированного трафика FTP

В одном файле захвата принудительное декодирование можно применять несколько раз. Wireshark отслеживает созданные правила в диалоговом окне Decode As..., где их все можно просмотреть и изменить.

По умолчанию принудительные правила декодирования не сохраняются при закрытии захвата. Исправить это можно кнопкой Save в диалоговом окне Decode As.... Правила декодирования протоколов сохранятся в текущем пользовательском профиле Wireshark и будут применяться при открытии любого захвата с этим профилем. Сохранённые правила можно удалить кнопкой со знаком «минус» в диалоговом окне.

Правила декодирования очень легко сохранить и забыть о них. Когда вы к этому не готовы, результатом может стать сильная путаница, поэтому помните о принудительном декодировании. Чтобы самому не стать жертвой такой оплошности, я обычно не сохраняю эти правила в основном профиле Wireshark.

Просмотр исходного кода анализатора

Прелесть приложения с открытым исходным кодом в том, что, если причина происходящего непонятна, можно заглянуть в код и выяснить её. Это особенно полезно, когда вы пытаетесь понять, почему определённый протокол интерпретирован неверно: можно исследовать отдельные анализаторы протоколов.

Исходный код анализаторов можно изучать прямо на сайте Wireshark: щёлкните ссылку Develop, затем Browse the Code. Она ведёт в репозиторий кода Wireshark, где доступен код выпусков последних версий программы. Анализаторы протоколов находятся в папке `epan/dissectors`, а каждый файл анализатора называется `packet-<protocolname>.c`.

Эти файлы бывают довольно сложными, но все следуют стандартному шаблону и, как правило, очень хорошо прокомментированы. Чтобы понять основные функции каждого анализатора, не обязательно быть специалистом по Си. Если вы хотите глубоко понимать увиденное в Wireshark, рекомендую взглянуть на анализаторы нескольких более простых протоколов.

Отслеживание потоков

Одна из самых приятных аналитических возможностей Wireshark - сборка данных из нескольких пакетов в единый удобочитаемый вид, который часто называют стенограммой пакетов. Вместо того чтобы просматривать данные, отправляемые клиентом серверу множеством небольших частей, и переходить от пакета к пакету, отслеживание потока упорядочивает данные и упрощает их просмотр.

Доступно отслеживание потоков четырёх видов.

Поток TCP. Собирает данные протоколов, использующих TCP, например HTTP и FTP.

Поток UDP. Собирает данные протоколов, использующих UDP, например DNS.

Поток SSL. Собирает данные зашифрованных протоколов, например HTTPS. Для расшифровки трафика необходимо предоставить ключи.

Поток HTTP. Собирает и распаковывает данные протокола HTTP. Это полезно, когда отслеживание данных HTTP через поток TCP не полностью декодирует полезную нагрузку HTTP.

В качестве примера рассмотрим простой обмен HTTP в файле `http_google.pcapng`. Щёлкните любой пакет TCP или HTTP в файле, нажмите правую кнопку мыши и выберите Follow TCP Stream. Поток TCP будет собран, а стенограмма обмена откроется в отдельном окне, как на рисунке 5-14.

Текст в этом окне показан двумя цветами: красный текст, обозначенный здесь более светлой серой заливкой, соответствует трафику от источника к назначению, а синий, показанный более тёмным серым, - трафику в обратном направлении, от назначения к источнику. Цвет зависит от того, какая сторона начала обмен. В нашем примере соединение с веб-сервером установил клиент, поэтому его данные показаны красным.

Обмен в потоке TCP начинается с исходного запроса GET корневого веб-каталога (`/`) и ответа сервера `HTTP/1.1 200 OK`, сообщающего об успешном запросе. Схожая последовательность повторяется в других потоках захвата: клиент запрашивает отдельные файлы, а сервер отвечает ими. Вы наблюдаете, как пользователь просматривает домашнюю страницу Google, но вместо перехода по каждому пакету можете легко прокрутить стенограмму. По сути, вы видите то же, что конечный пользователь, только изнутри.

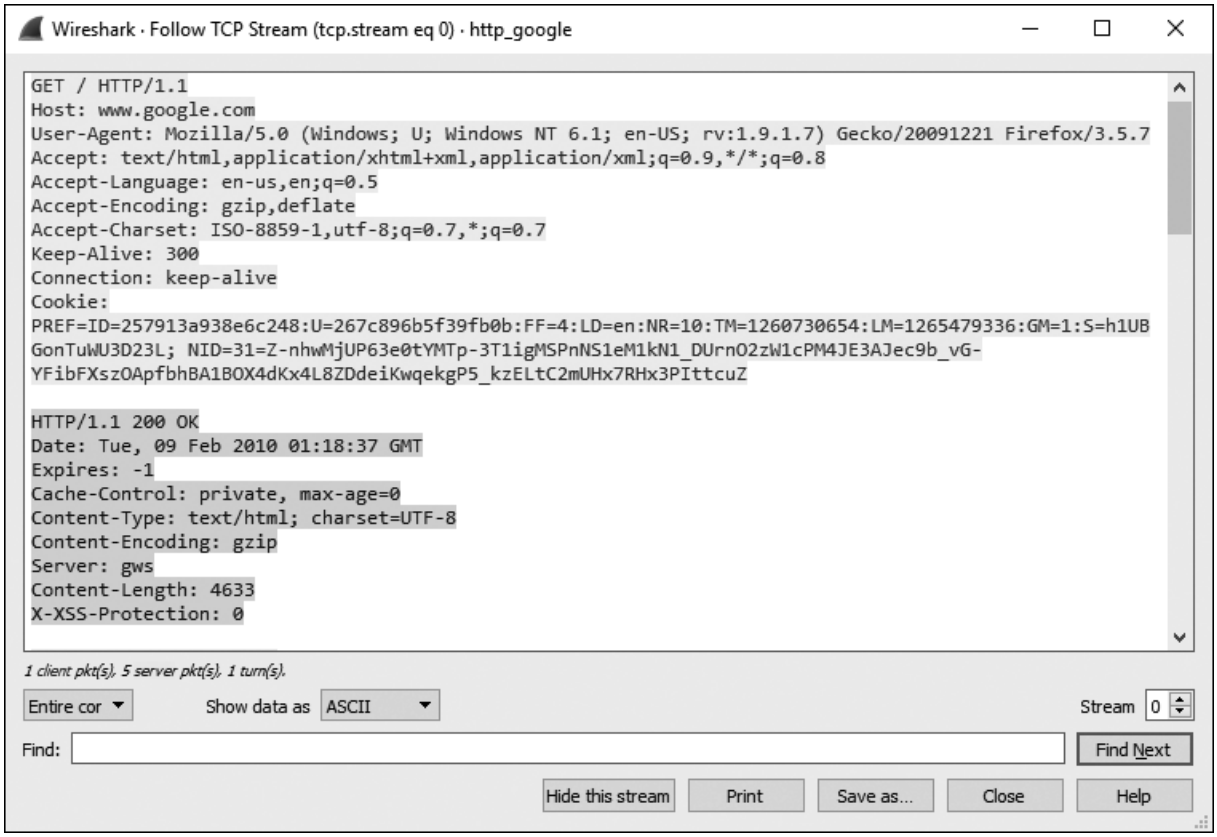


Рисунок 5-14. Окно Follow TCP Stream собирает обмен в удобочитаемый вид

Помимо просмотра необработанных данных в этом окне можно искать по тексту, сохранять его в файл, печатать и выбирать представление данных в формате ASCII, EBCDIC, шестнадцатеричном виде или как массив Си. Эти параметры, которые облегчают изучение крупных наборов данных, находятся в нижней части окна Follow Stream.

Отслеживание потоков SSL

Для отслеживания потоков TCP и UDP достаточно двух щелчков, но просмотр потоков SSL в удобочитаемом виде требует нескольких дополнительных действий. Поскольку трафик зашифрован, необходимо предоставить закрытый ключ, связанный с сервером, отвечающим за этот трафик. Способ получения ключа зависит от технологии сервера и выходит за рамки книги, но, получив его, загрузите ключ в Wireshark следующим образом:

1. Откройте настройки Wireshark командой Edit > Preferences.
2. Разверните раздел Protocols и щёлкните заголовок протокола SSL, показанный на рисунке 5-15. Щёлкните кнопку Edit рядом с подписью RSA keys list.
3. Щёлкните кнопку со знаком «плюс» (+).
4. Предоставьте необходимые сведения: IP-адрес сервера, отвечающего за шифрование, порт, протокол, расположение файла ключа и пароль файла ключа, если он использовался.
5. Перезапустите Wireshark.

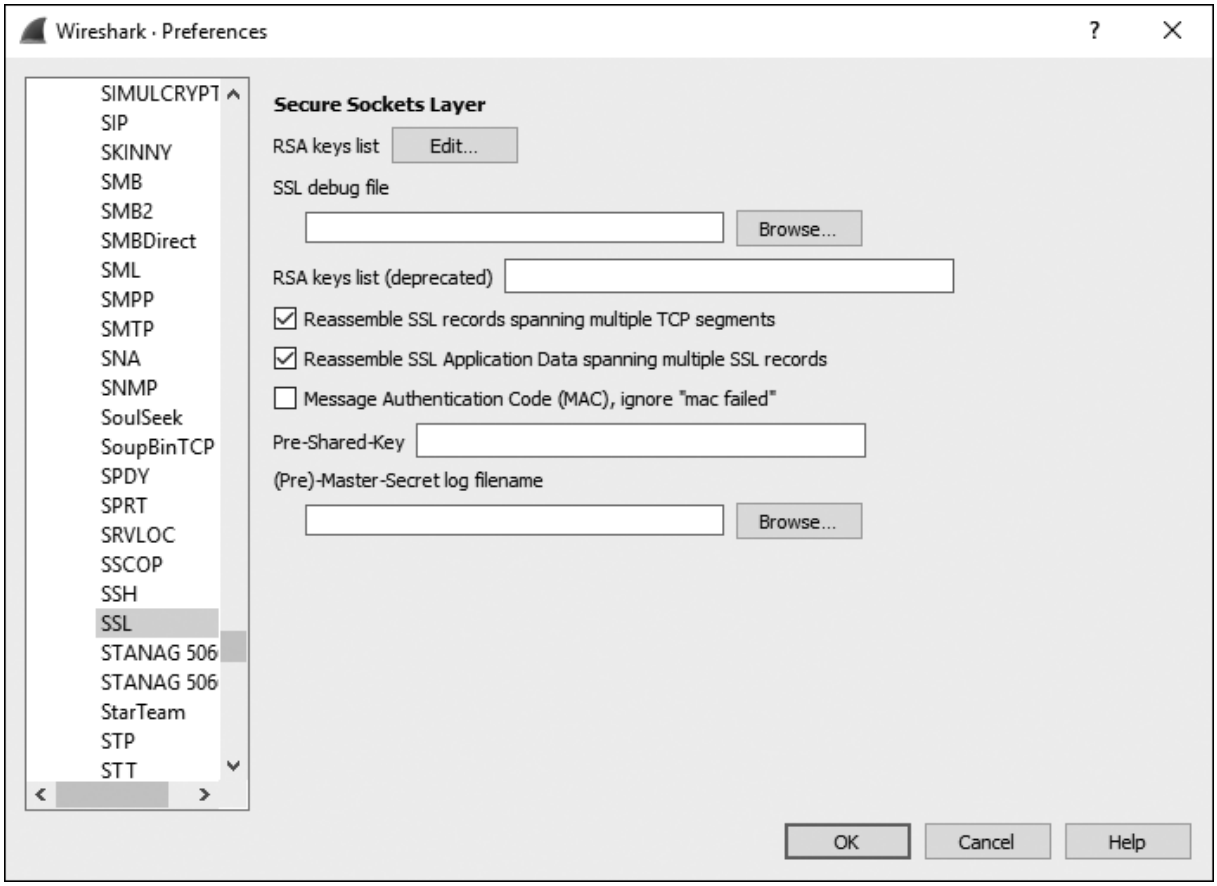


Рисунок 5-15. Добавление сведений для расшифровки SSL

После завершения процесса вы сможете захватывать зашифрованный трафик между клиентом и сервером. Щёлкните пакет HTTPS правой кнопкой мыши и выберите Follow SSL Stream, чтобы увидеть стенограмму открытым текстом.

Просмотр стенограмм пакетов - одна из самых часто используемых аналитических функций Wireshark. Вы привыкнете полагаться на неё, чтобы быстро выяснять, для чего именно применяются определённые протоколы. В следующих главах мы разберём ещё несколько ситуаций, в которых требуется просматривать стенограммы пакетов.

Размеры пакетов

Размер одного пакета или группы пакетов способен многое рассказать о ситуации. В обычных обстоятельствах максимальный размер кадра в сети Ethernet составляет 1518 байт. Если вычесть из этого числа заголовки Ethernet, IP и TCP, останется 1460 байт для передачи заголовка протокола уровня 7 или данных. Зная минимальные требования к передаче пакетов, можно изучить распределение их размеров в захвате и сделать обоснованные предположения о составе трафика. Это чрезвычайно помогает при попытке понять содержимое крупных файлов захвата. Для просмотра распределения пакетов по размеру Wireshark предоставляет диалоговое окно Packet Lengths.

Рассмотрим пример, открыв файл `download-slow.pcapng`. После его открытия выберите Statistics > Packet Lengths. Появится диалоговое окно Packet Lengths, показанное на рисунке 5-16.

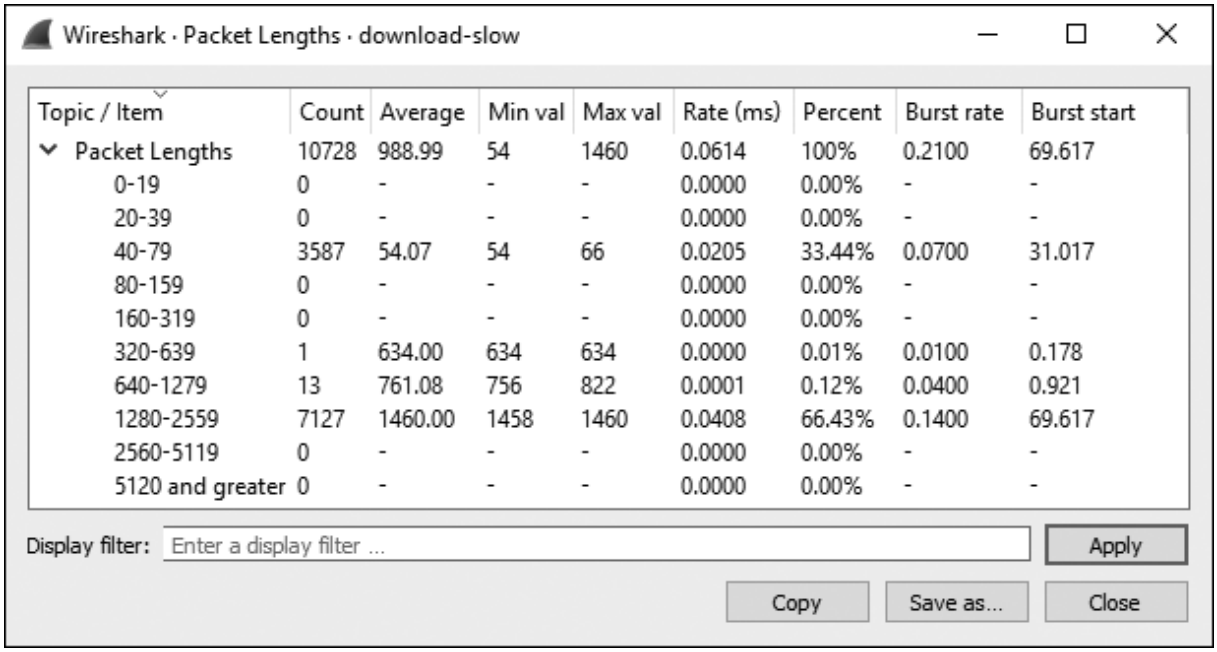


Рисунок 5-16. Диалоговое окно Packet Lengths помогает делать обоснованные предположения о трафике в файле захвата

Особое внимание обратите на строку со статистикой пакетов размером от 1280 до 2559 байт. Такие крупные пакеты обычно указывают на передачу данных, тогда как меньшие представляют управляющие последовательности протоколов. В данном случае велика доля крупных пакетов - 66,43 процента. Даже не глядя на пакеты в файле, можно обоснованно предположить, что захват содержит одну или несколько передач данных. Это может быть загрузка по HTTP, отправка по FTP или любой другой вид сетевого обмена, в котором данные передаются между узлами.

Большинство остальных пакетов - 33,44 процента - имеют размер от 40 до 79 байт. Пакеты этого диапазона обычно являются управляющими пакетами TCP без данных. Рассмотрим типичные размеры заголовков протоколов. Заголовок Ethernet занимает 14 байт плюс четырёхбайтовая контрольная сумма CRC, минимальный заголовок IP - 20 байт, а заголовок пакета TCP без данных и параметров - тоже 20 байт. Значит, стандартные управляющие пакеты TCP, например SYN, ACK, RST и FIN, будут иметь размер около 54 байт и попадут в этот диапазон. Разумеется, параметры IP или TCP увеличат размер. Подробно IP и TCP рассматриваются соответственно в главах 7 и 8.

Изучение размеров пакетов - прекрасный способ окинуть крупный захват с высоты птичьего полёта. При большом количестве крупных пакетов можно достаточно уверенно предположить, что передаются данные. Если большинство пакетов мало и, следовательно, данных передаётся немного, можно предположить, что захват состоит из управляющих команд протоколов. Это не незыблемые правила, но подобные предположения полезны перед более глубоким погружением.

Построение графиков

Графики - основа анализа и один из лучших способов получить общее представление о наборе данных. В Wireshark имеется несколько средств построения графиков, помогающих понимать данные захвата; прежде всего рассмотрим графики ввода-вывода.

Просмотр графиков ввода-вывода

Окно IO Graph программы Wireshark позволяет строить график пропускной способности данных в сети. Такие графики помогают находить всплески и спады пропускной способности, обнаруживать отставание производительности отдельных протоколов и сравнивать одновременные потоки данных.

Чтобы увидеть пример графика ввода-вывода компьютера, скачивающего файл из Интернета, откройте `download-fast.pcapng`. Щёлкните любой пакет TCP, чтобы выделить его, а затем выберите Statistics > IO Graph.

Окно IO Graph графически показывает поток данных с течением времени. В примере на рисунке 5-17 видно, что представленная загрузка в среднем передаёт около 500 пакетов в секунду, остаётся довольно равномерной на всём протяжении, а под конец постепенно сходит на нет.

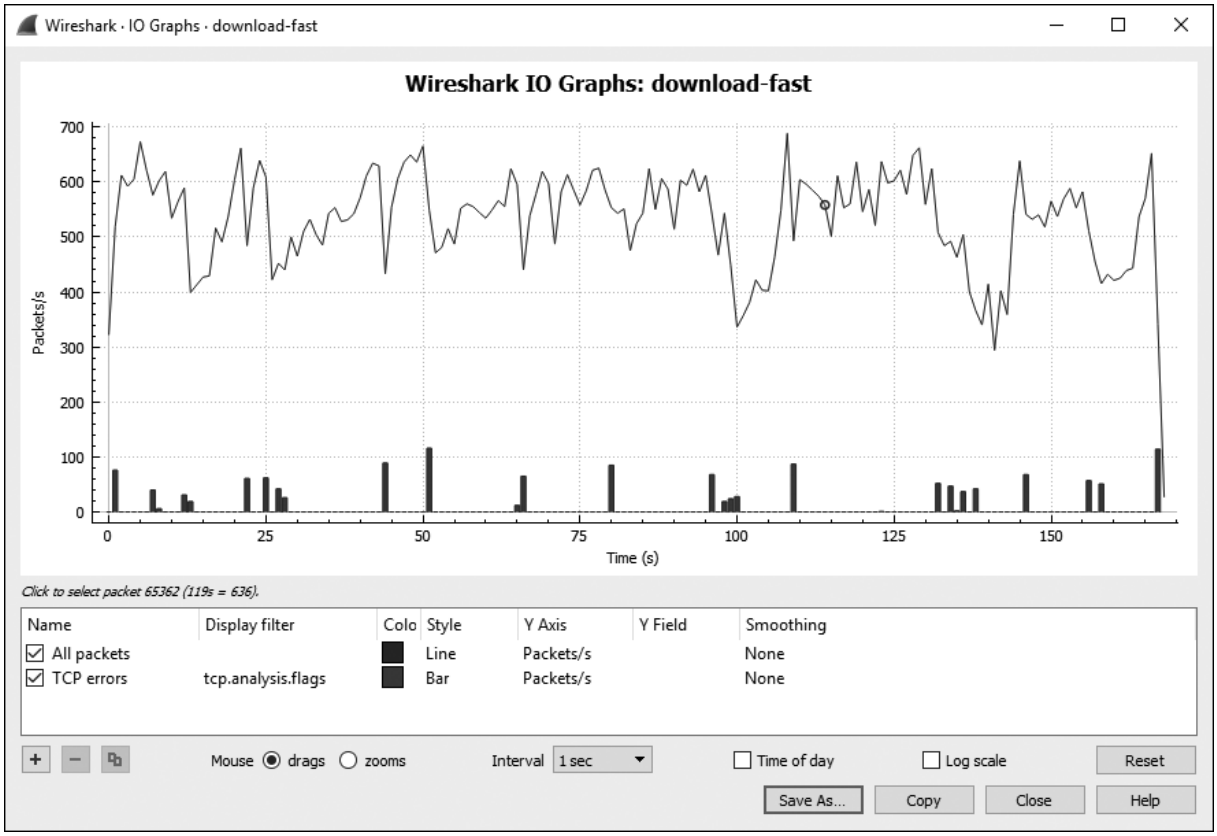


Рисунок 5-17. График ввода-вывода быстрой загрузки в основном равномерен

Сравним его с примером более медленной загрузки. Не закрывая текущий файл, откройте `download-slow.pcapng` в другом экземпляре Wireshark. Откройте график ввода-вывода этой загрузки, и вы увидите совершенно иную картину, показанную на рисунке 5-18.

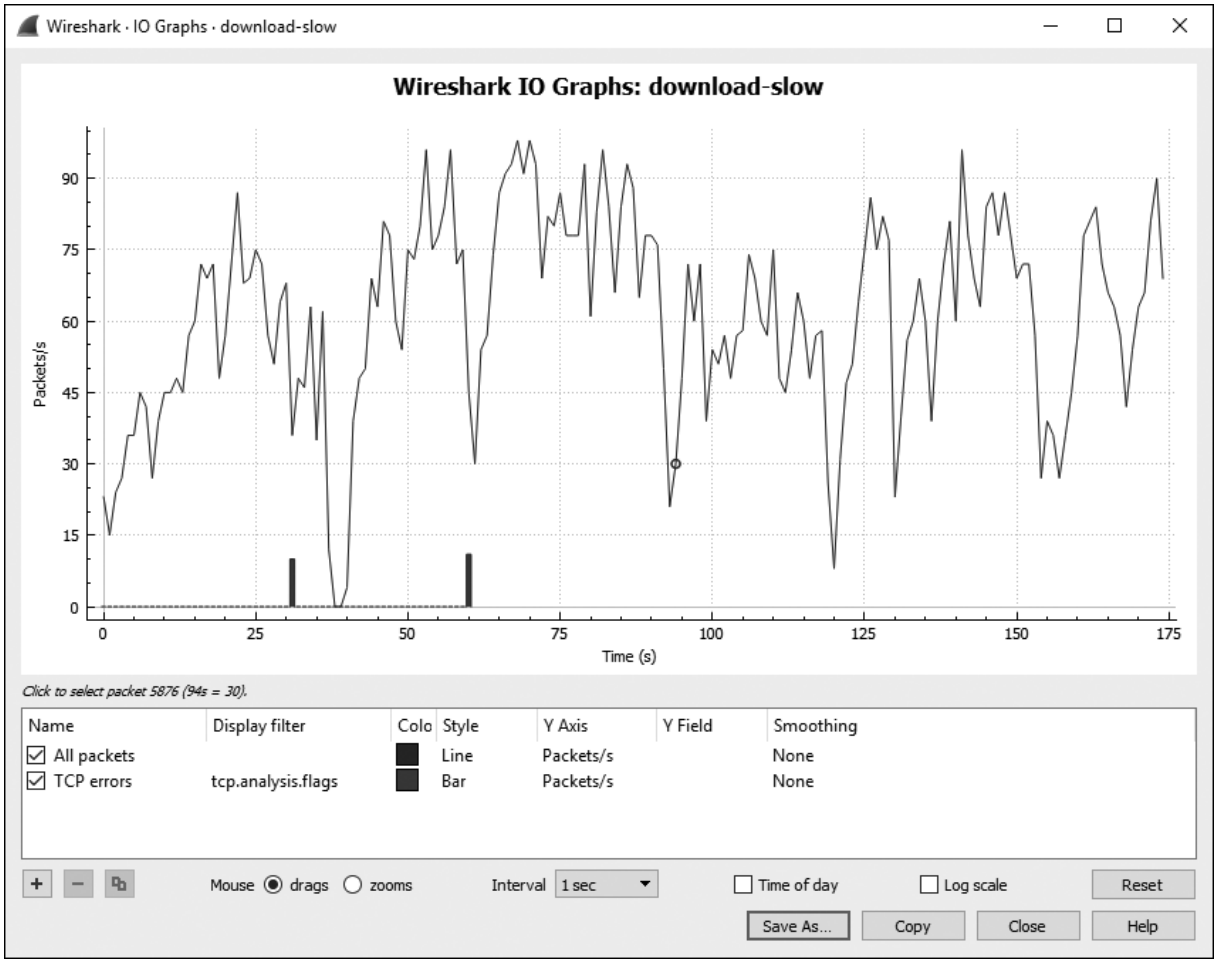


Рисунок 5-18. График ввода-вывода медленной загрузки совершенно неравномерен

Скорость этой загрузки находится между 0 и 100 пакетами в секунду и крайне непостоянна, иногда приближаясь к нулю. Несогласованность видна ещё яснее, если поместить графики двух файлов рядом (см. рисунок 5-19). Сравнивая графики, обращайте внимание на значения осей X и Y, чтобы сопоставлять сравнимые величины. Масштаб автоматически подстраивается под число пакетов и/или количество переданных данных, и именно в этом состоит ключевое различие двух графиков на рисунке 5-19. Шкала медленной загрузки охватывает от 0 до 100 пакетов в секунду, а шкала быстрой - от 0 до 700.

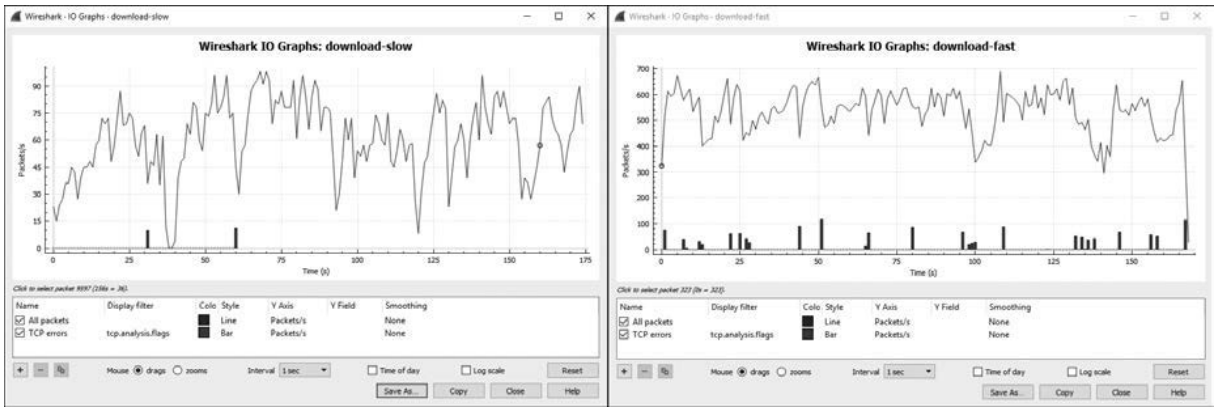


Рисунок 5-19. Одновременный просмотр нескольких графиков ввода-вывода помогает замечать различия

Настраиваемые параметры в нижней части этого окна позволяют применять несколько уникальных фильтров с тем же синтаксисом, что у фильтров отображения и захвата, и задавать их цвета.

Например, можно создать фильтры для определённых IP-адресов, назначить им уникальные цвета и просматривать различия пропускной способности каждого устройства. Давайте попробуем.

Откройте `http_espn.pcapng`, захваченный во время посещения устройством домашней страницы ESPN. В окне Conversations видно, что самый активный внешний IP-адрес - `205.234.218.129`. Отсюда можно заключить, что этот узел, вероятно, является основным поставщиком содержимого, от которого мы получаем данные при посещении `espn.com`. Однако в обменах участвует и несколько других IP-адресов, скорее всего, потому, что дополнительное содержимое скачивается у сторонних поставщиков и рекламодателей. Различие между прямой и сторонней доставкой содержимого можно показать графиком ввода-вывода на рисунке 5-20.

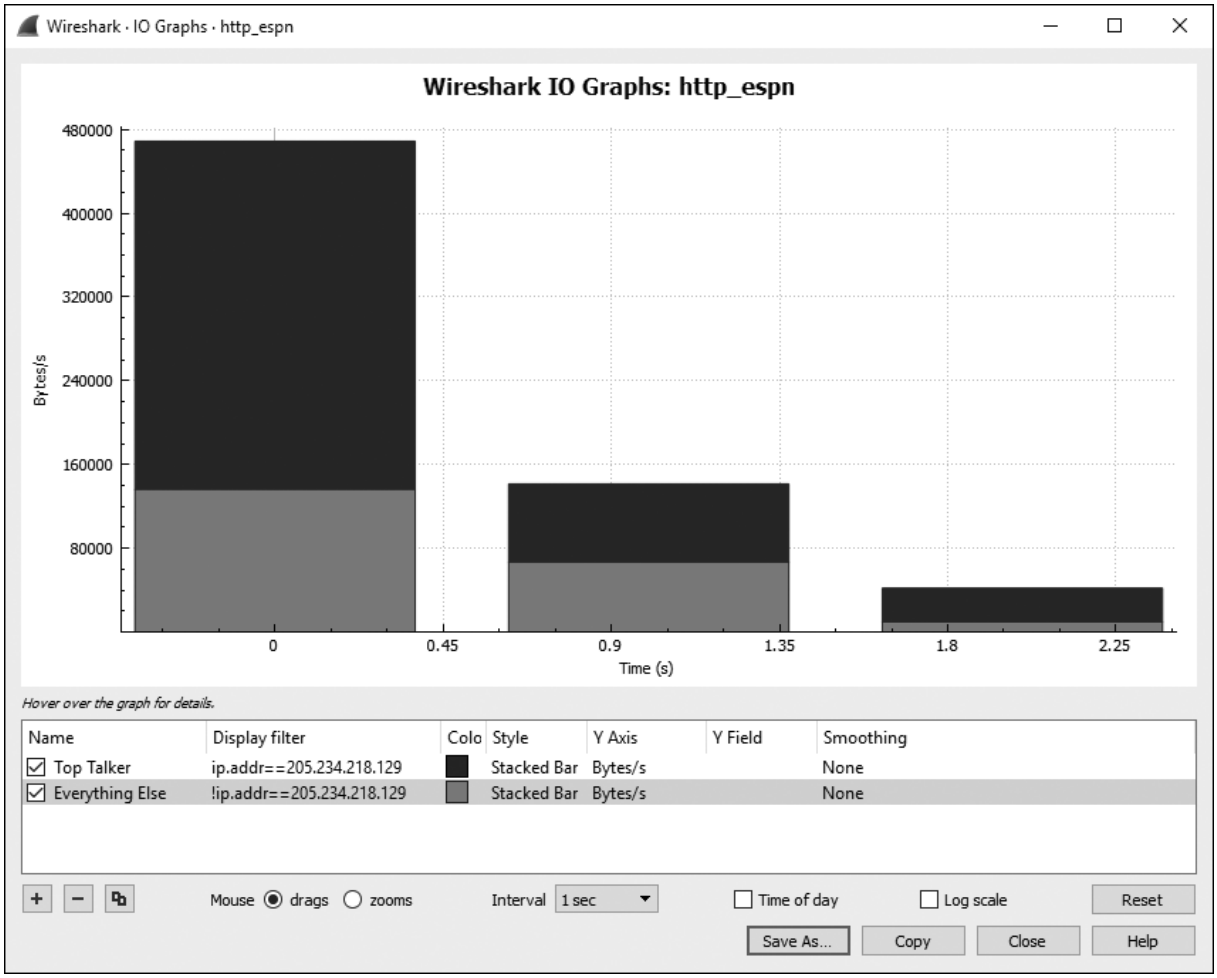


Рисунок 5-20. График ввода-вывода двух отдельных устройств

Два применённых на графике фильтра представлены строками в нижней части окна IO Graph. Фильтр Top Talker показывает ввод-вывод только для IP-адреса `205.234.218.129`, нашего основного поставщика содержимого. Это значение строится чёрным цветом в виде столбцов с накоплением. Второй фильтр, Everything Else, показывает ввод-вывод всего файла захвата, кроме адреса `205.234.218.129`, а значит, включает всех сторонних поставщиков содержимого. Это значение строится красным цветом, который здесь показан более светлым серым, также в виде столбцов с накоплением. Обратите внимание, что единица оси Y изменена на байты в секунду. После этих изменений очень легко увидеть разницу между основным и сторонними поставщиками содержимого и понять, как много данных на самом деле поступает из сторонних источников. Это увлекательное упражнение можно повторить для часто посещаемых сайтов; кроме того, такой приём полезен при сравнении ввода-вывода разных сетевых узлов.

Построение графика времени кругового обхода

Ещё одна графическая возможность Wireshark - построение графика времени кругового обхода для определённого файла захвата. Время кругового обхода (round-trip time, RTT) - время до получения подтверждения пакета. Фактически это время, за которое пакет достигает назначения, а его подтверждение возвращается к вам. RTT часто анализируют, чтобы находить медленные участки или узкие места обмена и определять наличие задержек.

Испытаем эту функцию. Откройте файл `download-fast.pcapng`. Чтобы увидеть его график RTT, выберите пакет TCP, затем Statistics > TCP Stream Graphs > Round Trip Time Graph. График RTT файла `download-fast.pcapng` показан на рисунке 5-21.

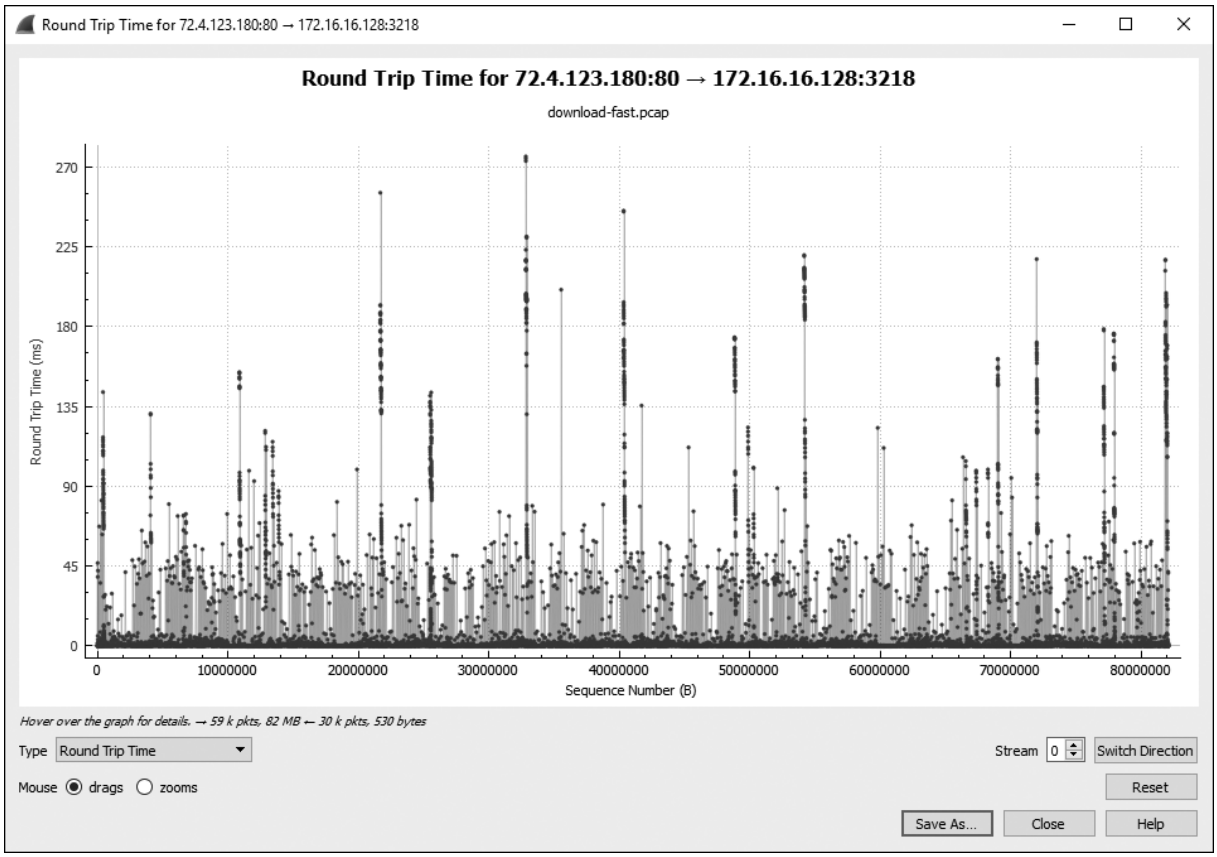


Рисунок 5-21. График RTT быстрой загрузки выглядит в основном равномерным, если не считать нескольких выбросов

Каждая точка графика представляет RTT пакета. В стандартном представлении значения отсортированы по порядковым номерам. Щёлкнув точку на графике, можно перейти непосредственно к этому пакету на панели Packet List.

> **Примечание.** График RTT однонаправленный, поэтому важно выбрать правильное направление анализируемого трафика. Если ваш график не похож на рисунок 5-21, возможно, придётся дважды щёлкнуть Switch Direction.

По-видимому, на графике RTT быстрой загрузки большинство значений меньше 0,05 секунды, а несколько более медленных точек лежит между 0,10 и 0,25 секунды. Хотя повышенных значений довольно много, большая часть RTT приемлема, поэтому для загрузки файла такое время кругового обхода считается допустимым. Исследуя график RTT в поисках проблем пропускной способности, обращайте внимание на большие задержки, которые проявляются как множество точек с высокими значениями по оси Y.

Построение графика потоков

График потоков полезен для наглядного представления соединений и прохождения данных во времени, что упрощает понимание взаимодействия устройств. Он показывает соединение между узлами в виде столбцов и упорядочивает трафик для визуальной интерпретации.

Чтобы создать график потока, откройте файл `dns_recursivequery_server.pcapng` и выберите Statistics > Flow Graph. Получившийся график показан на рисунке 5-22.

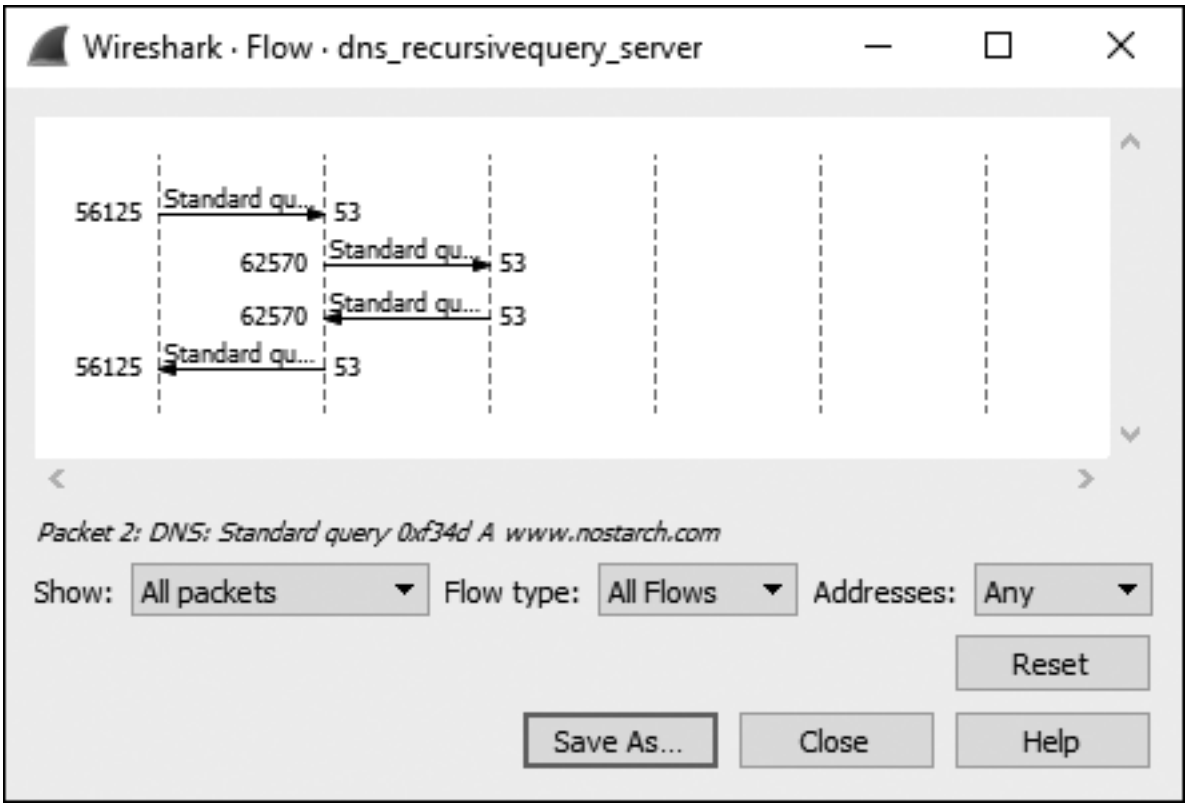


Рисунок 5-22. График потока TCP позволяет гораздо нагляднее представить соединение

На этом графике показан рекурсивный запрос DNS, то есть запрос, который принимает один узел и пересылает другому (DNS рассматривается в главе 9). Каждая вертикальная линия графика соответствует отдельному узлу. График потоков отлично показывает двусторонний обмен между двумя устройствами или, как в данном примере, взаимосвязь обменов нескольких устройств. Он также помогает понять нормальное течение обмена по протоколам, с которыми у вас меньше опыта.

Экспертная информация

Анализаторы каждого протокола в Wireshark определяют экспертные сведения, которые могут предупреждать об определённых состояниях пакетов этого протокола. Эти состояния разделены на четыре категории.

Chat. Основные сведения об обмене.

Note. Необычные пакеты, которые могут быть частью нормального обмена.

Warning. Необычные пакеты, которые, скорее всего, не являются частью нормального обмена.

Error. Ошибка в пакете или в анализаторе, который его интерпретирует.

Например, откройте файл `download-slow.pcapng`. Затем щёлкните Analyze и выберите Expert Information, чтобы открыть окно Expert Information. В нём отключите Group by summary, чтобы упорядочить результаты по серьёзности (см. рисунок 5-23).

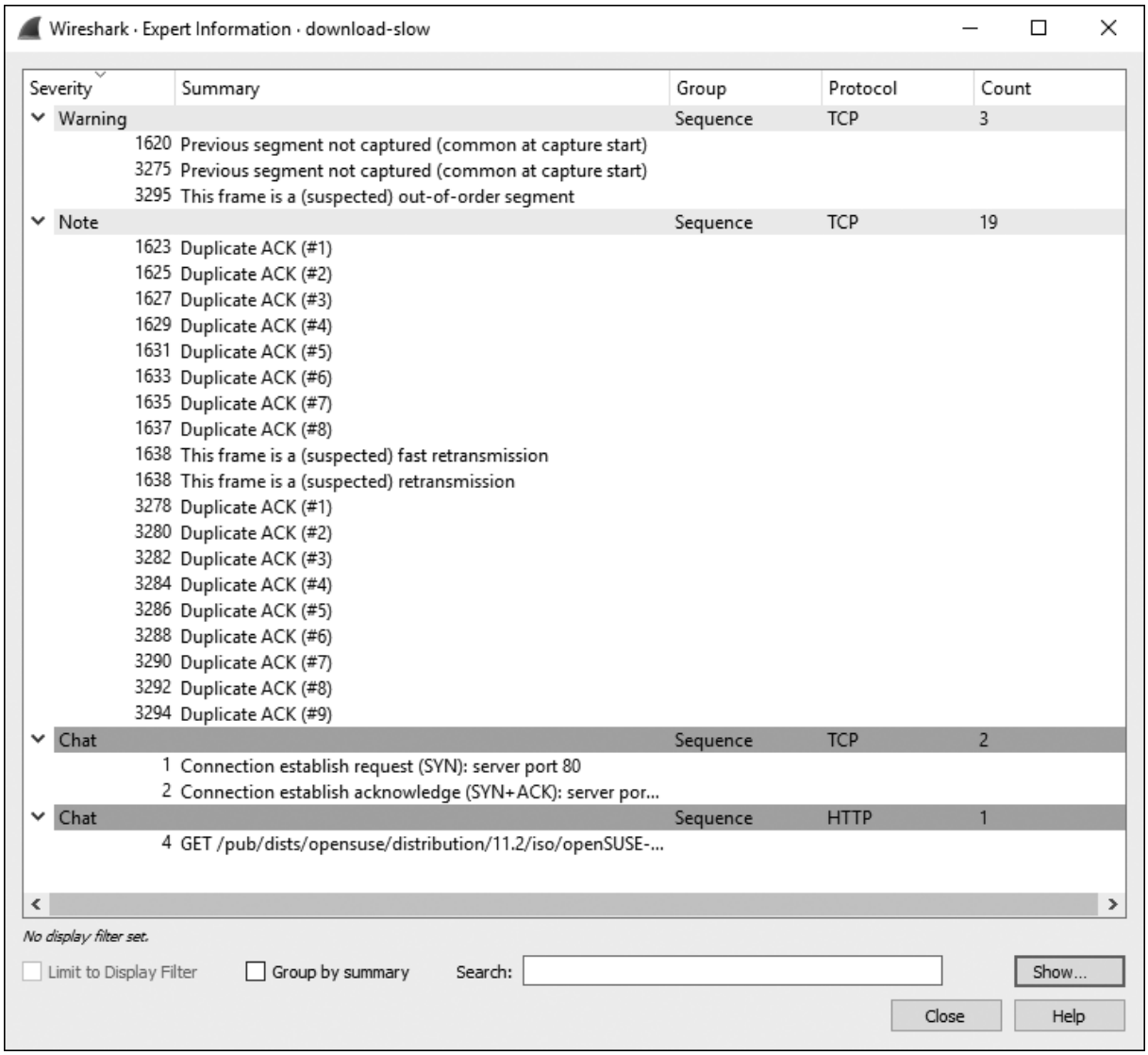


Рисунок 5-23. Окно Expert Information показывает сведения экспертной системы, запрограммированной в анализаторах протоколов

В окне есть разделы для каждой категории сведений. Здесь нет ошибок, но есть 3 предупреждения, 19 примечаний и 3 сообщения.

Большинство сообщений в этом файле захвата относится к TCP просто потому, что исторически экспертная система чаще всего применялась с этим протоколом. На данный момент для TCP настроено 29 сообщений экспертной информации, которые будут полезны при поиске неполадок в файлах захвата. Эти сообщения отмечают отдельный пакет, когда он удовлетворяет определённым критериям из приведённого ниже списка. (Смысл сообщений станет понятнее при изучении TCP в главе 8 и поиске неполадок медленных сетей в главе 11.)

Сообщения Chat

Window Update. Отправляется получателем, чтобы уведомить отправителя об изменении размера окна приёма TCP.

Сообщения Note

TCP Retransmission. Возникает из-за потери пакета, когда получено повторное подтверждение ACK или истёк таймер повторной передачи пакета.

Duplicate ACK. Когда узел не получает следующий ожидаемый порядковый номер, он создаёт повторное подтверждение ACK последних полученных данных.

Zero Window Probe. Проверяет состояние окна приёма TCP после передачи пакета нулевого окна (рассматривается в главе 11).

Keep Alive ACK. Отправляется в ответ на пакеты поддержания соединения.

Zero Window Probe ACK. Отправляется в ответ на пакеты проверки нулевого окна.

Window Is Full. Уведомляет передающий узел о заполнении окна приёма TCP получателя.

Сообщения Warning

Previous Segment Lost. Указывает на потерю пакета. Появляется, когда в потоке данных пропущен ожидаемый порядковый номер.

ACKed Lost Packet. Возникает, когда замечен пакет ACK, но отсутствует пакет, получение которого он подтверждает.

Keep Alive. Срабатывает при обнаружении пакета поддержания соединения.

Zero Window. Появляется, когда окно приёма TCP заполнено и отправлено уведомление о нулевом окне с просьбой к отправителю прекратить передачу данных.

Out-of-Order. С помощью порядковых номеров обнаруживает получение пакетов не по порядку.

Fast Retransmission. Повторная передача, происходящая в течение 20 миллисекунд после повторного подтверждения ACK.

Сообщения Error

Сообщений об ошибках нет.

Хотя некоторые рассмотренные в этой главе функции могут показаться нужными лишь в редких ситуациях, вы, вероятно, будете пользоваться ими чаще, чем ожидаете. Важно познакомиться с этими окнами и параметрами: я буду часто ссылаться на них в нескольких следующих главах.

Глава 6. Анализ пакетов в командной строке

Хотя во многих ситуациях можно обойтись графическим интерфейсом, иногда использование инструментов командной строки, таких как TShark или tcpdump, необходимо или предпочтительно. Вот несколько ситуаций, в которых вместо Wireshark может применяться средство командной строки:

• Wireshark показывает сразу очень много сведений. Средство командной строки позволяет ограничить вывод только относящимися к делу данными, например единственной строкой с IP-адресами.
• Инструменты командной строки лучше всего подходят для фильтрации файла захвата пакетов и передачи результатов непосредственно другой программе через каналы Unix.
• Работа с очень большим файлом захвата часто перегружает Wireshark, поскольку весь файл необходимо загрузить в оперативную память. Потоковая обработка крупных файлов средствами командной строки позволяет быстро отфильтровать файл до нужных пакетов.
• Если вы работаете с сервером и не имеете доступа к графическому инструменту, возможно, придётся полагаться на командную строку.

В этой главе я продемонстрирую возможности двух распространённых средств анализа пакетов в командной строке - TShark и tcpdump. Полезно знать оба, но в системах Windows я обычно пользуюсь TShark, а в Unix - tcpdump. Если вы работаете исключительно в Windows, разделы о tcpdump можно пропустить.

Установка TShark

«Терминальный Wireshark», или TShark, - приложение анализа пакетов, предоставляющее множество тех же функций, что Wireshark, но исключительно через интерфейс командной строки, без графического интерфейса. Если вы установили Wireshark, TShark, скорее всего, тоже установлен, если только вы явно не отказались от него во время установки Wireshark. Проверить наличие TShark можно следующим образом:

1. Откройте командную строку. Щёлкните меню Start, введите `cmd` и выберите Command Prompt.
2. Перейдите в каталог установки Wireshark. Если программа установлена в стандартное место, введите в командной строке `cd C:\Program Files\Wireshark`.
3. Запустите TShark и выведите сведения о версии командой `tshark -v`. Если TShark не установлен, появится ошибка о том, что команда не распознана. Если он установлен, будет выведена информация о версии:

```
C:\Program Files\Wireshark>tshark -v
TShark (Wireshark) 2.0.0 (v2.0.0-0-g9a73b82 from master-2.0
--snip--
```

Если вы не установили TShark, но хотите им пользоваться, просто повторно запустите установку Wireshark и убедитесь, что TShark выбран. (По умолчанию он выбран.)

Если вы хотите немедленно начать изучать возможности TShark, список доступных команд выводится аргументом `-h`. Некоторые из этих команд рассматриваются в данной главе.

```
C:\Program Files\Wireshark>tshark -h
```

Подобно Wireshark, TShark работает в нескольких операционных системах, но благодаря отсутствию зависимости от графических библиотек конкретной ОС пользовательский опыт на разных платформах более единообразен. Поэтому TShark очень похожим образом действует в

Windows, Linux и OS X. И всё же между платформами остаются различия. В книге мы сосредоточимся на запуске TShark в Windows, поскольку это основная операционная система, для которой он предназначался.

Установка tcpdump

Wireshark - самое популярное в мире графическое приложение анализа пакетов, а tcpdump с огромным отрывом является самым популярным приложением командной строки для этой цели. tcpdump разработан для операционных систем семейства Unix, очень легко устанавливается распространёнными диспетчерами пакетов и даже поставляется предустановленным во многих вариантах Linux.

Хотя большая часть книги ориентирована на Windows, для пользователей Unix включены разделы о tcpdump. В частности, мы будем применять Ubuntu 14.04 LTS. Если вы хотите пользоваться tcpdump в Windows, можно скачать и установить его аналог WinDump с сайта winpcap.org. Работа с tcpdump и WinDump не полностью одинакова, но эти анализаторы пакетов действуют схожим образом. Помните, однако, что WinDump поддерживается далеко не так активно, как tcpdump. Поэтому в нём могут отсутствовать некоторые новые функции и существовать уязвимости безопасности. (WinDump в книге не рассматривается.)

В Ubuntu tcpdump не предустановлен, однако благодаря системе управления пакетами APT установить его очень легко. Выполните следующие действия:

1. Откройте окно терминала и выполните `sudo apt-get update`, чтобы в репозиториях пакетов были сведения о последних версиях.
2. Выполните `sudo apt-get install tcpdump`.
3. Вам предложат установить несколько компонентов, необходимых для работы tcpdump. Разрешите установку, введя `Y` и нажав Enter.
4. После завершения установки выполните `tcpdump -h`, чтобы запустить tcpdump и вывести сведения о версии. Если команда завершится успешно и в терминале появится примерно следующий текст, можно начинать пользоваться tcpdump:

```
sanders@ppa:~$ tcpdump -h
tcpdump version 4.5.1
libpcap version 1.5.3
Usage: tcpdump [-aAbdDefhHIJKlLnNOpqRStuUvxX#] [ -B size ] [ -c count ]
            [ -C file_size ] [ -E algo:secret ] [ -F file ] [ -G seconds ]
            [ -i interface ] [ -j tstamptype ] [ -M secret ]
            [ -Q metadata-filter-expression ]
            [ -r file ] [ -s snaplen ] [ -T type ] [ --version ] [ -V file ]
            [ -w file ] [ -W filecount ] [ -y datalinktype ] [ -z command ]
            [ -Z user ] [ expression ]
```

Все доступные команды tcpdump можно вывести вызовом `man tcpdump`:

```
sanders@ppa:~$ man tcpdump
```

Мы обсудим применение нескольких из этих команд.

Захват и сохранение пакетов

Первая задача - захватить пакеты с линии и показать их на экране. Чтобы начать захват в TShark, просто выполните команду `tshark`. Она запустит захват пакетов с сетевого интерфейса и вывод на экран в окне терминала, которое будет выглядеть примерно так:

```
C:\Program Files\Wireshark>tshark
  1   0.000000 172.16.16.128 -> 74.125.95.104 TCP 66 1606      80 [SYN]
Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
  2   0.030107 74.125.95.104 -> 172.16.16.128 TCP 66 80      1606 [SYN, ACK]
Seq=0 Ack=1 Win=5720 Len=0 MSS=1406 SACK_PERM=1 WS=64
  3   0.030182 172.16.16.128 -> 74.125.95.104 TCP 54 1606      80 [ACK]
Seq=1 Ack=1 Win=16872 Len=0
  4   0.030248 172.16.16.128 -> 74.125.95.104 HTTP 681 GET / HTTP/1.1
  5   0.079026 74.125.95.104 -> 172.16.16.128 TCP 60 80      1606 [ACK]
Seq=1 Ack=628 Win=6976 Len=0
```

Чтобы начать захват в tcpdump, выполните команду `tcpdump`. После запуска окно терминала должно выглядеть примерно так:

```
sanders@ppa:~$ tcpdump
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
21:18:39.618072 IP 172.16.16.128.slm-api > 74.125.95.104.http: Flags [S],
seq 2082691767, win 8192, options [mss 1460,nop,wscale 2,nop,nop,sackOK],
length 0
21:18:39.648179 IP 74.125.95.104.http > 172.16.16.128.slm-api:
Flags [S.], seq 2775577373, ack 2082691768, win 5720, options [mss
1406,nop,nop,sackOK,nop,wscale 6], length 0
21:18:39.648254 IP 172.16.16.128.slm-api > 74.125.95.104.http: Flags [.],
ack 1, win 4218, length 0
21:18:39.648320 IP 172.16.16.128.slm-api > 74.125.95.104.http: Flags [P.],
seq 1:628, ack 1, win 4218, length 627: HTTP: GET / HTTP/1.1
21:18:39.697098 IP 74.125.95.104.http > 172.16.16.128.slm-api: Flags [.],
ack 628, win 109, length 0
```

> **Примечание.** Поскольку для захвата пакетов в Unix требуются административные привилегии, tcpdump, скорее всего, придётся запускать как пользователь root либо добавлять `sudo` перед командами из книги. Во многих случаях доступ к системе Unix выполняется от имени пользователя с ограниченными правами. Если при повторении примеров появляется ошибка доступа, вероятная причина именно в этом.

В зависимости от конфигурации системы TShark или tcpdump может по умолчанию выбрать не тот сетевой интерфейс, с которого требуется захватывать трафик. Если это произойдёт, интерфейс потребуется указать. Список доступных TShark интерфейсов можно вывести аргументом `-D`, который показывает их в виде нумерованного списка:

```
C:\Program Files\Wireshark>tshark -D
1. \Device\NPF_{1DE095C2-346D-47E6-B855-11917B74603A} (Local Area Connection*
2)
2. \Device\NPF_{1A494418-97D3-42E8-8C0B-78D79A1F7545} (Ethernet 2)
```

Чтобы использовать определённый интерфейс, передайте аргументу `-i` присвоенный этому интерфейсу номер из списка:

```
C:\Program Files\Wireshark>tshark -i 1
```

Эта команда будет захватывать пакеты исключительно из интерфейса Local Area Connection 2, которому в списке интерфейсов присвоен номер 1. Я рекомендую всегда указывать интерфейс, с которого выполняется захват. Инструменты виртуальных машин и VPN часто добавляют интерфейсы, поэтому необходимо быть уверенным, что захватываемые пакеты поступают из правильного источника.

В системе Linux или OS X, где запущен tcpdump, список доступных интерфейсов выводится командой `ifconfig`:

```
sanders@ppa:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:1f:a7:55
          inet addr:172.16.16.139  Bcast:172.16.16.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe1f:a755/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
```

```
          RX packets:5119 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3088 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:876746 (876.7 KB)  TX bytes:538083 (538.0 KB)
```

Интерфейс также указывается аргументом `-i`:

```
sanders@ppa:~$ tcpdump -i eth0
```

Эта команда будет захватывать пакеты исключительно из интерфейса `eth0`.

Когда всё правильно настроено, можно начать захват пакетов. Если устройство, трафик которого вы захватываете, проявляет в сети хотя бы небольшую активность, вы, вероятно, заметите, что строки, соответствующие отдельным пакетам, пролетают довольно быстро - возможно, слишком быстро, чтобы успевать их читать. Это можно исправить, сохранив пакеты в файл, а затем прочитав из него лишь несколько пакетов.

Чтобы сохранить собранные пакеты в файл в обоих инструментах, используйте аргумент `-w` вместе с именем файла. Захват будет продолжаться, пока вы не остановите его нажатием Ctrl-C. Если не указано иное, файл сохраняется в каталоге, из которого была запущена программа.

Пример этой команды в TShark:

```
C:\Program Files\Wireshark>tshark -i 1 -w packets.pcap
```

Эта команда запишет в `packets.pcap` все пакеты, захваченные из первого интерфейса в списке.

В tcpdump та же команда будет выглядеть так:

```
sanders@ppa:~$ tcpdump -i eth0 -w packets.pcap
```

Чтобы прочитать пакеты из сохранённого файла, используйте аргумент `-r` вместе с именем файла:

```
C:\Program Files\Wireshark>tshark -r packets.pcap
```

Эта команда выведет на экран все пакеты из `packets.pcap`.

Команда tcpdump почти такая же:

```
sanders@ppa:~$ tcpdump -r packets.pcap
```

Если файл, который вы пытаетесь прочитать, содержит много пакетов, может возникнуть уже описанная ситуация: пакеты будут прокручиваться на экране слишком быстро, чтобы их можно было читать. При чтении из файла число отображаемых пакетов ограничивается аргументом `-c`.

Например, следующая команда покажет в TShark только первые 10 пакетов файла захвата:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -c10
```

В tcpdump можно использовать тот же аргумент:

```
sanders@ppa:~$ tcpdump -r packets.pcap -c10
```

Аргумент `-c` можно применять и во время захвата. Следующая команда захватит только первые 10 обнаруженных пакетов. Их также можно сохранить, объединив `-c` с аргументом `-w`.

Так выглядит эта команда в TShark:

```
C:\Program Files\Wireshark>tshark -i 1 -w packets.pcap -c10
```

А так - в tcpdump:

```
sanders@ppa:~$ tcpdump -i eth0 -w packets.pcap -c10
```

Управление выводом

Одно из преимуществ инструментов командной строки состоит в том, что их вывод обычно формируется более продуманно. Графический интерфейс, как правило, показывает всё сразу, а искать нужное приходится вам. Инструменты командной строки обычно выводят лишь самый необходимый минимум и вынуждают применять дополнительные команды, чтобы исследовать данные глубже. TShark и tcpdump не исключение. Оба показывают по одной строке вывода для каждого пакета, а для просмотра таких сведений, как подробности протокола или отдельные байты, требуют дополнительных команд.

В выводе TShark каждая строка соответствует одному пакету, а её формат зависит от протоколов, использованных в этом пакете. TShark применяет те же диссекторы, что и Wireshark, и так же анализирует данные пакетов, поэтому при параллельном запуске вывод TShark будет соответствовать панели списка пакетов Wireshark. Поскольку в TShark имеются диссекторы протоколов уровня 7, он способен предоставить о пакетах с такими заголовками намного больше сведений, чем tcpdump.

В tcpdump каждая строка также соответствует одному пакету и форматируется по-разному в зависимости от используемого протокола. Поскольку tcpdump не применяет диссекторы протоколов Wireshark, инструмент не интерпретирует сведения о протоколах уровня 7. Это одно из самых существенных ограничений tcpdump. Вместо этого однострочное представление пакета форматируется в зависимости от протокола транспортного уровня - TCP или UDP (подробнее они рассматриваются в главе 8).

Для пакетов TCP используется следующий формат:

```
[Метка времени] [Протокол уровня 3] [IP источника].[Порт источника] > [IP назначения].
[Порт назначения]: [Флаги TCP], [Порядковый номер TCP], [Номер подтверждения TCP],
[Размер окна TCP], [Длина данных]
```

Для пакетов UDP используется такой формат:

```
[Метка времени] [Протокол уровня 3] [IP источника].[Порт источника] > [IP назначения].
[Порт назначения]: [Протокол уровня 4], [Длина данных]
```

Эти простые однострочные сводки прекрасно подходят для быстрого анализа, но рано или поздно пакет потребуется изучить подробно. В Wireshark для этого нужно щёлкнуть пакет на панели списка пакетов, после чего сведения появятся на панелях подробностей пакета и байтов пакета. Те же сведения доступны в командной строке при помощи нескольких параметров.

Самый простой способ получить больше сведений о каждом пакете - повысить подробность вывода.

В TShark подробность повышается прописной буквой `V`:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -V
```

Для пакетов, прочитанных из файла захвата `packets.pcap`, эта команда выдаст результат, подобный панели подробностей пакета Wireshark. Ниже показаны примеры пакета с обычной подробностью (краткой сводкой) и с расширенной подробностью (более детальными сводками, полученными аргументом `-V`).

Сначала стандартный вывод:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -c1
  1   0.000000 172.16.16.172 -> 4.2.2.1      ICMP Echo (ping) request
id=0x0001, seq=17/4352, ttl=128
```

А теперь часть более подробных сведений, полученных при расширенной подробности:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -V -c1
Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on
```

```
interface 0
    Interface id: 0 (\Device\NPF_{C30671C1-579D-4F33-9CC0-73EFFFE85A54})
    Encapsulation type: Ethernet (1)
    Arrival Time: Dec 21, 2015 12:52:43.116551000 Eastern Standard Time
    [Time shift for this packet: 0.000000000 seconds]
--snip--
```

В tcpdump подробность повышается строчной буквой v. В отличие от TShark, tcpdump позволяет отображать несколько уровней подробности для каждого пакета. Можно задать до трёх уровней, добавляя дополнительные буквы v:

```
sanders@ppa:~$ tcpdump -r packets.pcap -vvv
```

Ниже один и тот же пакет показан с обычной подробностью и с одним уровнем расширенной подробности. Даже при максимальной подробности этот вывод далеко не настолько детален, как вывод TShark.

```
sanders@ppa:~$ tcpdump -r packets.pcap -c1
reading from file packets.pcap, link-type EN10MB (Ethernet)
13:26:25.265937 IP 172.16.16.139 > a.resolvers.level3.net: ICMP echo request,
id 1759, seq 150, length 64
sanders@ppa:~$ tcpdump -r packets.pcap -c1 -v
reading from file packets.pcap, link-type EN10MB (Ethernet)
13:26:25.265937 IP (tos 0x0, ttl 64, id 37322, offset 0, flags [DF], proto
ICMP (1), length 84)
    172.16.16.139 > a.resolvers.level3.net: ICMP echo request, id 1759, seq
150, length 64
```

Доступные уровни подробности зависят от протокола исследуемого пакета. Расширенная подробность полезна, но и она показывает не всё. TShark и tcpdump сохраняют всё содержимое каждого пакета, которое также можно просмотреть в шестнадцатеричном виде или в формате ASCII.

В TShark шестнадцатеричное и ASCII-представление пакетов выводится аргументом `-x`, который можно объединить с аргументом `r`, чтобы прочитать и показать пакет из файла:

```
C:\Program Files\Wireshark>tshark -xr packets.pcap
```

Это представление, подобное панели байтов пакета Wireshark, показано на рис. 6-1.

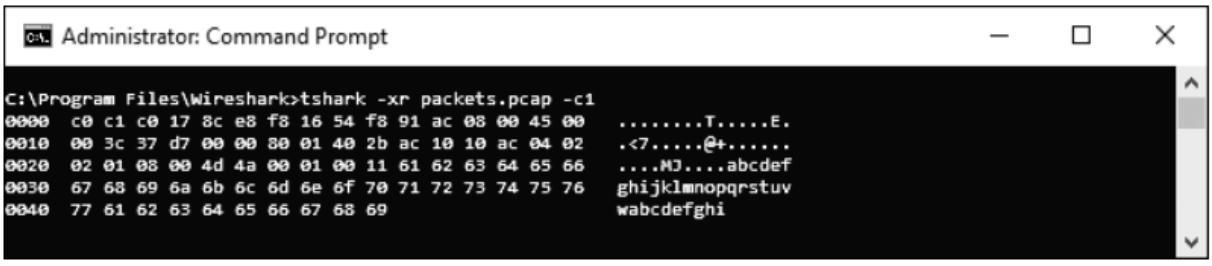


Рисунок 6-1. Просмотр необработанных пакетов в шестнадцатеричном виде и ASCII в TShark

В tcpdump шестнадцатеричное и ASCII-представление выводится параметром `-X`. Его также можно объединить с аргументом `r`, чтобы прочитать файл пакетов:

```
sanders@ppa:~$ tcpdump -Xr packets.pcap
```

Результат этой команды показан на рис. 6-2.

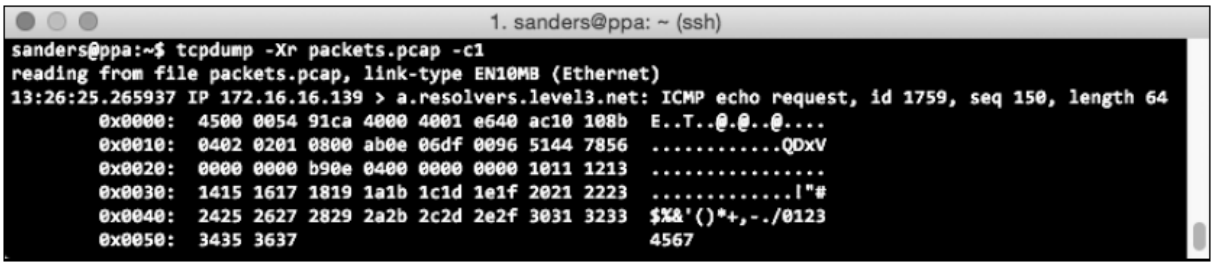


Рисунок 6-2. Просмотр необработанных пакетов в шестнадцатеричном виде и ASCII в tcpdump

При необходимости tcpdump позволяет задать и более точное представление. Только шестнадцатеричный вывод отображается аргументом `-x` (строчная буква), а только ASCII-вывод - аргументом `-A`.

Начав экспериментировать с этими параметрами вывода данных, легко оказаться перегруженным сведениями. При анализе в командной строке эффективнее всего использовать минимально необходимый объём информации. Начните с просмотра пакетов в стандартном представлении списка и переходите к более подробному выводу, когда сузите анализ до нескольких интересующих пакетов. Такой подход не позволит утонуть в данных.

Разрешение имён

Как и Wireshark, TShark и tcpdump попытаются выполнить разрешение имён, чтобы преобразовать адреса и номера портов в имена. Если вы повторяли какой-либо из предыдущих примеров, то могли заметить, что оно выполняется по умолчанию. Как уже говорилось, я обычно предпочитаю отключать эту функцию, чтобы исключить возможность появления в линии дополнительных пакетов из-за самого анализа.

В TShark разрешение имён отключается аргументом `-n`. Как и многие другие аргументы, его можно объединять с другими командами для повышения удобочитаемости:

```
C:\Program Files\Wireshark>tshark -ni 1
```

Отдельные аспекты разрешения имён можно включать или отключать аргументом `-N`. При его использовании все виды разрешения имён будут отключены, кроме явно включённых соответствующими значениями. Например, следующая команда включит только разрешение на транспортном уровне (имён портов):

```
C:\Program Files\Wireshark>tshark -i 1 -Nt
```

Можно объединить несколько значений. Эта команда включит разрешение имён транспортного уровня и MAC-адресов:

```
C:\Program Files\Wireshark>tshark -i 1 -Ntm
```

Для этого параметра доступны следующие значения:

- `m` - разрешение MAC-адресов;
- `n` - разрешение сетевых адресов;
- `t` - разрешение транспортного уровня (имён портов);
- `N` - использование внешних средств разрешения;
- `C` - параллельные DNS-запросы.

В tcpdump аргумент `-n` отключает разрешение имён IP-адресов, а `-nn` дополнительно отключает разрешение имён портов.

Этот аргумент также можно объединять с другими командами:

```
sanders@ppa:~$ tcpdump -nni eth1
```

В следующих примерах показан захват пакета сначала с включённым разрешением имён портов, а затем с отключённым (`-n`).

```
sanders@ppa:~$ tcpdump -r tcp_ports.pcap -c1
reading from file tcp_ports.pcap, link-type EN10MB (Ethernet)
14:38:34.341715 IP 172.16.16.128.2826 > 212.58.226.142.http: Flags [S], seq
3691127924, win 8192, options [mss 1460,nop,wscale 2,nop,nop,sackOK], length 0
sanders@ppa:~$ tcpdump -nr tcp_ports.pcap -c1
reading from file tcp_ports.pcap, link-type EN10MB (Ethernet)
14:38:34.341715 IP 172.16.16.128.2826 > 212.58.226.142.80: Flags [S], seq
3691127924, win 8192, options [mss 1460,nop,wscale 2,nop,nop,sackOK], length 0
```

Обе команды читают только первый пакет из файла захвата `tcp_ports.pcap`. В первой команде разрешение имён портов включено и преобразует порт 80 в `http`, а во второй порт отображается просто как число.

Применение фильтров

Фильтрация в TShark и tcpdump очень гибка, поскольку оба инструмента позволяют применять фильтры захвата BPF. Кроме того, TShark поддерживает фильтры отображения Wireshark. Как и в Wireshark, фильтры захвата TShark можно применять только во время захвата, а фильтры отображения - во время захвата или при показе уже захваченных пакетов. Сначала рассмотрим фильтры TShark.

Фильтры захвата применяются аргументом `-f`, после которого в кавычках указывается выражение BPF. Следующая команда захватит и сохранит только пакеты, направленные на порт 80 и использующие протокол TCP:

```
C:\Program Files\Wireshark>tshark -ni 1 -w packets.pcap -f "tcp port 80"
```

Фильтры отображения применяются аргументом `-Y`, после которого в кавычках указывается синтаксис фильтра Wireshark. Такой фильтр можно применить во время захвата:

```
C:\Program Files\Wireshark>tshark -ni 1 -w packets.pcap -Y "tcp.dstport == 80"
```

Тем же аргументом фильтры отображения можно применять к уже захваченным пакетам. Следующая команда покажет только соответствующие фильтру пакеты из `packets.pcap`:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -Y "tcp.dstport == 80"
```

В tcpdump фильтры указываются непосредственно в конце команды и заключаются в одинарные кавычки. Следующая команда также захватит и сохранит только пакеты, направленные на TCP-порт 80:

```
sanders@ppa:~$ tcpdump -nni eth0 -w packets.pcap 'tcp dst port 80'
```

Фильтр можно задать и при чтении пакетов. Эта команда покажет только соответствующие фильтру пакеты из `packets.pcap`:

```
sanders@ppa:~$ tcpdump -r packets.pcap 'tcp dst port 80'
```

Важно помнить: если исходный файл захвата был создан без фильтра, он по-прежнему содержит остальные пакеты; при чтении существующего файла вы лишь ограничиваете то, что выводится на экран.

А что, если файл захвата содержит множество разных пакетов, но вам требуется отфильтровать их подмножество и сохранить его в отдельный файл? Для этого можно объединить аргументы `-w` и `-r`:

```
sanders@ppa:~$ tcpdump -r packets.pcap 'tcp dst port 80' -w http_packets.pcap
```

Эта команда прочитает файл `packets.pcap`, отфильтрует только трафик, направленный на TCP-порт 80 (используемый HTTP), и запишет эти пакеты в новый файл `http_packets.pcap`. Это очень распространённый приём, когда требуется сохранить большой исходный файл `.pcap`, но анализировать за один раз лишь небольшую его часть. Я часто пользуюсь им, чтобы сократить очень большие файлы захвата посредством tcpdump, а затем проанализировать подмножество пакетов в Wireshark. С небольшими файлами захвата гораздо легче работать.

Помимо непосредственного указания фильтра в командной строке, tcpdump позволяет ссылаться на файл BPF, содержащий набор фильтров. Это удобно для очень большого или сложного фильтра, который иначе было бы трудно редактировать и поддерживать прямо в команде tcpdump. Файл фильтра задаётся аргументом `-F`:

```
sanders@ppa:~$ tcpdump -nni eth0 -F dns_servers.bpf
```

Если файл становится слишком большим, может возникнуть желание добавить в него примечания или комментарии, чтобы не забыть назначение каждой части фильтра. Помните, что файл фильтра BPF не допускает комментариев и выдаст ошибку при обнаружении чего-либо, кроме выражения фильтрации. Поскольку комментарии очень помогают разбираться в больших файлах фильтров, я обычно храню две копии каждого файла: одну без комментариев для tcpdump и одну с комментариями для справки.

Форматы отображения времени в TShark

Новичков часто сбивает с толку стандартная метка времени TShark. В ней время пакета указывается относительно начала захвата. Иногда такое представление предпочтительно, но во многих случаях нужно видеть время фактического захвата пакета, как это по умолчанию делает tcpdump. Такой же вывод можно получить в TShark, передав аргументу `-t` значение `ad`, то есть абсолютную дату:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -t ad
```

Ниже одни и те же пакеты показаны сначала со стандартными относительными метками времени, а затем с абсолютными:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -c2
  1   0.000000 172.16.16.172 -> 4.2.2.1      ICMP Echo (ping)
request  id=0x0001, seq=17/4352, ttl=128
  2   0.024500 4.2.2.1 -> 172.16.16.172      ICMP Echo (ping)
reply    id=0x0001, seq=17/4352, ttl=54 (request in 1)

C:\Program Files\Wireshark>tshark -r packets.pcap -t ad -c2
  1 2015-12-21 12:52:43.116551 172.16.16.172 -> 4.2.2.1      ICMP Echo (ping)
request  id=0x0001, seq=17/4352, ttl=128
  2 2015-12-21 12:52:43.141051      4.2.2.1 -> 172.16.16.172 ICMP Echo (ping)
reply    id=0x0001, seq=17/4352, ttl=54 (request in 1)
```

Аргумент `-t` позволяет задать любой формат отображения времени, доступный в Wireshark. Эти форматы перечислены в табл. 6-1.

Таблица 6-1. Форматы отображения времени, доступные в TShark

Значение	Метка времени	Пример
`a`	Абсолютное время захвата пакета (в вашем часовом поясе)	`15:47:58.004669`
`ad`	Абсолютное время захвата пакета с датой (в вашем часовом поясе)	`2015-10-09 15:47:58.004669`

Значение	Метка времени	Пример
d	Разница во времени с предыдущим захваченным пакетом	0.000140
dd	Разница во времени с предыдущим отображённым пакетом	0.000140
e	Время эпохи (число секунд с 1 января 1970 года, UTC)	1444420078.004669
r	Время, прошедшее между первым и текущим пакетами	0.000140
u	Абсолютное время захвата пакета (UTC)	19:47:58.004669
ud	Абсолютное время захвата пакета с датой (UTC)	2015-10-09 19:47:58.004669

К сожалению, tcpdump не предоставляет столь точного управления форматом отображения меток времени.

Сводная статистика в TShark

Ещё одна полезная функция TShark (и одно из его отличий от tcpdump) - возможность формировать из файла захвата набор статистических данных. Эта статистика воспроизводит многие возможности Wireshark, но предоставляет к ним удобный доступ из командной строки. Для формирования статистики используется аргумент `-z`, после которого указывается название требуемого вида вывода. Полный список доступной статистики можно просмотреть следующей командой:

```
C:\Program Files\Wireshark>tshark -z help
```

Аргумент `-z` предоставляет многие уже рассмотренные функции. В их числе вывод статистики конечных точек и разговоров следующей командой:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -z conv,ip
```

Она печатает таблицу статистики со сведениями об IP-разговорах в файле `packets.pcap`, как показано на рис. 6-3.

С помощью этого аргумента можно также просматривать сведения, относящиеся к конкретному протоколу. Как показано на рис. 6-4, параметр `http,tree` выводит в виде таблицы разбивку HTTP-запросов и ответов.

```
C:\Program Files\Wireshark>tshark -r packets.pcap -z http,tree
```

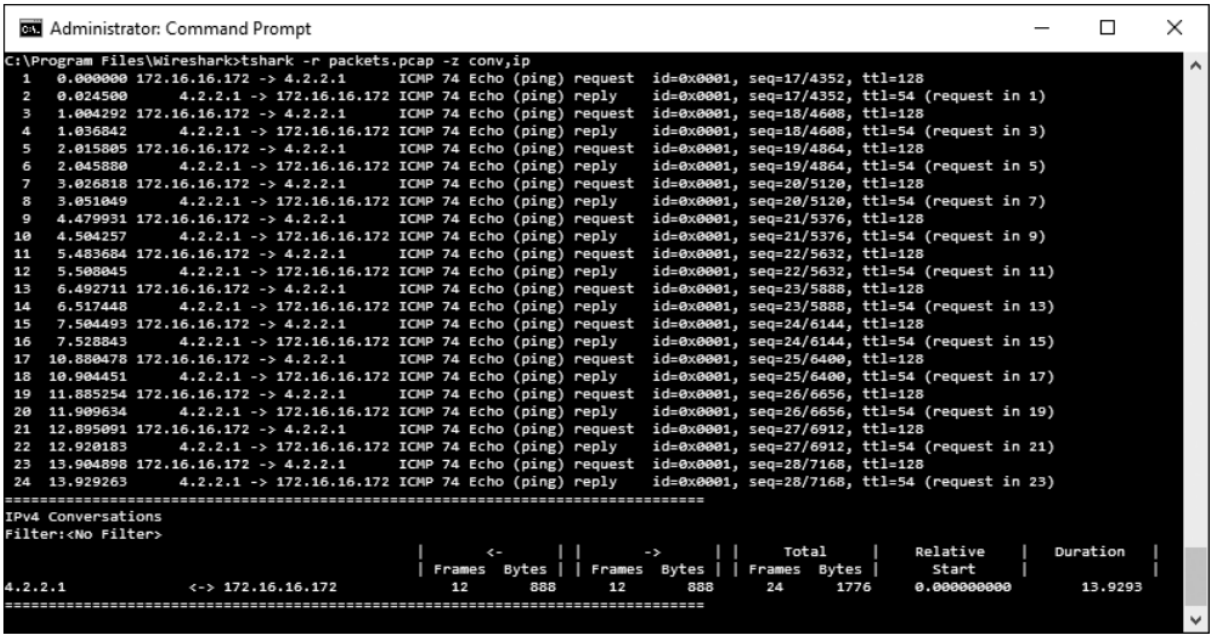


Рисунок 6-3. Просмотр статистики разговоров в TShark

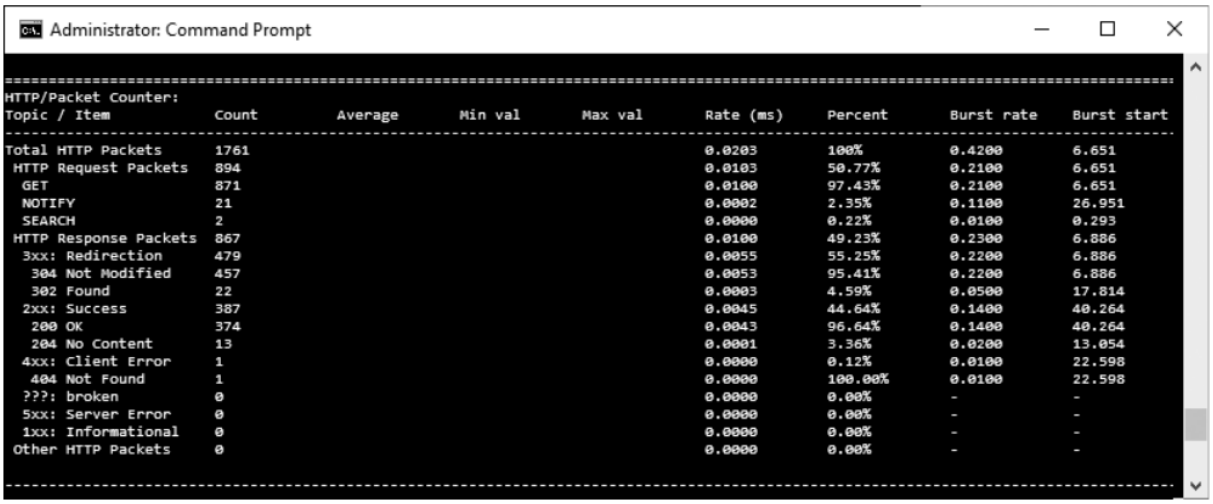


Рисунок 6-4. Просмотр статистики HTTP-запросов и ответов в TShark

Ещё одна полезная функция - просмотр вывода повторно собранного потока, подобный выполненному ранее выбору пункта Follow TCP Stream после щелчка пакета правой кнопкой мыши в Wireshark. Чтобы получить такой вывод, нужно применить параметр `follow` и указать тип потока, режим вывода и поток, который требуется показать. Поток можно определить по номеру, присвоенному ему в крайнем левом столбце при выводе статистики разговоров (как на рис. 6-3). Команда может выглядеть так:

```
C:\Program Files\Wireshark>tshark -r http_google.pcap -z follow,tcp,ascii,0
```

Эта команда выведет на экран в формате ASCII поток TCP 0 из файла `http_google.pcap`. Результат выглядит так:

```
C:\Program Files\Wireshark>tshark -r http_google.pcap -z
--snip--
===================================================================
Follow: tcp,ascii
Filter: tcp.stream eq 0
Node 0: 172.16.16.128:1606
```

```
Node 1: 74.125.95.104:80
627
GET / HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.1.7)
Gecko/20091221 Firefox/3.5.7
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Cookie: PREF=ID=257913a938e6c248:U=267c896b5f39fb0b:FF=4:LD=e
n:NR=10:TM=1260730654:LM=1265479336:GM=1:S=h1UBGonTuWU3D23L;
NID=31=Z-nhwMjUP63e0tYMTp-3T1igMSPnNS1eM1kN1_DUrnO2zW1cPM4JE3AJec9b_
vG-YFibFXszOApfbhBA1BOX4dKx4L8ZDdeiKwqekgP5_kzELtC2mUHx7RHx3PIttcuZ
        1406
HTTP/1.1 200 OK
Date: Tue, 09 Feb 2010 01:18:37 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=UTF-8
Content-Encoding: gzip
Server: gws
Content-Length: 4633
X-XSS-Protection: 0
```

Нужный поток можно также указать сведениями об адресах. Например, следующая команда извлечёт поток UDP для заданных конечных точек и портов:

```
C:\Program Files\Wireshark>tshark -r packets.pcap -z follow,udp,ascii,192.168.1.5:23429,4.2.2.1:53
```

Эта команда выведет из `packets.pcap` поток UDP между конечной точкой `192.168.1.5` на порте 23429 и конечной точкой `4.2.2.1` на порте 53.

Вот некоторые из моих любимых параметров статистики:

- `ip_hosts,tree` - отображает каждый IP-адрес в захвате, а также долю и процент трафика, приходящиеся на каждый адрес;
- `io,phs` - отображает иерархию протоколов со всеми протоколами, найденными в файле захвата;
- `http,tree` - отображает статистику HTTP-запросов и ответов;
- `http_req,tree` - отображает статистику каждого HTTP-запроса;
- `smb,srt` - отображает статистику команд SMB для анализа обмена данными Windows;
- `endpoints,wlan` - отображает беспроводные конечные точки;
- `expert` - отображает экспертную информацию (разговоры, ошибки и так далее) из захвата.

Аргумент `-z` предоставляет множество полезных параметров. Чтобы рассмотреть здесь их все, потребовалось бы слишком много страниц, но если вы собираетесь часто пользоваться TShark, стоит уделить время официальной документации и изучить все доступные возможности. Документация находится по адресу wireshark.org.

Сравнение TShark и tcpdump

Оба рассмотренных в этой главе приложения командной строки для анализа пакетов хорошо подходят для своих задач, и каждое из них позволит с той или иной затратой усилий выполнить необходимую работу. Стоит выделить несколько различий, чтобы вы могли выбрать лучший инструмент для конкретной задачи:

Операционная система. tcpdump доступен только в операционных системах семейства Unix, тогда как TShark работает в Windows и Unix-подобных системах.

Поддержка протоколов. Оба инструмента поддерживают распространённые протоколы уровней 3 и 4, но поддержка протоколов уровня 7 в tcpdump ограничена. TShark предоставляет широкую поддержку протоколов уровня 7 благодаря доступу к диссекторам протоколов Wireshark.

Возможности анализа. Для получения осмысленных результатов оба инструмента в значительной мере полагаются на анализ, выполняемый человеком, но TShark также предоставляет развитый набор аналитических и статистических функций, подобных функциям Wireshark. Они способны помочь в анализе, когда графический интерфейс недоступен.

Выбор приложения обычно в конечном счёте определяется доступностью инструмента и личными предпочтениями. К счастью, инструменты достаточно похожи, поэтому изучение одного неизбежно научит вас чему-то и о другом, сделает вас более разносторонним специалистом и расширит ваш набор инструментов.

Глава 7. Протоколы сетевого уровня

Независимо от того, устраняете ли вы проблемы с задержками, выявляете неправильно работающие приложения или локализуете угрозы безопасности, отыскивая аномальный трафик, сначала необходимо понять нормальный трафик. В следующих нескольких главах вы узнаете, как нормальный сетевой трафик работает на уровне пакетов, пока мы будем подниматься от самого низа модели OSI до её вершины. Каждому разделу о протоколе соответствует как минимум один файл захвата, который можно скачать и исследовать самостоятельно.

В этой главе мы сосредоточимся на протоколах сетевого уровня, составляющих рабочую основу сетевого обмена данными: ARP, IPv4, IPv6, ICMP и ICMPv6.

Следующие три главы о сетевых протоколах, пожалуй, самые важные в книге. Пропустить это обсуждение - всё равно что готовить праздничный ужин, не разогрев духовку. Даже если вы уже хорошо понимаете работу каждого протокола, хотя бы бегло прочитайте эти главы, чтобы повторить структуру пакетов каждого из них.

Протокол разрешения адресов (ARP)

Для сетевого обмена данными используются как логические, так и физические адреса. Логические адреса обеспечивают обмен между несколькими сетями и устройствами, которые не соединены непосредственно. Физические адреса обеспечивают обмен в одном сетевом сегменте между устройствами, напрямую соединёнными друг с другом коммутатором. В большинстве случаев для обмена данными эти два вида адресации должны работать совместно.

Представьте, что вы хотите связаться с устройством в своей сети. Это может быть какой-либо сервер или просто другая рабочая станция, с которой нужно обменяться файлами. Приложению, инициирующему обмен, уже известен IP-адрес удалённого узла (из DNS, рассматриваемой в главе 9), поэтому у системы должны быть все данные, необходимые для построения информации уровней 3-7 передаваемого пакета. Теперь ей недостаёт лишь информации канального уровня 2, содержащей MAC-адрес целевого узла.

MAC-адреса необходимы потому, что соединяющий сетевые устройства коммутатор использует таблицу ассоциативной памяти (Content Addressable Memory, CAM), в которой перечислены MAC-адреса всех устройств, подключённых к каждому его порту. Получив трафик, предназначенный определённому MAC-адресу, коммутатор по этой таблице определяет порт, через который следует его отправить. Если MAC-адрес назначения неизвестен, передающее устройство сначала проверяет наличие адреса в своём кеше; если его там нет, адрес необходимо разрешить посредством дополнительного обмена данными в сети.

Процесс, посредством которого сети TCP/IP (с IPv4) разрешают IP-адрес в MAC-адрес, называется протоколом разрешения адресов (Address Resolution Protocol, ARP) и определён в RFC 826. Для разрешения ARP нужны всего два пакета: запрос ARP и ответ ARP (см. рис. 7-1).

> **Примечание.** RFC, или Request for Comments, - техническая публикация Инженерного совета Интернета (Internet Engineering Task Force, IETF) и Общества Интернета (Internet Society, ISOC), служащая механизмом определения стандартов реализации протоколов. Документы RFC можно искать на главной странице редактора RFC: rfc-editor.org.

Передающий компьютер рассылает запрос ARP, который, по сути, говорит: «Всем привет. Мой IP-адрес - 192.168.0.101, а мой MAC-адрес - f2:f2:f2:f2:f2:f2. Мне нужно отправить данные тому, у кого IP-адрес 192.168.0.1, но его аппаратный адрес мне неизвестен. Пусть тот, кому принадлежит этот IP-адрес, сообщит свой MAC-адрес».

Этот пакет широковещательно передаётся каждому устройству в сетевом сегменте. Любое устройство, которому не принадлежит данный IP-адрес, просто отбрасывает пакет. Устройство с этим адресом отправляет ответ ARP примерно такого содержания: «Передающее устройство, тебе нужен я: мой IP-адрес - 192.168.0.1. Мой MAC-адрес - 02:f2:02:f2:02:f2».

После завершения разрешения передающее устройство обновляет свой кеш, добавляя сопоставление MAC- и IP-адресов принимающего устройства, и может начать отправку данных.

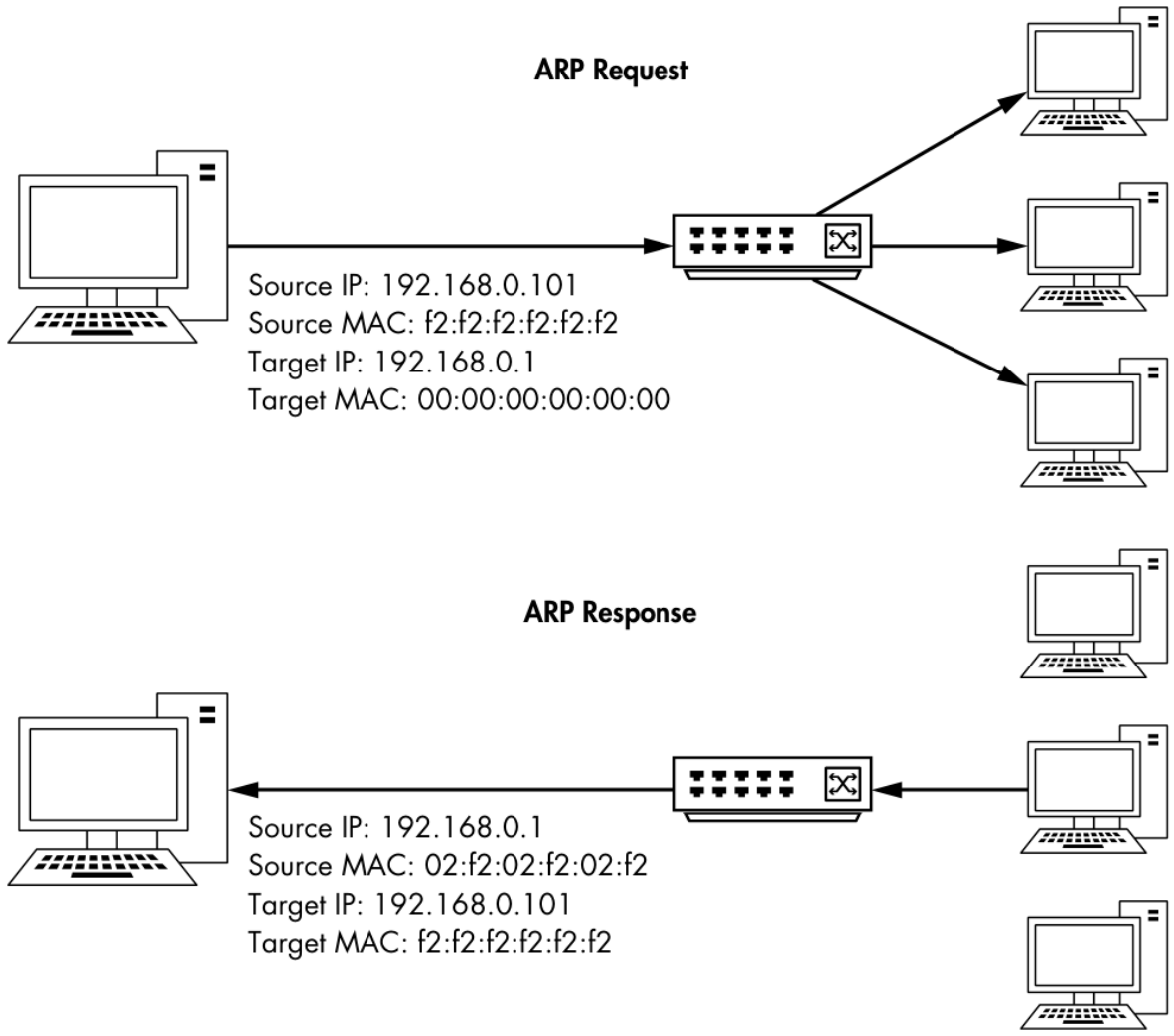


Рисунок 7-1. Процесс разрешения ARP

> **Примечание.** Таблицу ARP узла Windows можно просмотреть, введя `arp -a` в командной строке.

Наблюдение за этим процессом в действии поможет понять его работу. Но прежде чем переходить к примерам, рассмотрим заголовок пакета ARP.

Структура пакета ARP

Как показано на рис. 7-2, заголовок ARP содержит следующие поля:

Тип оборудования (Hardware Type). Тип уровня 2 - в большинстве случаев Ethernet (тип 1).

Тип протокола (Protocol Type). Протокол верхнего уровня, для которого используется запрос ARP.

Длина аппаратного адреса (Hardware Address Length). Длина используемого аппаратного адреса в октетах/байтах (6 для Ethernet).

Длина адреса протокола (Protocol Address Length). Длина логического адреса указанного типа протокола в октетах/байтах.

Операция (Operation). Функция пакета ARP: 1 для запроса или 2 для ответа.

Address Resolution Protocol (ARP)					
Offsets	Octet	0	1	3	4
Octet	Bit	0–7	8–15	0–7	8–15
0	0	Hardware Type		Protocol Type	
4	32	Hardware Address Length	Protocol Address Length	Operation	
8	64	Sender Hardware Address			
12	96	Sender Hardware Address		Sender Protocol Address	
16	128	Sender Protocol Address		Target Hardware Address	
20	160	Target Hardware Address			
24+	192+	Target Protocol Address			

Рисунок 7-2. Структура пакета ARP

Аппаратный адрес отправителя (Sender Hardware Address). Аппаратный адрес отправителя.

Адрес протокола отправителя (Sender Protocol Address). Адрес протокола верхнего уровня отправителя.

Аппаратный адрес цели (Target Hardware Address). Аппаратный адрес предполагаемого получателя (в запросах ARP все разряды равны нулю).

Адрес протокола цели (Target Protocol Address). Адрес протокола верхнего уровня предполагаемого получателя.

Теперь откройте файл `arp_resolution.pcapng`, чтобы увидеть процесс разрешения в действии. Рассматривая его, мы сосредоточимся на каждом пакете по отдельности.

Пакет 1: запрос ARP

Первый пакет представляет собой запрос ARP, показанный на рис. 7-3. Убедиться, что это действительно широковещательный пакет, можно по заголовку Ethernet на панели подробностей пакета Wireshark. Адрес назначения пакета - `ff:ff:ff:ff:ff:ff` (метка 1). Это широковещательный адрес Ethernet, и всё отправленное на него будет передано всем устройствам текущего сетевого сегмента. Адрес источника этого пакета в заголовке Ethernet указан как наш MAC-адрес (метка 2).

По этой структуре можно определить, что перед нами действительно запрос ARP в сети Ethernet, использующей IPv4. Указаны IP-адрес (`192.168.0.114`) и MAC-адрес (`00:16:ce:6e:8b:24`) отправителя (метка 3), а также IP-адрес цели (`192.168.0.1`) (метка 5). MAC-адрес цели - именно те сведения, которые мы пытаемся получить, - неизвестен, поэтому в поле целевого MAC-адреса указано `00:00:00:00:00:00` (метка 4).

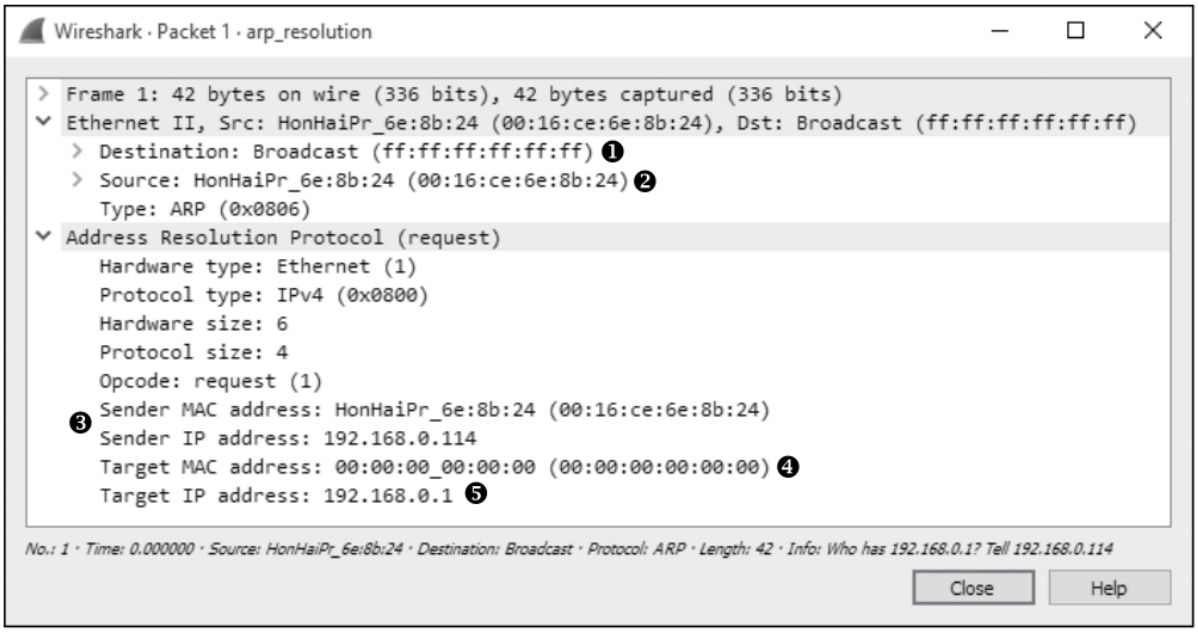


Рисунок 7-3. Пакет запроса ARP

Пакет 2: ответ ARP

В ответе на первоначальный запрос (см. рис. 7-4) адресом назначения в заголовке Ethernet теперь служит исходный MAC-адрес из первого пакета. Заголовок ARP похож на заголовок запроса ARP, но есть несколько изменений:

• Код операции пакета теперь равен `0x0002` (метка 1), что указывает на ответ, а не запрос.
• Адресные сведения поменялись местами: MAC- и IP-адреса отправителя теперь являются целевыми MAC- и IP-адресами (метка 3).
• Самое важное - теперь присутствуют все сведения, то есть мы получили MAC-адрес (`00:13:46:0b:22:ba`) (метка 2) узла с адресом `192.168.0.1`.

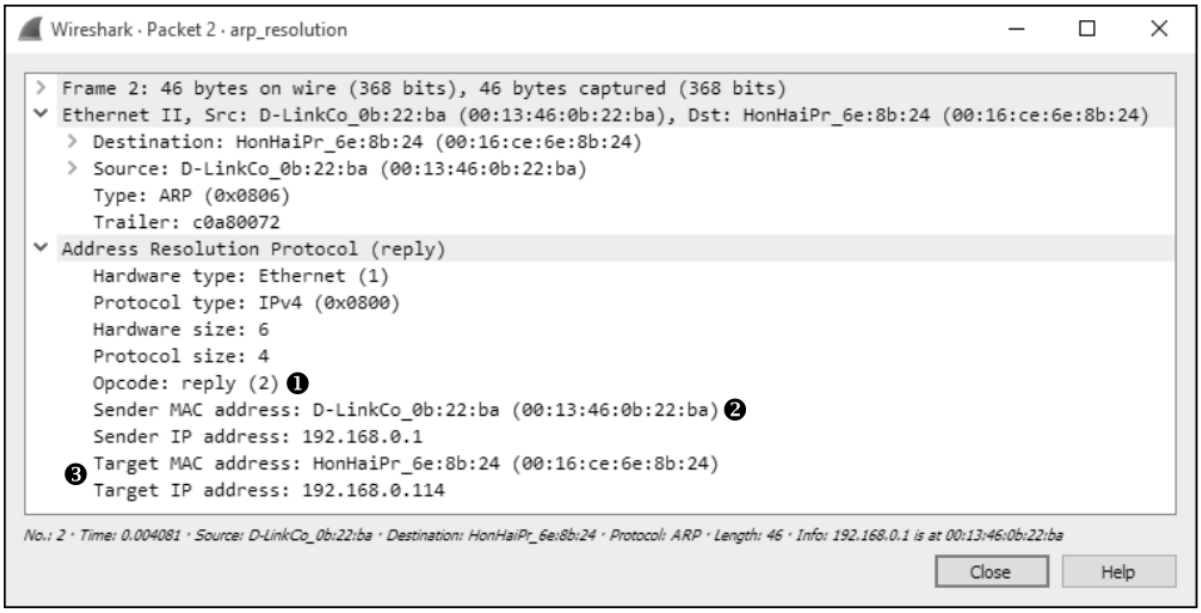


Рисунок 7-4. Пакет ответа ARP

Незапрошенный ARP

Там, откуда я родом, когда что-либо делают «без всякой необходимости» (gratuitously), это слово обычно несёт отрицательный оттенок. Однако незапрошенный ARP (gratuitous ARP) - полезная вещь.

Во многих случаях IP-адрес устройства может измениться. Тогда сохранённые в кешах сетевых узлов сопоставления IP- и MAC-адресов станут недействительными. Чтобы это не вызвало ошибок обмена данными, в сеть передаётся незапрошенный пакет ARP, заставляющий каждое получившее его устройство обновить кеш новым сопоставлением IP- и MAC-адресов (см. рис. 7-5).

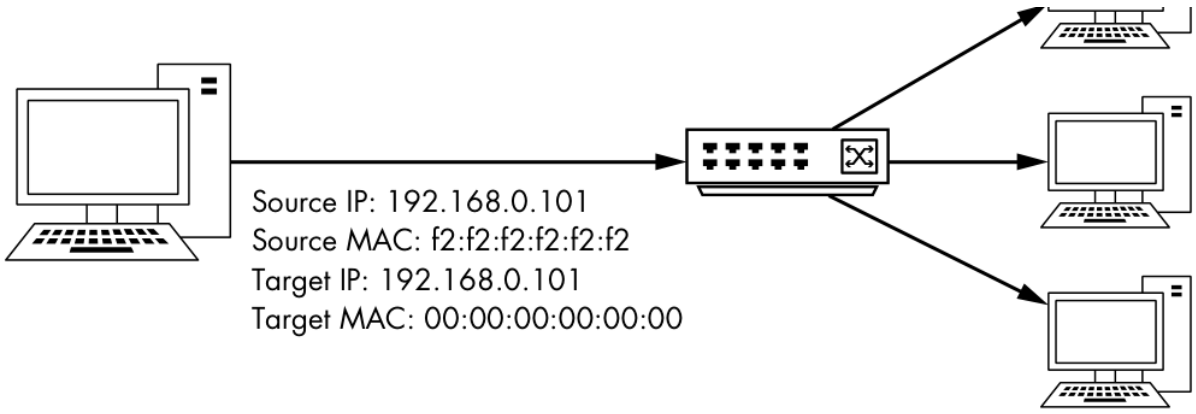


Рисунок 7-5. Процесс незапрошенного ARP

Незапрошенный пакет ARP может появиться в нескольких сценариях. Один из самых распространённых - изменение IP-адреса. Откройте файл захвата `arp_gratuitous.pcapng`, чтобы увидеть это в действии. В файле содержится только один пакет (см. рис. 7-6), потому что для незапрошенного ARP больше ничего не требуется.

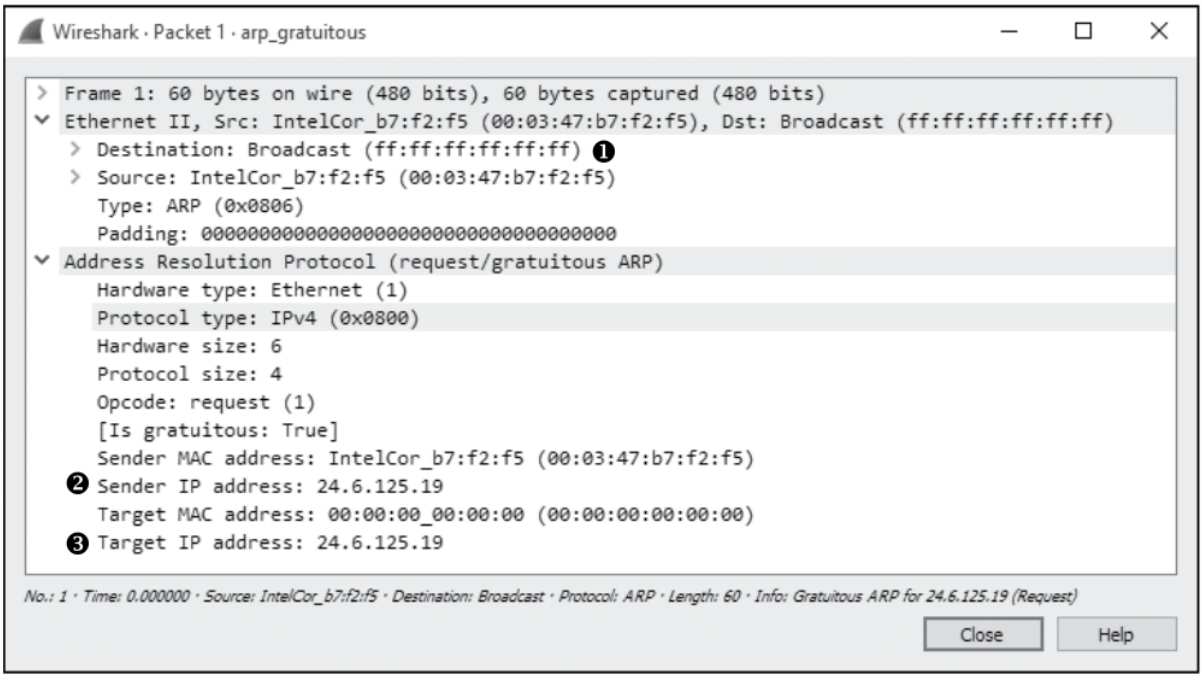


Рисунок 7-6. Незапрошенный пакет ARP

Из заголовка Ethernet видно, что пакет передаётся широковещательно, чтобы его получили все узлы сети (метка 1). Заголовок ARP выглядит как запрос ARP, за исключением того, что IP-адрес отправителя (метка 2) совпадает с целевым IP-адресом (метка 3). Получив этот пакет, другие сетевые узлы обновят свои таблицы ARP новым сопоставлением IP- и MAC-адресов. Поскольку этот пакет ARP не запрашивался, но приводит к обновлению клиентом своего кеша ARP, он считается незапрошенным.

Незапрошенные пакеты ARP встречаются в нескольких ситуациях. Как уже говорилось, такой пакет формируется при изменении IP-адреса устройства. Кроме того, некоторые операционные системы выполняют незапрошенный ARP при запуске. Наконец, отдельные системы используют незапрошенные пакеты ARP для поддержки балансировки нагрузки.

Протокол Интернета (IP)

Главное назначение протоколов уровня 3 модели OSI - обеспечивать обмен данными между сетями. Как вы только что увидели, MAC-адреса используются для обмена в одной сети на уровне 2. Подобным образом уровень 3 отвечает за адреса, используемые при межсетевом обмене. Это способны делать несколько протоколов, но наиболее распространён протокол Интернета (Internet Protocol, IP), у которого сейчас используются две версии - IP версии 4 и IP версии 6. Начнём с IP версии 4 (IPv4), определённого в RFC 791.

Протокол Интернета версии 4 (IPv4)

Чтобы понять функции IPv4, необходимо знать, как трафик проходит между сетями. IPv4 - рабочая основа процесса обмена данными; в конечном счёте именно он отвечает за перенос данных между устройствами независимо от местонахождения конечных точек обмена.

Простая сеть, в которой все устройства соединены концентраторами или коммутаторами, называется локальной сетью (local area network, LAN). Чтобы соединить две LAN, можно

использовать маршрутизатор. Сложные сети могут состоять из тысяч LAN, соединённых тысячами маршрутизаторов по всему миру. Сам Интернет представляет собой совокупность миллионов LAN и маршрутизаторов.

Адреса IPv4

Адреса IPv4 - это назначаемые 32-разрядные числа, служащие для уникальной идентификации подключённых к сети устройств. Ожидать от человека запоминания 32-значной последовательности единиц и нулей было бы чрезмерно, поэтому IP-адреса записываются в точечно-четверичной (или точечно-десятичной) нотации.

В точечно-четверичной нотации каждая из четырёх групп единиц и нулей, составляющих IP-адрес, преобразуется в десятичную систему и представляется числом от 0 до 255 в формате A.B.C.D (см. рис. 7-7). Рассмотрим, например, IP-адрес `11000000 10101000 00000000 00000001`. Очевидно, запоминать или записывать такое значение неудобно. К счастью, в точечно-четверичной нотации его можно представить как `192.168.0.1`.

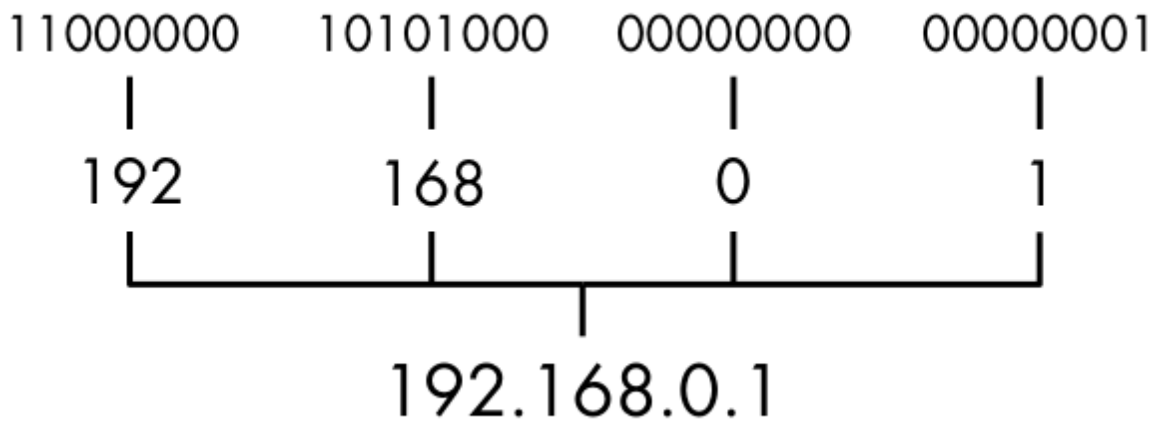


Рисунок 7-7. Точечно-четверичная нотация адреса IPv4

IP-адрес состоит из двух частей: сетевой части и части узла. Сетевая часть определяет LAN, к которой подключено устройство, а часть узла определяет само устройство в этой сети. Граница между сетевой частью и частью узла в IP-адресе не всегда одинакова. Она сообщается другим набором адресных данных, который называется маской сети (network mask, netmask), а иногда - маской подсети.

> **Примечание.** В этой книге под IP-адресом всегда подразумевается адрес IPv4. Далее в этой главе рассматривается IP версии 6, использующий другой набор правил адресации. Когда речь будет идти об адресе IPv6, это будет указано явно.

Маска сети определяет, какая часть IP-адреса относится к сети, а какая - к узлу. Маска сети также имеет длину 32 разряда; каждый установленный в 1 разряд отмечает часть IP-адреса, отведённую сетевой части. Остальные разряды устанавливаются в 0 и обозначают часть узла.

Например, рассмотрим IP-адрес `10.10.1.22`, который в двоичном виде представлен как `00001010 00001010 00000001 00010110`. Чтобы определить назначение каждой части IP-адреса, применим маску сети. В данном случае она равна `11111111 11111111 00000000 00000000`. Это означает, что первая половина IP-адреса (`10.10`, или `00001010 00001010`) отведена сетевой части, а последняя половина (`.1.22`, или `00000001 00010110`) определяет отдельный узел этой сети, как показано на

рис. 7-8.

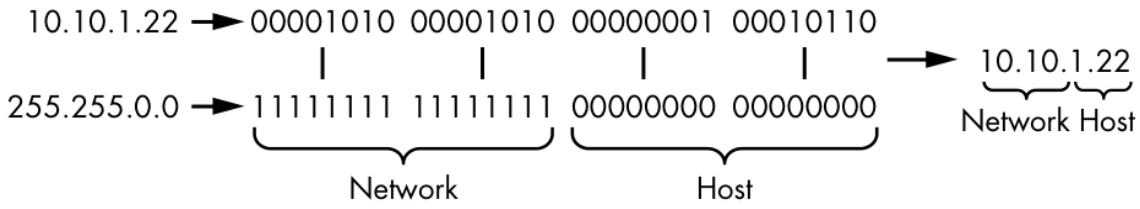


Рисунок 7-8. Маска сети определяет распределение разрядов IP-адреса

Как показано на рис. 7-8, маски сети также можно записывать в точечно-четверичной нотации. Например, маска `11111111 11111111 00000000 00000000` записывается как `255.255.0.0`.

IP-адреса и маски сети часто записываются в нотации бесклассовой междоменной маршрутизации (Classless Inter-Domain Routing, CIDR). В этой форме IP-адрес записывается полностью, после него ставится косая черта (`/`) и указывается число разрядов, представляющих сетевую часть IP-адреса. Например, IP-адрес `10.10.1.22` с маской сети `255.255.0.0` в нотации CIDR записывается как `10.10.1.22/16`.

Структура пакета IPv4

IP-адреса источника и назначения - важнейшие компоненты заголовка пакета IPv4, но содержащиеся в пакете сведения IP ими не ограничиваются. По сравнению с только что рассмотренным пакетом ARP заголовок IP довольно сложен: он предоставляет множество дополнительных функций, помогающих IP выполнять свою работу.

Как показано на рис. 7-9, заголовок IPv4 содержит следующие поля:

Версия (Version). Используемая версия IP (для IPv4 это всегда 4).

Длина заголовка (Header Length). Длина заголовка IP.

Тип обслуживания (Type of Service). Флаг приоритета и флаг типа обслуживания, которые маршрутизаторы применяют для определения приоритета трафика.

Общая длина (Total Length). Длина заголовка IP и включённых в пакет данных.

Идентификатор (Identification). Уникальный идентификационный номер для определения пакета или последовательности фрагментированных пакетов.

Флаги (Flags). Служат для определения того, входит ли пакет в последовательность фрагментированных пакетов.

Смещение фрагмента (Fragment Offset). Если пакет является фрагментом, значение этого поля используется для повторной сборки пакетов в правильном порядке.

Время жизни (Time to Live). Определяет время жизни пакета, измеряемое переходами через маршрутизаторы или секундами.

Протокол (Protocol). Определяет заголовок транспортного уровня, инкапсулирующий заголовок IPv4.

Контрольная сумма заголовка (Header Checksum). Механизм обнаружения ошибок, позволяющий проверить, что содержимое заголовка IP не повреждено и не искажено.

IP-адрес источника (Source IP Address). IP-адрес узла, отправившего пакет.

IP-адрес назначения (Destination IP Address). IP-адрес назначения пакета.

Параметры (Options). Зарезервировано для дополнительных параметров IP, в том числе исходной маршрутизации и меток времени.

Данные (Data). Фактические данные, передаваемые посредством IP.

Internet Protocol Version 4 (IPv4)							
Offsets	Octet	0		1	2		3
Octet	Bit	0–3	4–7	8–15	16–18	19–23	24–31
0	0	Version	Header Length	Type of Service	Total Length		
4	32	Identification			Flags	Fragment Offset	
8	64	Time to Live		Protocol	Header Checksum		
12	96	Source IP Address					
16	128	Destination IP Address					
20	160	Options					
24+	192+	Data					

Рисунок 7-9. Структура пакета IPv4

Время жизни

Значение времени жизни (Time to Live, TTL) определяет для IPv4 промежуток времени или максимальное число маршрутизаторов, через которые пакет может пройти до отбрасывания. TTL задаётся при создании пакета и обычно уменьшается на 1 при каждой пересылке пакета маршрутизатором. Например, если TTL пакета равен 2, первый достигнутый им маршрутизатор уменьшит TTL до 1 и перешлёт пакет второму маршрутизатору. Тот уменьшит TTL до нуля, и, если конечное назначение пакета не находится в этой сети, пакет будет отброшен (см. рис. 7-10).

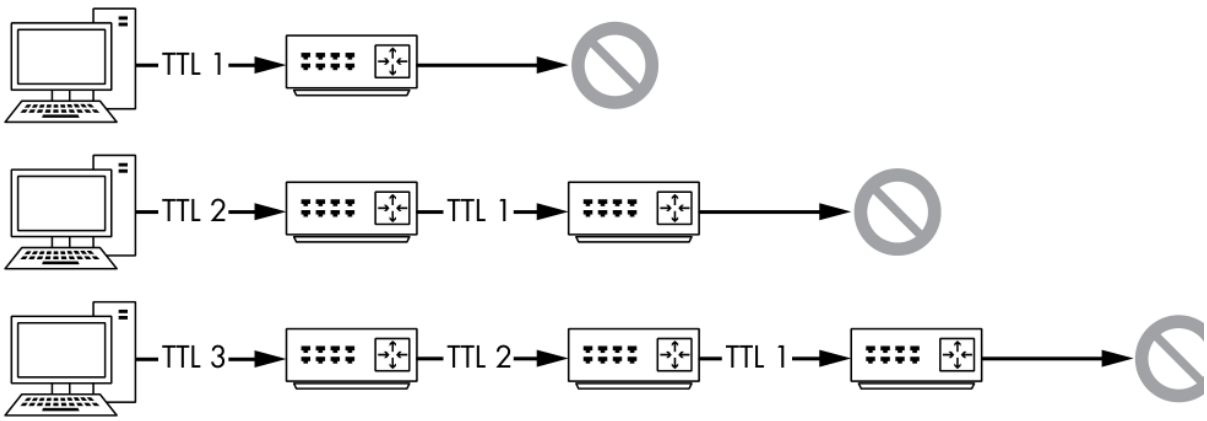


Рисунок 7-10. TTL пакета уменьшается при каждом прохождении маршрутизатора

Почему значение TTL важно? Обычно время жизни пакета интересует нас лишь с точки зрения времени его прохождения от источника к назначению. Однако представьте пакет, которому нужно достичь узла через Интернет, пройдя десятки маршрутизаторов. В какой-либо точке маршрута он может встретить неправильно настроенный маршрутизатор и потерять путь к конечному назначению. В такой ситуации маршрутизатор способен выполнить несколько действий, одно из которых может привести к бесконечной пересылке пакета по замкнутому кругу в сети.

Бесконечный цикл способен вызвать самые разные проблемы, но обычно приводит к аварийному завершению программы или всей операционной системы. Теоретически то же может произойти с пакетами в сети. Они будут бесконечно циркулировать между маршрутизаторами. По мере роста числа зацикленных пакетов доступная пропускная способность сети будет исчерпываться, пока не возникнет состояние отказа в обслуживании. Для предотвращения этого и был создан TTL.

Рассмотрим пример в Wireshark. Файл `ip_ttl_source.pcapng` содержит два пакета ICMP. Протокол ICMP (рассматриваемый далее в этой главе) использует IP для доставки пакетов, что можно увидеть, развернув раздел заголовка IP на панели подробностей пакета (см. рис. 7-11).

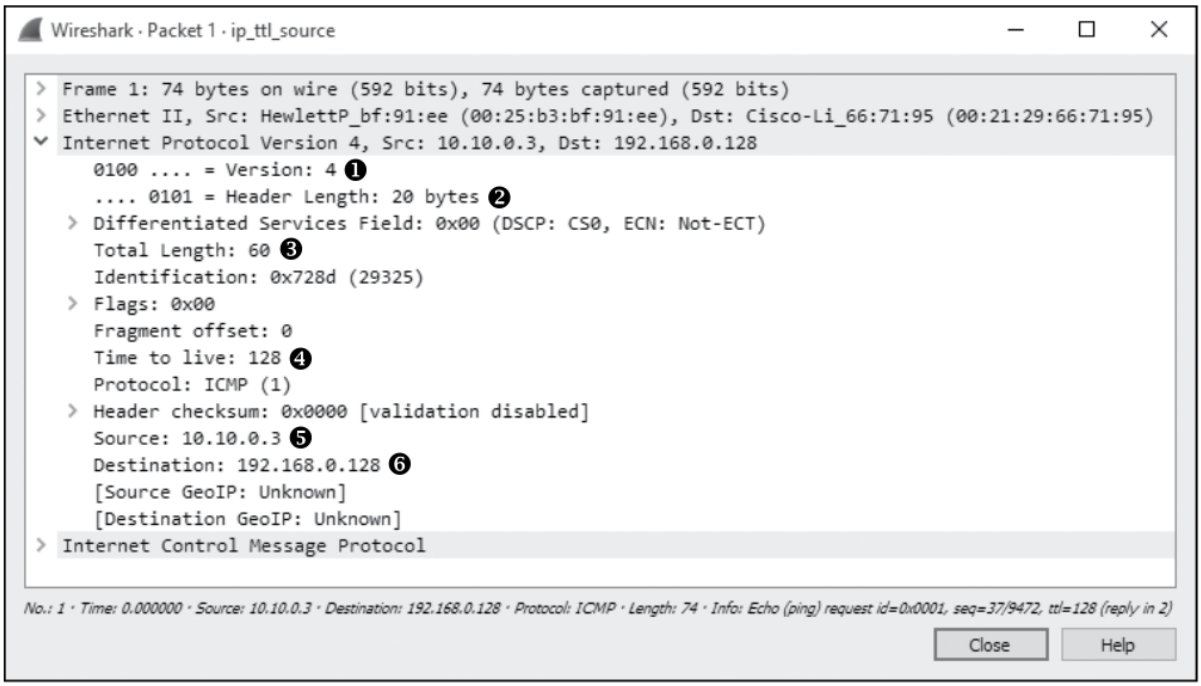


Рисунок 7-11. Заголовок IP исходного пакета

Видно, что используется IP версии 4 (метка 1), длина заголовка IP составляет 20 байт (метка 2), общая длина заголовка и полезной нагрузки - 60 байт (метка 3), а значение поля TTL - 128 (метка 4).

Основное назначение эхо-запроса ICMP - проверка обмена данными между устройствами. Данные отправляются с одного узла другому в виде запроса, а принимающий узел должен вернуть их в ответе. В данном файле одно устройство с адресом `10.10.0.3` (метка 5) отправляет запрос ICMP устройству с адресом `192.168.0.128` (метка 6). Этот исходный файл захвата был создан на узле-источнике `10.10.0.3`.

Теперь откройте файл `ip_ttl_dest.pcapng`. В нём данные захватывались на узле назначения `192.168.0.128`. Разверните заголовок IP первого пакета захвата и изучите значение TTL (см. рис. 7-12).

Вы сразу заметите, что TTL равен 127 (метка 1), то есть на 1 меньше первоначального значения 128. Даже не зная архитектуры сети, можно заключить, что эти устройства разделяет один маршрутизатор, прохождение которого уменьшило TTL на 1.

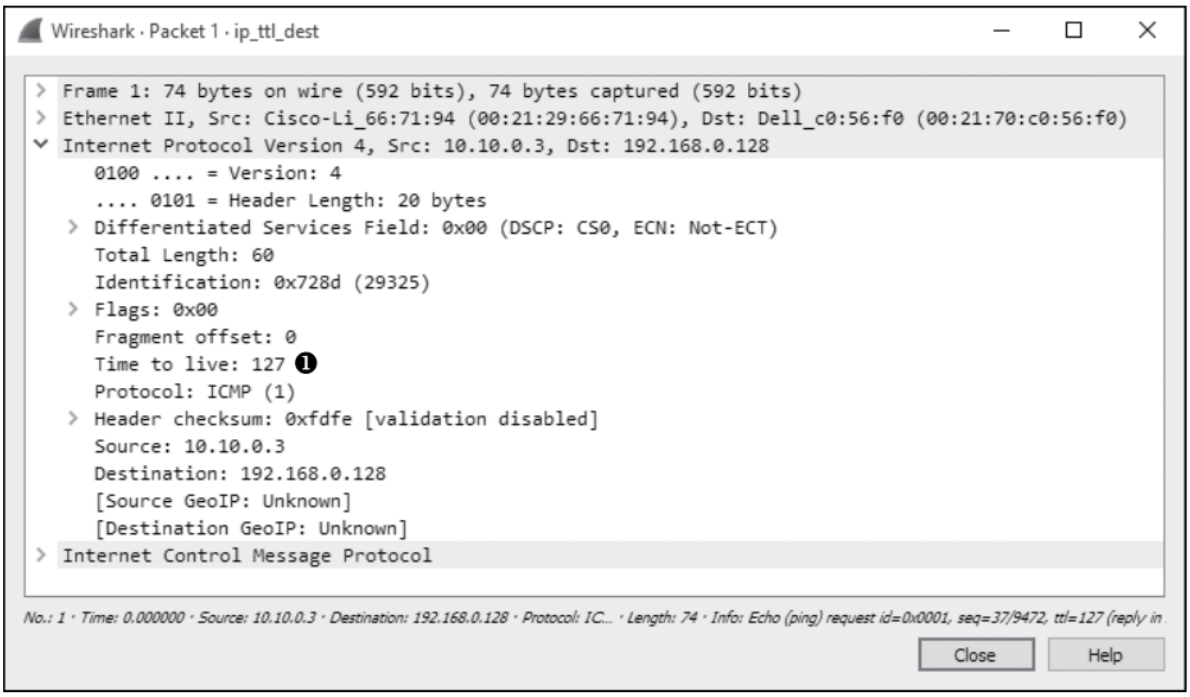


Рисунок 7-12. Заголовок IP показывает уменьшение TTL на 1

Фрагментация IP

Фрагментация пакетов - функция IP, обеспечивающая надёжную доставку данных через сети разных типов путём разделения потока данных на более мелкие фрагменты.

Фрагментация пакета зависит от размера максимального блока передачи (maximum transmission unit, MTU) используемого протокола канального уровня 2 и от конфигурации устройств, использующих этот протокол. В большинстве случаев протокол канального уровня 2 - Ethernet. Стандартный MTU Ethernet равен 1500, а значит, максимальный размер пакета, который можно передать по сети Ethernet, составляет 1500 байт (без учёта самого 14-байтового заголовка Ethernet).

> **Примечание.** Хотя существуют стандартные значения MTU, в большинстве случаев MTU устройства можно перенастроить вручную. MTU назначается отдельно каждому интерфейсу; его можно изменить в системах Windows и Linux, а также на интерфейсах управляемых маршрутизаторов.

Подготавливая пакет IP к передаче, устройство сравнивает размер данных пакета с MTU сетевого интерфейса, через который пакет будет отправлен, и определяет, нужна ли фрагментация. Если размер данных превышает MTU, пакет фрагментируется. Фрагментация пакета включает следующие действия:

1. Устройство разделяет данные на число пакетов, необходимое для успешной передачи.
2. В поле общей длины каждого заголовка IP записывается размер сегмента соответствующего фрагмента.
3. Во всех пакетах потока данных, кроме последнего, флаг «Есть ещё фрагменты» (More fragments) устанавливается в 1.
4. В заголовке IP фрагментов задаётся поле смещения фрагмента.
5. Пакеты передаются.

Файл `ip_frag_source.pcapng` был захвачен на компьютере с адресом `10.10.0.3`, который отправлял эхо-запрос устройству с адресом `192.168.0.128`. Обратите внимание: на панели списка пакетов в столбце Info перечислены два фрагментированных пакета IP, а следом - эхо-запрос ICMP.

Сначала изучим заголовок IP пакета 1 (см. рис. 7-13).

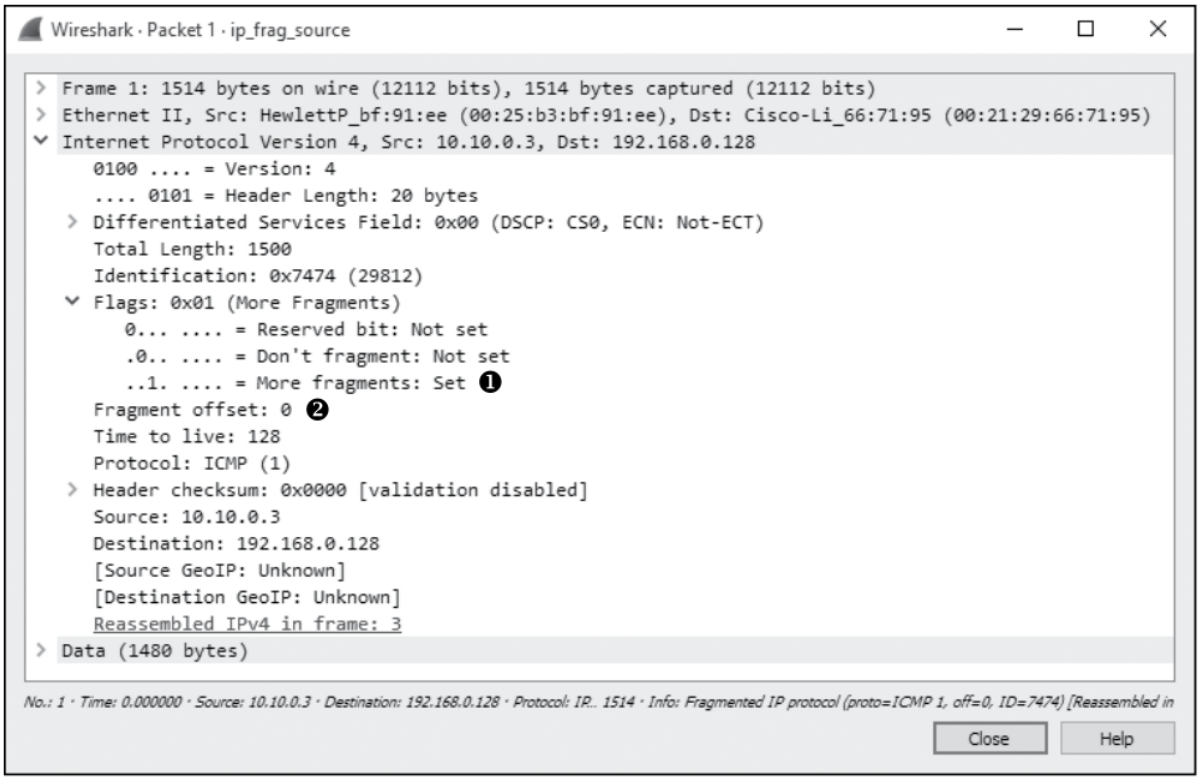


Рисунок 7-13. Значения More fragments и Fragment offset могут указывать на фрагментированный пакет

Определить, что пакет является частью фрагмента, можно по полям More fragments и Fragment offset. У пакетов-фрагментов либо положительное значение Fragment offset, либо установлен флаг More fragments. В первом пакете флаг More fragments установлен (метка 1), сообщая принимающему устройству, что следует ожидать ещё один пакет этой последовательности. Значение Fragment offset равно 0 (метка 2), а значит, данный пакет - первый в последовательности фрагментов.

В заголовке IP второго пакета (см. рис. 7-14) флаг More fragments также установлен (метка 1), но в этом случае значение Fragment offset равно 1480 (метка 2). Оно соответствует MTU 1500 байт за вычетом 20 байт заголовка IP.

В третьем пакете (см. рис. 7-15) флаг More fragments не установлен (метка 2), что отмечает его как последний фрагмент потока данных, а Fragment offset равен 2960 (метка 3), то есть результату `1480 + (1500 - 20)`. Все эти фрагменты можно распознать как части одной последовательности данных, поскольку значения поля Identification их заголовков IP совпадают (метка 1).

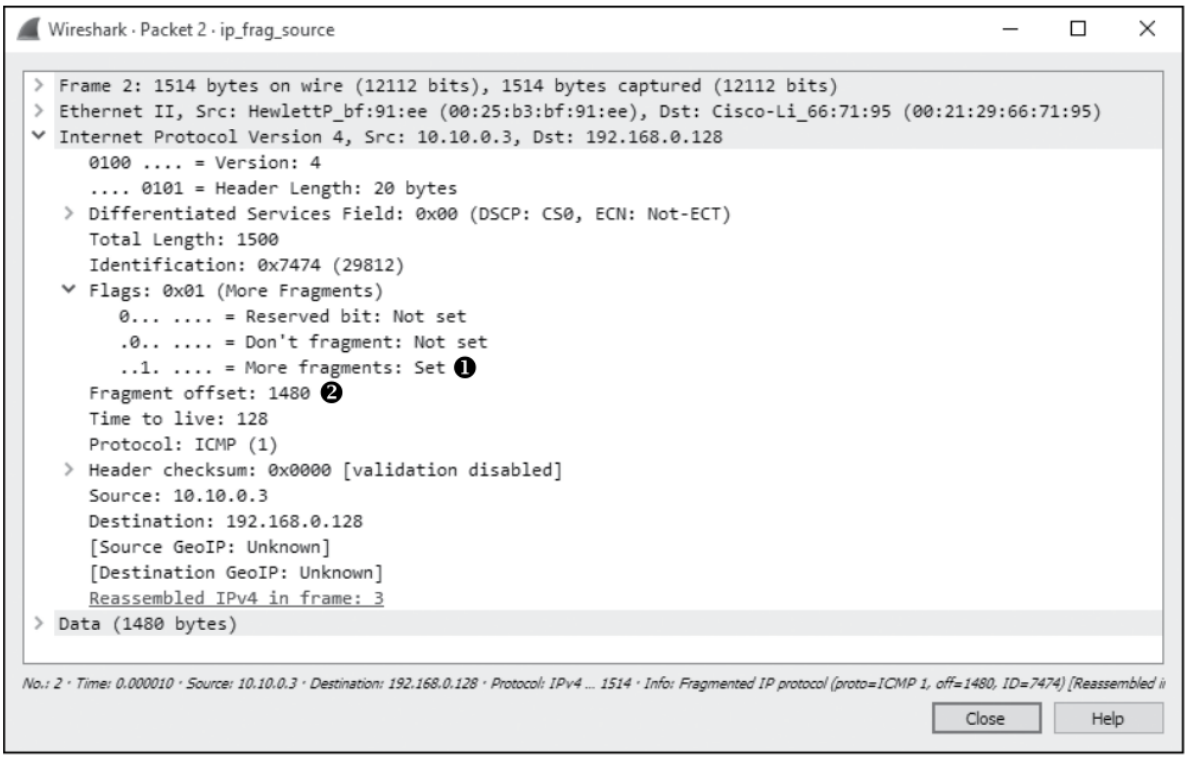


Рисунок 7-14. Значение Fragment offset увеличивается в зависимости от размера пакетов

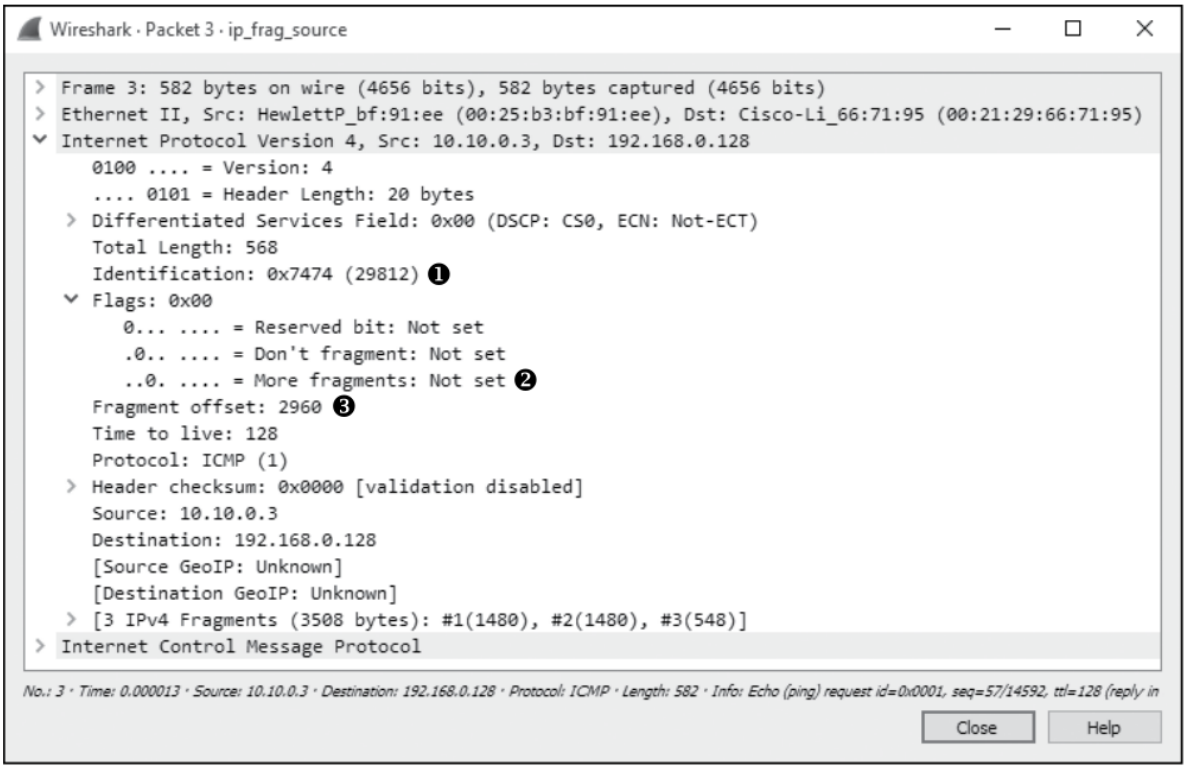


Рисунок 7-15. Неустановленный флаг More fragments указывает, что этот фрагмент последний

Хотя фрагментированные пакеты встречаются в сетях уже не так часто, как прежде, полезно понимать причины фрагментации: столкнувшись с ней, вы сможете диагностировать проблемы или заметить недостающие фрагменты.

Протокол Интернета версии 6 (IPv6)

Когда создавалась спецификация IPv4, никто не представлял, что в итоге появится столько подключённых к Интернету устройств, сколько существует сегодня. Максимальное адресное пространство IPv4 ограничивалось чуть менее чем 4,3 миллиарда адресов. Фактическое доступное пространство сокращается ещё сильнее, если вычесть диапазоны, зарезервированные для особых целей: тестирования, широковещательного трафика и внутренних адресов RFC 1918. Предпринималось несколько попыток отсрочить исчерпание адресов IPv4, но в конечном счёте устранить это ограничение можно было лишь разработкой новой версии спецификации IP.

Так появилась спецификация IPv6, первая версия которой была опубликована в 1998 году как RFC 2460. Эта версия предоставила несколько усовершенствований производительности, в том числе гораздо большее адресное пространство. В этом разделе мы рассмотрим структуру пакета IPv6 и обсудим, чем обмен данными IPv6 отличается от своего предшественника.

Адреса IPv6

Адреса IPv4 ограничены 32 разрядами, что даёт адресное пространство, измеряемое миллиардами. Длина адресов IPv6 составляет 128 разрядов, поэтому их адресное пространство измеряется ундециллионами (триллион триллионов триллионов). Впечатляющее расширение!

Поскольку адреса IPv6 имеют длину 128 разрядов, обращаться с ними в двоичной форме неудобно. Обычно адрес IPv6 записывается как восемь групп по 2 байта в шестнадцатеричной нотации, разделённых двоеточиями. Например, очень простой адрес IPv6 выглядит так:

```
1111:aaaa:2222:bbbb:3333:cccc:4444:dddd
```

Ваша первая мысль, вероятно, совпадёт с мыслью многих людей, привыкших запоминать адреса IPv4: адреса IPv6 практически невозможно выучить наизусть. Такова досадная плата за гораздо большее адресное пространство.

Одна особенность записи адресов IPv6, которая в некоторых случаях помогает, - возможность сворачивать группы нулей. Рассмотрим следующий адрес IPv6:

```
1111:0000:2222:0000:3333:4444:5555:6666
```

Содержащую нули группу можно полностью свернуть, чтобы она не отображалась:

```
1111::2222:0000:3333:4444:5555:6666
```

Однако свернуть можно лишь одну группу нулей, поэтому следующий адрес будет недопустимым:

```
1111::2222::3333:4444:5555:6666
```

Кроме того, из адресов IPv6 можно удалять ведущие нули. Рассмотрим пример, в котором нули находятся в начале четвёртой, пятой и шестой групп:

```
1111:0000:2222:0333:0044:0005:ffff:ffff
```

Этот адрес можно представить короче:

```
1111::2222:333:44:5:ffff:ffff
```

Пользоваться им всё ещё не так просто, как адресом IPv4, но всё же гораздо легче, чем более длинной записью.

В адресе IPv6 есть сетевая часть и часть узла, которые часто называют соответственно сетевым префиксом и идентификатором интерфейса. Распределение этих полей зависит от класса обмена данными IPv6. Трафик IPv6 делится на три класса: одноадресный, многоадресный и anycast. В большинстве случаев вы, вероятно, будете работать с локальным для канала одноадресным

трафиком, то есть обменом между двумя устройствами внутри сети. Формат локального для канала одноадресного адреса IPv6 показан на рис. 7-16.

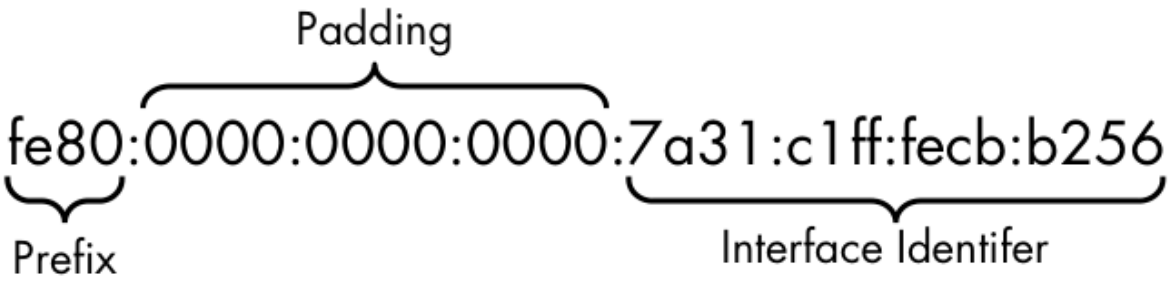


Рисунок 7-16. Части локального для канала одноадресного адреса IPv6

Локальные для канала адреса применяются, когда обмен предназначен другому устройству в той же сети. Такой адрес можно распознать по 10 старшим разрядам, установленным в `1111111010`, и следующим 54 разрядам, установленным в нули. Поэтому у локального для канала адреса первая половина имеет вид `fe80:0000:0000:0000`.

Вторая половина локального для канала адреса IPv6 - идентификатор интерфейса, уникально определяющий сетевой интерфейс конечного узла. В сетях Ethernet он может основываться на MAC-адресе интерфейса. Однако длина MAC-адреса составляет всего 48 разрядов. Чтобы заполнить всё 64-разрядное пространство, MAC-адрес делится пополам, а между его половинами в качестве заполнения вставляется значение `0xfffe`, создавая уникальный идентификатор. Наконец, седьмой разряд первого байта инвертируется. Это довольно сложно, но рассмотрим идентификатор интерфейса на рис. 7-17. Исходный MAC-адрес представленного этим идентификатором устройства был равен `78:31:c1:cb:b2:56`. В середину добавлены байты `0xfffe`, а инвертирование седьмого разряда первого байта заменило `8` на `a`.

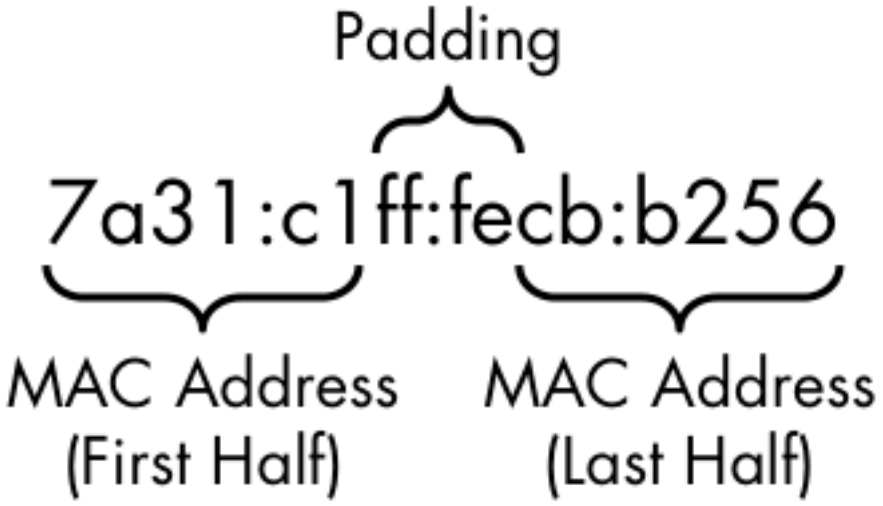


Рисунок 7-17. Идентификатор интерфейса использует MAC-адрес интерфейса и заполнение

Адреса IPv6, как и адреса IPv4, можно представлять в нотации CIDR. В следующем примере локальным для канала адресом представлено 64 разряда адресного пространства:

```
fe80:0000:0000:0000:/64
```

Состав адреса IPv6 меняется, когда он применяется для глобального одноадресного трафика, маршрутизируемого через общедоступный Интернет (см. рис. 7-18). Глобальный одноадресный адрес определяется первыми 3 разрядами, установленными в `001`, за которыми следует 45-разрядный глобальный префикс маршрутизации. Глобальный префикс маршрутизации, назначаемый организациям Администрацией адресного пространства Интернета (Internet Assigned Numbers Authority, IANA), служит для уникального определения пространства IP-адресов организации. Следующие 16 разрядов составляют идентификатор подсети, который можно использовать для иерархической адресации подобно части маски сети в адресе IPv4. Последние 64 разряда отводятся идентификатору интерфейса, как и в локальных для канала одноадресных адресах. Размеры префикса маршрутизации и идентификатора подсети могут различаться.

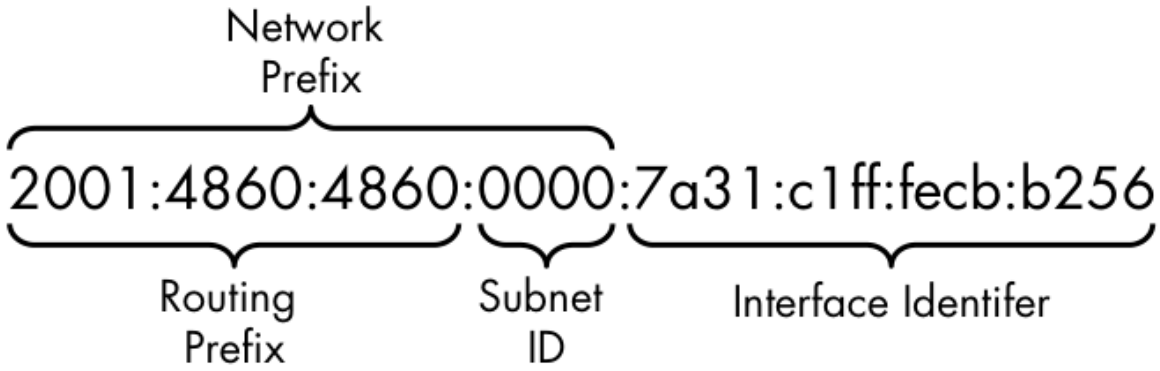


Рисунок 7-18. Части глобального одноадресного адреса IPv6

IPv6 обеспечивает намного большую эффективность, чем IPv4, при маршрутизации пакетов к назначению и рациональном использовании адресного пространства. Эта эффективность обусловлена более широким диапазоном доступных адресов, применением локальной для канала и глобальной адресации, а также уникальными идентификаторами узлов.

> **Примечание.** Человек легко отличает адрес IPv6 от IPv4 визуально, но многие программы этого не могут. Если нужно указать адрес IPv6, некоторые приложения, например браузеры или утилиты командной строки, требуют заключать адрес в квадратные скобки: `[1111::2222:333:44:5:ffff]`. Это требование не всегда хорошо документировано и стало источником разочарования для многих людей, изучающих IPv6.

Структура пакета IPv6

Структура заголовка IPv6 была расширена для поддержки новых возможностей, но при этом спроектирована так, чтобы разбирать её было проще. Вместо заголовка переменного размера с полем длины, которое нужно проверять при разборе, теперь используются заголовки фиксированного размера 40 байт. Дополнительные параметры предоставляются заголовками расширения. Преимущество состоит в том, что большинству маршрутизаторов для дальнейшей пересылки пакета достаточно обработать только 40-байтовый заголовок.

Как показано на рис. 7-19, заголовок IPv6 содержит следующие поля:

Версия (Version). Используемая версия IP (для IPv6 это всегда 6).

Класс трафика (Traffic Class). Служит для определения приоритета отдельных классов трафика.

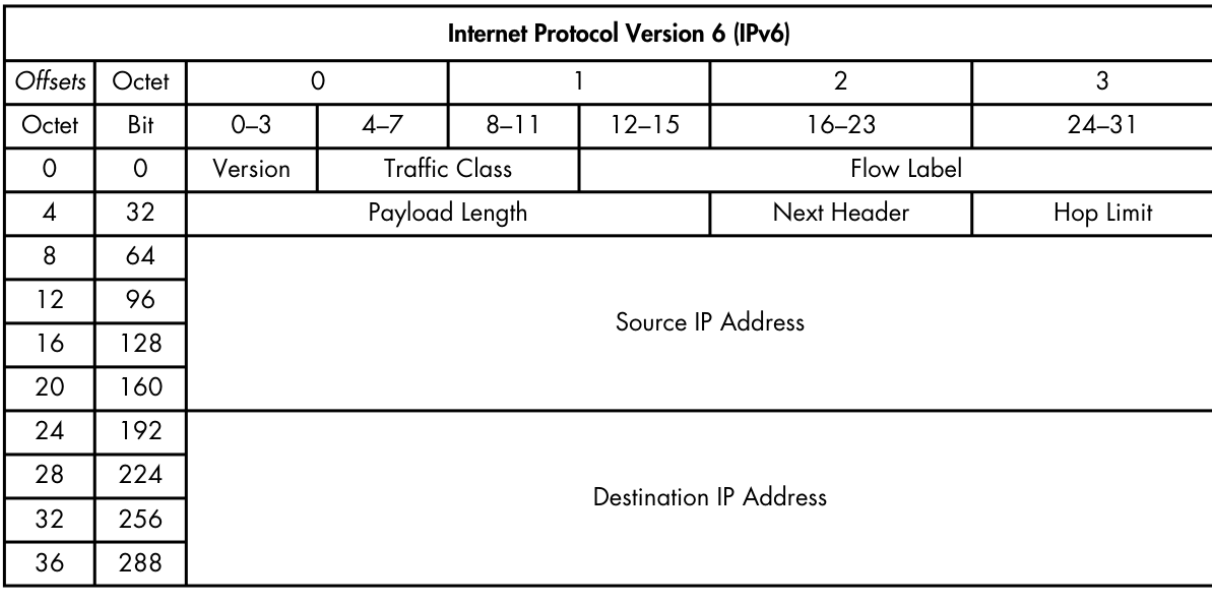


Рисунок 7-19. Структура пакета IPv6

Метка потока (Flow Label). Служит источнику для маркировки набора пакетов, относящихся к одному потоку. Обычно это поле применяется для управления качеством обслуживания (quality of service, QoS) и обеспечения прохождения пакетов одного потока по одному маршруту.

Длина полезной нагрузки (Payload Length). Длина полезной нагрузки данных, следующей за заголовком IPv6.

Следующий заголовок (Next Header). Определяет заголовок уровня 4, инкапсулирующий заголовок IPv6. Это поле заменяет поле Protocol в IPv4.

Предел переходов (Hop Limit). Определяет время жизни пакета, измеряемое переходами через маршрутизаторы. Это поле заменяет поле TTL в IPv4.

IP-адрес источника (Source IP Address). IP-адрес узла, отправившего пакет.

IP-адрес назначения (Destination IP Address). IP-адрес назначения пакета.

Сравним пакеты IPv4 и IPv6 и рассмотрим некоторые различия в файле `http_ip4and6.pcapng`. В этом захвате веб-сервер был настроен для ожидания подключений как IPv4, так и IPv6 на одном физическом узле. Один клиент, настроенный с адресами IPv4 и IPv6, по отдельности обращался к серверу по каждому адресу и загружал страницу `index.php` по HTTP при помощи приложения curl (рис. 7-20).

Открыв захват, вы легко определите принадлежность пакетов разговорам по адресам в столбцах Source и Destination области списка пакетов. Пакеты 1-10 представляют поток IPv4 (поток 0), а пакеты 11-20 - поток IPv6 (поток 1). Каждый из этих потоков можно отфильтровать из окна Conversations или вводом `tcp.stream == 0` либо `tcp.stream == 1` в строку фильтра.

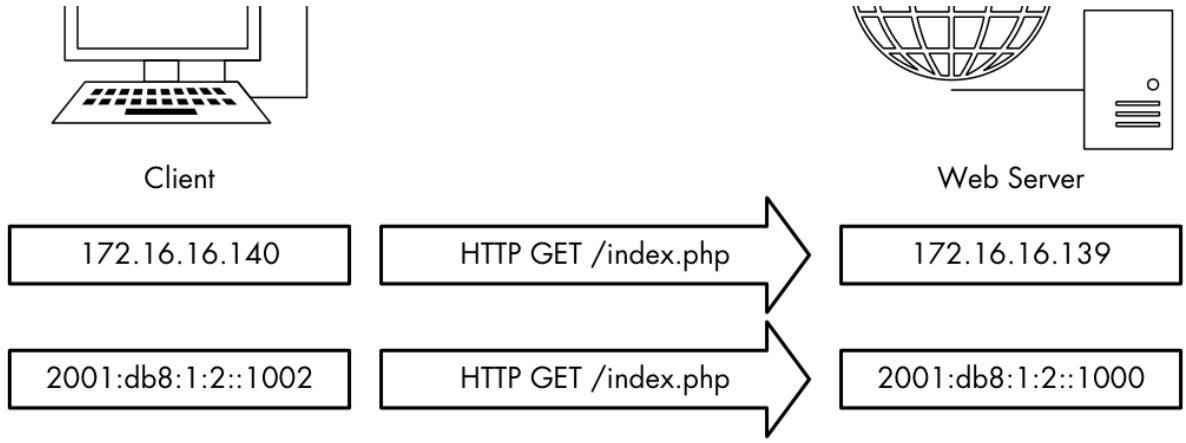


Рисунок 7-20. Соединения между одними и теми же физическими узлами посредством разных версий IP

HTTP - протокол, отвечающий за предоставление веб-страниц в Интернете, - подробно рассматривается в главе 8. В этом примере достаточно отметить, что предоставление веб-страниц выполняется одинаково независимо от того, какой сетевой протокол нижнего уровня используется. То же можно сказать о TCP, который также действует единообразно. Это наглядный пример инкапсуляции в действии. Хотя IPv4 и IPv6 работают по-разному, протоколы других уровней это не затрагивает.

На рис. 7-21 рядом показаны два пакета с одной функцией - пакеты 1 и 11. Оба являются пакетами TCP SYN, предназначенными для инициирования соединения клиента с сервером. Разделы Ethernet и TCP этих пакетов почти одинаковы. Однако разделы IP совершенно различны.

• Форматы адресов источника и назначения различаются (метки 6 и 10).
• Размер пакета IPv4 составляет 74 байта, а его общая длина - 60 байт (метка 1): в неё входят заголовок IPv4 и полезная нагрузка, а отдельно имеется 14-байтовый заголовок Ethernet. Размер пакета IPv6 составляет 96 байт, при этом полезная нагрузка IPv6 равна 40 байтам (метка 7), отдельно имеются 40-байтовый заголовок IPv6 и 14-байтовый заголовок Ethernet. Заголовок IPv6 имеет длину 40 байт - вдвое больше 20-байтового заголовка IPv4, чтобы вместить адреса большего размера.
• IPv4 определяет протокол в поле Protocol (метка 4), а IPv6 - в поле Next header, которое также может задавать заголовки расширения (метка 8).
• В IPv4 есть поле TTL (метка 3), тогда как в IPv6 ту же функцию выполняет поле Hop limit (метка 9).
• IPv4 включает контрольную сумму заголовка (метка 5), а IPv6 - нет.
• Пакет IPv4 не фрагментирован, но всё равно содержит значения соответствующих параметров (метка 2). Заголовок IPv6 этих сведений не содержит: если бы потребовалась фрагментация, она была бы реализована в заголовке расширения.

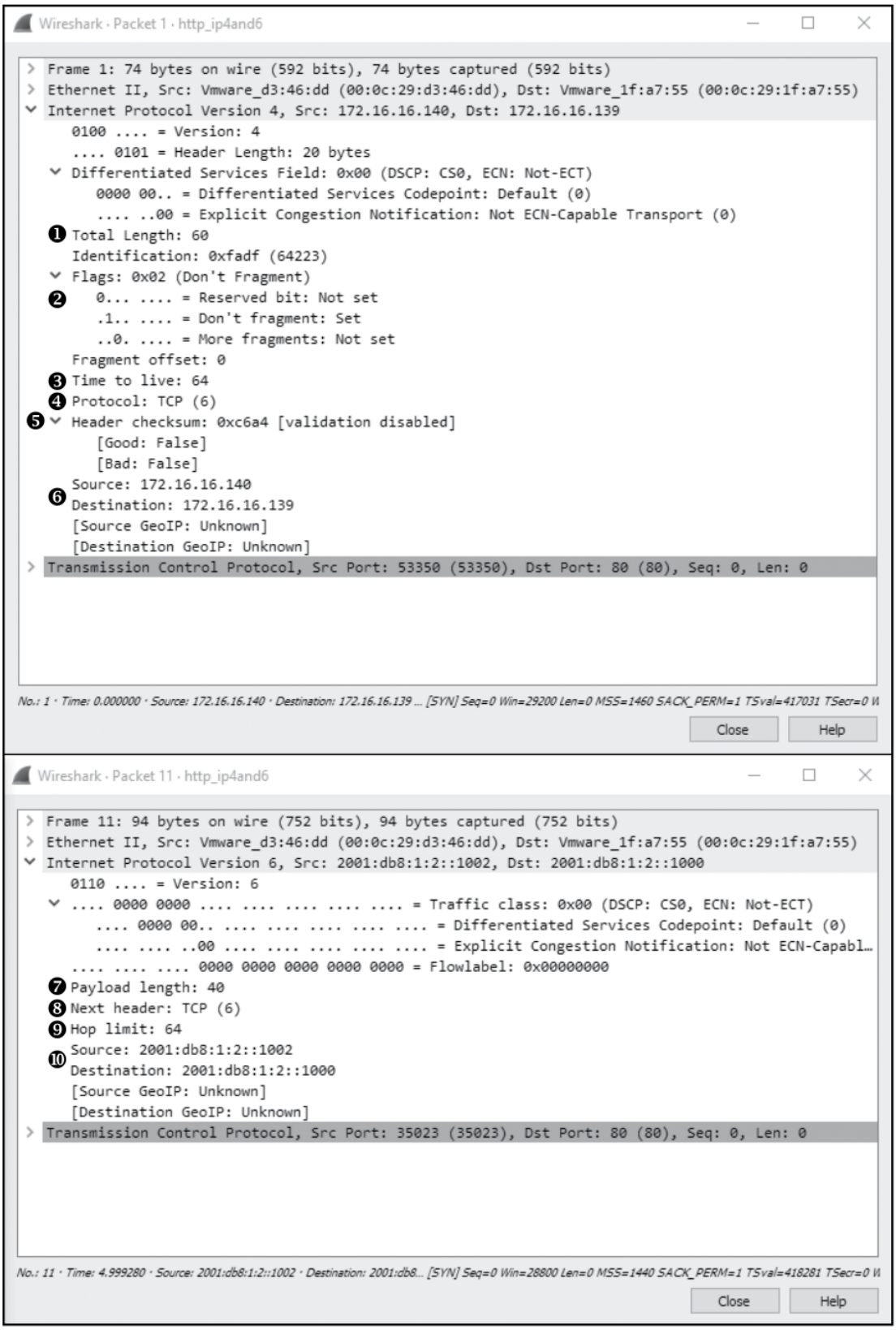


Рисунок 7-21. Сравнение пакетов IPv4 (вверху) и IPv6 (внизу), выполняющих одну функцию

Параллельное сравнение трафика IPv4 и IPv6 - отличный способ в полной мере понять различия в работе этих протоколов.

Обнаружение соседей и ARP

Когда ранее мы обсуждали разные классы трафика, я перечислил одноадресный, многоадресный и anycast-трафик, но не упомянул широковещательный. В IPv6 нет поддержки широковещательного трафика, поскольку широковещание считается неэффективным механизмом передачи. Раз широковещания нет, узлы не могут находить друг друга в сети посредством ARP. Как же устройства IPv6 обнаруживают друг друга?

Ответ даёт новая функция под названием «запрос соседей» (neighbor solicitation), входящая в протокол обнаружения соседей (Neighbor Discovery Protocol, NDP). Для выполнения основной работы она использует ICMPv6, который рассматривается в последнем разделе этой главы. Для решения этой задачи ICMPv6 применяет многоадресную рассылку - вид обмена данными, при котором поток данных получают и обрабатывают только подписанные на него узлы. Многоадресный трафик легко распознать, поскольку для него зарезервировано собственное пространство IP-адресов (`ff00::/8`).

Хотя процесс разрешения адресов опирается на другой протокол, он по-прежнему использует очень простой обмен «запрос - ответ». Например, представим, что узел с IPv6-адресом `2001:db8:1:2::1003` хочет связаться с другим узлом, имеющим адрес `2001:db8:1:2::1000`. Как и в IPv4, устройство-источник должно определить адрес канального уровня (MAC-адрес) узла, с которым оно хочет обмениваться данными, поскольку этот обмен происходит внутри сети. Процесс показан на рис. 7-22.

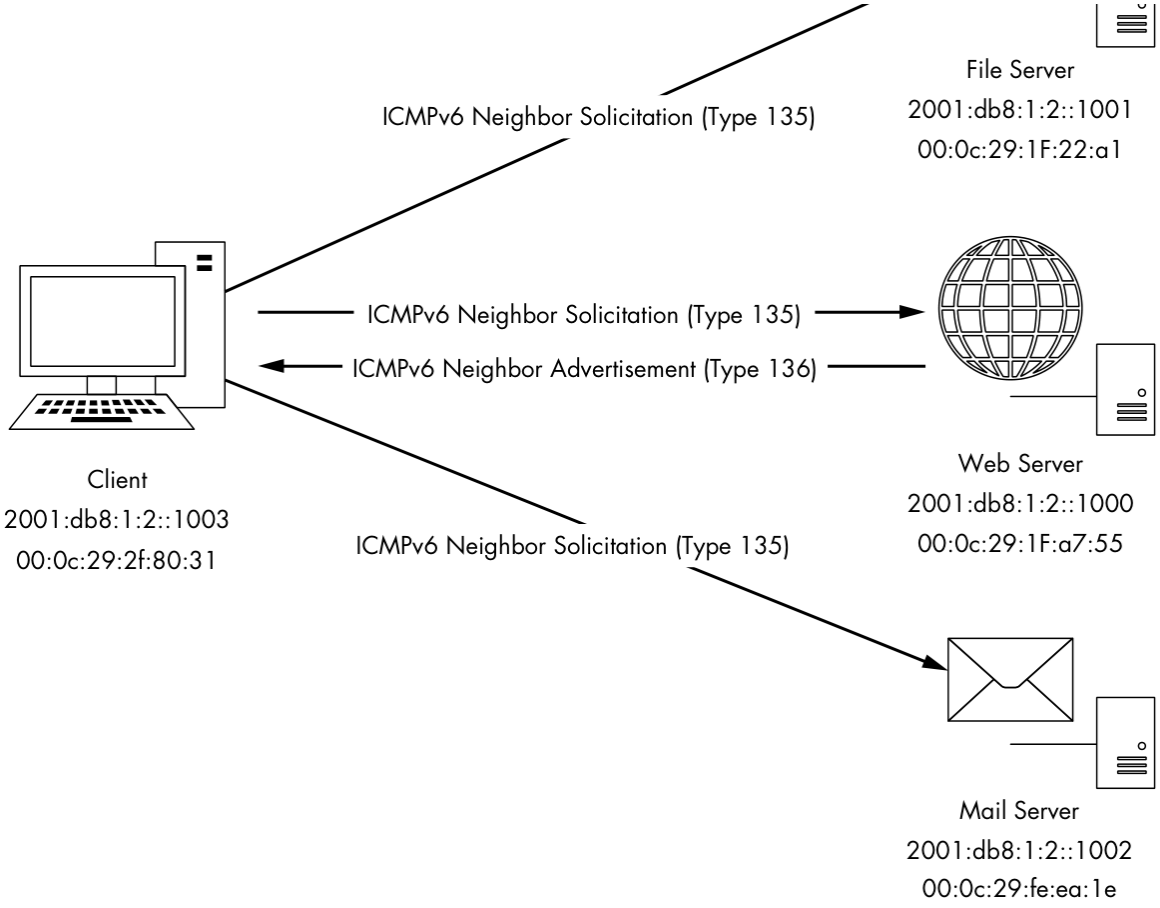


Рисунок 7-22. Процесс запроса соседей для разрешения адресов

В этом процессе узел `2001:db8:1:2::1003` посредством многоадресной рассылки отправляет каждому сетевому устройству пакет запроса соседей (Neighbor Solicitation, ICMPv6 типа 135), который спрашивает: «Какой MAC-адрес у устройства с IP-адресом `2001:db8:1:2::1000`? Мой

MAC-адрес - `00:0C:29:2f:80:31`».

Устройство, которому назначен этот IPv6-адрес, получит многоадресную передачу и ответит исходному узлу пакетом объявления соседа (Neighbor Advertisement, ICMPv6 типа 136). Этот пакет сообщает: «Мой сетевой адрес - `2001:db8:1:2::1000`, а MAC-адрес - `00:0c:29:1f:a7:55`». После получения сообщения можно начинать обмен данными.

Этот процесс показан в действии в файле захвата `icmpv6_neighbor_solicitation.pcapng`. Захват воплощает только что рассмотренный пример, в котором `2001:db8:1:2::1003` хочет связаться с `2001:db8:1:2::1000`. Рассмотрите первый пакет и разверните раздел ICMPv6 в окне подробностей пакета (рис. 7-23). Видно, что пакет имеет тип ICMP 135 (метка 2) и был отправлен с `2001:db8:1:2::1003` на многоадресный адрес `ff02::1:ff00:1000` (метка 1). Узел-источник указал целевой IPv6-адрес, с которым хотел связаться (метка 3), и собственный MAC-адрес уровня 2 (метка 4).

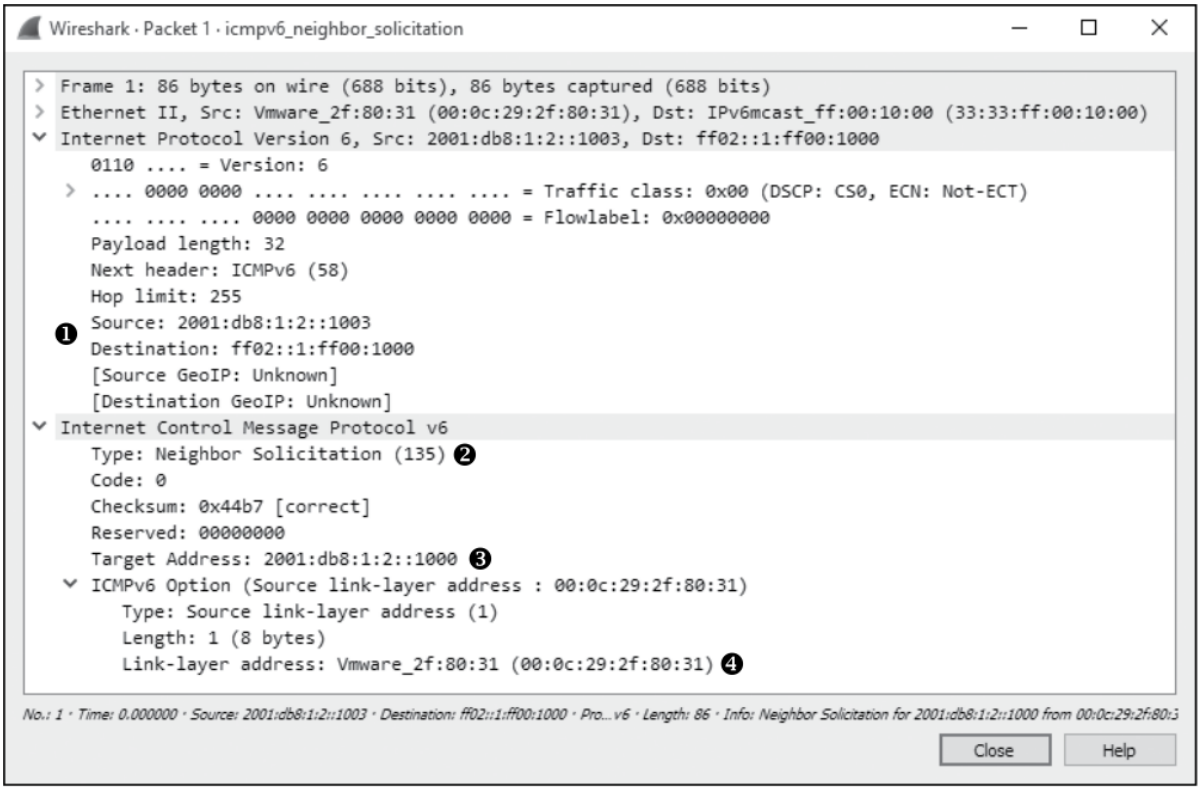


Рисунок 7-23. Пакет запроса соседей

Ответ на запрос находится во втором пакете файла захвата. Развернув раздел ICMPv6 в окне подробностей пакета (рис. 7-24), можно увидеть, что пакет имеет тип ICMP 136 (метка 2), отправлен с `2001:db8:1:2::1000` обратно на `2001:db8:1:2::1003` (метка 1) и содержит MAC-адрес `00:0c:29:1f:a7:55`, связанный с `2001:db8:1:2::1000` (метка 3).

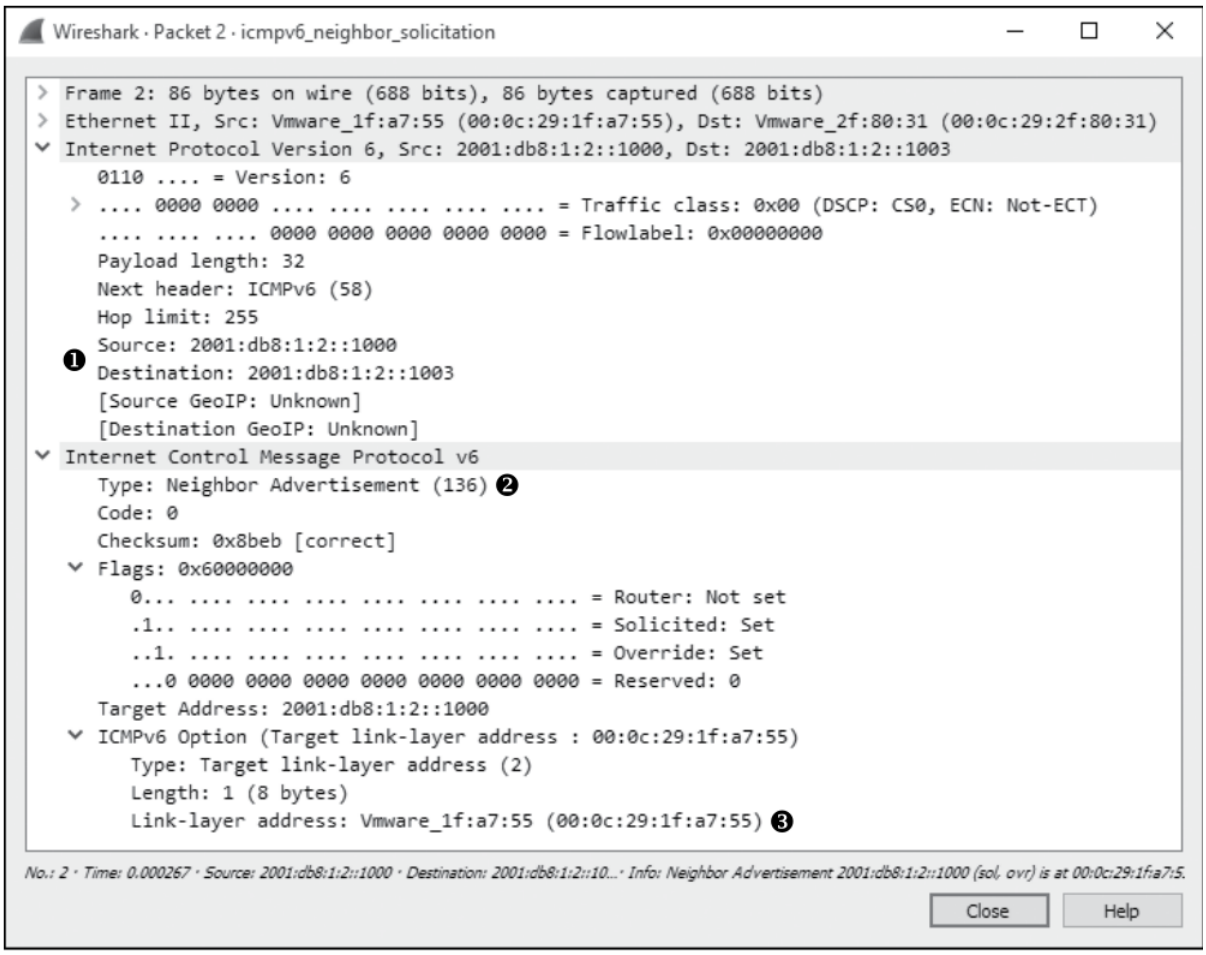


Рисунок 7-24. Пакет объявления соседа

После завершения процесса узлы `2001:db8:1:2::1003` и `2001:db8:1:2::1000` начинают обычный обмен эхо-запросами и эхо-ответами ICMPv6. Это свидетельствует об успешном запросе соседей и разрешении адреса канального уровня.

Фрагментация IPv6

Поддержка фрагментации была встроена в заголовок IPv4, поскольку во времена огромного разнообразия сетевых MTU она гарантировала, что пакеты смогут пересекать сети всех видов. В IPv6 фрагментация применяется реже, поэтому поддерживающие её параметры в заголовок IPv6 не включены. Предполагается, что устройство, передающее пакеты IPv6, прежде чем отправить их, выполнит процесс обнаружения MTU и определит максимальный размер отправляемых пакетов. Если маршрутизатор получает пакет, который слишком велик для MTU сети дальнейшей пересылки, он отбрасывает пакет и возвращает исходному узлу сообщение ICMPv6 «Слишком большой пакет» (Packet Too Big, тип 2). Получив его, исходный узел попытается повторно отправить пакет с меньшим MTU, если протокол верхнего уровня поддерживает такое действие. Процесс повторяется, пока не будет найден достаточно малый MTU или пока полезную нагрузку уже нельзя будет фрагментировать (рис. 7-25). Маршрутизатор никогда не отвечает за самостоятельную фрагментацию пакетов: именно устройство-источник определяет подходящий MTU для пути передачи и выполняет надлежащую фрагментацию.

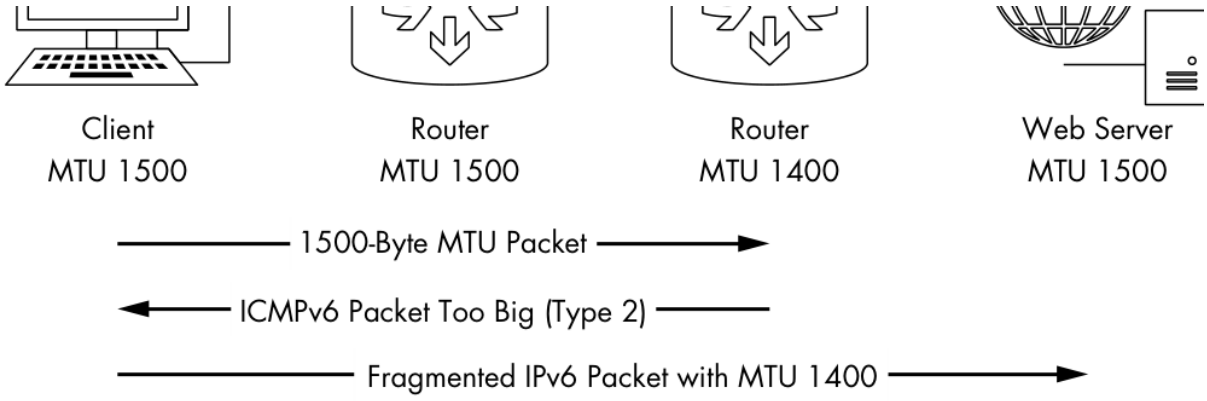


Рисунок 7-25. Обнаружение MTU пути IPv6

Если протокол верхнего уровня, используемый совместно с IPv6, не способен ограничить размер полезной нагрузки пакета, фрагментация всё же необходима. Для поддержки такого сценария к пакету IPv6 можно добавить заголовок расширения фрагментации. Пример захвата с фрагментацией IPv6 находится в файле `ipv6_fragments.pcapng`.

Поскольку MTU принимающего устройства меньше, чем у передающего, в файле захвата каждый эхо-запрос и эхо-ответ ICMPv6 представлен двумя фрагментированными пакетами. Заголовок фрагментации первого пакета показан на рис. 7-26.

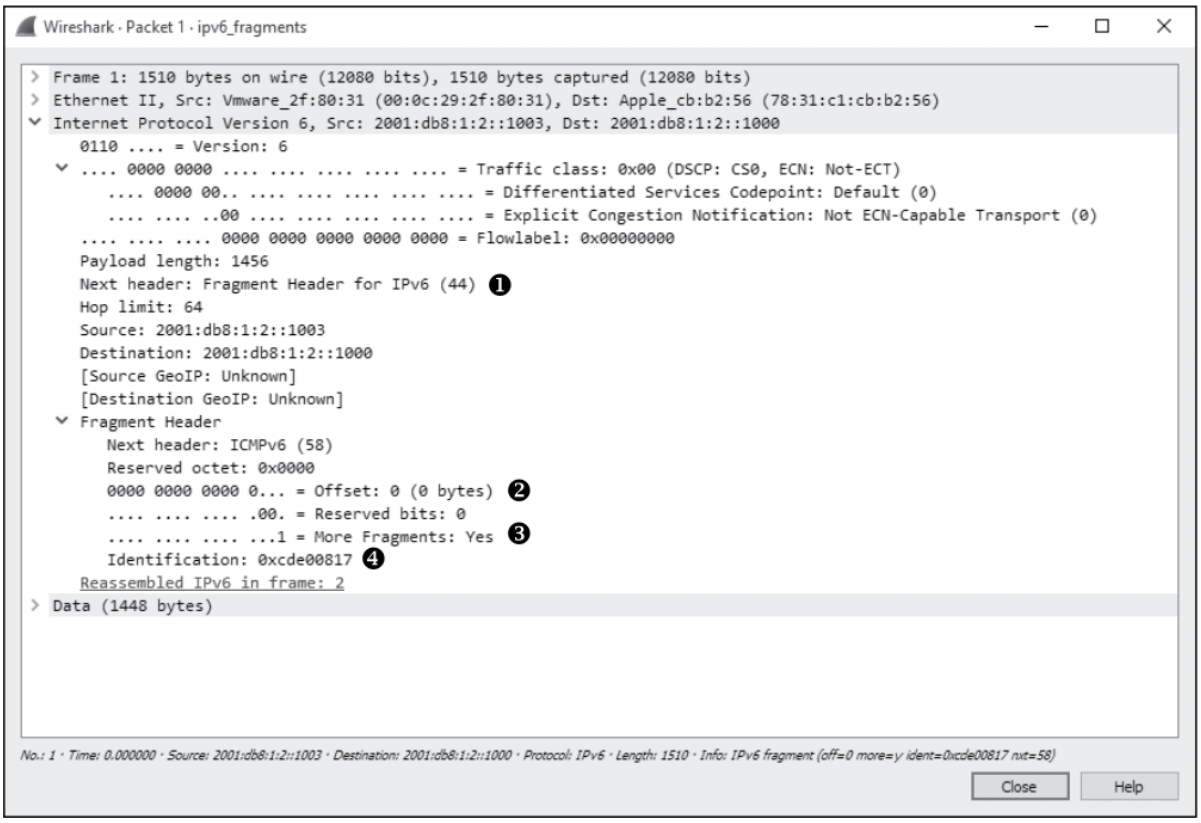


Рисунок 7-26. Заголовок расширения фрагмента IPv6

В 8-байтовом заголовке расширения содержатся все те же характеристики фрагментации, что и в пакете IPv4: смещение фрагмента (метка 2), флаг More Fragments (метка 3) и поле Identification (метка 4). Вместо присутствия в каждом пакете он добавляется в конец только тех пакетов, которым требуется фрагментация. Этот более эффективный процесс по-прежнему позволяет принимающей системе правильно собирать фрагменты. Кроме того, если заголовок расширения присутствует, поле Next header указывает на него, а не на инкапсулируемый протокол (метка 1).

Переходные протоколы IPv6

IPv6 решает вполне реальную проблему, но его внедрение идёт медленно из-за усилий, необходимых для перевода на него сетевой инфраструктуры. Чтобы облегчить переход, несколько протоколов позволяют туннелировать обмен IPv6 через сети, поддерживающие только IPv4. Туннелирование здесь означает, что обмен IPv6 инкапсулируется внутри обмена IPv4, как могут инкапсулироваться и другие протоколы. Обычно инкапсуляция выполняется одним из трёх способов:

От маршрутизатора к маршрутизатору. Туннель используется для инкапсуляции трафика IPv6 от передающих и принимающих узлов их сетей при прохождении через сеть IPv4. Этот способ позволяет целым сетям обмениваться данными по IPv6 через промежуточные каналы IPv4.

От узла к маршрутизатору. Инкапсуляция на уровне маршрутизатора применяется для передачи трафика от узла IPv6 через сеть IPv4. Этот способ позволяет отдельному узлу обмениваться данными по IPv6 с сетью IPv6, когда сам узел находится в сети, поддерживающей только IPv4.

От узла к узлу. Туннель между двумя конечными точками используется для инкапсуляции трафика IPv6 между узлами, поддерживающими IPv4 или IPv6. Этот способ позволяет конечным точкам IPv6 обмениваться данными напрямую через сеть IPv4.

Хотя переходные протоколы не будут подробно рассматриваться в книге, полезно знать об их существовании на случай, если при анализе на уровне пакетов вам когда-либо потребуется их исследовать. Вот несколько распространённых протоколов:

6to4. Этот переходный протокол, также известный как IPv6 поверх IPv4, позволяет передавать пакеты IPv6 через сеть IPv4. Он поддерживает ретрансляторы и маршрутизаторы, обеспечивая обмен IPv6 от маршрутизатора к маршрутизатору, от узла к маршрутизатору и от узла к узлу.

Teredo. Этот протокол, применяемый для одноадресного обмена IPv6 через сеть IPv4 с NAT (трансляцией сетевых адресов), передаёт пакеты IPv6 поверх IPv4, инкапсулируя их в транспортный протокол UDP.

ISATAP. Этот внутрисайтовый протокол обеспечивает внутри сети обмен между устройствами, поддерживающими только IPv4 или только IPv6, способом «от узла к узлу».

Протокол управляющих сообщений Интернета (ICMP)

Протокол управляющих сообщений Интернета (Internet Control Message Protocol, ICMP) - служебный протокол TCP/IP, отвечающий за предоставление сведений о доступности устройств, служб или маршрутов в сети TCP/IP. Большинство методов и инструментов устранения неполадок сети опирается на распространённые типы сообщений ICMP. Протокол ICMP определён в RFC 792.

Структура пакета ICMP

ICMP является частью IP и передаёт свои сообщения посредством IP. У ICMP сравнительно небольшой заголовок, который меняется в зависимости от назначения. Как показано на рис. 7-27, заголовок ICMP содержит следующие поля:

Тип (Type). Тип или класс сообщения ICMP согласно спецификации RFC.

Код (Code). Подкласс сообщения ICMP согласно спецификации RFC.

Контрольная сумма (Checksum). Служит для проверки целостности заголовка и данных ICMP после прибытия.

Переменная часть (Variable). Часть, содержимое которой зависит от полей Type и Code.

Internet Control Message Protocol (ICMP)					
Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	Type	Code	Checksum	
4+	32+	Variable			

Рисунок 7-27. Заголовок ICMP

Типы и сообщения ICMP

Как уже отмечалось, структура пакета ICMP зависит от его назначения, определяемого значениями полей Type и Code.

Поле Type ICMP можно считать классом пакета, а Code - его подклассом. Например, значение 3 в поле Type означает «назначение недоступно». Одних этих сведений может быть недостаточно для устранения проблемы, но если в поле Code того же пакета указано 3, то есть «порт недоступен», можно заключить, что проблема связана с портом, с которым вы пытаетесь установить связь.

Примечание. Полный список доступных типов и кодов ICMP см. по адресу iana.org.

Эхо-запросы и эхо-ответы

Наибольшую известность ICMP принесла утилита ping. Она используется для проверки возможности связи с устройством. Хотя сама ping не входит в спецификацию ICMP, для выполнения своей основной функции она применяет ICMP.

Чтобы воспользоваться ping, введите в командной строке `ping ipaddress`, заменив `ipaddress` фактическим IP-адресом устройства в вашей сети. Если целевое устройство включено, у компьютера есть маршрут связи с ним и межсетевой экран не блокирует обмен, вы должны увидеть ответы на команду `ping`.

В примере на рис. 7-28 показаны четыре успешных ответа с их размером, временем полного цикла (round trip time, RTT), то есть временем до прибытия пакета и получения ответа, и использованным TTL. Утилита Windows также выводит сводку с числом отправленных, полученных и потерянных пакетов. При сбое связи должно появиться сообщение с объяснением причины.

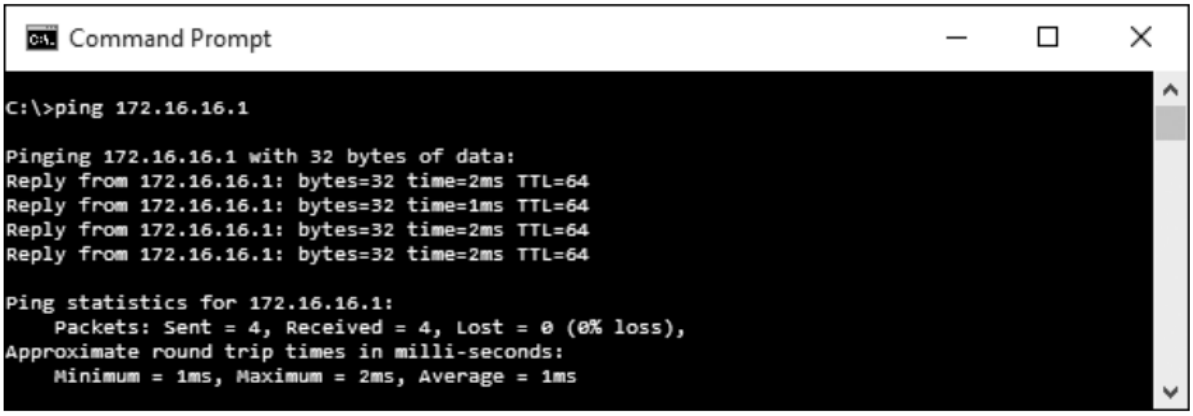


Рисунок 7-28. Применение команды ping для проверки возможности связи

По сути, команда ping по одному отправляет пакеты устройству и ожидает ответ, чтобы определить, есть ли связь с этим устройством, как показано на рис. 7-29.

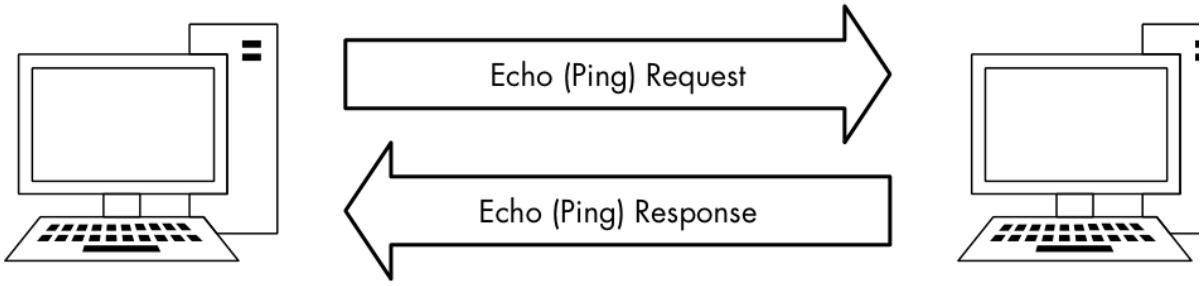


Рисунок 7-29. Команда ping состоит всего из двух шагов

> **Примечание.** Хотя ping давно служит основным инструментом ИТ-специалистов, при использовании межсетевых экранов на узлах её результаты могут вводить в заблуждение. Многие современные межсетевые экраны ограничивают способность устройства отвечать на пакеты ICMP. Это хорошо для безопасности, поскольку может остановить потенциальных злоумышленников, применяющих ping для проверки доступности узла, но одновременно затрудняет устранение неполадок: неприятно проверять связь с устройством посредством ping и не получать ответа, когда вы точно знаете, что обмен с ним возможен.

Работа утилиты ping - отличный пример простого обмена ICMP. Пакеты в файле `icmp_echo.pcapng` показывают, что происходит при запуске ping.

Первый пакет (см. рис. 7-30) показывает, что узел `192.168.100.138` отправляет пакет узлу `192.168.100.1` (метка 1). Развернув раздел ICMP этого пакета, можно определить тип пакета ICMP по полям Type и Code. Здесь пакет имеет тип 8 (метка 2) и код 0 (метка 3), что означает эхо-запрос. (Wireshark должен сообщить, чему фактически соответствует показанная пара типа и кода.) Этот эхо-запрос - первая половина обмена. Это простой пакет ICMP, отправленный посредством IP и содержащий небольшой объём данных. Помимо типа, кода и контрольной суммы в нём есть порядковый номер, который используется для сопоставления запросов с ответами, а переменная часть пакета ICMP содержит случайную текстовую строку.

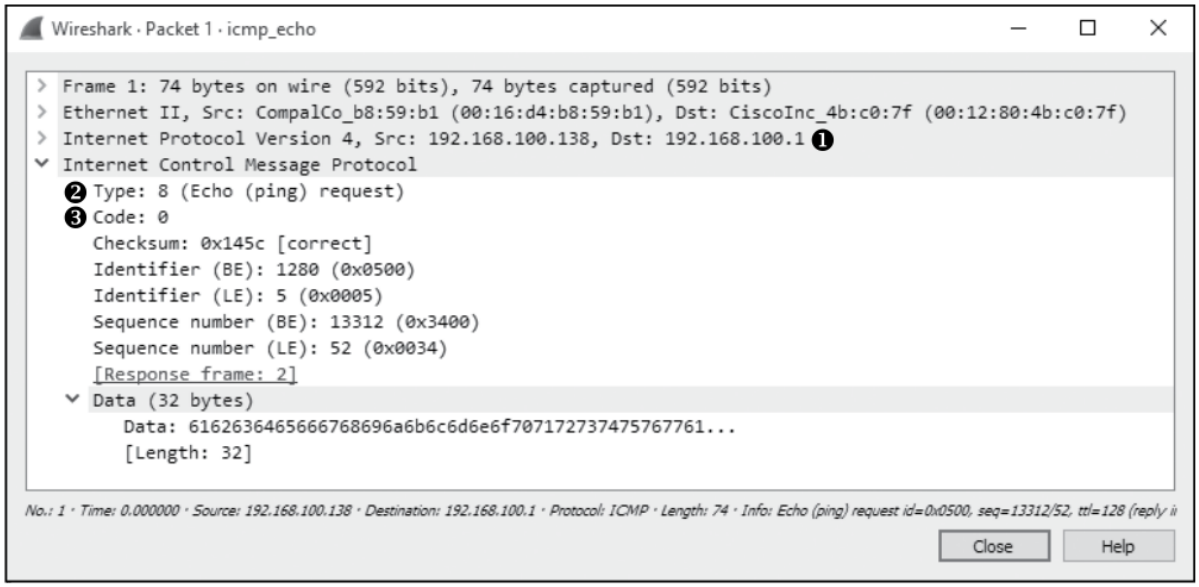


Рисунок 7-30. Пакет эхо-запроса ICMP

> **Примечание.** Термины «эхо» и «ping» часто используются как взаимозаменяемые, однако помните, что ping - это название инструмента. Инструмент ping служит для отправки пакетов эхо-запроса ICMP.

Второй пакет последовательности является ответом на запрос (см. рис. 7-31). Раздел ICMP пакета содержит тип 0 (метка 1) и код 0 (метка 2), то есть перед нами эхо-ответ. Порядковый номер и идентификатор второго пакета совпадают с первым (метка 3), поэтому мы знаем, что этот эхо-ответ соответствует эхо-запросу из предыдущего пакета. Wireshark показывает значения этих полей в формате с прямым порядком байтов (big-endian, BE) и обратным порядком байтов (little-endian, LE). Иными словами, данные представлены в разном порядке в зависимости от того, как их может обрабатывать конкретная конечная точка. Пакет ответа также содержит ту же 32-байтовую строку данных, которая передавалась с исходным запросом (метка 4). После получения второго пакета узлом `192.168.100.138` ping сообщит об успехе.

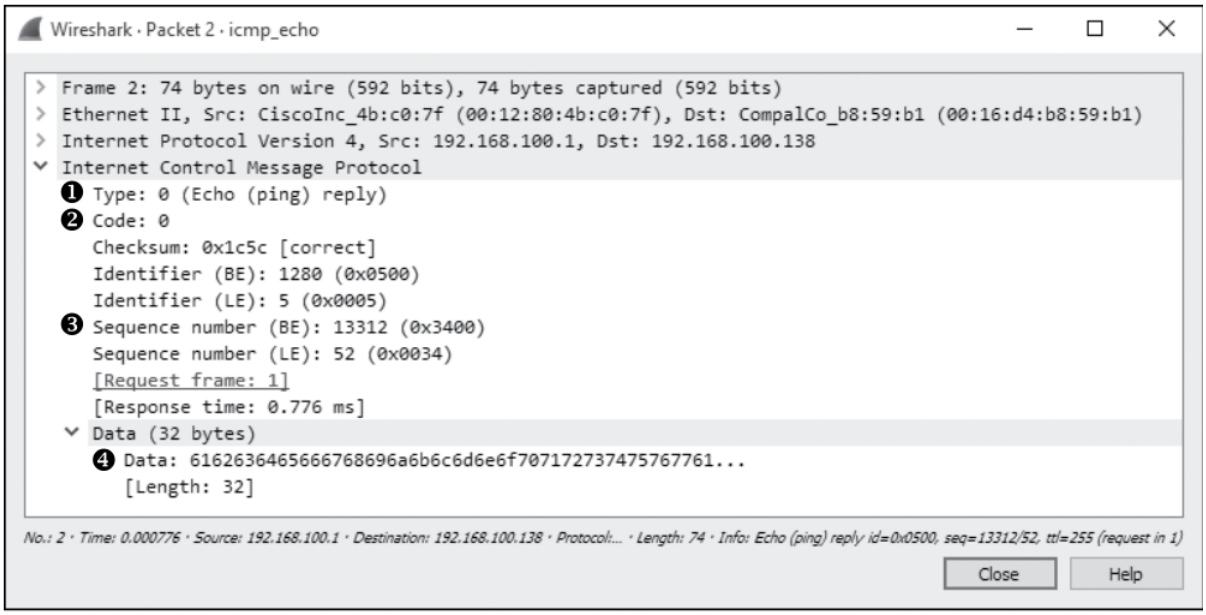


Рисунок 7-31. Пакет эхо-ответа ICMP

Обратите внимание: варианты команды ping позволяют увеличить размер заполняющих данных в эхо-запросах и тем самым принудительно фрагментировать пакеты для разных видов устранения сетевых неполадок. Это может потребоваться при устранении проблем в сетях, которым нужен меньший размер фрагмента.

> **Примечание.** Случайный текст, используемый в эхо-запросе ICMP, может очень заинтересовать потенциального злоумышленника. Содержащиеся в заполнении сведения позволяют злоумышленникам профилировать операционную систему устройства. Кроме того, они могут помещать в это поле небольшие фрагменты данных для скрытого обмена.

traceroute

Утилита traceroute служит для определения пути от одного устройства к другому. В простой сети путь может проходить всего через один маршрутизатор или вовсе не проходить через маршрутизаторы. Однако в сложной сети пакету для достижения конечного назначения может потребоваться пройти десятки маршрутизаторов. Поэтому для устранения неполадок связи крайне важно иметь возможность проследить точный путь пакета от одного назначения к другому.

С помощью ICMP (и при небольшой помощи IP) traceroute может построить маршрут движения пакетов. Например, первый пакет файла `icmp_traceroute.pcapng` очень похож на эхо-запрос, рассмотренный в предыдущем разделе (см. рис. 7-32).

Пакеты в этом захвате были сформированы командой `tracert 4.2.2.1`. Чтобы воспользоваться traceroute в Windows, введите в командной строке `tracert ipaddress`, заменив `ipaddress` фактическим IP-адресом устройства, путь к которому нужно определить. В Linux или Mac используется команда `traceroute ipaddress`.

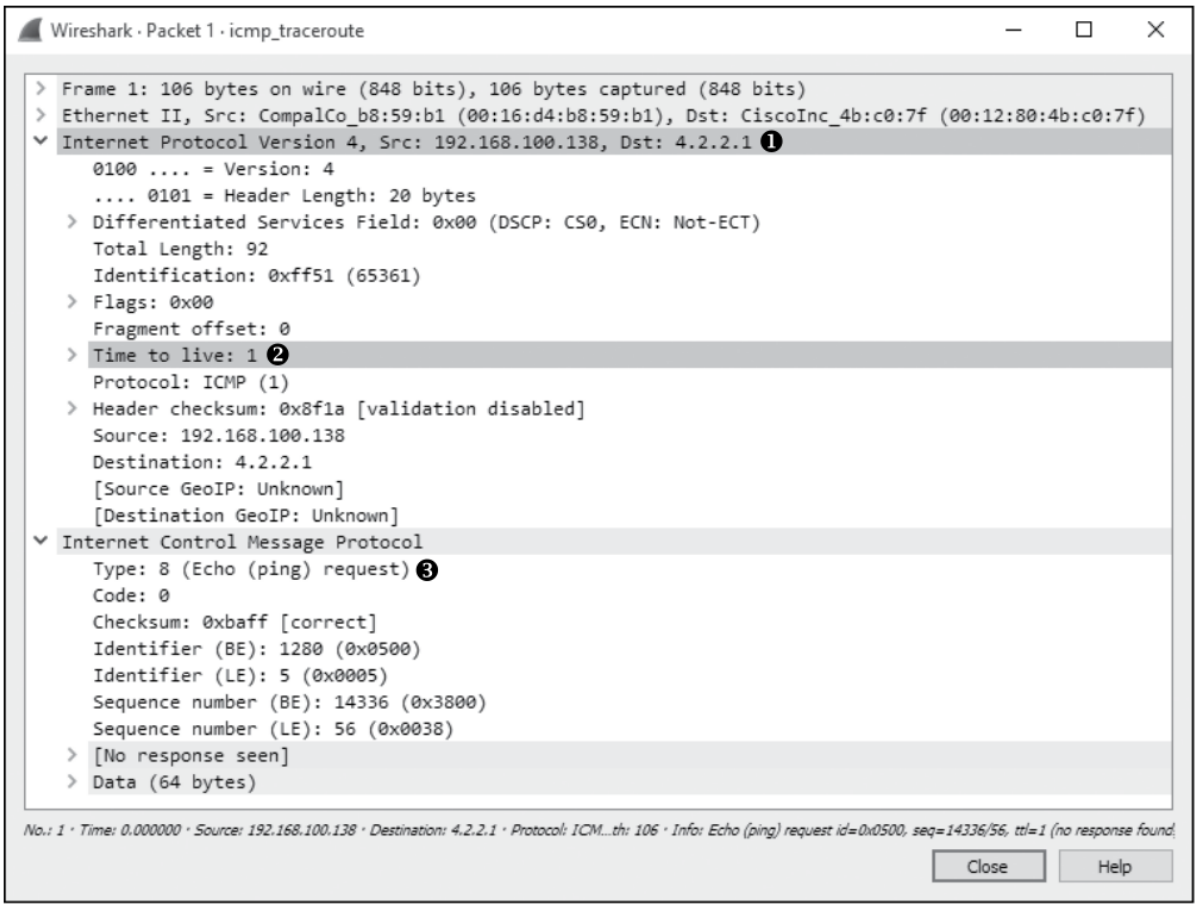


Рисунок 7-32. Пакет эхо-запроса ICMP со значением TTL 1

На первый взгляд этот пакет кажется простым эхо-запросом (метка 3) от `192.168.100.138` к `4.2.2.1` (метка 1), а всё содержимое раздела ICMP имеет обычный формат пакета эхо-запроса. Однако, развернув заголовок IP пакета, вы заметите странность: TTL пакета равен 1 (метка 2), то есть пакет будет отброшен первым же встреченным маршрутизатором. Поскольку адрес назначения `4.2.2.1` находится в Интернете, мы знаем, что между устройством-источником и назначением есть как минимум один маршрутизатор, поэтому пакет никак не сможет достичь назначения. Для нас это хорошо, поскольку работа traceroute основана на том, что пакет достигнет только первого пройденного маршрутизатора.

Второй пакет, как и ожидалось, является ответом от первого маршрутизатора на пути к назначению (см. рис. 7-33). Этот пакет достиг устройства `192.168.100.1`, его TTL уменьшился до 0, и дальнейшая передача стала невозможной, поэтому маршрутизатор отправил ответ ICMP. Пакет имеет тип 11 (метка 1) и код 0 (метка 2). Эти данные сообщают, что назначение было недоступно из-за превышения TTL пакета при передаче.

Такой пакет ICMP иногда называют двуглавым, потому что в хвостовой части его раздела ICMP находится копия заголовка IP (метка 3) и данных ICMP (метка 4), отправленных в исходном эхо-запросе. Эти сведения могут оказаться очень полезными при устранении неполадок.

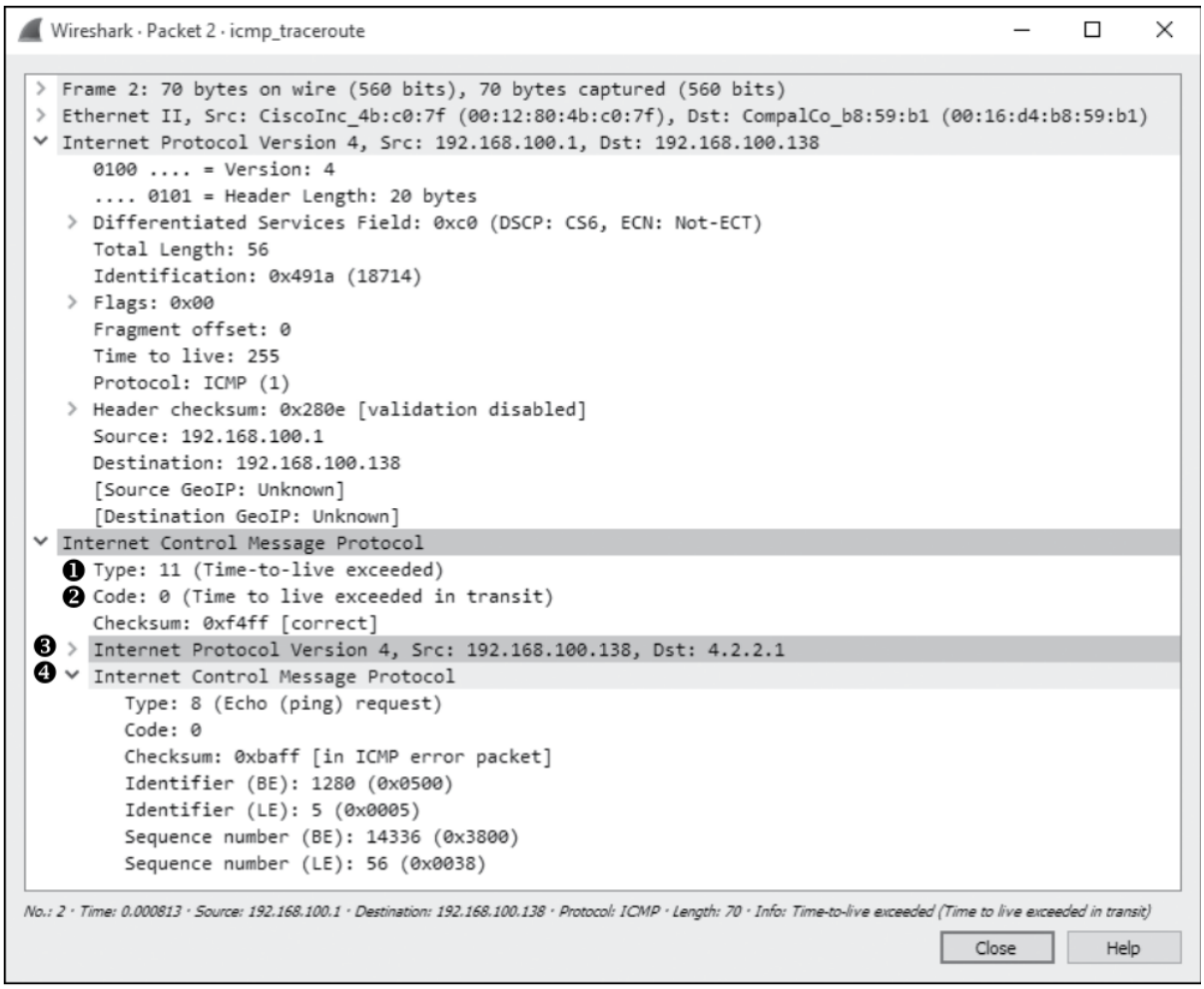


Рисунок 7-33. Ответ ICMP от первого маршрутизатора на пути

Процесс отправки пакетов с TTL 1 повторяется ещё дважды до пакета 7. Здесь наблюдается то же, что в первом пакете, но теперь TTL в заголовке IP равен 2, поэтому перед отбрасыванием пакет гарантированно достигнет маршрутизатора второго перехода. Как и ожидалось, мы получаем ответ от маршрутизатора следующего перехода `12.180.241.1` с теми же сообщениями ICMP о недоступности назначения и превышении TTL.

Процесс продолжается с увеличением TTL на 1, пока не будет достигнуто назначение `4.2.2.1`. Однако прямо перед этим, как видно на рис. 7-34, истекает время ожидания запроса в строке 8. Как запрос на пути может завершиться по тайм-ауту, а весь процесс при этом успешно продолжиться? Обычно это происходит, когда маршрутизатор настроен не отвечать на запросы ICMP. Маршрутизатор всё же получает запрос и передаёт данные следующему маршрутизатору, поэтому в строке 9 на рис. 7-34 виден следующий переход. Просто он не сформировал пакет ICMP о превышении времени жизни, как остальные переходы. Не получив ответа, tracert считает, что время ожидания запроса истекло, и переходит к следующему.

Итак, в ходе traceroute выполнен обмен с каждым маршрутизатором на пути и построена карта маршрута к назначению. Пример такой карты показан на рис. 7-34.

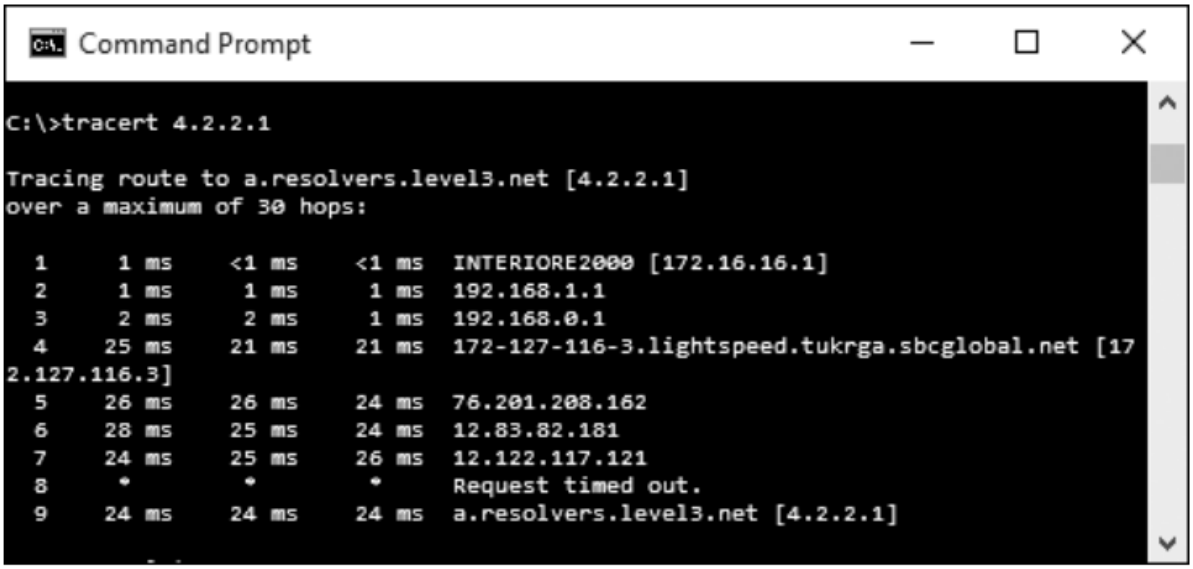


Рисунок 7-34. Пример вывода утилиты traceroute

> **Примечание.** Обсуждение traceroute здесь в основном ориентировано на Windows, поскольку эта утилита использует исключительно ICMP. Утилита traceroute в Linux несколько универсальнее и способна задействовать другие протоколы для трассировки пути маршрута.

ICMP версии 6 (ICMPv6)

Обновлённая версия IP активно полагается на ICMP в таких функциях, как запрос соседей и обнаружение пути, что было показано в предыдущих примерах. Протокол ICMPv6 установлен в RFC 4443 для поддержки набора функций, необходимых IPv6, а также дополнительных усовершенствований. Отдельно ICMPv6 в книге не рассматривается, поскольку он использует ту же структуру пакетов, что и ICMP.

Пакеты ICMPv6 обычно относятся либо к сообщениям об ошибках, либо к информационным сообщениям. Полный список доступных типов и кодов IANA находится по адресу iana.org.

В этой главе вы познакомились с несколькими важнейшими протоколами, которые встретятся при анализе пакетов. ARP, IP и ICMP лежат в основе всего сетевого обмена и критически важны практически для любой повседневной задачи. В главе 8 мы рассмотрим распространённые протоколы транспортного уровня TCP и UDP.

Глава 8. Протоколы транспортного уровня

В этой главе мы продолжим рассматривать отдельные протоколы и их представление на уровне пакетов. Поднимаясь по модели OSI, мы обратимся к транспортному уровню и двум самым распространённым транспортным протоколам - TCP и UDP.

Протокол управления передачей (TCP)

Конечная цель протокола управления передачей (Transmission Control Protocol, TCP) - обеспечить сквозную надёжность доставки данных. TCP, определённый в RFC 793, управляет упорядочением данных и восстановлением после ошибок и в конечном счёте гарантирует, что данные попадут туда, куда должны. TCP считается протоколом с установлением соединения: перед передачей данных он устанавливает формальное соединение, отслеживает доставку пакетов и обычно пытается формально закрыть каналы обмена после завершения передачи. Многие часто используемые протоколы прикладного уровня полагаются на TCP и IP при доставке пакетов к конечному назначению.

Структура пакета TCP

TCP предоставляет множество функций, что отражается в сложности его заголовка. Как показано на рис. 8-1, заголовок TCP содержит следующие поля:

Порт источника (Source Port). Порт, используемый для передачи пакета.

Порт назначения (Destination Port). Порт, на который будет передан пакет.

Порядковый номер (Sequence Number). Номер для идентификации сегмента TCP. Это поле помогает удостовериться, что части потока данных не отсутствуют.

Номер подтверждения (Acknowledgment Number). Порядковый номер, который ожидается в следующем пакете от другого участвующего в обмене устройства.

Флаги (Flags). Флаги URG, ACK, PSH, RST, SYN и FIN, определяющие тип передаваемого пакета TCP.

Размер окна (Window Size). Размер буфера получателя TCP в байтах.

Контрольная сумма (Checksum). Служит для проверки целостности содержимого заголовка и данных TCP после прибытия.

Указатель срочности (Urgent Pointer). Если установлен флаг URG, это поле исследуется для получения дополнительных указаний о том, с какого места внутри пакета центральному процессору следует начать чтение данных.

Параметры (Options). Различные необязательные поля, которые можно задать в пакете TCP.

Transmission Control Protocol (TCP)						
Offsets	Octet	0		1	2	
Octet	Bit	0–3	4–7	8–15	16–23	
0	0	Source Port			Destination Port	
4	32	Sequence Number				
8	64	Acknowledgment Number				
12	96	Data Offset	Reserved	Flags	Window Size	
16	128	Checksum			Urgent Pointer	
20+	160+	Options				

Рисунок 8-1. Заголовок TCP

Порты TCP

Весь обмен TCP происходит с использованием портов источника и назначения, которые присутствуют в каждом заголовке TCP. Порт похож на гнездо старого телефонного коммутатора. Оператор следил за панелью лампочек и разъёмов. Когда загоралась лампочка, он соединялся со звонящим, спрашивал, с кем тот хочет поговорить, а затем соединял его с другим абонентом, вставляя кабель. Для каждого звонка требовались порт источника (звонящий) и порт назначения (получатель). Порты TCP работают почти так же.

Чтобы передать данные определённому приложению на удалённом сервере или устройстве, пакет TCP должен знать порт, на котором удалённая служба ожидает соединений. При попытке обратиться к приложению через порт, отличный от настроенного, обмен завершится неудачей.

Порт источника в такой последовательности не особенно важен и может выбираться случайно. Удалённый сервер просто определит порт для обмена из первоначально отправленного ему пакета (см. рис. 8-2).

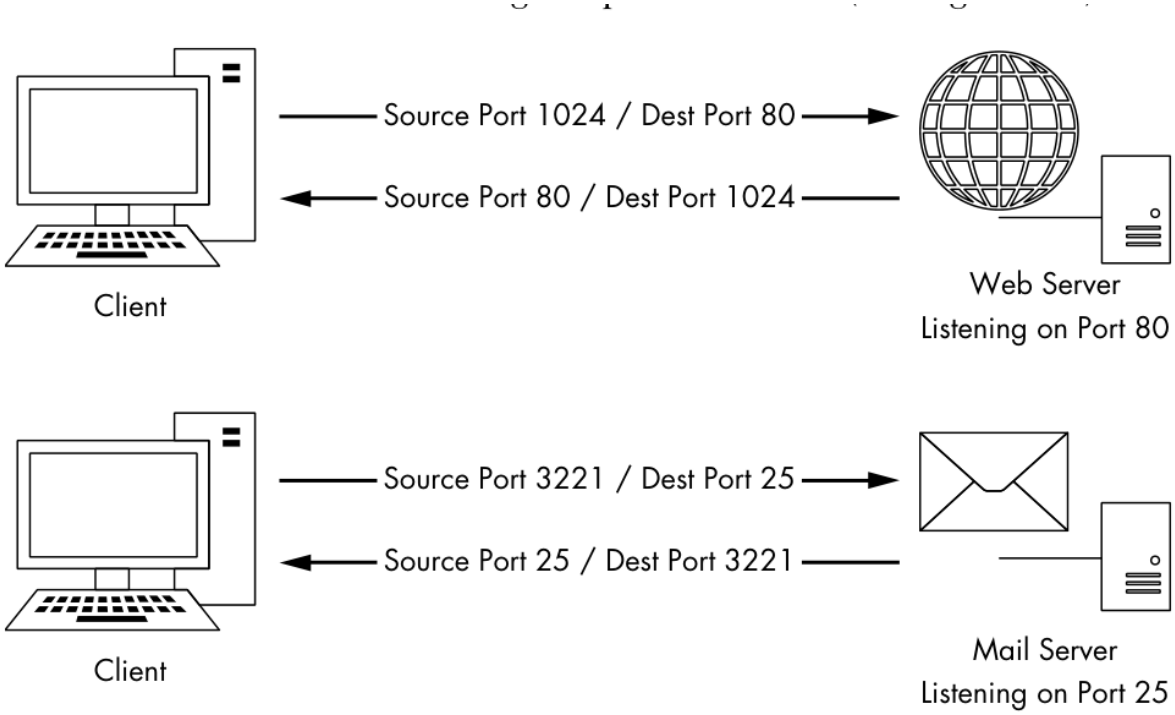


Рисунок 8-2. TCP использует порты для передачи данных

Для обмена по TCP доступно 65 535 портов. Обычно они делятся на две группы:

• Группа системных портов (также называемых стандартными или общеизвестными портами) охватывает порты 1-1023 (порт 0 не учитывается, поскольку зарезервирован). Общеизвестные устоявшиеся службы обычно используют порты из системной группы.
• Группа эфемерных портов охватывает порты 1024-65535 (хотя в некоторых операционных системах диапазон определяется иначе). В любой момент через один порт может обмениваться данными только одна служба, поэтому современные операционные системы выбирают порты источника случайным образом, стремясь сделать обмен уникальным. Такие порты источника обычно находятся в эфемерном диапазоне.

Рассмотрим пару пакетов TCP и определим номера используемых ими портов, открыв файл `tcp_ports.pcapng`. В нём содержится HTTP-обмен клиента, посещающего два веб-сайта. Как уже говорилось, HTTP использует для обмена TCP, поэтому прекрасно иллюстрирует стандартный трафик TCP.

В первом пакете этого файла (см. рис. 8-3) первые два значения представляют порт источника и порт назначения. Пакет отправляется с `172.16.16.128` на `212.58.226.142`. Порт источника - эфемерный порт 2826 (метка 1). (Помните, что операционная система выбирает порты источника случайно, хотя затем их значения могут последовательно увеличиваться от этого случайного выбора.) Порт назначения - системный порт 80 (метка 2), стандартный порт веб-серверов, использующих HTTP.

Рисунок 8-3. Порты источника и назначения находятся в заголовке TCP

Обратите внимание, что Wireshark обозначает эти порты как `slc-systemlog` (2826) и `http` (80). Wireshark хранит список портов и их наиболее распространённых применений. Хотя общеизвестными применениями помечаются прежде всего системные порты, со многими эфемерными портами тоже связаны часто используемые службы. Такие обозначения могут сбивать с толку, поэтому обычно лучше отключать их, выключив разрешение имён транспортного уровня. Для этого выберите Edit > Preferences > Name Resolution и снимите флажок Enable Transport Name Resolution. Если вы хотите оставить этот параметр включённым, но изменить способ определения некоторого порта в Wireshark, отредактируйте файл `services` в системном каталоге Wireshark. Его содержимое основано на списке распространённых портов IANA (пример редактирования файла разрешения имён см. в разделе «Использование собственного файла hosts» на странице 86).

Второй пакет отправляется обратно с `212.58.226.142` на `172.16.16.128` (см. рис. 8-4). Как и IP-адреса, порты источника и назначения теперь поменялись местами (метка 1).

В большинстве случаев обмен на основе TCP происходит одинаково: случайный порт источника выбирается для связи с известным портом назначения. После отправки первоначального пакета удалённое устройство обменивается с устройством-источником данными через установленные порты.

В этом образце файла захвата есть ещё один поток обмена. Попробуйте найти номера портов, которые он использует.

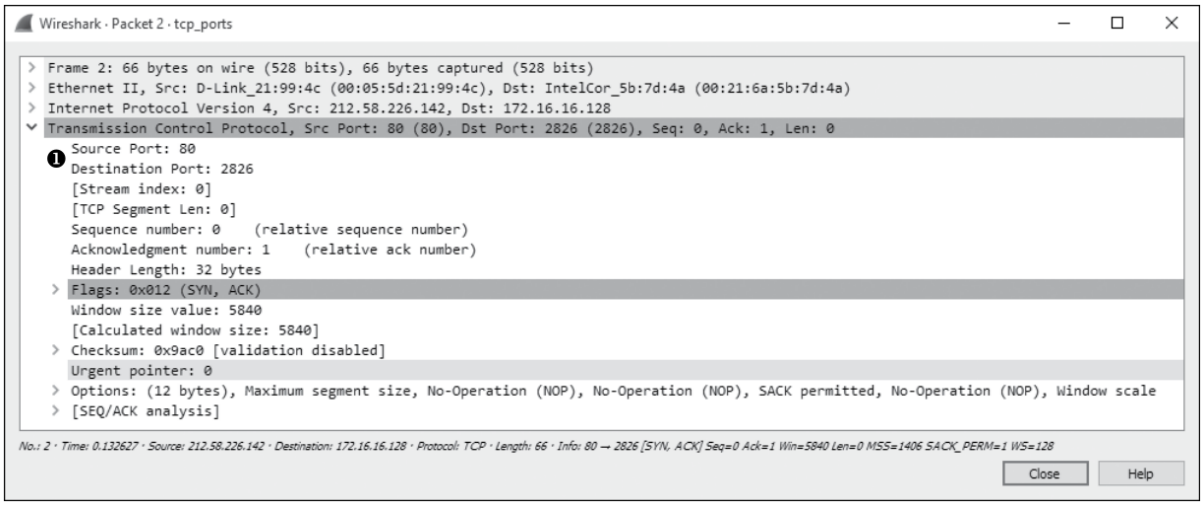


Рисунок 8-4. Перестановка номеров портов источника и назначения для обратного обмена

> **Примечание.** По мере чтения книги вы будете узнавать всё больше о портах, связанных с распространёнными протоколами и службами. Со временем вы сможете профилировать службы и устройства по используемым ими портам. Полный список распространённых портов находится в файле `services` системного каталога Wireshark.

Трёхэтапное рукопожатие TCP

Любой обмен на основе TCP должен начинаться с рукопожатия между двумя узлами. Этот процесс служит нескольким целям:

• Позволяет передающему узлу убедиться, что узел-получатель работает и способен обмениваться данными.
• Позволяет передающему узлу проверить, ожидает ли получатель соединения на порте, через который передающий узел пытается связаться.
• Позволяет передающему узлу отправить получателю свой начальный порядковый номер, чтобы оба узла могли поддерживать правильный порядок потока пакетов.

Рукопожатие TCP выполняется в три этапа, как показано на рис. 8-5. Сначала устройство, желающее начать обмен (узел A), отправляет пакет TCP своей цели (узлу B). В этом первоначальном пакете нет данных, кроме заголовков протоколов нижних уровней. В заголовке TCP установлен флаг SYN, а также указаны начальный порядковый номер и максимальный размер сегмента (maximum segment size, MSS), которые будут использоваться при обмене. Узел B отвечает похожим пакетом с установленными флагами SYN и ACK и собственным начальным порядковым номером. Наконец, узел A отправляет узлу B последний пакет, в котором установлен только флаг ACK. После завершения процесса у обоих устройств должны быть все сведения, необходимые для правильного начала обмена.

> **Примечание.** Пакеты TCP часто называют по установленным в них флагам. Например, вместо «пакет TCP с установленным флагом SYN» говорят «пакет SYN». Поэтому пакеты процесса рукопожатия TCP называются SYN, SYN/ACK и ACK.

Чтобы увидеть процесс в действии, откройте `tcp_handshake.pcapng`. В Wireshark имеется функция, заменяющая порядковые номера пакетов TCP относительными числами для упрощения анализа. Для наших целей отключим её, чтобы видеть фактические порядковые номера. Выберите Edit > Preferences, разверните заголовок Protocols и выберите TCP. В окне снимите флажок Relative Sequence Numbers и нажмите OK.

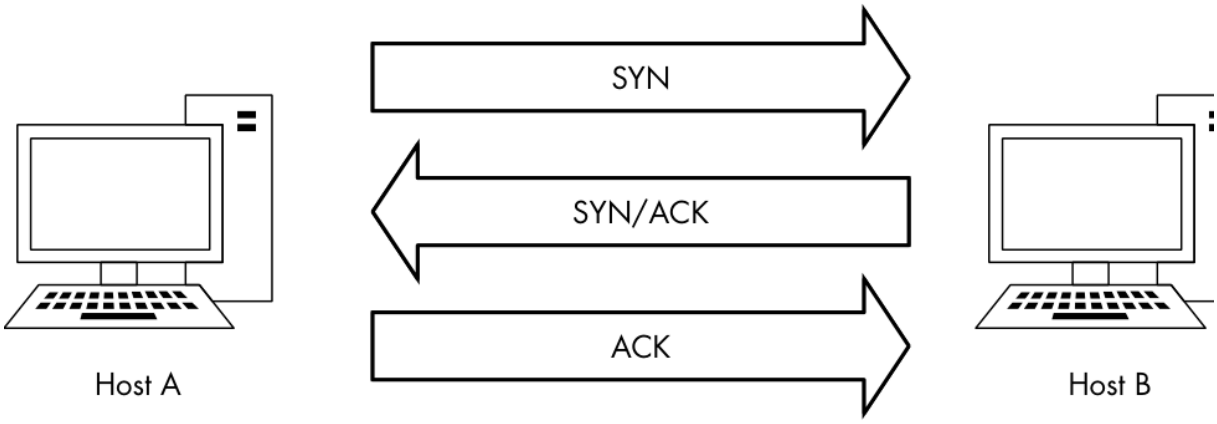


Рисунок 8-5. Трёхэтапное рукопожатие TCP

Первый пакет захвата представляет исходный пакет SYN (метка 2; см. рис. 8-6). Он передаётся с `172.16.16.128` через порт 2826 на `212.58.226.142`, порт 80. Видно, что передаваемый порядковый номер равен `3691127924` (метка 1).

Второй пакет рукопожатия - ответ SYN/ACK (метка 3) от `212.58.226.142` (см. рис. 8-7). В пакете также содержатся начальный порядковый номер этого узла (`233779340`) (метка 1) и номер подтверждения (`3691127925`) (метка 2). Показанный номер подтверждения на 1 больше порядкового номера предыдущего пакета, поскольку это поле задаёт следующий порядковый номер, который узел ожидает получить.

Последний пакет - ACK (метка 2), отправленный с `172.16.16.128` (см. рис. 8-8). Как и ожидалось, он содержит порядковый номер `3691127925` (метка 1), заданный полем Acknowledgment number предыдущего пакета.

Рукопожатие происходит перед каждой последовательностью обмена TCP. Если вы просматриваете насыщенный файл захвата в поисках начала последовательности обмена, сочетание SYN-SYN/ACK-ACK служит отличным ориентиром.

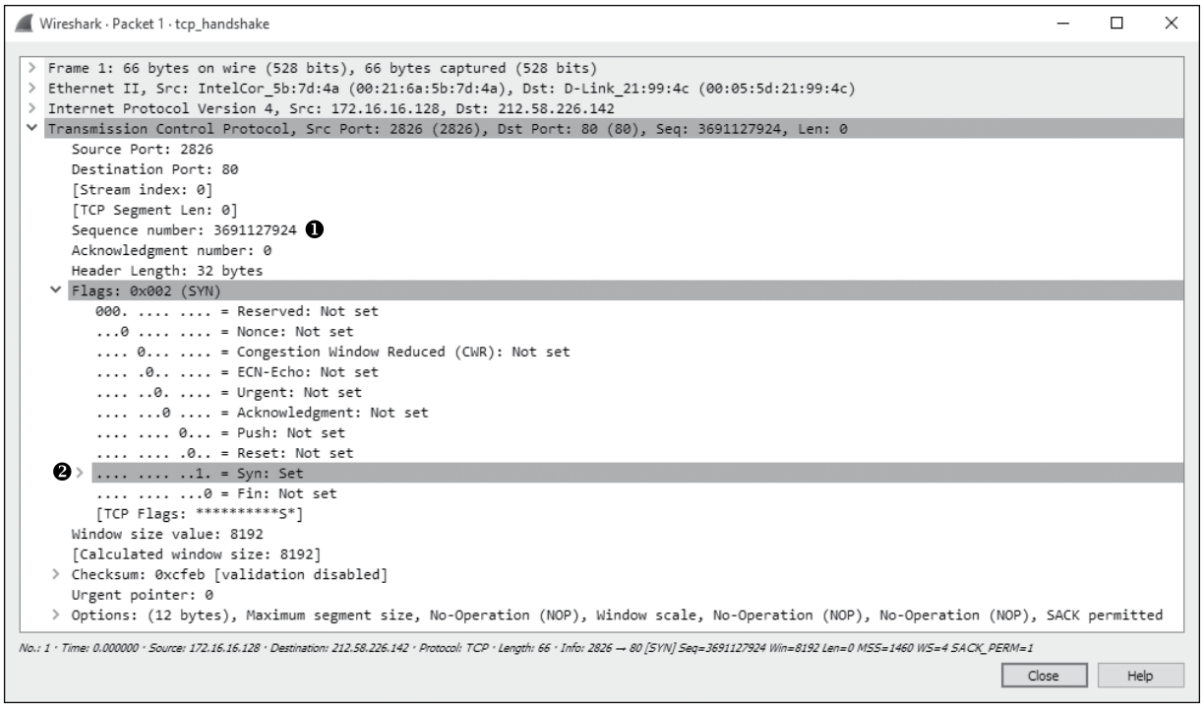


Рисунок 8-6. Исходный пакет SYN

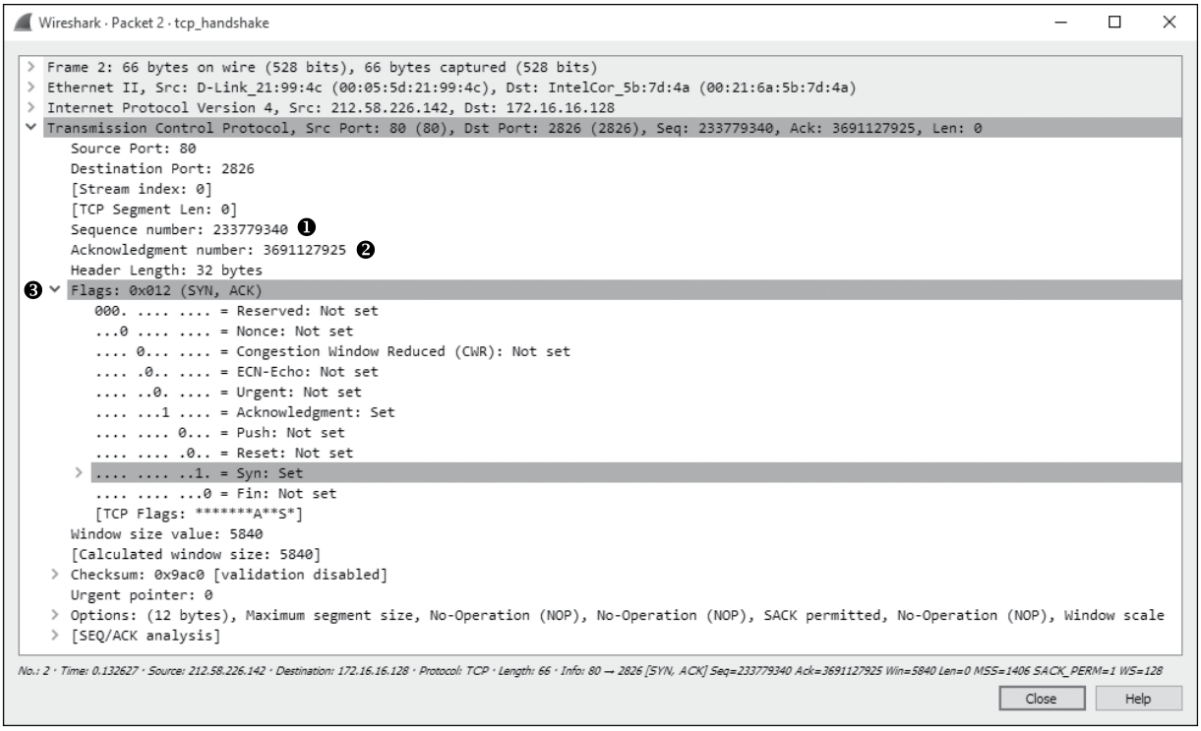


Рисунок 8-7. Ответ SYN/ACK

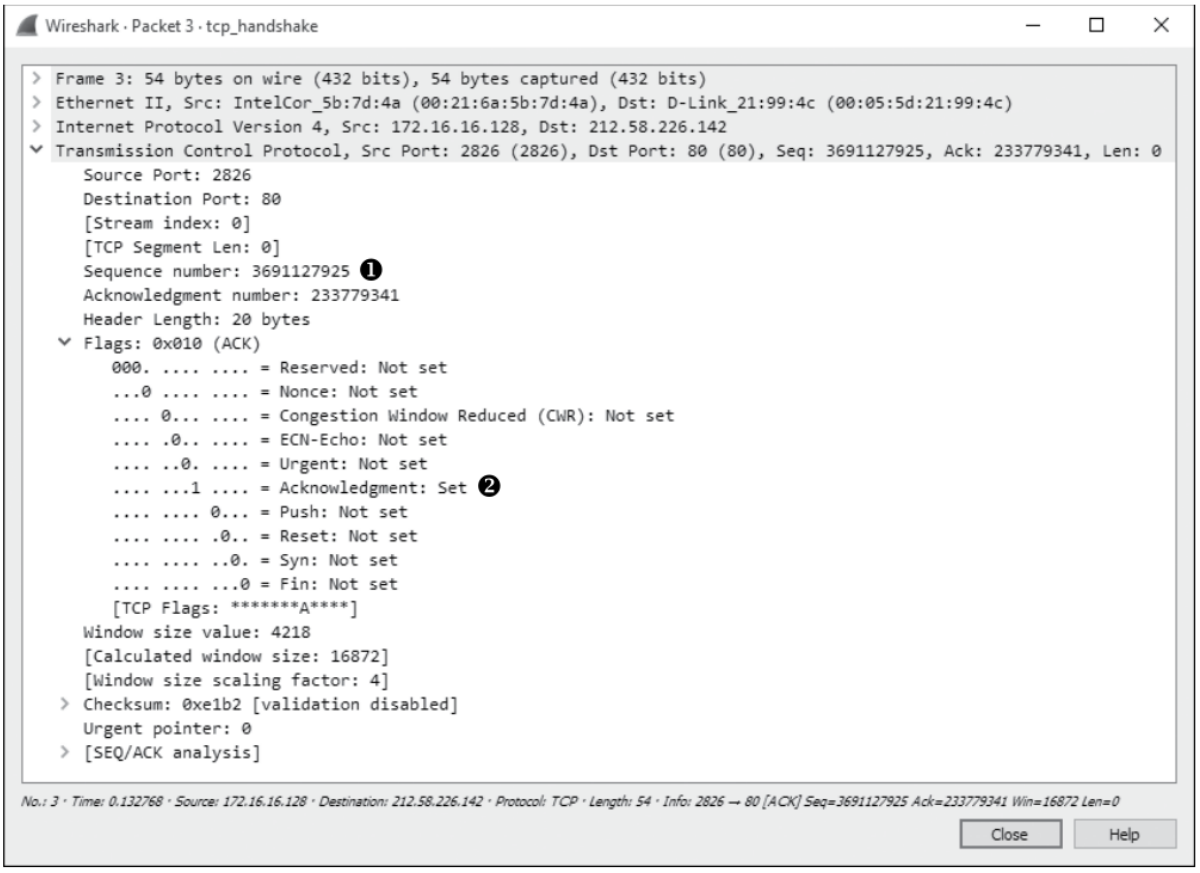


Рисунок 8-8. Заключительный ACK

Завершение соединения TCP

Большинство приветствий рано или поздно заканчивается прощанием, а в TCP за каждым рукопожатием следует завершение соединения. Завершение TCP используется для корректного закрытия соединения между двумя устройствами после окончания обмена. Процесс состоит из четырёх пакетов и применяет флаг FIN как признак конца соединения.

В последовательности завершения узел A сообщает узлу B об окончании обмена, отправляя пакет TCP с установленными флагами FIN и ACK. Узел B отвечает пакетом ACK и передаёт собственный пакет FIN/ACK. Узел A отвечает пакетом ACK, завершая обмен. Процесс показан на рис. 8-9.

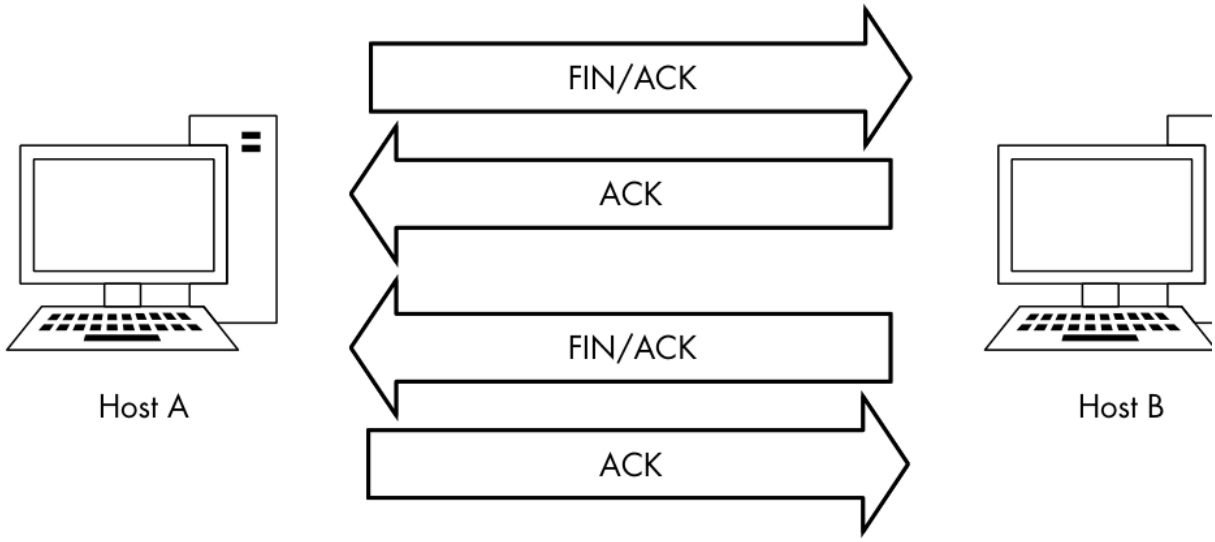


Рисунок 8-9. Процесс завершения соединения TCP

Чтобы увидеть процесс в Wireshark, откройте файл `tcp_teardown.pcapng`. В первом пакете последовательности (см. рис. 8-10) видно, что устройство с адресом `67.228.110.120` инициирует завершение, отправляя пакет с установленными флагами FIN и ACK (метка 1).

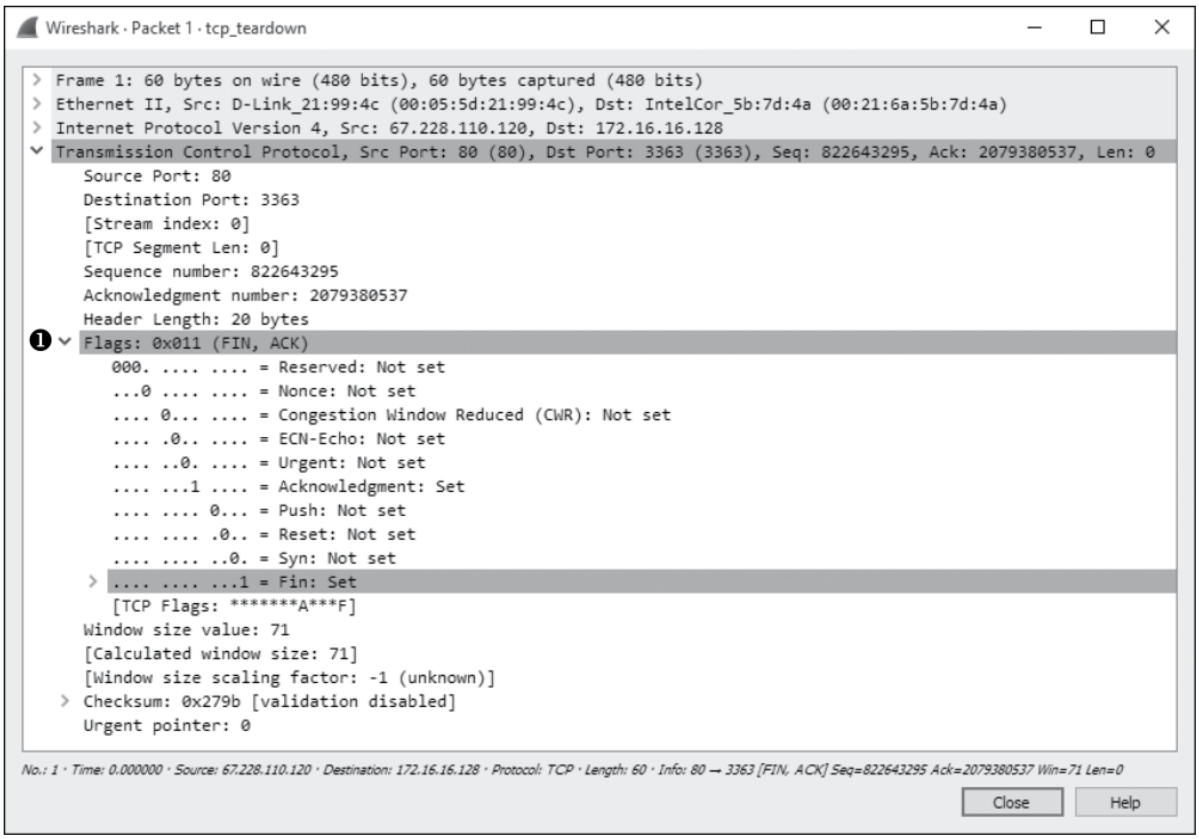


Рисунок 8-10. Пакет FIN/ACK инициирует процесс завершения соединения

После отправки этого пакета узел `172.16.16.128` подтверждает его получение пакетом ACK и отправляет пакет FIN/ACK. Процесс завершается, когда `67.228.110.120` отправляет заключительный ACK. В этот момент обмен между двумя устройствами прекращается. Если им потребуется возобновить обмен, придётся выполнить новое рукопожатие TCP.

Сбросы TCP

В идеальном мире каждое соединение заканчивалось бы корректным завершением TCP. На практике соединения часто обрываются внезапно. Например, узел может быть неправильно настроен или потенциальный злоумышленник может сканировать порты. В таких случаях, когда пакет отправлен устройству, которое не желает его принимать, может быть передан пакет TCP с установленным флагом RST. Флаг RST указывает, что соединение было внезапно закрыто, либо отклоняет попытку соединения.

Файл `tcp_refuseconnection.pcapng` показывает пример сетевого трафика с пакетом RST. Первый пакет файла поступает от узла `192.168.100.138`, который пытается связаться с `192.168.100.1` через порт 80. Этому узлу неизвестно, что `192.168.100.1` не ожидает соединений на порте 80, поскольку это маршрутизатор Cisco без настроенного веб-интерфейса. На этом порте не настроена служба, способная принимать соединения. В ответ на попытку обмена `192.168.100.1` отправляет узлу `192.168.100.138` пакет, сообщающий, что обмен через порт 80 невозможен. На рис. 8-11 это внезапное окончание попытки обмена показано в заголовке TCP второго пакета. Пакет RST не содержит ничего, кроме флагов RST и ACK (метка 1), и дальнейшего обмена не происходит.

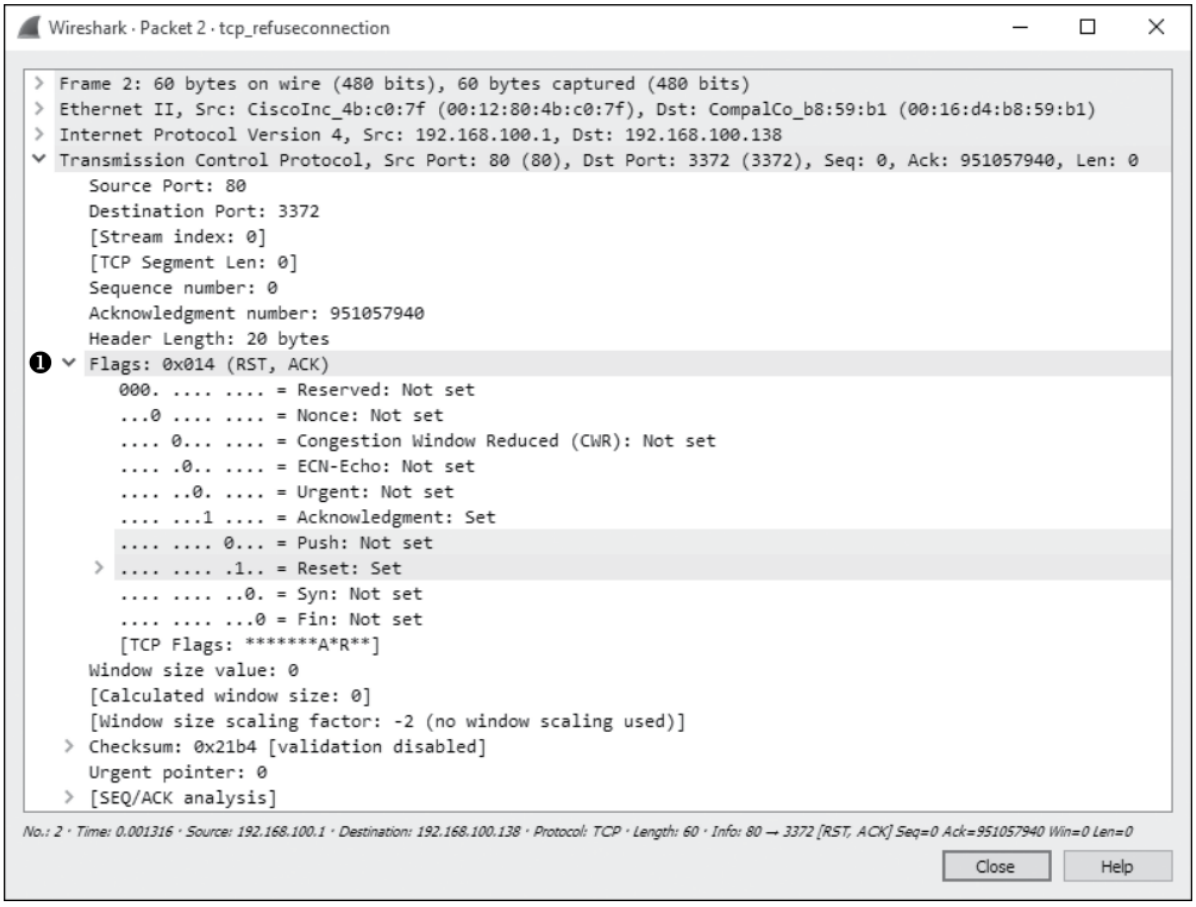


Рисунок 8-11. Флаги RST и ACK означают окончание обмена

Пакет RST прекращает обмен независимо от того, прибыл ли он в начале попытки обмена, как в данном примере, или был отправлен в середине обмена между узлами.

Протокол пользовательских датаграмм (UDP)

Протокол пользовательских датаграмм (User Datagram Protocol, UDP) - другой протокол уровня 4, широко используемый в современных сетях. TCP предназначен для надёжной доставки данных со встроенной проверкой ошибок, тогда как цель UDP - быстрая передача. Поэтому UDP предоставляет обслуживание по мере возможности и обычно называется протоколом без установления соединения. В отличие от TCP с его рукопожатием и завершением, протокол без установления соединения формально не открывает и не закрывает соединение между узлами.

Можно предположить, что трафик UDP - протокола без установления соединения, не предоставляющего надёжного обслуживания, - в лучшем случае будет нестабильным. Так и было бы, но протоколы, полагающиеся на UDP, обычно имеют собственные встроенные механизмы надёжности или используют отдельные функции ICMP, чтобы сделать обмен несколько надёжнее. Например, протоколы прикладного уровня DNS и DHCP, для которых очень важна скорость передачи пакетов по сети, используют UDP в качестве протокола транспортного уровня, но самостоятельно управляют проверкой ошибок и таймерами повторной передачи.

Структура пакета UDP

Заголовок UDP намного меньше и проще заголовка TCP. Как показано на рис. 8-12, он содержит следующие поля:

Порт источника (Source Port). Порт, используемый для передачи пакета.

Порт назначения (Destination Port). Порт, на который будет передан пакет.

Длина пакета (Packet Length). Длина пакета в байтах.

Контрольная сумма (Checksum). Служит для проверки целостности содержимого заголовка и данных UDP после прибытия.

User Datagram Protocol (UDP)

Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	Source Port		Destination Port	
4	32	Packet Length		Checksum	

Рисунок 8-12. Заголовок UDP

Файл `udp_dnsrequest.pcapng` содержит один пакет. Он представляет запрос DNS, использующий UDP. Развернув заголовок UDP пакета, вы увидите четыре поля (см. рис. 8-13).

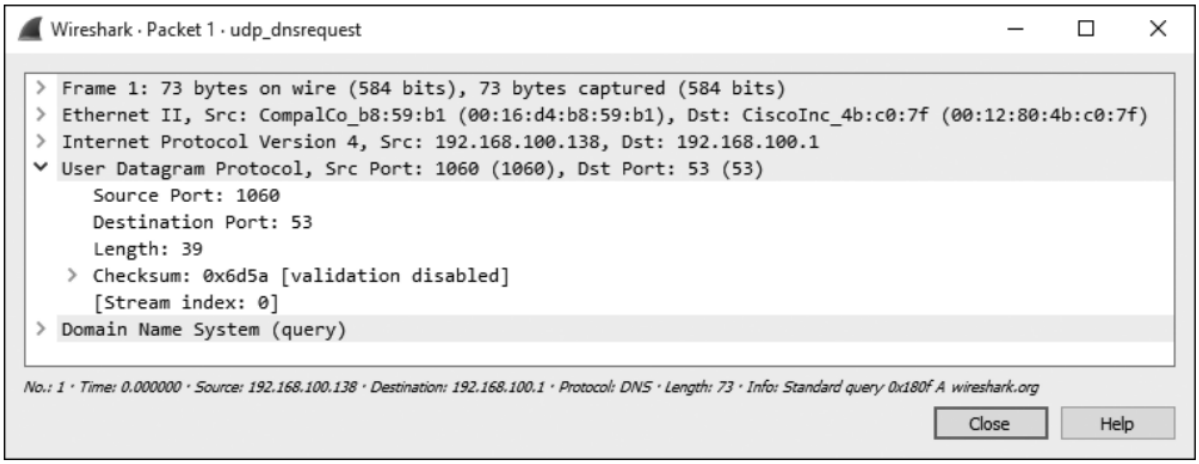


Рисунок 8-13. Содержимое пакета UDP очень просто

Главное, что следует помнить: UDP не заботится о надёжности доставки. Поэтому любое использующее UDP приложение при необходимости должно самостоятельно принимать специальные меры для надёжной доставки. Этим UDP отличается от TCP, который формально устанавливает и завершает соединение и располагает средствами проверки успешной передачи пакетов.

В этой главе вы познакомились с протоколами транспортного уровня TCP и UDP. Подобно сетевым протоколам, TCP и UDP лежат в основе большей части повседневного обмена данными, и умение эффективно их анализировать критически важно для становления квалифицированным аналитиком пакетов. В главе 9 мы рассмотрим распространённые протоколы прикладного уровня.

Глава 9. Распространённые протоколы верхнего уровня

В этой главе мы продолжим рассматривать функции отдельных протоколов и то, как они выглядят в Wireshark. Мы обсудим пять наиболее распространённых протоколов верхнего уровня (уровня 7): DHCP, DNS, HTTP и SMTP.

Протокол динамической настройки узла (DHCP)

На заре сетей устройству, желавшему обмениваться данными по сети, требовалось вручную назначить адрес. По мере роста сетей этот ручной процесс быстро становился обременительным. Для решения проблемы был создан протокол начальной загрузки (Bootstrap Protocol, BOOTP), автоматически назначавший адреса подключённым к сети устройствам. Позднее BOOTP был заменён более совершенным протоколом динамической настройки узла (Dynamic Host Configuration Protocol, DHCP).

DHCP - протокол прикладного уровня, позволяющий устройству автоматически получить IP-адрес, а также адреса других важных сетевых ресурсов, например DNS-серверов и маршрутизаторов. Сегодня большинство серверов DHCP предоставляет клиентам и другие параметры, включая адреса используемых в сети шлюза по умолчанию и DNS-серверов.

Структура пакета DHCP

Пакеты DHCP могут передавать клиенту множество сведений. Как показано на рис. 9-1, в пакете DHCP присутствуют следующие поля:

Код операции (OpCode). Указывает, является ли пакет запросом или ответом DHCP.

Тип оборудования (Hardware Type). Тип аппаратного адреса (10-мегабитный Ethernet, IEEE 802, ATM и так далее).

Длина аппаратного адреса (Hardware Length). Длина аппаратного адреса.

Переходы (Hops). Используется агентами ретрансляции при поиске сервера DHCP.

Идентификатор транзакции (Transaction ID). Случайное число для сопоставления запросов с ответами.

Прошедшие секунды (Seconds Elapsed). Число секунд с момента, когда клиент впервые запросил адрес у сервера DHCP.

Флаги (Flags). Типы трафика, которые способен принимать клиент DHCP (одноадресный, широковещательный и так далее).

Dynamic Host Configuration Protocol (DHCP)					
Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	OpCode	Hardware Type	Hardware Length	Hops
4	32	Transaction ID			
8	64	Seconds Elapsed		Flags	
12	96	Client IP Address			
16	128	Your IP Address			
20	160	Server IP Address			
24	192	Gateway IP Address			
28	224	Client IP Address			
32	256	Client Hardware Address (16 bytes)			
36	288				
40	320				
44	352				
48+	384+	Server Host Name (64 bytes)			
		Boot File (128 bytes)			
		Options			

Рисунок 9-1. Структура пакета DHCP

IP-адрес клиента (Client IP Address). IP-адрес клиента (полученный из поля Your IP Address).

Ваш IP-адрес (Your IP Address). IP-адрес, предложенный сервером DHCP (в конечном счёте становится значением поля Client IP Address).

IP-адрес сервера (Server IP Address). IP-адрес сервера DHCP.

IP-адрес шлюза (Gateway IP Address). IP-адрес шлюза сети по умолчанию.

Аппаратный адрес клиента (Client Hardware Address). MAC-адрес клиента.

Имя узла сервера (Server Host Name). Имя узла сервера (необязательно).

Загрузочный файл (Boot File). Загрузочный файл для DHCP (необязательно).

Параметры (Options). Служат для расширения структуры пакета DHCP дополнительными возможностями.

Процесс инициализации DHCP

Главная цель DHCP - назначить адреса клиентам во время инициализации. Процесс обновления происходит между одним клиентом и сервером DHCP, как показано в файле `dhcp_nolease_initialization.pcapng`. Инициализацию DHCP часто называют процессом DORA, поскольку в ней используются четыре типа пакетов DHCP: обнаружение (discover), предложение (offer), запрос (request) и подтверждение (acknowledgment), как показано на рис. 9-2. Рассмотрим каждый тип пакета DORA.

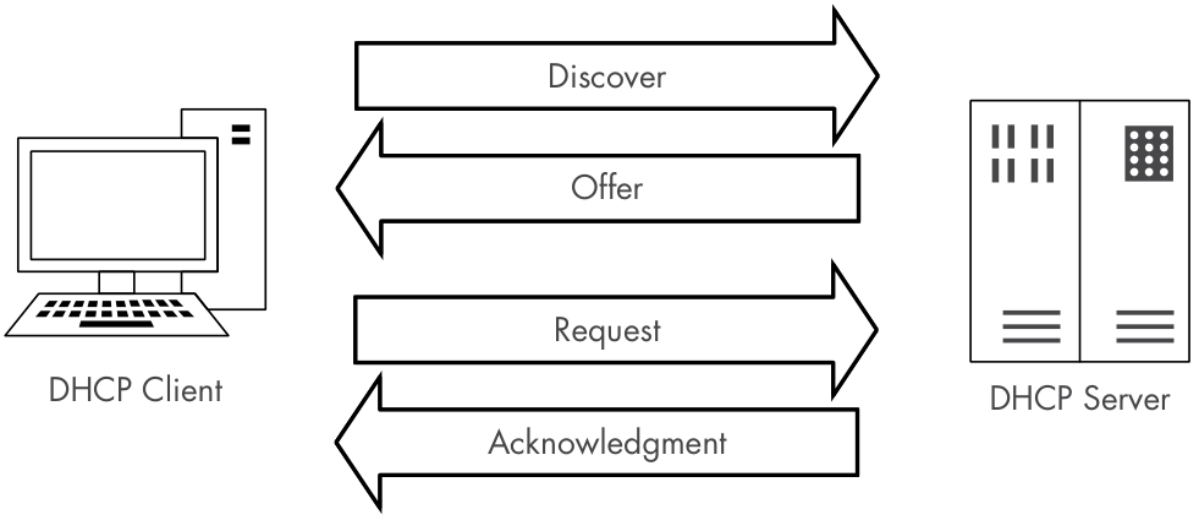


Рисунок 9-2. Процесс DORA DHCP

Пакет обнаружения

Как видно в указанном файле захвата, первый пакет отправляется с `0.0.0.0`, порт 68, на `255.255.255.255`, порт 67. Клиент использует `0.0.0.0`, поскольку у него ещё нет IP-адреса. Пакет направляется на `255.255.255.255`, потому что это не зависящий от сети широковещательный адрес, гарантирующий отправку пакета каждому сетевому устройству. Поскольку устройство не знает адрес сервера DHCP, первый пакет передаётся в попытке найти сервер DHCP, который его услышит.

Изучая панель подробностей пакета, прежде всего мы замечаем, что DHCP использует UDP в качестве протокола транспортного уровня. Для DHCP очень важна скорость получения клиентом запрошенных сведений. В DHCP имеются собственные встроенные средства надёжности, поэтому UDP подходит идеально. Подробности процесса обнаружения видны в разделе DHCP первого пакета на панели подробностей пакета, как показано на рис. 9-3.

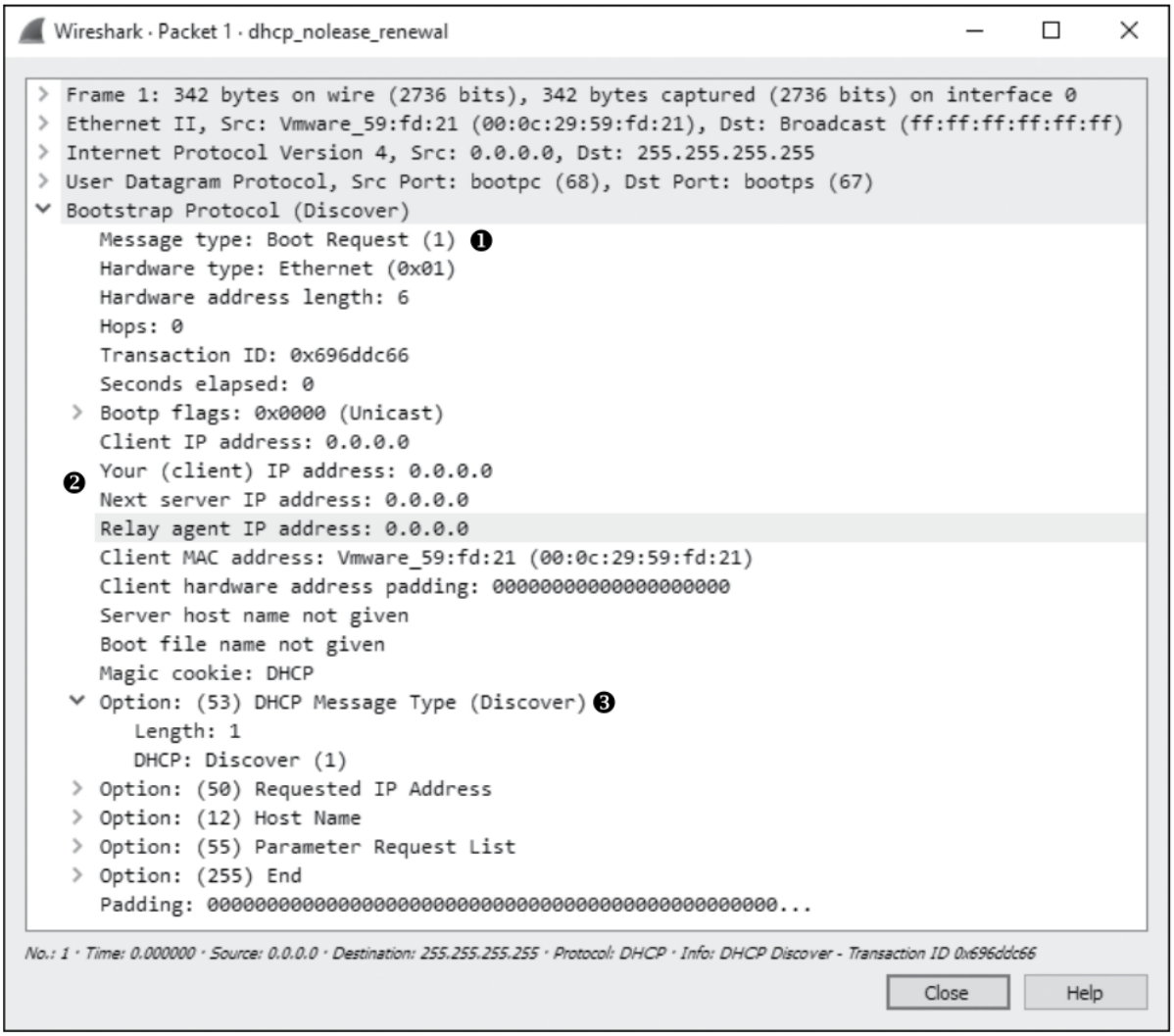


Рисунок 9-3. Пакет обнаружения DHCP

> **Примечание.** Поскольку при работе с DHCP Wireshark всё ещё ссылается на BOOTP, на панели подробностей пакета вы увидите раздел Bootstrap Protocol, а не DHCP. Тем не менее во всей книге я буду называть его разделом DHCP пакета.

Этот пакет является запросом, что определяется значением (1) в поле Message type (метка 1). Большинство полей пакета обнаружения либо полностью заполнены нулями (как видно в полях IP-адресов, метка 2), либо понятны сами по себе с учётом списка полей DHCP из предыдущего раздела. Основное содержимое пакета находится в его четырёх полях Option (метка 3).

Тип сообщения DHCP (DHCP Message Type). Это параметр типа 53 с длиной 1 и значением Discover (1). Эти значения указывают, что перед нами пакет обнаружения DHCP.

Идентификатор клиента (Client Identifier). Предоставляет дополнительные сведения о клиенте, запрашивающем IP-адрес.

Запрошенный IP-адрес (Requested IP Address). Указывает IP-адрес, который клиент хотел бы получить. Это может быть ранее использованный IP-адрес или `0.0.0.0`, означающий отсутствие предпочтений.

Список запрашиваемых параметров (Parameter Request List). Перечисляет различные элементы конфигурации (IP-адреса других важных сетевых устройств и элементы, не относящиеся к IP), которые клиент хотел бы получить от сервера DHCP.

Пакет предложения

В заголовке IP второго пакета файла указаны действительные IP-адреса: пакет движется с `192.168.1.5` на `192.168.1.10`, как показано на рис. 9-4. У клиента ещё нет адреса `192.168.1.10`, поэтому сервер сначала попытается связаться с ним по аппаратному адресу, предоставленному ARP. Если обмен невозможен, сервер просто передаст предложение широковещательно.

Раздел DHCP этого второго пакета, называемого пакетом предложения, указывает, что Message type имеет значение reply (метка 1). Пакет содержит тот же Transaction ID, что и предыдущий (метка 2), а значит, этот ответ действительно относится к исходному запросу.

Сервер DHCP отправляет пакет предложения, чтобы предложить клиенту свои услуги. Для этого он сообщает сведения о себе и адресные данные, которые хочет предоставить клиенту. На рис. 9-4 IP-адрес `192.168.1.10` в поле Your (client) IP address предлагается клиенту (метка 3) сервером `192.168.1.5`, указанным в поле Next server IP address (метка 4).

Первый перечисленный параметр определяет пакет как DHCP Offer (метка 5). Следующие параметры предоставлены сервером и указывают дополнительные сведения, которые он может предложить вместе с IP-адресом клиента. Он предлагает:

• время аренды IP-адреса 10 минут;
• маску подсети `255.255.255.0`;
• широковещательный адрес `192.168.1.255`;
• адрес маршрутизатора `192.168.1.254`;
• имя домена `mydomain.example`;
• адреса серверов доменных имён `192.168.1.1` и `192.168.1.2`.

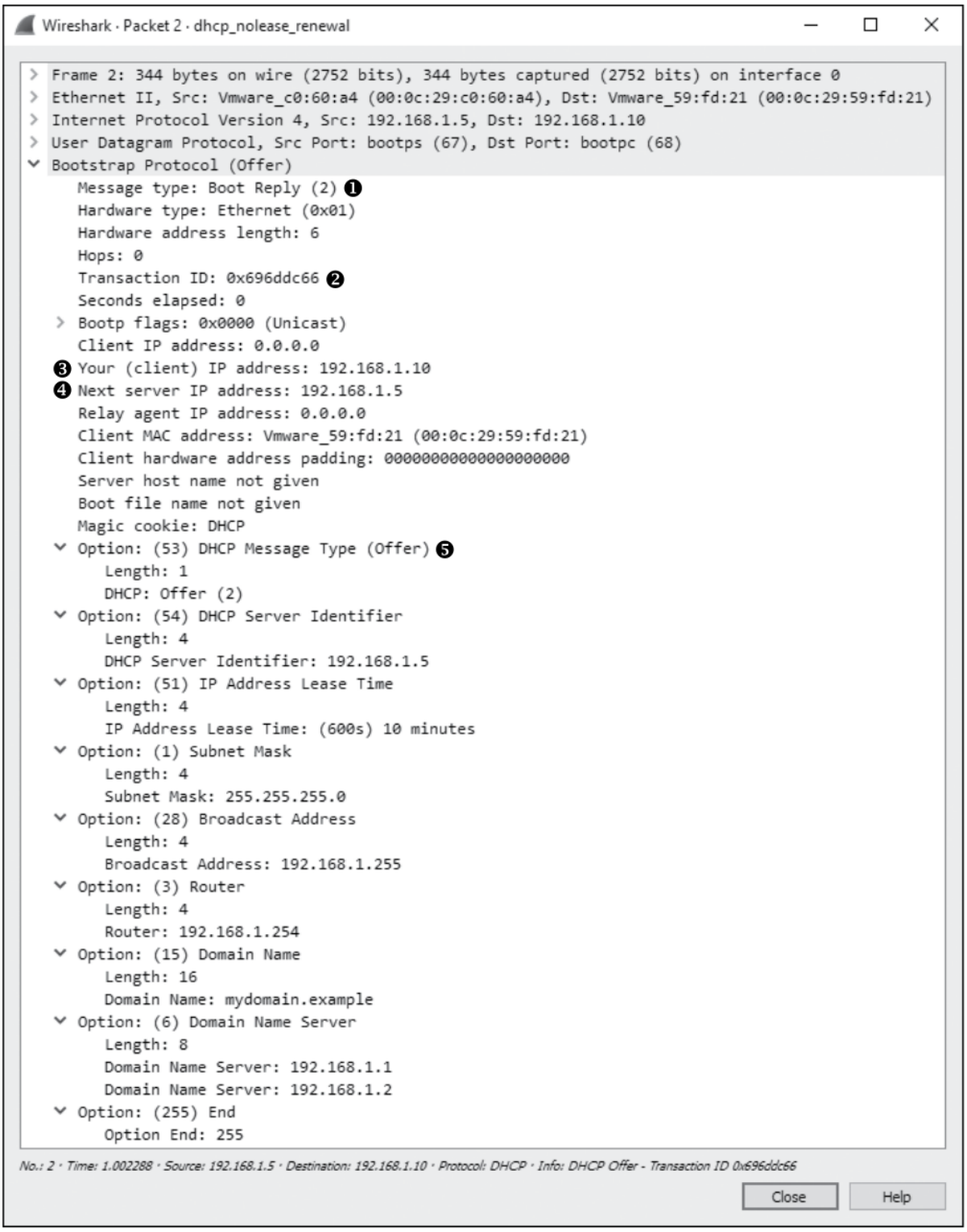


Рисунок 9-4. Пакет предложения DHCP

Пакет запроса

Получив предложение от сервера DHCP, клиент должен принять его пакетом запроса DHCP, как показано на рис. 9-5.

Третий пакет захвата всё ещё поступает с IP-адреса `0.0.0.0`, поскольку процесс получения IP-адреса пока не завершён (метка 1). Однако теперь пакет знает сервер DHCP, с которым обменивается данными.

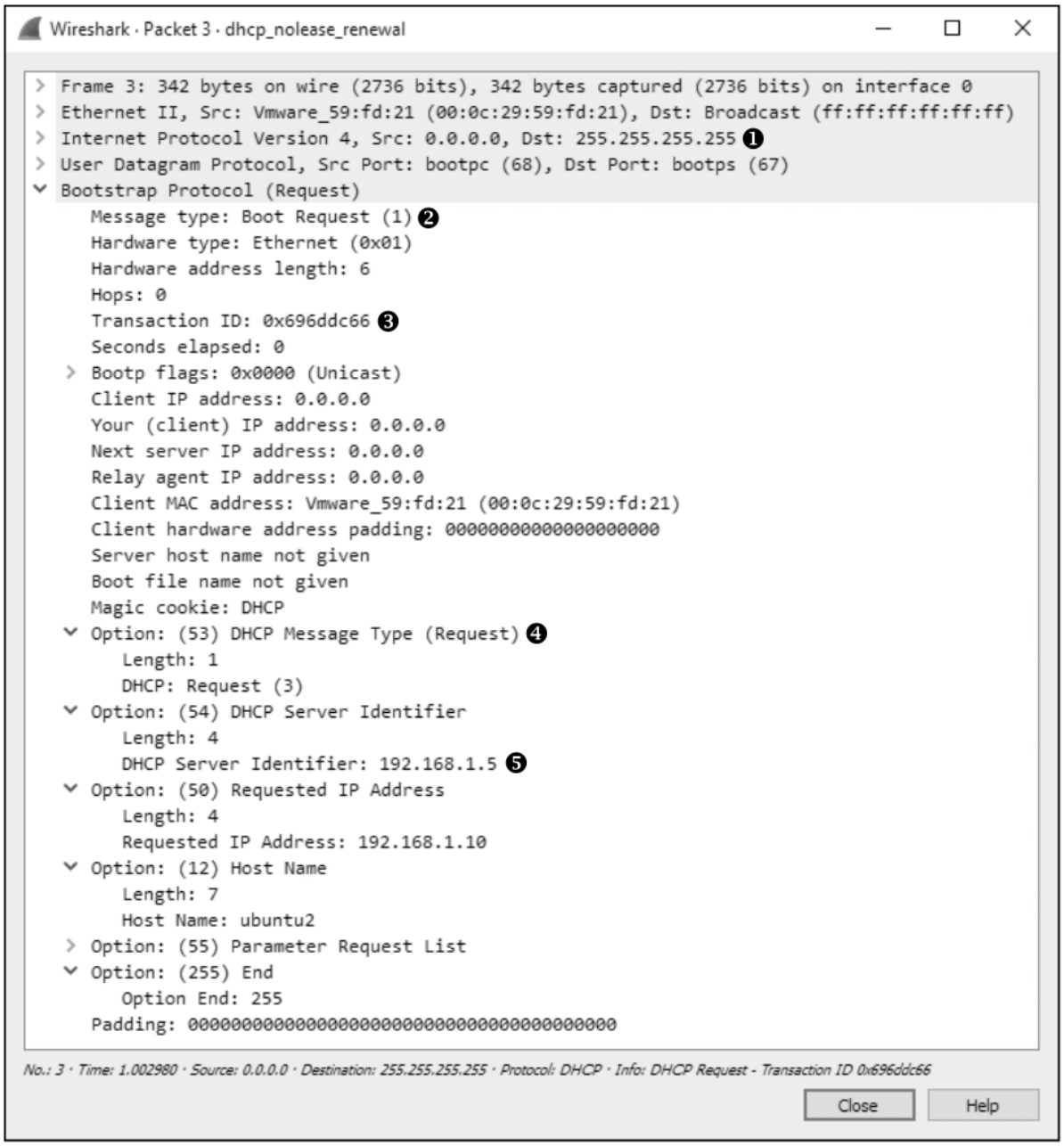


Рисунок 9-5. Пакет запроса DHCP

Поле Message type показывает, что пакет является запросом (метка 2), а поле Transaction ID совпадает с первыми двумя пакетами (метка 3), то есть все они относятся к одному процессу. Этот пакет похож на пакет обнаружения тем, что все поля IP-адресации заполнены нулями.

Наконец, в полях Option видно, что это DHCP Request (метка 4). Обратите внимание: запрошенный IP-адрес уже не пуст, а поле DHCP Server Identifier также содержит адрес (метка 5).

Пакет подтверждения

На последнем этапе процесса сервер DHCP отправляет клиенту запрошенные IP-адреса в пакете подтверждения и записывает эти сведения в свою базу данных, как показано на рис. 9-6. Теперь у клиента есть IP-адрес, и он может начать обмениваться данными в сети.

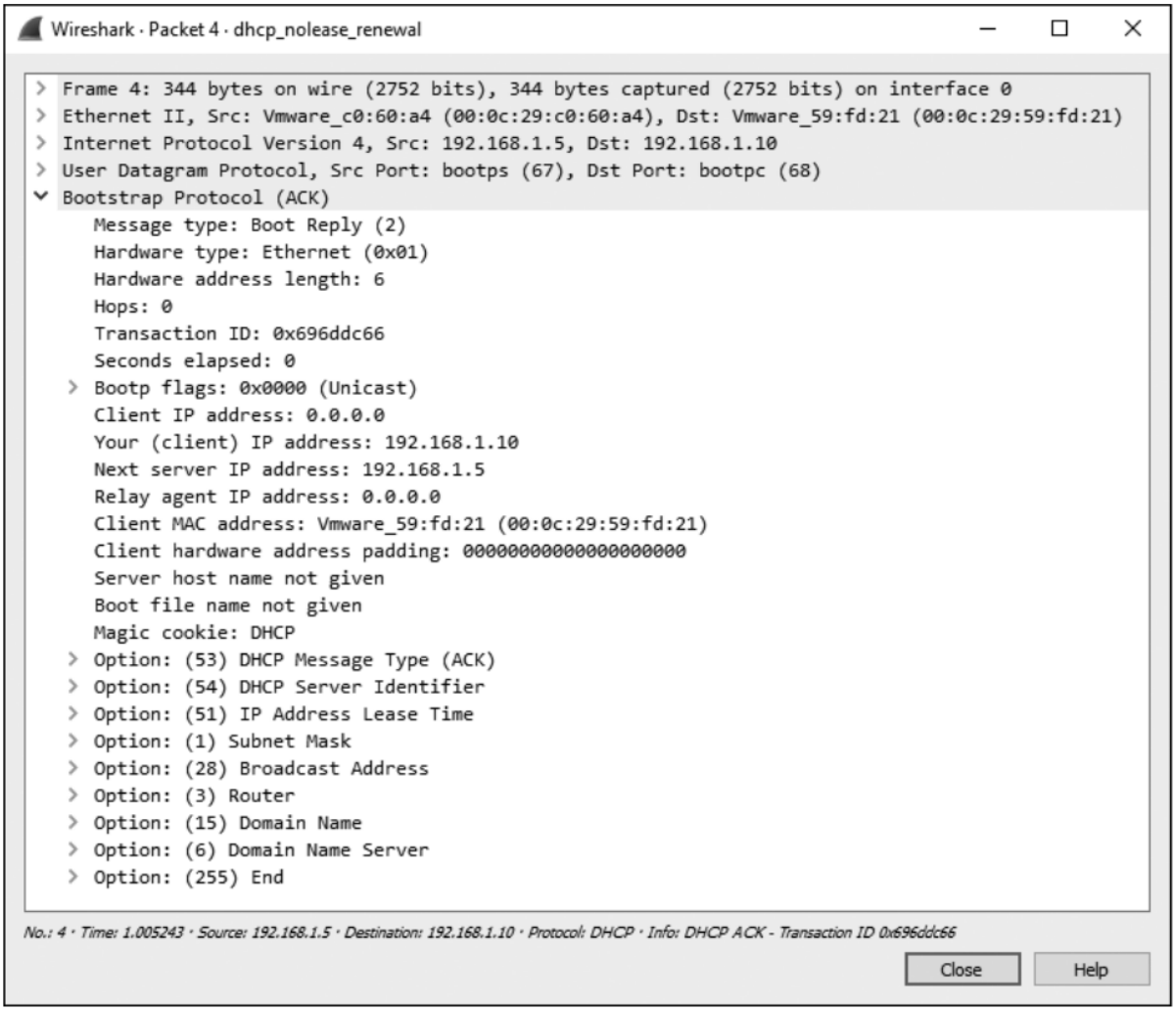


Рисунок 9-6. Пакет подтверждения DHCP

Обновление DHCP во время аренды

Когда сервер DHCP назначает устройству IP-адрес, он передаёт его клиенту в аренду. Это означает, что клиенту разрешено использовать IP-адрес лишь ограниченное время, после чего аренду необходимо обновить. Только что рассмотренный процесс DORA выполняется при первом получении клиентом IP-адреса или после истечения срока аренды. В обоих случаях устройство считается находящимся вне аренды.

Когда клиент с действующим арендованным IP-адресом перезагружается, для повторного получения своего IP-адреса он должен выполнить сокращённую версию процесса DORA. Этот процесс называется обновлением во время аренды.

При обновлении аренды пакеты обнаружения и предложения не нужны. Обновление во время аренды можно считать тем же процессом DORA, что применяется при обновлении вне аренды, но ему требуется выполнить меньше действий: остаются только запрос и подтверждение. Пример

захвата обновления во время аренды находится в файле `dhcp_inlease_renewal.pcapng`.

Параметры и типы сообщений DHCP

Настоящая гибкость DHCP заключается в доступных параметрах. Как вы уже видели, параметры DHCP пакета могут различаться по размеру и содержимому. Общий размер пакета зависит от сочетания используемых параметров. Полный список многочисленных параметров DHCP находится по адресу iana.org.

Единственный параметр, обязательный во всех пакетах DHCP, - тип сообщения (параметр 53). Он определяет, как клиент или сервер DHCP будет обрабатывать содержащиеся в пакете сведения. Существует восемь типов сообщений, перечисленных в табл. 9-1.

Таблица 9-1. Типы сообщений DHCP

Номер типа	Тип сообщения	Описание
1	Discover	Используется клиентом для поиска доступных серверов DHCP
2	Offer	Отправляется сервером клиенту в ответ на пакет обнаружения
3	Request	Отправляется клиентом для запроса предложенных сервером параметров
4	Decline	Отправляется клиентом серверу, чтобы указать на недопустимые параметры в пакете
5	ACK	Отправляется сервером клиенту с запрошенными параметрами конфигурации
6	NAK	Отправляется клиентом серверу для отклонения запроса параметров конфигурации
7	Release	Отправляется клиентом серверу для отмены аренды путём освобождения параметров конфигурации
8	Inform	Отправляется клиентом серверу для запроса параметров конфигурации, когда у клиента уже есть IP-адрес

DHCP версии 6 (DHCPv6)

Из структуры пакета DHCP на рис. 9-1 видно, что в нём недостаточно места для адресов той длины, которая требуется при распределении IPv6. Вместо доработки DHCP для этой цели в RFC 3315 был разработан DHCPv6. Поскольку DHCPv6 не основан на концепции BOOTP, формат его пакета намного проще (рис. 9-7).

Dynamic Host Configuration Protocol Version 6 (DHCPv6)					
Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	Message Type	Transaction ID		
4+	32+	Options			

Рисунок 9-7. Структура пакета DHCPv6

Показанная здесь структура пакета содержит только два постоянных значения, работающих так же, как их аналоги в DHCP. Остальная структура меняется в зависимости от типа сообщения, указанного в первом байте. В разделе Options каждый параметр определяется 2-байтовым кодом параметра и 2-байтовым полем длины. Полный список типов сообщений и кодов параметров, которые могут появляться в этих полях, находится по адресу iana.org.

DHCPv6 решает ту же задачу, что и DHCP, но для понимания потока обмена DHCPv6 аббревиатуру DORA необходимо заменить новой - SARR. Процесс показан на рис. 9-8 для клиента, который в данный момент находится вне аренды.

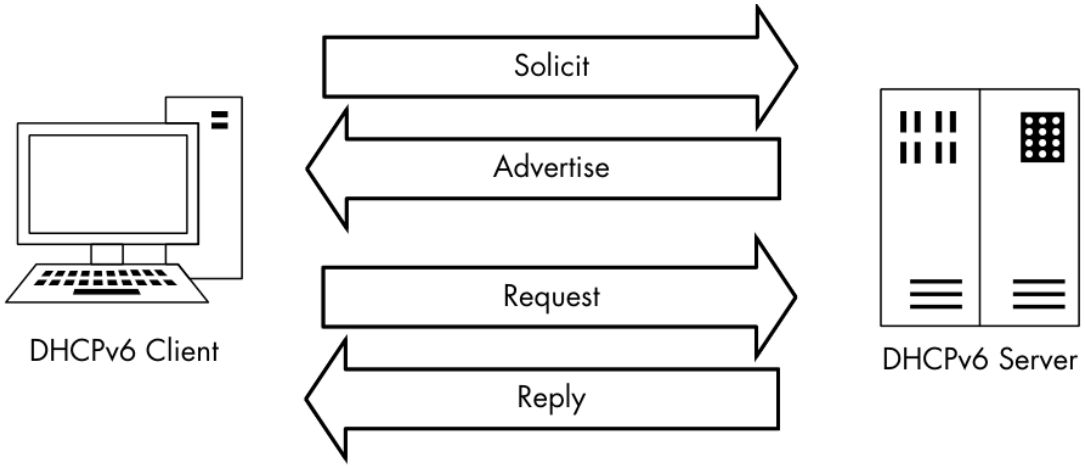
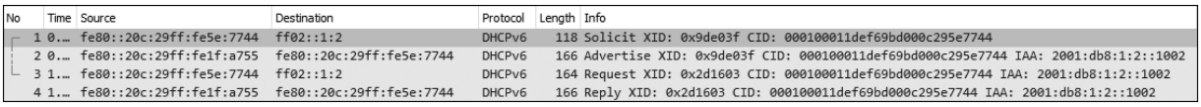


Рисунок 9-8. Процесс SARR DHCPv6 при обновлении вне аренды

Процесс SARR состоит из четырёх этапов:

1. **Solicit (поиск).** Клиент отправляет исходный пакет на специальный многоадресный адрес (`ff02::1:2`), пытаясь найти доступные серверы DHCPv6 в сети.
2. **Advertise (объявление).** Доступный сервер отвечает непосредственно клиенту, сообщая о готовности предоставить адресные и конфигурационные сведения.
3. **Request (запрос).** Клиент посредством многоадресной рассылки отправляет серверу формальный запрос конфигурационных сведений.
4. **Reply (ответ).** Сервер отправляет непосредственно клиенту все доступные запрошенные конфигурационные сведения, после чего процесс завершается.

Сводка этого процесса показана на рис. 9-9, взятом из файла `dhcp6_outlease_acquisition.pcapng`. В примере процесс SARR выполняется, когда новый сетевой узел (`fe80::20c:29ff:fe5e:7744`) получает конфигурационные сведения от сервера DHCPv6 (`fe80::20c:29ff:fe1f:a755`). Каждый пакет представляет один этап SARR: исходные пакеты solicit и advertise связаны идентификатором транзакции `0x9de03f`, а пакеты request и reply - идентификатором `0x2d1603`. Хотя на рисунке это не показано, обмен происходит через порты 546 и 547, стандартные для DHCPv6.



No	Time	Source	Destination	Protocol	Length	Info
1	0.…	fe80::20c:29ff:fe5e:7744	ff02::1:2	DHCPv6	118	Solicit XID: 0x9de03f CID: 000100011def69bd000c295e7744
2	0.…	fe80::20c:29ff:fe1f:a755	fe80::20c:29ff:fe5e:7744	DHCPv6	166	Advertise XID: 0x9de03f CID: 000100011def69bd000c295e7744 IAA: 2001:db8:1:2::1002
3	1.…	fe80::20c:29ff:fe5e:7744	ff02::1:2	DHCPv6	164	Request XID: 0x2d1603 CID: 000100011def69bd000c295e7744 IAA: 2001:db8:1:2::1002
4	1.…	fe80::20c:29ff:fe1f:a755	fe80::20c:29ff:fe5e:7744	DHCPv6	166	Reply XID: 0x2d1603 CID: 000100011def69bd000c295e7744 IAA: 2001:db8:1:2::1002

Рисунок 9-9. Получение клиентом IPv6-адреса посредством DHCPv6

В целом структура пакетов трафика DHCPv6 выглядит совсем иначе, но большинство концепций остаётся прежним. Процесс по-прежнему требует той или иной формы обнаружения сервера DHCP и формального получения конфигурационных сведений. Все транзакции связываются идентификаторами в каждой паре пакетов, которыми обмениваются клиент и сервер. Традиционные механизмы DHCP не поддерживают адресацию IPv6, поэтому, если устройства в вашей сети автоматически получают IPv6-адреса от сервера, вероятно, в сети уже работают службы DHCPv6. Для дальнейшего сравнения DHCP и DHCPv6 рекомендую открыть рядом захваты пакетов, рассмотренные в этой главе, и пройти их по шагам.

Система доменных имён (DNS)

Система доменных имён (Domain Name System, DNS) - один из важнейших протоколов Интернета, поскольку служит тем самым связующим веществом, которое удерживает всё вместе. DNS связывает доменные имена, например `www.google.com`, с IP-адресами, например `74.125.159.99`. Когда мы хотим связаться с сетевым устройством и не знаем его IP-адреса, то обращаемся к нему по имени DNS.

Серверы DNS хранят базу ресурсных записей с сопоставлениями IP-адресов и имён DNS и предоставляют её клиентам и другим серверам DNS.

> **Примечание.** Поскольку архитектура серверов DNS сложна, мы рассмотрим лишь несколько распространённых типов трафика DNS. Различные RFC, относящиеся к DNS, можно изучить по адресу isc.org.

Структура пакета DNS

Как видно на рис. 9-10, структура пакета DNS несколько отличается от ранее рассмотренных типов пакетов. В пакете DNS могут присутствовать следующие поля:

Идентификатор DNS (DNS ID Number). Служит для сопоставления запросов DNS с ответами DNS.

Запрос/ответ (Query/Response, QR). Указывает, является пакет запросом или ответом DNS.

Код операции (OpCode). Определяет тип запроса, содержащегося в сообщении.

Авторитетные ответы (Authoritative Answers, AA). Если значение установлено в пакете ответа, указывает, что ответ поступил от сервера имён, авторитетного для домена.

Усечение (Truncation, TC). Указывает, что ответ был усечён, поскольку оказался слишком большим для пакета.

Требуется рекурсия (Recursion Desired, RD). Если значение установлено в запросе, указывает, что клиент DNS просит выполнить рекурсивный запрос, когда целевой сервер имён не содержит запрошенных сведений.

Рекурсия доступна (Recursion Available, RA). Если значение установлено в ответе, указывает, что сервер имён поддерживает рекурсивные запросы.

Domain Name System (DNS)															
Offsets	Octet	0				1				2				3	
Octet	Bit	0–7				8–15				16–23				24–31	
0	0	DNS ID Number								QR	OpCode	AA	TC	RD	RA	Z	RCode
4	32	Question Count								Answer Count					
8	64	Name Server (Authority) Record Count								Additional Records Count					
12+	96+	Questions Section								Answers Section					
		Authority Section								Additional Information Section					

Рисунок 9-10. Структура пакета DNS

Зарезервировано (Reserved, Z). Согласно RFC 1035 должно быть полностью заполнено нулями, однако иногда используется как расширение поля RCode.

Код ответа (Response Code, RCode). Применяется в ответах DNS для указания наличия ошибок.

Число вопросов (Question Count). Число записей в разделе Questions.

Число ответов (Answer Count). Число записей в разделе Answers.

Число записей сервера имён (авторитетных записей) (Name Server (Authority) Record Count). Число ресурсных записей сервера имён в разделе Authority.

Число дополнительных записей (Additional Records Count). Число прочих ресурсных записей в разделе Additional Information.

Раздел вопросов (Questions Section). Раздел переменного размера, содержащий один или несколько запросов сведений, отправляемых серверу DNS.

Раздел ответов (Answers Section). Раздел переменного размера, несущий одну или несколько ресурсных записей с ответами на запросы.

Раздел авторитетных сведений (Authority Section). Раздел переменного размера с ресурсными записями, указывающими на авторитетные серверы имён, которые можно использовать для продолжения разрешения.

Раздел дополнительных сведений (Additional Information Section). Раздел переменного размера с ресурсными записями, содержащими дополнительные сведения, относящиеся к запросу, но не являющиеся совершенно необходимыми для ответа.

Простой запрос DNS

DNS работает в формате «запрос - ответ». Клиент, желающий разрешить имя DNS в IP-адрес, отправляет запрос серверу DNS, а сервер возвращает запрошенные сведения в ответе. В простейшем случае процесс занимает два пакета, как видно в файле захвата `dns_query_response.pcapng`.

Первый пакет, показанный на рис. 9-11, представляет запрос DNS от клиента `192.168.0.114` серверу `205.152.37.23` через порт 53 - стандартный порт DNS.

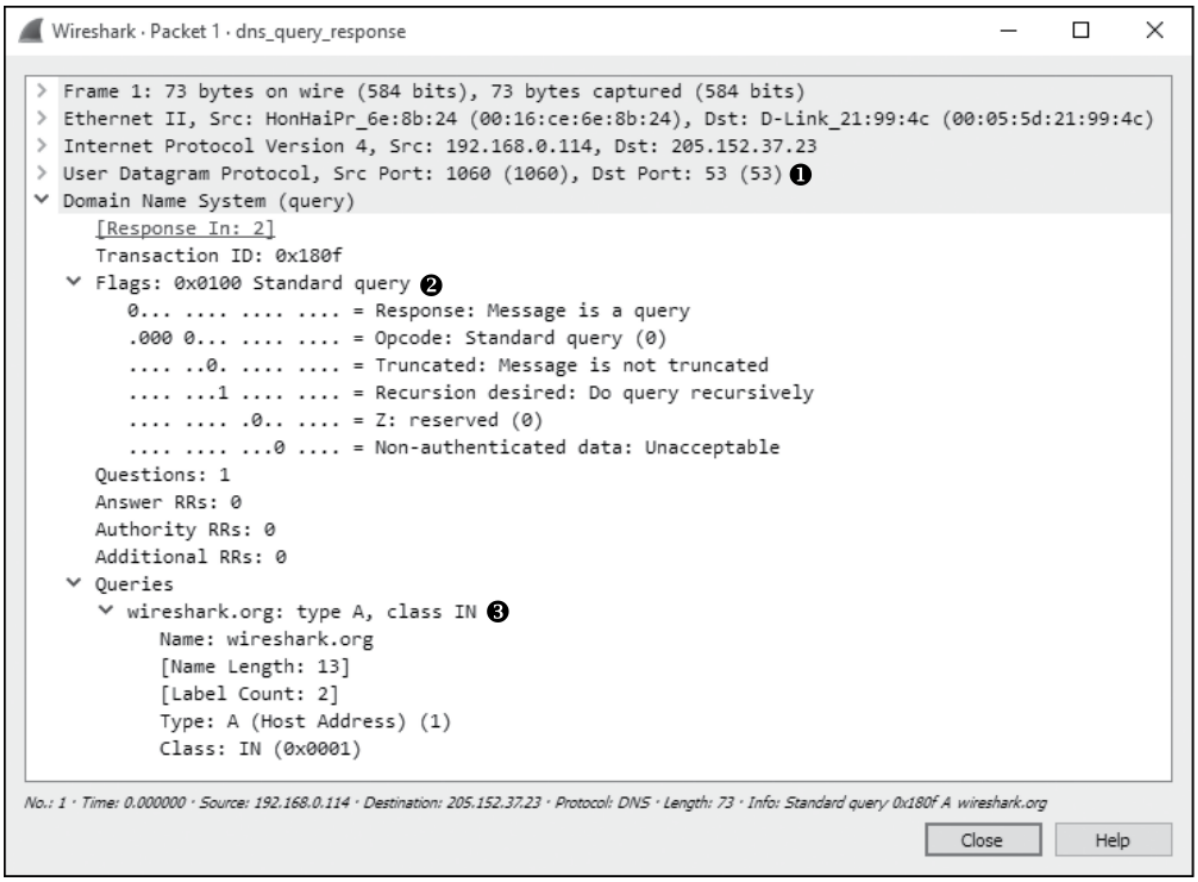


Рисунок 9-11. Пакет запроса DNS

Начав исследовать заголовки пакета, вы увидите, что DNS также полагается на UDP (метка 1).

В разделе DNS пакета видно, что Wireshark объединяет небольшие поля в начале пакета в один раздел Flags. Развернув его, вы увидите, что сообщение действительно является стандартным запросом (метка 2), не усечено и требует рекурсии (она будет рассмотрена чуть позже). Указан только один вопрос, который можно найти, развернув раздел Queries. Там видно, что запрашивается имя `wireshark.org` для адреса узла (тип A) Интернета (IN) (метка 3). По сути, пакет спрашивает: «Какой IP-адрес связан с доменом `wireshark.org`?»

Ответ на запрос содержится в пакете 2, показанном на рис. 9-12. Поскольку у пакета тот же идентификационный номер (метка 1), мы знаем, что он содержит правильный ответ на исходный запрос.

Раздел Flags подтверждает, что это ответ и что при необходимости доступна рекурсия (метка 2). Пакет содержит только один вопрос и одну ресурсную запись (метка 3), поскольку включает исходный вопрос вместе с ответом. Развернув раздел Answers, мы получаем ответ на запрос: IP-адрес `wireshark.org` - `128.121.50.122` (метка 4). С этими сведениями клиент теперь может строить пакеты IP и начать обмен с `wireshark.org`.

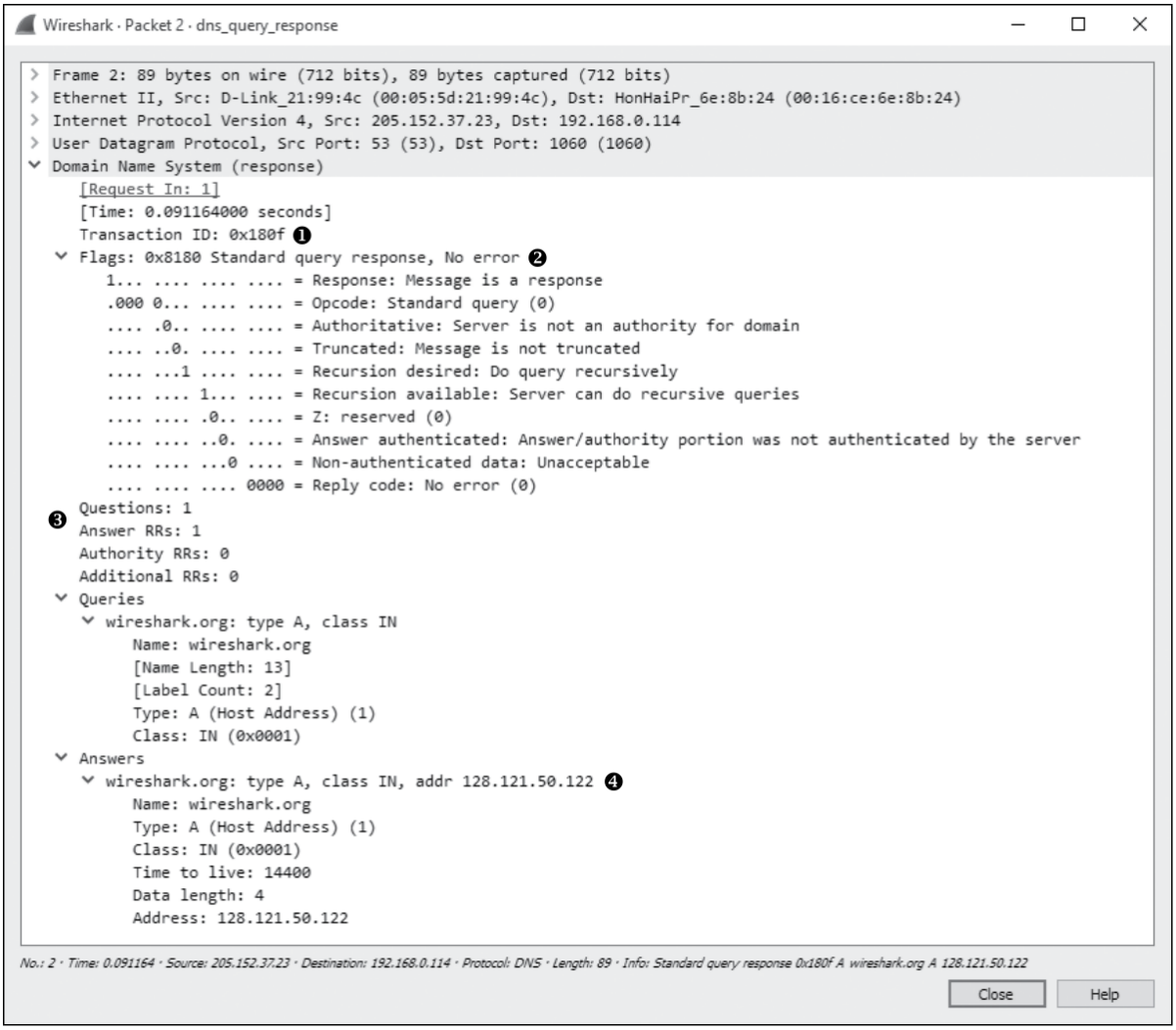


Рисунок 9-12. Пакет ответа DNS

Типы вопросов DNS

Поля Type в запросах и ответах DNS указывают тип ресурсной записи, к которой относится запрос или ответ. Некоторые распространённые типы сообщений/ресурсных записей перечислены в табл. 9-2. Вы будете встречать их в обычном трафике и далее в книге. (Список в табл. 9-2 краток и далеко не исчерпывающий. Все типы ресурсных записей DNS см. по адресу iana.org.)

Таблица 9-2. Распространённые типы ресурсных записей DNS

Значение	Тип	Описание
1	A	Адрес узла IPv4
2	NS	Авторитетный сервер имён
5	CNAME	Каноническое имя псевдонима
15	MX	Почтовый обменник
16	TXT	Текстовая строка
28	AAAA	Адрес узла IPv6
251	IXFR	Инкрементная передача зоны

Значение	Тип	Описание
252	AXFR	Полная передача зоны

Рекурсия DNS

Из-за иерархической структуры DNS в Интернете серверы DNS должны обмениваться данными друг с другом, чтобы отвечать на запросы клиентов. Можно ожидать, что внутренний сервер DNS знает сопоставление имени и IP-адреса локального сервера интрасети, но нельзя ожидать от него знания IP-адреса Google или Dell.

Когда серверу DNS нужно найти IP-адрес, он от имени запрашивающего клиента обращается к другому серверу DNS, фактически выступая в роли клиента. Этот процесс называется рекурсией.

Чтобы увидеть процесс рекурсии с точки зрения клиента и сервера DNS, откройте файл `dns_recursivequery_client.pcapng`. Он содержит захват трафика DNS клиента из двух пакетов. Первый пакет - исходный запрос, отправленный клиентом DNS `172.16.0.8` своему серверу DNS `172.16.0.102`, как показано на рис. 9-13.

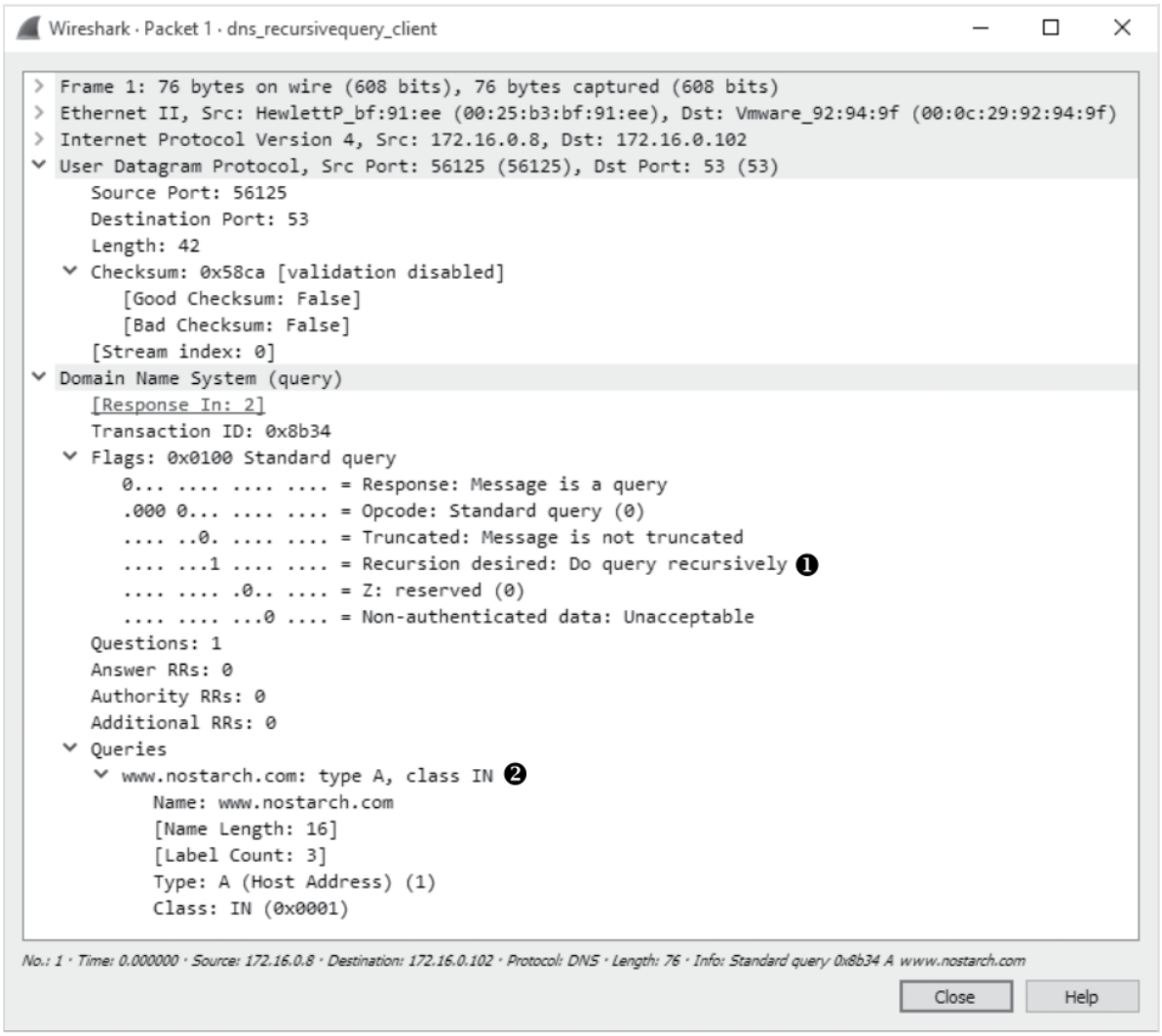


Рисунок 9-13. Запрос DNS с установленным битом Recursion desired

Развернув раздел DNS пакета, вы увидите, что это стандартный запрос записи типа A для DNS-имени `www.nostarch.com` (метка 2). Чтобы узнать о пакете больше, разверните раздел Flags:

там видно, что требуется рекурсия (метка 1).

Второй пакет - ожидаемый ответ на исходный запрос, показанный на рис. 9-14.

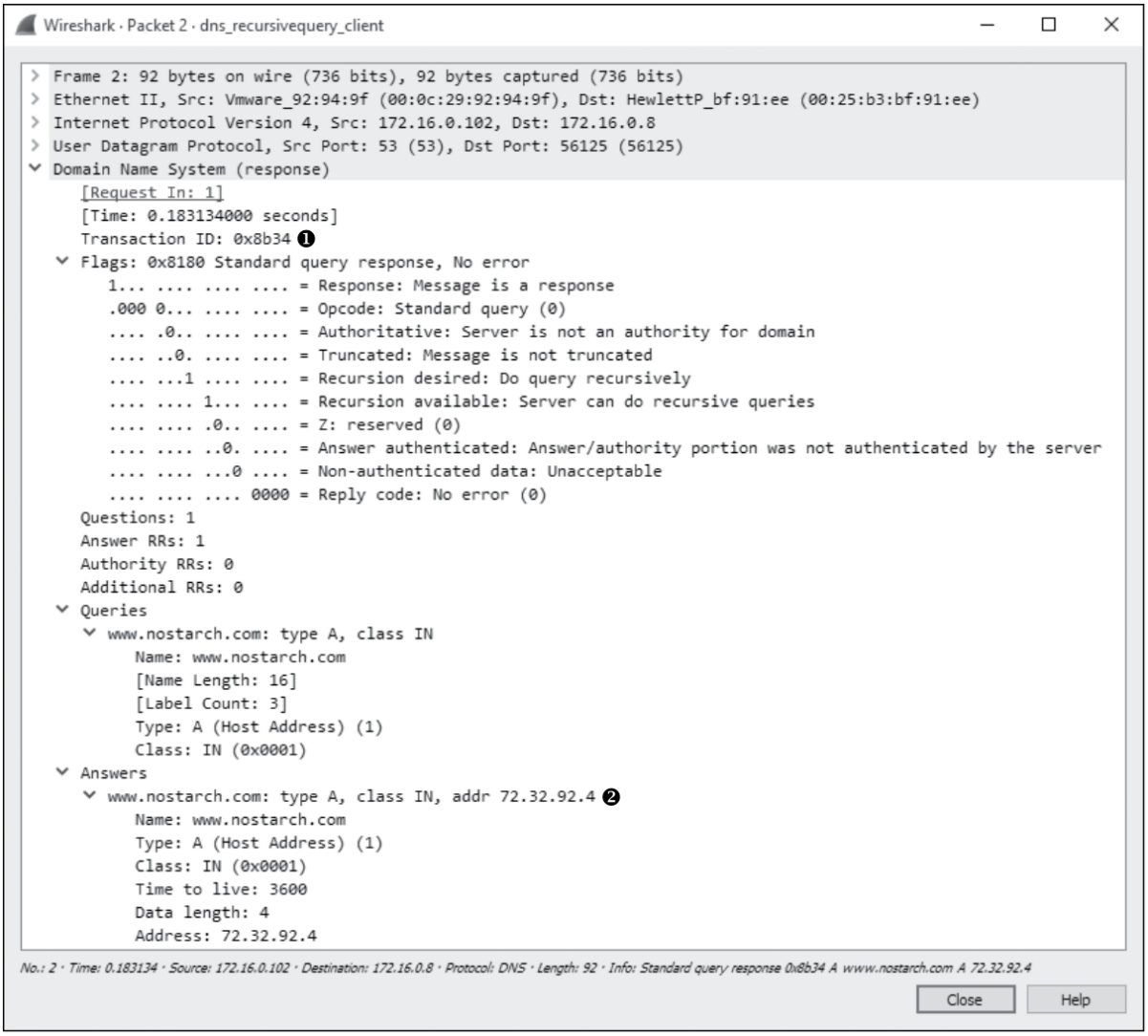
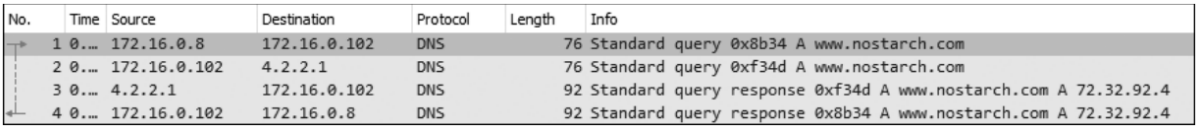


Рисунок 9-14. Ответ на запрос DNS

Идентификатор транзакции пакета совпадает с запросом (метка 1), ошибки не указаны, и мы получаем ресурсную запись типа A, связанную с `www.nostarch.com` (метка 2).

Убедиться, что ответ на запрос был получен посредством рекурсии, можно, наблюдая трафик сервера DNS во время её выполнения, как показано в файле `dns_recursivequery_server.pcapng`. В нём захвачен трафик локального сервера DNS при инициировании запроса (рис. 9-15).



No.	Time	Source	Destination	Protocol	Length	Info
1	0.…	172.16.0.8	172.16.0.102	DNS	76	Standard query 0x8b34 A www.nostarch.com
2	0.…	172.16.0.102	4.2.2.1	DNS	76	Standard query 0xf34d A www.nostarch.com
3	0.…	4.2.2.1	172.16.0.102	DNS	92	Standard query response 0xf34d A www.nostarch.com A 72.32.92.4
4	0.…	172.16.0.102	172.16.0.8	DNS	92	Standard query response 0x8b34 A www.nostarch.com A 72.32.92.4

Рисунок 9-15. Рекурсия DNS с точки зрения сервера

Первый пакет - тот же исходный запрос, который мы видели в предыдущем файле захвата. К этому моменту сервер DNS получил запрос, проверил локальную базу данных и понял, что не знает, какой IP-адрес соответствует DNS-имени `www.nostarch.com`. Поскольку пакет был отправлен с установленным битом Recursion desired, сервер DNS может задать этот вопрос другому серверу DNS в попытке найти ответ, как видно во втором пакете.

Во втором пакете сервер DNS `172.16.0.102` передаёт новый запрос серверу `4.2.2.1` (метка 1), на который он настроен пересылать восходящие запросы, как показано на рис. 9-16. Запрос повторяет исходный, фактически превращая сервер DNS в клиента. Определить, что это новый запрос, можно по идентификатору транзакции, отличному от идентификатора в предыдущем файле захвата (метка 2).

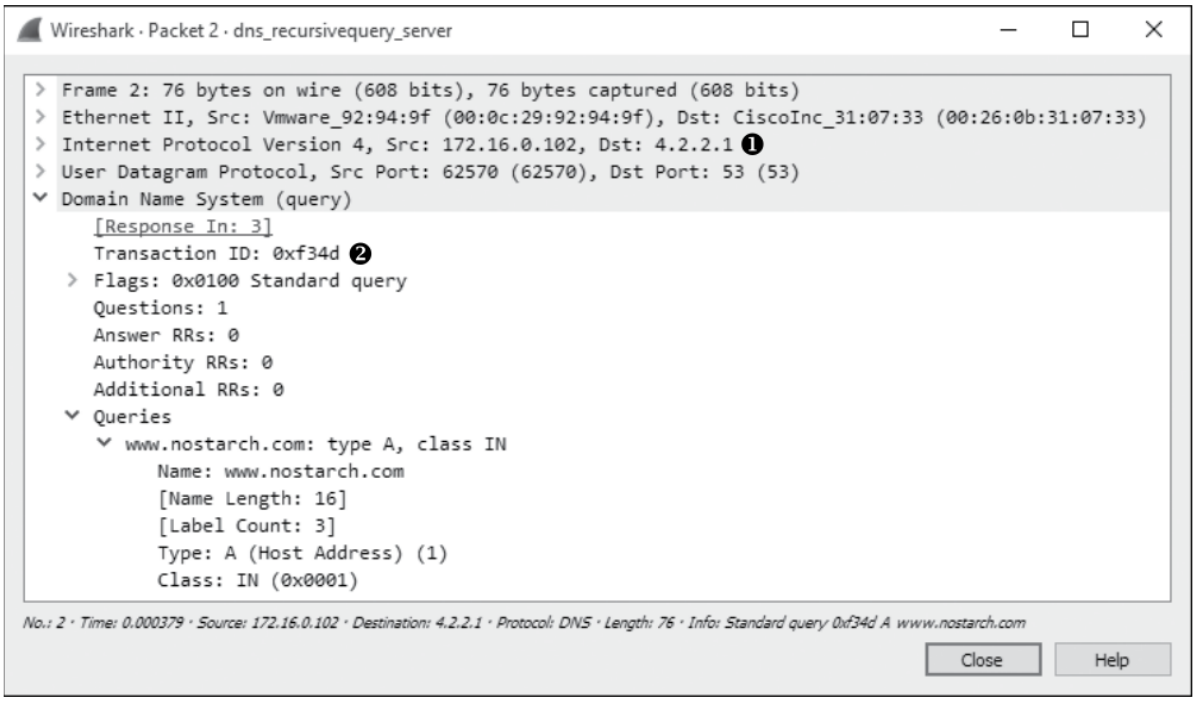


Рисунок 9-16. Рекурсивный запрос DNS

После получения этого пакета сервером `4.2.2.1` локальный сервер DNS получает ответ, показанный на рис. 9-17.

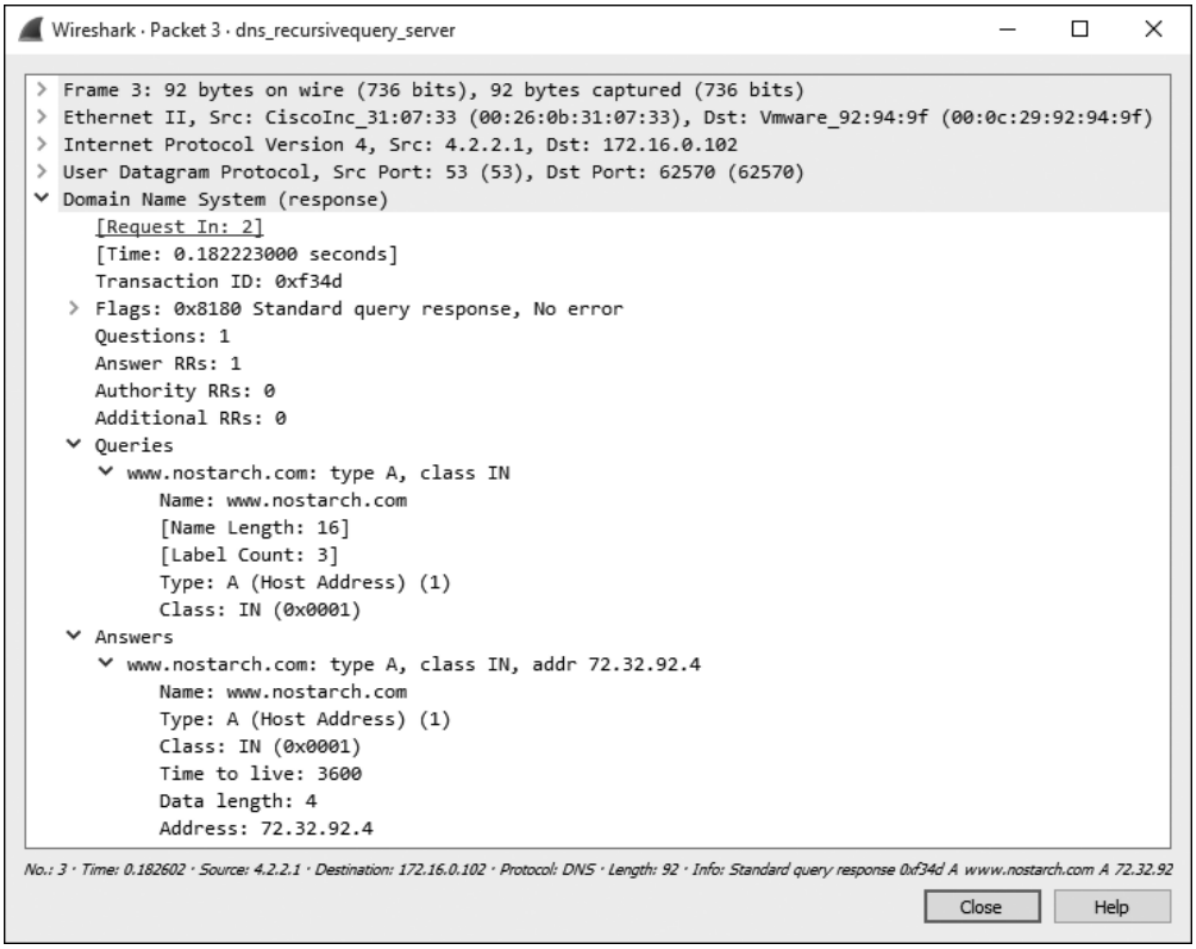


Рисунок 9-17. Ответ на рекурсивный запрос DNS

Получив ответ, локальный сервер DNS может передать клиенту DNS четвёртый и последний пакет с запрошенными сведениями.

Хотя в примере показан лишь один уровень рекурсии, для одного запроса DNS она может выполняться многократно. Здесь ответ поступил от сервера DNS `4.2.2.1`, но тот мог рекурсивно переслать запрос другому серверу, чтобы найти ответ. Простой запрос может обойти весь мир, прежде чем наконец получит правильный ответ. Процесс рекурсивного запроса DNS показан на рис. 9-18.

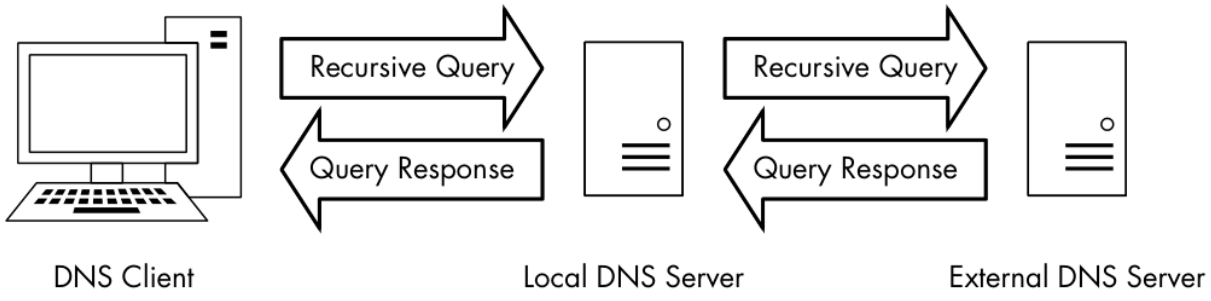


Рисунок 9-18. Рекурсивный запрос DNS

Передача зон DNS

Зона DNS - пространство имён (или группа DNS-имён), управление которым делегировано серверу DNS. Например, у ресторана «Emma's Diner» может быть один сервер DNS, отвечающий за `emmasdiner.com`. В таком случае устройства как внутри, так и за пределами ресторана, желающие разрешить `emmasdiner.com` в IP-адрес, должны обращаться к этому серверу DNS как авторитетному для зоны. Если ресторан вырастет, он может добавить второй сервер DNS для управления только почтовой частью пространства имён DNS, например `mail.emmasdiner.com`; этот сервер станет авторитетным для почтового поддомена. При необходимости можно добавить дополнительные серверы DNS для других поддоменов, как показано на рис. 9-19.

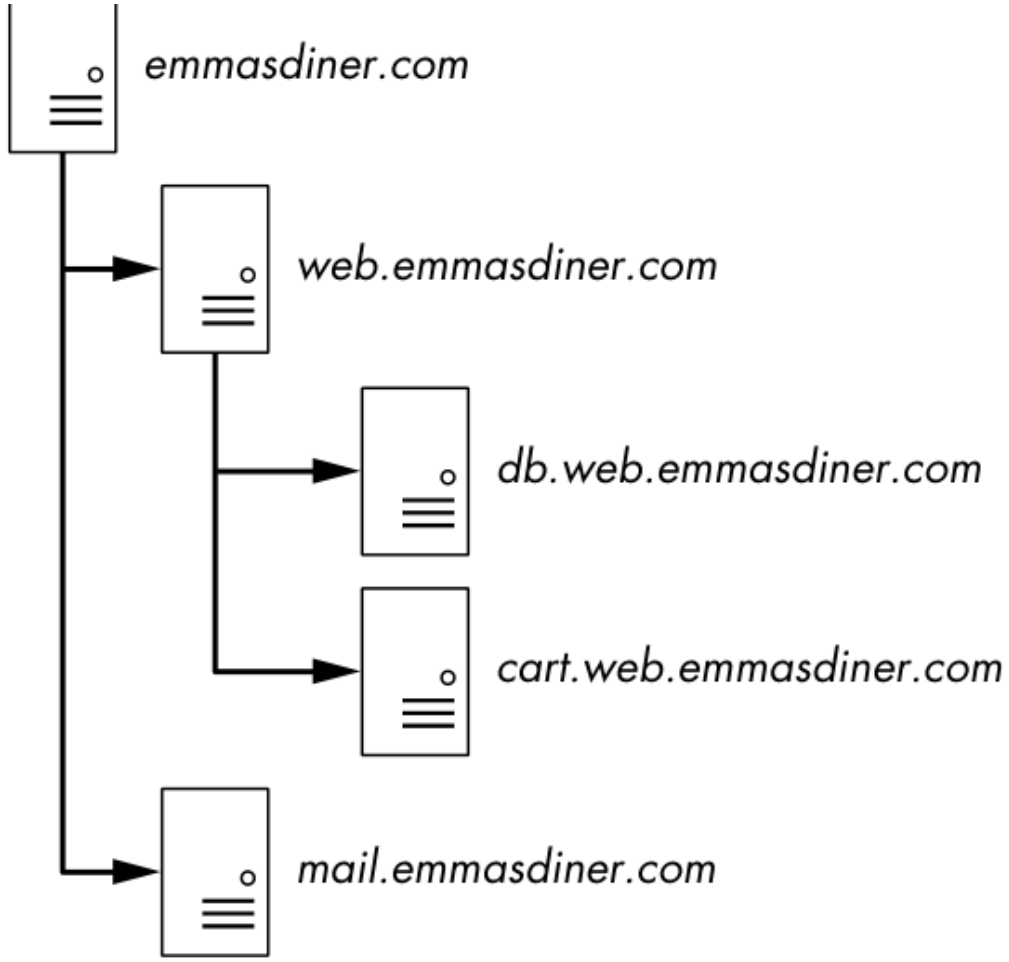


Рисунок 9-19. Зоны DNS распределяют ответственность за пространства имён

Передача зоны происходит, когда данные зоны переносятся между двумя устройствами, обычно для обеспечения избыточности. Например, в организациях с несколькими серверами DNS администраторы часто настраивают вторичный сервер DNS так, чтобы он хранил копию сведений о зоне DNS основного сервера на случай его недоступности. Существует два типа передачи зон:

Полная передача зоны (AXFR). При такой передаче между устройствами отправляется вся зона.

Инкрементная передача зоны (IXFR). При такой передаче отправляется только часть сведений зоны.

Файл `dns_axfr.pcapng` содержит пример полной передачи зоны между узлами `172.16.16.164` и `172.16.16.139`. При первом взгляде на файл может показаться, что открыт не тот захват, поскольку вместо пакетов UDP видны пакеты TCP. Хотя DNS полагается на UDP, для некоторых задач, например передачи зон, он использует TCP, потому что TCP надёжнее при таком объёме

передаваемых данных. Первые три пакета захвата составляют трёхэтапное рукопожатие TCP.

Четвёртый пакет начинает запрос передачи зоны между `172.16.16.164` и `172.16.16.139`. В нём нет сведений DNS. Он помечен как «TCP segment of a reassembled PDU», поскольку данные запроса передачи зоны отправлялись в нескольких пакетах. Пакеты 4 и 6 содержат данные пакета. Пакет 5 подтверждает получение пакета 4. Они отображаются так из-за способа, которым Wireshark разбирает и показывает пакеты TCP для повышения удобочитаемости. Для наших целей можно считать пакет 6 полным запросом передачи зоны DNS, показанным на рис. 9-20.

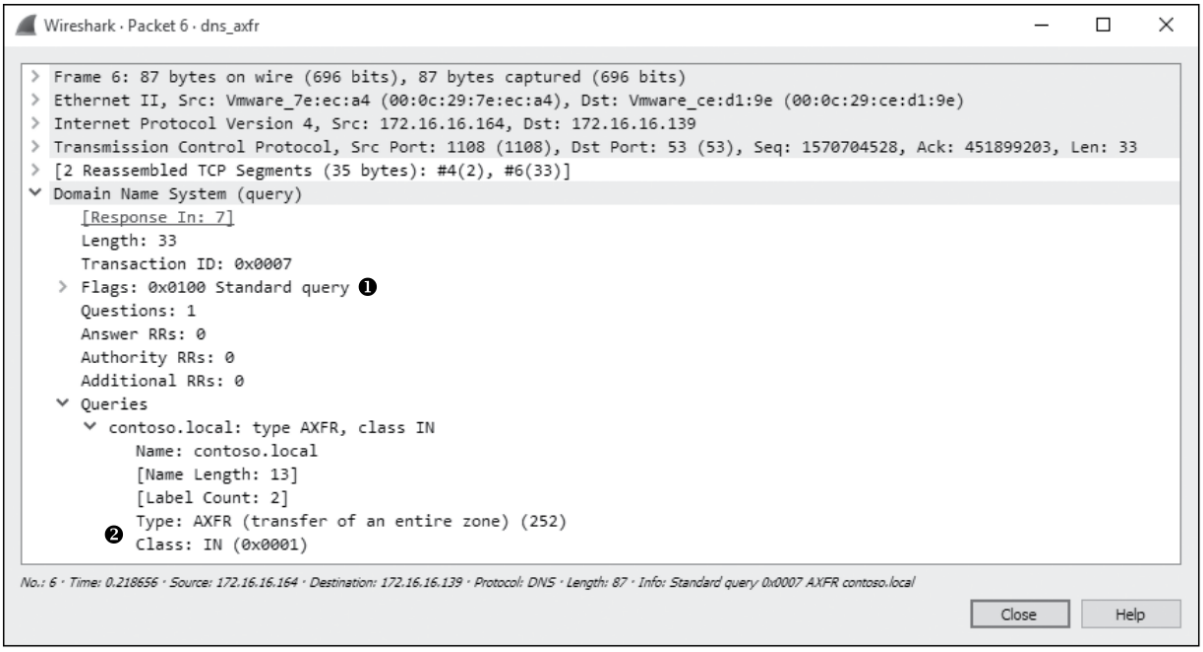


Рисунок 9-20. Запрос полной передачи зоны DNS

Запрос передачи зоны является стандартным запросом (метка 1), но вместо единственного типа записи он запрашивает тип AXFR (метка 2), то есть хочет получить от сервера всю зону DNS. Сервер отвечает записями зоны в пакете 7, как показано на рис. 9-21. Как видно, передача зоны содержит немало данных, а это один из простейших примеров! После завершения передачи зоны файл захвата заканчивается процессом закрытия TCP-соединения.

> **Предупреждение.** Данные, содержащиеся в передаче зоны, могут быть очень опасны в чужих руках. Например, перечислив содержимое единственного сервера DNS, можно составить карту всей инфраструктуры сети.

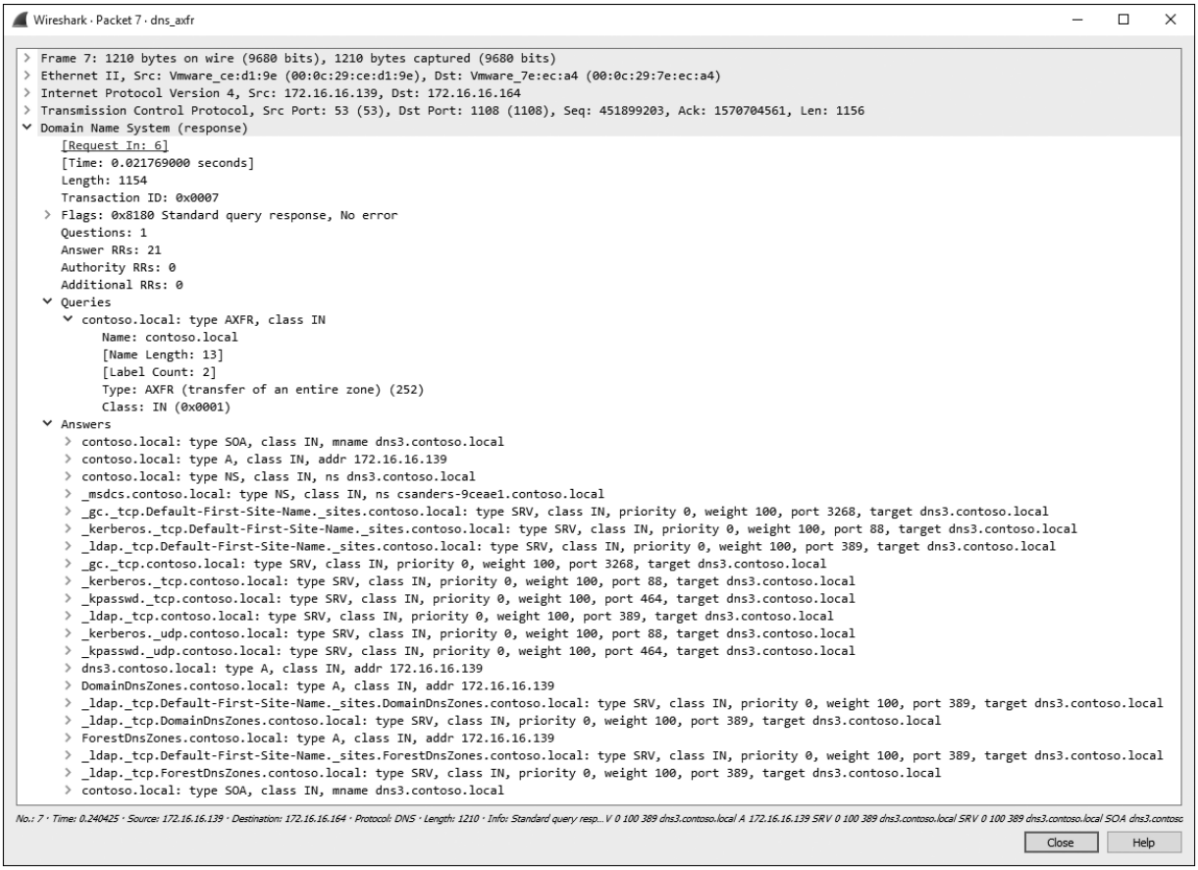


Рисунок 9-21. Выполняющаяся полная передача зоны DNS

Протокол передачи гипертекста (HTTP)

Протокол передачи гипертекста (Hypertext Transfer Protocol, HTTP) - механизм доставки Всемирной паутины, позволяющий веб-браузерам подключаться к веб-серверам для просмотра веб-страниц. В большинстве организаций HTTP с огромным отрывом составляет наибольшую долю трафика, проходящего по линии. Каждый раз, когда вы ищете что-либо в Google, отправляете твит или проверяете результаты баскетбольных матчей Университета Кентукки на espn.com, вы используете HTTP.

Мы не будем рассматривать структуру пакетов передачи HTTP, поскольку реализаций протокола HTTP настолько много, что структура может очень сильно различаться. Из-за этой изменчивости такое упражнение оставлено вам. Здесь мы рассмотрим некоторые практические применения HTTP, например получение и публикацию содержимого.

Просмотр веб-страниц посредством HTTP

Чаще всего HTTP применяется для просмотра в браузере веб-страниц на веб-сервере. Файл захвата `http_google.pcapng` показывает такую передачу HTTP с использованием TCP в качестве протокола транспортного уровня. Обмен начинается с трёхэтапного рукопожатия между клиентом `172.16.16.128` и веб-сервером Google `74.125.95.104`.

После установления соединения первый пакет отмечен как пакет HTTP от клиента серверу, как показано на рис. 9-22.

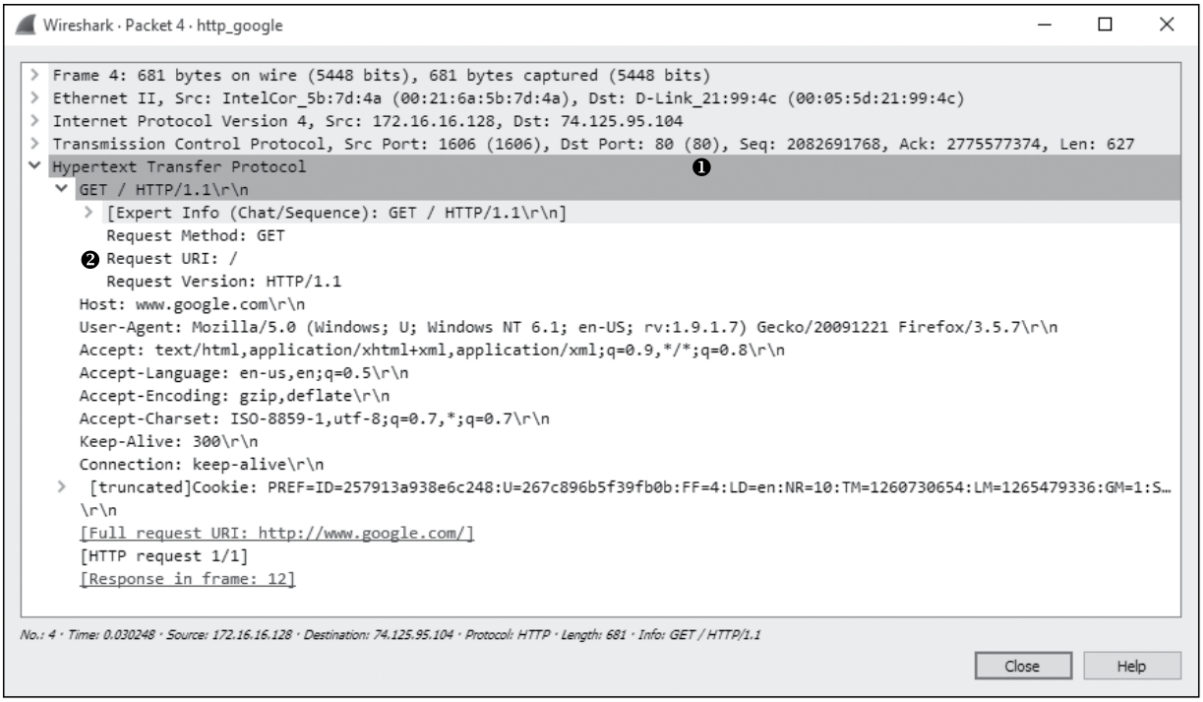


Рисунок 9-22. Исходный пакет HTTP-запроса GET

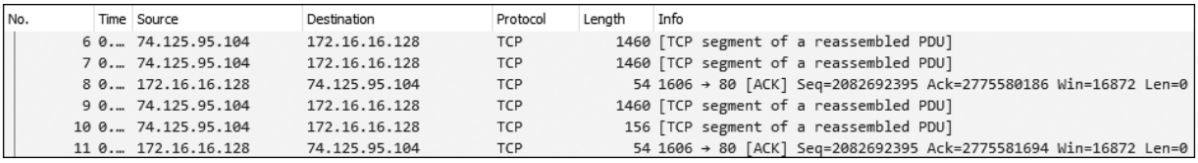
Пакет HTTP доставляется по TCP на порт 80 сервера (метка 1), стандартный порт обмена HTTP (часто используются и некоторые другие порты, например 8080 и 8888).

Пакеты HTTP определяются одним из восьми методов запроса, заданных в спецификации HTTP версии 1.1 (см. iana.org); метод указывает действие, которое передатчик пакета выполнит над получателем. Как показано на рис. 9-22, метод этого пакета - `GET`, унифицированный указатель ресурса (Uniform Resource Indicator, URI) запроса - `/`, а версия запроса - `HTTP/1.1` (метка 2). Эти сведения сообщают, что клиент отправляет запрос на загрузку (`GET`) корневого веб-каталога (`/`) веб-сервера посредством HTTP версии 1.1.

Затем узел отправляет веб-серверу сведения о себе. В их числе браузер (`User-Agent`), используемые языки, принимаемые браузером (`Accept-Languages`), и сведения cookie (внизу захвата). Сервер может применить эти данные, чтобы определить, какие сведения вернуть клиенту для обеспечения совместимости.

Получив HTTP-запрос `GET` в пакете 4, сервер отвечает подтверждением TCP ACK и начинает передавать запрошенные данные в пакетах 6-11. HTTP применяется только для выдачи команд прикладного уровня между клиентом и сервером. Почему же все эти пакеты HTTP отображаются как TCP в заголовке протокола списка пакетов? Когда начинается передача данных, окно списка пакетов Wireshark определяет их как TCP, а не HTTP, поскольку отдельные пакеты не содержат заголовков запроса/ответа HTTP. Поэтому там, где передаются данные, в столбце Protocol виден TCP вместо HTTP. Тем не менее это по-прежнему часть процесса обмена HTTP.

Сервер отправляет данные в пакетах 6 и 7, клиент подтверждает их в пакете 8, затем следуют ещё два пакета данных 9 и 10 и ещё одно подтверждение в пакете 11, как показано на рис. 9-23. Wireshark показывает все эти пакеты как сегменты TCP, а не пакеты HTTP, хотя за их передачу по-прежнему отвечает HTTP.



No.	Time	Source	Destination	Protocol	Length	Info
6	0.…	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]
7	0.…	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]
8	0.…	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082692395 Ack=2775580186 Win=16872 Len=0
9	0.…	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]
10	0.…	74.125.95.104	172.16.16.128	TCP	156	[TCP segment of a reassembled PDU]
11	0.…	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082692395 Ack=2775581694 Win=16872 Len=0

Рисунок 9-23. TCP передаёт данные между браузером клиента и веб-сервером

После передачи данных Wireshark повторно собирает поток данных для просмотра, как показано на рис. 9-24.

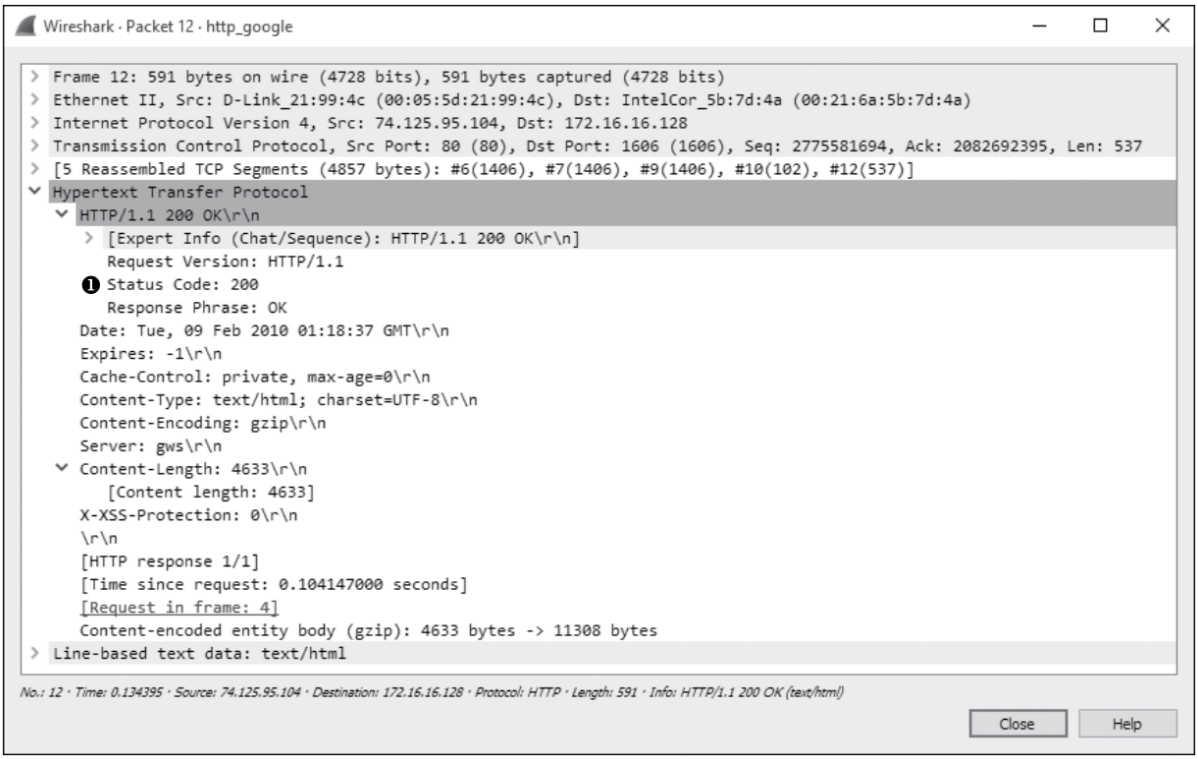


Рисунок 9-24. Заключительный пакет HTTP с кодом ответа 200

> **Примечание.** Во многих случаях при просмотре списка пакетов вы не увидите читаемых данных HTML, поскольку для повышения эффективности использования полосы пропускания они сжаты gzip. На это указывает поле Content-Encoding в ответе HTTP веб-сервера. Данные декодируются и становятся легко читаемыми лишь при просмотре полного потока.

HTTP использует ряд предопределённых кодов ответа для указания результатов метода запроса. В примере виден пакет с кодом состояния 200 (метка 1), означающим успешное выполнение метода запроса. Пакет также содержит метку времени и дополнительные сведения о кодировании содержимого и параметрах конфигурации веб-сервера. Когда клиент получает пакет, транзакция завершается.

Отправка данных посредством HTTP

Рассмотрев загрузку данных с веб-сервера, обратимся к отправке данных на него. Файл `http_post.pcapng` содержит очень простой пример загрузки: пользователь публикует комментарий на веб-сайте. После исходного трёхэтапного рукопожатия клиент (`172.16.16.128`) отправляет пакет HTTP веб-серверу (`69.163.176.56`), как показано на рис. 9-25.

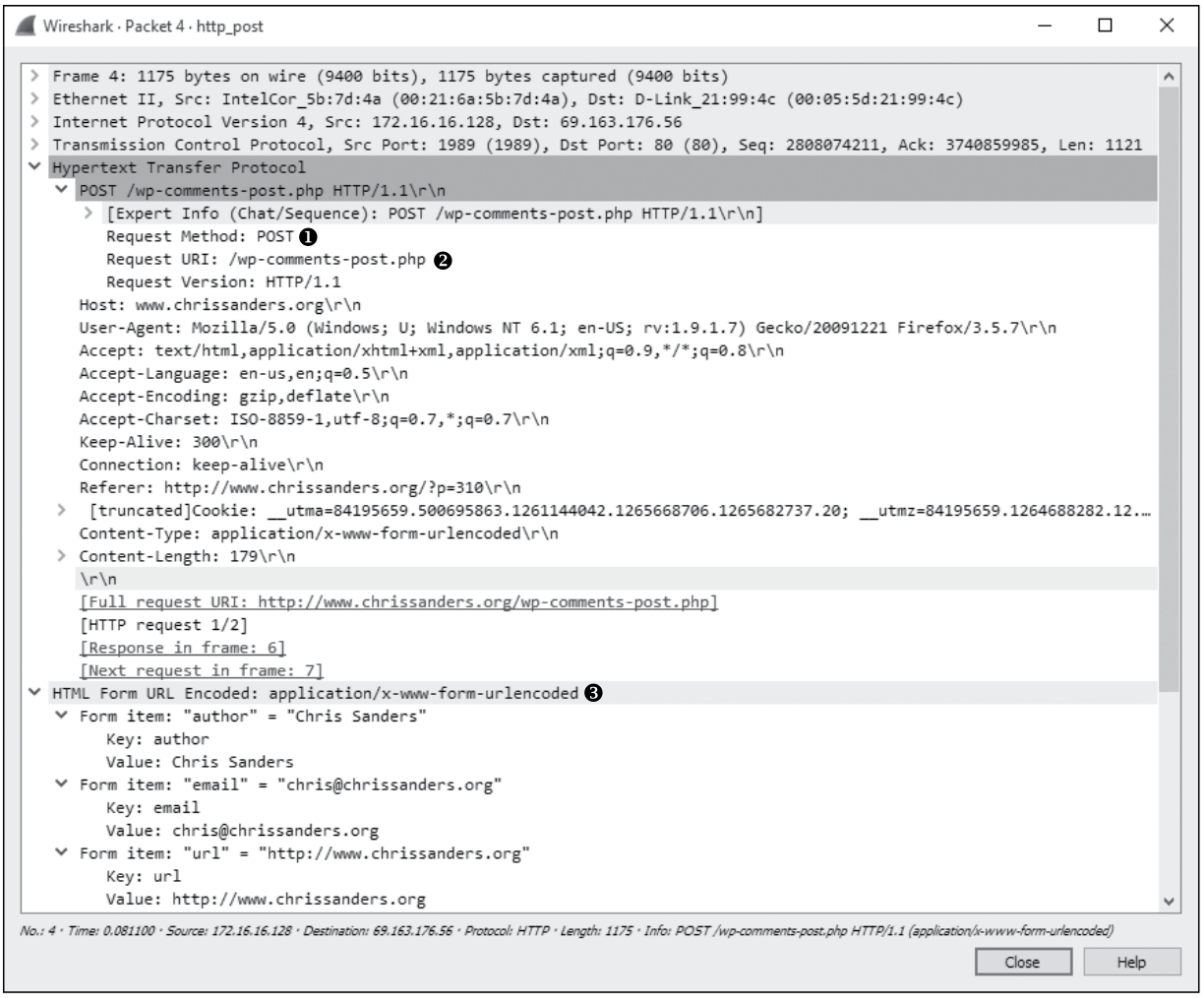


Рисунок 9-25. Пакет HTTP POST

Пакет использует метод `POST` (метка 1), чтобы загрузить данные на веб-сервер для обработки. Применённый здесь метод POST задаёт URI `/wp-comments-post.php` (метка 2) и версию HTTP `HTTP/1.1`. Чтобы увидеть содержимое опубликованных данных, разверните раздел HTML Form URL Encoded пакета (метка 3).

После передачи данных в этом POST отправляется пакет ACK. Как показано на рис. 9-26, сервер отвечает пакетом 6 с кодом ответа 302 (метка 1), означающим «найдено».

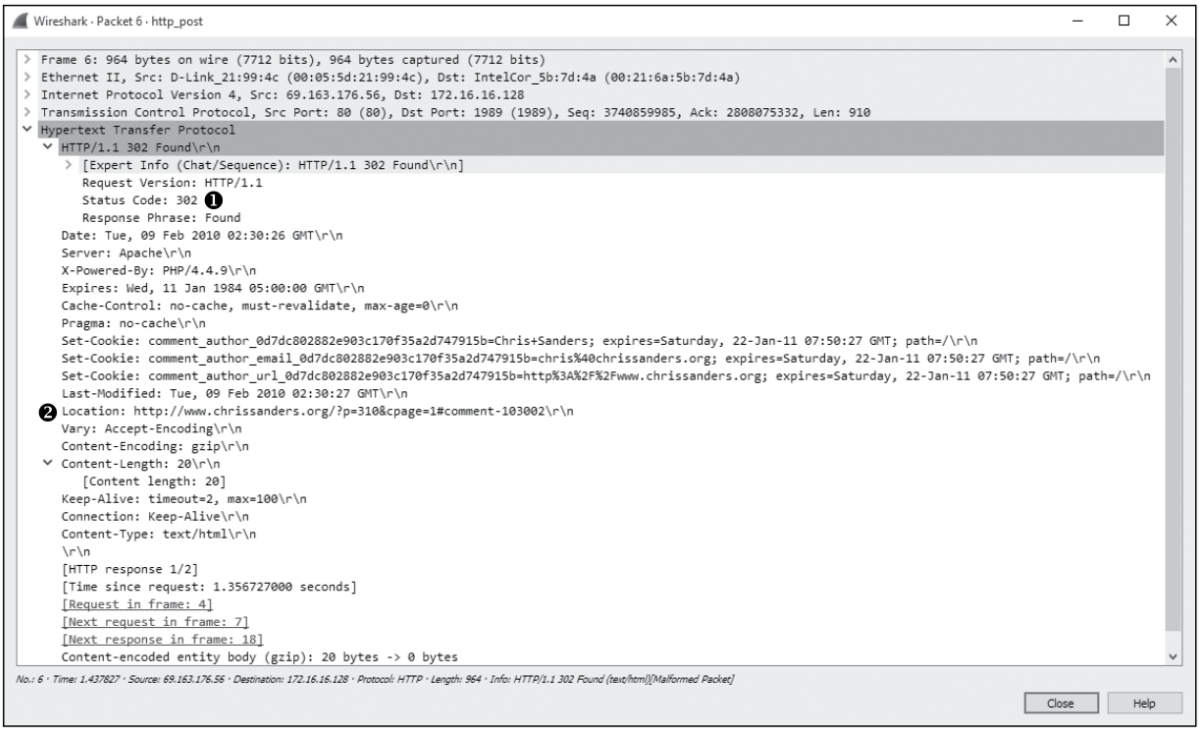


Рисунок 9-26. Ответ HTTP 302 используется для перенаправления

Код ответа 302 - распространённое средство перенаправления в HTTP. Поле Location пакета задаёт место, куда следует направить клиента (метка 2). В данном случае это исходная веб-страница, на которой был опубликован комментарий. Клиент выполняет новый запрос GET для получения содержимого из нового расположения и отправляет его в следующих нескольких пакетах. Наконец, сервер передаёт код состояния 200, и обмен завершается.

Простой протокол передачи почты (SMTP)

Если просмотр веб-страниц - самое распространённое действие пользователя, то отправка и получение электронной почты, вероятно, занимает второе место. Простой протокол передачи почты (Simple Mail Transfer Protocol, SMTP), применяемый такими платформами, как Microsoft Exchange и Postfix, является стандартом отправки электронной почты.

Как и у HTTP, структура пакета SMTP может меняться в зависимости от реализации и набора функций, поддерживаемых клиентом и сервером. В этом разделе мы рассмотрим некоторые основные функции SMTP и увидим, как отправка электронной почты выглядит на уровне пакетов.

Отправка и получение электронной почты

Архитектура электронной почты похожа на почтовую службу США. Написав письмо, вы кладёте его в почтовый ящик, почтовый работник забирает его и доставляет в отделение, где письмо сортируют. Оттуда его либо доставляют в другой почтовый ящик, обслуживаемый тем же отделением, либо перевозят в другое отделение, отвечающее за доставку. Письмо может пройти через несколько отделений или даже через узловые центры, созданные исключительно для распределения отправлений по отделениям в отдельных географических регионах. Этот поток показан на рис. 9-27.

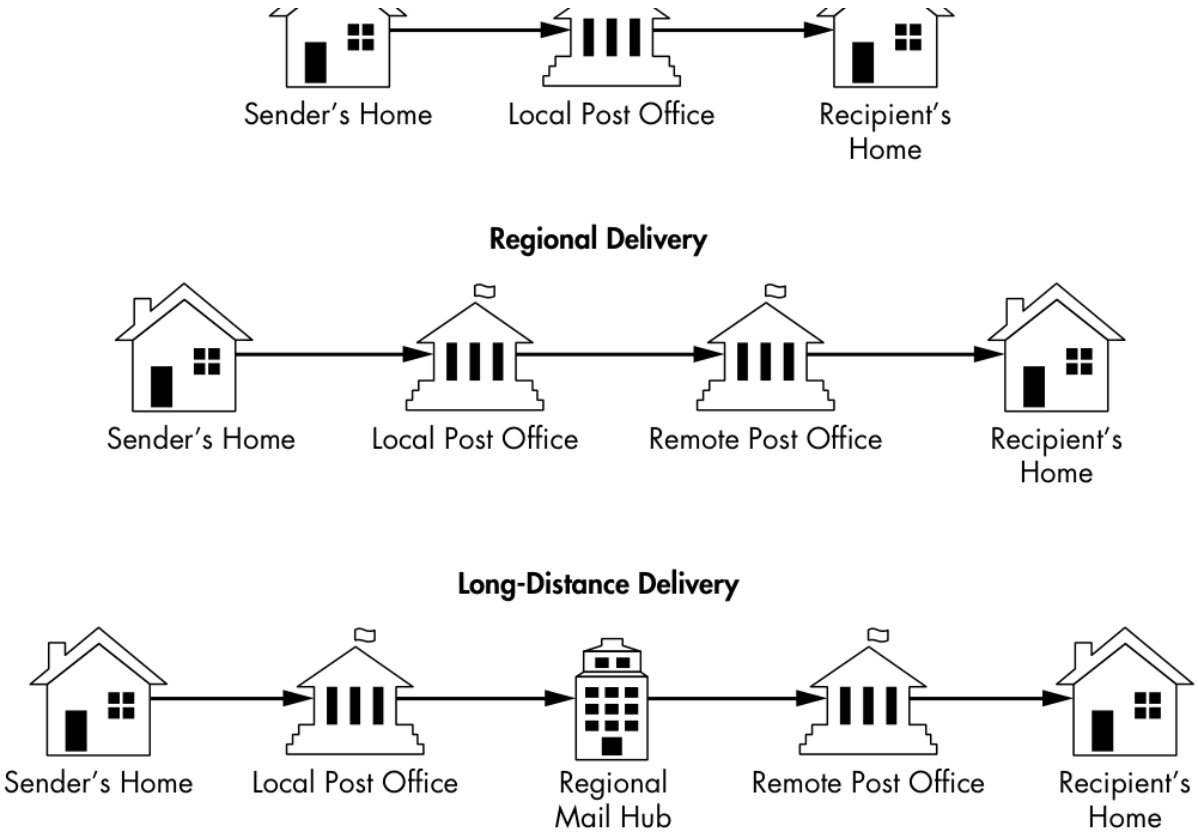


Рисунок 9-27. Отправка письма посредством почтовой службы

Доставка электронной почты работает очень похоже, но терминология несколько иная. На уровне отдельного пользователя физический почтовый ящик заменяется цифровым, который хранит электронную почту и обеспечивает её отправку и получение. Доступ к нему осуществляется через почтовый пользовательский агент (Mail User Agent, MUA), то есть почтовый клиент вроде Microsoft Outlook или Mozilla Thunderbird. Когда вы отправляете сообщение, оно передаётся от MUA агенту передачи почты (Mail Transfer Agent, MTA). MTA часто называют почтовым сервером; популярные серверные почтовые приложения - Microsoft Exchange и Postfix. Если отправляемое письмо предназначено тому же домену, из которого поступило, MTA может связать его с почтовым ящиком получателя без дополнительного обмена. Если письмо отправляется в другой домен, MTA должен посредством DNS найти адрес почтового сервера получателя и передать сообщение ему. Стоит отметить, что почтовый сервер часто состоит и из других компонентов, например агента доставки почты (Mail Delivery Agent, MDA) или агента отправки почты (Mail Submission Agent, MSA), но с точки зрения сети нас обычно интересуют лишь понятия клиента и сервера. Общая схема показана на рис. 9-28.

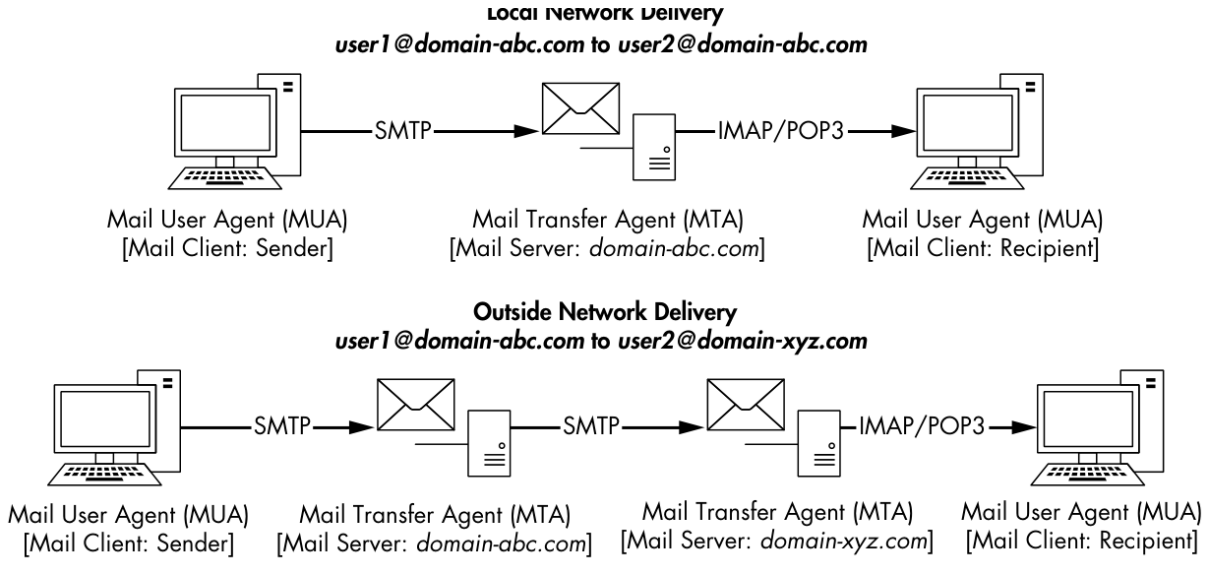


Рисунок 9-28. Отправка электронной почты посредством SMTP

Для простоты мы будем называть MUA почтовым клиентом, а MTA - почтовым сервером.

Отслеживание сообщения электронной почты

Получив общее представление о передаче электронных писем, можно перейти к пакетам этого процесса. Начнём со сценария на рис. 9-29.

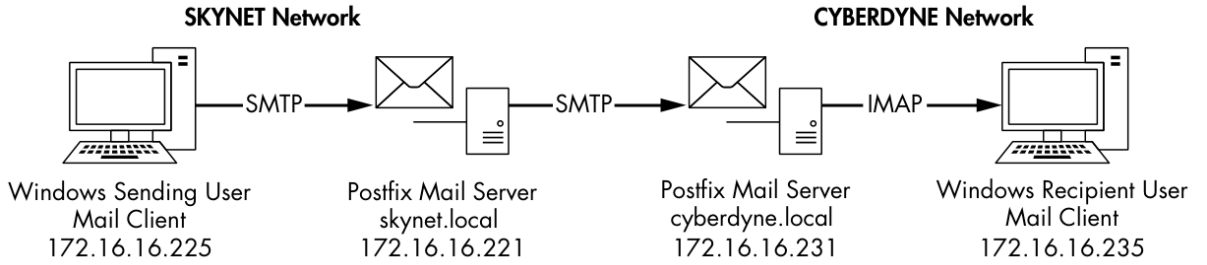


Рисунок 9-29. Отслеживание электронной почты от отправителя к получателю

Сценарий состоит из трёх этапов:

1. Пользователь отправляет сообщение со своей рабочей станции (`172.16.16.225`). Почтовый клиент передаёт сообщение по SMTP локальному почтовому серверу (`172.16.16.221`, домен `skynet.local`).
2. Локальный почтовый сервер получает сообщение и по SMTP передаёт его удалённому почтовому серверу (`172.16.16.231`, домен `cyberdyne.local`).
3. Удалённый почтовый сервер получает сообщение и связывает его с нужным почтовым ящиком. Почтовый клиент на рабочей станции пользователя (`172.16.16.235`) получает это сообщение по протоколу IMAP.

Этап 1: от клиента к локальному серверу

Начнём пошаговое рассмотрение с этапа 1, представленного файлом `mail_sender_client_1.pcapng`. Файл начинается в момент, когда пользователь нажимает кнопку Send в почтовом клиенте, инициируя в пакетах 1-3 рукопожатие TCP между рабочей станцией и локальным почтовым сервером.

> **Примечание.** При анализе захватов пакетов из этого раздела можно игнорировать ошибки `ETHERNET FRAME CHECK SEQUENCE INCORRECT`. Они являются артефактом лабораторной среды, в которой создавались захваты.

После установления соединения управление принимает SMTP и начинает передавать пользовательское сообщение серверу. Можно было бы исследовать каждый запрос и ответ SMTP по отдельности, прокручивая пакеты и просматривая раздел SMTP окна подробностей пакета, но есть более простой способ. Поскольку SMTP - простой транзакционный протокол, а наш пример передаётся открытым текстом, можно проследить поток TCP и увидеть всю транзакцию в одном окне. Щёлкните любой пакет захвата правой кнопкой мыши и выберите Follow > TCP Stream. Полученный поток показан на рис. 9-30.

После установления соединения почтовый сервер отправляет клиенту в пакете 4 баннер службы, сообщая о готовности принять команду. Здесь он представляется сервером Postfix, работающим в операционной системе Ubuntu Linux (метка 1). Он также сообщает о способности принимать команды расширенного SMTP (Extended SMTP, ESMTP). ESMTP - расширение спецификации SMTP, позволяющее использовать при передаче почты дополнительные команды.

Почтовый клиент отвечает командой `EHLO` в пакете 5 (метка 2). `EHLO` - команда приветствия, которая идентифицирует отправляющий узел при поддержке ESMTP. Если ESMTP недоступен, клиент возвращается к команде `HELO`. В этом примере отправитель идентифицируется своим IP-адресом, хотя можно использовать и DNS-имя.

В пакете 7 сервер отвечает списком элементов, среди которых `VRFY`, `STARTTLS` и `SIZE 10240000` (метка 3). Этот список отражает команды, поддерживаемые сервером SMTP, и позволяет клиенту узнать, какие команды разрешено применять при передаче сообщения. Такое согласование функций происходит в начале каждой транзакции SMTP до отправки сообщения. Передача сообщения начинается с пакета 8 и занимает большую часть оставшегося захвата.

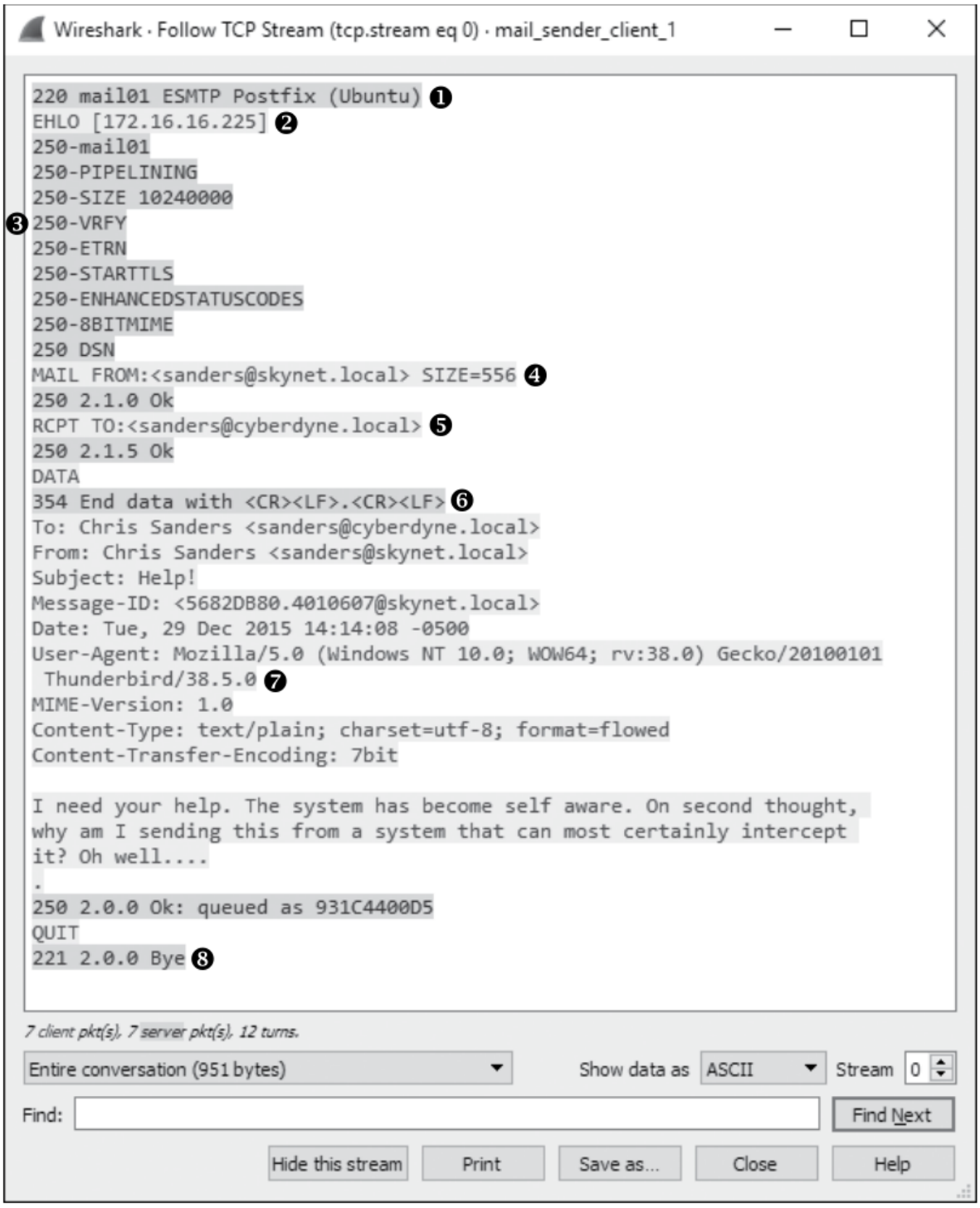


Рисунок 9-30. Просмотр потока TCP от почтового клиента к локальному серверу

SMTP управляется простыми командами и значениями параметров, которые отправляет клиент, после чего сервер возвращает код ответа. Это очень похоже на такие протоколы, как HTTP и TELNET, и рассчитано на простоту. Пример запроса и ответа виден в пакетах 8 и 9: клиент выдаёт команду `MAIL` с параметром `FROM:<sanders@skynet.local> SIZE=556` (метка 4), а сервер отвечает кодом 250 (Requested mail action okay, completed) и параметром `2.1.0 Ok`. Здесь клиент сообщает адрес электронной почты отправителя и размер сообщения, а сервер отвечает, что данные получены и допустимы. Похожая транзакция снова происходит в пакетах 10 и 11: клиент выдаёт команду `RCPT` с параметром `TO:<sanders@cyberdyne.local>` (метка 5), а сервер возвращает ещё один код `250 2.1.5 Ok`.

> **Примечание.** Все доступные команды и параметры SMTP можно изучить по адресу iana.org. Доступные коды ответа приведены по адресу iana.org.

Остаётся передать само сообщение. Клиент начинает процесс в пакете 12 командой `DATA`. Сервер отвечает кодом 354 и сообщением (метка 6), указывающим, что он создал буфер для сообщения, и предлагающим клиенту начать передачу. Строка с кодом 354 сообщает клиенту, что конец передачи следует обозначить точкой (`<CR><LF>.<CR><LF>`). Сообщение передаётся открытым текстом, после чего отправляется код ответа об успешной передаче. Вы заметите, что вместе с текстом сообщения передаются дополнительные сведения, включая дату, тип и кодировку содержимого, а также пользовательский агент, связанный с передачей. Они сообщают, что конечный пользователь отправил письмо посредством Mozilla Thunderbird (метка 7).

После окончания передачи почтовый клиент закрывает соединение SMTP, выдавая в пакете 18 команду `QUIT` без параметров. Сервер отвечает в пакете 19 кодом 221 (`<domain> service closing transmission channel`) и параметром `2.0.0 Bye` (метка 8). TCP-соединение корректно закрывается в пакетах 20-23.

Этап 2: от локального сервера к удалённому

Теперь рассмотрим тот же сценарий с точки зрения локального почтового сервера, отвечающего за домен `skynet.local`; его адрес - `172.16.16.221`. Захват находится в файле `mail_sender_server_2.pcapng`, полученном непосредственно на почтовом сервере. Как можно ожидать, первые примерно 20 пакетов повторяют захват этапа 1, поскольку это те же пакеты, захваченные из другого источника.

Если бы отправленное сообщение предназначалось другому почтовому ящику домена `skynet.local`, дальнейшего трафика SMTP мы бы не увидели; вместо него наблюдалось бы получение письма почтовым клиентом по протоколу POP3 или IMAP. Но поскольку сообщение предназначено домену `cyberdyne.local`, локальный сервер SMTP должен передать его удалённому серверу SMTP, отвечающему за этот домен. Процесс начинается в пакете 22 с рукопожатия TCP между локальным сервером `172.16.16.221` и удалённым почтовым сервером `172.16.16.231`.

> **Примечание.** В реальной ситуации почтовый сервер находит другой сервер посредством особого типа записи DNS - записи почтового обменника (mail exchange, MX). Поскольку сценарий создан в лаборатории, а IP-адрес удалённого почтового сервера был заранее настроен на локальном сервере, здесь такого трафика не будет. При устранении неполадок доставки электронной почты следует учитывать возможные проблемы DNS наряду с проблемами протоколов самой почты.

После установления соединения в окне списка пакетов видно, что для доставки сообщения удалённому серверу используется SMTP. Этот разговор удобнее просматривать, проследив поток TCP транзакции. Он показан на рис. 9-31. Если вам нужна помощь в выделении соединения, примените в строке фильтра выражение `tcp.stream == 1`.

Рисунок 9-31. Просмотр потока TCP от локального почтового сервера к удалённому почтовому серверу

Эта транзакция почти идентична показанной на рис. 9-30. По сути, сообщение просто передаётся между серверами. Удалённый сервер представляется как `mail02` (метка 1), локальный - как `mail01` (метка 2), стороны обмениваются списком поддерживаемых команд (метка 3), а сообщение передаётся целиком; перед строкой To к нему добавлено немного дополнительных данных из предыдущей транзакции (метка 4). Всё это происходит между пакетами 27 и 35, после чего канал обмена закрывается завершением TCP.

Серверу в конечном счёте неважно, поступает сообщение от почтового клиента или другого сервера SMTP, поэтому применяются те же правила и процедуры (за исключением возможных ограничений управления доступом). В реальном мире локальный и удалённый почтовые серверы могут поддерживать разные наборы функций или основываться на совершенно разных платформах. Именно поэтому начальный обмен SMTP так важен: он позволяет принимающему серверу сообщить отправителю поддерживаемый набор функций. Когда клиенту или серверу SMTP известны функции принимающего сервера, команды SMTP можно скорректировать для эффективной передачи сообщения. Благодаря этому SMTP широко применим между множеством клиентских и серверных технологий, а при отправке письма вам не нужно много знать о сетевой инфраструктуре получателя.

Этап 3: от удалённого сервера к удалённому клиенту

Теперь сообщение достигло удалённого сервера, отвечающего за доставку электронной почты в почтовые ящики домена `cyberdyne.local`. Рассмотрим захват пакетов с точки зрения удалённого сервера - `mail_receiver_server_3.pcapng`, показанный на рис. 9-32.

Рисунок 9-32. Просмотр потока TCP от локального почтового сервера к удалённому почтовому серверу

Первые 15 пакетов этого захвата снова выглядят очень знакомо: они представляют обмен тем же сообщением, где адрес источника соответствует локальному почтовому серверу (метка 1), а адрес назначения - удалённому (метка 2). После завершения последовательности сервер SMTP может связать сообщение с нужным почтовым ящиком, чтобы предполагаемый получатель забрал его почтовым клиентом.

Как уже говорилось, SMTP применяется главным образом для отправки электронной почты и является самым распространённым протоколом для этой цели с огромным отрывом. Получение

почты из ящика на сервере устроено менее однозначно, и из-за разных потребностей для этой задачи создано несколько протоколов. Наиболее распространены протокол почтового отделения версии 3 (Post Office Protocol version 3, POP3) и протокол доступа к сообщениям Интернета (Internet Message Access Protocol, IMAP). В нашем примере удалённый клиент получает сообщения с почтового сервера по IMAP в пакетах 16-34.

IMAP в книге не рассматривается, но в этом примере его изучение всё равно мало помогло бы, поскольку обмен зашифрован. В пакете 21 клиент (`172.16.16.235`) отправляет почтовому серверу (`172.16.16.231`) команду `STARTTLS`, показанную на рис. 9-33 (метка 1).

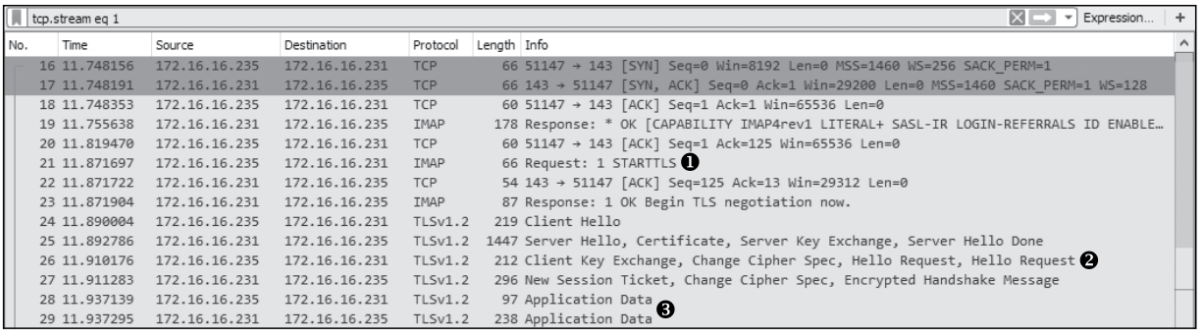


Рисунок 9-33. Команда STARTTLS указывает, что трафик IMAP будет зашифрован

Команда сообщает серверу, что клиент хочет безопасно получать сообщения с использованием шифрования TLS. В пакетах 24-27 между конечными точками согласуется защищённый канал (метка 2), а в остальных пакетах сообщение безопасно получается по протоколу TLS (Transport Layer Security, безопасность транспортного уровня) (метка 3). Если щёлкнуть любой из этих пакетов для просмотра данных или попытаться проследить поток TCP (рис. 9-34), содержимое окажется нечитаемым. Это защищает письмо от перехвата злоумышленником, который может пытаться захватить сеанс или прослушивать трафик.

После получения последних пакетов процесс отправки сообщения от пользователя одного домена пользователю другого завершается.

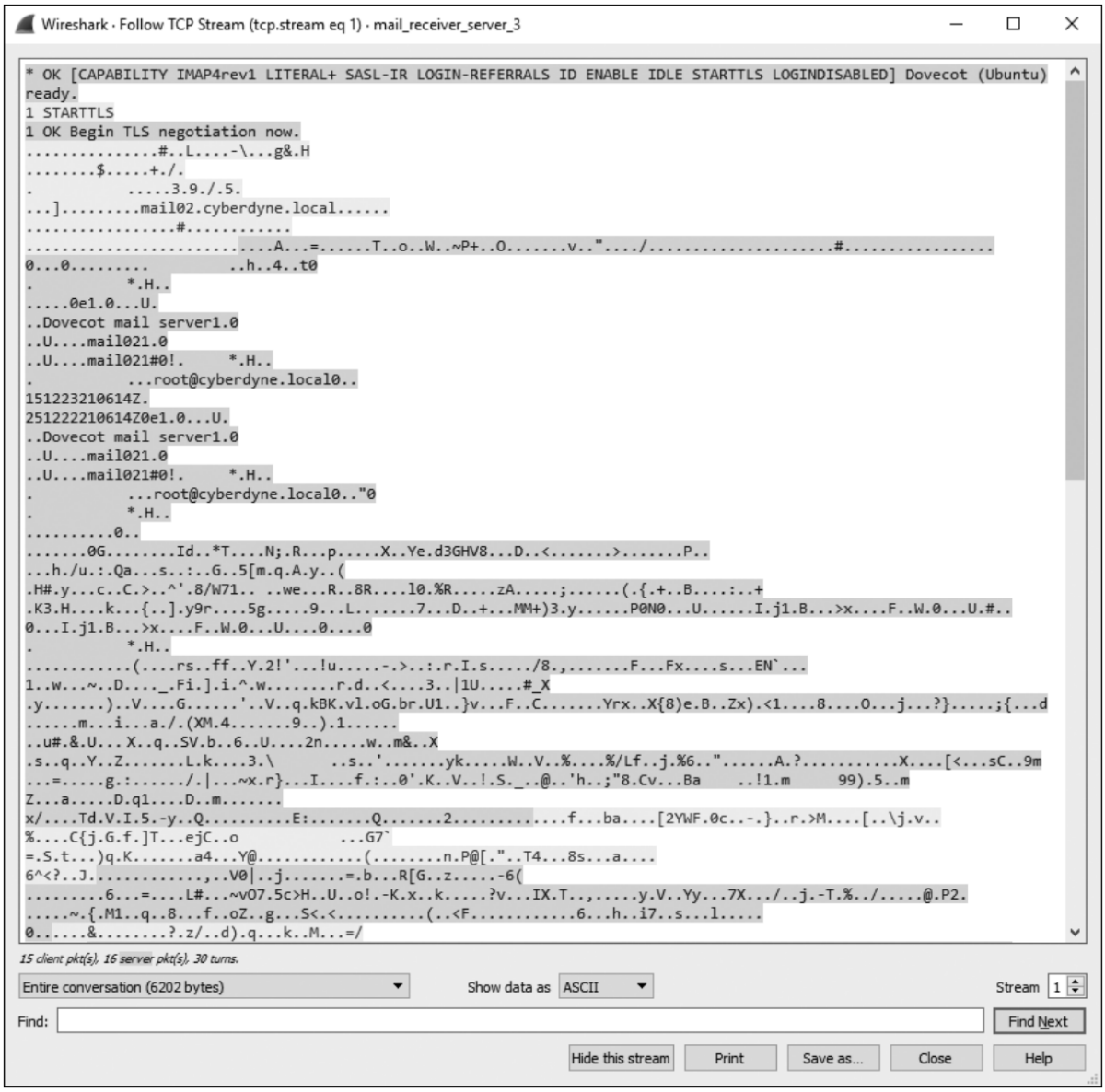


Рисунок 9-34. Трафик IMAP зашифрован во время загрузки сообщения клиентом

Отправка вложений посредством SMTP

SMTP никогда не предназначался для передачи файлов, однако удобство отправки файла по электронной почте сделало её для многих основным способом обмена. Рассмотрим краткий пример того, как отправка файла по SMTP выглядит на уровне пакетов.

В захвате `mail_sender_attachment.pcapng` пользователь отправляет со своего клиента (`172.16.16.225`) электронное письмо другому пользователю в той же сети через локальный почтовый сервер SMTP (`172.16.16.221`). Сообщение содержит немного текста и вложенный файл изображения.

Отправка вложения по SMTP не слишком отличается от отправки текста. Для сервера всё это просто данные, и, хотя обычно выполняется специальное кодирование, для доставки по-прежнему используется команда `DATA`. Чтобы увидеть процесс, откройте файл захвата и проследите поток TCP транзакции SMTP. Поток показан на рис. 9-35.

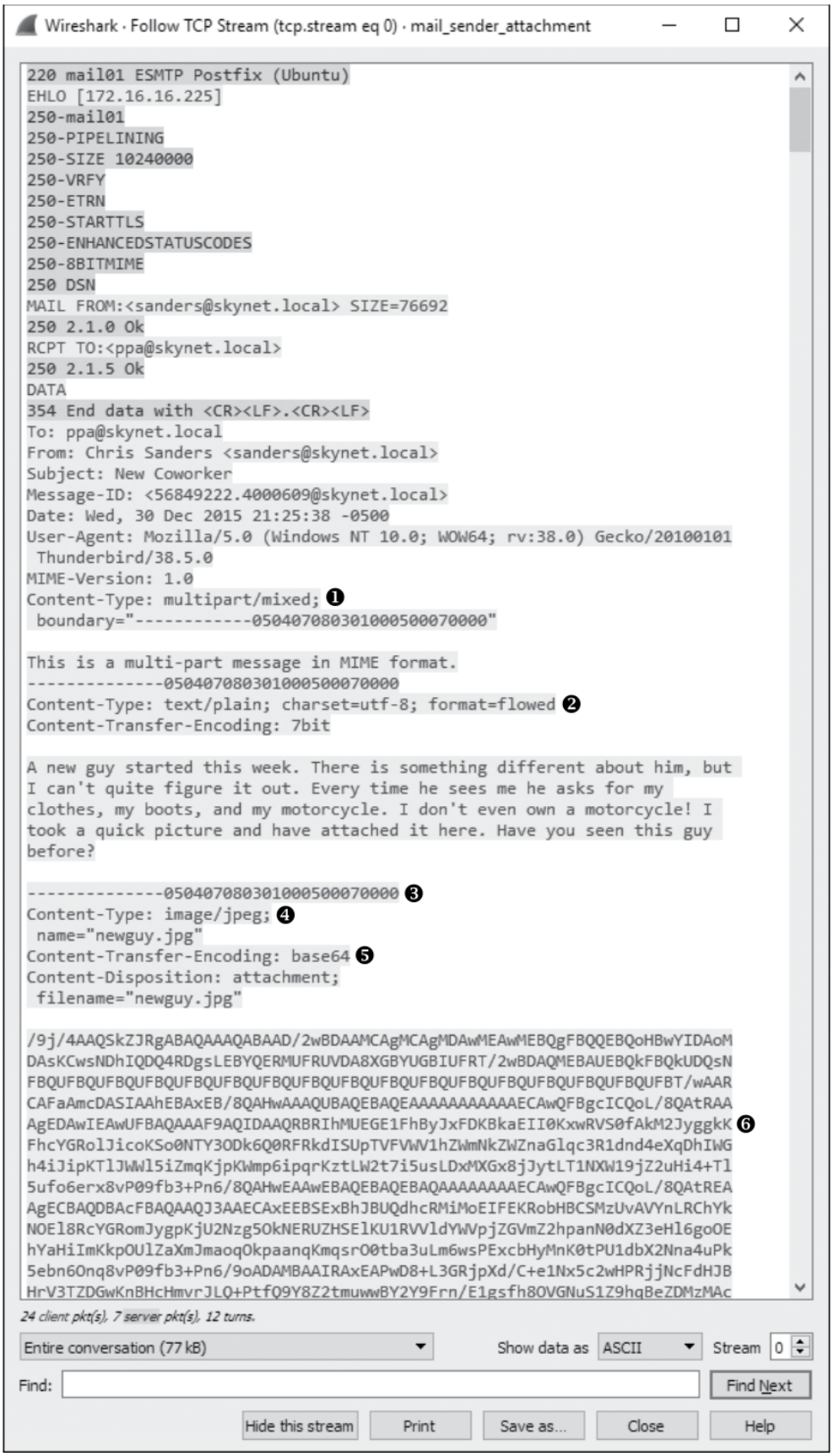


Рисунок 9-35. Пользователь отправляет вложение посредством SMTP

Пример начинается, как предыдущие сценарии, с идентификации службы и обмена поддерживаемыми протоколами. Когда клиент готов передать сообщение, он сообщает адреса

From и To, а команда DATA предписывает серверу открыть буфер для получения сведений. Здесь и появляются различия.

В предыдущем примере клиент непосредственно передал серверу текст, и на этом всё закончилось. Теперь клиент должен отправить сообщение открытым текстом, а также двоичные данные вложенного изображения. Для этого он указывает Content-Type `multipart/mixed` с границей `------------0504070803010005000700000` (метка 1). Это сообщает серверу, что передаются данные нескольких типов, каждый со своим уникальным типом MIME и кодировкой, а типы разделяются заданным значением границы. Поэтому, получив эти данные, другой почтовый клиент сможет интерпретировать их по границам, уникальному типу MIME и кодировке каждого фрагмента.

В нашем примере сообщение состоит из двух уникальных частей. Первая - сам текст письма, определённый типом содержимого `text/plain` (метка 2). После него находится маркер границы и начало новой части сообщения (метка 3). Эта часть содержит файл изображения и определяется типом содержимого `image/jpeg` (метка 4). Стоит также отметить, что Content-Transfer-Encoding имеет значение `base64` (метка 5), то есть для разбора данные необходимо преобразовать из base64. Остальная передача содержит закодированный файл изображения (метка 6).

Ни в коем случае не путайте это кодирование с функцией безопасности. Кодирование base64 обращается почти мгновенно, и любой перехвативший обмен злоумышленник сможет без особых усилий получить файл изображения. Если вы хотите самостоятельно вырезать изображение из захвата пакетов, похожий сценарий извлечения изображения из передачи файла по HTTP рассматривается в разделе о трояне удалённого доступа главы 12. Прочитав его, вернитесь к этому захвату и попробуйте узнать, кто этот таинственный новый коллега пользователя.

Заключительные мысли

В этой главе были представлены самые распространённые протоколы, которые встретятся при исследовании трафика прикладного уровня. В следующих главах, разбирая множество реальных сценариев, мы рассмотрим новые протоколы и дополнительные функции уже изученных.

Чтобы больше узнать об отдельных протоколах, прочитайте соответствующие RFC или книгу Чарльза М. Козиерока «The TCP/IP Guide» (No Starch Press, 2005). Также обратитесь к списку ресурсов в приложении A.

Глава 10. Базовые реальные сценарии

Начиная с этой главы мы погрузимся в самую суть анализа пакетов и применим Wireshark для исследования реальных сетевых проблем. Я представлю серию проблемных сценариев, описывая контекст проблемы и сведения, которыми располагал аналитик в тот момент. Заложив основу, мы перейдём к анализу: я расскажу, как захватывались нужные пакеты, и шаг за шагом проведу вас по пути к диагнозу. После анализа я укажу возможные решения и кратко изложу усвоенные уроки.

Помните, что анализ - очень динамичный процесс. Поэтому мои методы исследования каждого сценария могут отличаться от тех, которыми воспользовались бы вы. Каждый подходит к решению проблем и рассуждению через собственную призму. Самое важное - чтобы результат анализа решил проблему, но даже если этого не произошло, критически важно учиться и на неудачах. В конце концов, опыт - это то, что мы получаем, когда не получаем желаемого.

Кроме того, большинство обсуждаемых в этой главе проблем, вероятно, можно решить методами, не обязательно связанными с анализатором пакетов, но разве это интересно? Когда я только учился анализировать пакеты, мне помогало исследовать обычные проблемы необычными способами, применяя методы анализа пакетов, поэтому я и предлагаю вам эти сценарии.

Отсутствующее веб-содержимое

В первом сценарии наш пользователь - Пакетный Пит, любитель студенческого баскетбола, который не засиживается допоздна и обычно пропускает матчи Западного побережья. Каждое утро, садясь за рабочую станцию, он первым делом открывает espn.com, чтобы посмотреть итоговые результаты предыдущего вечера. Сегодня утром Пит обнаруживает, что страница ESPN загружается очень долго, а когда наконец открывается, большая часть изображений и содержимого отсутствует (рис. 10-1). Поможем Питу диагностировать проблему.

Рисунок 10-1. ESPN загружается неправильно

Подключение к линии

Проблема ограничена рабочей станцией Пита и не затрагивает остальные, поэтому начнём захват пакетов непосредственно на ней. Установим Wireshark и захватим пакеты во время открытия веб-сайта ESPN. Эти пакеты находятся в файле `http_espn_fail.pcapng`.

Анализ

Мы знаем, что проблема Пита состоит в невозможности просмотреть открываемый веб-сайт, поэтому главным образом будем исследовать протокол HTTP. Если вы прочитали предыдущую главу, у вас должно быть общее представление о трафике HTTP между клиентом и сервером. Хорошая отправная точка - HTTP-запросы к удалённому серверу. Их можно найти, применив фильтр запросов `GET` (`http.request.method == "GET"`), или просто выбрав в главном раскрывающемся меню Statistics > HTTP > Requests (рис. 10-2).

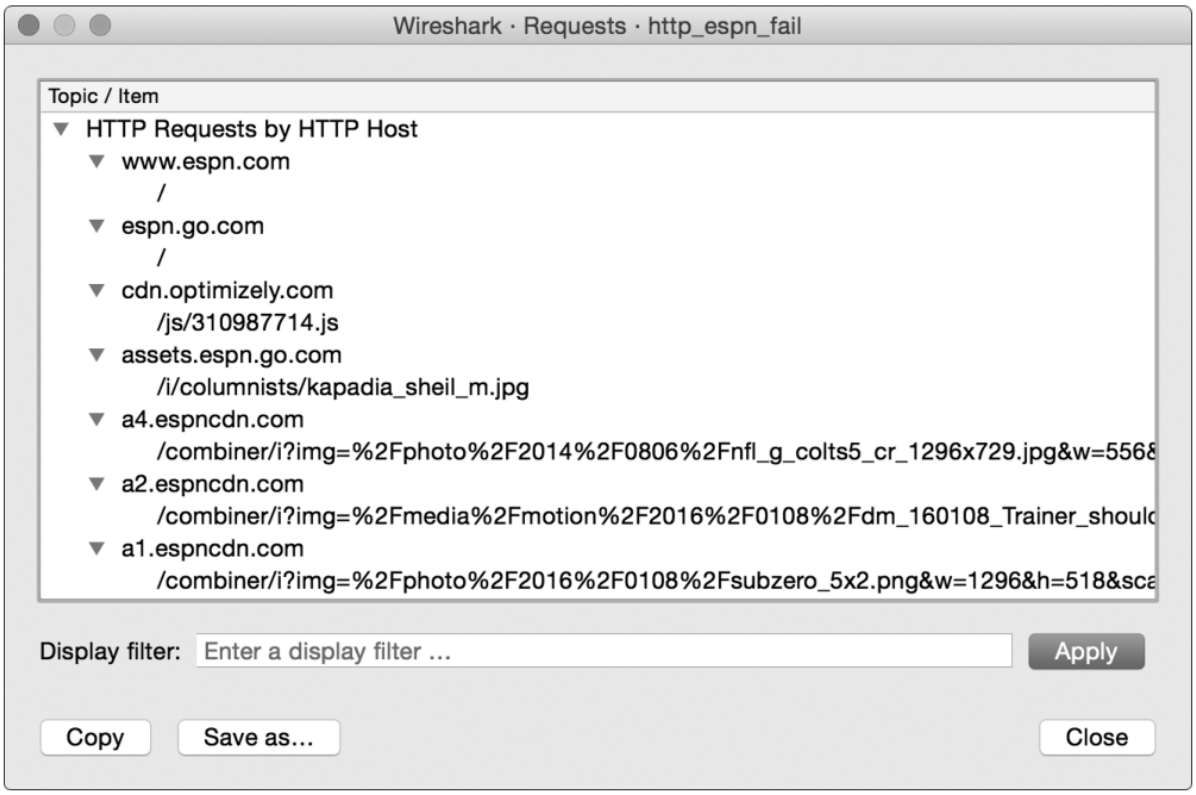


Рисунок 10-2. Просмотр HTTP-запросов к ESPN

Судя по обзору, захват ограничен семью разными HTTP-запросами, и все они, похоже, связаны с веб-сайтом ESPN. Каждый запрос содержит строку `espn` в доменном имени, за исключением `cdn.optimizely.com` - сети доставки содержимого (content delivery network, CDN), используемой для показа рекламы на множестве сайтов. При посещении веб-сайтов с рекламой или другим внешним содержимым запросы к различным CDN встречаются часто.

Поскольку очевидных зацепок нет, следующим шагом исследуем иерархию протоколов файла захвата, выбрав Statistics > Protocol Hierarchy. Это позволит заметить неожиданные протоколы или необычное распределение трафика между ними (рис. 10-3). Помните, что экран иерархии протоколов основывается на текущем фильтре отображения. Обязательно очистите ранее применённый фильтр, чтобы получить ожидаемые результаты по всему захвату пакетов.

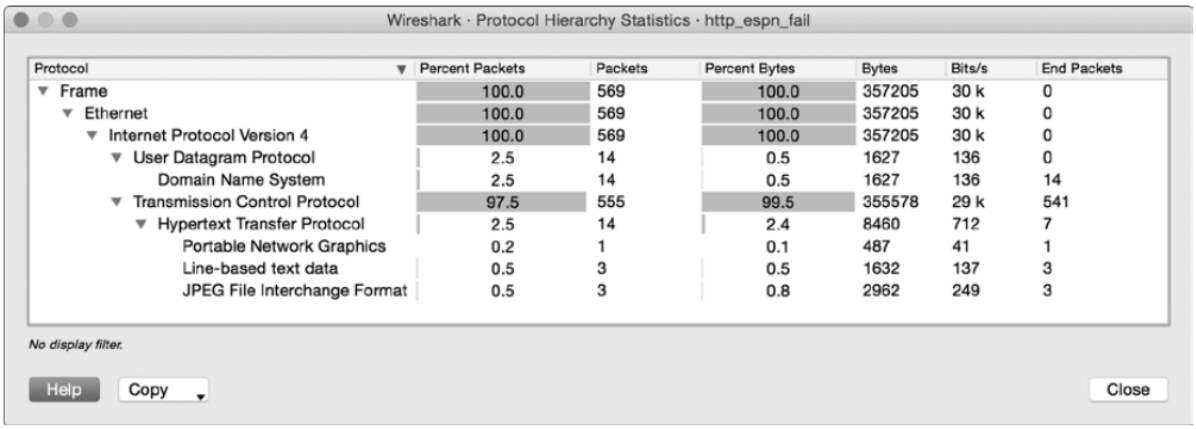


Рисунок 10-3. Исследование иерархии протоколов сеанса просмотра

Иерархия протоколов не слишком сложна, и можно быстро определить, что работают лишь два протокола прикладного уровня: HTTP и DNS. Как вы узнали в главе 9, DNS преобразует доменные имена в IP-адреса. Поэтому, когда вы открываете сайт вроде espn.com, системе может

потребоваться отправить запрос DNS, чтобы найти IP-адрес удалённого веб-сервера, если он ещё неизвестен. После получения ответа DNS с нужным IP-адресом сведения можно добавить в локальный кеш и начать обмен HTTP (с использованием TCP).

Хотя ничего необычного здесь не видно, 14 пакетов DNS заслуживают внимания. Запрос DNS одного доменного имени обычно помещается в один пакет, и ответ также составляет один пакет (если только он не очень велик, тогда DNS применяет TCP). Поскольку здесь 14 пакетов DNS, возможно, было сформировано до семи запросов DNS (7 запросов + 7 ответов = 14 пакетов). На рис. 10-2 действительно показаны HTTP-запросы к семи разным доменам, но Пит ввёл в браузере всего один URL. Почему выполняются все дополнительные запросы?

В простом мире для посещения веб-страницы было бы достаточно обратиться к одному серверу и получить всё содержимое в одном разговоре HTTP. В действительности отдельная веб-страница может предоставлять содержимое с нескольких серверов. Весь текст может находиться в одном месте, графика - в другом, а встроенные видеоролики - в третьем. И это не считая рекламы, которая может размещаться у нескольких поставщиков на десятках отдельных серверов. Каждый раз, когда клиент HTTP разбирает код HTML и находит ссылку на содержимое другого узла, он пытается запросить это содержимое у данного узла, что способно породить дополнительные запросы DNS и HTTP. Именно это произошло, когда Пит посетил ESPN. Хотя он намеревался просмотреть содержимое только одного источника, в HTML-коде нашлись ссылки на дополнительное содержимое, и браузер автоматически запросил его из нескольких других доменов.

Теперь, когда причина дополнительных запросов понятна, следующим шагом исследуем отдельные разговоры, связанные с каждым запросом (Statistics > Conversations). Просмотр окна Conversations (рис. 10-4) даёт важную подсказку.

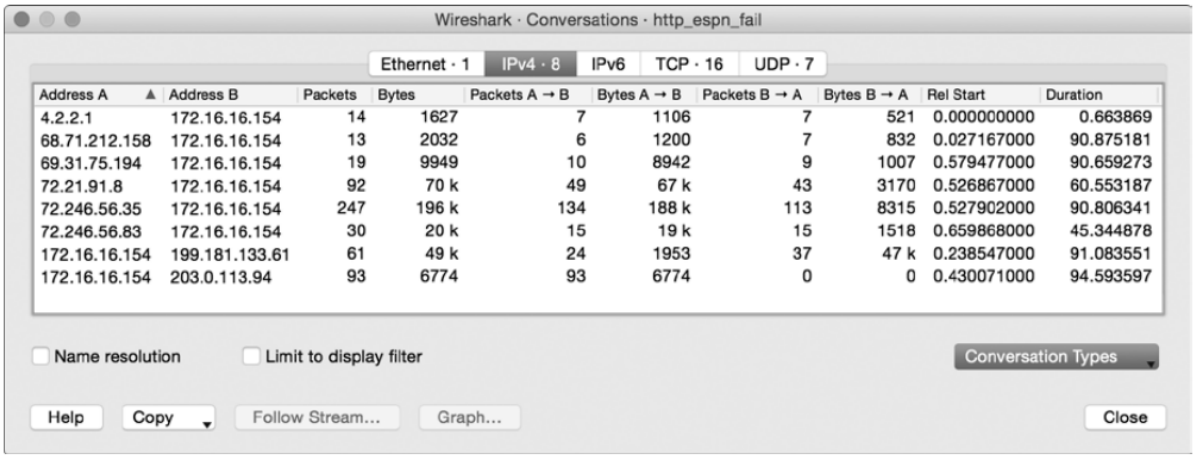
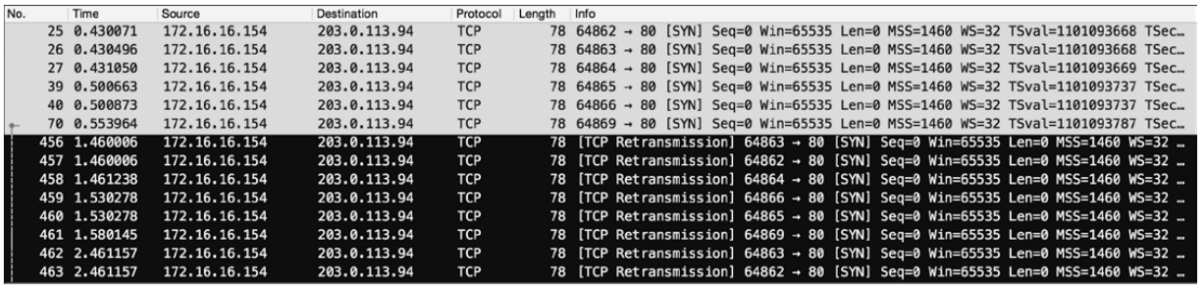


Рисунок 10-4. Исследование IP-разговоров

Ранее мы обнаружили семь запросов DNS и семь соответствующих запросов HTTP. Поэтому разумно ожидать и семь соответствующих IP-разговоров, но на самом деле их восемь. Чем это объяснить?

Можно предположить, что захват «загрязнён» дополнительным разговором, не связанным с рассматриваемой проблемой. Необходимо следить, чтобы нерелевантный трафик не мешал анализу, но в данном разговоре проблема не в этом. Если исследовать каждый HTTP-запрос и отметить IP-адрес, на который он отправлялся, останется один разговор без соответствующего HTTP-запроса. Его конечные точки - рабочая станция Пита (`172.16.16.154`) и удалённый IP-адрес `203.0.113.94`. Этот разговор представлен нижней строкой на рис. 10-4. Мы видим, что неизвестному узлу отправлено 6774 байта, но обратно не поступило ни одного: это стоит изучить.

Отфильтровав этот разговор (щёлкните его правой кнопкой мыши и выберите Apply As Filter > Selected > A<->B), можно применить знания TCP, чтобы определить, что пошло не так (рис. 10-5).



No.	Time	Source	Destination	Protocol	Length	Info
25	0.430071	172.16.16.154	203.0.113.94	TCP	78	64862 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093668 TSec…
26	0.430496	172.16.16.154	203.0.113.94	TCP	78	64863 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093668 TSec…
27	0.431050	172.16.16.154	203.0.113.94	TCP	78	64864 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093669 TSec…
39	0.500663	172.16.16.154	203.0.113.94	TCP	78	64865 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093737 TSec…
40	0.500873	172.16.16.154	203.0.113.94	TCP	78	64866 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093737 TSec…
70	0.553964	172.16.16.154	203.0.113.94	TCP	78	64869 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1101093787 TSec…
456	1.460006	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64863 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
457	1.460006	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64862 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
458	1.461238	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64864 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
459	1.530278	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64866 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
460	1.530278	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64865 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
461	1.580145	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64869 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
462	2.461157	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64863 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …
463	2.461157	172.16.16.154	203.0.113.94	TCP	78	[TCP Retransmission] 64862 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 …

Рисунок 10-5. Исследование неожиданного соединения

При нормальном обмене TCP ожидается стандартная последовательность рукопожатия SYN-SYN/ACK-ACK. Здесь рабочая станция Пита отправила пакет SYN на `203.0.113.94`, но ответа SYN/ACK мы так и не видим. Более того, рабочая станция Пита безрезультатно отправила несколько пакетов SYN и в конце концов начала передавать пакеты повторной передачи TCP. Подробнее особенности повторной передачи TCP будут рассмотрены в главе 11, но главный вывод здесь таков: один узел отправляет пакеты, на которые никогда не получает ответа. В столбце Time видно, что повторные передачи продолжаются 95 секунд без ответа. Для сетевого обмена это медленнее патоки.

Мы обнаружили семь запросов DNS, семь запросов HTTP и восемь IP-разговоров. Поскольку известно, что захват не загрязнён лишними данными, разумно предположить, что загадочный восьмой разговор IP, вероятно, и вызывает медленную и неполную загрузку веб-страницы Пита. По какой-то причине рабочая станция пытается связаться с устройством, которое либо не существует, либо просто не ожидает соединений. Чтобы понять причину, мы посмотрим не на то, что присутствует в файле захвата, а на то, чего в нём нет.

Когда Пит открыл espn.com, браузер обнаружил ресурсы, размещённые в других доменах. Для получения этих данных рабочая станция сформировала запросы DNS, чтобы найти их IP-адреса, а затем подключилась к ним по TCP для отправки HTTP-запроса содержимого. Для разговора с `203.0.113.94` запрос DNS отсутствует. Откуда же рабочая станция Пита узнала этот адрес?

Если вы помните обсуждение DNS в главе 9 или знакомы с ним иным образом, то знаете, что большинство систем реализует ту или иную форму кеширования DNS. Благодаря этому они могут обращаться к уже полученному локальному сопоставлению DNS-имени и IP-адреса, не создавая DNS-запрос при каждом посещении домена, с которым часто обмениваются данными. Со временем такие сопоставления истекают, и требуется новый запрос. Однако, если сопоставление DNS и IP изменилось, а при следующем посещении устройство не сформировало DNS-запрос для получения нового адреса, оно попытается подключиться к адресу, который больше недействителен.

Именно это произошло с Питом. На его рабочей станции уже было кешированное сопоставление DNS и IP для домена, размещающего содержимое ESPN. Поскольку запись существовала в кеше, запрос DNS не создавался, и система сразу попыталась подключиться к старому адресу. Однако этот адрес больше не был настроен отвечать на запросы. В результате истекло время ожидания запросов, а содержимое не загрузилось.

К счастью для Пита, кеш DNS можно очистить вручную несколькими нажатиями клавиш в командной строке или окне терминала. Либо он может просто повторить попытку через несколько минут, когда запись кеша DNS, вероятно, истечёт и будет сформирован новый запрос.

Усвоенные уроки

Немало работы лишь для того, чтобы узнать, что Кентукки обыграл Дьюк с преимуществом в 90 очков, но зато мы глубже поняли отношения между сетевыми узлами. В этом сценарии мы смогли приблизиться к решению, оценив несколько точек данных, связанных с запросами и разговорами в захвате. Затем мы заметили несколько несоответствий, которые привели нас к неудачному обмену между клиентом и одним из серверов доставки содержимого ESPN.

В реальном мире диагностика проблем редко сводится к прокрутке списка пакетов в поисках тех, которые выглядят странно. Устранение даже простейших неполадок может создавать очень большие захваты, где для обнаружения аномалий необходимо использовать аналитические и статистические функции Wireshark. Знакомство с таким стилем анализа критически важно для успешного устранения неполадок на уровне пакетов.

Если вы хотите увидеть пример нормального обмена между веб-браузером и ESPN, самостоятельно откройте сайт во время захвата трафика и попробуйте определить все серверы, отвечающие за доставку содержимого.

Неотвечающая служба погоды

Во втором сценарии снова участвует наш приятель Пакетный Пит. Среди множества увлечений Пит считает себя метеорологом-любителем и не проводит больше нескольких часов без проверки текущих условий и прогноза. При этом он не полагается исключительно на местный прогноз новостей, а держит возле дома небольшую метеостанцию, которая отправляет данные на wunderground.com для агрегирования и просмотра. Сегодня Пит решил проверить, насколько за ночь упала температура, но обнаружил, что станция уже более девяти часов, примерно с полуночи, не отправляла отчёты в Wunderground (рис. 10-6).

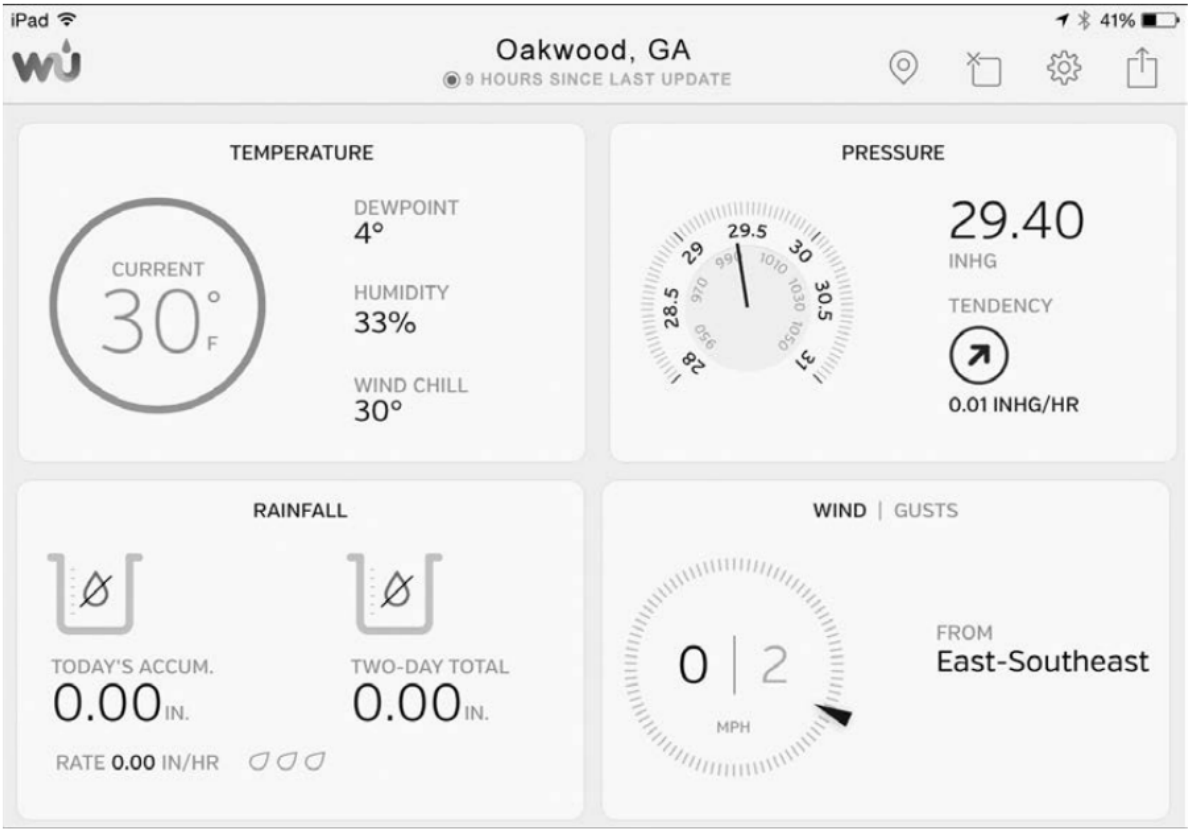


Рисунок 10-6. Метеостанция не отправляла отчёт девять часов

Подключение к линии

В сети Пита установленная на крыше метеостанция соединяется по радиоканалу с приёмником внутри дома. Приёмник подключён к сетевому коммутатору и через Интернет отправляет статистику в Wunderground. Архитектура показана на рис. 10-7.

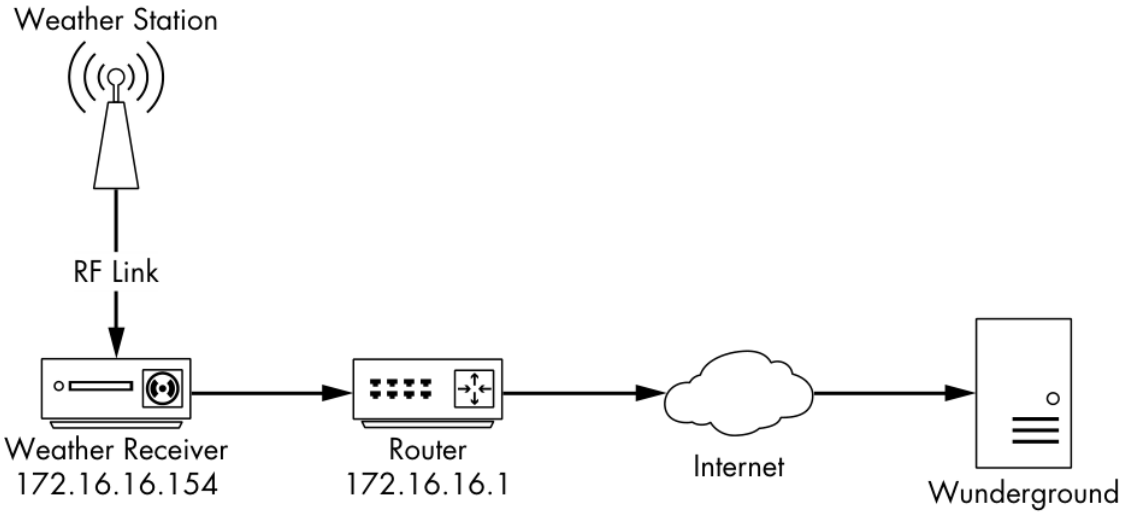


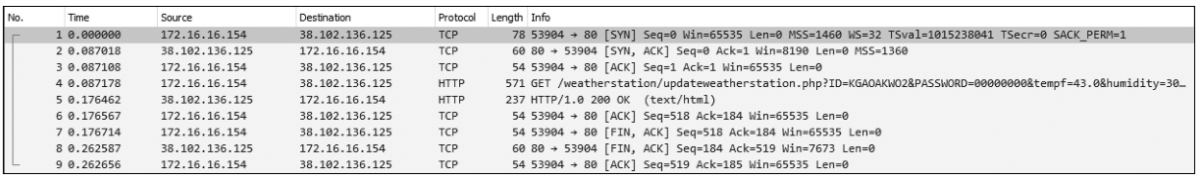
Рисунок 10-7. Архитектура сети метеостанции

У приёмника есть простая веб-страница управления, однако после входа Пит увидел лишь загадочное сообщение о времени последней синхронизации без дополнительных рекомендаций по

устранению неполадок: программное обеспечение не ведёт подробных журналов ошибок. Поскольку приёмник служит центром обмена инфраструктуры метеостанции, разумно захватить пакеты, передаваемые к этому устройству и от него, чтобы попытаться диагностировать проблему. Это домашняя сеть, поэтому зеркалирование портов на коммутаторе SOHO, вероятно, недоступно. Лучше всего использовать недорогой ответвитель или отравление кеша ARP для перехвата пакетов. Захваченные пакеты находятся в файле `weather_broken.pcapng`.

Анализ

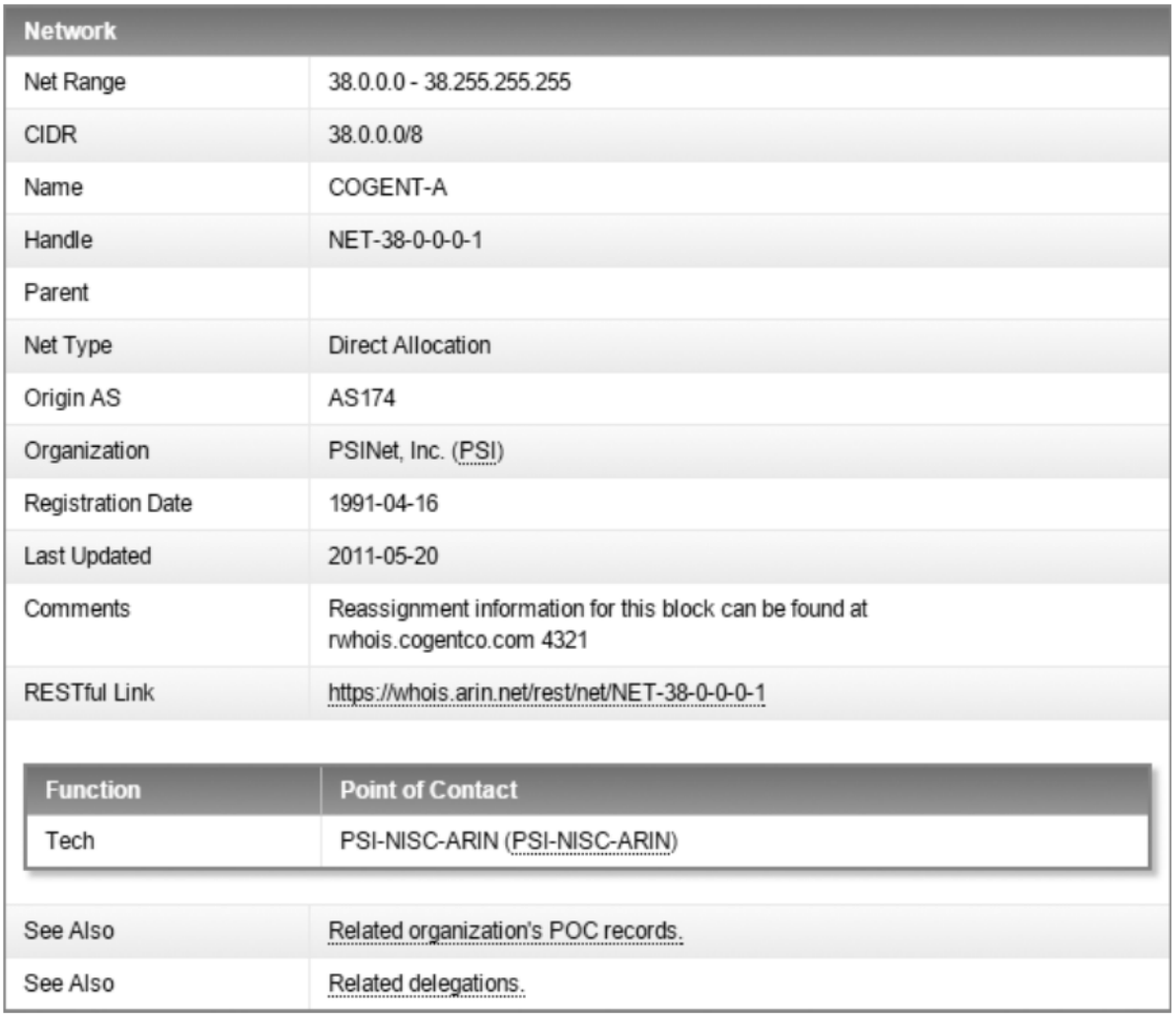
Открыв захват, вы увидите, что мы снова имеем дело с обменом HTTP. Захват пакетов ограничен одним разговором между локальным приёмником метеостанции Пита `172.16.16.154` и неизвестным удалённым устройством в Интернете `38.102.136.125` (рис. 10-8).



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.154	38.102.136.125	TCP	78	53904 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1015238041 TSecr=0 SACK_PERM=1
2	0.087018	38.102.136.125	172.16.16.154	TCP	60	80 → 53904 [SYN, ACK] Seq=0 Ack=1 Win=8190 Len=0 MSS=1360
3	0.087108	172.16.16.154	38.102.136.125	TCP	54	53904 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.087178	172.16.16.154	38.102.136.125	HTTP	571	GET /weatherstation/updateweatherstation.php?ID=KGAOAKWO2&PASSWORD=00000000&tempf=43.0&humidity=30…
5	0.176462	38.102.136.125	172.16.16.154	HTTP	237	HTTP/1.0 200 OK (text/html)
6	0.176567	172.16.16.154	38.102.136.125	TCP	54	53904 → 80 [ACK] Seq=518 Ack=184 Win=65535 Len=0
7	0.176714	172.16.16.154	38.102.136.125	TCP	54	53904 → 80 [FIN, ACK] Seq=518 Ack=184 Win=65535 Len=0
8	0.262587	38.102.136.125	172.16.16.154	TCP	60	80 → 53904 [FIN, ACK] Seq=184 Ack=519 Win=7673 Len=0
9	0.262656	172.16.16.154	38.102.136.125	TCP	54	53904 → 80 [ACK] Seq=519 Ack=185 Win=65535 Len=0

Рисунок 10-8. Изолированный обмен приёмника метеостанции

Прежде чем исследовать характеристики разговора, попробуем определить неизвестный IP-адрес. Без обширного исследования мы, возможно, не узнаем, именно ли с этим адресом должен связываться приёмник погоды Пита, но по крайней мере сможем посредством запроса WHOIS проверить, входит ли он в инфраструктуру Wunderground. Запрос WHOIS можно выполнить на большинстве веб-сайтов регистрации доменов или региональных интернет-регистраторов, например whois.arin.net. В данном случае IP-адрес, похоже, принадлежит Cogent, поставщику интернет-услуг (internet service provider, ISP) (рис. 10-9). Здесь также упоминается PSINet Inc., но быстрый поиск показывает, что в начале 2000-х годов большинство активов PSINet приобрела Cogent.



Network	
Net Range	38.0.0.0 - 38.255.255.255
CIDR	38.0.0.0/8
Name	COGENT-A
Handle	NET-38-0-0-0-1
Parent	
Net Type	Direct Allocation
Origin AS	AS174
Organization	PSINet, Inc. (PSI)
Registration Date	1991-04-16
Last Updated	2011-05-20
Comments	Reassignment information for this block can be found at rwhois.cogentco.com 4321
RESTful Link	https://whois.arin.net/rest/net/NET-38-0-0-0-1

Function	Point of Contact
Tech	PSI-NISC-ARIN (PSI-NISC-ARIN)

See Also	Related organization's POC records.
See Also	Related delegations.

Рисунок 10-9. Данные WHOIS определяют владельца IP-адреса

Иногда, если IP-адрес зарегистрирован непосредственно на организацию, запрос WHOIS вернёт её название. Однако компания часто просто использует адресное пространство IP поставщика услуг, не регистрируя его прямо на себя. В таких случаях полезен поиск номера автономной системы (autonomous system number, ASN), связанного с IP-адресом. Для поддержки определённых видов маршрутизации в общедоступном Интернете организации обязаны регистрировать ASN. Сопоставление IP и ASN можно найти несколькими способами (некоторые запросы WHOIS предоставляют его автоматически), но мне нравится автоматизированный инструмент поиска Team Cymru (asn.cymru.com). Применив его к `38.102.136.125`, мы видим связь с AS 36347, которая относится к «Wunderground - The Weather Channel, LLC, US» (рис. 10-10). Это сообщает, что устройство, с которым обменивается метеостанция, по крайней мере находится в нужном окружении. Если бы нам не удалось определить правильную принадлежность адреса, стоило бы проверить, не связывается ли приёмник Пита с неправильным устройством, но адрес подтверждается.

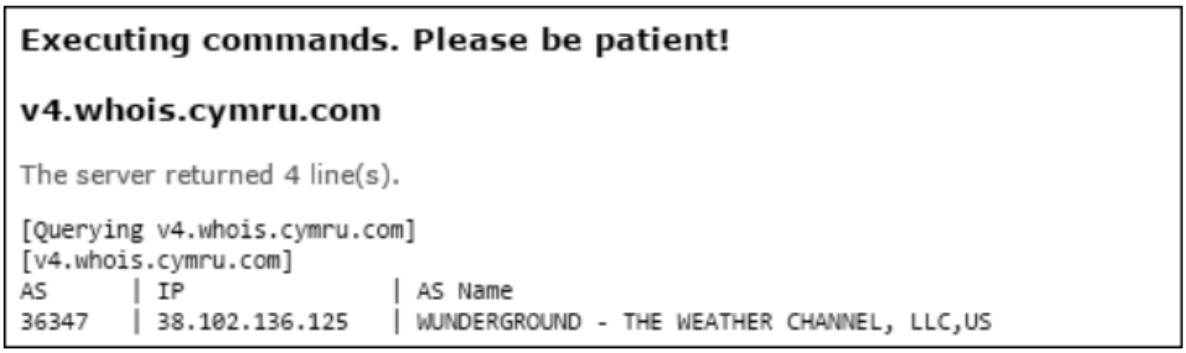


Рисунок 10-10. Сопоставление внешнего IP-адреса с ASN

Охарактеризовав неизвестный узел, можно углубиться в подробности обмена. Разговор сравнительно короткий: рукопожатие TCP, единственный запрос и ответ HTTP `GET`, затем закрытие TCP. Рукопожатие и закрытие выглядят успешными, поэтому проблема, вероятно, содержится в самом HTTP-запросе. Для подробного исследования проследим поток TCP (рис. 10-11).

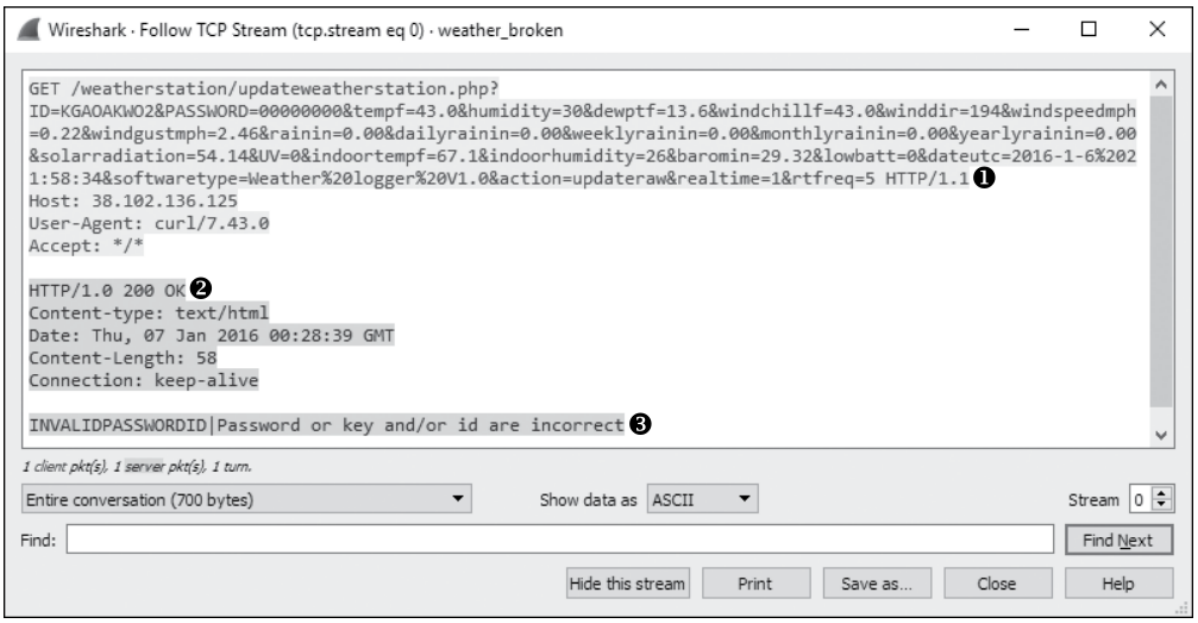


Рисунок 10-11. Отслеживание потока TCP обмена приёмника метеостанции

Обмен HTTP начинается с запроса GET от приёмника погоды Пита к Wunderground. Содержимое HTTP не передавалось, но значительный объём данных находился в URL (метка 1). Передача данных через строку запроса URL распространена в веб-приложениях, и, похоже, приёмник отправляет таким способом обновления погоды. Например, видны поля `tempf=43.0`, `dewptf=13.6` и `windchillf=43.0`. Сервер сбора Wunderground разбирает список полей и параметров URL и сохраняет их в базе данных.

На первый взгляд с запросом `GET` к серверу Wunderground всё в порядке. Но соответствующий ответ сообщает об ошибке. Сервер вернул код `HTTP/1.0 200 OK`, указывающий, что запрос GET получен и успешно обработан (метка 2), однако тело ответа содержит полезное сообщение: `INVALIDPASSWORDID|Password or key and/or id are incorrect` (метка 3).

Вернувшись к URL запроса, вы увидите, что первые два переданных параметра - `ID` и `PASSWORD`. Они служат для определения позывного метеостанции и её аутентификации на сервере Wunderground.

В данном случае идентификатор метеостанции Пита правильный, а пароль - нет. По неизвестной причине он заменён нулями. Поскольку последний известный успешный обмен произошёл в полночь, возможно, было применено обновление либо приёмник перезагрузился и потерял настройку пароля.

> **Примечание.** Хотя многие разработчики передают параметры в URL, передавать так пароли, как здесь, обычно считается плохой практикой. Причина в том, что при использовании HTTP без дополнительного шифрования, например HTTPS, запрашиваемые URL передаются открытым текстом. Поэтому злоумышленник, прослушивающий линию, может перехватить пароль.

На этом этапе Пит смог открыть настройки приёмника и ввести новый пароль. Вскоре метеостанция снова начала синхронизировать данные. Пример успешного обмена метеостанции находится в файле `weather_working.pcapng`. Поток обмена показан на рис. 10-12.

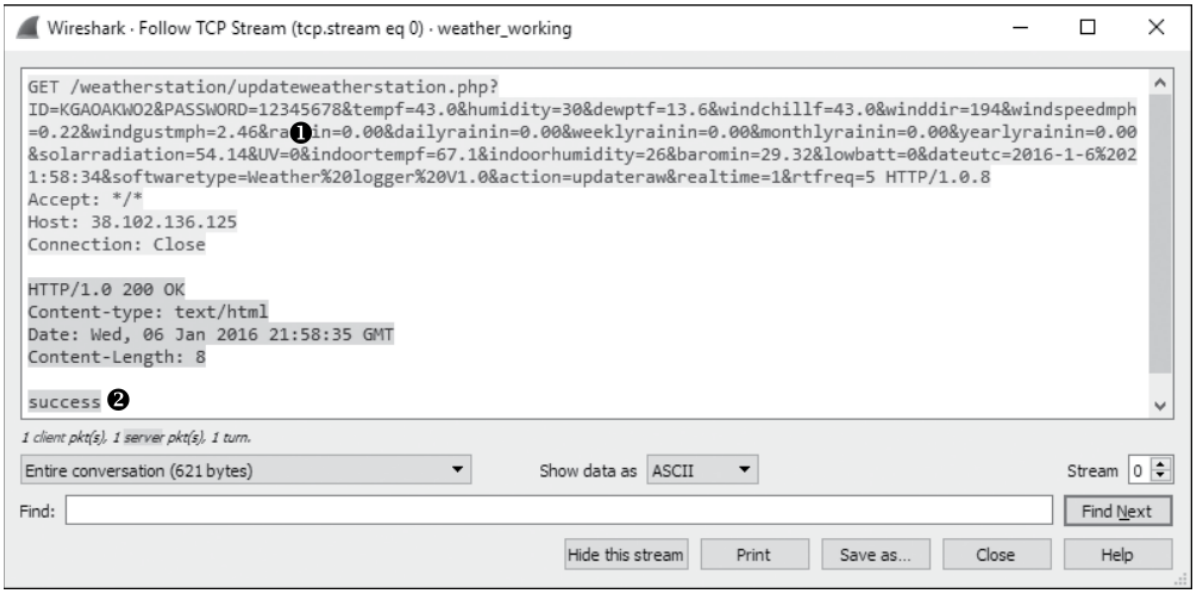


Рисунок 10-12. Успешный обмен метеостанции

Теперь пароль правильный (метка 1), а сервер Wunderground возвращает в теле HTTP-ответа сообщение об успехе (метка 2).

Усвоенные уроки

В этом сценарии мы столкнулись со сторонней службой, обеспечивающей сетевой обмен посредством функций другого протокола (HTTP). Проблемы обмена со сторонними службами приходится исправлять часто, и методы анализа пакетов прекрасно подходят для устранения таких неполадок, когда надлежащая документация или журналирование ошибок отсутствуют. Это встречается всё чаще по мере того, как вокруг появляется всё больше устройств Интернета вещей (Internet of Things, IoT), подобных этой метеостанции.

Исправление таких проблем требует умения исследовать неизвестные последовательности трафика и определять, как они должны работать. Некоторые приложения, например передача погодных данных на основе HTTP в этом сценарии, довольно просты. Другие очень сложны и требуют нескольких транзакций, добавления шифрования или даже собственных протоколов, которые Wireshark может не уметь разбирать изначально.

Исследуя всё больше сторонних служб, вы со временем начнёте замечать общие шаблоны, которыми разработчики обеспечивают сетевой обмен. Эти знания повысят эффективность устранения неполадок.

Нет доступа к Интернету

Во многих сценариях может потребоваться диагностировать и устранить проблемы подключения к Интернету. Рассмотрим несколько распространённых проблем.

Проблемы настройки шлюза

Следующий сценарий представляет частую проблему: пользователь не может получить доступ к Интернету. Мы проверили, что ему доступны все внутренние ресурсы сети, включая общие папки других рабочих станций и приложения на локальных серверах.

Архитектура сети проста: все клиенты и серверы подключены к ряду простых коммутаторов. Доступ в Интернет осуществляется через один маршрутизатор, служащий шлюзом по умолчанию, а адресные сведения IP предоставляются DHCP. Это очень распространённый сценарий в небольших офисах.

Подключение к линии

Чтобы определить причину, попросим пользователя попытаться открыть интернет-ресурс, пока анализатор слушает линию. С помощью сведений из главы 2 (см. рис. 2-15) определяем наиболее подходящий способ размещения анализатора.

Коммутаторы нашей сети не поддерживают зеркалирование портов. Нам и так придётся прервать работу пользователя для проверки, поэтому можно считать допустимым ещё раз отключить его. Хотя это не сценарий с высокой пропускной способностью, при наличии ответвитель был бы уместен. Полученный файл называется `nowebaccess1.pcapng`.

Анализ

Захват трафика начинается с запроса и ответа ARP, как показано на рис. 10-13. В пакете 1 компьютер пользователя с MAC-адресом `00:25:b3:bf:91:ee` и IP-адресом `172.16.0.8` отправляет широковещательный пакет ARP всем компьютерам сетевого сегмента, пытаясь найти MAC-адрес, связанный с IP-адресом шлюза по умолчанию `172.16.0.10`.

No.	Time	Source	Destination	Protocol	Length	Info
1	04:32:21.445645	00:25:b3:bf:91:ee	ff:ff:ff:ff:ff:ff	ARP	42	Who has 172.16.0.10? Tell 172.16.0.8
2	04:32:21.445735	00:24:81:a1:f6:79	00:25:b3:bf:91:ee	ARP	60	172.16.0.10 is at 00:24:81:a1:f6:79

Рисунок 10-13. Запрос и ответ ARP для шлюза компьютера по умолчанию

Ответ поступает в пакете 2, и компьютер пользователя узнаёт, что `172.16.0.10` находится по адресу `00:24:81:a1:f6:79`. Получив ответ, компьютер располагает маршрутом к шлюзу, который должен направить его в Интернет.

После ответа ARP компьютер должен попытаться разрешить DNS-имя веб-сайта в IP-адрес посредством DNS в пакете 3. Как показано на рис. 10-14, для этого компьютер отправляет пакет

запроса DNS основному серверу DNS `4.2.2.2` (метка 1).

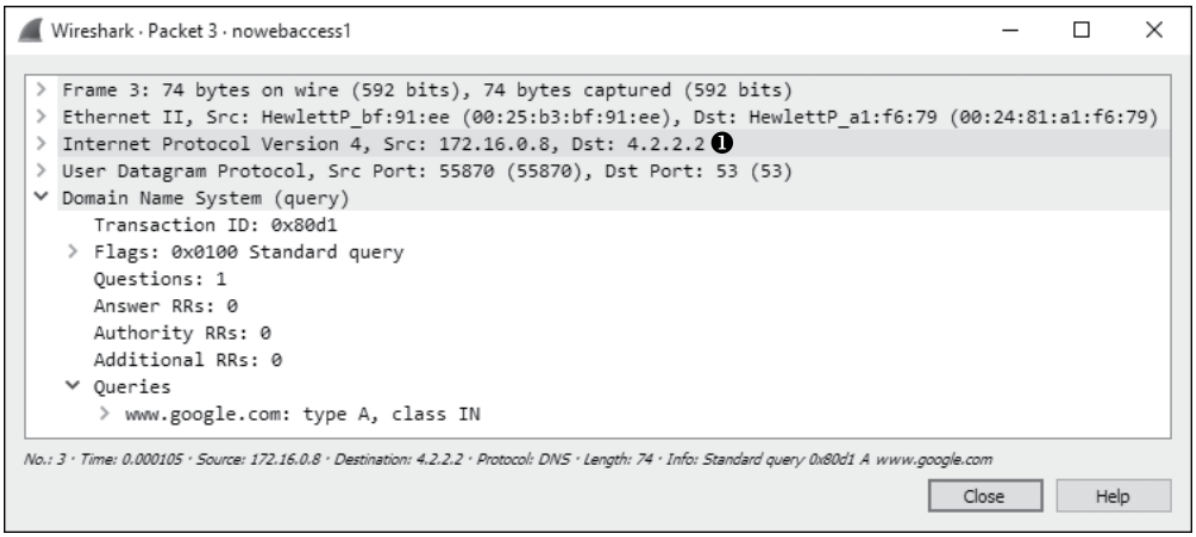


Рисунок 10-14. Запрос DNS, отправленный серверу 4.2.2.2

В обычных условиях сервер DNS отвечает на запрос очень быстро, но здесь этого не происходит. Вместо ответа виден тот же запрос DNS, второй раз отправленный на другой адрес назначения. Как показано на рис. 10-15, в пакете 4 второй запрос DNS отправляется вторичному серверу DNS, настроенному на компьютере, - `4.2.2.1` (метка 1).

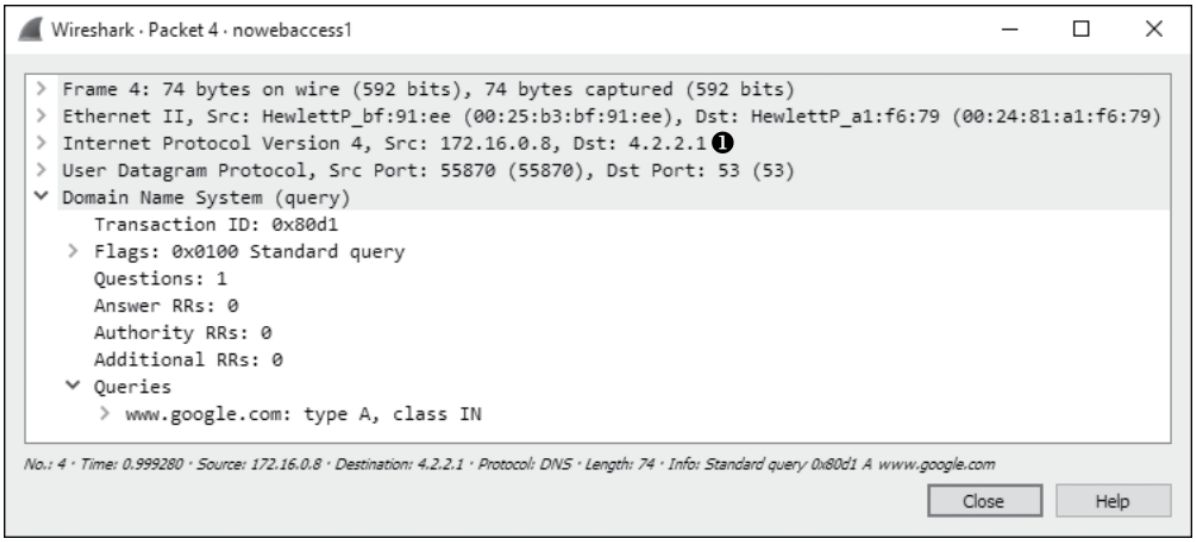
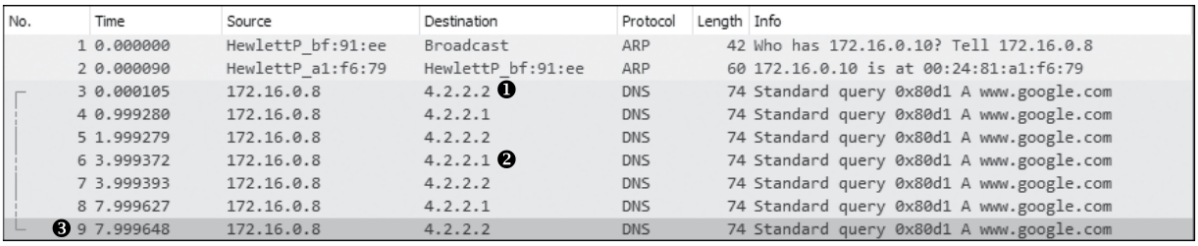


Рисунок 10-15. Второй запрос DNS, отправленный серверу 4.2.2.1

Сервер DNS снова не отвечает, и через 1 секунду запрос повторно отправляется на `4.2.2.2`. В следующие несколько секунд процесс повторяется с чередованием настроенных основного (метка 1) и вторичного (метка 2) серверов DNS, как показано на рис. 10-16. Весь процесс занимает около 8 секунд (метка 3), после чего браузер пользователя сообщает, что веб-сайт недоступен.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	HewlettP_bf:91:ee	Broadcast	ARP	42	Who has 172.16.0.10? Tell 172.16.0.8
2	0.000090	HewlettP_a1:f6:79	HewlettP_bf:91:ee	ARP	60	172.16.0.10 is at 00:24:81:a1:f6:79
3	0.000105	172.16.0.8	4.2.2.2 ❶	DNS	74	Standard query 0x80d1 A www.google.com
4	0.999280	172.16.0.8	4.2.2.1	DNS	74	Standard query 0x80d1 A www.google.com
5	1.999279	172.16.0.8	4.2.2.2	DNS	74	Standard query 0x80d1 A www.google.com
6	3.999372	172.16.0.8	4.2.2.1 ❷	DNS	74	Standard query 0x80d1 A www.google.com
7	3.999393	172.16.0.8	4.2.2.2	DNS	74	Standard query 0x80d1 A www.google.com
8	7.999627	172.16.0.8	4.2.2.1	DNS	74	Standard query 0x80d1 A www.google.com
❸ 9	7.999648	172.16.0.8	4.2.2.2	DNS	74	Standard query 0x80d1 A www.google.com

Рисунок 10-16. Запросы DNS повторяются до прекращения обмена

На основании увиденных пакетов можно начать локализовывать источник проблемы. Во-первых, мы видим успешный запрос ARP к устройству, которое считаем маршрутизатором-шлюзом сети по умолчанию, а значит, знаем, что устройство работает и обменивается данными. Мы также знаем, что компьютер пользователя действительно передаёт пакеты в сеть, поэтому можно предположить отсутствие проблемы со стеком протоколов самого компьютера. Проблема явно начинает проявляться при запросе DNS.

В этой сети запросы DNS разрешаются внешним сервером в Интернете (`4.2.2.2` или `4.2.2.1`). Поэтому для правильного разрешения отвечающий за маршрутизацию пакетов в Интернет маршрутизатор должен успешно переслать серверу запросы DNS, а сервер - ответить. Всё это должно произойти до того, как HTTP сможет запросить саму веб-страницу.

Поскольку у остальных пользователей нет проблем с подключением к Интернету, сетевой маршрутизатор и удалённый сервер DNS, вероятно, не являются источником проблемы. Остаётся исследовать только сам компьютер пользователя.

При более глубоком исследовании проблемного компьютера выясняется, что вместо адреса, назначенного DHCP, в нём вручную заданы адресные сведения, причём адрес шлюза по умолчанию указан неверно. Заданный шлюзом по умолчанию адрес не является маршрутизатором и не может пересылать пакеты запроса DNS за пределы сети.

Усвоенные уроки

Проблему в этом сценарии вызвал неправильно настроенный клиент. Хотя сама причина оказалась простой, она значительно повлияла на пользователя. У человека без знания сети или способности быстро проанализировать пакеты, как сделали мы, устранение такой простой ошибки настройки могло бы занять немало времени. Как видно, анализ пакетов не ограничивается крупными и сложными проблемами.

Обратите внимание: поскольку в начале сценария нам не был известен IP-адрес маршрутизатора-шлюза сети, Wireshark не определил проблему точно, но указал, где искать, сэкономив драгоценное время. Вместо исследования маршрутизатора-шлюза, обращения к поставщику интернет-услуг или поиска ресурсов для устранения неполадок удалённого сервера DNS мы смогли сосредоточиться на самом компьютере, который и был источником проблемы.

> **Примечание.** Если бы мы лучше знали схему IP-адресации этой сети, анализ прошёл бы ещё быстрее. Проблему можно было определить сразу, заметив, что запрос ARP отправлен IP-адресу, отличному от адреса маршрутизатора-шлюза. Такие простые ошибки настройки часто вызывают сетевые проблемы и обычно быстро устраняются при помощи небольшого анализа пакетов.

Нежелательное перенаправление

В этом сценарии пользователь снова испытывает затруднения с доступом в Интернет со своей рабочей станции. Однако, в отличие от пользователя предыдущего сценария, Интернет ему доступен. Проблема в том, что он не может открыть домашнюю страницу google.com. При попытке обратиться к любому домену Google пользователь перенаправляется на страницу браузера с сообщением «Internet Explorer cannot display the web page». Проблема затрагивает только этого пользователя.

Как и в предыдущем сценарии, это небольшая сеть с несколькими простыми коммутаторами и одним маршрутизатором, служащим шлюзом по умолчанию.

Подключение к линии

Для начала анализа просим пользователя попытаться открыть google.com, а сами посредством ответвителя прослушиваем сформированный трафик. Полученный файл называется `nowebaccess2.pcapng`.

Анализ

Захват начинается с запроса и ответа ARP, показанных на рис. 10-17. В пакете 1 компьютер пользователя с MAC-адресом `00:25:b3:bf:91:ee` и IP-адресом `172.16.0.8` отправляет широковещательный пакет ARP всем компьютерам сетевого сегмента, пытаясь найти MAC-адрес, связанный с IP-адресом узла `172.16.0.102`. Мы не сразу узнаём этот адрес.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	00:25:b3:bf:91:ee	ff:ff:ff:ff:ff:ff	ARP	42	Who has 172.16.0.102? Tell 172.16.0.8
2	0.000334	00:21:70:c0:56:f0	00:25:b3:bf:91:ee	ARP	60	172.16.0.102 is at 00:21:70:c0:56:f0

Рисунок 10-17. Запрос и ответ ARP для другого устройства сети

В пакете 2 компьютер пользователя узнаёт, что IP-адрес `172.16.0.102` находится по адресу `00:21:70:c0:56:f0`. На основании предыдущего сценария можно предположить, что это адрес маршрутизатора-шлюза и он используется для дальнейшей пересылки пакетов внешнему серверу DNS. Однако, как показано на рис. 10-18, следующий пакет - не запрос DNS, а пакет TCP от `172.16.0.8` к `172.16.0.102`. В нём установлен флаг SYN (метка 3), а значит, это первый пакет рукопожатия нового соединения на основе TCP между двумя узлами.

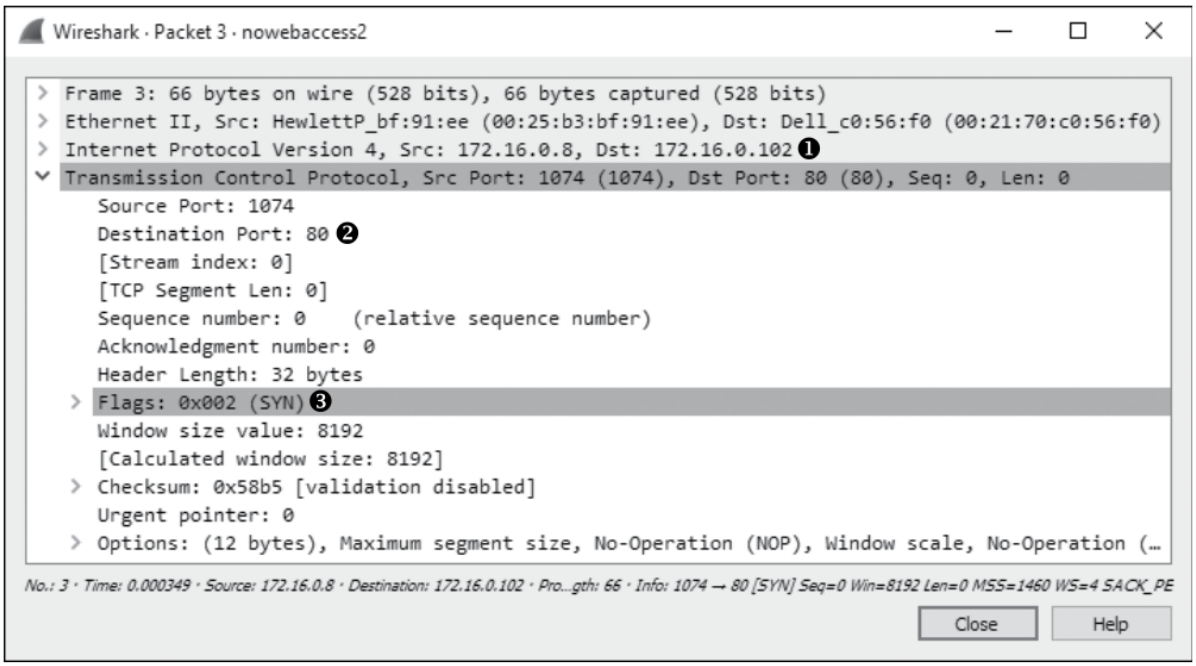


Рисунок 10-18. Пакет TCP SYN от одного внутреннего узла другому

Примечательно, что попытка TCP-соединения направлена на порт 80 (метка 2) узла `172.16.0.102` (метка 1), обычно связанный с трафиком HTTP.

Как показано на рис. 10-19, попытка соединения внезапно прерывается, когда узел `172.16.0.102` в ответ отправляет пакет TCP (пакет 4) с установленными флагами RST и ACK (метка 1).

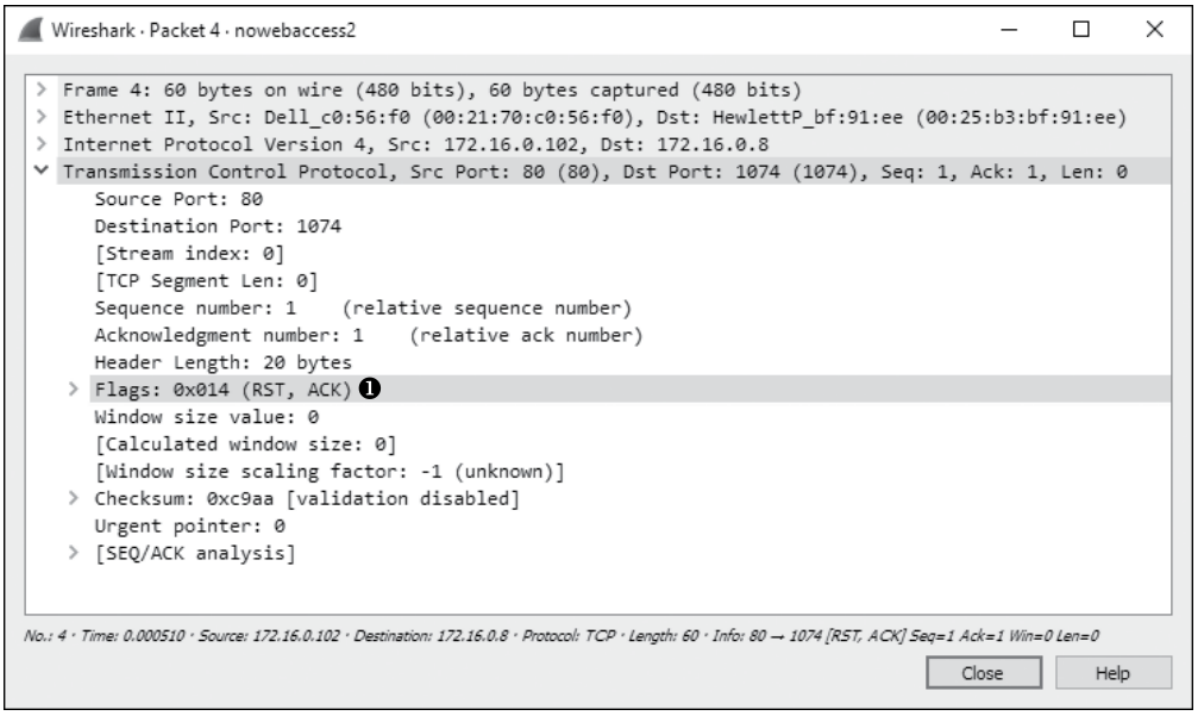
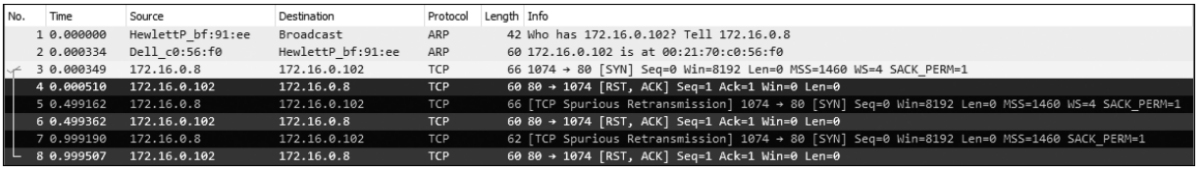


Рисунок 10-19. Пакет TCP RST, отправленный в ответ на TCP SYN

Вспомните из главы 8, что пакет с установленным флагом RST служит для завершения TCP-соединения. Здесь узел `172.16.0.8` попытался установить TCP-соединение с узлом `172.16.0.102` через порт 80. К сожалению, поскольку на этом узле не настроены службы, ожидающие запросов на порте 80, для закрытия соединения отправляется пакет TCP RST. Процесс

повторяется трижды, прежде чем обмен окончательно прекращается, как показано на рис. 10-20. В этот момент пользователь получает в браузере сообщение о невозможности отобразить страницу.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	HewlettP_bf:91:ee	Broadcast	ARP	42	Who has 172.16.0.102? Tell 172.16.0.8
2	0.000334	Dell_c0:56:f0	HewlettP_bf:91:ee	ARP	60	172.16.0.102 is at 00:21:70:c0:56:f0
3	0.000349	172.16.0.8	172.16.0.102	TCP	66	1074 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
4	0.000510	172.16.0.102	172.16.0.8	TCP	60	80 → 1074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5	0.499162	172.16.0.8	172.16.0.102	TCP	66	[TCP Spurious Retransmission] 1074 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
6	0.499362	172.16.0.102	172.16.0.8	TCP	60	80 → 1074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
7	0.999190	172.16.0.8	172.16.0.102	TCP	62	[TCP Spurious Retransmission] 1074 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 SACK_PERM=1
8	0.999507	172.16.0.102	172.16.0.8	TCP	60	80 → 1074 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Рисунок 10-20. Пакеты TCP SYN и RST встречаются в общей сложности трижды

Исследовав настройки другого, правильно работающего сетевого устройства, мы начинаем подозревать неладное в запросе и ответе ARP из пакетов 1 и 2: запрос ARP предназначен не фактическому MAC-адресу маршрутизатора-шлюза, а неизвестному устройству. После запроса и ответа ARP мы ожидали бы запрос DNS к настроенному серверу DNS для поиска IP-адреса, связанного с google.com, но его нет. Созданию запроса DNS могут помешать два условия:

• Устройство, инициирующее соединение, уже имеет сопоставление DNS-имени и IP-адреса в кеше DNS (как в первом сценарии этой главы).
• Устройство, подключающееся к DNS-имени, уже имеет сопоставление DNS-имени и IP-адреса в файле `hosts`.

При дальнейшем исследовании клиентского компьютера выясняется, что его файл `hosts` содержит запись для google.com, связанную с внутренним IP-адресом `172.16.0.102`. Эта ошибочная запись и является источником проблем пользователя.

Обычно компьютер считает файл `hosts` авторитетным источником сопоставлений DNS-имён и IP-адресов и проверяет его до запроса к внешнему источнику. В этом сценарии компьютер пользователя проверил файл `hosts`, нашёл запись для google.com и решил, что google.com фактически находится в его локальном сетевом сегменте. Затем он отправил узлу запрос ARP, получил ответ и попытался инициировать TCP-соединение с `172.16.0.102` через порт 80. Но поскольку удалённая система не была настроена как веб-сервер, она не принимала попытки соединения.

После удаления записи из файла `hosts` компьютер пользователя начал обмениваться данными правильно и получил доступ к google.com.

> **Примечание.** Чтобы исследовать файл `hosts` в Windows, откройте `C:\Windows\System32\drivers\etc\hosts`. В Linux просмотрите `/etc/hosts`.

Этот очень распространённый сценарий много лет используется вредоносными программами для перенаправления пользователей на веб-сайты с вредоносным кодом. Представьте, что злоумышленник изменил файл `hosts` так, чтобы при каждом входе в интернет-банк вы перенаправлялись на поддельный сайт, созданный для кражи учётных данных!

Усвоенные уроки

Продолжая анализировать трафик, вы будете узнавать и о том, как различные протоколы работают, и о том, как их сломать. В этом сценарии запрос DNS не отправлялся из-за неправильной настройки клиента, а не из-за внешних ограничений или ошибок конфигурации.

Исследовав проблему на уровне пакетов, мы смогли быстро заметить неизвестный IP-адрес и определить отсутствие DNS - ключевого компонента процесса обмена. Эти сведения позволили определить клиента как источник проблемы.

Проблемы выше по потоку

Как и в двух предыдущих сценариях, здесь пользователь жалуется на отсутствие доступа к Интернету со своей рабочей станции. Он сузил проблему до одного веб-сайта - google.com. Дальнейшее исследование показывает, что проблема затрагивает всех в организации: никто не может открыть домены Google.

Сеть настроена так же, как в двух предыдущих сценариях: несколько простых коммутаторов и один маршрутизатор, соединяющий сеть с Интернетом.

Подключение к линии

Для устранения неполадки сначала открываем google.com, чтобы сформировать трафик. Поскольку проблема охватывает всю сеть, в идеале её можно воспроизвести на любом сетевом устройстве посредством большинства способов захвата. Файл, полученный захватом через ответвитель, называется `nowebaccess3.pcapng`.

Анализ

Этот захват пакетов начинается с трафика DNS, а не с привычного ARP. Поскольку первый пакет захвата направлен на внешний адрес, а пакет 2 содержит ответ с этого адреса, можно предположить, что процесс ARP уже произошёл и сопоставление MAC- и IP-адресов маршрутизатора-шлюза уже находится в кеше ARP узла `172.16.0.8`.

Как показано на рис. 10-21, первый пакет захвата идёт от узла `172.16.0.8` к адресу `4.2.2.1` (метка 1) и является пакетом DNS (метка 2). Исследовав содержимое, мы видим запрос записи A для `www.google.com` (метка 3), которая сопоставит DNS-имя с IP-адресом.

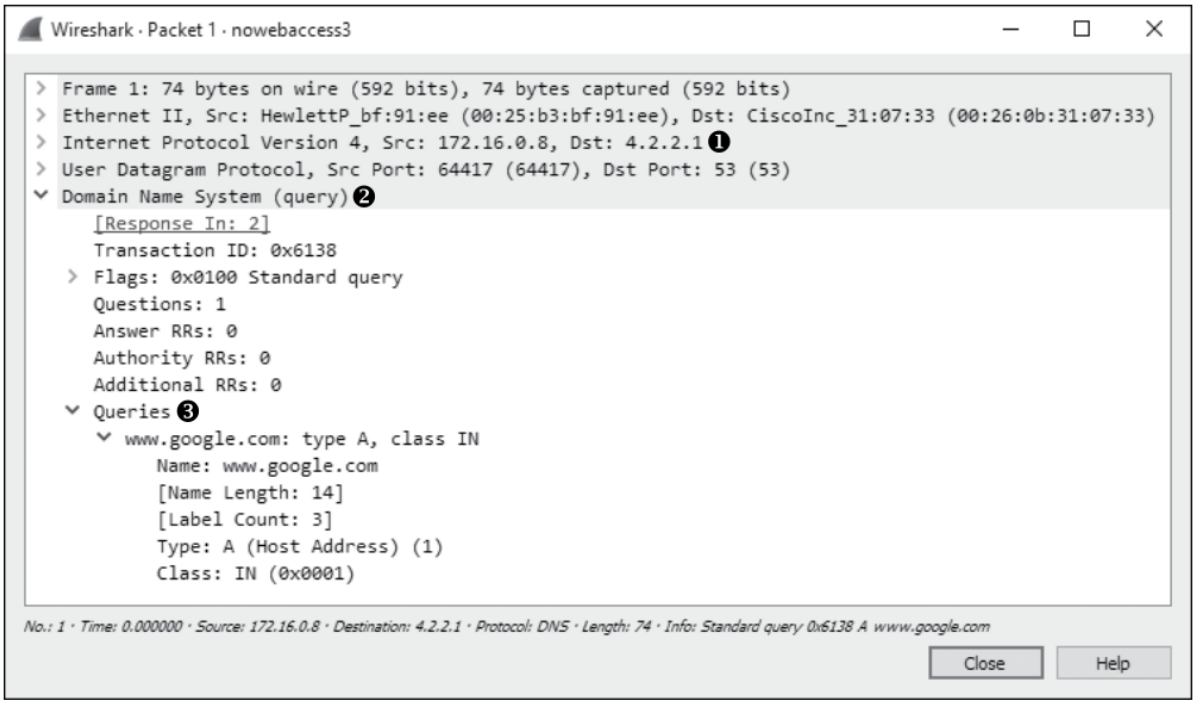


Рисунок 10-21. Запрос DNS записи A для www.google.com

Ответ от `4.2.2.1` на запрос - второй пакет файла захвата, показанный на рис. 10-22. Здесь видно, что ответивший сервер имён предоставил несколько ответов на запрос (метка 1). Пока всё выглядит правильно, и обмен происходит как должен.

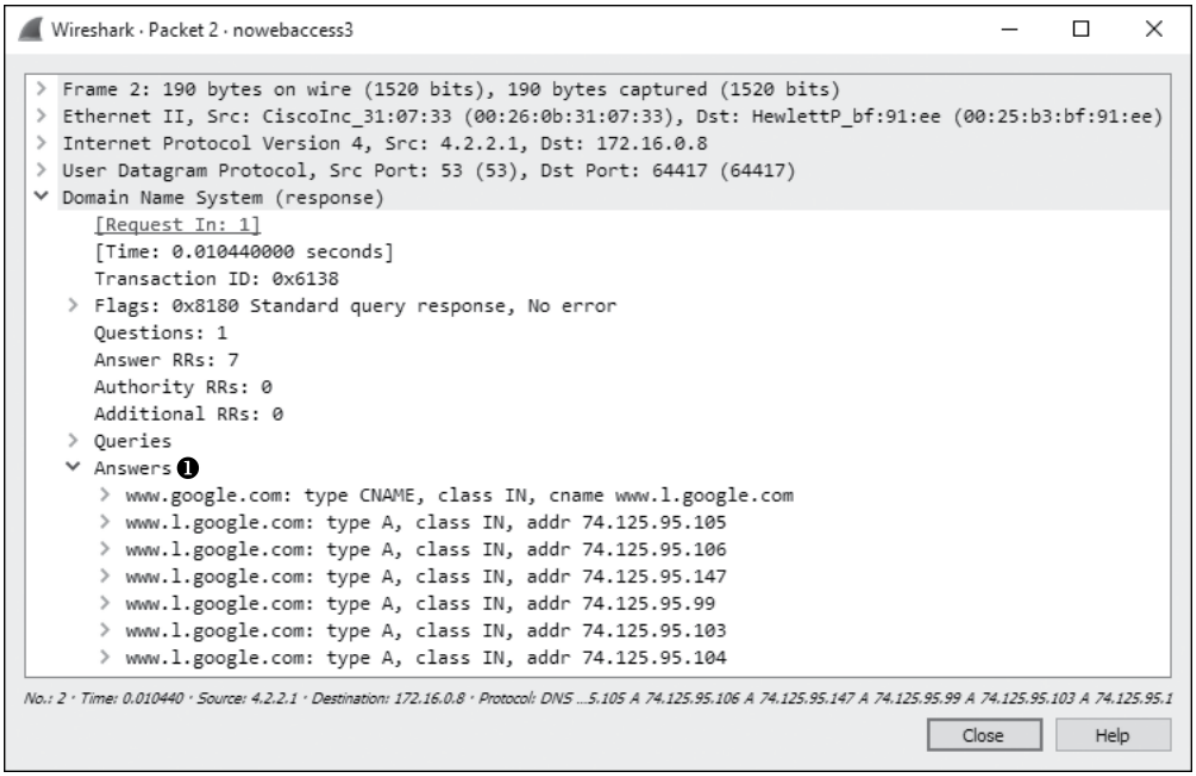


Рисунок 10-22. Ответ DNS с несколькими записями A

Теперь, когда компьютер пользователя определил IP-адрес веб-сервера, он может попытаться связаться с ним. Как показано на рис. 10-23, процесс начинается в пакете 3: пакет TCP отправляется с `172.16.0.8` на `74.125.95.105` (метка 1). Этот адрес назначения взят из первой записи A, предоставленной ответом DNS в пакете 2. В пакете TCP установлен флаг SYN (метка 3), и он пытается связаться с удалённым сервером через порт 80 (метка 2).

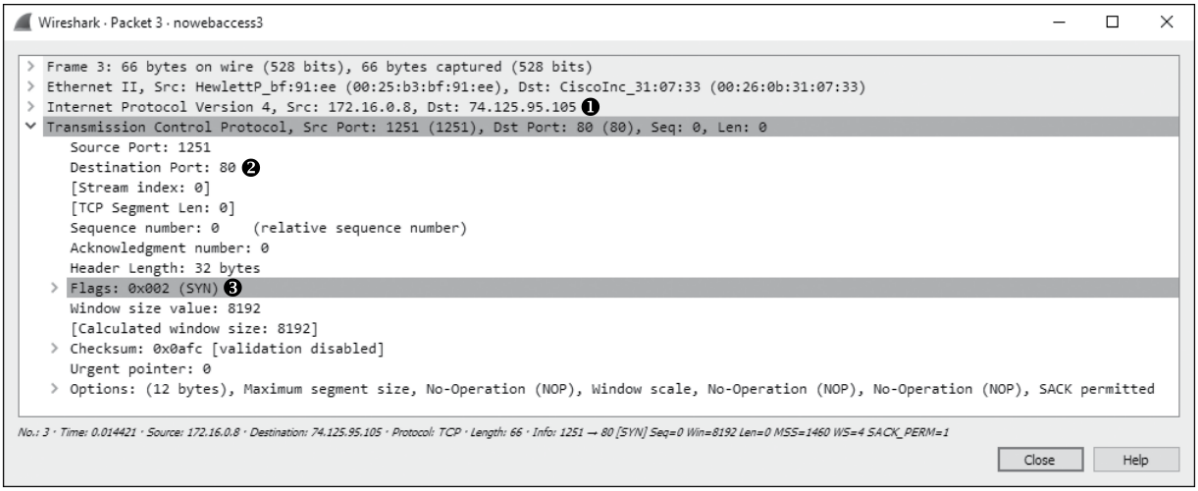
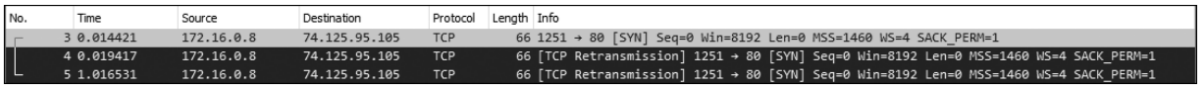


Рисунок 10-23. Пакет SYN пытается инициировать соединение через порт 80

Поскольку это процесс рукопожатия TCP, в ответ должен быть отправлен пакет TCP SYN/ACK, но вместо этого через короткое время источник передаёт назначению ещё один пакет SYN. Примерно через секунду процесс повторяется ещё раз, как показано на рис. 10-24, после чего обмен

прекращается, а браузер сообщает, что веб-сайт не найден.



No.	Time	Source	Destination	Protocol	Length	Info
3	0.014421	172.16.0.8	74.125.95.105	TCP	66	1251 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
4	0.019417	172.16.0.8	74.125.95.105	TCP	66	[TCP Retransmission] 1251 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
5	1.016531	172.16.0.8	74.125.95.105	TCP	66	[TCP Retransmission] 1251 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1

Рисунок 10-24. Пакет TCP SYN отправляется трижды, но ответ не получен

При устранении этой проблемы учтём, что рабочая станция нашей сети способна связаться с внешним миром, поскольку запрос DNS к внешнему серверу `4.2.2.1` успешен. Сервер DNS отвечает, по всей видимости, действительным адресом, и узлы пытаются подключиться к одному из этих адресов. Локальная рабочая станция, с которой мы пытаемся установить соединение, также выглядит исправной.

Проблема в том, что удалённый сервер просто не отвечает на запросы соединения; пакет TCP RST не отправляется. Это может происходить по нескольким причинам: неправильно настроенный веб-сервер, повреждённый стек протоколов веб-сервера или устройство фильтрации пакетов в удалённой сети (например, межсетевой экран). Если предположить, что локального устройства фильтрации пакетов нет, все остальные возможные решения находятся в удалённой сети и вне нашего контроля. В данном случае веб-сервер работал неправильно, и ни одна попытка доступа к нему не завершалась успешно. После устранения проблемы на стороне Google обмен смог продолжиться.

Усвоенные уроки

В этом сценарии мы не могли самостоятельно исправить проблему. Анализ определил, что её источник находится не в узлах нашей сети, маршрутизаторе или внешнем сервере DNS, предоставляющем службу разрешения имён. Проблема лежала за пределами нашей сетевой инфраструктуры.

Иногда открытие того, что проблема на самом деле не наша, не только снимает напряжение, но и помогает сохранить лицо, когда руководство требует ответа. Мне доводилось спорить со многими поставщиками интернет-услуг, оборудования и программного обеспечения, утверждавшими, что проблема не на их стороне, но, как вы только что увидели, пакеты не лгут.

Непостоянно работающий принтер

В следующем сценарии администратор службы поддержки ИТ испытывает трудности с устранением проблемы печати. Пользователи отдела продаж сообщают, что высокопроизводительный принтер работает неправильно. Когда пользователь отправляет на него большое задание, принтер печатает несколько страниц, а затем останавливается до окончания задания. Было безуспешно опробовано несколько изменений настроек драйвера. Сотрудники службы поддержки хотят убедиться, что проблема не связана с сетью.

Подключение к линии

Общий элемент проблемы - принтер, поэтому начнём с размещения анализатора как можно ближе к нему. Установить Wireshark на самом принтере нельзя, но в этой сети используются усовершенствованные коммутаторы уровня 3, поэтому можно применить зеркалирование портов. Зеркалируем порт принтера на свободный порт и подключим к нему ноутбук с Wireshark. После настройки попросим пользователя отправить принтеру большое задание и будем наблюдать вывод. Полученный файл захвата называется `inconsistent_printer.pcapng`.

Анализ

В начале файла захвата соединение инициируется рукопожатием TCP между сетевой рабочей станцией, отправляющей задание (`172.16.0.8`), и принтером (`172.16.0.253`). После рукопожатия в пакете 4 принтеру отправляется пакет данных TCP размером 1460 байт (метка 1; рис. 10-25). Объём данных виден в крайней правой части столбца Info панели списка пакетов либо внизу сведений заголовка TCP на панели подробностей пакета.

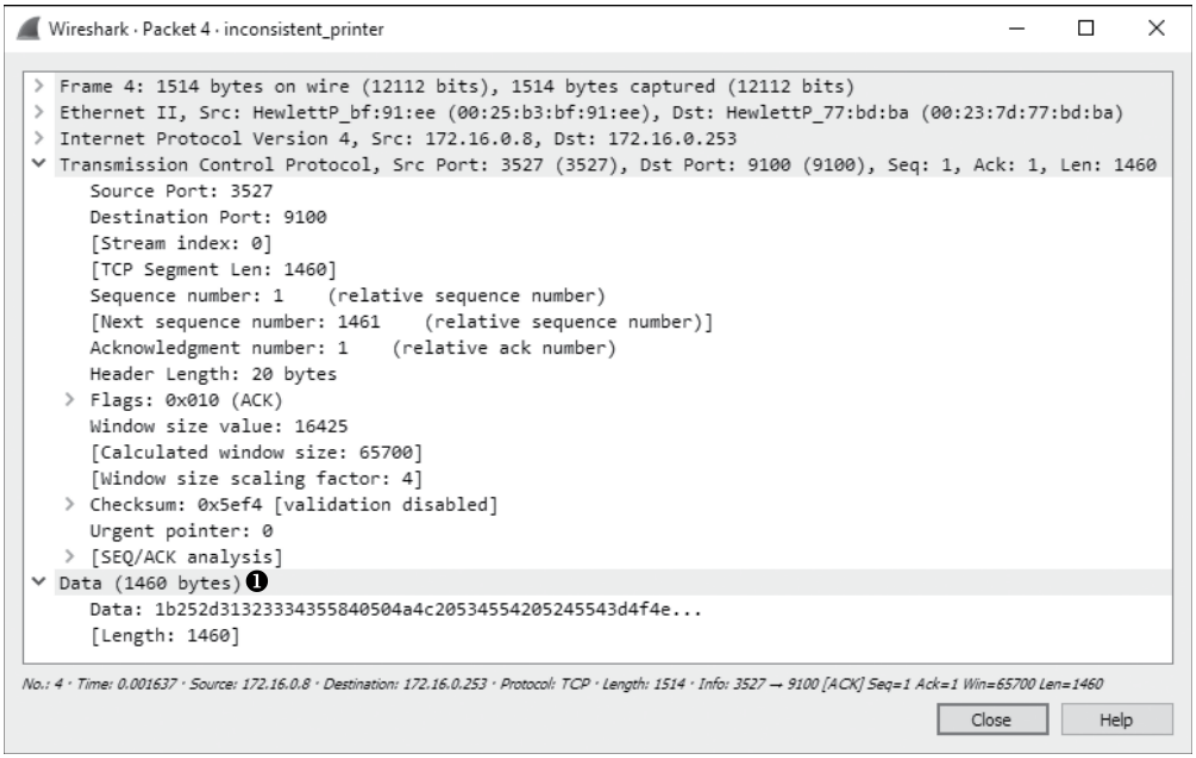
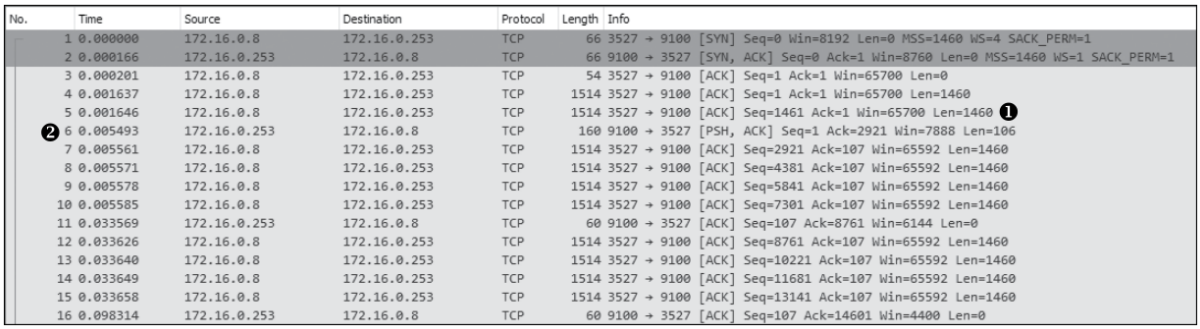


Рисунок 10-25. Передача данных принтеру по TCP

После пакета 4 отправляется ещё один пакет данных, также содержащий 1460 байт (метка 1), как видно на рис. 10-26. Принтер подтверждает эти данные в пакете 6 (метка 2).



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.0.8	172.16.0.253	TCP	66	3527 → 9100 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.000166	172.16.0.253	172.16.0.8	TCP	66	9100 → 3527 [SYN, ACK] Seq=0 Ack=1 Win=8760 Len=0 MSS=1460 WS=1 SACK_PERM=1
3	0.000201	172.16.0.8	172.16.0.253	TCP	54	3527 → 9100 [ACK] Seq=1 Ack=1 Win=65700 Len=0
4	0.001637	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=1 Ack=1 Win=65700 Len=1460
5	0.001646	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=1461 Ack=1 Win=65700 Len=1460 ❶
❷ 6	0.005493	172.16.0.253	172.16.0.8	TCP	160	9100 → 3527 [PSH, ACK] Seq=1 Ack=2921 Win=7888 Len=106
7	0.005561	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=2921 Ack=107 Win=65592 Len=1460
8	0.005571	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=4381 Ack=107 Win=65592 Len=1460
9	0.005578	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=5841 Ack=107 Win=65592 Len=1460
10	0.005585	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=7301 Ack=107 Win=65592 Len=1460
11	0.033569	172.16.0.253	172.16.0.8	TCP	60	9100 → 3527 [ACK] Seq=107 Ack=8761 Win=6144 Len=0
12	0.033626	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=8761 Ack=107 Win=65592 Len=1460
13	0.033640	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=10221 Ack=107 Win=65592 Len=1460
14	0.033649	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=11681 Ack=107 Win=65592 Len=1460
15	0.033658	172.16.0.8	172.16.0.253	TCP	1514	3527 → 9100 [ACK] Seq=13141 Ack=107 Win=65592 Len=1460
16	0.098314	172.16.0.253	172.16.0.8	TCP	60	9100 → 3527 [ACK] Seq=107 Ack=14601 Win=4400 Len=0

Рисунок 10-26. Нормальная передача данных и подтверждения TCP

Поток данных продолжается до последних пакетов захвата. Пакет 121 является пакетом повторной передачи TCP и признаком проблемы, как показано на рис. 10-27.

Пакет повторной передачи TCP отправляется, когда одно устройство передаёт пакет TCP удалённому устройству, а то не подтверждает передачу. После достижения порога повторной передачи отправитель считает, что удалённое устройство не получило данные, и передаёт пакет повторно. Процесс повторяется несколько раз, после чего обмен фактически прекращается.

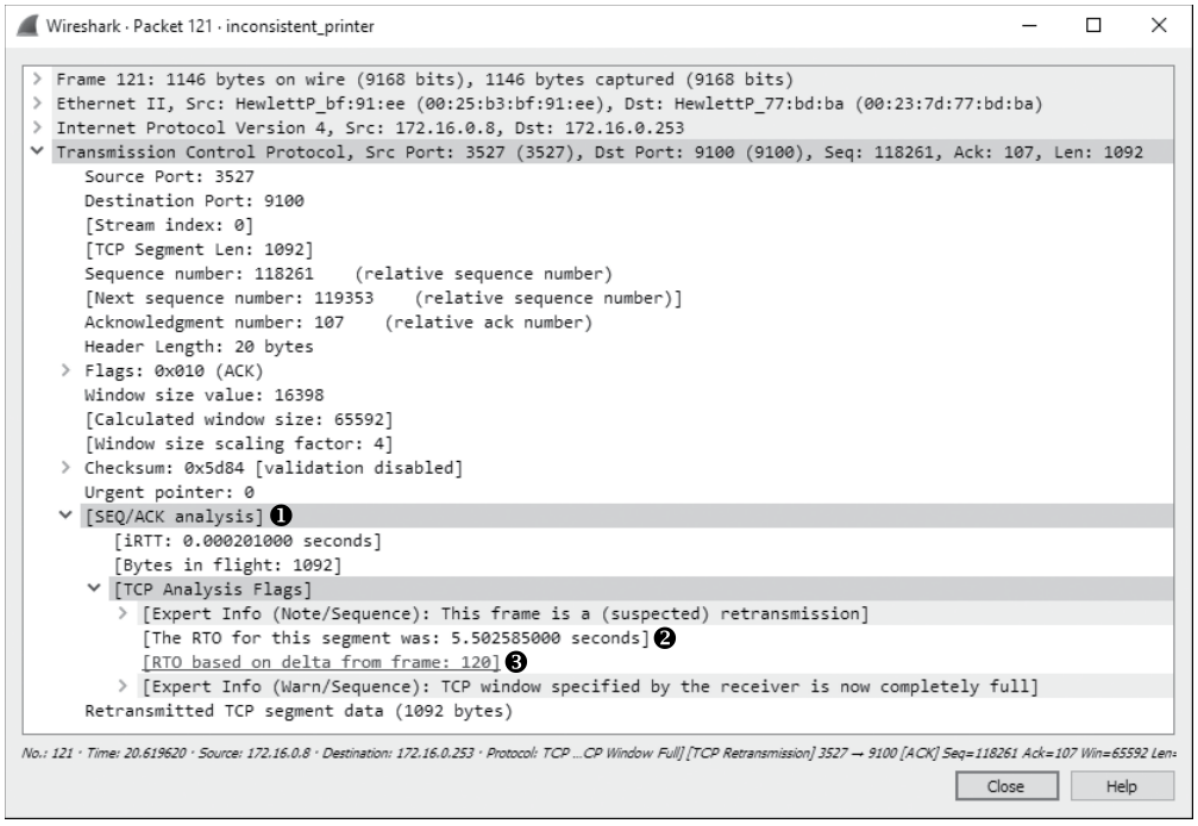


Рисунок 10-27. Эти пакеты повторной передачи TCP указывают на возможную проблему

В этом сценарии повторная передача направляется от клиентской рабочей станции принтеру, поскольку принтер не подтвердил переданные данные. Как показано на рис. 10-27, развернув раздел анализа SEQ/ACK заголовка TCP (метка 1) и дополнительные сведения под ним, можно увидеть, почему это повторная передача. Согласно сведениям, обработанным Wireshark, пакет 121 повторно передаёт пакет 120 (метка 3). Кроме того, тайм-аут повторной передачи (retransmission timeout, RTO) пакета составлял около 5,5 секунды (метка 2).

При анализе задержки между пакетами формат отображения времени можно менять в соответствии с задачей. Здесь нам нужно увидеть, через сколько времени после отправки

предыдущего пакета произошли повторные передачи, поэтому выберите View > Time Display Format > Seconds Since Previous Captured Packet. Теперь, как показано на рис. 10-28, ясно видно, что повторная передача в пакете 121 происходит через 5,5 секунды после отправки исходного пакета 120 (метка 1).

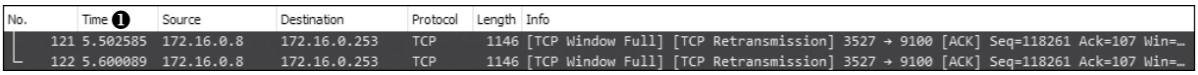


Рисунок 10-28. Просмотр времени между пакетами полезен при устранении неполадок

Следующий пакет - ещё одна повторная передача пакета 120. Его RTO равен 11,10 секунды, включая 5,5 секунды RTO предыдущего пакета. Взгляд на столбец Time панели списка пакетов показывает, что повторная передача была отправлена через 5,6 секунды после предыдущей. По всей видимости, это последний пакет файла захвата, и неслучайно принтер перестаёт печатать примерно в этот момент.

В этом сценарии у нас есть преимущество: мы имеем дело только с двумя устройствами внутри собственной сети, поэтому нужно определить, кто виноват - клиентская рабочая станция или принтер. Видно, что данные некоторое время передаются правильно, а затем принтер просто перестаёт отвечать рабочей станции. Рабочая станция прилагает все усилия, чтобы доставить данные назначению, о чём свидетельствуют повторные передачи, но ответа не получает. Проблема воспроизводится независимо от компьютера, отправляющего задание печати, поэтому предполагаем, что её источник - принтер.

Дальнейший анализ показывает неисправность оперативной памяти принтера. При отправке больших заданий он печатает лишь определённое число страниц, вероятно, пока не обращается к некоторым областям памяти. Затем проблема памяти лишает принтер способности принимать новые данные, и он прекращает обмен с узлом, передающим задание.

Усвоенные уроки

Хотя проблема принтера не была вызвана сетью, с помощью Wireshark мы смогли точно определить её местонахождение. В отличие от предыдущих сценариев, этот был сосредоточен исключительно на трафике TCP. Поскольку TCP заботится о надёжной передаче данных, при внезапном прекращении обмена между двумя устройствами он часто оставляет нам полезные сведения.

В данном случае, когда обмен внезапно остановился, мы смогли определить место проблемы лишь по встроенной функции повторной передачи TCP. В следующих сценариях мы часто будем опираться на подобные функции для устранения более сложных неполадок.

Нет подключения к филиалу

В этом сценарии у компании есть центральная штаб-квартира и недавно развёрнутый удалённый филиал. Большая часть ИТ-инфраструктуры находится в центральном офисе и использует домен на основе Windows Server. Инфраструктуру поддерживают контроллер домена, сервер DNS и сервер приложений, на котором размещено веб-программное обеспечение, ежедневно используемое сотрудниками. Филиал соединён маршрутизаторами, образующими канал глобальной сети (wide area network, WAN). В филиале находятся рабочие станции пользователей и подчинённый сервер DNS, который должен получать сведения о ресурсных записях от вышестоящего сервера DNS в корпоративной штаб-квартире. На рис. 10-29 показана схема обоих офисов и их соединения.

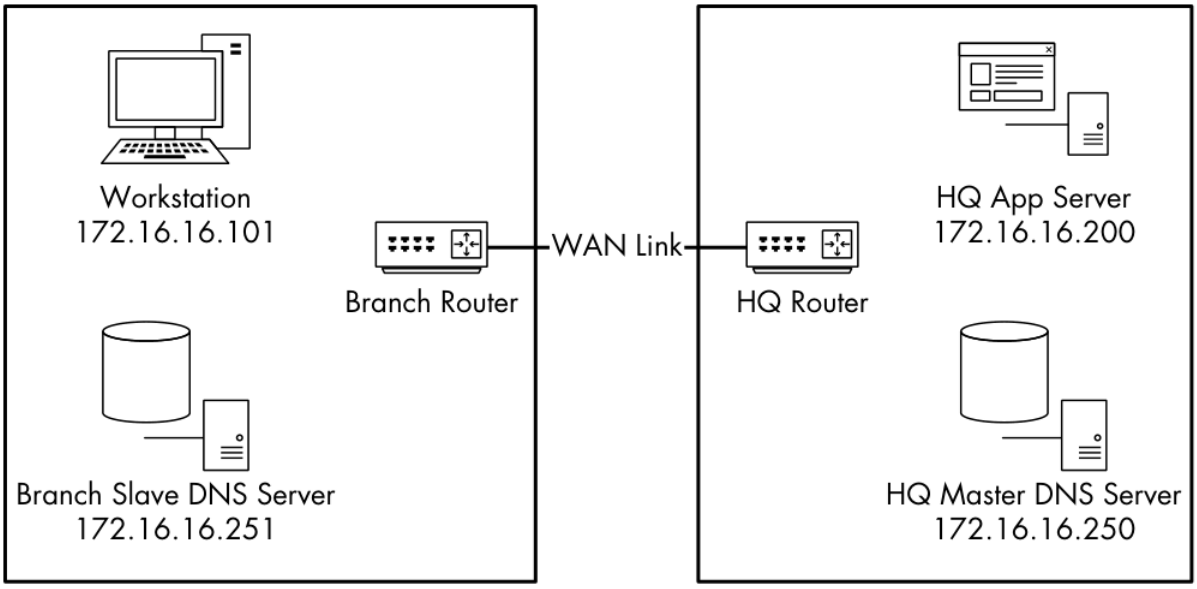


Рисунок 10-29. Компоненты, относящиеся к проблеме изолированного филиала

Группа развёртывания вводит в эксплуатацию новую инфраструктуру филиала и обнаруживает, что из его сети никто не может получить доступ к серверу веб-приложения интрасети. Сервер находится в главном офисе, и доступ к нему осуществляется через канал WAN. Проблема подключения затрагивает всех пользователей филиала. При этом всем им доступны Интернет и другие ресурсы внутри филиала.

Подключение к линии

Поскольку проблема находится в обмене между главным офисом и филиалом, начать сбор данных можно в нескольких местах. Причина может быть в клиентах внутри филиала, поэтому сначала зеркалируем порт одного из этих компьютеров, чтобы проверить, что он видит в линии. Получив сведения, мы сможем по ним выбрать другие точки сбора, которые помогут решить проблему. Исходный файл захвата с одного из клиентов называется `stranded_clientside.pcapng`.

Анализ

Как показано на рис. 10-30, первый захват начинается, когда пользователь рабочей станции `172.16.16.101` пытается обратиться к приложению на сервере приложений штаб-квартиры `172.16.16.200`. Захват содержит всего два пакета. В первом пакете, по всей видимости, на `172.16.16.251` отправлен запрос DNS (метка 1) записи A (метка 3) для `appserver` (метка 2). Это DNS-имя сервера `172.16.16.200` в центральном офисе.

Как видно на рис. 10-31, ответ на пакет сообщает о сбое сервера (метка 1), то есть что-то мешает успешному разрешению DNS-запроса. Обратите внимание: поскольку это ошибка (сбой сервера), пакет не отвечает на запрос (метка 2).

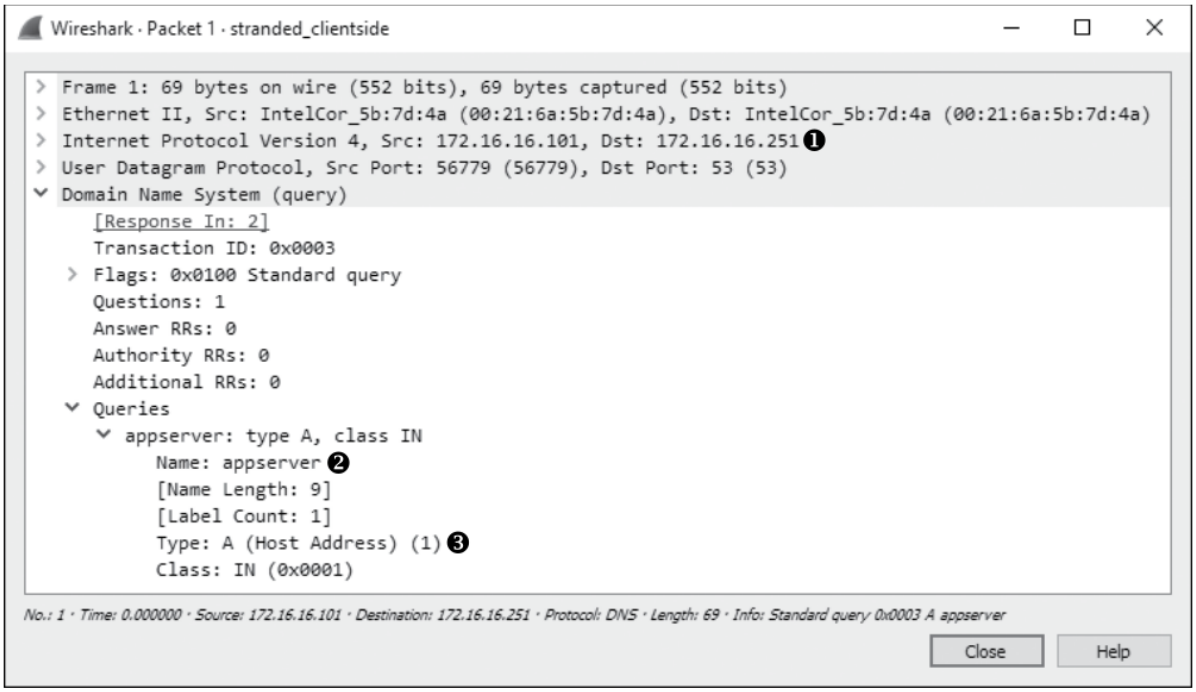


Рисунок 10-30. Обмен начинается с запроса DNS записи A для appserver

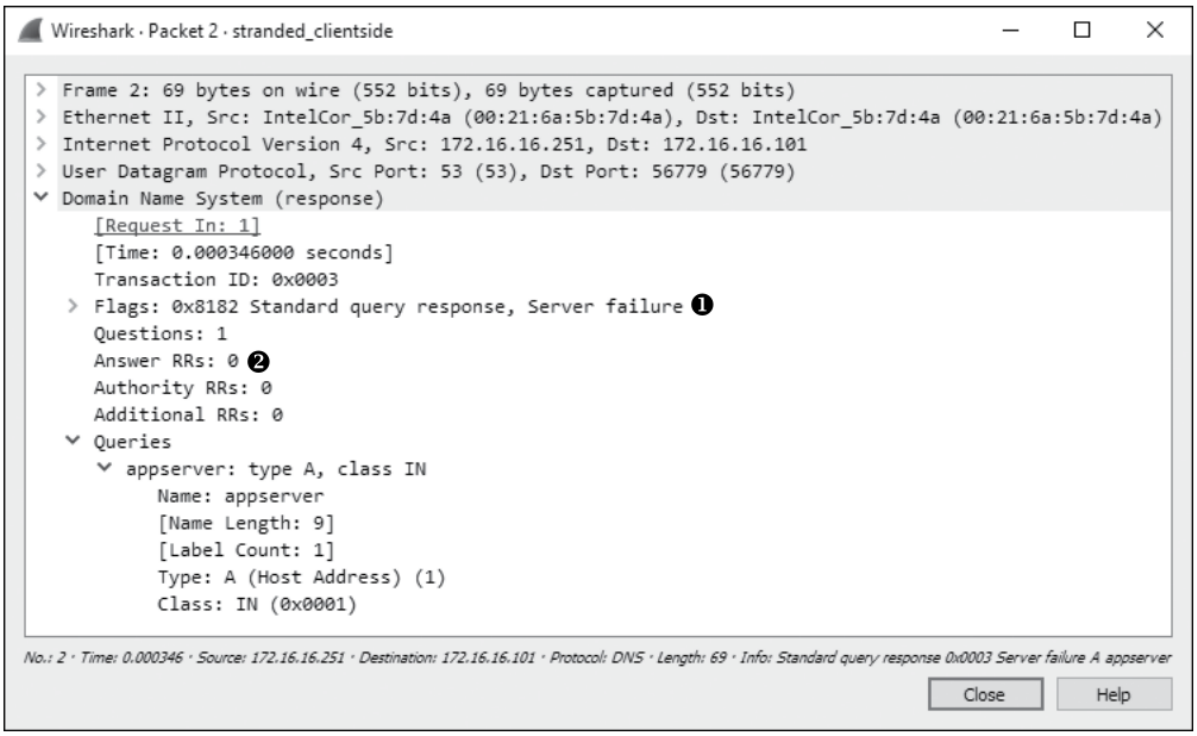


Рисунок 10-31. Ответ на запрос указывает на проблему выше по потоку

Теперь известно, что проблема обмена связана с DNS. Поскольку запросы DNS в филиале разрешаются местным сервером DNS `172.16.16.251`, именно туда мы направимся дальше.

Для захвата нужного трафика сервера DNS филиала оставим анализатор на месте и просто изменим назначение зеркалирования порта, чтобы теперь на него зеркалировался трафик сервера DNS, а не рабочей станции. Полученный файл называется `stranded_branchdns.pcapng`.

Как показано на рис. 10-32, захват начинается с уже виденных запроса и ответа, а также одного дополнительного пакета. Этот дополнительный пакет выглядит несколько странно: он пытается связаться с основным сервером DNS в центральном офисе (`172.16.16.250`) (метка 1) через стандартный порт сервера DNS 53 (метка 3), но это не привычный UDP (метка 2).

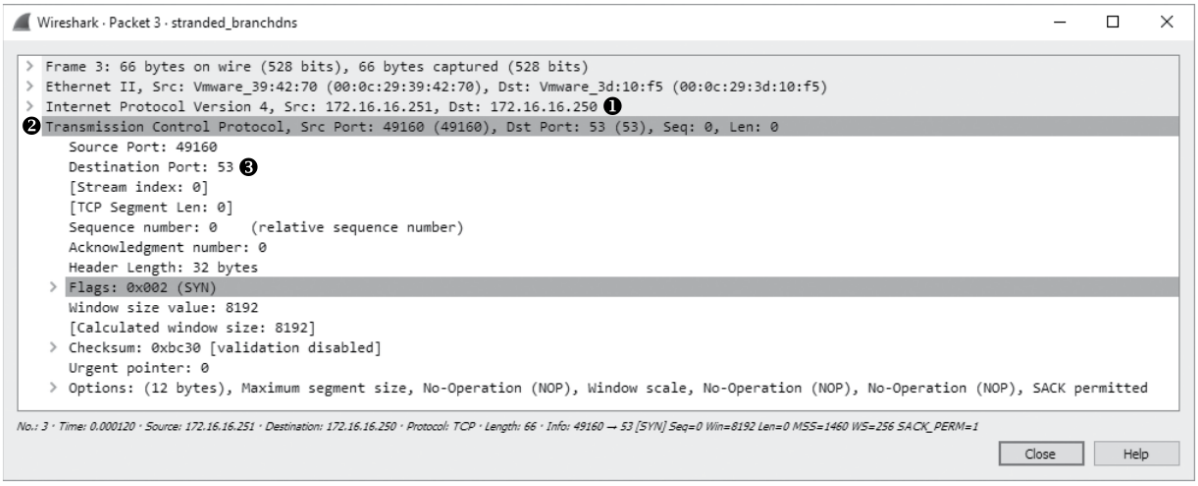


Рисунок 10-32. Пакет SYN использует порт 53, но не UDP

Чтобы понять назначение пакета, вспомним обсуждение DNS в главе 9. Обычно DNS использует UDP, но переходит на TCP, когда ответ на запрос превышает определённый размер. В таком случае сначала виден трафик UDP, порождающий трафик TCP. TCP также применяется DNS при

передаче зоны, когда ресурсные записи переносятся между серверами DNS, и здесь, вероятно, происходит именно это.

Сервер DNS филиала является подчинённым серверу DNS центрального офиса, то есть полагается на него при получении ресурсных записей. Сервер приложений, к которому пытаются обратиться пользователи филиала, находится в центральном офисе, поэтому центральный сервер DNS является авторитетным для него. Чтобы сервер филиала мог разрешить DNS-запрос сервера приложений, ресурсная запись DNS этого сервера должна быть передана с сервера DNS центрального офиса на сервер DNS филиала. Вероятно, именно это породило пакет SYN в захвате.

Отсутствие ответа на пакет SYN сообщает, что проблема DNS вызвана неудачной передачей зоны между серверами DNS филиала и центрального офиса. Теперь можно пойти дальше и определить причину сбоя. Возможные виновники сужаются до маршрутизаторов между офисами или самого сервера DNS центрального офиса. Чтобы выяснить, кто виноват, можно прослушать трафик центрального сервера DNS и проверить, достигает ли его пакет SYN.

Я не включил файл захвата трафика сервера DNS центрального офиса, потому что такого трафика не было. Пакет SYN никогда не достиг сервера. Технические специалисты проверили настройки маршрутизаторов, соединяющих два офиса, и обнаружили, что входящий трафик порта 53 на маршрутизаторе центрального офиса настроен разрешать только UDP и блокировать входящий трафик TCP. Эта простая ошибка настройки не позволяла передавать зоны между серверами и тем самым мешала клиентам филиала разрешать запросы устройств центрального офиса.

Усвоенные уроки

Многому в исследовании проблем сетевого обмена можно научиться из криминальных сериалов. Когда происходит преступление, детективы начинают с опроса тех, кого оно затронуло сильнее всего. Они следуют за полученными зацепками, и процесс продолжается, пока виновник не найден.

В этом сценарии мы начали с исследования цели (рабочей станции) и получили зацепки, обнаружив проблему обмена DNS. Они привели нас к серверу DNS филиала, затем к центральному серверу DNS и наконец к маршрутизатору, который и был источником проблемы.

При анализе попробуйте воспринимать пакеты как улики. Они не всегда сразу называют преступника, но часто в конце концов приводят к виновнику.

Повреждение данных программного обеспечения

Одни из самых частых споров в ИТ происходят между разработчиками и сетевыми администраторами. Разработчики всегда винят в ошибках программ плохую сетевую архитектуру и неисправное оборудование. В свою очередь, сетевые администраторы склонны обвинять плохой код в сетевых ошибках и медленном обмене.

В этом сценарии программист разработал приложение для отслеживания продаж в нескольких магазинах и отправки отчётов в центральную базу данных. Чтобы экономить полосу пропускания в обычные рабочие часы, приложение не обновляется в реальном времени. Данные накапливаются в течение дня, а ночью передаются в виде файла значений, разделённых запятыми (comma-separated value, CSV), для вставки в центральную базу.

Новое приложение работает неправильно. Сервер получает отправленные магазинами файлы, но вставляемые в базу данные неверны. Некоторые разделы отсутствуют, данные находятся не на

своих местах, а отдельные части потеряны. К неудовольствию сетевого администратора, программист обвиняет сеть. Он убеждён, что файлы повреждаются во время передачи из магазинов в центральное хранилище. Наша цель - определить, прав ли он.

Подключение к линии

Для сбора нужных данных можно захватить пакеты в одном из магазинов или в центральном офисе. Поскольку проблема затрагивает все магазины, если она связана с сетью, то должна проявляться в центральном офисе - единственном общем для всех магазинов элементе (кроме самого программного обеспечения).

Сетевые коммутаторы поддерживают зеркалирование портов, поэтому зеркалируем порт, к которому подключён сервер, и прослушаем его трафик. Захват будет ограничен одним случаем отправки магазином файла CSV серверу сбора. Полученный файл называется `tickedoffdeveloper.pcapng`.

Анализ

Мы ничего не знаем о разработанном программистом приложении, кроме общей схемы движения сведений по сети. Файл захвата, по-видимому, начинается с трафика FTP, поэтому исследуем, действительно ли именно он служит механизмом переноса файла.

Сначала посмотрим список пакетов (рис. 10-33). Видно, что `172.16.16.128` (метка 1) инициирует обмен с `172.16.16.121` (метка 2) посредством рукопожатия TCP. Поскольку обмен инициирует `172.16.16.128`, можно предположить, что это клиент, а `172.16.16.121` - сервер, который собирает и обрабатывает данные. После завершения рукопожатия мы начинаем видеть запросы FTP от клиента и ответы сервера (метка 3).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128 ❶	172.16.16.121 ❷	TCP	66	2555 → 21 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.000071	172.16.16.121	172.16.16.128	TCP	66	21 → 2555 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
3	0.000242	172.16.16.128	172.16.16.121	TCP	60	2555 → 21 [ACK] Seq=1 Ack=1 Win=17520 Len=0
4	0.002749	172.16.16.121	172.16.16.128	FTP	96	Response: 220 FileZilla Server version 0.9.34 beta
5	0.002948	172.16.16.128	172.16.16.121	FTP	70 ❸	Request: USER salesxfer
6	0.003396	172.16.16.121	172.16.16.128	FTP	91	Response: 331 Password required for salesxfer
7	0.003514	172.16.16.128	172.16.16.121	FTP	69	Request: PASS p@ssw0rd
8	0.004862	172.16.16.121	172.16.16.128	FTP	69	Response: 230 Logged on

Рисунок 10-33. Исходный обмен помогает определить клиента и сервер

Мы знаем, что здесь должна происходить передача данных, поэтому воспользуемся знаниями FTP, чтобы найти пакет, с которого она начинается. FTP-соединение и передачу данных инициирует клиент, поэтому от `172.16.16.128` должна поступить команда FTP `STOR`, используемая для загрузки данных на сервер FTP. Проще всего найти её, создав фильтр.

Поскольку файл захвата переполнен командами запросов FTP, вместо поиска среди сотен протоколов и параметров в построителе выражений можно создать нужный фильтр непосредственно на панели списка пакетов. Сначала выберите пакет с командой запроса FTP. Возьмём пакет 5, поскольку он находится вверху списка. Затем разверните раздел FTP на панели подробностей пакета и раздел USER. Щёлкните правой кнопкой мыши поле Request Command: USER, выберите Prepare a Filter, затем Selected.

Это подготовит фильтр всех пакетов, содержащих команду запроса FTP `USER`, и поместит его в поле фильтра. Затем, как показано на рис. 10-34, отредактируйте фильтр, заменив слово `USER` словом `STOR` (метка 1).

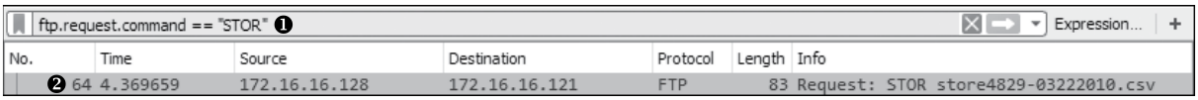


Рисунок 10-34. Фильтр помогает найти начало передачи данных

Фильтр можно было бы сузить, указав IP-адрес клиента как источник соединения и добавив && `ip.src == 172.16.16.128`, но поскольку захват уже ограничен одним клиентом, здесь это не требуется.

Примените фильтр, нажав Enter, и вы увидите, что в захвате есть только один экземпляр команды `STOR` - в пакете 64 (метка 2).

Теперь, когда известно начало передачи данных, щёлкните пакет, чтобы выбрать его, и очистите фильтр кнопкой X над панелью списка пакетов. На экране снова должны быть видны все пакеты, а пакет 64 должен оставаться выбранным.

Исследуя захват с пакета 64, мы видим, что он задаёт передачу файла `store4829-03222010.csv` (метка 1), как показано на рис. 10-35.

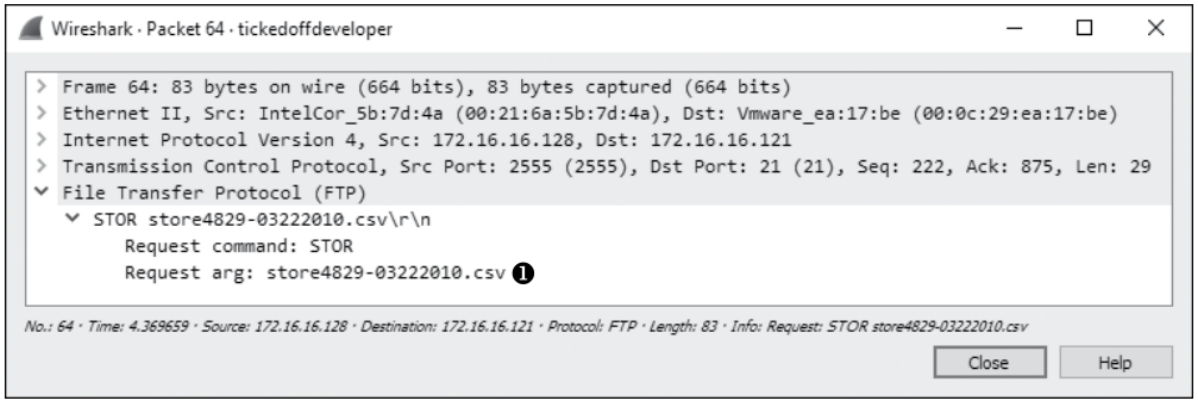


Рисунок 10-35. Файл CSV передаётся посредством FTP

Пакеты после команды `STOR` используют другой порт, но определяются как часть передачи FTP-DATA. Мы подтвердили передачу данных, но ещё не установили, прав программист или нет. Для этого нужно показать, осталось ли содержимое файла целым после прохождения сети, поэтому извлечём переданный файл из захваченных пакетов.

Когда файл передаётся по сети в незашифрованном виде, он разбивается на сегменты и повторно собирается в месте назначения. В этом сценарии пакеты были захвачены после прибытия к назначению, но до повторной сборки. Все данные присутствуют; нужно лишь собрать их, извлекая файл как поток данных. Для сборки выберите любой пакет потока FTP-DATA (например, пакет 66) и нажмите Follow TCP Stream. Результат показан на рис. 10-36. Он выглядит как обычный текстовый файл формата CSV с данными заказов на продажу.

Данные видны, поскольку передаются открытым текстом по FTP, но по одному потоку нельзя быть уверенным в целостности файла. Чтобы собрать данные и извлечь их в исходном формате, нажмите Save As и укажите имя файла из пакета 64. Затем нажмите Save.

Результатом сохранения должен стать файл CSV - точная побайтовая копия файла, первоначально переданного системой магазина. Его можно проверить, сравнив хеш MD5 исходного и извлечённого файлов. Хеши MD5 должны совпадать, как показано на рис. 10-37.

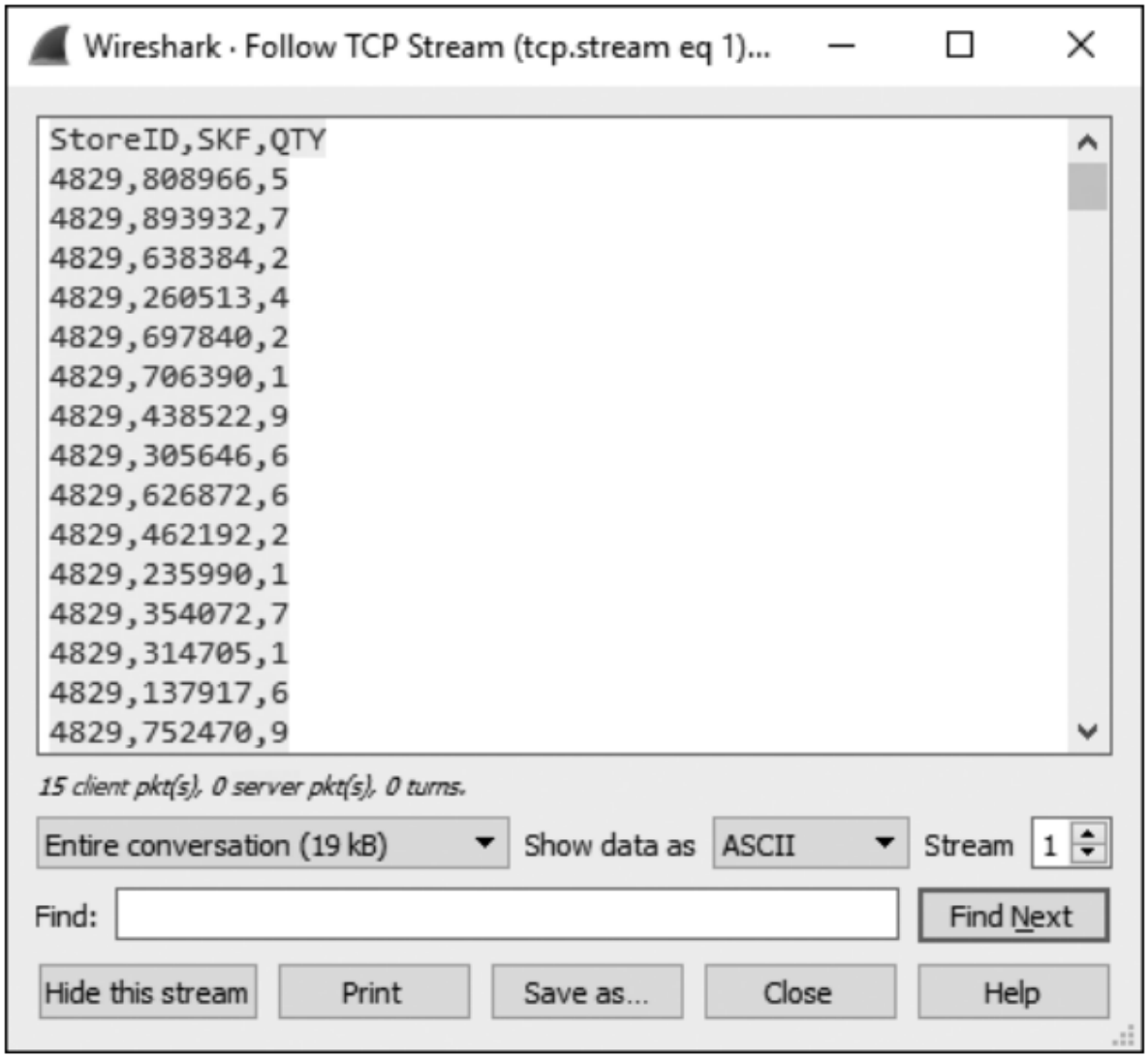


Рисунок 10-36. Поток TCP показывает данные, которые, по всей видимости, передаются

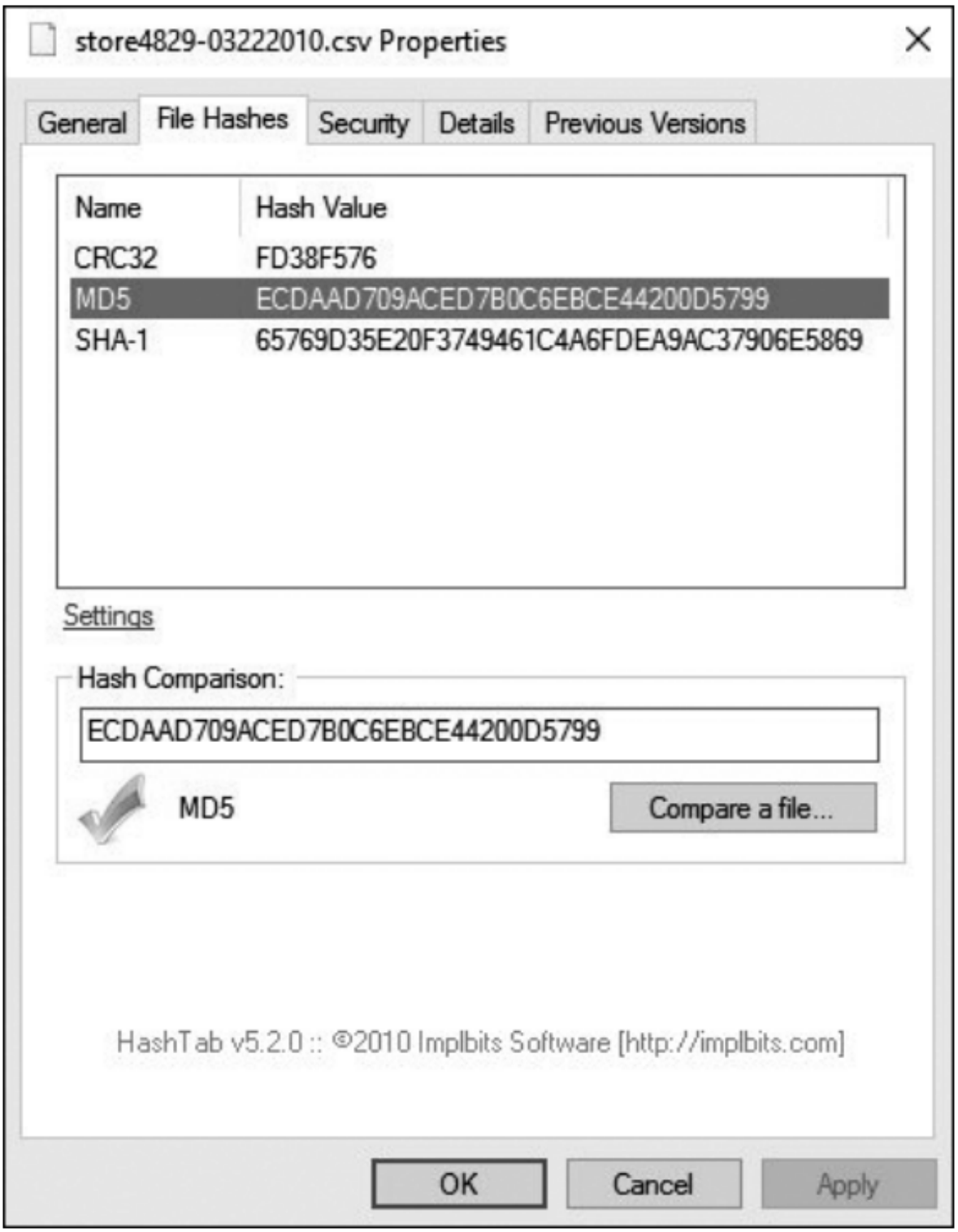


Рисунок 10-37. Хеши MD5 исходного и извлечённого файлов совпадают

После сравнения файлов можно утверждать, что сеть не виновата в повреждении базы данных внутри приложения. Файл, переданный из магазина серверу сбора, прибывает целым, поэтому любое повреждение должно происходить при его обработке приложением на стороне сервера.

Усвоенные уроки

Одно из прекрасных свойств анализа на уровне пакетов состоит в том, что не приходится иметь дело с нагромождением приложений. Плохо написанных приложений намного больше, чем хороших, но на уровне пакетов это не имеет значения. Здесь программист беспокоился обо всех загадочных компонентах, от которых зависело приложение, однако в конечном счёте сложная передача данных, потребовавшая сотен строк кода, всё равно является не более чем FTP, TCP и IP. Используя знания этих основных протоколов, мы смогли убедиться в правильном ходе обмена и даже извлечь файлы, доказав исправность сети. Крайне важно помнить: какой бы сложной ни была проблема, всё сводится к пакетам.

Заключительные мысли

В этой главе мы рассмотрели несколько сценариев, в которых анализ пакетов позволил лучше понять проблемный обмен. Посредством базового анализа распространённых протоколов мы смогли своевременно найти и устранить сетевые проблемы. Возможно, в вашей сети не встретятся в точности те же сценарии, но представленные методы анализа должны пригодиться при исследовании ваших собственных уникальных проблем.

Глава 11. Борьба с медленной сетью

Работая сетевым администратором, значительную часть времени вы будете исправлять компьютеры и службы, которые работают медленнее, чем должны. Но если кто-то говорит, что сеть работает медленно, это ещё не означает, что виновата сеть.

Прежде чем бороться с медленной сетью, сначала нужно определить, действительно ли она работает медленно. В этой главе вы узнаете, как это сделать.

Начнём с функций восстановления после ошибок и управления потоком TCP. Затем рассмотрим, как обнаружить источник замедления сети. Наконец, изучим способы создания базовых показателей сетей и работающих в них устройств и служб. После этой главы вы будете гораздо лучше подготовлены к выявлению, диагностике и устранению неполадок медленных сетей.

> **Примечание.** Для устранения неполадок медленной сети можно применять несколько методов. Я решил сосредоточиться главным образом на TCP, потому что чаще всего это всё, с чем вам придётся работать. TCP позволяет выполнять пассивный ретроспективный анализ, а не создавать дополнительный трафик (в отличие от ICMP).

Функции восстановления TCP после ошибок

Функции восстановления TCP после ошибок - наши лучшие инструменты для обнаружения, диагностики и в конечном счёте устранения высокой задержки в сети. В компьютерных сетях задержка - мера времени между передачей и получением пакета.

Задержку можно измерять в одном направлении (от одного источника к назначению) или для полного цикла (от источника к назначению и обратно). Если обмен между устройствами быстр и пакет проходит из одной точки в другую за малое время, говорят, что у обмена низкая задержка. И наоборот, если пакетам требуется значительное время для прохождения между источником и назначением, задержка обмена считается высокой. Высокая задержка - главный враг всех сетевых администраторов, дорожащих своим рассудком (и работой).

В главе 8 мы обсуждали, как TCP применяет порядковые номера и номера подтверждения для надёжной доставки пакетов. В этой главе снова рассмотрим их и увидим, как TCP реагирует, когда из-за высокой задержки эти номера поступают не по порядку (или не поступают вовсе).

Повторные передачи TCP

Способность узла повторно передавать пакеты - одна из основополагающих функций восстановления TCP после ошибок. Она предназначена для борьбы с потерей пакетов.

У потери пакетов много возможных причин, включая неправильно работающие приложения, маршрутизаторы под высокой нагрузкой трафика или временные перебои службы. На уровне пакетов всё происходит быстро, а потеря часто бывает временной, поэтому TCP критически важно уметь обнаруживать её и восстанавливаться.

Основной механизм определения необходимости повторной передачи пакета - таймер повторной передачи. Он поддерживает значение, называемое тайм-аутом повторной передачи (retransmission timeout, RTO). При каждой передаче пакета по TCP таймер запускается и останавливается при получении ACK этого пакета. Время между передачей пакета и получением ACK называется

временем полного цикла (round-trip time, RTT). Несколько таких значений усредняются, и среднее используется для определения окончательного RTO.

До определения RTO передающая операционная система полагается на стандартную настроенную величину RTT, применяемую при первоначальном обмене между узлами. Затем она корректируется по RTT полученных пакетов, чтобы определить RTO.

После определения RTO таймер повторной передачи используется для каждого переданного пакета, чтобы установить, произошла ли потеря. Процесс повторной передачи TCP показан на рис. 11-1.

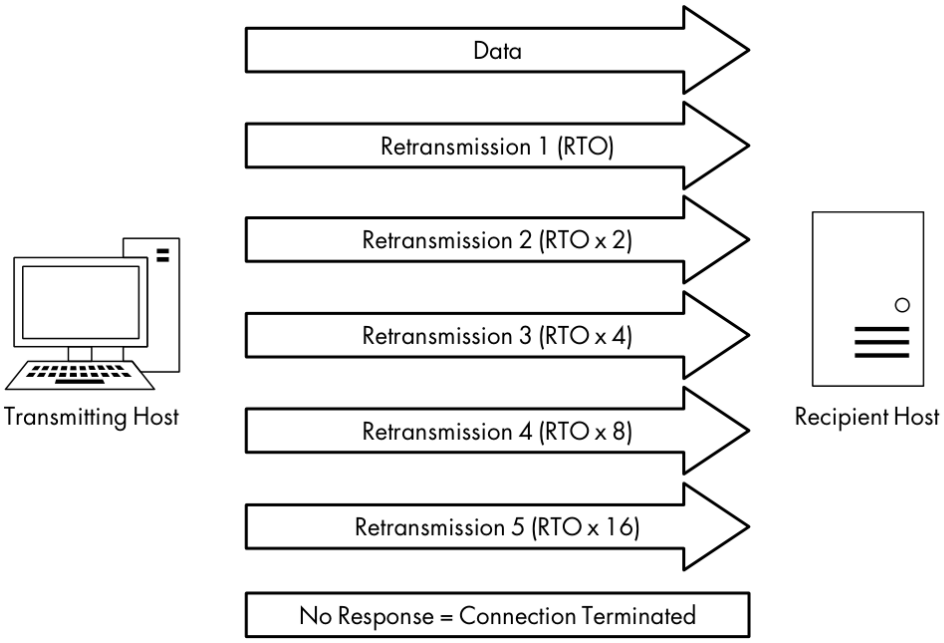


Рисунок 11-1. Концептуальное представление процесса повторной передачи TCP

Если пакет отправлен, но получатель не вернул TCP ACK, передающий узел считает исходный пакет потерянным и повторяет его. При повторной передаче RTO удваивается; если ACK не поступит до достижения этого значения, произойдёт ещё одна повторная передача. Если и она не получит ACK, RTO снова удваивается. Процесс продолжается с удвоением RTO для каждой повторной передачи, пока не поступит ACK либо отправитель не достигнет настроенного максимального числа попыток. Подробнее процесс описан в RFC 6298.

Максимальное число попыток повторной передачи зависит от значения, настроенного в передающей операционной системе. По умолчанию узлы Windows выполняют не более пяти попыток, а большинство узлов Linux - не более 15. В обеих операционных системах этот параметр можно настроить.

Чтобы увидеть пример повторной передачи TCP, откройте файл `tcp_retransmissions.pcapng`, содержащий шесть пакетов. Первый пакет показан на рис. 11-2.

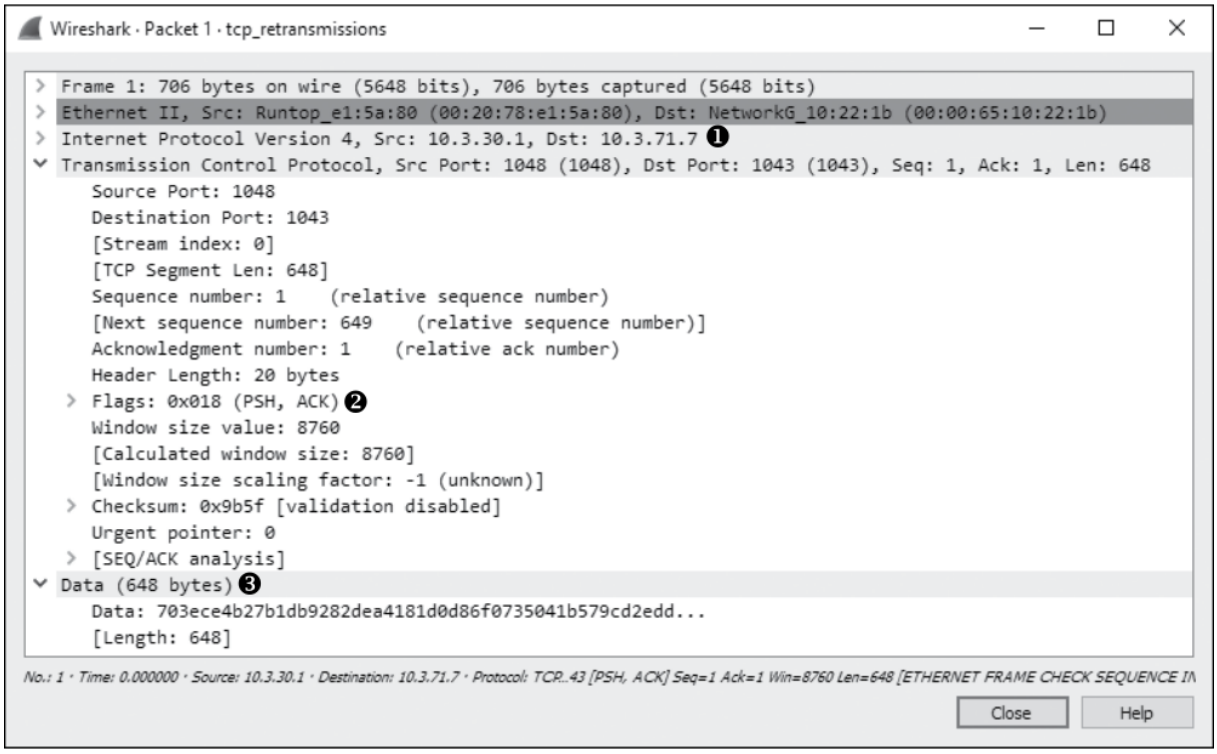


Рисунок 11-2. Простой пакет TCP, содержащий данные

Это пакет TCP PSH/ACK (метка 2) с 648 байтами данных (метка 3), отправленный с `10.3.30.1` на `10.3.71.7` (метка 1). Перед нами обычный пакет данных.

В нормальных условиях вскоре после отправки первого пакета ожидается ответ TCP ACK. Однако здесь следующий пакет является повторной передачей. Это можно определить на панели списка пакетов: в столбце Info будет явно написано `[TCP Retransmission]`, а пакет отобразится красным текстом на чёрном фоне. Примеры повторных передач в списке пакетов показаны на рис. 11-3.

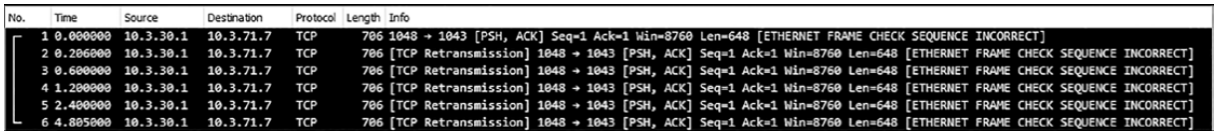


Рисунок 11-3. Повторные передачи на панели списка пакетов

Определить повторную передачу можно также, исследовав пакет на панели подробностей, как показано на рис. 11-4.

Обратите внимание: на панели подробностей пакета под заголовком анализа SEQ/ACK у повторно передаваемого пакета присутствуют дополнительные сведения (метка 1). Эти полезные данные предоставляет Wireshark; в самом пакете их нет. Анализ SEQ/ACK сообщает, что это действительно повторная передача (метка 2), RTO равен 0,206 секунды (метка 3) и основан на разнице времени с пакетом 1 (метка 4).

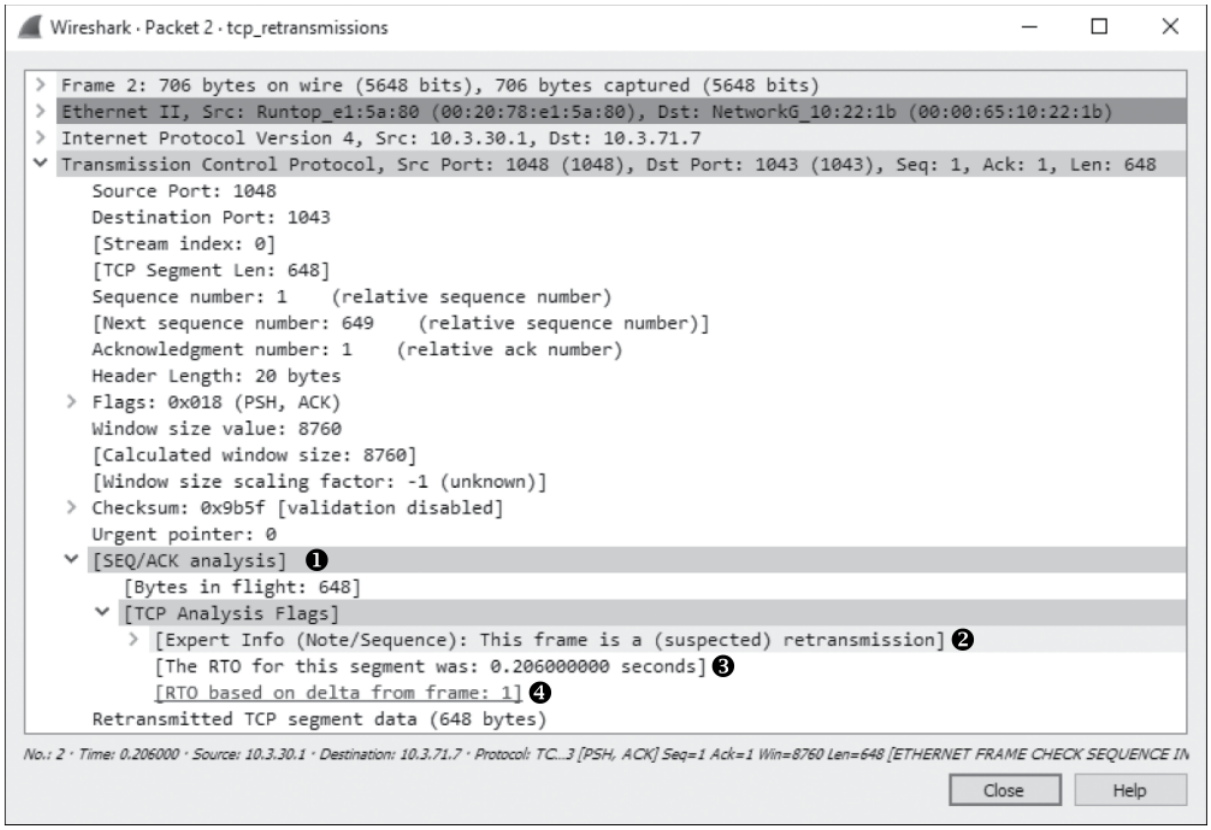


Рисунок 11-4. Отдельный пакет повторной передачи

Обратите внимание: этот пакет совпадает с исходным (кроме полей идентификатора IP и Checksum). Чтобы проверить это, сравните панель байтов повторно переданного пакета с исходным.

Исследование остальных пакетов должно дать похожие результаты: различаться будут только поля идентификатора IP и Checksum, а также значение RTO. Чтобы наглядно увидеть промежуток времени между пакетами, посмотрите столбец Time панели списка пакетов, как показано на рис. 11-5. Здесь наблюдается экспоненциальный рост времени при удвоении RTO после каждой повторной передачи.

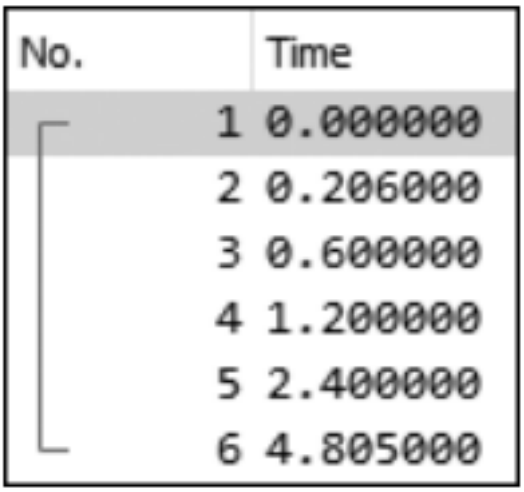


Рисунок 11-5. Столбец Time показывает рост значения RTO

Функция повторной передачи TCP используется передающим устройством для обнаружения потери пакетов и восстановления после неё. Далее рассмотрим дублирующие подтверждения TCP

- функцию, которую получатель данных использует для обнаружения потери пакетов и восстановления.

Дублирующие подтверждения и быстрая повторная передача TCP

Дублирующий ACK - пакет TCP, отправляемый получателем при поступлении пакетов не по порядку. TCP использует поля порядкового номера и номера подтверждения заголовка, чтобы надёжно обеспечить получение и повторную сборку данных в том же порядке, в котором они были отправлены.

> **Примечание.** Правильный термин для пакета TCP в действительности - сегмент TCP, но большинство людей называет его пакетом.

При установлении нового TCP-соединения одним из важнейших элементов обмена во время рукопожатия является начальный порядковый номер (initial sequence number, ISN). После установки ISN каждой стороны соединения порядковый номер каждого следующего переданного пакета увеличивается на размер его полезной нагрузки данных.

Представьте узел с ISN 5000, отправляющий получателю 500-байтовый пакет. Получив пакет, узел-получатель отвечает TCP ACK с номером подтверждения 5500 согласно следующей формуле:

```
Входящий порядковый номер + число полученных байтов данных = исходящий номер подтверждения
```

В результате расчёта возвращаемый передающему узлу номер подтверждения является следующим порядковым номером, который получатель ожидает получить. Пример показан на рис. 11-6.

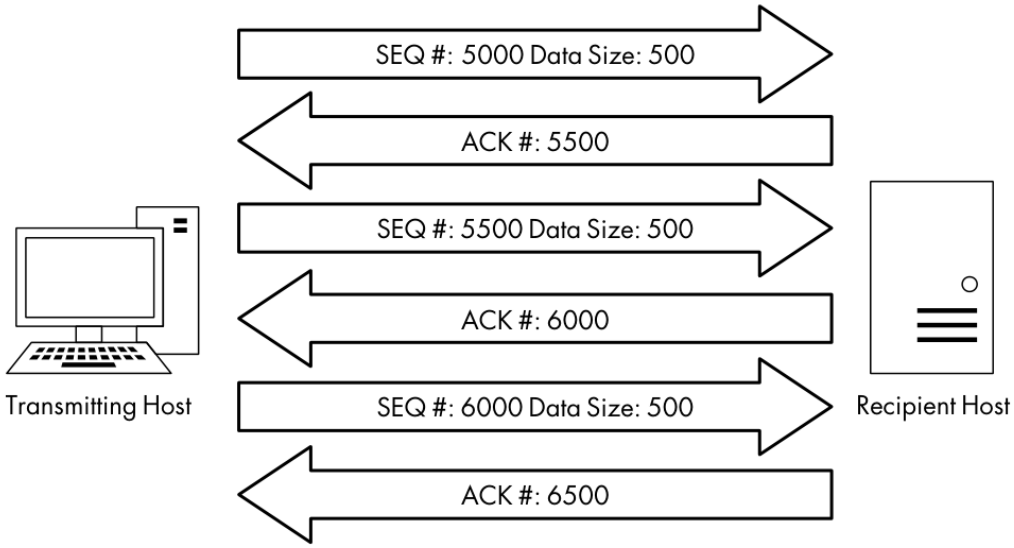


Рисунок 11-6. Порядковые номера и номера подтверждения TCP

Получатель данных может обнаружить потерю пакета благодаря порядковым номерам. Отслеживая поступающие номера, он определяет, когда они приходят не по порядку.

Получив неожиданный порядковый номер, получатель предполагает, что пакет был потерян при передаче. Для правильной сборки данных ему нужен недостающий пакет, поэтому он повторно отправляет ACK с ожидаемым порядковым номером потерянного пакета, чтобы побудить передающий узел повторить этот пакет.

Когда передающий узел получает от получателя три дублирующих ACK, он считает, что пакет действительно потерян при передаче, и немедленно выполняет быструю повторную передачу.

После её запуска все остальные передаваемые пакеты помещаются в очередь до отправки быстро повторяемого пакета. Процесс показан на рис. 11-7.

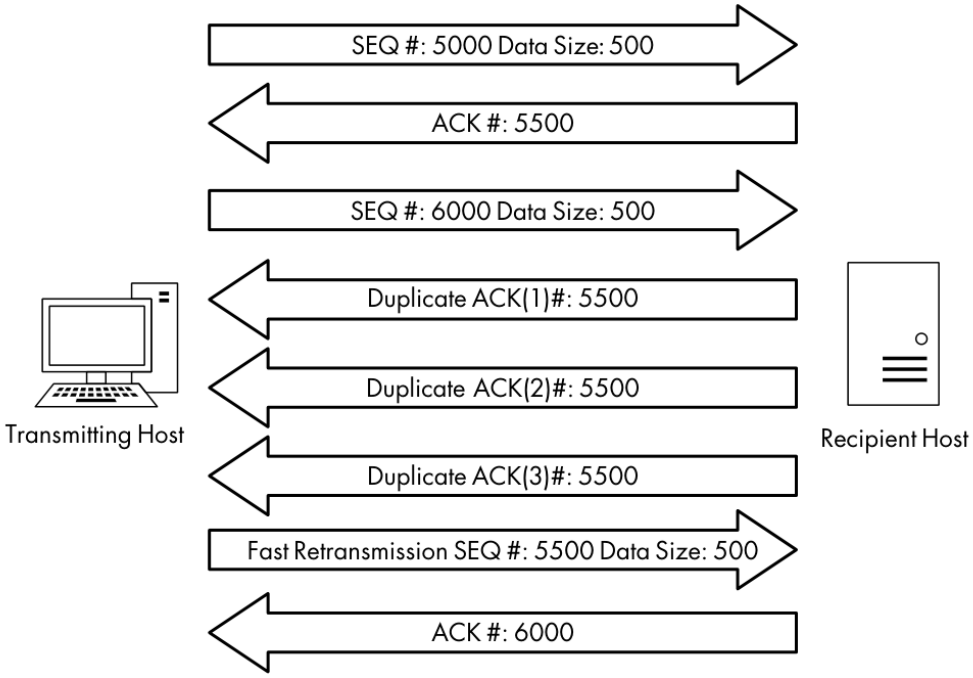


Рисунок 11-7. Дублирующие ACK от получателя приводят к быстрой повторной передаче

Пример дублирующих ACK и быстрой повторной передачи находится в файле `tcp_dupack.pcapng`. Первый пакет захвата показан на рис. 11-8.

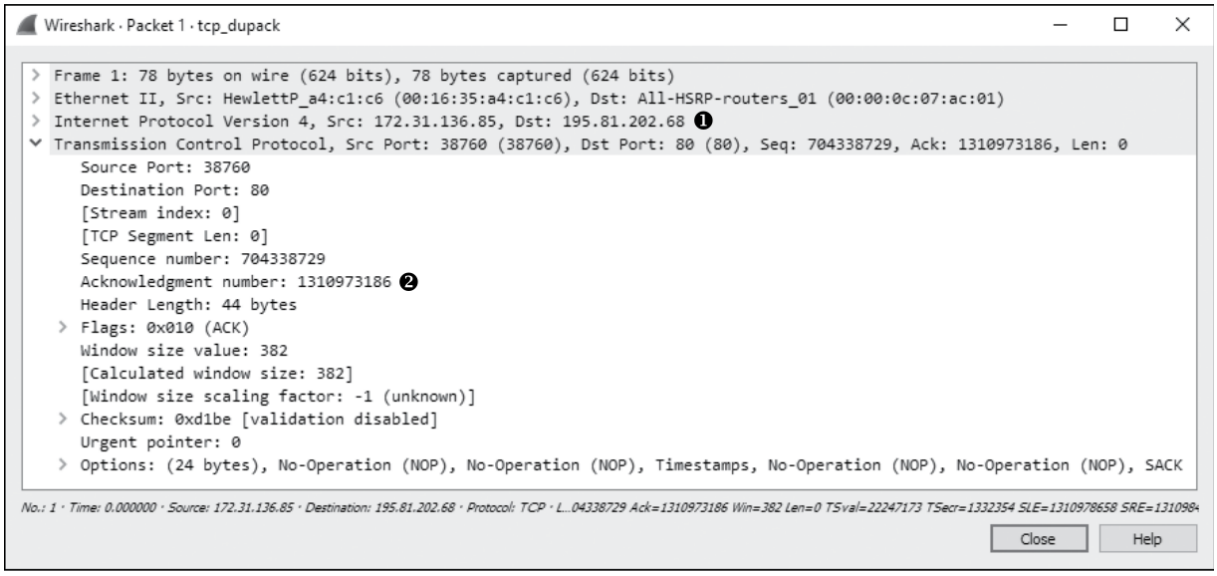


Рисунок 11-8. ACK с указанием следующего ожидаемого порядкового номера

Этот пакет, TCP ACK от получателя данных (`172.31.136.85`) передатчику (`195.81.202.68`) (метка 1), содержит подтверждение данных, отправленных в предыдущем пакете, который не включён в файл захвата.

> **Примечание.** По умолчанию Wireshark использует относительные порядковые номера, упрощающие их анализ, но примеры и снимки экрана в следующих разделах эту функцию не применяют. Чтобы отключить её, выберите Edit > Preferences. В окне Preferences выберите Protocols, затем раздел TCP и снимите флажок Relative sequence numbers.

Номер подтверждения этого пакета равен 1310973186 (метка 2). Он должен совпадать с порядковым номером следующего полученного пакета, как показано на рис. 11-9.

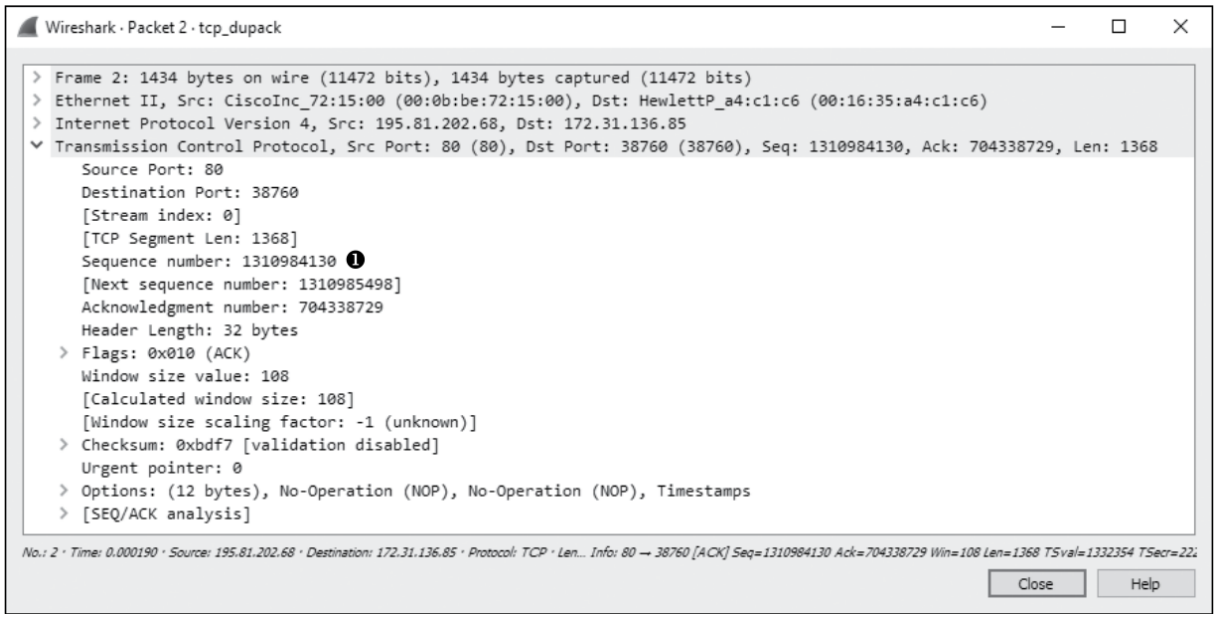


Рисунок 11-9. Порядковый номер этого пакета не соответствует ожидаемому

К сожалению для нас и получателя, порядковый номер следующего пакета - 1310984130 (метка 1), а это не то, чего мы ожидали. Пакет, пришедший не по порядку, указывает, что ожидаемый пакет каким-то образом потерян при передаче. Узел-получатель замечает нарушение порядка и отправляет дублирующий ACK в третьем пакете захвата, как показано на рис. 11-10.

Определить, что это пакет дублирующего ACK, можно по любому из следующих признаков:

• В столбце Info панели подробностей пакета он должен отображаться красным текстом на чёрном фоне.
• На панели подробностей пакета под заголовком анализа SEQ/ACK (рис. 11-10). Развернув его, вы увидите, что пакет указан как дублирующий ACK пакета 1 (метка 1).

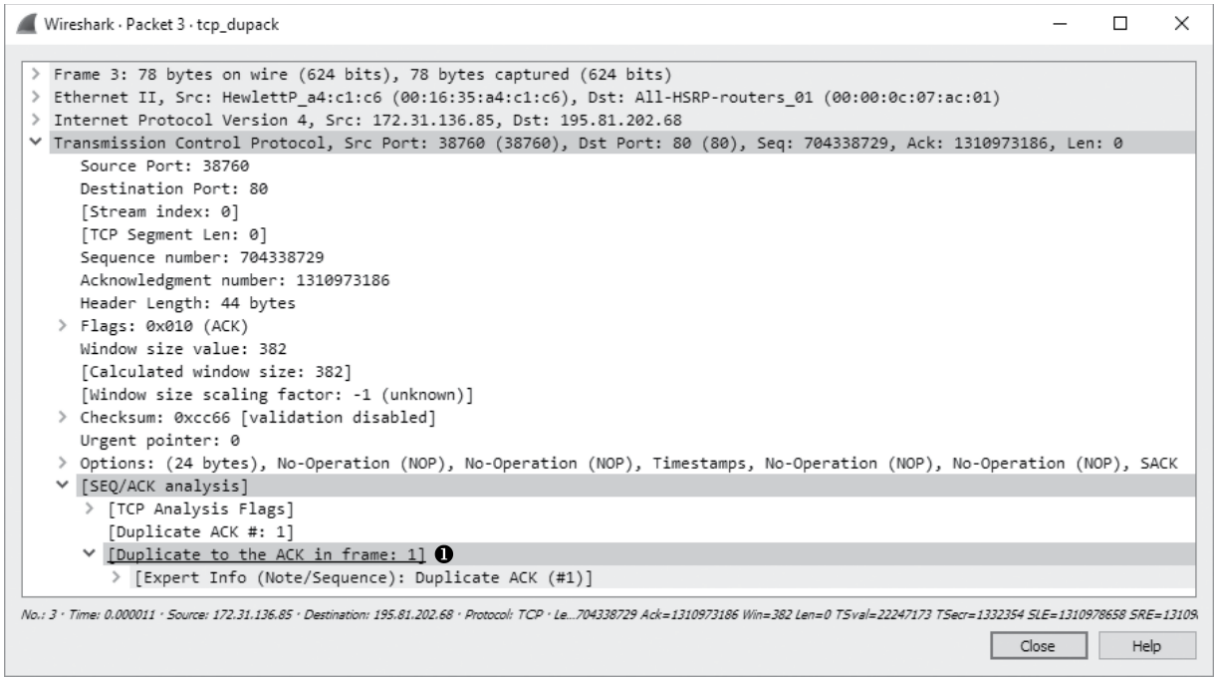
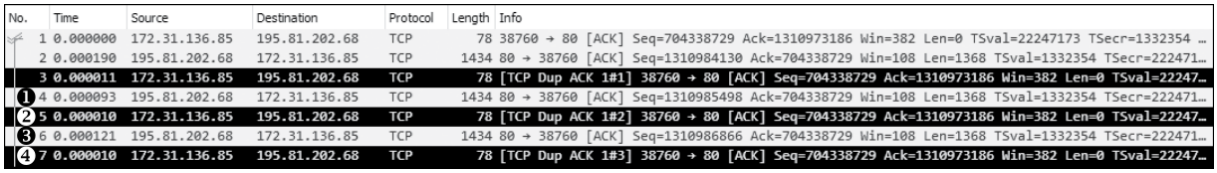


Рисунок 11-10. Первый пакет дублирующего ACK

В следующих нескольких пакетах этот процесс продолжается, как показано на рис. 11-11.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.31.136.85	195.81.202.68	TCP	78	38760 → 80 [ACK] Seq=704338729 Ack=1310973186 Win=382 Len=0 TSval=22247173 TSecr=1332354 …
2	0.000190	195.81.202.68	172.31.136.85	TCP	1434	80 → 38760 [ACK] Seq=1310984130 Ack=704338729 Win=108 Len=1368 TSval=1332354 TSecr=222471…
3	0.000011	172.31.136.85	195.81.202.68	TCP	78	[TCP Dup ACK 1#1] 38760 → 80 [ACK] Seq=704338729 Ack=1310973186 Win=382 Len=0 TSval=22247…
❶ 4	0.000093	195.81.202.68	172.31.136.85	TCP	1434	80 → 38760 [ACK] Seq=1310985498 Ack=704338729 Win=108 Len=1368 TSval=1332354 TSecr=222471…
❷ 5	0.000010	172.31.136.85	195.81.202.68	TCP	78	[TCP Dup ACK 1#2] 38760 → 80 [ACK] Seq=704338729 Ack=1310973186 Win=382 Len=0 TSval=22247…
❸ 6	0.000121	195.81.202.68	172.31.136.85	TCP	1434	80 → 38760 [ACK] Seq=1310986866 Ack=704338729 Win=108 Len=1368 TSval=1332354 TSecr=222471…
❹ 7	0.000010	172.31.136.85	195.81.202.68	TCP	78	[TCP Dup ACK 1#3] 38760 → 80 [ACK] Seq=704338729 Ack=1310973186 Win=382 Len=0 TSval=22247…

Рисунок 11-11. Из-за поступающих не по порядку пакетов создаются дополнительные дублирующие ACK

Четвёртый пакет в файле захвата - ещё один фрагмент данных, отправленный передающим узлом с неправильным порядковым номером (метка 1). В результате узел-получатель отправляет свой второй дублирующий ACK (метка 2). Затем получатель принимает ещё один пакет с неправильным порядковым номером (метка 3). Это приводит к отправке третьего и последнего дублирующего ACK (метка 4).

Как только передающий узел получает от получателя третий дублирующий ACK, он вынужден остановить всю передачу пакетов и повторно отправить потерянный пакет. На рис. 11-12 показана быстрая повторная передача потерянного пакета.

Пакет повторной передачи снова можно найти по столбцу Info на панели списка пакетов. Как и в предыдущих примерах, он явно выделен красным текстом на чёрном фоне. Раздел анализа SEQ/ACK этого пакета (рис. 11-12) сообщает, что предполагается быстрая повторная передача (метка 1). Напомним, сведения, помечающие пакет как быструю повторную передачу, не являются значением, установленным в самом пакете, а предоставляются Wireshark. Последний пакет захвата - ACK, подтверждающий получение быстро повторно переданного пакета.

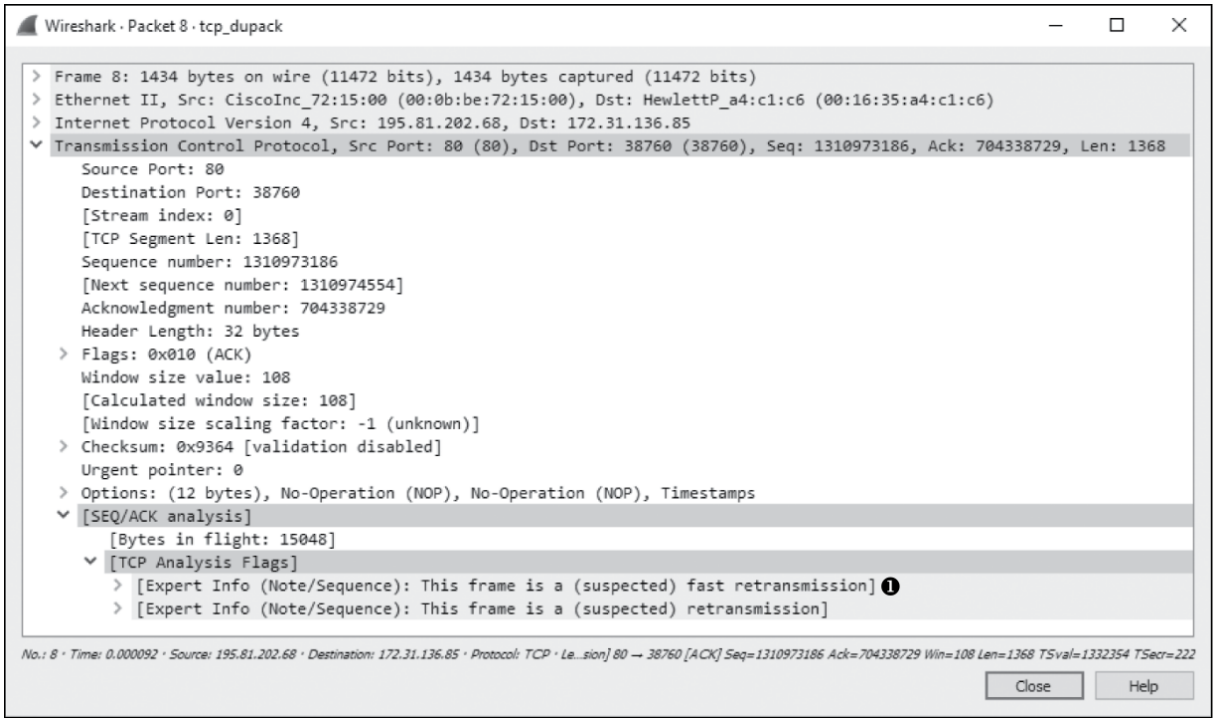


Рисунок 11-12. Три дублирующих ACK вызывают быструю повторную передачу потерянного пакета

Примечание. Одной из функций, способных повлиять на поток данных в TCP-соединениях при потере пакетов, является выборочное подтверждение (Selective Acknowledgment). В только что рассмотренном захвате пакетов поддержка Selective ACK была согласована и включена во время начального трёхэтапного рукопожатия. Поэтому при потере пакета и получении дублирующего ACK повторно передаётся только потерянный пакет, даже если после него успешно получены другие пакеты. Если бы Selective ACK не был включён, пришлось бы повторно передать также все пакеты, следующие за потерянным. Selective ACK делает восстановление после потери данных гораздо эффективнее. Эту функцию поддерживает большинство современных реализаций стека TCP/IP, поэтому обычно она включена.

Управление потоком TCP

Повторные передачи и дублирующие ACK - реактивные функции TCP, предназначенные для восстановления после потери пакетов. Однако TCP был бы весьма плохим протоколом, если бы в нём не существовало упреждающего способа предотвращать потерю пакетов.

Для обнаружения возможной потери пакетов и регулирования скорости передачи данных TCP реализует механизм скользящего окна. Этот механизм использует окно приёма получателя данных для управления потоком.

Окно приёма - указанное получателем данных и хранящееся в заголовке TCP значение в байтах, которое сообщает передающему устройству, сколько данных получатель готов хранить в пространстве своего буфера TCP. В этом буфере данные временно хранятся до тех пор, пока их не удастся передать вверх по стеку ожидающему их обработки протоколу прикладного уровня. Следовательно, передающий узел может за один раз отправить лишь объём данных, указанный в поле Window size. Чтобы передатчик мог отправить дополнительные данные, получатель должен подтвердить получение предыдущих данных. Кроме того, он должен освободить пространство буфера TCP, обработав занимающие его данные. Принцип работы окна приёма показан на рис. 11-13.

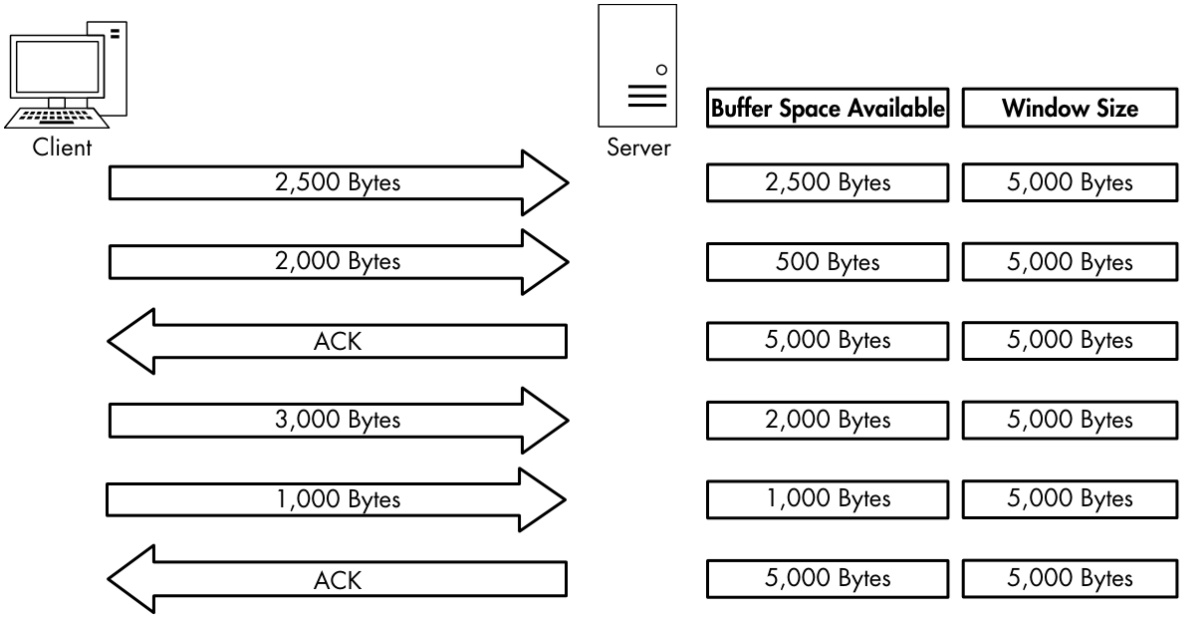


Рисунок 11-13. Окно приёма не позволяет перегрузить получателя данных

На рис. 11-13 клиент отправляет данные серверу, который сообщил размер окна приёма 5000 байт. Клиент отправляет 2500 байт, уменьшая свободное пространство буфера сервера до 2500 байт, а затем ещё 2000 байт, после чего в буфере остаётся 500 байт. Сервер отправляет подтверждение этих данных, а после их обработки снова получает в своё распоряжение пустой буфер. Затем процесс повторяется: клиент отправляет 3000 байт и ещё 1000 байт, уменьшая свободное пространство буфера сервера до 1000 байт. Клиент ещё раз подтверждает эти данные и обрабатывает содержимое своего буфера.

Регулирование размера окна

Этот процесс регулирования размера окна довольно прост, однако он не всегда работает идеально. При получении данных стек TCP создаёт и отправляет в ответ подтверждение, но помещённые в буфер получателя данные не всегда обрабатываются немедленно.

Занятый сервер, обрабатывающий пакеты от нескольких клиентов, вполне может медленно освобождать свой буфер и поэтому оказаться неспособным разместить новые данные. Без управления потоком переполненный буфер мог бы привести к потере пакетов и повреждению данных. К счастью, если сервер становится слишком загруженным и не может обрабатывать данные со скоростью, объявленной его окном приёма, он способен изменить размер окна. Для этого сервер уменьшает значение размера окна в заголовке TCP пакета ACK, который отправляет обратно передающим ему данные узлам. Пример показан на рис. 11-14.

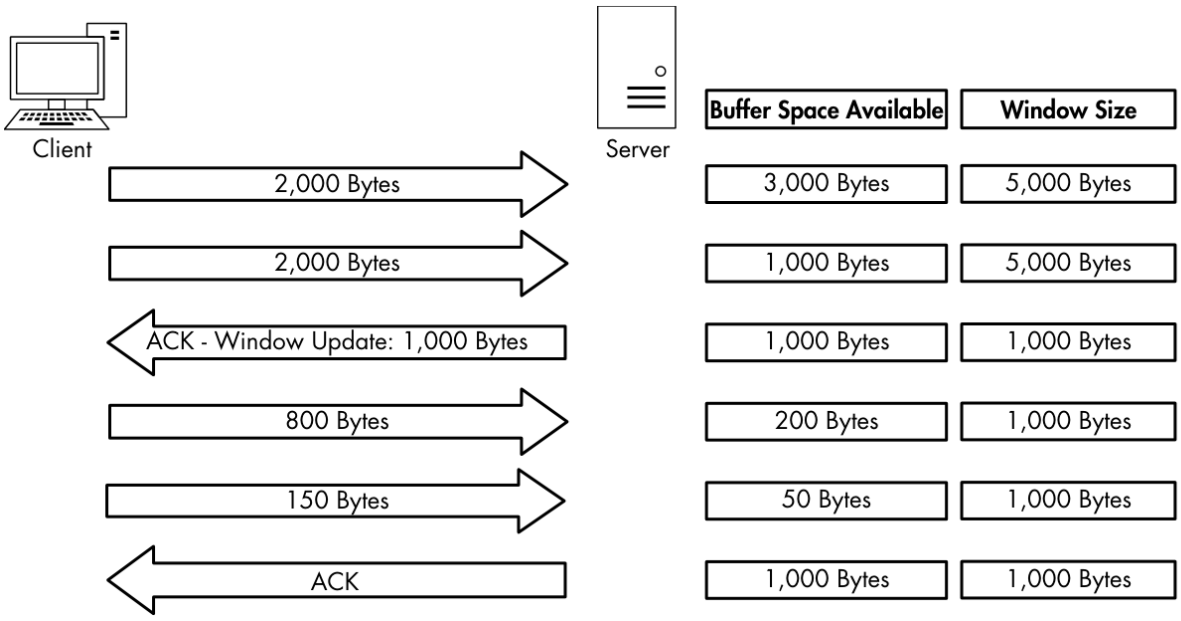


Рисунок 11-14. Когда сервер становится загружен, размер окна можно изменить

На рис. 11-14 сервер начинает с объявленного размера окна 5000 байт. Клиент отправляет 2000 байт, а затем ещё 2000 байт, оставляя свободными лишь 1000 байт буфера. Сервер замечает, что его буфер быстро заполняется, и понимает: если передача данных продолжится с такой скоростью, вскоре начнётся потеря пакетов. Чтобы этого избежать, сервер отправляет клиенту подтверждение с обновлённым размером окна 1000 байт. Клиент отвечает отправкой меньших объёмов данных, и теперь скорость обработки содержимого буфера сервером позволяет поддерживать постоянный поток данных.

Изменение размера работает в обе стороны. Когда сервер получает возможность обрабатывать данные быстрее, он может отправить пакет ACK с увеличенным размером окна.

Остановка потока данных уведомлением о нулевом окне

Из-за недостатка памяти, нехватки вычислительных ресурсов или иной проблемы сервер может перестать обрабатывать данные, поступающие от клиента. Такая остановка могла бы привести к отбрасыванию пакетов и прекращению обмена, однако окно приёма позволяет свести отрицательные последствия к минимуму.

В этой ситуации сервер может отправить пакет с нулевым размером окна. Получив его, клиент прекратит любую передачу данных, но иногда будет поддерживать соединение с сервером открытым, отправляя пакеты проверки активности (keep-alive). Клиент может отправлять их через равные интервалы, чтобы проверять состояние сервера. Как только сервер снова сможет обрабатывать данные, он ответит ненулевым размером окна, и обмен возобновится. Пример уведомления о нулевом окне показан на рис. 11-15.

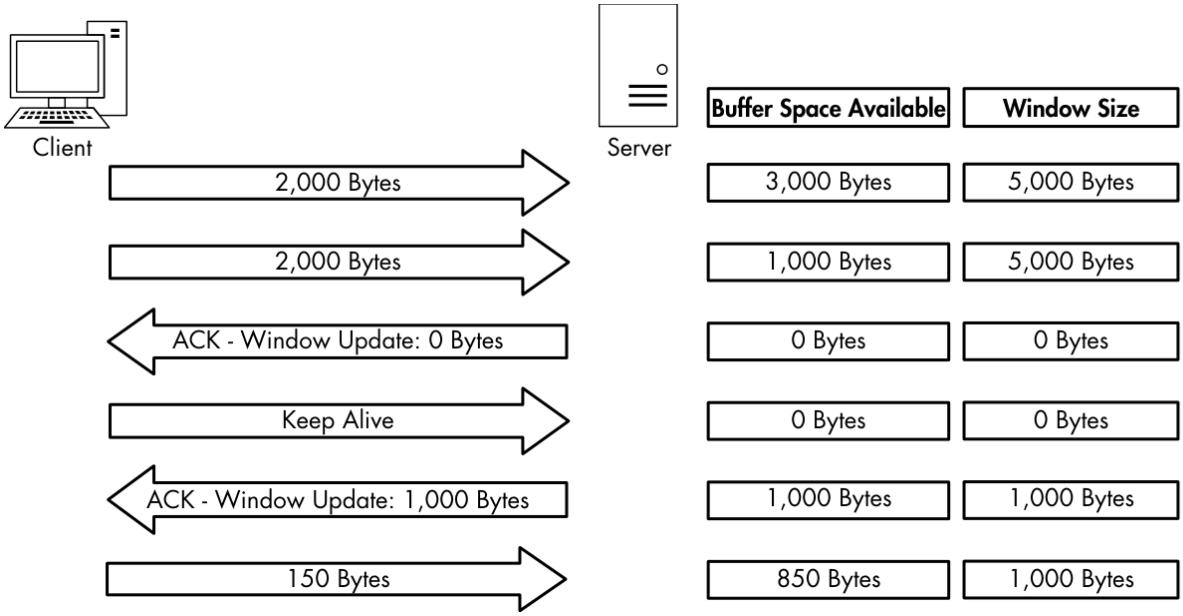


Рисунок 11-15. Когда размер окна устанавливается равным 0 байт, передача данных прекращается

На рис. 11-15 сервер начинает принимать данные с размером окна 5000 байт. Получив от клиента в общей сложности 4000 байт данных, сервер сталкивается с очень высокой нагрузкой на процессор и больше не может обрабатывать поступающие от клиента данные. Тогда сервер отправляет пакет, в котором поле Window size value установлено в 0. Клиент прекращает передачу данных и отправляет пакет keep-alive. Получив его, сервер отвечает пакетом, уведомляющим клиента, что теперь он снова может принимать данные, а размер его окна равен 1000 байт. Клиент возобновляет отправку данных, но с меньшей скоростью, чем прежде.

Скользящее окно TCP на практике

Разобрав теорию скользящего окна TCP, теперь исследуем его в файле захвата `tcp_zerowindowrecovery.pcapng`.

В начале файла находятся несколько пакетов TCP ACK, идущих от `192.168.0.20` к `192.168.0.30`. Главное интересующее нас значение - поле Window size value, которое видно и в столбце Info панели списка пакетов, и в заголовке TCP на панели подробностей. Сразу заметно, что на протяжении первых трёх пакетов значение этого поля уменьшается, как показано на рис. 11-16.

No.	Time ❶	Source	Destination	Protocol	Length	Info ❷
1	0.000000	192.168.0.20	192.168.0.30	TCP	60	2235 → 1720 [ACK] Seq=1422793785 Ack=2710996659 Win=8760 Len=0
2	0.000237	192.168.0.20	192.168.0.30	TCP	60	2235 → 1720 [ACK] Seq=1422793785 Ack=2710999579 Win=5840 Len=0
3	0.000193	192.168.0.20	192.168.0.30	TCP	60	2235 → 1720 [ACK] Seq=1422793785 Ack=2711002499 Win=2920 Len=0

Рисунок 11-16. Размер окна этих пакетов уменьшается

Размер окна снижается с 8760 байт в первом пакете до 5840 байт во втором, а затем до 2920 байт в третьем (метка 2). Такое уменьшение размера окна - классический признак увеличения задержки на узле. Обратите внимание по столбцу Time, что это происходит очень быстро (метка 1). При столь быстром уменьшении окно нередко сокращается до нуля - именно это и происходит в четвёртом пакете, показанном на рис. 11-17.

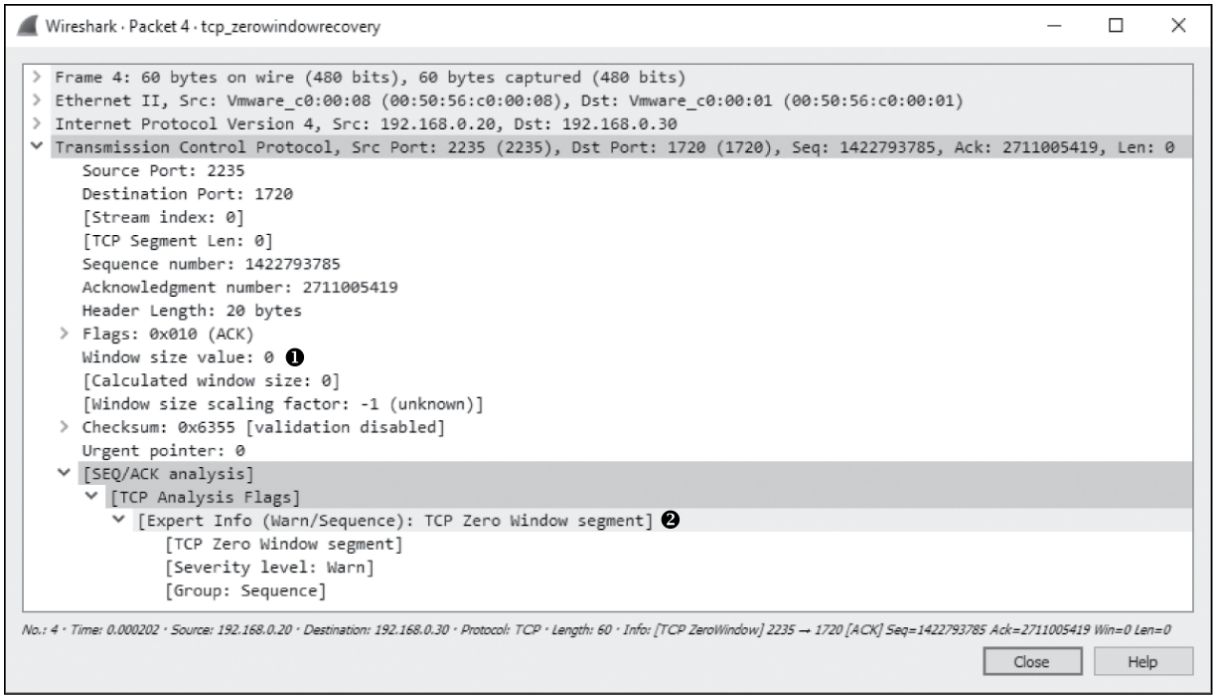


Рисунок 11-17. Этот пакет с нулевым окном сообщает, что узел больше не может принимать данные

Четвёртый пакет также отправлен от `192.168.0.20` к `192.168.0.30`, но его назначение - сообщить `192.168.0.30`, что отправитель больше не может принимать данные. Нулевое значение видно в заголовке TCP (метка 1). Wireshark также указывает, что это пакет с нулевым окном, в столбце Info панели списка пакетов и в разделе анализа SEQ/ACK заголовка TCP (метка 2).

После отправки пакета с нулевым окном устройство `192.168.0.30` не станет передавать дополнительные данные, пока не получит от `192.168.0.20` обновление окна, уведомляющее об увеличении его размера. К счастью, причина нулевого окна в этом файле захвата была лишь временной, поэтому уже в следующем пакете отправляется обновление окна, показанное на рис. 11-18.

В данном случае размер окна увеличен до вполне достаточных 64 240 байт (метка 1). Wireshark снова сообщает, что это обновление окна, под заголовком анализа SEQ/ACK.

Получив пакет обновления, узел `192.168.0.30` может снова начать отправлять данные, что он и делает в пакетах 6 и 7. Весь период остановки передачи происходит очень быстро. Если бы он продлился хотя бы немного дольше, в сети мог бы возникнуть сбой, который привёл бы к замедлению или неудаче передачи данных.

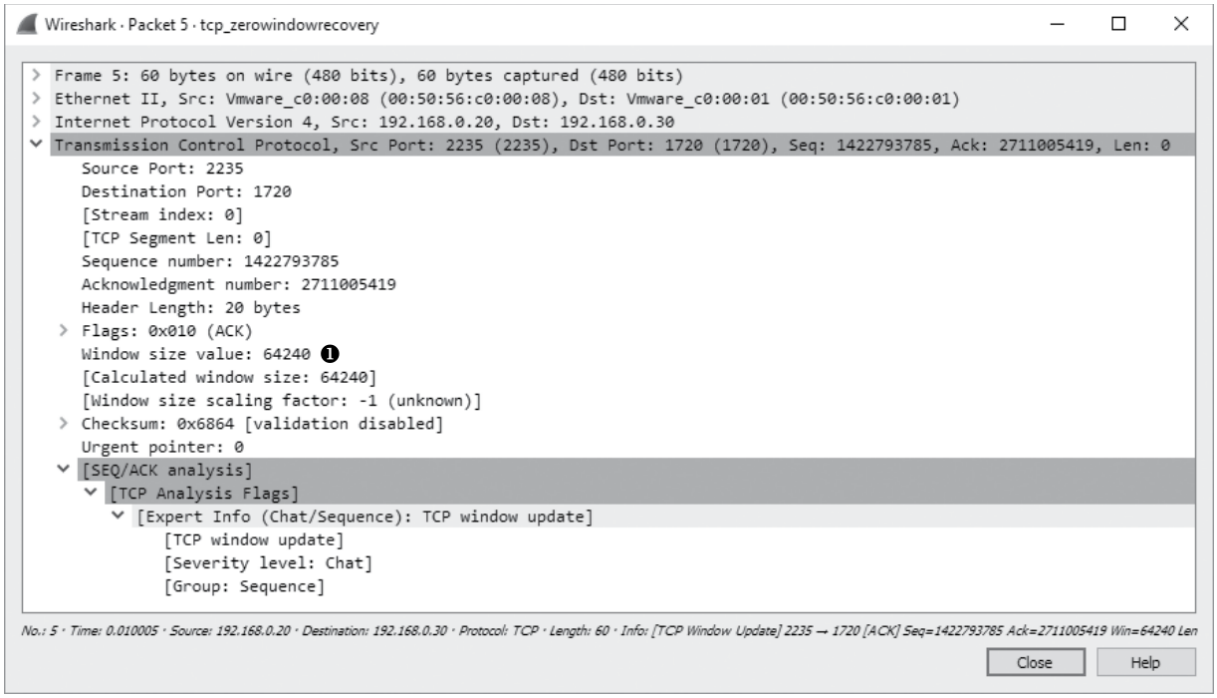


Рисунок 11-18. Пакет обновления окна TCP сообщает другому узлу, что тот может возобновить передачу

Для последнего примера скользящего окна исследуйте файл `tcp_zerowindowdead.pcapng`. Первый пакет захвата представляет собой обычный трафик HTTP, отправленный от `195.81.202.68` к `172.31.136.85`. Сразу за ним в обратном направлении от `172.31.136.85` отправляется пакет с нулевым окном, показанный на рис. 11-19.

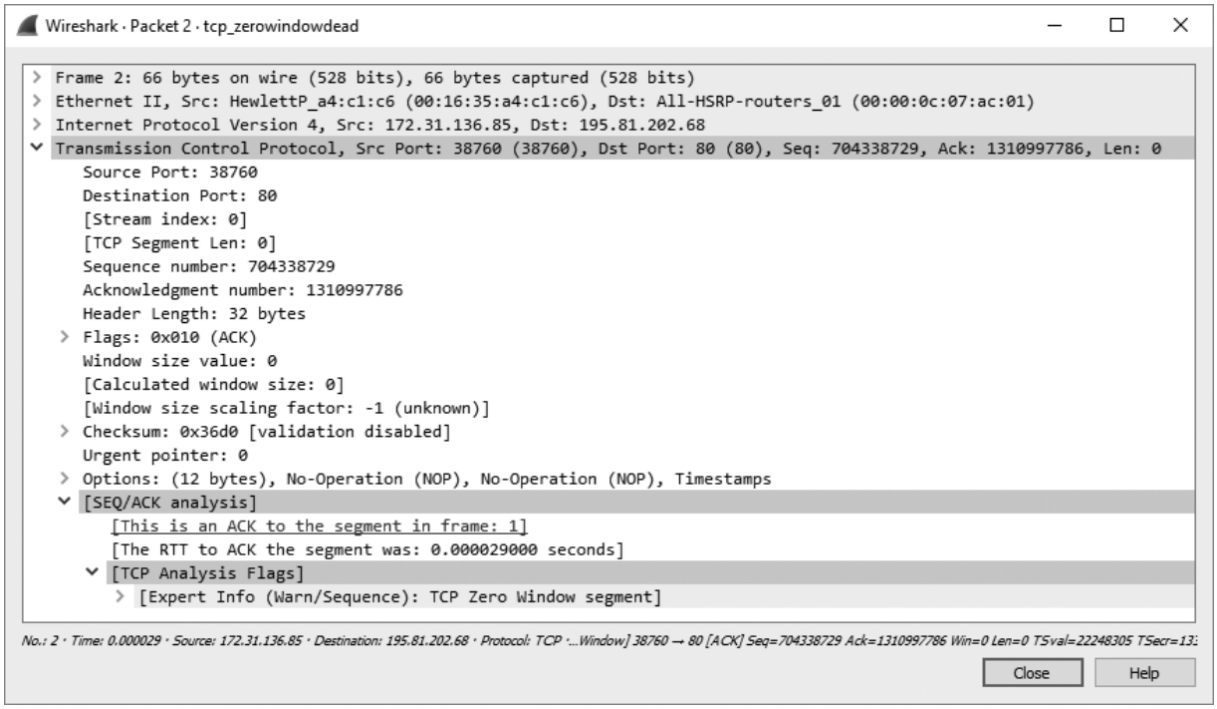


Рисунок 11-19. Пакет с нулевым окном останавливает передачу данных

Этот пакет очень похож на пакет с нулевым окном на рис. 11-17, но результат оказывается совсем иным. Вместо обновления окна от узла `172.31.136.85` и возобновления обмена мы видим пакет keep-alive, показанный на рис. 11-20.

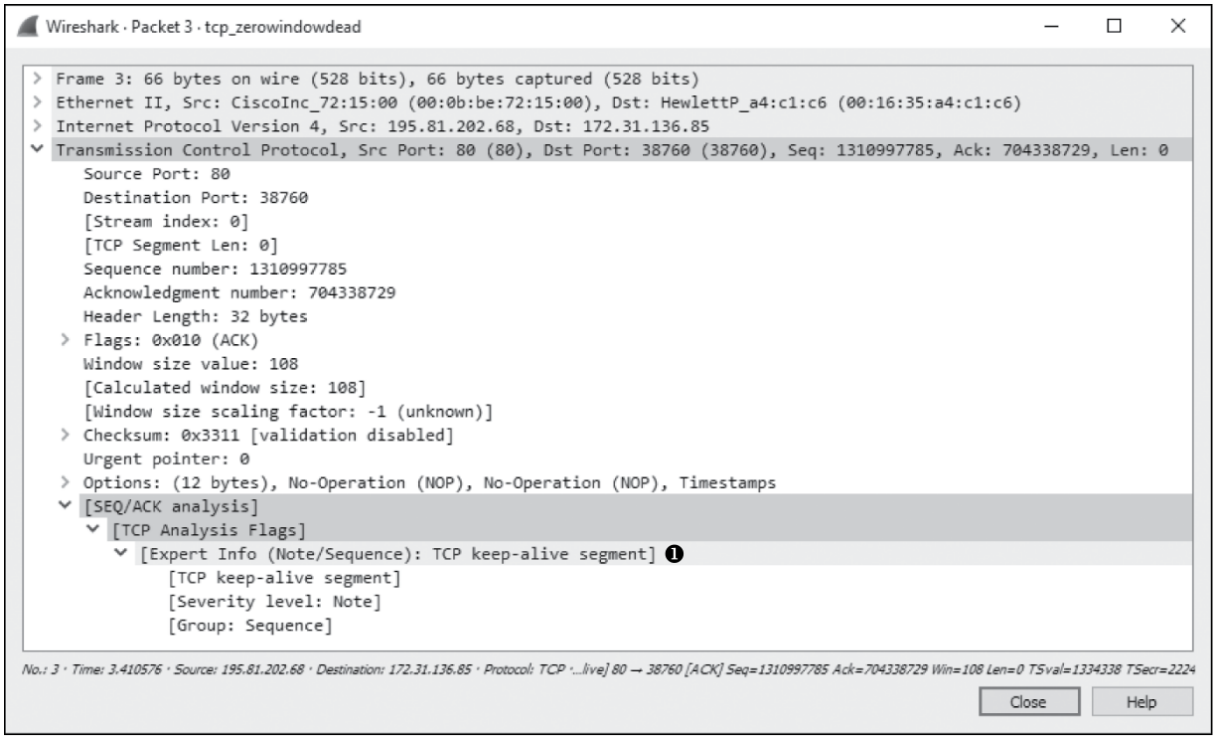


Рисунок 11-20. Этот пакет keep-alive подтверждает, что узел с нулевым окном всё ещё активен

Wireshark помечает пакет как keep-alive в разделе анализа SEQ/ACK заголовка TCP на панели подробностей пакета (метка 1). Столбец Time сообщает, что пакет отправлен через 3,4 секунды после последнего принятого пакета. Этот процесс повторяется ещё несколько раз: один узел отправляет пакет с нулевым окном, а другой - пакет keep-alive, как показано на рис. 11-21.

No.	Time ❶	Source	Destination	Protocol	Length	Info
2	0.000029	172.31.136.85	195.81.202.68	TCP	66	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338729 Ack=1310997786 Win=0 Len=0 TSv…
3	3.410576	195.81.202.68	172.31.136.85	TCP	66	[TCP Keep-Alive] 80 → 38760 [ACK] Seq=1310997785 Ack=704338729 Win=108 Len=0 T…
4	0.000031	172.31.136.85	195.81.202.68	TCP	66	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338729 Ack=1310997786 Win=0 Len=0 TSv…
5	6.784127	195.81.202.68	172.31.136.85	TCP	66	[TCP Keep-Alive] 80 → 38760 [ACK] Seq=1310997785 Ack=704338729 Win=108 Len=0 T…
6	0.000029	172.31.136.85	195.81.202.68	TCP	66	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338729 Ack=1310997786 Win=0 Len=0 TSv…
7	13.536714	195.81.202.68	172.31.136.85	TCP	66	[TCP Keep-Alive] 80 → 38760 [ACK] Seq=1310997785 Ack=704338729 Win=108 Len=0 T…
8	0.000047	172.31.136.85	195.81.202.68	TCP	66	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338729 Ack=1310997786 Win=0 Len=0 TSv…

Рисунок 11-21. Узел и клиент продолжают отправлять соответственно пакеты с нулевым окном и keep-alive

Пакеты keep-alive возникают с интервалами 3,4, 6,8 и 13,5 секунды (метка 1). В зависимости от операционных систем обменивающихся устройств этот процесс может продолжаться довольно долго. Если сложить значения в столбце Time, видно, что соединение остаётся остановленным почти 25 секунд. Представьте попытку пройти аутентификацию на контроллере домена или загрузить файл из Интернета при 25-секундной задержке - это неприемлемо!

Что можно узнать из пакетов управления ошибками и потоком TCP

Поместим повторные передачи, дублирующие ACK и механизм скользящего окна в общий контекст. При устранении проблем с задержкой помните о следующих особенностях.

Пакеты повторной передачи

Повторные передачи происходят потому, что клиент обнаружил: сервер не получает отправляемые ему данные. Поэтому в зависимости от того, какую сторону обмена вы анализируете, повторных передач можно вообще не увидеть. Если вы захватываете данные со стороны сервера, а он действительно не получает пакеты, отправляемые и повторно передаваемые клиентом, ситуация может остаться неясной, поскольку пакеты повторной передачи в захват не попадут. Подозревая потерю пакетов на стороне сервера, попробуйте, если это возможно, захватить трафик со стороны клиента и проверить наличие повторных передач.

Пакеты дублирующего ACK

Я склонен считать дублирующий ACK своего рода противоположностью повторной передачи, поскольку он отправляется, когда сервер обнаруживает, что пакет от обменивающегося с ним клиента потерян при передаче. В большинстве случаев при захвате трафика с обеих сторон обмена дублирующие ACK будут видны. Помните, что их вызывает получение пакетов не по порядку. Например, если из трёх отправленных пакетов сервер получил только первый и третий, он отправит дублирующий ACK, чтобы добиться от клиента быстрой повторной передачи второго пакета. Поскольку первый и третий пакеты получены, условие, приведшее к отбрасыванию второго, скорее всего, было временным, и дублирующий ACK, вероятно, будет успешно отправлен и принят. Разумеется, так бывает не всегда, поэтому, если вы подозреваете потерю пакетов на стороне сервера, но не видите дублирующих ACK, рассмотрите возможность захвата пакетов со стороны клиента.

Пакеты zero window и keep-alive

Скользящее окно напрямую связано с неспособностью сервера принимать и обрабатывать данные. Любое уменьшение размера окна или состояние нулевого окна является прямым результатом проблемы на сервере, поэтому при обнаружении любого из этих явлений в линии сосредоточьте расследование на сервере. Пакеты обновления окна обычно видны с обеих сторон сетевого обмена.

Определение источника большой задержки

В некоторых случаях причиной задержки может быть не потеря пакетов. Обмен между двумя узлами может быть медленным, но не проявлять обычных признаков повторной передачи TCP или дублирующих ACK. Тогда для определения источника большой задержки потребуется другой метод.

Один из самых эффективных способов найти источник большой задержки - исследовать начальное рукопожатие соединения и первые несколько следующих за ним пакетов. В качестве примера рассмотрим простое соединение клиента с веб-сервером, когда клиент пытается открыть размещённый на сервере сайт. Нас интересуют первые шесть пакетов этой последовательности: рукопожатие TCP, начальный HTTP-запрос GET, подтверждение этого запроса и первый пакет данных, отправленный сервером клиенту.

Примечание. Чтобы выполнять действия из этого раздела, установите в Wireshark правильный формат отображения времени: выберите View > Time Display Format > Seconds Since Previous Displayed Packet.

Нормальный обмен

Немного позже в этой главе мы подробно обсудим базовые профили сети. Пока достаточно знать, что для сравнения с условиями большой задержки нужен базовый профиль нормального обмена. В примерах используется файл `latency1.pcapng`. Подробности рукопожатия TCP и обмена HTTP уже рассматривались, поэтому возвращаться к ним мы не будем. Более того, панель подробностей пакета нам вообще не понадобится. Нас интересует только столбец Time, показанный на рис. 11-22.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.030107	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM=1 WS=64
3	0.000075	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
4	0.000066	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/1.1
5	0.048778	74.125.95.104	172.16.16.128	TCP	60	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
6	0.022176	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]

Рисунок 11-22. Этот обмен происходит очень быстро и может считаться нормальным

Вся последовательность обмена выполняется очень быстро, менее чем за 0,1 секунды.

В нескольких следующих файлах захвата будет представлен тот же шаблон трафика, но с другими временными интервалами между пакетами.

Медленный обмен: задержка в линии

Теперь обратимся к файлу захвата `latency2.pcapng`. Обратите внимание, что все пакеты остались прежними, за исключением значений времени у двух из них, как показано на рис. 11-23.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.878530	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM=1 WS=64
3	0.016604	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
4	0.000335	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/1.1
5	1.155228	74.125.95.104	172.16.16.128	TCP	60	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
6	0.015866	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]

Рисунок 11-23. Пакеты 2 и 5 демонстрируют большую задержку

Просматривая эти шесть пакетов по очереди, мы сразу сталкиваемся с первым признаком задержки. Клиент (`172.16.16.128`) отправляет начальный пакет SYN, чтобы начать рукопожатие TCP, а ответный SYN/ACK от сервера (`74.125.95.104`) поступает с задержкой 0,87 секунды. Это первый признак задержки в линии, вызванной устройством между клиентом и сервером.

Сделать вывод о задержке в линии позволяет характер передаваемых пакетов. Получив SYN, сервер должен выполнить лишь минимальную обработку, чтобы отправить ответ, поскольку никаких действий выше транспортного уровня не требуется. Даже при очень высокой нагрузке сервер обычно быстро отвечает на SYN пакетом SYN/ACK. Поэтому сервер можно исключить из числа возможных причин большой задержки.

Клиент тоже исключается, поскольку на этом этапе он не выполняет никакой обработки, кроме простого получения пакета SYN/ACK. Исключив и клиента, и сервер, уже по первым двум пакетам захвата мы понимаем, что потенциальный источник медленного обмена находится между ними.

Далее видно, что пакет ACK, завершающий трёхэтапное рукопожатие, передаётся быстро, как и отправленный клиентом HTTP-запрос GET. Вся обработка, создающая эти два пакета,

выполняется локально на клиенте после получения SYN/ACK, поэтому при отсутствии высокой вычислительной нагрузки у клиента оба пакета должны передаваться быстро.

В пакете 5 снова наблюдается чрезвычайно большое значение времени. Похоже, после отправки начального HTTP-запроса GET ответный пакет ACK от сервера был получен лишь через 1,15 секунды. Получив HTTP-запрос GET, сервер перед началом отправки данных передал TCP ACK, что опять же требует от него очень небольшой обработки. Это ещё один признак задержки в линии.

При задержке в линии она почти всегда проявляется и в SYN/ACK начального рукопожатия, и в других пакетах ACK на протяжении обмена. Эти сведения не указывают точный источник большой задержки в сети, но показывают, что им не являются ни клиент, ни сервер: задержку создаёт находящееся между ними устройство. Теперь можно исследовать различные межсетевые экраны, маршрутизаторы и прокси-серверы, чтобы найти виновника.

Медленный обмен: задержка клиента

Следующий сценарий задержки содержится в файле `latency3.pcapng`, показанном на рис. 11-24.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.023790	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM=1 WS=64
3	0.014894	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
4	1.345023	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/1.1
5	0.046121	74.125.95.104	172.16.16.128	TCP	60	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
6	0.016182	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]

Рисунок 11-24. Медленный пакет в этом захвате - начальный HTTP-запрос GET

Захват начинается нормально: рукопожатие TCP выполняется очень быстро и без признаков задержки. Всё выглядит хорошо вплоть до пакета 4 - HTTP-запроса GET после завершения рукопожатия. Между ним и предыдущим полученным пакетом проходит 1,34 секунды.

Чтобы определить источник этой задержки, нужно исследовать происходящее между пакетами 3 и 4. Пакет 3 - завершающий рукопожатие TCP пакет ACK от клиента серверу, а пакет 4 - запрос GET, также отправленный клиентом серверу. Общая черта в том, что оба пакета отправляет клиент и они не зависят от сервера. Поскольку все эти действия сосредоточены на клиенте, запрос GET должен появиться вскоре после отправки ACK.

К несчастью для конечного пользователя, переход от ACK к GET происходит не быстро. Создание и передача пакета GET требуют обработки вплоть до прикладного уровня, а задержка этой обработки означает, что клиент не смог выполнить действие своевременно. Следовательно, в большой задержке обмена в конечном счёте виноват клиент.

Медленный обмен: задержка сервера

Последний сценарий задержки исследуется в файле `latency4.pcapng`, показанном на рис. 11-25. Это пример задержки сервера.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM=1
2	0.018583	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM=1 WS=64
3	0.016197	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
4	0.000172	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/1.1
5	0.047936	74.125.95.104	172.16.16.128	TCP	60	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
6	0.982983	74.125.95.104	172.16.16.128	TCP	1460	[TCP segment of a reassembled PDU]

Рисунок 11-25. Большая задержка проявляется только в последнем пакете этого захвата

Рукопожатие TCP между двумя узлами выполняется безупречно и быстро, так что начало выглядит хорошо. Следующие несколько пакетов тоже радуют: начальный запрос GET и ответный ACK доставляются быстро. Признаки большой задержки появляются лишь в последнем пакете файла.

Шестой пакет - первый пакет данных HTTP, отправленный сервером в ответ на запрос GET клиента; он поступает с задержкой 0,98 секунды после того, как сервер отправил TCP ACK для запроса GET. Переход между пакетами 5 и 6 очень похож на переход между ACK рукопожатия и запросом GET в предыдущем сценарии. Однако на этот раз нас интересует сервер.

Пакет 5 - ACK, отправленный сервером в ответ на запрос GET клиента. Сразу после его передачи сервер должен начать отправлять данные. Доступ к данным, их упаковку и передачу в этом пакете выполняет протокол HTTP, а поскольку это протокол прикладного уровня, серверу требуется определённая обработка. Задержка получения пакета указывает, что сервер не смог обработать данные за разумное время, и в конечном счёте именно он оказывается источником задержки в этом файле захвата.

Схема определения источника задержки

Всего по шести пакетам нам удалось в нескольких сценариях определить источник большой сетевой задержки между клиентом и сервером. Схема на рис. 11-26 должна помочь вам устранять собственные проблемы с задержкой. Эти принципы применимы почти к любому обмену на основе TCP.

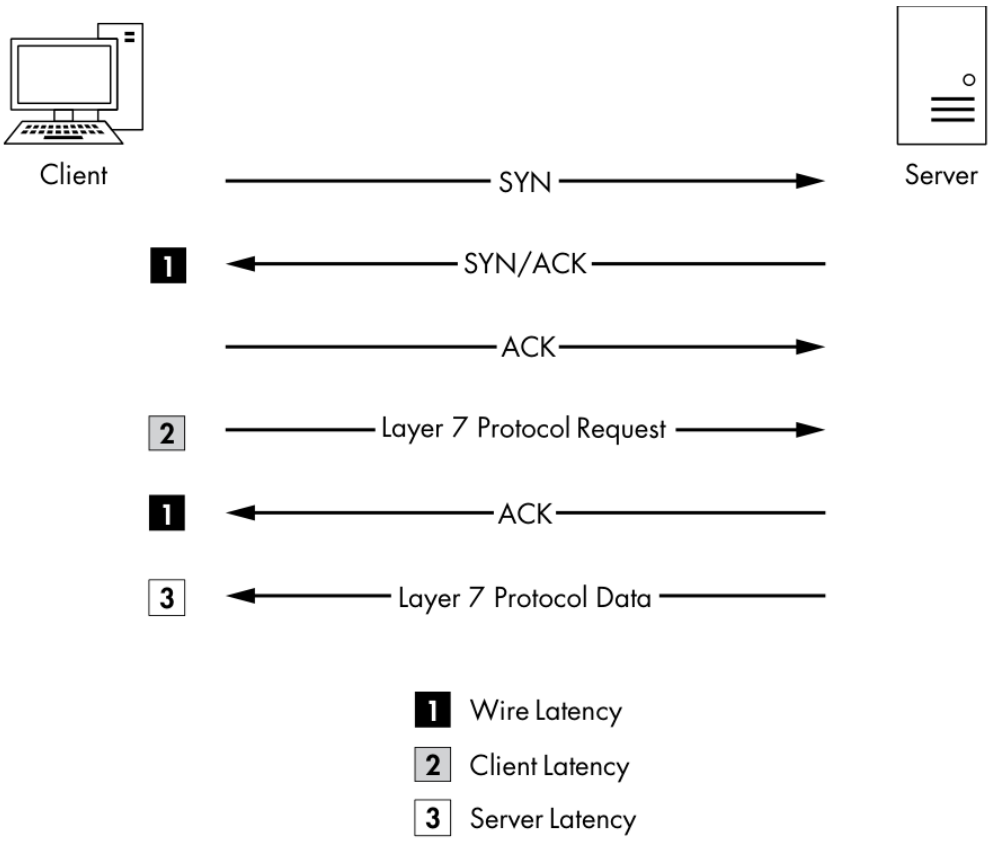


Рисунок 11-26. Эту схему можно использовать для устранения собственных проблем с задержкой

> **Примечание.** Обратите внимание, что задержка UDP почти не обсуждалась. UDP рассчитан на скорость, но не на надёжность, поэтому у него нет встроенных функций обнаружения задержки и восстановления после неё. Вместо этого надёжность доставки данных обеспечивают работающие с ним протоколы прикладного уровня и ICMP.

Создание базового профиля сети

Когда все прочие методы не помогают, базовый профиль сети может стать одним из важнейших источников данных при устранении сетевой медлительности. Для наших целей базовый профиль сети - это выборка трафика из различных точек сети, включающая значительный объём того, что мы считаем нормальным сетевым трафиком. Его назначение - служить основой для сравнения, когда сеть или находящиеся в ней устройства ведут себя неправильно.

Например, представьте ситуацию, когда несколько клиентов в сети жалуются на медленный вход на локальный сервер веб-приложения. Захватив этот трафик и сравнив его с базовым профилем сети, можно обнаружить, что веб-сервер отвечает нормально, однако внешние DNS-запросы, возникающие из-за встроенного в веб-приложение внешнего содержимого, выполняются вдвое медленнее обычного.

Медленный внешний DNS-сервер можно было бы заметить и без базового профиля, но в случае едва заметных изменений это не всегда возможно. Десять DNS-запросов, каждый из которых обрабатывается на 0,1 секунды дольше обычного, не лучше одного DNS-запроса с дополнительной задержкой в целую секунду, однако первую ситуацию гораздо труднее выявить без базового профиля сети.

Поскольку двух одинаковых сетей не бывает, состав базового профиля может сильно различаться. В следующих разделах приведены примеры его компонентов. Возможно, к вашей сетевой инфраструктуре применимы все эти элементы или лишь немногие из них. В любом случае каждый компонент базового профиля должен относиться к одной из трёх основных категорий: площадка, узел или приложение.

Базовый профиль площадки

Назначение базового профиля площадки - получить общий снимок трафика на каждой физической площадке сети. В идеале он должен охватывать каждый сегмент WAN.

Такой профиль может включать следующие компоненты.

Используемые протоколы

Чтобы видеть трафик всех устройств, захватывайте его от всех устройств сегмента на границе сети, то есть на маршрутизаторе или межсетевом экране, и используйте окно Protocol Hierarchy Statistics (Statistics > Protocol Hierarchy). Позже результат иерархии можно применять для сравнения: определять исчезновение обычно присутствующих протоколов, появление в сети новых протоколов и необычно большие объёмы трафика определённых типов по протоколам.

Широковещательный трафик

Сюда входит весь широковещательный трафик сегмента. Перехват в любой точке площадки должен позволить захватить его целиком и узнать, кто или что обычно создаёт в сети большой объём широковещательных передач. Тогда можно быстро определить, не стало ли их слишком много или, наоборот, слишком мало.

Последовательности аутентификации

Сюда входит трафик процессов аутентификации случайно выбранных клиентов во всех службах, например Active Directory, веб-приложениях и специализированном программном обеспечении организации. Аутентификация - одна из областей, где службы часто работают медленно. Базовый профиль позволяет определить, виновата ли она в медленном обмене.

Скорость передачи данных

Обычно этот компонент представляет собой измерение крупной передачи данных с площадки на различные другие площадки сети. Чтобы определить скорость передачи и стабильность соединения, можно использовать сводку захвата и функции построения графиков Wireshark, продемонстрированные в главе 5. Вероятно, это самый важный базовый профиль площадки. Когда любое входящее в сегмент или исходящее из него соединение кажется медленным, можно выполнить ту же передачу данных, что и в базовом профиле, и сравнить результаты. Это покажет, действительно ли соединение медленное, и, возможно, даже поможет найти область, где начинается замедление.

Базовый профиль узла

Вероятно, создавать базовый профиль каждого отдельного узла сети не требуется. Это следует делать только для серверов с большим объёмом трафика или критически важных серверов. Проще говоря, если медленная работа сервера приведёт к гневным звонкам руководства, у вас должен быть его базовый профиль.

Базовый профиль узла включает следующие компоненты.

Используемые протоколы

Этот профиль хорошо подходит для применения окна Protocol Hierarchy Statistics при захвате трафика узла. Позже его можно использовать для сравнения: определять исчезновение обычно присутствующих протоколов, появление на узле новых протоколов и необычно большие объёмы трафика определённых типов по протоколам.

Трафик в периоды простоя и нагрузки

Этот профиль состоит из обычных захватов нормального рабочего трафика в периоды пиковой и минимальной нагрузки. Зная количество соединений и используемую ими пропускную способность в разное время суток, можно определить, является ли медлительность результатом пользовательской нагрузки или другой проблемы.

Запуск и завершение работы

Для получения этого профиля необходимо захватить трафик, создаваемый при запуске и завершении работы узла. Если компьютер отказывается загружаться или выключаться либо любой из этих процессов происходит необычно медленно, по базовому профилю можно определить, связана ли причина с сетью.

Последовательности аутентификации

Для получения этого профиля нужно захватить трафик процессов аутентификации во всех службах узла. Аутентификация - одна из областей, где службы часто работают медленно. Базовый профиль позволяет определить, виновата ли она в медленном обмене.

Связи и зависимости

Этот профиль представляет собой более длительный захват, предназначенный для определения других узлов, от которых зависит данный узел и которые зависят от него. Эти связи и зависимости можно просмотреть в окне Conversations (Statistics > Conversations). Например, веб-сервер может зависеть от узла SQL Server. Мы не всегда знаем о скрытых зависимостях между узлами, и базовый профиль позволяет их определить. Затем можно установить, не вызвана ли неправильная работа узла отказом или большой задержкой одной из зависимостей.

Базовый профиль приложения

Последняя категория - базовый профиль приложения. Его следует создавать для всех критически важных сетевых бизнес-приложений.

Базовый профиль приложения включает следующие компоненты.

Используемые протоколы

И для этого профиля используйте окно Protocol Hierarchy Statistics в Wireshark, но на этот раз захватывайте трафик узла, на котором работает приложение. Позже с этим списком можно сравнивать текущую ситуацию, чтобы выявлять неправильную работу или полное отсутствие протоколов, от которых зависит приложение.

Запуск и завершение работы

Этот профиль включает захват трафика, создаваемого во время запуска и завершения работы приложения. Если приложение не запускается либо один из этих процессов происходит необычно медленно, по базовому профилю можно определить причину.

Связи и зависимости

Для этого профиля нужен более длительный захват, в котором с помощью окна Conversations можно определить другие узлы и приложения, от которых зависит данное приложение. Мы не всегда знаем о скрытых зависимостях между приложениями, и базовый профиль позволяет их определить. Затем можно установить, не вызвана ли неправильная работа приложения отказом или большой задержкой одной из зависимостей.

Скорость передачи данных

С помощью сводки захвата и функций построения графиков Wireshark можно определить скорость передачи и стабильность соединений с сервером приложения во время его нормальной работы. Когда поступает сообщение о медленной работе приложения, этот базовый профиль поможет установить, вызвана ли проблема высокой утилизацией ресурсов или большой пользовательской нагрузкой.

Дополнительные замечания о базовых профилях

При создании базового профиля сети помните ещё о нескольких моментах:

• Каждый базовый профиль следует захватывать не менее трёх раз: один раз при малом объёме трафика ранним утром, один раз при большом объёме трафика в середине дня и один раз при отсутствии трафика поздней ночью.
• По возможности не выполняйте захват непосредственно с узлов, для которых создаёте базовый профиль. В период интенсивного трафика это может увеличить нагрузку на устройство, ухудшить его производительность и сделать профиль недействительным из-за отброшенных пакетов.
• Базовый профиль будет содержать весьма конфиденциальные сведения о вашей сети, поэтому обязательно защитите его. Храните его в безопасном месте, доступном только соответствующим лицам, но в то же время достаточно доступном, чтобы воспользоваться им при необходимости. Рассмотрите хранение на USB-накопителе или в зашифрованном разделе.
• Сохраняйте все связанные с базовым профилем файлы `.pcap` и `.pcapng` и создайте памятку с наиболее часто используемыми значениями, например связями или средней скоростью передачи данных.

Заключение

Эта глава была посвящена устранению неполадок в медленных сетях. Мы рассмотрели некоторые из наиболее полезных функций TCP для обнаружения нарушений надёжности и восстановления, показали, как определить источник большой задержки в сетевом обмене, а также обсудили важность базового профиля сети и некоторые его компоненты. С помощью описанных здесь методов в сочетании с функциями построения графиков и анализа Wireshark вы будете хорошо подготовлены к устранению неполадок, когда поступит жалоба на медленную работу сети.

Глава 12. Анализ пакетов для обеспечения безопасности

Хотя основная часть этой книги посвящена применению анализа пакетов для устранения сетевых неполадок, в реальной работе значительный объём анализа выполняется в целях безопасности. Например, аналитик вторжений может исследовать сетевой трафик потенциальных злоумышленников, а специалист по компьютерной криминалистике - пытаться определить масштаб заражения вредоносной программой на скомпрометированном узле.

Анализ пакетов при расследовании инцидентов безопасности всегда представляет сложную задачу, поскольку в нём присутствует неизвестный элемент - устройство, контролируемое злоумышленником. Вы не можете подойти к рабочему месту злоумышленника, задать вопрос или создать базовый профиль его нормального трафика; в вашем распоряжении только тот обмен между его системой и вашей, который удастся захватить. К счастью, чтобы удалённо взломать одну из ваших систем, злоумышленник должен тем или иным образом взаимодействовать с сетью. Разумеется, он тоже это знает, поэтому располагает множеством приёмов для сокрытия своих методов.

В этой главе мы займём позицию специалиста по безопасности и исследуем различные аспекты компрометации системы на сетевом уровне. Мы рассмотрим сетевую разведку, вредоносное перенаправление трафика и распространённые методы вредоносных программ. В некоторых случаях нам предстоит сыграть роль аналитика вторжений и разобрать трафик на основании предупреждений системы обнаружения вторжений (intrusion-detection system, IDS). Эта глава даст представление о сетевой безопасности, которое может оказаться критически важным, даже если сейчас ваша работа непосредственно с безопасностью не связана.

Разведка

Первым шагом злоумышленника часто становится углублённое исследование целевой системы. Этот этап, обычно называемый сбором следов (footprinting), нередко выполняется с помощью различных общедоступных ресурсов, например сайта целевой компании или Google. Завершив исследование, злоумышленник обычно начинает сканировать IP-адрес или DNS-имя цели в поисках открытых портов и работающих служб.

Сканирование позволяет определить, активна и доступна ли цель. Представьте, например, что грабители планируют ограбить крупнейший банк города, расположенный по адресу Мейн-стрит, 123. Несколько недель они разрабатывают сложный план, но, прибыв по адресу, узнают, что банк переехал на Вайн-стрит, 555. Или, что ещё хуже, грабители собираются войти в банк в обычные рабочие часы и похитить деньги из хранилища, но на месте обнаруживают, что сегодня банк закрыт. И при ограблении банка, и при атаке на сеть первым препятствием будет необходимость убедиться, что цель активна и доступна.

Сканирование также сообщает злоумышленнику, на каких портах цель ожидает соединений. Вернёмся к аналогии с грабителями банка и представим, что они пришли туда, совершенно не зная физической планировки здания. Они не представляли бы, как добраться до хранилища, поскольку им неизвестны слабые места физической безопасности банка.

В этом разделе рассматриваются несколько наиболее распространённых методов сканирования, применяемых для выявления узлов, их открытых портов и уязвимостей в сети.

Примечание. До сих пор стороны соединения в книге назывались передатчиком и получателем либо клиентом и сервером. В этой главе каждая сторона обмена называется либо злоумышленником, либо целью.

Сканирование SYN

Первым против системы часто выполняется сканирование TCP SYN, также называемое скрытым или полуоткрытым сканированием. Это самый распространённый вид сканирования по нескольким причинам:

• Оно очень быстрое и надёжное.
• Оно даёт точные результаты на всех платформах независимо от реализации стека TCP.
• Оно создаёт меньше шума, чем другие методы сканирования.

Сканирование TCP SYN опирается на трёхэтапное рукопожатие, чтобы определить открытые порты целевого узла. Злоумышленник отправляет пакеты TCP SYN на ряд портов цели, словно пытаясь установить по ним канал обычного обмена. После получения такого пакета целью возможен один из нескольких результатов, показанных на рис. 12-1.

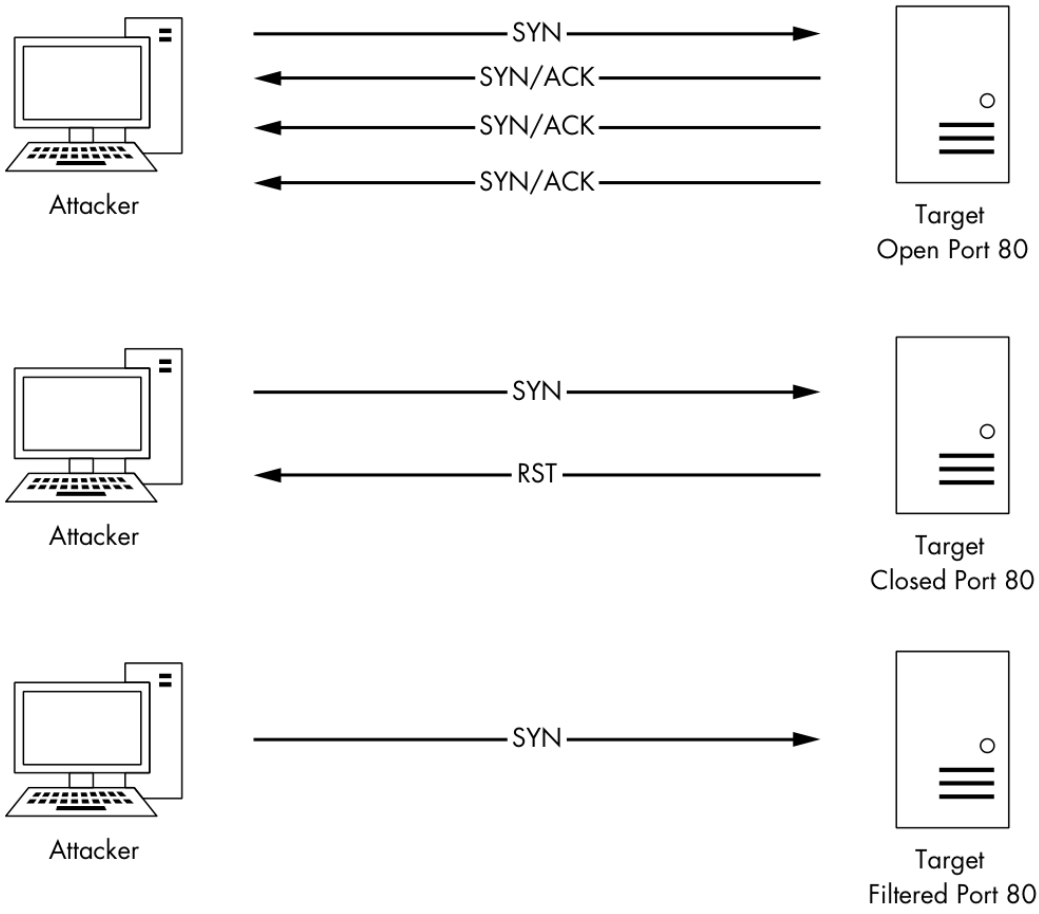


Рисунок 12-1. Возможные результаты сканирования TCP SYN

Если работающая на целевой машине служба ожидает соединений на порту, получившем SYN, она отвечает злоумышленнику пакетом TCP SYN/ACK - второй частью рукопожатия TCP. Теперь злоумышленник знает, что порт открыт и на нём работает служба. В обычных обстоятельствах для завершения рукопожатия соединения был бы отправлен последний TCP ACK. Однако в данном случае злоумышленнику это не нужно, поскольку на этом этапе он не собирается продолжать обмен с узлом, и поэтому не пытается завершить рукопожатие TCP.

Если на сканируемом порту ни одна служба не ожидает соединений, злоумышленник не получит SYN/ACK. В зависимости от конфигурации операционной системы цели в ответ может поступить пакет RST, указывающий, что порт закрыт. Другой вариант - полное отсутствие ответа. Оно может означать, что порт фильтруется промежуточным устройством, например межсетевым экраном, или самим узлом. С другой стороны, ответ мог просто потеряться при передаче. Поэтому, хотя такой результат обычно указывает на закрытый порт, окончательного вывода сделать нельзя.

Файл `synscan.pcapng` содержит прекрасный пример сканирования SYN с помощью Nmap. Nmap - мощное приложение для сканирования сети, разработанное Гордоном "Фёдором" Лайоном. Оно способно выполнить практически любой вид сканирования, который только можно представить. Nmap можно бесплатно загрузить с сайта http://www.nmap.com/download.html.

Наш пример захвата содержит около 2000 пакетов, то есть сканирование имеет немалый масштаб. Один из лучших способов оценить охват такого сканирования - открыть окно Conversations, показанное на рис. 12-2. В нём должна быть только одна беседа IPv4 (метка 1) между злоумышленником (`172.16.0.8`) и целью (`64.13.134.52`). Кроме того, между этими двумя узлами присутствует 1994 беседы TCP (метка 2) - фактически новая беседа для каждой пары портов, участвующей в обмене.

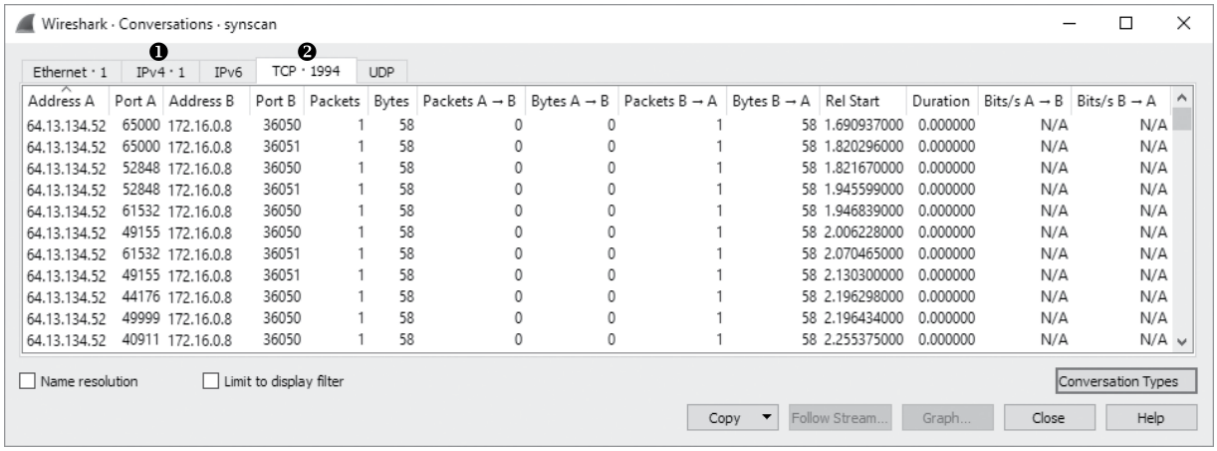


Рисунок 12-2. Окно Conversations показывает разнообразие происходящих обменов TCP

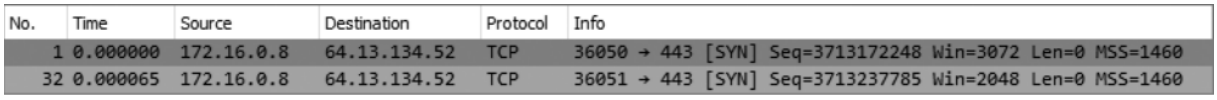
Сканирование выполняется очень быстро, поэтому прокрутка файла захвата - не лучший способ найти ответ, связанный с каждым начальным пакетом SYN. До получения ответа на исходный пакет может быть отправлено ещё несколько пакетов. К счастью, найти нужный трафик помогут фильтры.

Использование фильтров при сканировании SYN

В качестве примера фильтрации рассмотрим первый пакет захвата - SYN, отправленный цели на порт 443 (HTTPS). Чтобы проверить наличие ответа, можно создать фильтр, отображающий весь трафик к порту 443 и от него. Быстрее всего это сделать так:

1. Выберите первый пакет файла захвата.
2. Разверните заголовок TCP на панели подробностей пакета.
3. Щёлкните правой кнопкой мыши поле Destination Port, выберите Prepare as Filter, затем Selected.
4. В строку фильтра будет помещён фильтр для всех пакетов с портом назначения 443. Поскольку нам нужны также все пакеты с исходным портом 443, щёлкните строку фильтра в верхней части экрана и удалите из фильтра часть `dst`.

Получившийся фильтр выведет два пакета. Оба являются TCP SYN, отправленными злоумышленником цели, как показано на рис. 12-3.



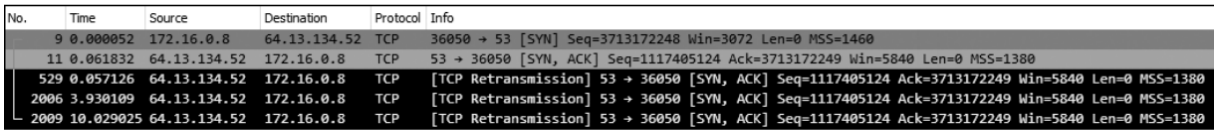
No.	Time	Source	Destination	Protocol	Info
1	0.000000	172.16.0.8	64.13.134.52	TCP	36050 → 443 [SYN] Seq=3713172248 Win=3072 Len=0 MSS=1460
32	0.000065	172.16.0.8	64.13.134.52	TCP	36051 → 443 [SYN] Seq=3713237785 Win=2048 Len=0 MSS=1460

Рисунок 12-3. Две попытки установить соединение с помощью пакетов SYN

> **Примечание.** В этом разделе пакеты показаны с форматом отображения времени Seconds Since Previous Displayed Packet.

Поскольку ни на один из пакетов ответа нет, возможно, он фильтруется целевым узлом или промежуточным устройством либо порт закрыт. В конечном счёте результат сканирования порта 443 не позволяет сделать определённый вывод.

Попробуем применить тот же метод к другому пакету и проверить, будет ли результат иным. Очистите предыдущий фильтр и выберите в списке пакет 9. Это SYN для порта 53, обычно связанного с DNS. С помощью описанного выше метода или путём изменения последнего фильтра создайте фильтр, отображающий весь трафик TCP порта 53. После применения фильтра вы увидите пять пакетов, как показано на рис. 12-4.

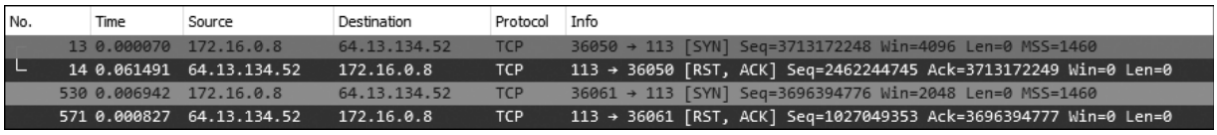


No.	Time	Source	Destination	Protocol	Info
9	0.000052	172.16.0.8	64.13.134.52	TCP	36050 → 53 [SYN] Seq=3713172248 Win=3072 Len=0 MSS=1460
11	0.061832	64.13.134.52	172.16.0.8	TCP	53 → 36050 [SYN, ACK] Seq=1117405124 Ack=3713172249 Win=5840 Len=0 MSS=1380
529	0.057126	64.13.134.52	172.16.0.8	TCP	[TCP Retransmission] 53 → 36050 [SYN, ACK] Seq=1117405124 Ack=3713172249 Win=5840 Len=0 MSS=1380
2006	3.930109	64.13.134.52	172.16.0.8	TCP	[TCP Retransmission] 53 → 36050 [SYN, ACK] Seq=1117405124 Ack=3713172249 Win=5840 Len=0 MSS=1380
2009	10.029025	64.13.134.52	172.16.0.8	TCP	[TCP Retransmission] 53 → 36050 [SYN, ACK] Seq=1117405124 Ack=3713172249 Win=5840 Len=0 MSS=1380

Рисунок 12-4. Пять пакетов указывают, что порт открыт

Первый из них - выбранный нами в начале захвата SYN, пакет 9. Второй - ответ от цели. Это TCP SYN/ACK, то есть ожидаемый ответ при установлении трёхэтапного рукопожатия. В обычных обстоятельствах следующим пакетом был бы ACK от узла, отправившего начальный SYN. Однако злоумышленник не хочет завершать соединение и не отвечает. Поэтому цель ещё трижды повторно передаёт SYN/ACK, после чего прекращает попытки. Поскольку при попытке обмена с узлом через порт 53 получен ответ SYN/ACK, можно обоснованно предположить, что на этом порту ожидает соединений некоторая служба.

Повторим процесс ещё раз для пакета 13. Это SYN, отправленный на порт 113, обычно связанный с протоколом Ident, который часто применяется службами идентификации и аутентификации IRC. Применив фильтр того же типа к указанному в пакете порту, вы увидите четыре пакета, как показано на рис. 12-5.



No.	Time	Source	Destination	Protocol	Info
13	0.000070	172.16.0.8	64.13.134.52	TCP	36050 → 113 [SYN] Seq=3713172248 Win=4096 Len=0 MSS=1460
14	0.061491	64.13.134.52	172.16.0.8	TCP	113 → 36050 [RST, ACK] Seq=2462244745 Ack=3713172249 Win=0 Len=0
530	0.006942	172.16.0.8	64.13.134.52	TCP	36061 → 113 [SYN] Seq=3696394776 Win=2048 Len=0 MSS=1460
571	0.000827	64.13.134.52	172.16.0.8	TCP	113 → 36061 [RST, ACK] Seq=1027049353 Ack=3696394777 Win=0 Len=0

Рисунок 12-5. За SYN следует RST, указывающий, что порт закрыт

Первый пакет - начальный SYN, сразу за которым следует RST от цели. Это указывает, что цель не принимает соединения на выбранном порту и, вероятнее всего, на нём не работает никакая служба.

Выявление открытых и закрытых портов

Теперь, когда вы понимаете различные виды ответов, которые может вызвать сканирование SYN, нужно найти быстрый способ определять открытые и закрытые порты. Решение снова находится в окне Conversations. В нём можно отсортировать беседы TCP по числу пакетов, разместив наибольшие значения сверху: щёлкайте заголовок столбца Packets, пока стрелка не будет указывать вниз, как на рис. 12-6.

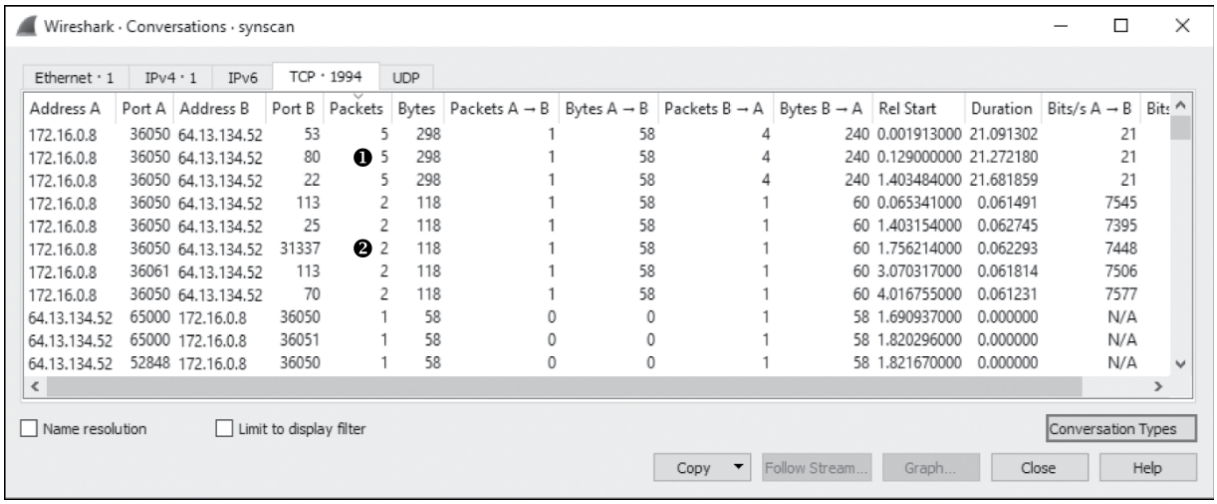


Рисунок 12-6. Поиск открытых портов с помощью окна Conversations

В беседах для трёх просканированных портов содержится по пять пакетов (метка 1). Мы знаем, что порты 53, 80 и 22 открыты, поскольку эти пять пакетов представляют собой начальный SYN, соответствующий SYN/ACK и повторные передачи SYN/ACK от цели.

Для пяти портов в обмене участвовали лишь по два пакета (метка 2). Первый - начальный SYN, второй - RST от цели. Эти результаты указывают, что порты 113, 25, 31337, 113 и 70 закрыты.

Примечание переводчика. Порт 113 дважды указан в исходном перечне.

Остальные записи окна Conversations содержат только по одному пакету: целевой узел так и не ответил на начальный SYN. Эти порты, скорее всего, закрыты, но точно мы этого не знаем.

Метод подсчёта пакетов сработал для данного узла, но его результаты не будут единообразными для всех сканируемых узлов, поэтому полагаться исключительно на него нельзя. Лучше научитесь распознавать нормальные стимул и ответ, а также понимать, что могут означать ненормальные ответы на нормальные стимулы.

Определение операционной системы по отпечатку

Сведения об операционной системе цели имеют для злоумышленника большую ценность. Зная её, он может правильно настроить все методы атаки для данной системы. Кроме того, в случае успешного доступа к системе злоумышленник будет знать расположение определённых критически важных файлов и каталогов в целевой файловой системе.

Определением операционной системы по отпечатку (operating system fingerprinting) называется группа методов, позволяющих установить работающую в системе ОС без физического доступа к ней. Существует два вида такого определения: пассивное и активное.

Пассивное определение по отпечатку

При пассивном определении исследуются определённые поля пакетов, отправленных целью, чтобы установить используемую операционную систему. Метод считается пассивным, поскольку вы лишь прослушиваете пакеты, которые отправляет целевой узел, и сами активно ему ничего не передаёте. Для злоумышленников такой вид определения ОС идеален, так как позволяет действовать скрытно.

Но как определить операционную систему узла только по отправляемым им пакетам? Это возможно из-за отсутствия стандартизированных значений в спецификациях, определённых RFC протоколов. Хотя поля заголовков TCP, UDP и IP заданы очень конкретно, значения по умолчанию обычно определены не для каждого поля. Поэтому реализация стека TCP/IP каждой операционной системы должна сама устанавливать для них значения по умолчанию. В таблице 12-1 перечислены некоторые наиболее распространённые поля и значения по умолчанию, по которым их можно связать с различными операционными системами. Помните, что с выпуском новых версий ОС эти значения могут измениться.

Таблица 12-1. Распространённые значения для пассивного определения по отпечатку

Заголовок протокола	Поле	Значение по умолчанию	Платформа
IP	Initial time to live	64	Nmap, BSD, OS X, Linux
IP	Initial time to live	128	Novell, Windows
IP	Initial time to live	255	Cisco IOS, Palm OS, Solaris
IP	Don't fragment flag	Set	BSD, OS X, Linux, Novell, Windows, Palm OS, Solaris
IP	Don't fragment flag	Not set	Nmap, Cisco IOS
TCP	Maximum segment size	0	Nmap
TCP	Maximum segment size	1440-1460	Windows, Novell
TCP	Maximum segment size	1460	BSD, OS X, Linux, Solaris
TCP	Window size	1024-4096	Nmap
TCP	Window size	65535	BSD, OS X
TCP	Window size	Variable	Linux
TCP	Window size	16384	Novell
TCP	Window size	4128	Cisco IOS
TCP	Window size	24820	Solaris
TCP	Window size	Variable	Windows
TCP	SackOK	Set	Linux, Windows, OS X, OpenBSD
TCP	SackOK	Not set	Nmap, FreeBSD, Novell, Cisco IOS, Solaris

Пакеты в файле `passiveosfingerprinting.pcapng` прекрасно демонстрируют этот метод. В файле два пакета. Оба являются TCP SYN, отправленными на порт 80, но поступают от разных узлов. Пользуясь только содержащимися в пакетах значениями и таблицей 12-1, мы должны суметь определить архитектуру операционной системы каждого узла. Подробности пакетов показаны на рис. 12-7.

Используя таблицу 12-1 как справочник, можно составить таблицу 12-2 с разбором соответствующих полей этих пакетов.

Таблица 12-2. Разбор пакетов определения операционной системы по отпечатку

Заголовок протокола	Поле	Значение пакета 1	Значение пакета 2
IP	Initial time to live	128	64
IP	Don't fragment flag	Set	Set
TCP	Maximum segment size	1440 байт	1460 байт

Заголовок протокола	Поле	Значение пакета 1	Значение пакета 2
TCP	Window size	64 240 байт	2920 байт
TCP	SackOK	Set	Set

На основании этих значений можно заключить, что пакет 1, скорее всего, отправлен устройством под управлением Windows, а пакет 2 - устройством под управлением Linux.

Помните, что приведённый в таблице 12-1 перечень распространённых идентифицирующих полей для пассивного определения по отпечатку далеко не исчерпывающий. Существует множество особенностей, способных вызвать отклонение от ожидаемых значений. Поэтому полностью полагаться на результаты пассивного определения операционной системы по отпечатку нельзя.

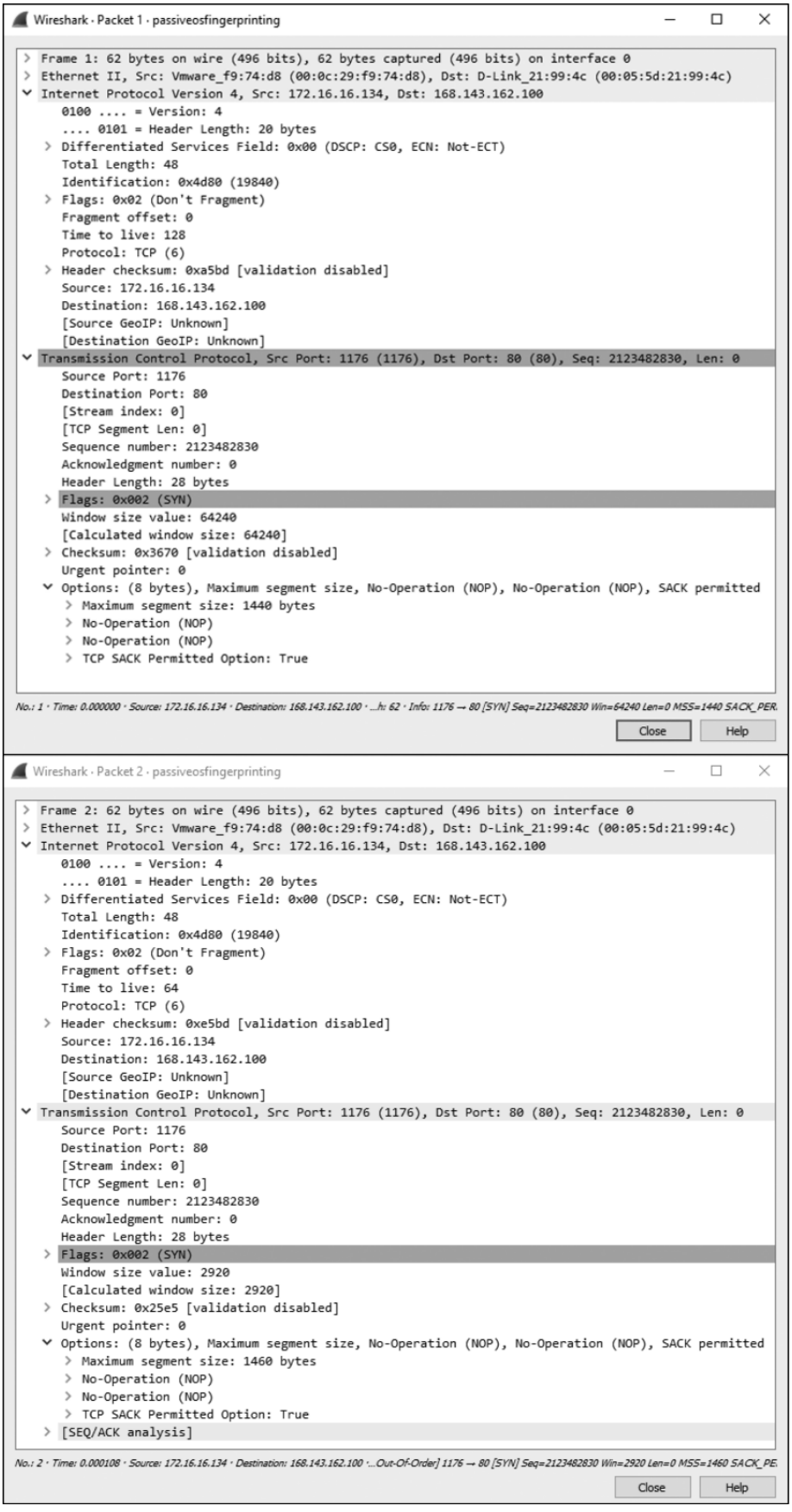


Рисунок 12-7. Эти пакеты могут сообщить, из какой операционной системы они были отправлены

> **Примечание.** Во многих случаях злоумышленники полагаются на автоматизированные инструменты пассивного определения операционной системы цели. Один из инструментов, использующих методы определения ОС по отпечатку, - p0f. Он анализирует соответствующие поля захвата пакетов и выводит предполагаемую операционную систему. С помощью таких средств, как p0f, иногда можно узнать не только архитектуру операционной системы, но даже её версию или уровень исправлений. Загрузить p0f можно с сайта http://lcamtuf.coredump.cx/p0f.shtml.

Активное определение по отпечатку

Если пассивное наблюдение за трафиком не даёт желаемых результатов, может потребоваться более прямой подход - активное определение по отпечатку. Теперь злоумышленник активно отправляет цели специально сформированные пакеты, чтобы вызвать ответы, раскрывающие операционную систему целевой машины. Разумеется, поскольку этот подход предполагает непосредственный обмен с целью, скрытным его назвать нельзя, но он может быть весьма эффективным.

Файл `activeosfingerprinting.pcapng` содержит пример активного определения операционной системы по отпечатку, запущенного утилитой сканирования Nmap. Несколько пакетов файла являются результатом отправки Nmap различных проб, предназначенных для получения ответов, по которым можно идентифицировать операционную систему. Nmap записывает ответы на эти пробы, строит отпечаток и сравнивает его с базой значений, чтобы сделать вывод.

> **Примечание.** Методы, применяемые Nmap для активного определения операционной системы по отпечатку, весьма сложны. Подробнее о том, как Nmap выполняет эту задачу, читайте в исчерпывающем руководстве *Nmap Network Scanning* (2008), написанном автором инструмента Гордоном "Фёдором" Лайоном.

Манипулирование трафиком

Одна из ключевых мыслей, которую я старался показать на протяжении книги, состоит в том, что, исследуя правильные пакеты, можно многое узнать о системе и её пользователях. Поэтому неудивительно, что злоумышленники часто стремятся сами захватить эти пакеты. По пакетам, создаваемым системой, злоумышленник может узнать об операционной системе, используемых приложениях, учётных данных аутентификации и многом другом.

В этом разделе на уровне пакетов рассматриваются два метода: применение злоумышленником отравления кеша ARP для перехвата и захвата целевого трафика, а также перехват файлов cookie HTTP для атак с захватом сеанса.

Отравление кеша ARP

В главе 7 мы обсуждали, как протокол ARP позволяет устройствам сопоставлять IP-адреса с MAC-адресами внутри сети, а в главе 2 - как отравление кеша ARP может служить полезным методом подключения к линии и перехвата трафика узлов, пакеты которых требуется проанализировать. При законном применении отравление кеша ARP очень помогает устранять неполадки. Однако при злонамеренном использовании оно становится смертоносной формой атаки посредника (man-in-the-middle, MITM).

При атаке MITM злоумышленник перенаправляет трафик между двумя узлами, чтобы перехватывать или изменять передаваемые данные. Существует множество форм MITM, включая подмену DNS и захват SSL. При отравлении кеша ARP специально сформированные пакеты ARP заставляют два узла считать, что они обмениваются друг с другом, хотя в действительности каждый из них связывается с третьей стороной, которая ретранслирует пакеты в качестве посредника. Так незаконное применение нормальной функциональности протокола используется во вредоносных целях.

Файл `arppoison.pcapng` содержит пример отравления кеша ARP. На первый взгляд после его открытия трафик выглядит нормально. Однако, проследив за пакетами, вы увидите, что наша цель `172.16.0.107` открывает Google и выполняет поиск. В результате возникает довольно много трафика HTTP с примесью запросов DNS.

Мы знаем, что отравление кеша ARP происходит на уровне 2, поэтому при беглом просмотре пакетов на панели списка заметить нарушение может быть трудно. Чтобы получить преимущество, добавим на панель списка пакетов пару столбцов:

1. Выберите Edit > Preferences.
2. Щёлкните Columns в левой части окна Preferences.
3. Нажмите кнопку со знаком плюс (+), чтобы добавить новый столбец.
4. В области Title введите `Source MAC` и нажмите Enter.
5. В раскрывающемся списке Type выберите Hw src addr (resolved).
6. Щёлкните новую запись и перетащите её непосредственно за столбец Source.
7. Нажмите кнопку со знаком плюс (+), чтобы добавить ещё один столбец.
8. В области Title введите `Dest MAC` и нажмите Enter.
9. В раскрывающемся списке Type выберите Hw dest addr (resolved).
10. Щёлкните новую запись и перетащите её непосредственно за столбец Destination.
11. Нажмите OK, чтобы применить изменения.

После выполнения этих действий экран должен выглядеть, как на рис. 12-8. Теперь у вас есть два дополнительных столбца с MAC-адресами источника и назначения пакетов.

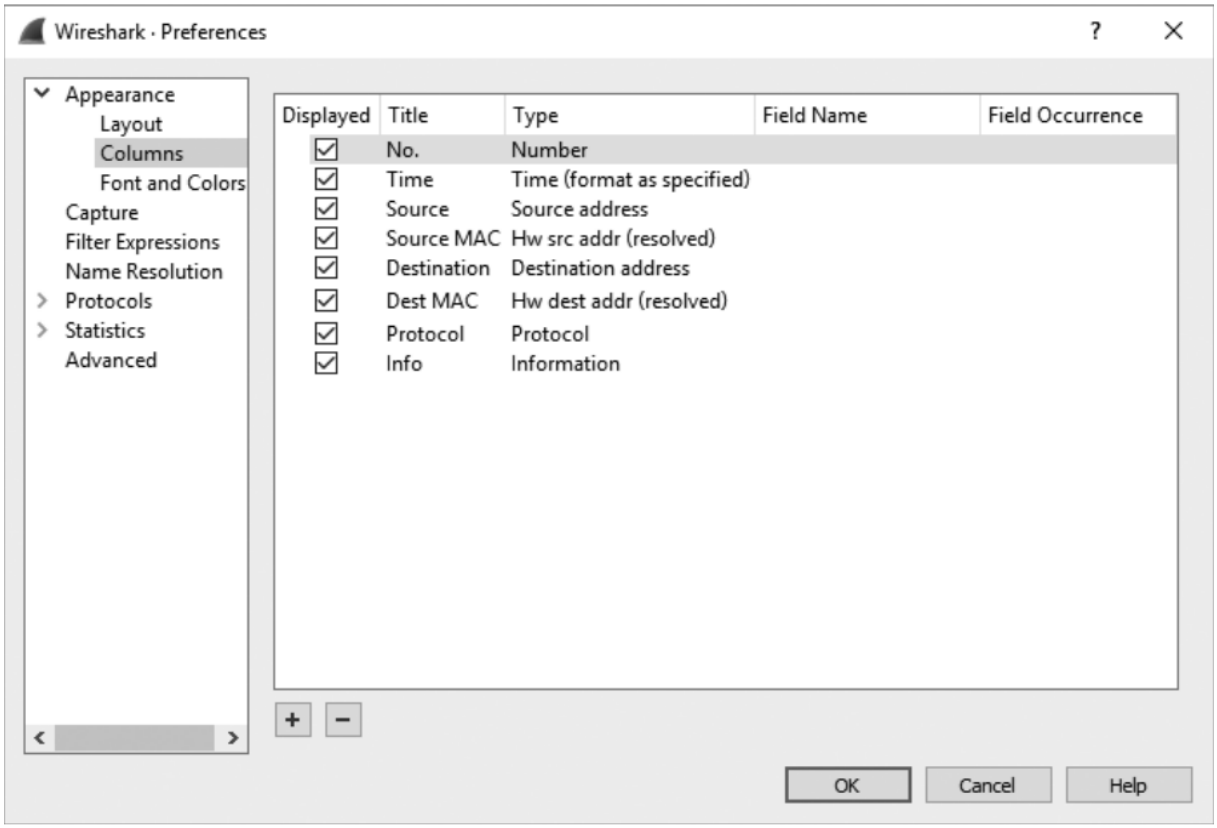


Рисунок 12-8. Экран настройки столбцов с новыми столбцами исходного и целевого аппаратных адресов

Если разрешение имён MAC всё ещё включено, у обменивающихся устройств должны отображаться MAC-адреса, указывающие на оборудование Dell и Cisco. Это очень важно запомнить: по мере прокрутки захвата мы увидим изменение в пакете 54, где между узлом Dell, нашей целью, и недавно появившимся узлом HP, злоумышленником, происходит необычный обмен ARP, показанный на рис. 12-9.

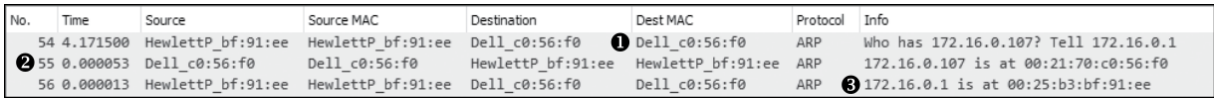


Рисунок 12-9. Странный трафик ARP между устройствами Dell и HP

Прежде чем двигаться дальше, обратите внимание на конечные точки этого обмена, перечисленные в таблице 12-3.

Таблица 12-3. Наблюдаемые конечные точки

Роль	Тип устройства	IP-адрес	MAC-адрес
Цель	Dell	`172.16.0.107`	`00:21:70:c0:56:f0`
Маршрутизатор	Cisco	`172.16.0.1`	`00:26:0b:31:07:33`
Злоумышленник	HP	Неизвестен	`00:25:b3:bf:91:ee`

Но что делает этот трафик странным? Вспомните из обсуждения ARP в главе 7, что существует два основных типа пакетов ARP: запрос и ответ. Запрос отправляется широковещательно всем узлам сети, чтобы найти машину с MAC-адресом, связанным с определённым IP-адресом. Затем ответившая машина отправляет ответ запросившему устройству одноадресным пакетом. Исходя из этого, в показанной на рис. 12-9 последовательности обмена можно выявить несколько странностей.

Во-первых, пакет 54 - запрос ARP от злоумышленника с MAC-адресом `00:25:b3:bf:91:ee`, отправленный одноадресно непосредственно цели с MAC-адресом `00:21:70:c0:56:f0` (метка 1). Такой запрос должен быть широковещательно отправлен всем узлам сети, но здесь он выделяет одну цель. Кроме того, хотя пакет отправлен злоумышленником и содержит его MAC-адрес в заголовке ARP, в нём указан IP-адрес маршрутизатора, а не самого злоумышленника.

За этим пакетом следует ответ цели злоумышленнику, содержащий сведения о её MAC-адресе (метка 2). Настоящая магия происходит в пакете 56: злоумышленник отправляет цели пакет с незапрошенным ответом ARP, сообщающим, что адрес `172.16.0.1` находится по его MAC-адресу `00:25:b3:bf:91:ee` (метка 3). Проблема в том, что настоящий MAC-адрес узла `172.16.0.1` не `00:25:b3:bf:91:ee`, а `00:26:0b:31:07:33`. Мы знаем это, поскольку ранее в захвате видели обмен маршрутизатора `172.16.0.1` с целью. Так как протокол ARP изначально небезопасен и принимает незапрошенные обновления своей таблицы, теперь цель станет отправлять злоумышленнику трафик, который должен был идти маршрутизатору.

> **Примечание.** Поскольку этот захват сделан на целевой машине, всей картины вы не видите. Чтобы атака сработала, злоумышленник должен отправить такую же последовательность пакетов маршрутизатору и заставить его считать злоумышленника целью. Чтобы увидеть эти пакеты, потребовался бы другой захват с маршрутизатора или устройства злоумышленника.

Когда и цель, и маршрутизатор обмануты, обмен между ними проходит через злоумышленника, как показано на рис. 12-10.

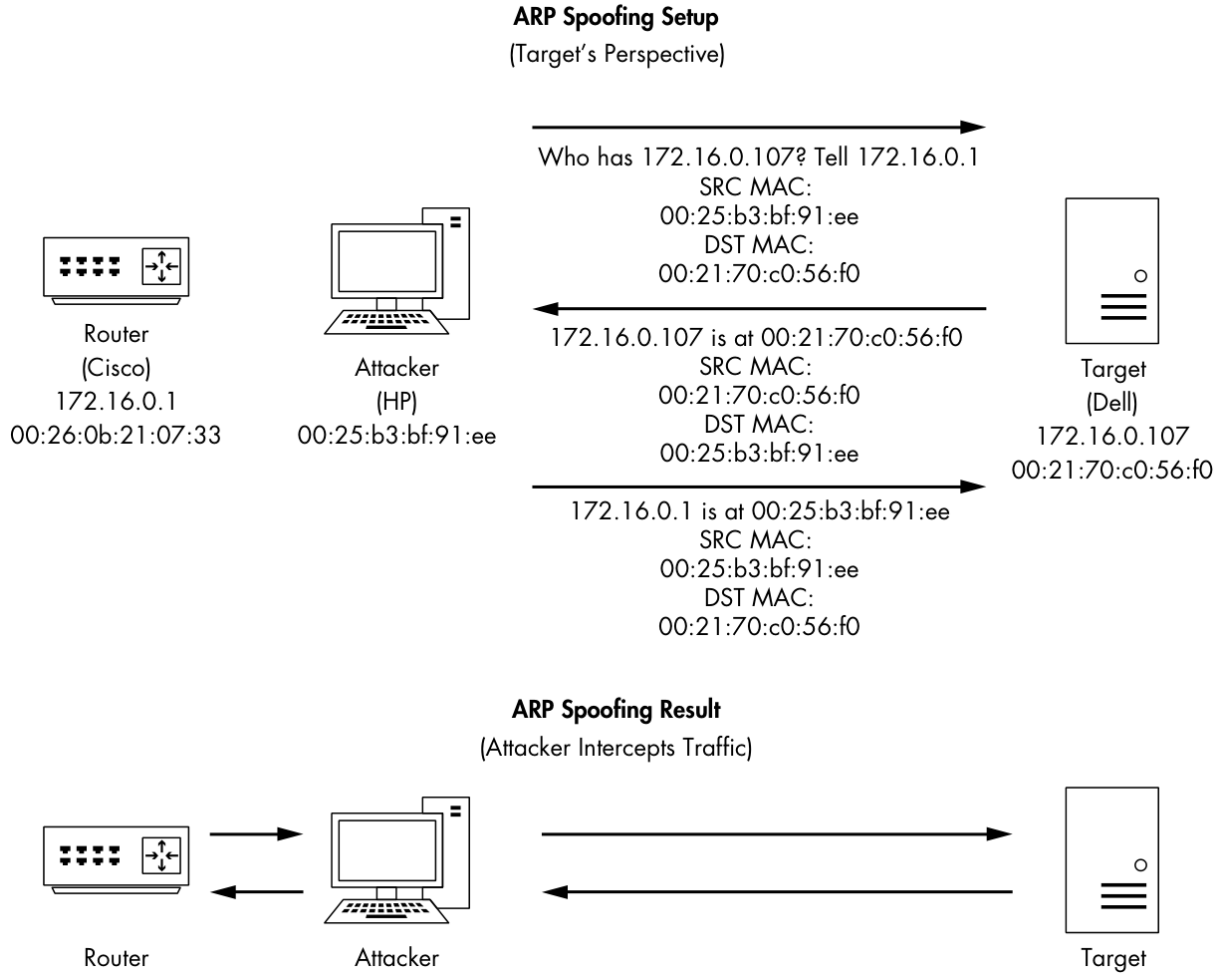


Рисунок 12-10. Отравление кеша ARP как атака MITM

Пакет 57 подтверждает успех атаки. Сравнив его с пакетом, отправленным до загадочного трафика ARP, например с пакетом 40 на рис. 12-11, вы увидите, что IP-адрес удалённого сервера Google остаётся тем же (метка 2), а целевой MAC-адрес изменился (метка 1). Это изменение сообщает, что теперь до поступления на маршрутизатор трафик проходит через злоумышленника.

Поскольку атака очень незаметна, обнаружить её чрезвычайно трудно. Обычно для этого нужна помощь IDS, специально настроенной на выявление такой атаки, или работающего на устройствах программного обеспечения, которое обнаруживает внезапные изменения записей таблицы ARP. Скорее всего, вы сами будете применять отравление кеша ARP для захвата пакетов в анализируемых сетях, поэтому важно знать и то, как этот метод может использоваться против вас.

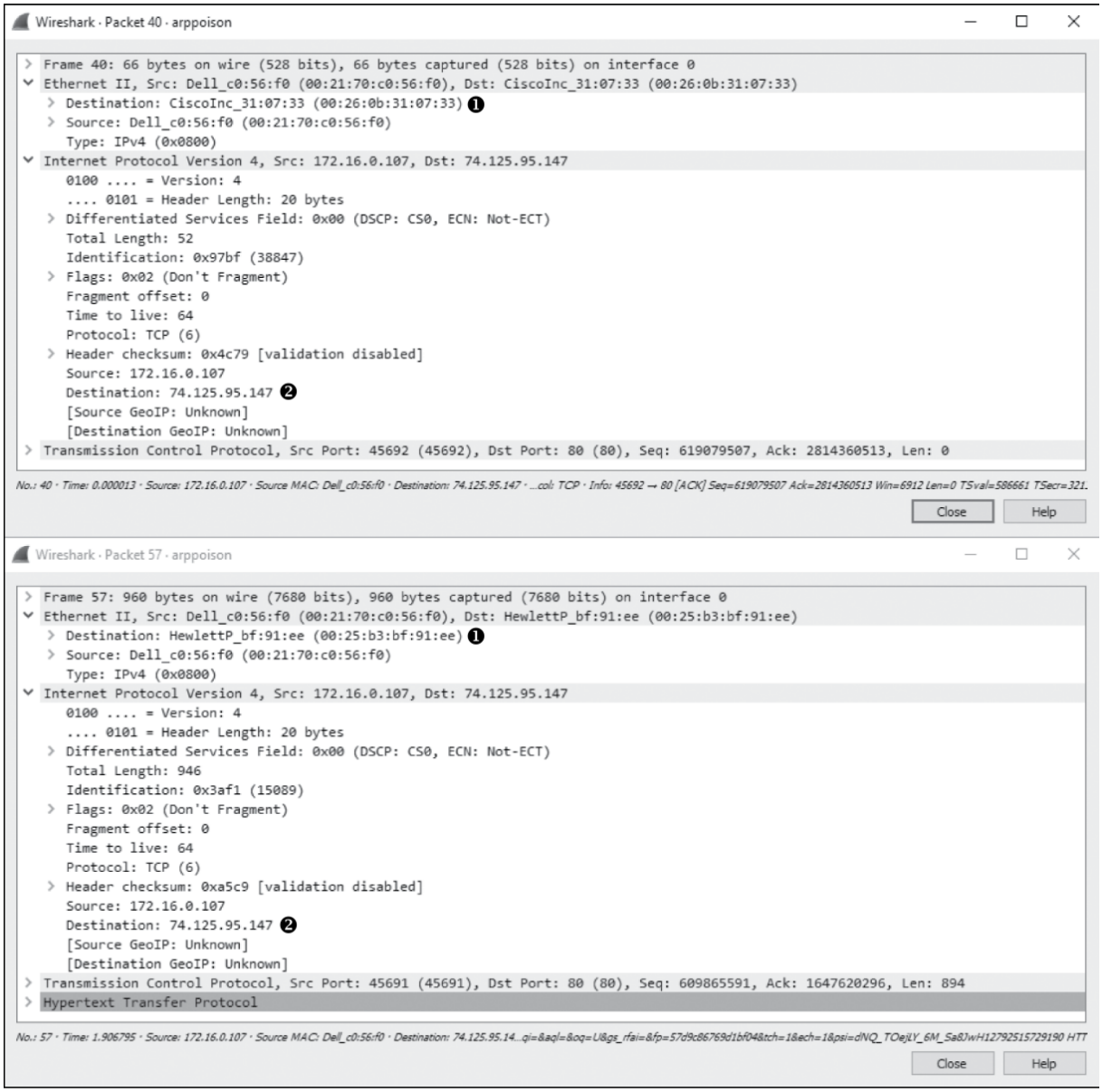


Рисунок 12-11. Изменение целевого MAC-адреса показывает, что атака завершилась успешно

Захват сеанса

Теперь, когда вы знаете, как отравление кеша ARP применяется во вредоносных целях, я хочу продемонстрировать использующий его метод - захват сеанса. При захвате сеанса злоумышленник компрометирует файл cookie сеанса HTTP, с которым мы скоро познакомимся, и применяет его, чтобы выдать себя за другого пользователя. Для этого злоумышленник может посредством отравления кеша ARP перехватить трафик цели и найти в нём нужные сведения cookie сеанса. Затем эти сведения позволяют ему обратиться к целевому веб-приложению от имени целевого пользователя.

Сценарий начинается с файла `sessionhijacking.pcapng`. Этот захват содержит трафик цели (`172.16.16.164`), обменивающейся с веб-приложением (`172.16.16.181`). Цель не знает, что стала жертвой злоумышленника (`172.16.16.154`), который активно перехватывает её обмен. Пакеты собраны с точки зрения веб-сервера - вероятно, именно такую картину увидит защитник, если против его серверной инфраструктуры будет применена атака с захватом сеанса.

> **Примечание.** Веб-приложение в этом примере называется Damn Vulnerable Web Application (DVWA). Оно намеренно уязвимо для множества видов атак и часто используется как учебное средство. Подробнее об атаках на веб-приложения и связанных с ними пакетах можно узнать на сайте http://www.dvwa.co.uk/.

Трафик захвата состоит главным образом из двух бесед. Первую - обмен цели с веб-сервером - можно выделить фильтром `ip.addr == 172.16.16.164 && ip.addr == 172.16.16.181`. Это обычный трафик просмотра веб-страниц, в котором нет ничего особенно примечательного. Нас прежде всего интересует значение cookie в запросах. Например, в запросе GET из пакета 14 файл cookie находится в окне подробностей пакета, как показано на рис. 12-12. В данном случае cookie идентифицирует сеанс значением PHPSESSID `ncobrqrb7fj2a2sinddtk567q4` (метка 1).

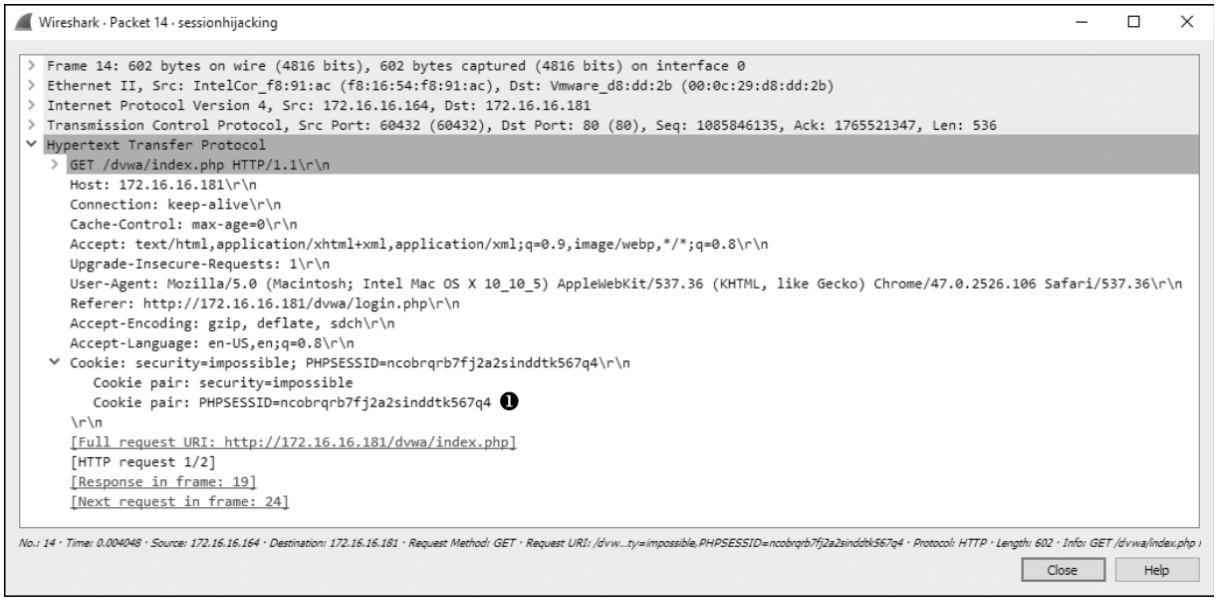


Рисунок 12-12. Просмотр файла cookie сеанса цели

Веб-сайты используют cookie для отслеживания сеансов отдельных узлов. При посещении сайта новому пользователю выдаётся идентификатор сеанса PHPSESSID, однозначно определяющий его. Для аутентификации многие приложения ждут, пока пользователь с данным идентификатором сеанса успешно пройдёт проверку, а затем создают в базе данных запись, признающую этот идентификатор представителем аутентифицированного сеанса. Любой пользователь с таким идентификатором сможет обратиться к приложению с этой аутентификацией. Разумеется, разработчики хотят верить, что конкретный идентификатор принадлежит только одному

пользователю, поскольку эти значения создаются уникальными. Однако такой способ обработки идентификаторов сеанса небезопасен: злоумышленник может похитить чужой идентификатор и выдать себя за его владельца. Существуют методы предотвращения захвата сеансов, но многие сайты, включая DVWA, по-прежнему уязвимы.

Цель не понимает, что её трафик перехватывается злоумышленником, а тот получил доступ к cookie сеанса, показанному на рис. 12-12. Злоумышленнику остаётся лишь связаться с веб-сервером, используя это значение cookie. Задачу можно решить определёнными типами прокси-серверов, но ещё проще воспользоваться расширением браузера, например Cookie Manager для Chrome. С его помощью злоумышленник указывает значение PHPSESSID, полученное из трафика цели, как показано на рис. 12-13.

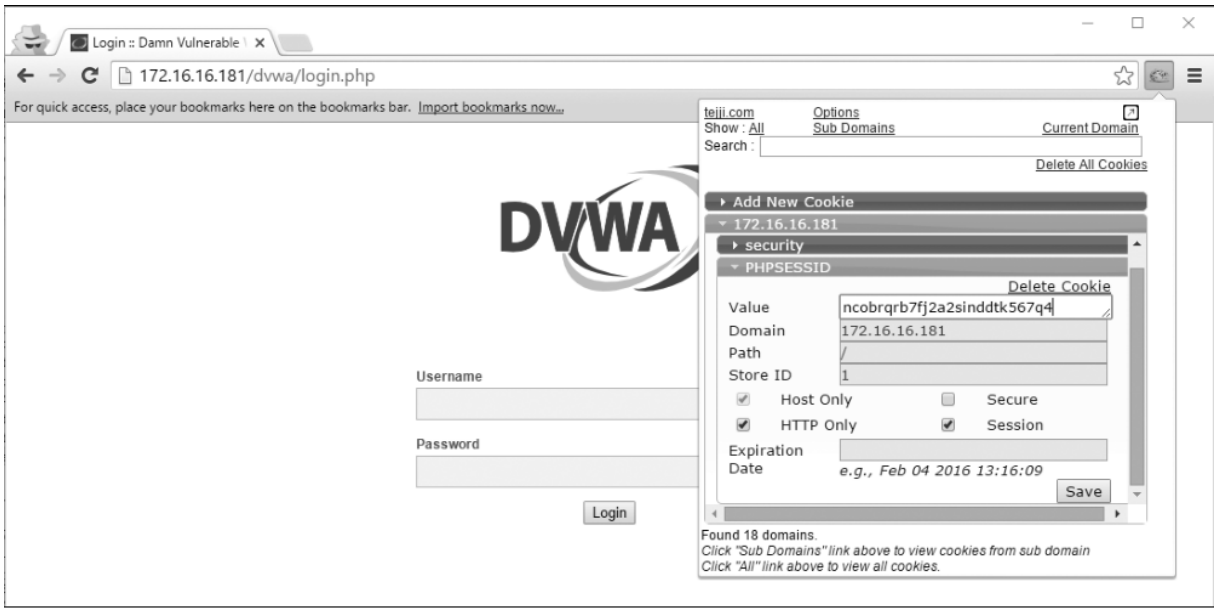


Рисунок 12-13. Использование расширения Cookie Manager для выдачи себя за цель

Если очистить ранее применённый к файлу захвата фильтр и начать прокрутку, в конце концов появится обмен IP-адреса злоумышленника с веб-сервером. Ограничить представление этим обменом можно с помощью фильтра `ip.addr == 172.16.16.154 && ip.addr == 172.16.16.181`.

Прежде чем исследовать его подробнее, добавим на панель списка пакетов столбец со значениями cookie. Если в предыдущем разделе об отравлении кеша ARP вы добавляли столбцы, сначала удалите их. Затем по инструкциям из того раздела добавьте новый пользовательский столбец на основе имени поля `http.cookie_pair`. Разместите его после поля Destination. Экран должен выглядеть, как на рис. 12-14.

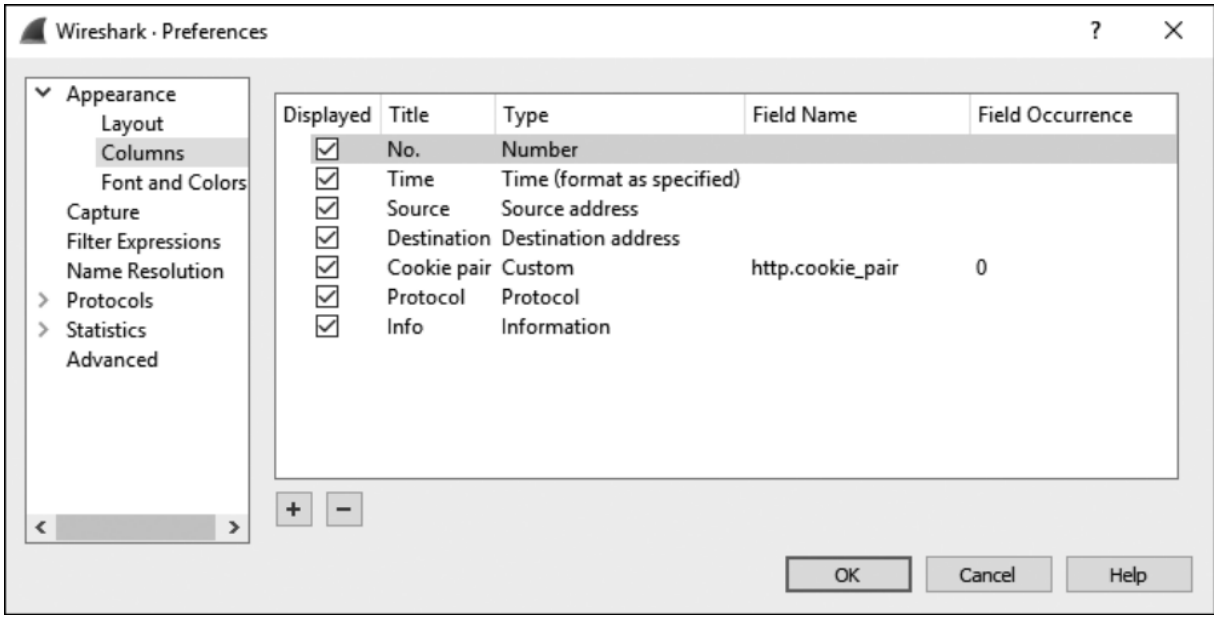


Рисунок 12-14. Настройка столбцов для исследования захвата сеанса

После настройки новых столбцов измените фильтр отображения так, чтобы он выводил только запросы HTTP, поскольку обмен TCP здесь бесполезен. Новый фильтр:

```
(ip.addr==172.16.16.154 && ip.addr==172.16.16.181) && (http.request.method || http.response.code)
```

Получившиеся пакеты показаны на рис. 12-15.

No.	Time	Source	Destination	Cookie pair	Protocol	Info
77	16.563004	172.16.16.154	172.16.16.181	security=low,PHPSESSID=lup70ajeuodkrhrvbmsjtgrd71	HTTP	❶ GET /dvwa/ HTTP/1.1
79	16.565584	172.16.16.181	172.16.16.154		HTTP	HTTP/1.1 302 Found ❷
80	16.570187	172.16.16.154	172.16.16.181	security=low,PHPSESSID=lup70ajeuodkrhrvbmsjtgrd71	HTTP	❸ GET /dvwa/login.php HTTP/1.1
81	16.575123	172.16.16.181	172.16.16.154		HTTP	HTTP/1.1 200 OK (text/html) ❹
115	60.040166	172.16.16.154	172.16.16.181	security=low,PHPSESSID=ncobrqrb7fj2a2sinddtk567q4	HTTP	❺ GET /dvwa/ HTTP/1.1
118	60.042241	172.16.16.181	172.16.16.154		HTTP	HTTP/1.1 200 OK (text/html) ❻
120	64.292056	172.16.16.154	172.16.16.181	security=low,PHPSESSID=ncobrqrb7fj2a2sinddtk567q4	HTTP	❼ GET /dvwa/setup.php HTTP/1.1
122	64.293401	172.16.16.181	172.16.16.154		HTTP	HTTP/1.1 200 OK (text/html) ❽

Рисунок 12-15. Злоумышленник выдаёт себя за целевого пользователя

Теперь перед вами обмен между злоумышленником и сервером. В первых четырёх пакетах злоумышленник запрашивает каталог `/dvwa/` (метка 1) и получает в ответ код 302 - обычный способ, которым веб-серверы перенаправляют посетителей на другие URL сервера. Здесь злоумышленник перенаправляется на страницу входа `/dvwa/login.php` (метка 2). Его машина запрашивает страницу входа (метка 3), и та успешно возвращается (метка 4). В обоих запросах используется идентификатор сеанса `lup70ajeuodkrhrvbmsjtgrd71`.

Затем следует новый запрос каталога `/dvwa/`, но на этот раз обратите внимание на другой идентификатор сеанса (метка 5). Теперь это `ncobrqrb7fj2a2sinddtk567q4` - то же значение, которое ранее использовала цель. Значит, злоумышленник изменил трафик так, чтобы применить похищенный идентификатор. Вместо перенаправления на страницу входа запрос получает код состояния HTTP 200, и страница доставляется в том виде, в котором её увидела бы аутентифицированная цель (метка 6). Злоумышленник переходит на другую страницу, `dvwa/setup.php`, с идентификатором цели (метка 7), и она также успешно возвращается (метка 8). Злоумышленник просматривает сайт DVWA так, словно аутентифицирован от имени цели, и при этом не знает ни её имени пользователя, ни пароля.

Это лишь один пример того, как злоумышленник может превратить анализ пакетов в наступательный инструмент. В целом можно обоснованно считать, что если злоумышленник видит связанные с вашим обменом пакеты, это способно привести к тем или иным вредоносным действиям. По этой причине специалисты по безопасности рекомендуют защищать передаваемые

данные шифрованием.

Вредоносные программы

Хотя совершенно законное программное обеспечение может применяться во вредоносных целях, термином *вредоносная программа* обычно называют код, написанный специально со злым умыслом. Вредоносные программы принимают множество форм, включая самораспространяющиеся черви и троянские программы, маскирующиеся под законное ПО. С точки зрения защитника сети большинство вредоносных программ остаются необнаруженными и неизвестными, пока их не удастся захватить и проанализировать. Такой анализ включает несколько этапов, в том числе исследование поведения шаблонов сетевого обмена программы. Иногда анализ выполняется в криминалистической лаборатории обратной разработки вредоносного ПО, но чаще он происходит в реальной среде, когда аналитик безопасности обнаруживает заражённое устройство в своей сети.

В этом разделе мы рассмотрим по пакетам несколько примеров настоящих вредоносных программ и их поведения.

Операция "Аврора"

В январе 2010 года было обнаружено, что в ходе операции "Аврора" использовалась до того момента неизвестная уязвимость Internet Explorer. Она позволила злоумышленникам получить удалённое управление целевыми машинами в Google и других компаниях.

Для выполнения вредоносного кода пользователю было достаточно посетить сайт с помощью уязвимой версии Internet Explorer. После этого злоумышленники немедленно получали доступ к его машине с теми же правами, что и у вошедшего в систему пользователя. Чтобы заманить цели, применялся целевой фишинг: злоумышленники отправляли письмо, призванное побудить получателей перейти по ссылке на вредоносный сайт.

В случае "Авроры" мы начинаем наблюдение сразу после того, как целевой пользователь щёлкает ссылку в целевом фишинговом письме. Получившиеся пакеты находятся в файле `aurora.pcapng`.

Захват начинается с трёхэтапного рукопожатия между целью (`192.168.100.206`) и злоумышленником (`192.168.100.202`). Первоначальное соединение идёт к порту 80, что позволяет предположить трафик HTTP. Это подтверждается в четвёртом пакете - HTTP-запросе GET для `/info` (метка 1), показанном на рис. 12-16.

Как видно на рис. 12-17, машина злоумышленника подтверждает получение запроса GET и в пакете 6 сообщает код ответа 302 (Moved Temporarily) (метка 1). Этот код состояния обычно применяется для перенаправления браузера на другую страницу, как и происходит здесь. Вместе с кодом 302 поле Location указывает расположение `/info?rFfWELUjLJHpP` (метка 2).

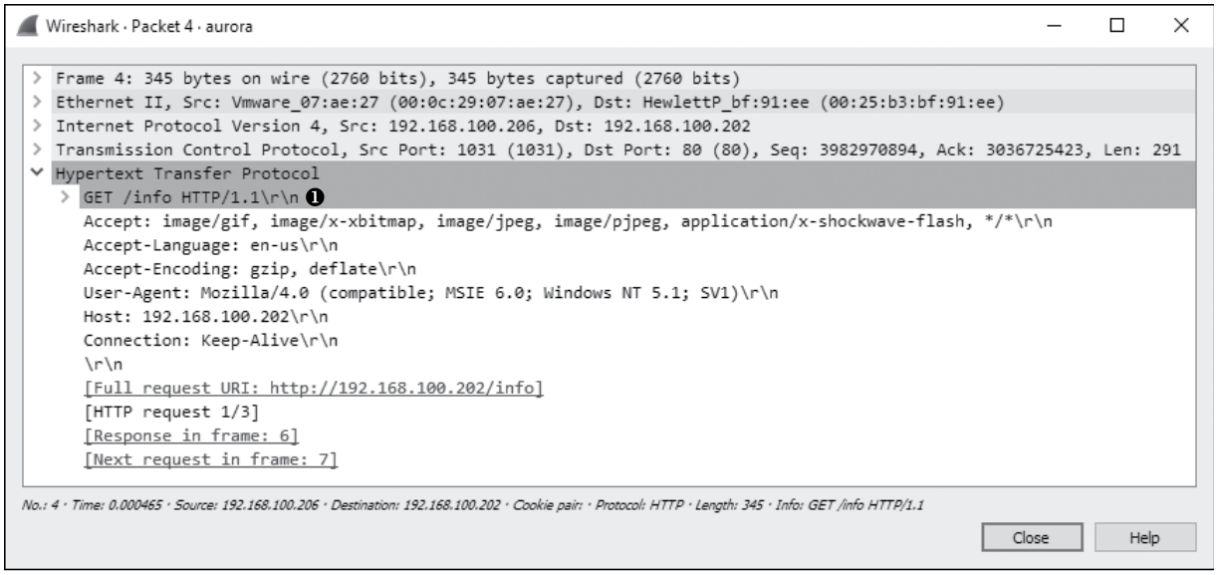


Рисунок 12-16. Цель выполняет запрос GET для `/info`

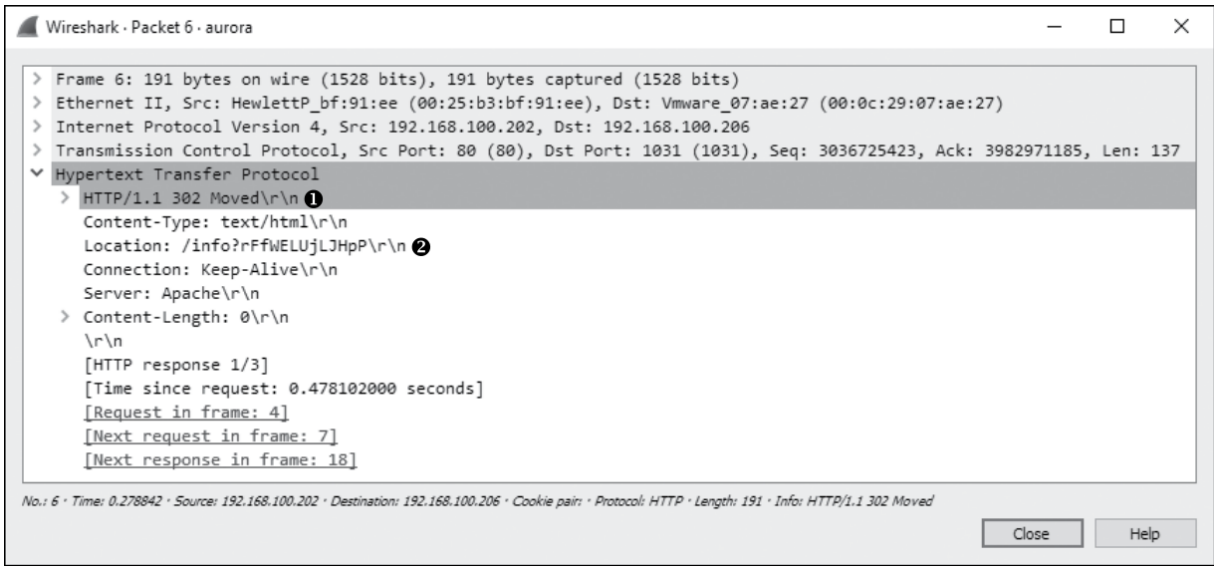


Рисунок 12-17. Этим пакетом браузер клиента перенаправляется

Получив пакет HTTP 302, клиент в пакете 7 создаёт ещё один запрос GET к URL `/info?rFfWELUjLJHpP`, на который в пакете 8 приходит ACK. Следующие несколько пакетов после ACK представляют передачу данных от злоумышленника к цели. Чтобы исследовать эти данные подробнее, щёлкните правой кнопкой мыши один из пакетов потока, например пакет 9, и выберите Follow > TCP Stream. В выводе потока видны начальный запрос GET, перенаправление 302 и второй запрос GET, как показано на рис. 12-18.

После этого начинают происходить действительно странные вещи. Злоумышленник отвечает на запрос GET весьма необычно выглядящим содержимым, первая часть которого показана на рис. 12-19.

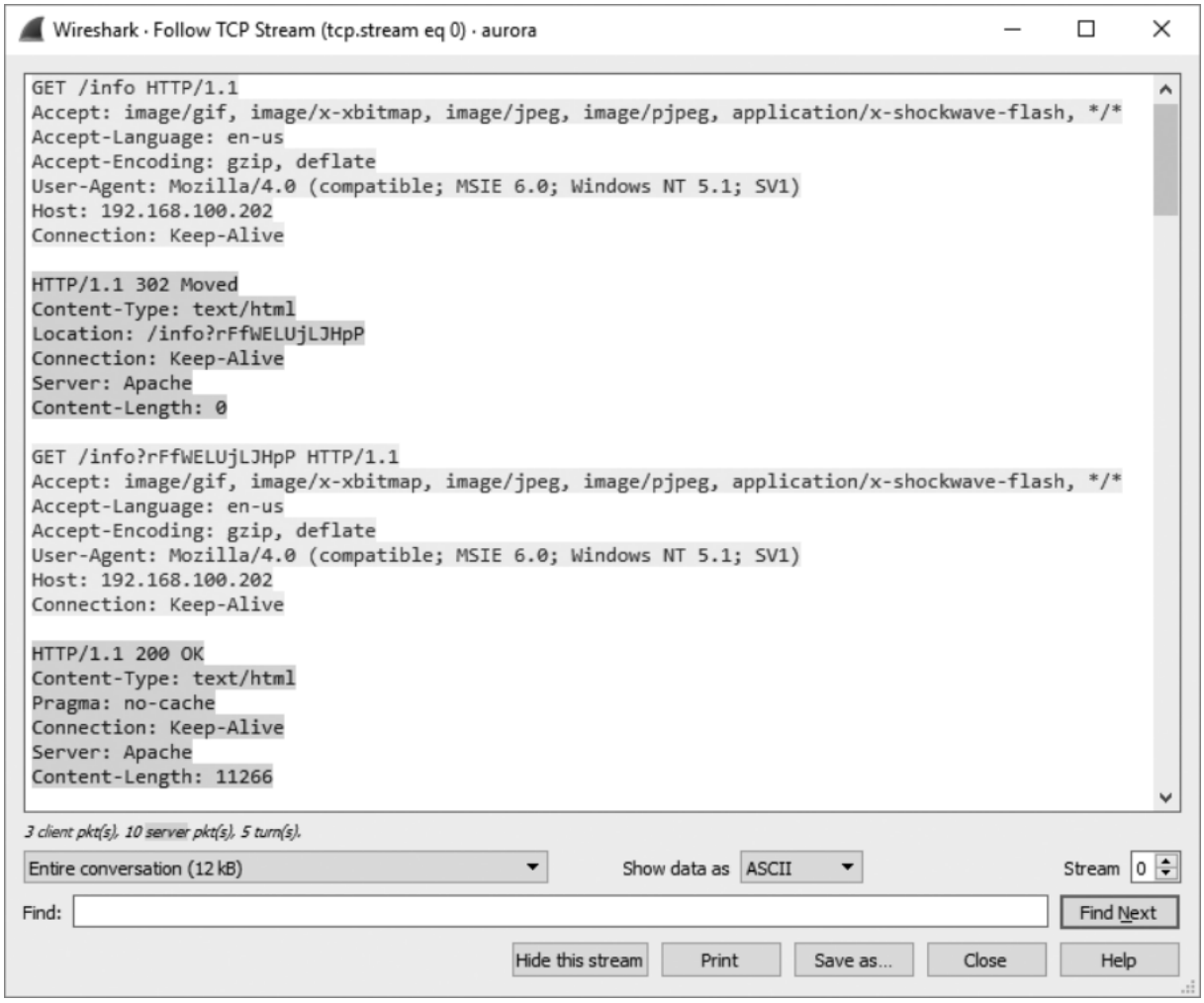


Рисунок 12-18. Поток данных, передаваемый клиенту

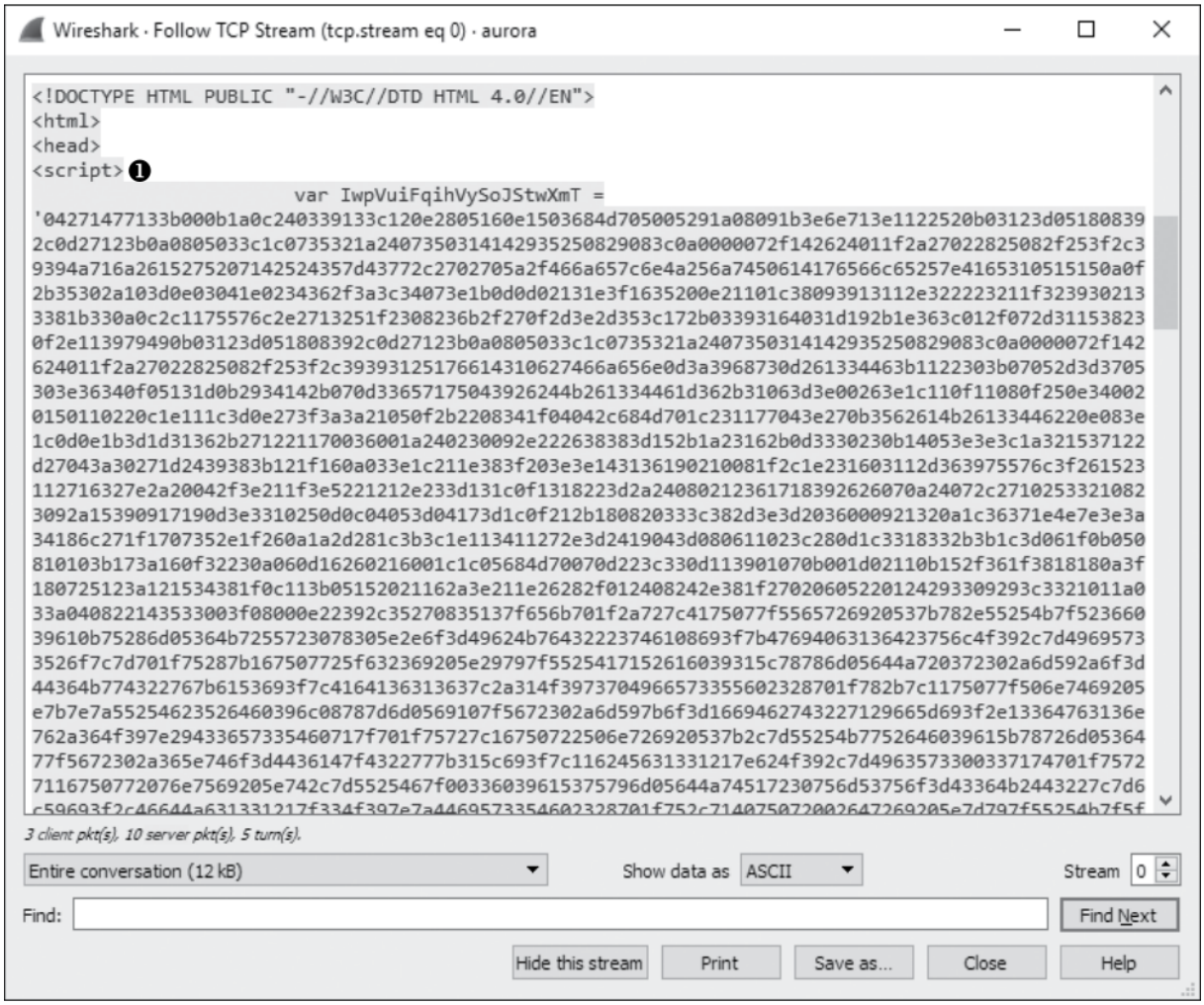


Рисунок 12-19. Это перемешанное содержимое внутри тега `<script>`*, по-видимому, закодировано*

Содержимое выглядит как последовательность случайных цифр и букв внутри тега `<script>` (метка 1). В HTML этот тег обозначает применение клиентского языка сценариев более высокого уровня, код которого выполняется на HTTP-клиенте. Обычно внутри тега находятся различные инструкции сценария. Однако бессмыслица в данном случае указывает, что содержимое могло быть закодировано для сокрытия от обнаружения. Поскольку мы знаем, что это трафик эксплуатации уязвимости, можно предположить: обфусцированный фрагмент текста содержит шестнадцатеричное заполнение и шелл-код, применяемые для эксплуатации уязвимой службы.

> **Примечание.** Обфускация сценариев - распространённый метод, с помощью которого вредоносные программы уклоняются от обнаружения и скрывают вредоносное содержимое. Деобфускация сценариев выходит за рамки книги, но вы разовьёте этот навык, если продолжите исследовать обмен вредоносных программ. Многие опытные аналитики способны мгновенно распознать вредоносный сценарий при беглом визуальном осмотре. Если хотите испытать себя, попробуйте вручную деобфусцировать сценарий из этого примера.

Во второй части содержимого от злоумышленника, показанной на рис. 12-20, наконец появляется читаемый текст. Даже без глубоких знаний программирования видно, что на основании нескольких переменных здесь, по-видимому, выполняется разбор строк. Это последний фрагмент текста перед закрывающим тегом `</script>`.

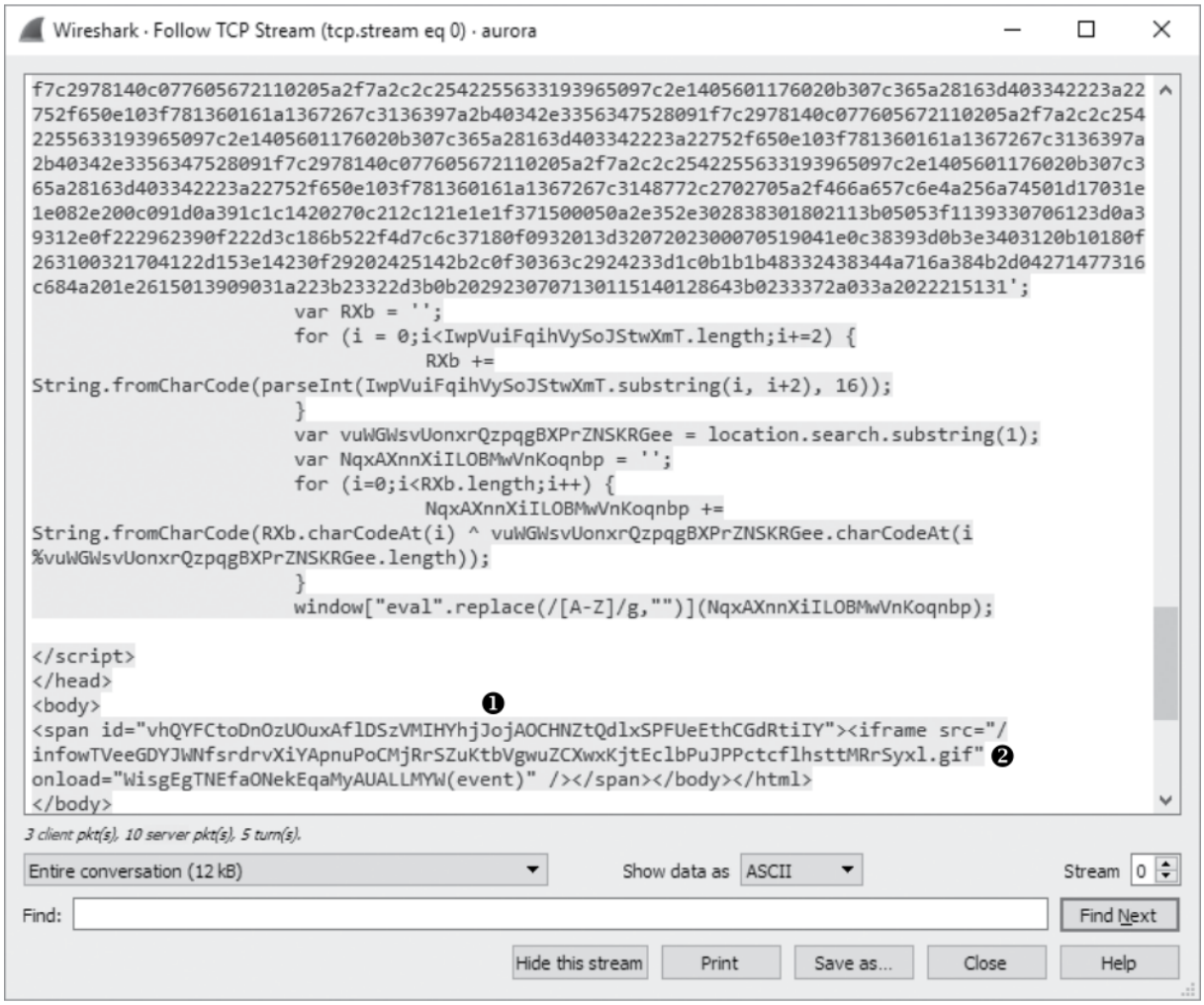


Рисунок 12-20. Эта часть содержимого, отправленного сервером, включает читаемый текст и подозрительный iframe

Последний раздел данных от злоумышленника клиенту, также показанный на рис. 12-20, состоит из двух частей. Первая - `<span id="vhQYFCtoDnOzUOuxAflDSzVMIHYhjJojAOCHNZtQdlxSPFUeEthCGdRtiIY">` (метка 1). Вторая, заключённая между тегами `<span></span>`, - `<iframe src="/infowTVeeGDYJWNfsrdrvXiYApnuPoCMjRrSZuKtbVgwuZCXwxKjtEclbPuJPPctcflhsttMRrSyxl.gif" onload="WisgEgTNEfaONekEqaMyAUALLMYW(event)" />` (метка 2). И снова это содержимое может указывать на вредоносную активность из-за подозрительно длинных случайных строк нечитаемого и, возможно, обфусцированного текста.

Часть кода внутри тега `<span>` представляет собой iframe - распространённый способ, которым злоумышленники встраивают в HTML-страницу дополнительное неожиданное содержимое. Тег `<iframe>` создаёт встроенный фрейм, способный остаться незамеченным пользователем. Здесь он ссылается на GIF-файл со странным именем. Как показано на рис. 12-21, увидев ссылку на этот файл, браузер цели запрашивает его пакетом GET 21 (метка 1), после чего GIF немедленно отправляется (метка 2).

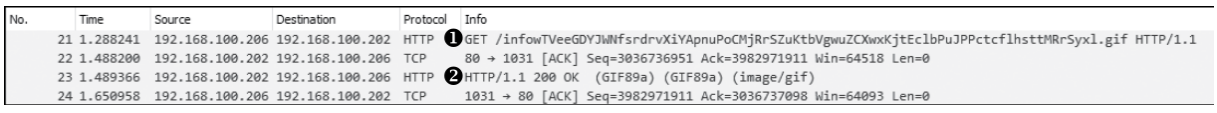


Рисунок 12-21. Цель запрашивает и загружает GIF, указанный в iframe

Самая странная часть захвата начинается с пакета 25, когда цель устанавливает обратное соединение со злоумышленником через порт 4321. Просмотр этого второго потока обмена на панели подробностей пакета даёт мало сведений, поэтому для более ясной картины снова откроем поток TCP. На рис. 12-22 показан вывод окна Follow TCP Stream.

В нём мы видим то, что должно немедленно вызвать тревогу: командную оболочку Windows (метка 1). Оболочка отправлена от цели серверу, а значит, попытка эксплуатации злоумышленника была успешной и полезная нагрузка доставлена. После запуска эксплойта клиент передал злоумышленнику командную оболочку. В захвате виден даже обмен злоумышленника с целью: он вводит команду `dir` (метка 2), чтобы просмотреть содержимое каталога на целевой машине (метка 3).

Если эксплойт скомпрометировал процесс, работающий от имени администратора, или мигрировал в такой процесс, злоумышленник может делать с целевой машиной практически всё, что пожелает. Единственным щелчком мыши и всего за несколько секунд цель передала злоумышленнику полный контроль над своим компьютером.

Подобные эксплойты обычно кодируются так, чтобы при передаче по линии их нельзя было распознать и сетевая IDS не смогла их обнаружить. Поэтому без предварительных сведений об эксплойте или хотя бы образца его кода может быть трудно точно установить, что происходит в целевой системе, не выполнив дальнейший анализ. К счастью, в этом захвате нам удалось выделить некоторые характерные признаки вредоносного кода: обфусцированный текст в тегах `<script>`, необычный iframe и командную оболочку, видимую как обычный текст.

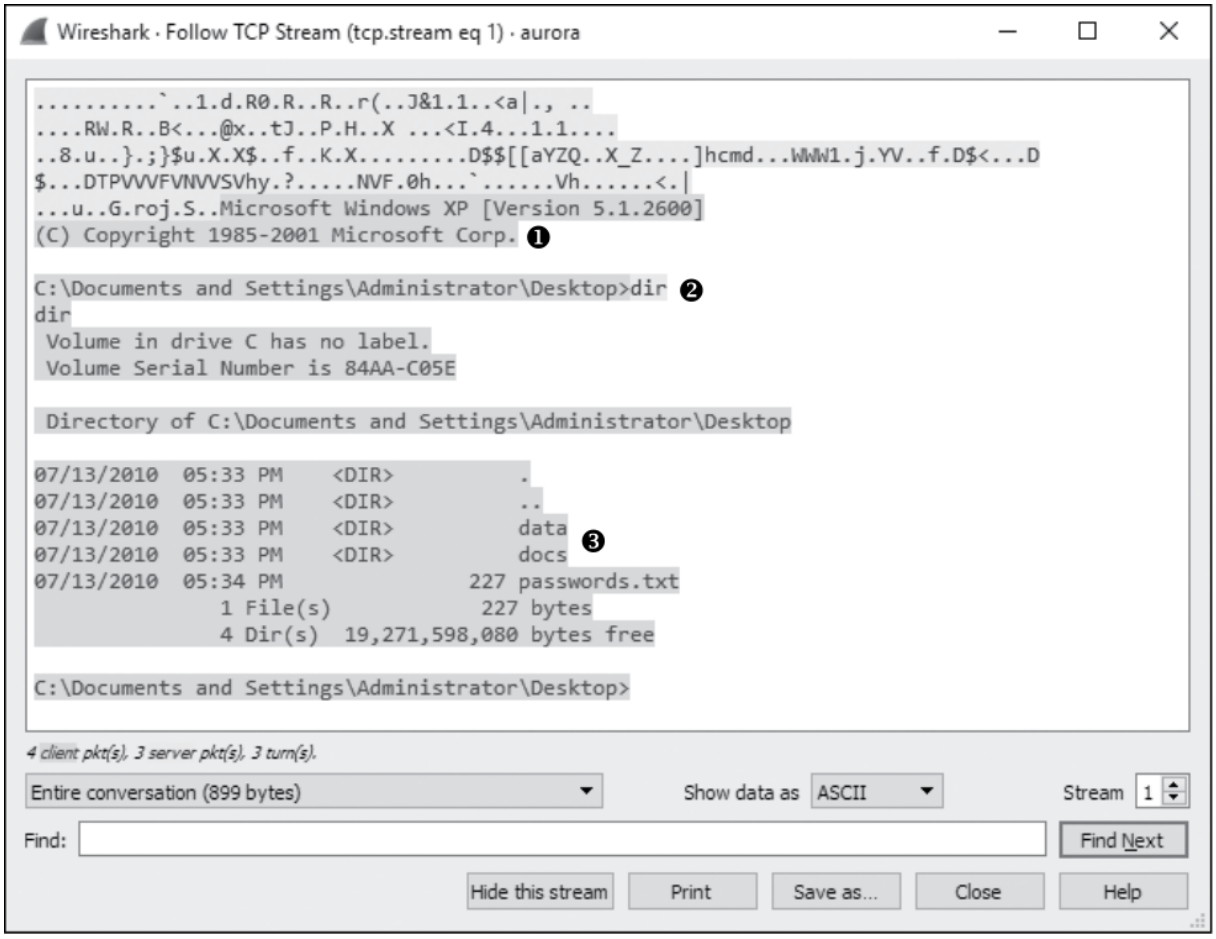


Рисунок 12-22. Злоумышленник взаимодействует с командной оболочкой через это соединение

Подведём итог работы эксплойта "Аврора" в данном случае:

• Цель получает от злоумышленника выглядящее законным письмо, щёлкает находящуюся в нём ссылку и отправляет запрос GET вредоносному сайту злоумышленника.
• Веб-сервер злоумышленника выдаёт цели перенаправление 302, после чего браузер цели автоматически отправляет запрос GET к новому URL.
• Веб-сервер злоумышленника передаёт клиенту веб-страницу с обфусцированным кодом JavaScript, который включает эксплойт уязвимости и iframe со ссылкой на запрашиваемое GIF-изображение.
• Когда страница отображается в браузере цели, ранее переданный код JavaScript деобфусцируется и выполняется на её машине, эксплуатируя уязвимость Internet Explorer.
• После эксплуатации уязвимости выполняется скрытая в обфусцированном коде полезная нагрузка, открывающая новый сеанс от цели к злоумышленнику через порт 4321.
• Полезная нагрузка создаёт командную оболочку и передаёт её злоумышленнику, чтобы тот мог с ней взаимодействовать.

С точки зрения защитника этот файл захвата можно использовать для создания сигнатуры IDS, способной помочь обнаруживать последующие случаи такой атаки. Например, можно отфильтровать необфусцированную часть захвата, такую как код в виде обычного текста в конце обфусцированного содержимого внутри тега `<script>`, или написать сигнатуру для всего трафика HTTP с перенаправлением 302 на сайт, в URL которого присутствует `info`. Чтобы такая сигнатура стала пригодна для производственной среды, её пришлось бы дополнительно настроить, но это хорошее начало. Разумеется, важно помнить, что сигнатуры можно обойти. Если злоумышленник просто изменит несколько наблюдавшихся здесь строк или доставит эксплойт другим способом, наши сигнатуры могут стать бесполезны. Так ведётся вечная борьба между нападающими и защитниками.

> **Примечание.** Умение создавать сигнатуры трафика на основе образцов вредоносного трафика - критически важный навык для человека, который пытается защитить сеть от неизвестных угроз. Анализ захватов, подобных описанному здесь, прекрасно развивает навыки написания таких сигнатур. Подробнее об обнаружении вторжений и сигнатурах атак можно узнать на сайте проекта Snort: http://www.snort.org/.

Троянская программа удалённого доступа

До сих пор мы исследовали события безопасности, заранее располагая некоторыми сведениями о происходящем. Это прекрасный способ узнать, как выглядят атаки, но он не слишком соответствует реальности. В большинстве настоящих сценариев люди, отвечающие за защиту сети, не исследуют каждый проходящий через неё пакет. Вместо этого они используют ту или иную IDS, которая на основании заранее определённой сигнатуры атаки предупреждает об аномалиях сетевого трафика, требующих дальнейшего изучения.

Следующий пример мы начнём с простого предупреждения, словно аналитик в реальной среде. В данном случае IDS создаёт следующее предупреждение:

```
[**] [1:132456789:2] CyberEYE RAT Session Establishment [**]
[Classification: A Network Trojan was detected] [Priority: 1]
07/18-12:45:04.656854 172.16.0.111:4433 -> 172.16.0.114:6641
TCP TTL:128 TOS:0x0 ID:6526 IpLen:20 DgmLen:54 DF
***AP*** Seq: 0x53BAEB5E  Ack: 0x18874922  Win: 0xFAF0  TcpLen: 20
```

Следующий шаг - просмотреть правило сигнатуры, вызвавшее это предупреждение:

```
alert tcp any any -> $HOME_NET any (msg:"CyberEYE RAT Session Establishment";
content:"|41 4E 41 42 49 4C 47 49 7C|"; classtype:trojan-activity;
sid:132456789; rev:2;)
```

Правило настроено выдавать предупреждение при обнаружении пакета, который входит во внутреннюю сеть из любой сети и содержит шестнадцатеричную последовательность `41 4E 41 42 49 4C 47 49 7C`. В удобочитаемом ASCII она преобразуется в `ANA BILGI`. При её обнаружении создаётся предупреждение о возможном присутствии троянской программы удалённого доступа (remote-access trojan, RAT) CyberEYE. RAT - вредоносная программа, которая незаметно работает на компьютере цели и предоставляет злоумышленнику средства удалённого доступа к её машине.

> **Примечание.** CyberEYE - некогда популярный инструмент турецкого происхождения, применявшийся для создания исполняемых файлов RAT и управления скомпрометированными узлами. По иронии правило Snort здесь срабатывает на строку `ANA BILGI`, что по-турецки означает "ОСНОВНЫЕ СВЕДЕНИЯ".

Теперь исследуем связанный с предупреждением трафик в файле `ratinfected.pcapng`. Обычно предупреждение Snort захватывает только единственный пакет, который его вызвал, но, к счастью, у нас есть вся последовательность обмена между узлами. Чтобы сразу перейти к сути, найдите упомянутую в правиле Snort шестнадцатеричную строку:

1. Выберите Edit > Find Packet или нажмите Ctrl-F.
2. В раскрывающемся меню выберите Hex Value.
3. Введите в текстовую область значение `41 4E 41 42 49 4C 47 49 7C`.
4. Нажмите Find.

Теперь, как показано на рис. 12-23, в области данных пакета 4 будет видно первое вхождение шестнадцатеричной строки (метка 1).

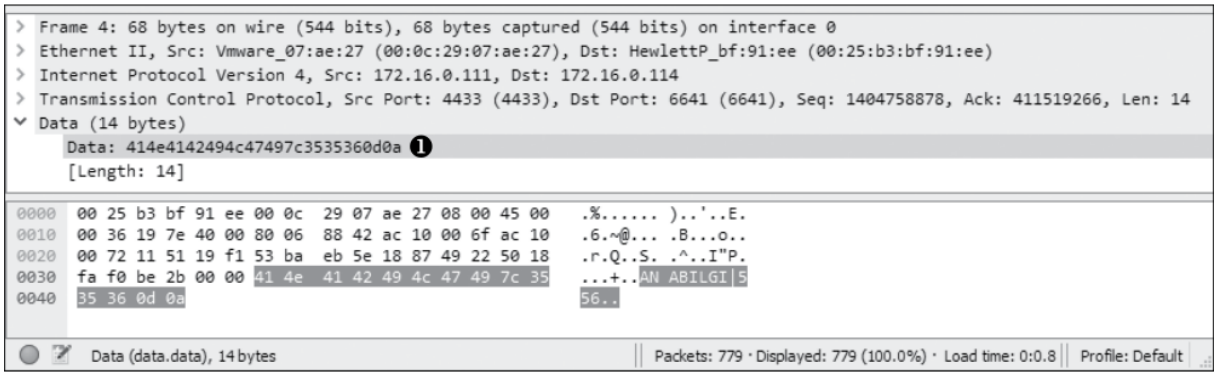


Рисунок 12-23. Здесь, в пакете 4, впервые появляется строка содержимого из предупреждения Snort

Если ещё несколько раз нажать Find, строка обнаружится также в пакетах 5, 10, 32, 156, 280, 405, 531 и 652. Хотя весь обмен файла захвата происходит между злоумышленником (`172.16.0.111`) и целью (`172.16.0.114`), некоторые вхождения строки, по-видимому, относятся к разным беседам. Пакеты 4 и 5 обмениваются через порты 4433 и 6641, а большинство остальных вхождений возникает между портом 4433 и другими случайно выбранными эфемерными портами. Наличие нескольких бесед можно подтвердить на вкладке TCP окна Conversations, показанной на рис. 12-24.

Разные беседы файла захвата можно визуально разделить, раскрасив их:

1. В строке фильтра над панелью списка пакетов введите `(tcp.flags.syn == 1) && (tcp.flags.ack == 0)`, затем нажмите Enter. Будет выбран начальный пакет SYN каждой беседы в трафике.
2. Щёлкните первый пакет правой кнопкой мыши и выберите Colorize Conversation.
3. Выберите TCP, затем цвет.
4. Повторите процесс для остальных пакетов SYN, назначая каждой беседе другой цвет.
5. Закончив, нажмите X, чтобы удалить фильтр.

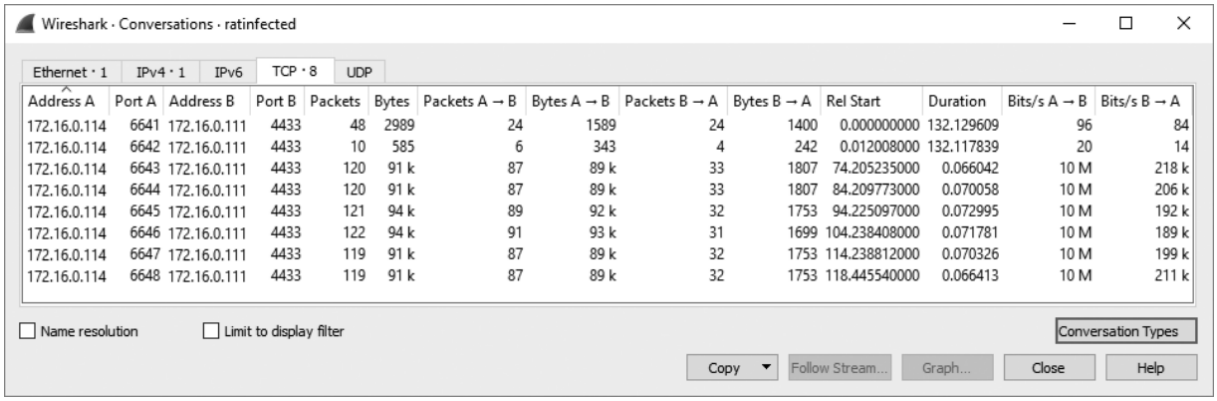


Рисунок 12-24. Между злоумышленником и целью существуют три отдельные беседы

Раскрасив беседы, можно очистить фильтр и увидеть, как они связаны друг с другом, что поможет проследить процесс обмена между двумя узлами. Первая беседа через порты 6641/4433 открывает обмен между ними, поэтому начать лучше с неё. Щёлкните правой кнопкой мыши любой пакет беседы и выберите Follow TCP Stream, чтобы увидеть переданные данные, как показано на рис. 12-25.

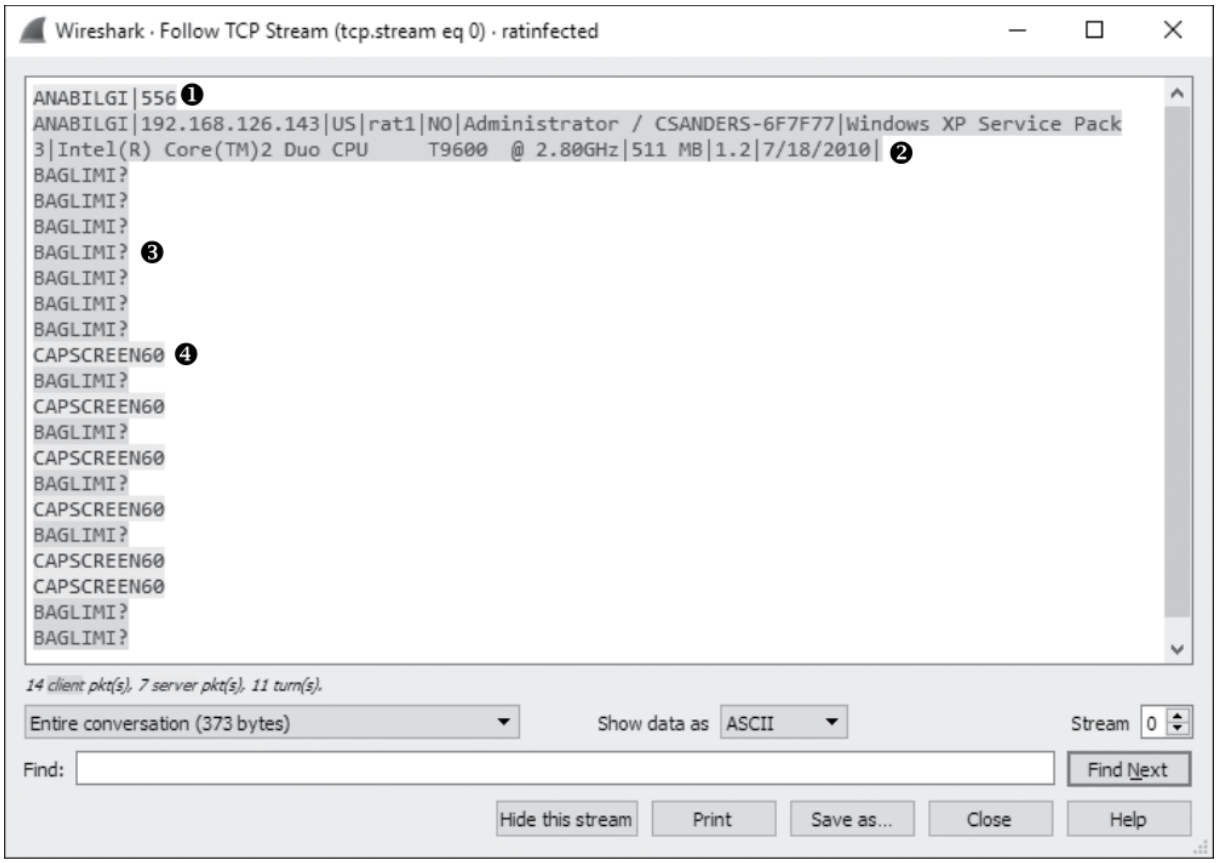


Рисунок 12-25. Первая беседа даёт интересные результаты

Сразу видно, что строка `ANABILGI|556` отправляется злоумышленником цели (метка 1). В ответ цель возвращает основные сведения о системе, включая имя компьютера `CSANDERS-6F7F77` и используемую операционную систему `Windows XP Service Pack 3` (метка 2), а затем начинает многократно передавать злоумышленнику строку `BAGLIMI?` (метка 3). Единственный ответ злоумышленника - строка `CAPSCREEN60` (метка 4), которая появляется шесть раз.

Возвращаемая злоумышленником строка `CAPSCREEN60` выглядит интересно, поэтому проследим, куда она ведёт. Очистите все фильтры отображения и найдите текстовую строку `CAPSCREEN60` в

пакетах через диалог поиска, выбрав вариант String и убедившись, что в области поиска выбран вариант Packet bytes.

Первое вхождение строки обнаруживается в пакете 27. Примечательно, что сразу после её отправки злоумышленником клиенту последний подтверждает получение пакета, а в пакете 29 начинается новая беседа. Благодаря применённым ранее правилам раскраски заметить её начало будет проще.

Если открыть вывод TCP-потока этой новой беседы, показанный на рис. 12-26, мы увидим знакомую строку `ANABILGI|12`, затем `SH|556` и, наконец, `CAPSCREEN|C:\WINDOWS\jpgevhook.dat|84972` (метка 1). Обратите внимание на путь к файлу после строки `CAPSCREEN`, за которым следует нечитаемый текст. Самое интересное, что перед ним находится строка `JFIF` (метка 2), которая, как подскажет быстрый поиск в Google, часто встречается в начале файлов JPG.

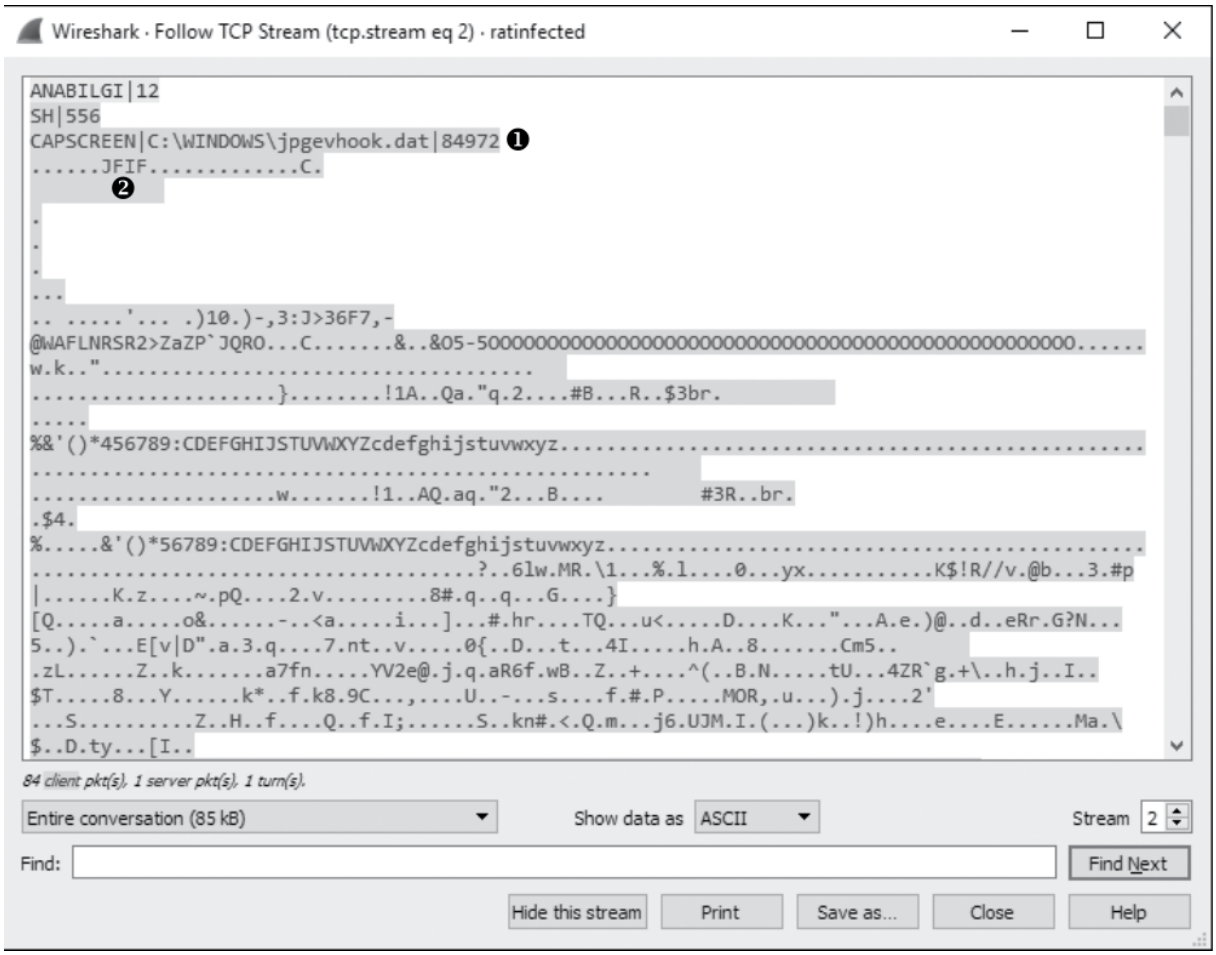


Рисунок 12-26. По-видимому, злоумышленник инициирует запрос файла JPG

Теперь можно обоснованно заключить, что злоумышленник инициировал запрос на передачу этого изображения JPG. Но ещё важнее то, что в трафике начинает проявляться структура команд. По-видимому, `CAPSCREEN` - команда злоумышленника, запускающая передачу JPG. В действительности при каждой отправке команды `CAPSCREEN` результат одинаков. Чтобы проверить это, просмотрите TCP-поток каждой беседы, в которой присутствует `CAPSCREEN`, либо воспользуйтесь функцией графика ввода-вывода Wireshark:

1. Выберите Statistics > IO Graph.
2. Нажмите кнопку со знаком плюс (+), чтобы добавить пять линий.
3. Вставьте соответственно фильтры `tcp.stream eq 2`, `tcp.stream eq 3`, `tcp.stream eq 4`, `tcp.stream eq 5` и `tcp.stream eq 6` в поле Display Filter пяти новых линий. Также присвойте

каждой линии имя.
4. Измените масштаб оси Y каждой записи на Bytes/s.
5. Нажмите кнопки Graph 1, Graph 2, Graph 3, Graph 4 и Graph 5, чтобы включить точки данных для заданных фильтров.

Получившийся график показан на рис. 12-27.

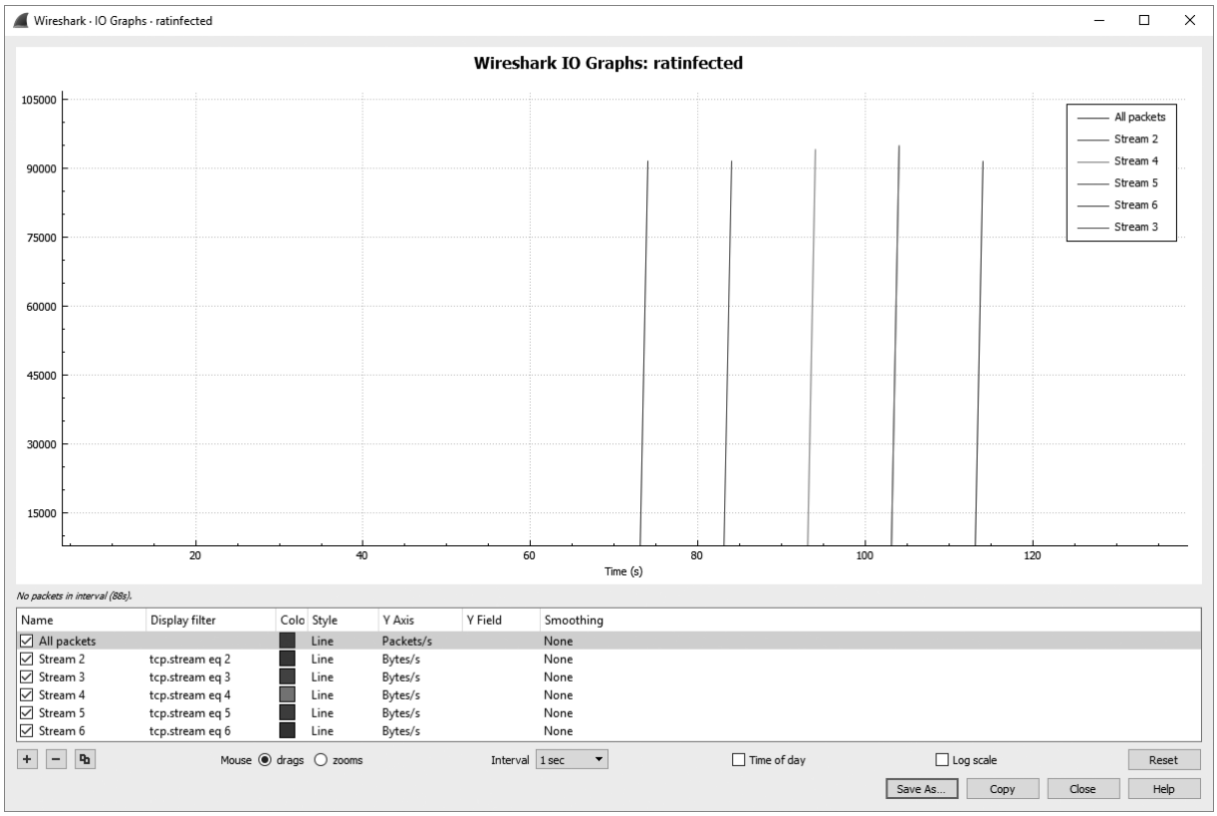


Рисунок 12-27. График показывает повторение похожей активности

Судя по графику, каждая беседа содержит примерно одинаковый объём данных и длится примерно одинаковое время. Теперь можно заключить, что активность повторяется несколько раз.

Возможно, у вас уже появились предположения о содержимом передаваемого изображения JPG. Попробуем просмотреть один из этих файлов. Чтобы извлечь данные JPG из Wireshark, выполните следующие действия:

1. Сначала проследите TCP-поток соответствующих пакетов, как при работе с рис. 12-25. Хорошо подойдёт пакет 29.
2. Нужно изолировать обмен так, чтобы видеть только поток данных от цели к злоумышленнику. Для этого нажмите стрелку рядом с раскрывающимся списком Entire Conversation (85033 bytes). Обязательно выберите нужное направление трафика: `172.16.0.114:6643 --> 172.16.0.111:4433 (85 kB)`.
3. В раскрывающемся списке Show data as выберите RAW.
4. Сохраните данные кнопкой Save As, обязательно указав для файла расширение `.jpg`.

Если попытаться открыть изображение сейчас, вы можете удивиться, что это не удаётся. Осталось выполнить ещё один шаг. В отличие от сценария главы 10, где файл был чисто извлечён из трафика FTP, здесь к данным добавлено лишнее содержимое. Первые две строки TCP-потока на самом деле являются частью последовательности команд вредоносной программы, а не данных JPG, как показано на рис. 12-28. При сохранении потока были записаны и эти посторонние данные. Поэтому средство просмотра, которое ищет заголовок JPG, встречает не соответствующее ожиданиям содержимое и не может открыть изображение.

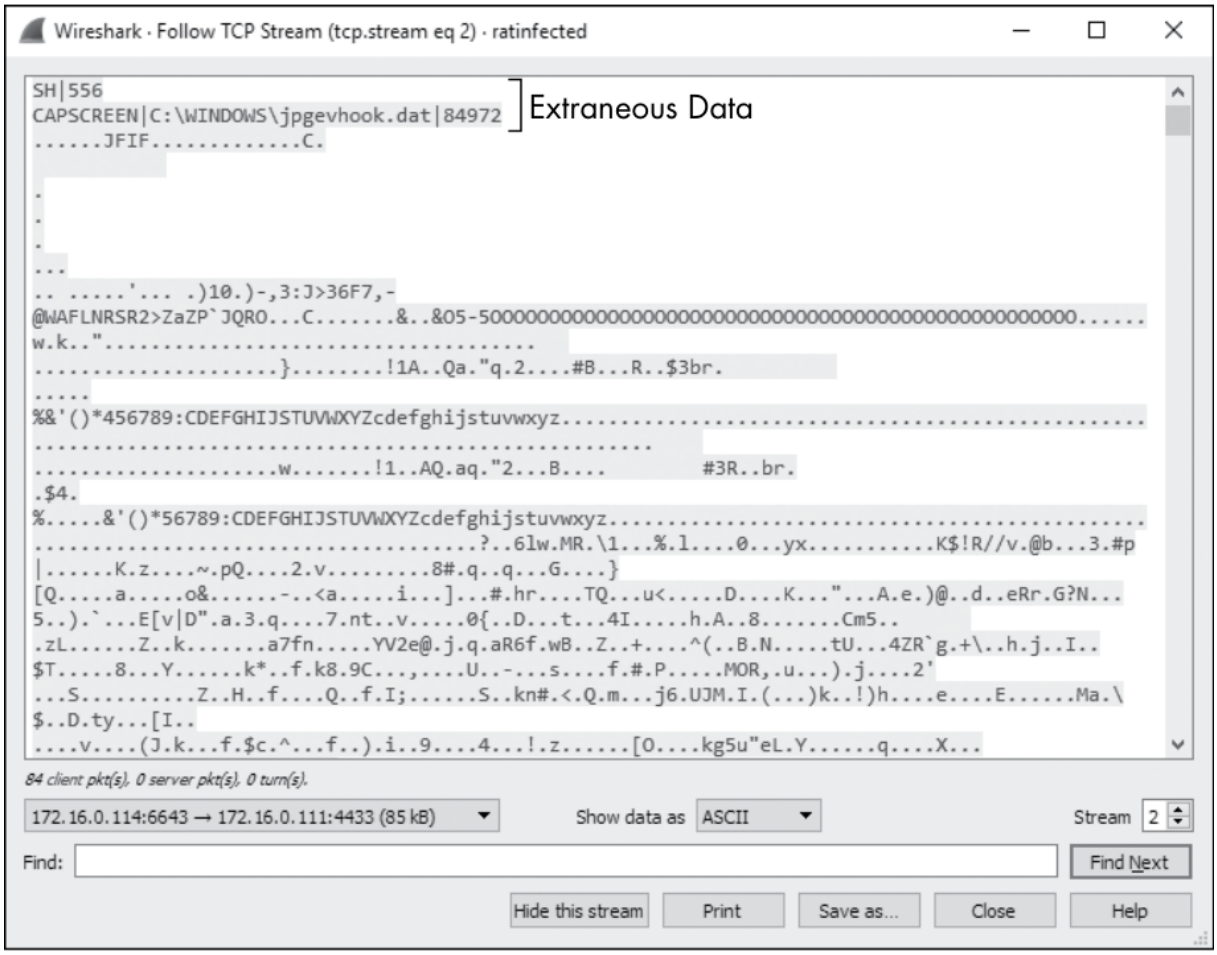


Рисунок 12-28. Добавленные вредоносной программой посторонние данные не позволяют правильно открыть файл

Исправить проблему нетрудно: потребуется лишь немного изменить файл шестнадцатеричным редактором. Этот процесс называется вырезанием файла (file carving). Чтобы вырезать файл из экспортированных данных, выполните следующие действия:

1. Просматривая TCP Stream, показанный на рис. 12-28, нажмите кнопку Save as. Выберите запоминающееся имя и сохраните файл в месте, к которому вскоре сможете снова обратиться.
2. Загрузите и установите WinHex с сайта https://www.x-ways.net/winhex/.
3. Запустите WinHex и откройте только что сохранённый из Wireshark файл.
4. Мышью выделите все посторонние данные в начале файла. Это должно быть всё до байтов `FF D8 FF E0`, обозначающих начало нового файла JPG, но не сами эти байты. Выделяемые байты показаны на рис. 12-29.

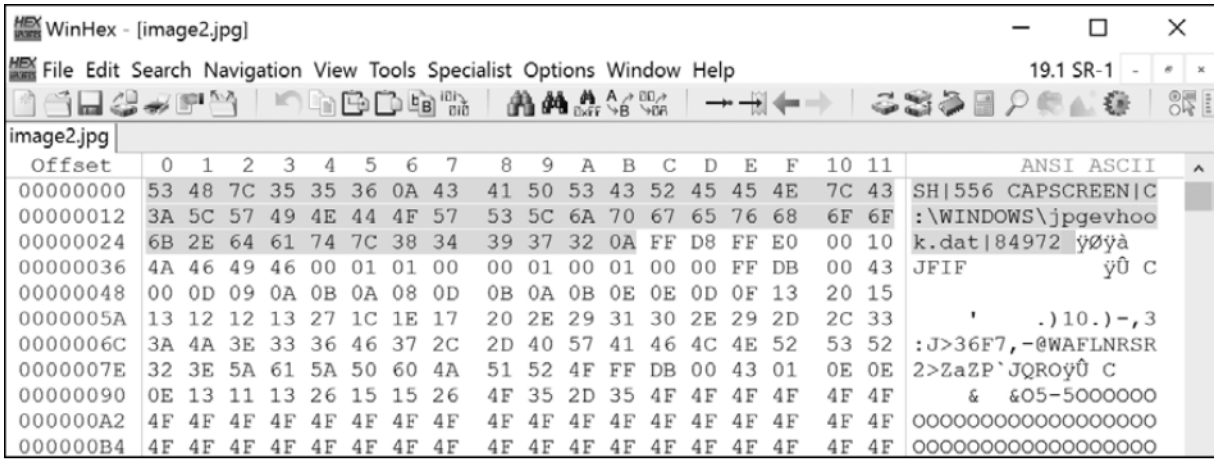


Рисунок 12-29. Удаление посторонних байтов из файла JPG

5. Нажмите Delete, чтобы удалить выбранные данные.
6. Нажмите кнопку Save на главной панели инструментов WinHex, чтобы сохранить изменения.

> **Примечание.** Для этой задачи в Windows мне нравится WinHex, но подойдёт любой знакомый вам шестнадцатеричный редактор.

После удаления ненужных байтов данных файл должен открыться. Станет ясно, что троянская программа делает снимки экрана рабочего стола цели и передаёт их злоумышленнику, как показано на рис. 12-30. После завершения этих последовательностей обмен заканчивается обычной процедурой закрытия TCP.

Этот сценарий прекрасно демонстрирует ход мыслей аналитика вторжений при исследовании трафика на основании предупреждения IDS:

• Исследовать предупреждение и создавшую его сигнатуру.
• Подтвердить, что соответствие сигнатуре присутствует в трафике в правильном контексте.
• Исследовать трафик и выяснить, что злоумышленник сделал со скомпрометированной машиной.
• Начать сдерживание проблемы, прежде чем с целевой машины утекут дополнительные конфиденциальные сведения.

Рисунок 12-30. Передаваемый JPG является снимком экрана целевого компьютера

Набор эксплойтов и программа-вымогатель

В последнем сценарии мы рассмотрим ещё одно расследование, начинающееся с предупреждения IDS. Мы исследуем пакеты, которые в реальном времени создаёт заражённая система, а затем попытаемся проследить источник компрометации. В примере используется настоящая вредоносная программа, которой вполне может быть заражено устройство в вашей сети.

История начинается с предупреждения IDS от Snort в консоли Sguil, показанного на рис. 12-31. Sguil - инструмент для управления, просмотра и исследования предупреждений IDS от одного или нескольких датчиков. Его пользовательский интерфейс нельзя назвать самым привлекательным, но инструмент существует уже давно и очень популярен среди аналитиков безопасности.

В Sguil доступно много сведений об этом предупреждении. Верхнее окно (метка 1) показывает его сводку: время создания, IP-адреса и порты источника и назначения, протокол и сообщение о событии, сформированное совпавшей сигнатурой IDS. Здесь `192.168.122.145`, локальная доверенная система, обменивается с неизвестной внешней системой `184.170.149.44` через порт 80, обычно связанный с трафиком HTTP. Внешняя система считается враждебной, поскольку появилась в связи с сигнатурой, указывающей на вредоносный обмен, и о ней почти ничего не известно. Совпавшая с трафиком сигнатура характерна для контрольного обмена семейства вредоносных программ CryptoWall, а значит, на доверенной системе, вероятно, установлена разновидность этой программы.

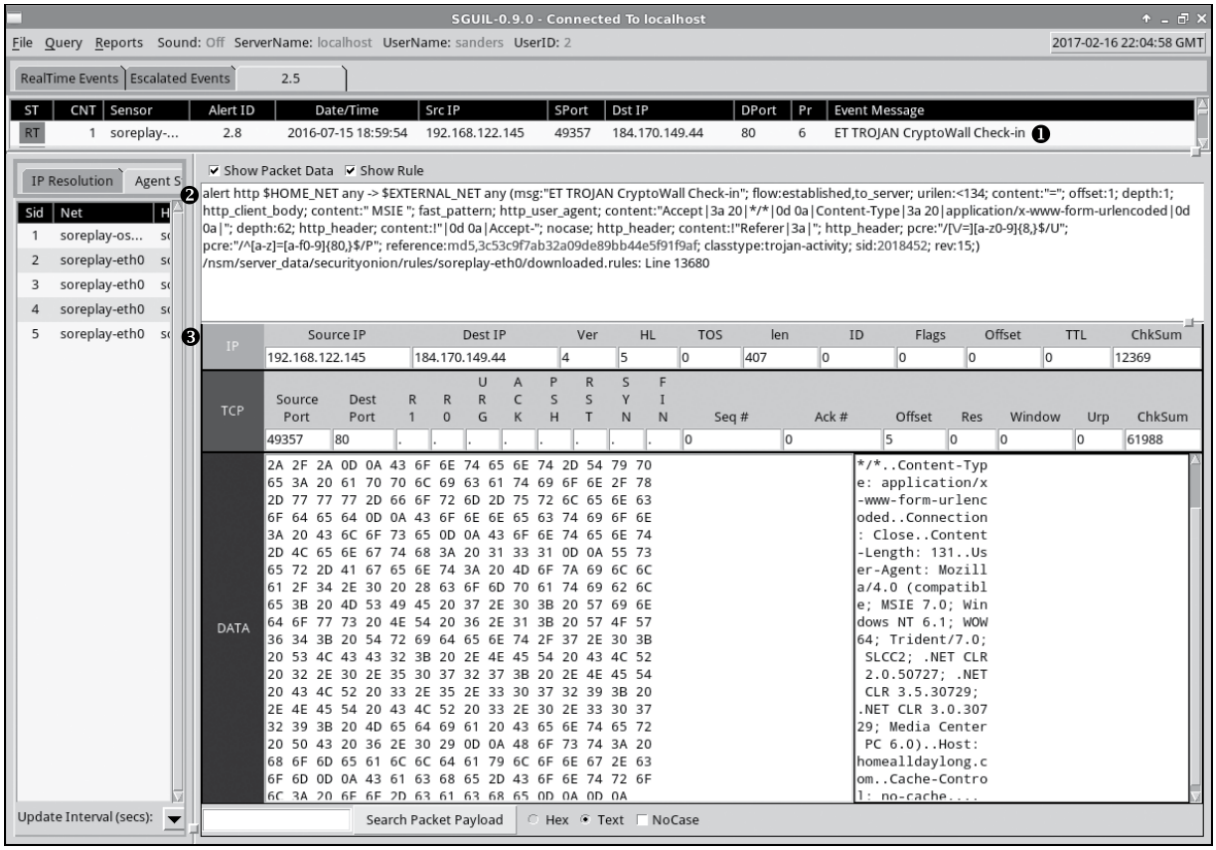


Рисунок 12-31. Это предупреждение IDS указывает на заражение CryptoWall 4

Консоль Sguil предоставляет синтаксис совпавшего правила (метка 2) и данные отдельного пакета, соответствующего правилу (метка 3). Обратите внимание, что сведения о пакете разделены на заголовок протокола и данные, как при представлении пакета в Wireshark. К сожалению, Sguil показывает только один совпавший пакет, а нам нужно копнуть глубже. Следующий шаг - исследовать в Wireshark связанный с предупреждением трафик, попытаться подтвердить его и понять происходящее. Этот трафик содержится в файле `cryptowall4_c2.pcapng`.

Захват содержит обмен, происходивший примерно во время предупреждения, и не отличается особой сложностью. Первая беседа занимает пакеты с 1 по 16; её легко просмотреть, проследив TCP-поток, как показано на рис. 12-32. В начале захвата локальная система открывает TCP-соединение с враждебным узлом через порт 80 и выполняет запрос POST к URL http://homealldaylong.com/76N1Lm.php?x4tk7t4jo6 (метка 1), содержащий небольшой объём буквенно-цифровых данных (метка 2). Враждебный узел отвечает буквенно-цифровой строкой (метка 4) и кодом HTTP 200 OK (метка 3), после чего соединение штатно завершается.

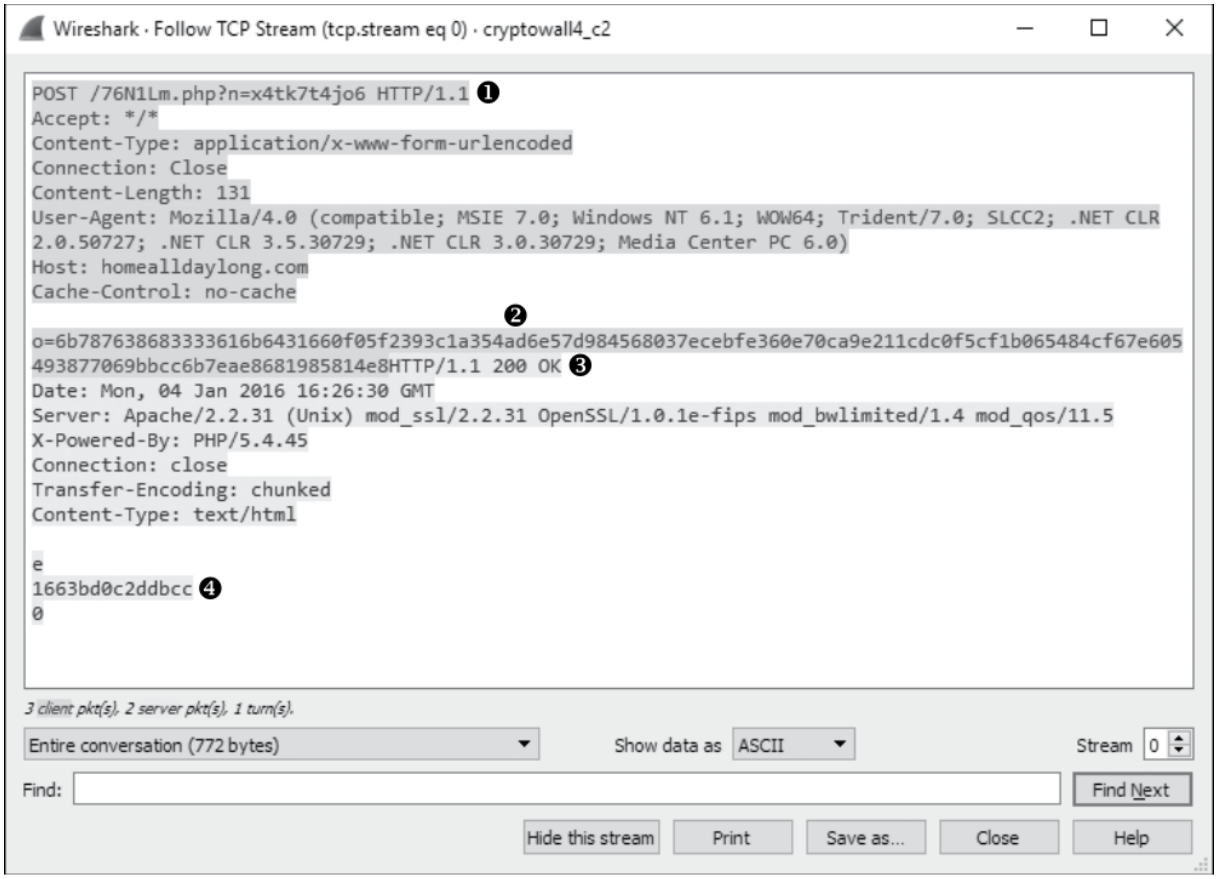
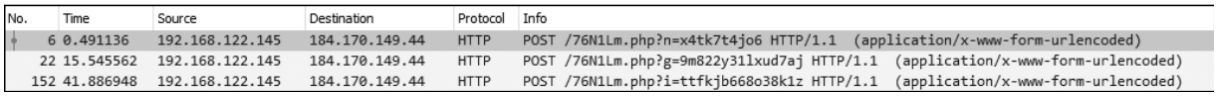


Рисунок 12-32. Между узлами через HTTP передаётся небольшой объём данных

Если просмотреть остальную часть файла захвата, окажется, что между этими узлами повторяется та же последовательность с разными объёмами передаваемых данных. Примените фильтр `http.request.method == "POST"`, чтобы увидеть три разных соединения с похожей структурой URL, как на рис. 12-33.



No.	Time	Source	Destination	Protocol	Info
6	0.491136	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?n=x4tk7t4jo6 HTTP/1.1 (application/x-www-form-urlencoded)
22	15.545562	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?g=9m822y31lxud7aj HTTP/1.1 (application/x-www-form-urlencoded)
152	41.886948	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?i=ttfkjb668o38k1z HTTP/1.1 (application/x-www-form-urlencoded)

Рисунок 12-33. Структура URL показывает разные данные, передаваемые одной и той же странице

Часть `76N1Lm.php`, то есть веб-страница, остаётся прежней, а остальное содержимое - параметр и данные, передаваемые странице, - меняется. Повторяющаяся последовательность обмена вместе со структурой запросов соответствует поведению управления и контроля (command and control, C2) вредоносной программы и сигнатуре, создавшей предупреждение. Поэтому локальная система, скорее всего, действительно заражена CryptoWall, как и указывает сигнатура. Дополнительно проверить это можно по похожему образцу на популярной исследовательской странице CryptoWall Tracker: https://www.cryptowalltracker.org/cryptowall-4.html#networktraffic.

> **Примечание.** Расшифровка данных, которыми доверенная и враждебная системы обмениваются во время последовательности C2, была бы слишком сложной для этой книги. Если вам интересно, подробнее о процессе можно прочитать здесь: https://www.cryptowalltracker.org/communication-protocol.html.

Теперь, подтвердив обмен C2 вредоносной программы, следует устранить проблему и заняться заражённой машиной. Это особенно важно в случае такого вредоносного ПО, как CryptoLocker: оно пытается зашифровать данные пользователя и предоставляет ключ расшифровки, только если тот

заплатит крупный выкуп, откуда и происходит термин *программа-вымогатель*. Действия по устранению проблемы выходят за рамки книги, но в реальном сценарии именно их аналитик безопасности выполнил бы далее.

Обычно затем возникает вопрос: как доверенная машина вообще была заражена? Если это удастся определить, можно обнаружить другие устройства, заражённые похожим образом иными вредоносными программами, либо разработать механизмы защиты или обнаружения для предотвращения заражений в будущем.

Пакеты предупреждения показывали только активную последовательность C2 после заражения. В сетях, где ведутся мониторинг безопасности и непрерывный захват пакетов, многие датчики настроены хранить данные пакетов ещё несколько часов или дней для криминалистических расследований. В конце концов, не каждая организация способна реагировать на предупреждения в ту же секунду. Временное хранение позволяет просмотреть данные пакетов доверенного узла непосредственно перед началом уже увиденной последовательности C2. Эти пакеты находятся в файле `ek_to_cryptowall4.pcapng`.

Беглая прокрутка захвата показывает, что исследовать предстоит гораздо больше пакетов, но все они относятся к HTTP. Поскольку мы уже знаем, как работает HTTP, сразу перейдём к сути и ограничим видимые пакеты запросами при помощи фильтра отображения `http.request`. Он выводит 11 HTTP-запросов от доверенного узла, как показано на рис. 12-34.

No.	Time	Source	Destination	Protocol	Info
4	0.534405	192.168.122.145	113.20.11.49	HTTP	GET /index.php/services HTTP/1.1
35	5.265859	192.168.122.145	45.32.238.202	HTTP	GET /contrary/1653873/quite-someone-visitor-nonsense-tonight-sweet-await-gigantic-dance-third HTTP/1.1
39	6.109508	192.168.122.145	45.32.238.202	HTTP	GET /occasional/bXJkeHFlYXhmaA HTTP/1.1
123	9.126714	192.168.122.145	45.32.238.202	HTTP	GET /goodness/1854996/earnest-fantastic-thorough-weave-grotesque-forth-awaken-fountain HTTP/1.1
130	14.020289	192.168.122.145	45.32.238.202	HTTP	GET /observation/enVjZ2dtcnpz HTTP/1.1
441	30.245463	192.168.122.145	213.186.33.18	HTTP	POST /VOEHSQ.php?v=x4tk7t4jo6 HTTP/1.1 (application/x-www-form-urlencoded)
456	41.772768	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?n=x4tk7t4jo6 HTTP/1.1 (application/x-www-form-urlencoded)
472	45.628284	192.168.122.145	213.186.33.18	HTTP	POST /VOEHSQ.php?w=9m822y31lxud7aj HTTP/1.1 (application/x-www-form-urlencoded)
487	56.827194	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?g=9m822y31lxud7aj HTTP/1.1 (application/x-www-form-urlencoded)
619	71.971402	192.168.122.145	213.186.33.18	HTTP	POST /VOEHSQ.php?h=ttfkjb668o38k1z HTTP/1.1 (application/x-www-form-urlencoded)
634	83.168580	192.168.122.145	184.170.149.44	HTTP	POST /76N1Lm.php?i=ttfkjb668o38k1z HTTP/1.1 (application/x-www-form-urlencoded)

Рисунок 12-34. От доверенного узла поступает 11 HTTP-запросов

Первый запрос идёт от доверенного узла `192.168.122.145` к неизвестному внешнему узлу `113.20.11.49`. Исследование части HTTP этого пакета на рис. 12-35 сообщает, что пользователь запросил страницу http://www.sydneygroup.com.au/index.php/services/ (метка 1), перейдя из результатов поиска Bing по запросу `sydneygroup.com.au` (метка 2). Пока всё выглядит нормально.

Затем доверенный узел в пакетах 35, 39, 123 и 130 выполняет четыре запроса к другому неизвестному внешнему узлу `45.32.238.202`. Как вы видели в предыдущих примерах, при просмотре веб-страницы браузер часто получает содержимое с дополнительных узлов, если встроенные материалы или реклама хранятся на сторонних серверах. Само по себе это не вызывает тревоги, хотя домен в этих запросах выглядит несколько случайным и подозрительным.

Рисунок 12-35. HTTP-запрос к неизвестному внешнему узлу

Интересные события начинаются с запроса GET в пакете 39. Проследив TCP-поток этого обмена на рис. 12-36, вы заметите запрос файла с именем `bXJkeHFlYXhmaA` (метка 1). Имя выглядит немного странно, и у файла даже нет расширения.

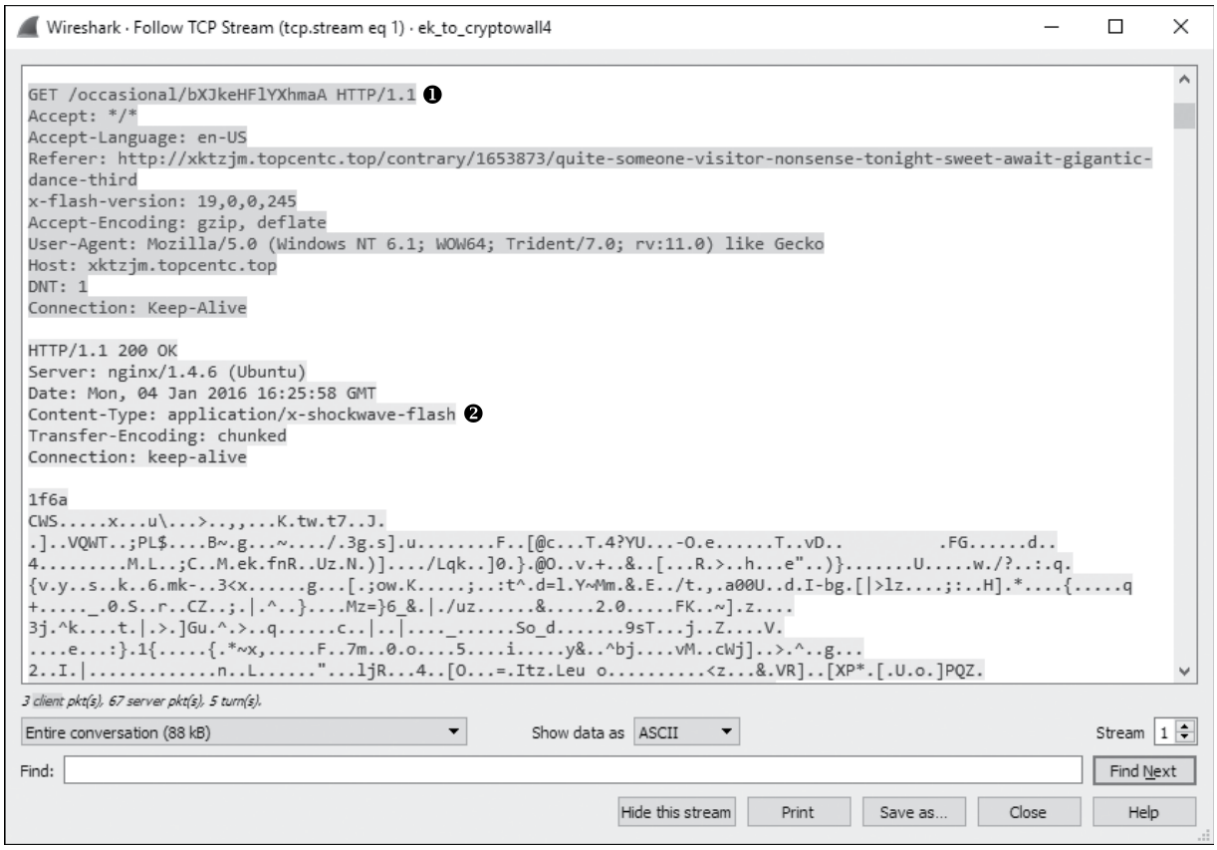


Рисунок 12-36. Загружается Flash-файл со странным именем

При ближайшем рассмотрении видно, что веб-сервер определяет содержимое файла как `x-shockwave-flash` (метка 2). Flash - популярное расширение для потоковой передачи мультимедиа в браузере. Загрузка устройством содержимого Flash не является аномалией, однако следует помнить, что Flash печально известен программными уязвимостями и часто остаётся без исправлений. После запроса Flash-файл успешно загружается.

После загрузки Flash-файла в пакете 130 запрашивается другой файл с похожим именем. Проследив этот TCP-поток на рис. 12-37, вы увидите запрос файла `enVjZ2dtcnpz` (метка 1). Тип файла не обозначен ни расширением, ни сервером. За запросом следует загрузка клиентом 358 400 байт нечитаемых данных (метка 2).

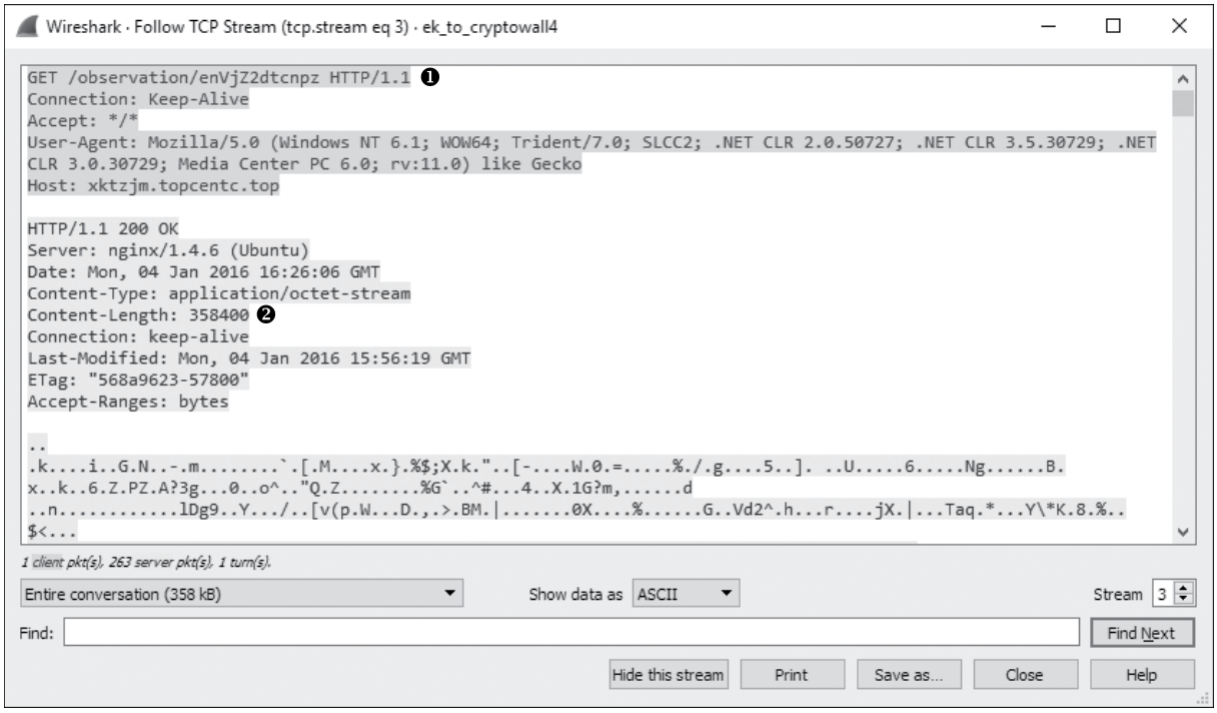


Рисунок 12-37. Загружается ещё один файл со странным именем, но его тип не указан

Менее чем через 20 секунд после загрузки этого файла в списке HTTP-запросов с рис. 12-34 появляется нечто знакомое. Начиная с пакета 441, доверенный узел выполняет запросы HTTP POST к двум разным серверам по уже наблюдавшемуся шаблону C2. Вероятно, мы нашли источник заражения. За него отвечали два загруженных файла: первый, запрошенный в пакете 39, доставил эксплойт Flash, а второй, запрошенный в пакете 130, - вредоносную программу.

> **Примечание.** Для декодирования и анализа содержащихся в захвате файлов можно применить методы анализа вредоносных программ. Если вы хотите узнать больше об их обратной разработке, рекомендую книгу Майкла Сикорски и Эндрю Хонига *Practical Malware Analysis* (2012), ещё одно издание No Starch Press и одну из моих любимых книг.

Этот сценарий представляет один из самых распространённых методов заражения. Пользователь просматривал Интернет и попал на сайт, заражённый вредоносным кодом перенаправления из набора эксплойтов. Такие наборы заражают законные серверы и определяют отпечаток клиентов, чтобы выявить их уязвимости. Заражённая страница называется целевой страницей набора; её задача - перенаправить клиента на другой сайт с эксплойтом, который набор считает эффективным против данной системы.

Только что рассмотренные пакеты относятся к набору эксплойтов Angler, который, возможно, наблюдался чаще всех остальных в 2015 и 2016 годах. Когда пользователь попал на заражённый Angler сайт, набор установил, что он уязвим для определённой уязвимости Flash. Flash-файл был доставлен, система подверглась эксплуатации, а вторичная полезная нагрузка вредоносной программы CryptoWall была загружена и установлена на узле. Вся последовательность показана на рис. 12-38.

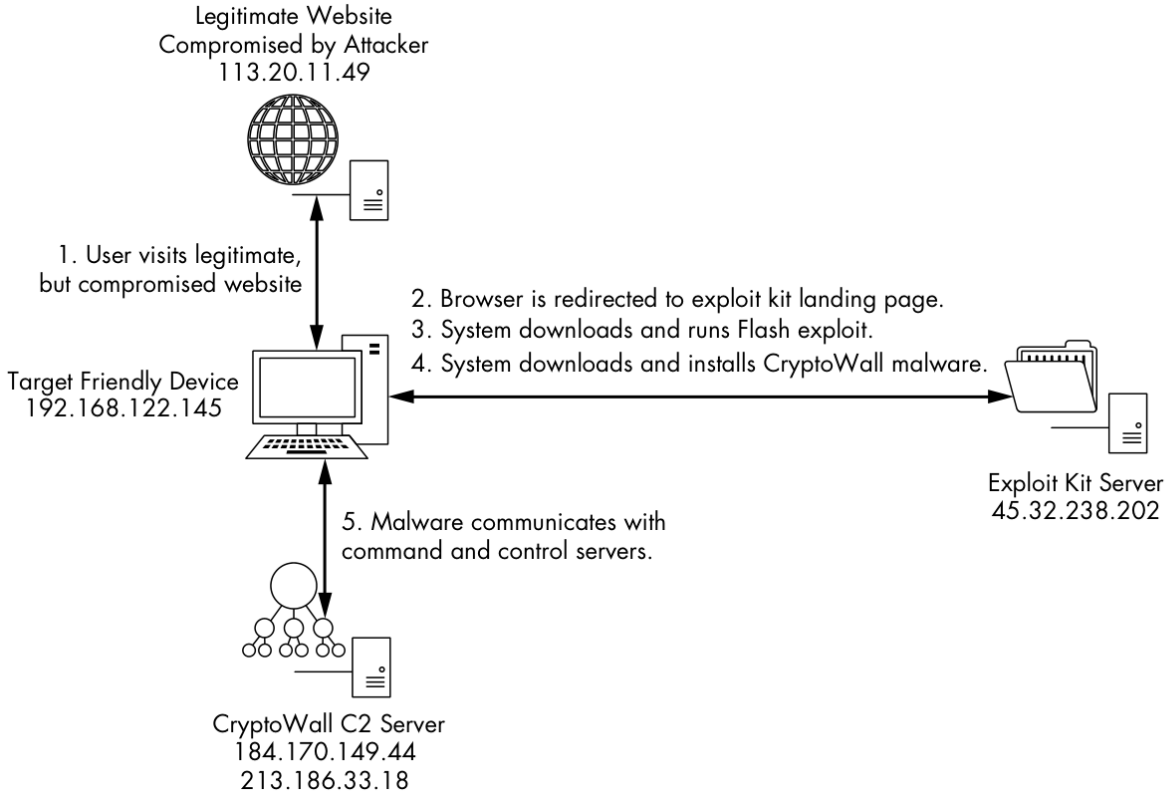


Рисунок 12-38. Последовательность заражения набором эксплойтов

Заключение

Можно написать целые книги о разборе захватов пакетов в сценариях безопасности, анализе распространённых атак и реагировании на предупреждения IDS. В этой главе мы исследовали несколько распространённых методов сканирования и перечисления ресурсов, обычную атаку MITM и пару примеров того, как система может подвергнуться эксплуатации и что в результате способно произойти.

Глава 13. Анализ беспроводных пакетов

Мир беспроводных сетей несколько отличается от традиционных проводных. Хотя мы по-прежнему имеем дело с распространёнными протоколами обмена, такими как TCP и IP, при переходе к нижним уровням модели OSI правила немного меняются. Здесь из-за природы беспроводных сетей и физического уровня особое значение приобретает канальный уровень. Вместо простых проводных протоколов вроде Ethernet, которые со временем почти не изменились, приходится учитывать тонкости таких быстро развивающихся беспроводных протоколов, как 802.11. Это накладывает новые ограничения на доступные нам данные и способы их захвата.

Учитывая эти дополнительные обстоятельства, неудивительно, что захвату и анализу пакетов в беспроводных сетях посвящена целая глава. Мы подробно обсудим, почему беспроводные сети уникальны с точки зрения анализа пакетов и как преодолевать возникающие трудности. Разумеется, делать это будем на настоящих практических примерах захватов беспроводной сети.

Физические особенности

Первое, что следует учитывать при захвате и анализе данных, передаваемых по беспроводной сети, - физическая среда передачи. До сих пор мы не рассматривали физический уровень, поскольку обмен происходил по физическим кабелям. Теперь мы связываемся через невидимые радиоволны, а пакеты буквально пролетают рядом с нами.

Перехват одного канала за раз

Отличительная особенность захвата трафика беспроводной локальной сети (wireless local area network, WLAN) состоит в том, что беспроводной спектр является общей средой. В отличие от проводных сетей, где у каждого клиента есть собственный сетевой кабель до коммутатора, средой беспроводного обмена служит общее для клиентов и ограниченное по объёму пространство эфира. Одна WLAN занимает только часть спектра 802.11. Поэтому несколько систем могут работать в одной физической области, используя разные части спектра.

> **Примечание.** Беспроводные сети основаны на стандарте 802.11, разработанном Институтом инженеров электротехники и электроники (Institute of Electrical and Electronics Engineers, IEEE). На протяжении главы термины *беспроводная сеть* и *WLAN* обозначают сети, соответствующие 802.11. Самые популярные версии этого стандарта - 802.11a, b, g и n. Каждая обладает уникальным набором функций и характеристик, причём новые стандарты, например n, обеспечивают более высокую скорость. Все они используют один и тот же спектр.

Разделение пространства становится возможным благодаря делению спектра на рабочие каналы. Канал - просто часть беспроводного спектра 802.11. В США доступны 11 каналов, а в некоторых других странах разрешено больше. Это важно, поскольку WLAN в каждый момент может работать только на одном канале, а мы способны одновременно перехватывать пакеты лишь одного канала, как показано на рис. 13-1. Поэтому при устранении неполадок WLAN, работающей на канале 6, систему необходимо настроить на захват трафика, наблюдаемого именно там.

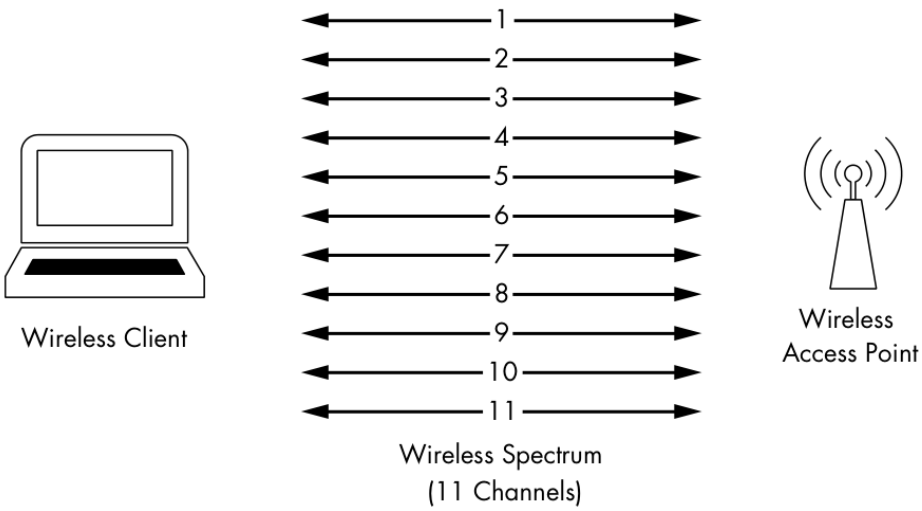


Рисунок 13-1. Беспроводной перехват может быть утомительным, поскольку одновременно выполняется лишь на одном канале

Традиционный беспроводной перехват выполняется только на одном канале за раз, но есть исключение: некоторые приложения беспроводного сканирования применяют метод переключения каналов (channel hopping), быстро переходя между ними для сбора данных. Один из самых популярных инструментов этого типа, Kismet (http://www.kismetwireless.net/), способен переключаться до десяти раз в секунду, благодаря чему очень эффективно перехватывает сразу несколько каналов.

Помехи беспроводного сигнала

При беспроводном обмене иногда нельзя полагаться на целостность передаваемых по воздуху данных: сигналу может что-нибудь помешать. В беспроводных сетях предусмотрены функции обработки помех, но они срабатывают не всегда. Поэтому при захвате пакетов беспроводной сети нужно внимательно следить за окружением и убеждаться в отсутствии значительных источников помех: крупных отражающих поверхностей, больших твёрдых объектов, микроволновых печей, телефонов с частотой 2,4 ГГц, толстых стен и материалов высокой плотности. Всё это может привести к потере, дублированию и неправильному формированию пакетов.

Следует учитывать и взаимные помехи каналов. Хотя одновременно можно перехватывать только один канал, есть небольшая оговорка: в спектре беспроводной сети доступно несколько каналов передачи, но пространство ограничено, поэтому они немного перекрываются, как показано на рис. 13-2. Если трафик присутствует на каналах 4 и 5, а вы перехватываете один из них, скорее всего, в захват попадут и пакеты другого. Обычно сосуществующие в одной области сети проектируют на неперекрывающихся каналах 1, 6 и 11, поэтому вы, вероятно, не столкнётесь с этой проблемой. Но на всякий случай следует понимать, почему она возникает.

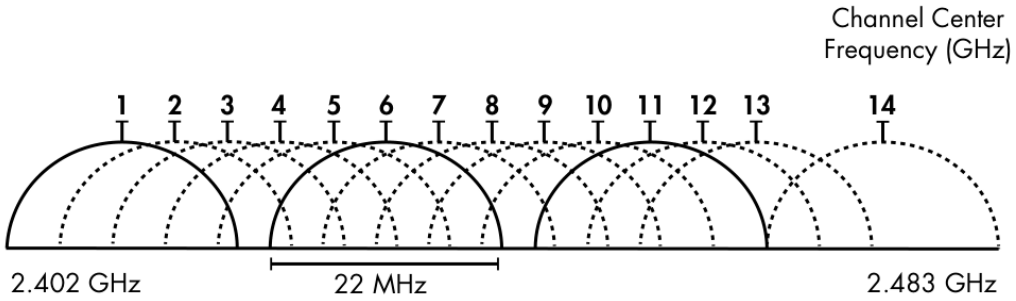


Рисунок 13-2. Из-за ограниченного пространства спектра каналы перекрываются

Обнаружение и анализ помех сигнала

Устранить помехи беспроводного сигнала, исследуя пакеты в Wireshark, невозможно. Если вы собираетесь регулярно или профессионально устранять неполадки WLAN, проверять помехи наверняка придётся часто. Эта задача выполняется анализатором спектра - инструментом, который показывает данные или помехи во всём спектре.

Коммерческие анализаторы спектра могут стоить несколько тысяч долларов, но для обычного повседневного применения существует прекрасное решение. Компания MetaGeek выпускает аппаратное USB-устройство Wi-Spy, которое отслеживает сигналы во всём спектре 802.11. В сочетании с программным обеспечением MetaGeek inSSIDer или Chanalyzer оно графически отображает спектр, помогая устранять неполадки. Пример вывода Chanalyzer показан на рис. 13-3.

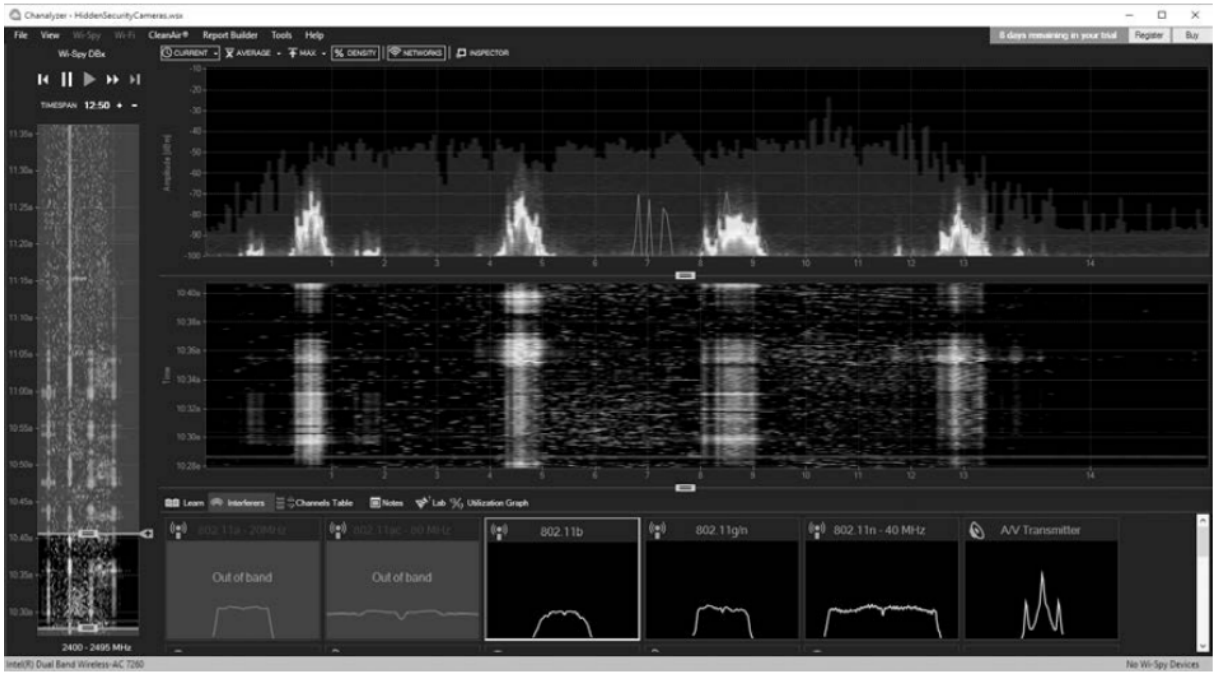


Рисунок 13-3. Этот вывод Chanalyzer показывает четыре сигнала, равномерно распределённых по спектру Wi-Fi

Режимы беспроводной карты

Прежде чем начать перехват беспроводных пакетов, рассмотрим разные режимы, в которых беспроводная карта может работать применительно к захвату пакетов.

Доступны четыре режима беспроводного сетевого адаптера.

Управляемый режим

Этот режим используется, когда беспроводной клиент подключается непосредственно к беспроводной точке доступа (wireless access point, WAP). В таком случае связанный с беспроводным адаптером драйвер полагается на WAP в управлении всем процессом обмена.

Одноранговый режим

Этот режим применяется в беспроводной сети, где устройства подключаются непосредственно друг к другу. Два желающих обмениваться беспроводных клиента совместно выполняют обязанности, которые обычно брала бы на себя WAP.

Режим ведущего

Некоторые беспроводные адаптеры более высокого класса поддерживают также режим ведущего. Он позволяет адаптеру совместно со специализированным драйвером превратить компьютер в WAP для других устройств.

Режим мониторинга

Это самый важный режим для наших целей. Он применяется, когда беспроводной клиент должен прекратить передачу и приём данных и вместо этого лишь прослушивать пролетающие по эфиру пакеты. Чтобы Wireshark мог захватывать беспроводные пакеты, адаптер и его драйвер должны поддерживать режим мониторинга, также известный как RFMON.

Большинство пользователей применяют беспроводные карты только в управляемом или одноранговом режиме. Графическое представление работы каждого режима показано на рис. 13-4.

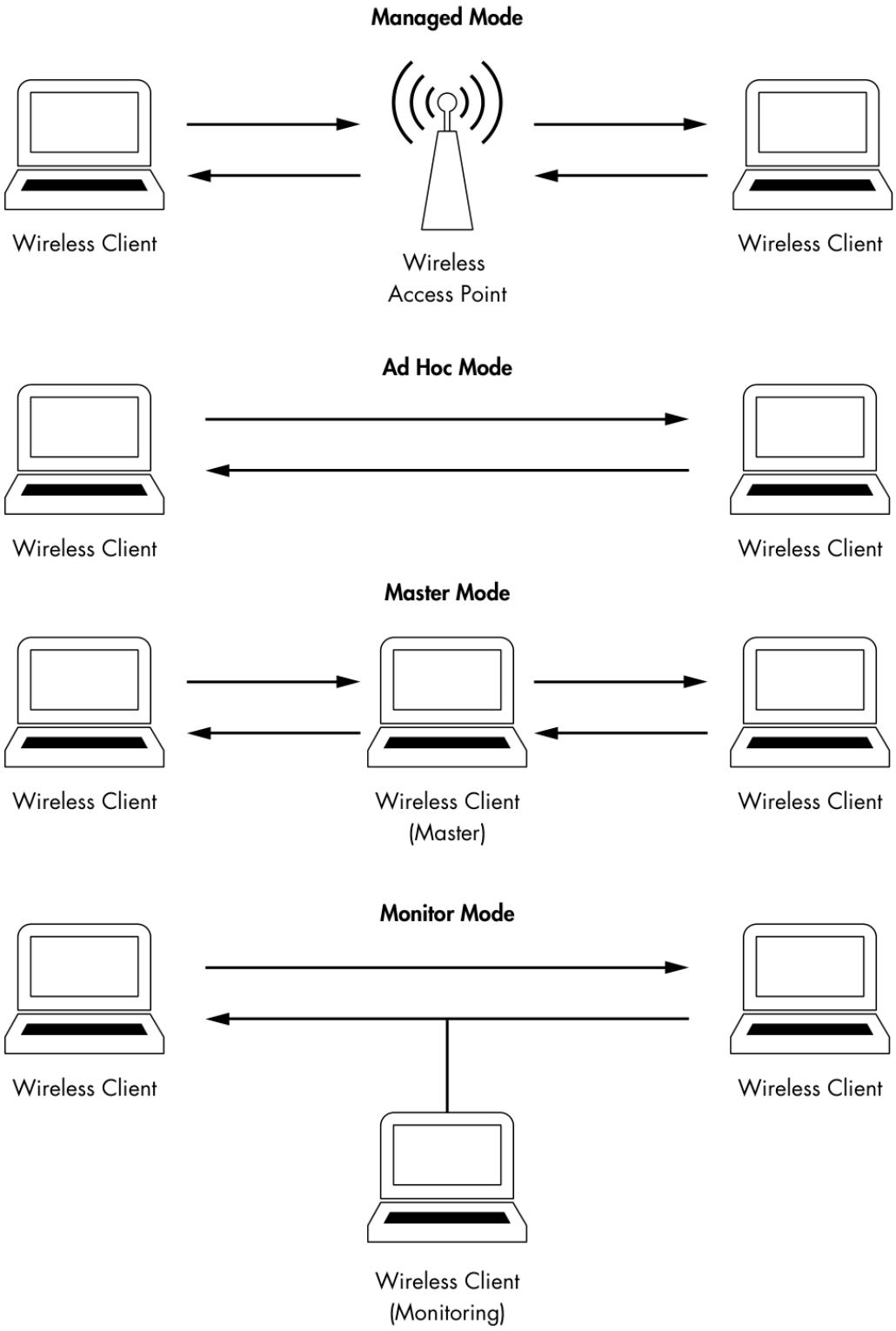


Рисунок 13-4. Различные режимы беспроводной карты

Примечание. Меня часто спрашивают, какую беспроводную карту я рекомендую для анализа пакетов. Я пользуюсь продукцией ALFA Network и настоятельно её рекомендую. Эти устройства считаются одними из лучших на рынке, когда требуется гарантированно захватить все возможные пакеты; кроме того, они недороги и портативны. Продукция ALFA доступна у большинства интернет-магазинов компьютерного оборудования.

Беспроводной перехват в Windows

Даже если беспроводной адаптер поддерживает режим мониторинга, большинство драйверов адаптеров для Windows не позволяют в него переключиться. Это означает, что вы сможете захватывать лишь пакеты, входящие и исходящие через беспроводной интерфейс устройства, с которого подключаетесь к сети. Для захвата пакетов между всеми устройствами канала потребуется дополнительное оборудование.

Настройка AirPcap

AirPcap компании Riverbed Technologies (http://www.riverbed.com/) предназначен для преодоления ограничений, которые Windows накладывает на анализ беспроводных пакетов. AirPcap - небольшое USB-устройство, похожее на флеш-накопитель, как показано на рис. 13-5. Оно захватывает беспроводной трафик одного или нескольких заданных каналов. AirPcap использует драйвер WinPcap и специальную клиентскую утилиту настройки.

Рисунок 13-5. AirPcap очень компактен, поэтому его удобно носить вместе с ноутбуком

Программа настройки AirPcap, показанная на рис. 13-6, проста в использовании и предлагает лишь несколько параметров.

Interface

Здесь выбирается устройство для захвата. В некоторых сложных сценариях анализа может потребоваться несколько устройств AirPcap для одновременного перехвата на нескольких каналах.

Blink Led

При нажатии этой кнопки светодиоды устройства AirPcap начинают мигать. Главным образом это позволяет определить конкретный используемый адаптер, если устройств AirPcap несколько.

Channel

В этом поле выбирается канал, который должен прослушивать AirPcap.

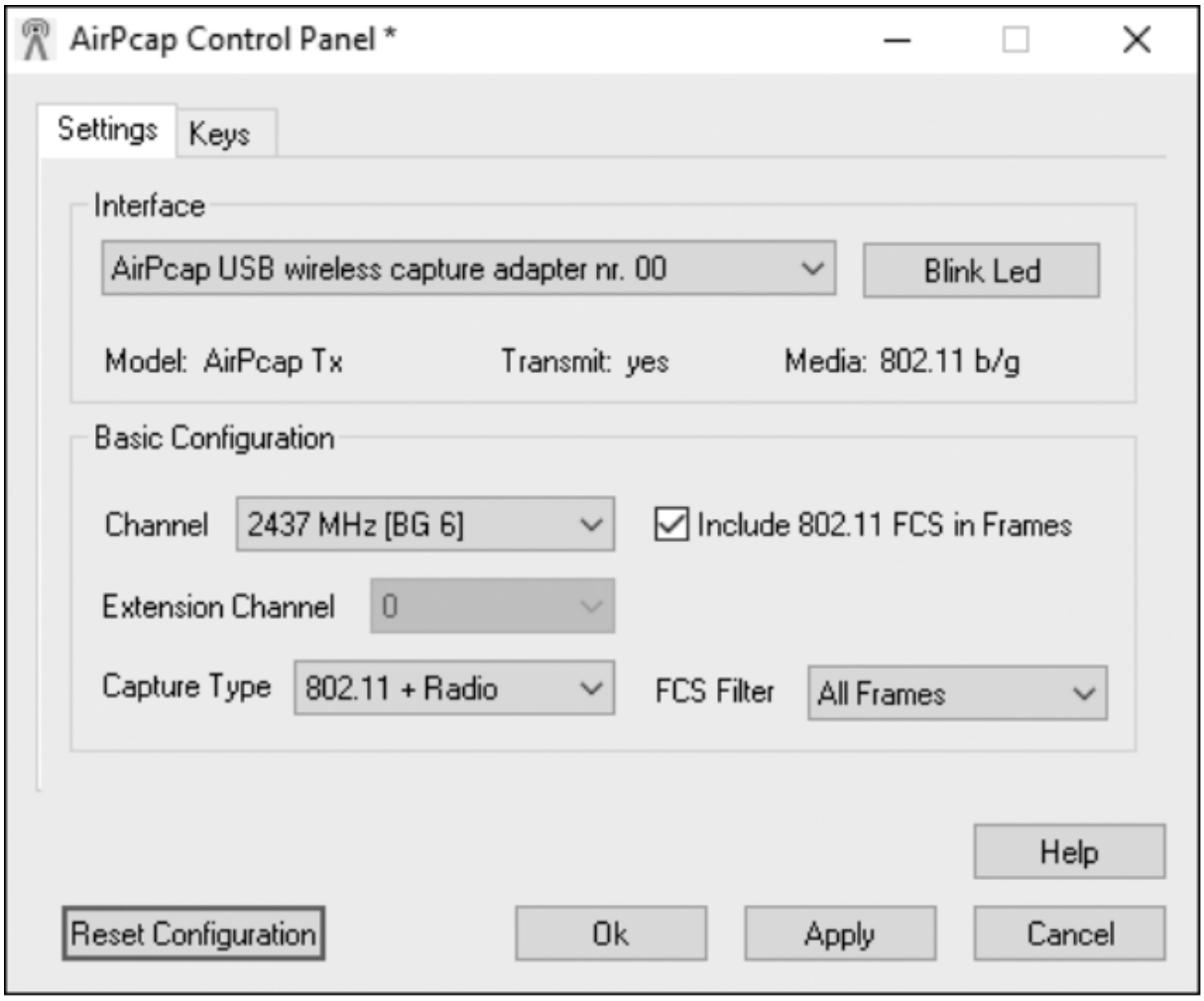


Рисунок 13-6. Программа настройки AirPcap

Extension Channel

Здесь можно выбрать канал расширения - функцию адаптеров 802.11n, позволяющую создавать более широкие каналы.

Include 802.11 FCS in Frames

По умолчанию некоторые системы удаляют из беспроводных пакетов последние четыре бита контрольной суммы. Эта контрольная сумма, называемая последовательностью проверки кадра (frame check sequence, FCS), позволяет убедиться, что пакет не был повреждён при передаче. Если нет конкретной причины поступить иначе, установите этот флажок, чтобы включить контрольные суммы FCS.

Capture Type

Здесь доступны три варианта: 802.11 Only, 802.11 + Radio и 802.11 + PPI. Вариант 802.11 Only включает стандартный заголовок пакета 802.11 во все захваченные пакеты. 802.11 + Radio добавляет к этому заголовку предшествующий заголовок radiotap с дополнительными сведениями о пакете, такими как скорость передачи, частота, уровень сигнала и уровень шума. Вариант 802.11 + PPI добавляет заголовок Per-Packet Information с дополнительными сведениями о пакетах 802.11n.

FCS Filter

Даже если снять флажок Include 802.11 FCS in Frames, этот параметр позволяет отфильтровать пакеты, которые FCS считает повреждёнными. Вариант Valid Frames выводит только пакеты, которые, по мнению FCS, могут быть успешно приняты.

WEP Configuration

Эта область, доступная на вкладке Keys панели управления AirPcap, позволяет вводить ключи расшифровки WEP для перехватываемых сетей. Чтобы интерпретировать данные, зашифрованные WEP, в это поле необходимо ввести правильные ключи WEP. Ключи WEP обсуждаются в разделе "Успешная аутентификация WEP" на странице 309 книги.

Захват трафика с помощью AirPcap

После установки и настройки AirPcap процесс захвата должен быть вам знаком. Просто запустите Wireshark и выберите интерфейс AirPcap, чтобы начать сбор пакетов, как показано на рис. 13-7.

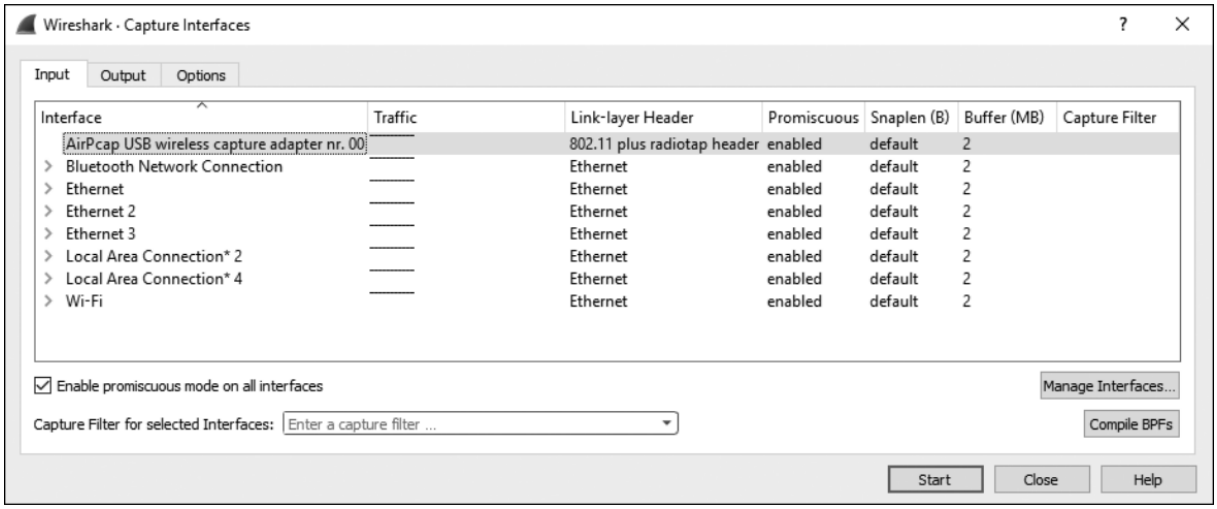


Рисунок 13-7. Выбор интерфейса AirPcap для захвата пакетов

Помните, что захватываются пакеты того канала, который выбран в утилите настройки AirPcap. Если нужных пакетов нет, вероятно, вы смотрите не тот канал. Чтобы изменить его, остановите активный захват, выберите новый канал в утилите AirPcap и снова запустите захват. Менять канал во время активного захвата нельзя.

Если нужно проверить канал захвата из Wireshark, проще всего просмотреть статистику беспроводного захвата. Для этого в главном раскрывающемся меню выберите Wireless > WLAN Traffic. Открывшееся окно покажет обнаруженные устройства и сведения о них, включая канал 802.11, как на рис. 13-8.

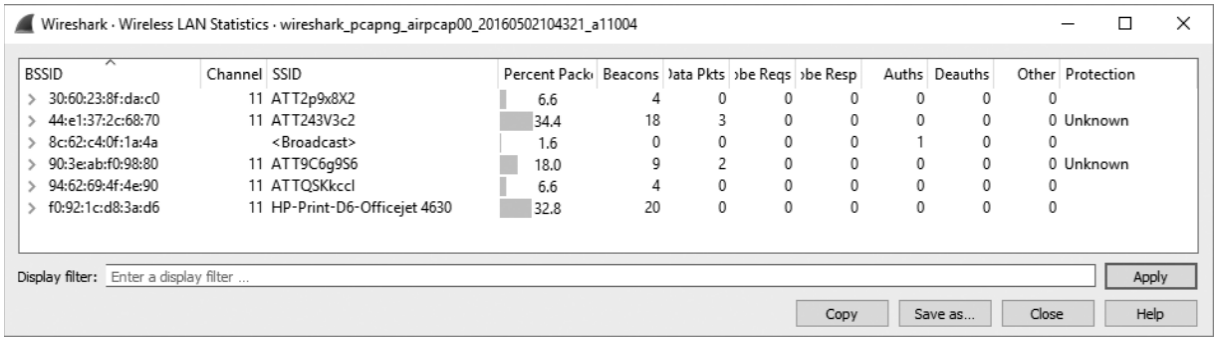


Рисунок 13-8. Окно Wireless LAN Statistics указывает, что данные захвачены при прослушивании канала 11

Беспроводной перехват в Linux

Для перехвата в Linux достаточно включить режим мониторинга беспроводного адаптера и запустить Wireshark. К сожалению, процедура включения этого режима различается у разных моделей адаптеров, поэтому дать точные универсальные указания невозможно. Более того, некоторые беспроводные адаптеры вообще не требуют отдельного включения режима мониторинга. Лучше всего быстро поискать в Google модель своего адаптера и выяснить, требуется ли это и как именно сделать.

Один из наиболее распространённых способов включения режима мониторинга в Linux - встроенные беспроводные расширения. Обращаться к ним позволяет команда `iwconfig`. Если ввести её в консоли, результат будет похож на следующий:

```
$ iwconfig
Eth0  no wireless extensions
Lo0   no wireless extensions
Eth1  IEEE 802.11g
      ESSID: "Tesla Wireless Network"
      Mode: Managed  Frequency: 2.462 GHz  Access Point: 00:02:2D:8B:70:2E
      Bit Rate: 54 Mb/s  Tx-Power-20 dBm  Sensitivity=8/0
      Retry Limit: 7  RTS thr: off  Fragment thr: off
      Power Management: off
      Link Quality=75/100  Signal level=-71 dBm  Noise level=-86 dBm
      Rx invalid nwid: 0  Rx invalid crypt: 0  Rx invalid frag: 0
      Tx excessive retries: 0  Invalid misc: 0  Missed beacon: 2
```

Вывод `iwconfig` показывает, что интерфейс Eth1 поддерживает беспроводную настройку. Это очевидно, поскольку для него отображаются данные протокола 802.11g (метка 2), тогда как интерфейсы Eth0 и Lo0 возвращают фразу `no wireless extensions` (метка 1).

Вместе со всеми предоставленными командой сведениями, например беспроводным идентификатором расширенного набора служб (extended service set ID, ESSID) и частотой, обратите внимание: во второй строке под Eth1 текущим режимом указан Managed. Именно это мы хотим изменить.

Чтобы перевести интерфейс Eth1 в режим мониторинга, нужно войти как пользователь root непосредственно или с помощью команды смены пользователя `su`:

```
$ su
Password: <введите здесь пароль root>
```

Получив права root, можно вводить команды настройки беспроводного интерфейса. Чтобы перевести Eth1 в режим мониторинга, введите:

```
# iwconfig eth1 mode monitor
```

После перевода адаптера в режим мониторинга повторный запуск `iwconfig` должен отразить изменение. Теперь убедитесь, что интерфейс Eth1 работает, введя:

```
# iwconfig eth1 up
```

Команда `iwconfig` также позволяет изменить прослушиваемый канал. Чтобы перевести интерфейс Eth1 на канал 3, введите:

```
# iwconfig eth1 channel 3
```

> **Примечание.** Каналы можно менять прямо во время захвата пакетов, поэтому делайте это когда пожелаете. Для упрощения процесса команду `iwconfig` можно включить в сценарий.

Завершив настройку, запустите Wireshark и начните захват пакетов.

Структура пакета 802.11

Главное различие между беспроводными и проводными пакетами - наличие дополнительного заголовка 802.11. Этот заголовок уровня 2 содержит дополнительные сведения о пакете и среде его передачи. Существуют три типа пакетов 802.11.

Управление

Эти пакеты применяются для установления соединения между узлами на уровне 2. К важным подтипам пакетов управления относятся пакеты аутентификации, связывания и маяка.

Контроль

Контрольные пакеты обеспечивают доставку пакетов управления и данных и связаны с управлением перегрузкой. Распространённые подтипы включают пакеты запроса на отправку и разрешения отправки.

Данные

Эти пакеты содержат собственно данные и являются единственным типом, который можно пересылать из беспроводной сети в проводную.

Тип и подтип беспроводного пакета определяют его структуру, поэтому возможных вариантов очень много. Исследуем один из них по единственному пакету файла `80211beacon.pcapng`. В нём находится пример пакета управления, называемого маяком, который показан на рис. 13-9.

Маяк - один из самых информативных беспроводных пакетов. WAP широковещательно отправляет его по беспроводному каналу, чтобы уведомить всех прослушивающих беспроводных клиентов о своей доступности и определить параметры, которые необходимо установить для подключения. В нашем примере поле Type/Subtype заголовка 802.11 определяет пакет как маяк (метка 1).

Заголовок кадра управления 802.11 содержит множество дополнительных сведений, в том числе следующие:

Timestamp

Время передачи пакета.

Beacon Interval

Интервал повторной передачи пакета маяка.

Capabilities Information

Сведения об аппаратных возможностях WAP.

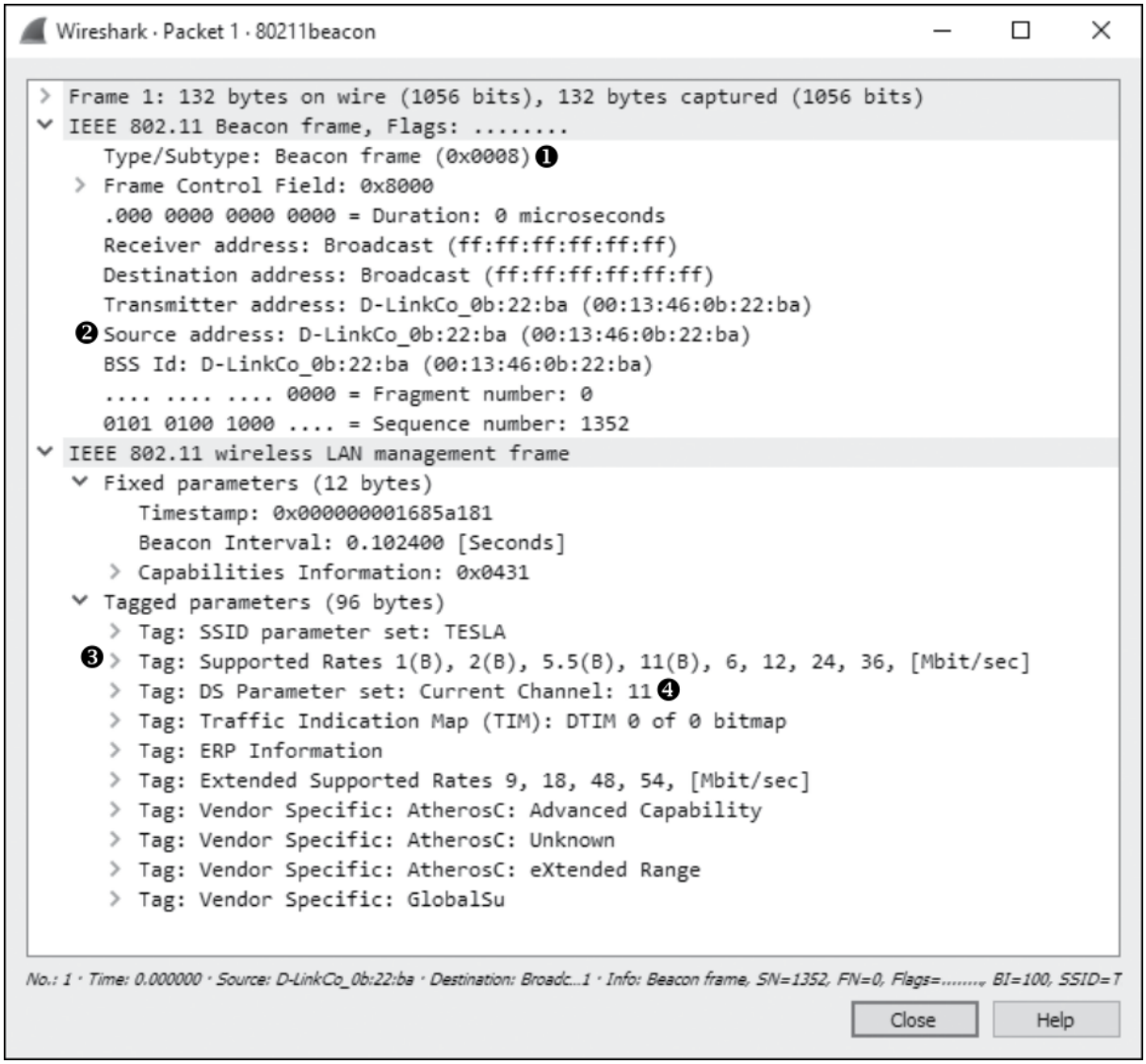


Рисунок 13-9. Это пакет маяка 802.11

SSID parameter set

SSID, то есть имя сети, широковещательно объявляемое WAP.

Supported Rates

Скорости передачи данных, поддерживаемые WAP.

DS Parameter set

Канал, на котором WAP выполняет широковещательную передачу.

Заголовок также содержит адреса источника и назначения и сведения, зависящие от производителя.

На основании этих данных можно довольно многое определить о WAP, передающей маяк в файле примера. Очевидно, что это устройство D-Link (метка 2), использующее стандарт 802.11b (B) (метка 3) на канале 11 (метка 4).

Хотя точное содержимое и назначение пакетов управления 802.11 меняются, общая структура остаётся похожей на этот пример.

Добавление столбцов для беспроводной сети на панель списка пакетов

В предыдущих главах мы использовали гибкий интерфейс Wireshark и добавляли столбцы, подходящие для конкретной ситуации. Прежде чем продолжить беспроводной анализ, полезно добавить на панель списка пакетов следующие три столбца.

• Channel - показывает канал, на котором собран пакет.
• Signal Strength - показывает мощность сигнала захваченного пакета в дБм.
• Data Rate - показывает скорость передачи захваченного пакета.

Эти показатели способны сильно помочь при устранении неполадок беспроводных соединений. Например, даже если клиентское беспроводное ПО сообщает об отличной мощности сигнала, захват и проверка этих столбцов могут показать значение, не подтверждающее такое утверждение.

Чтобы добавить столбцы на панель списка пакетов, выполните следующие действия:

1. Выберите Edit > Preferences.
2. Перейдите в раздел Columns и нажмите +.
3. В поле Title введите `Channel`, в раскрывающемся списке Type выберите Custom, а в поле Field Name укажите фильтр `wlan_radio.channel`.
4. Повторите процесс для столбцов Signal Strength и Data Rate, присвойте им соответствующие названия и выберите в списке Field Name соответственно `wlan_radio.signal_dbm` и `wlan_radio.data_rate`. На рис. 13-10 показано, как должно выглядеть окно Preferences после добавления всех трёх столбцов.

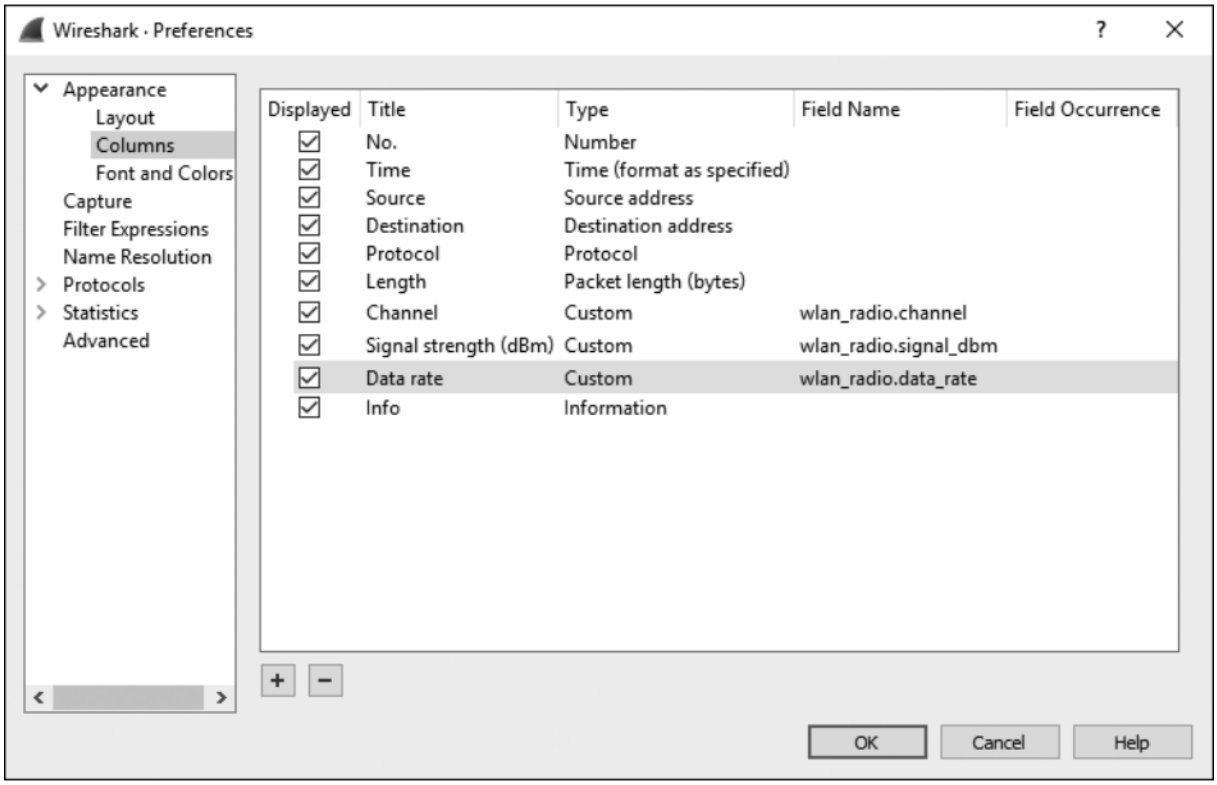


Рисунок 13-10. Добавление столбцов IEEE для беспроводной сети на панель списка пакетов

5. Нажмите OK, чтобы сохранить изменения.

Фильтры для беспроводной сети

В главе 4 мы обсуждали преимущества фильтров захвата и отображения. Фильтровать трафик в проводной инфраструктуре гораздо проще, поскольку у каждого устройства есть собственный выделенный кабель. В беспроводной сети весь трафик клиентов сосуществует на общих каналах, поэтому захват любого канала может содержать трафик десятков клиентов. Этот раздел посвящён некоторым фильтрам пакетов, помогающим находить конкретный трафик.

Фильтрация трафика определённого BSS ID

У каждой WAP в сети есть уникальное идентифицирующее имя, называемое идентификатором базового набора служб (basic service set identifier, BSS ID). Это имя передаётся в каждом беспроводном пакете управления и данных от точки доступа.

Зная имя исследуемого BSS ID, достаточно найти пакет, отправленный конкретной WAP. Wireshark показывает передающую WAP в столбце Info панели списка пакетов, поэтому найти эти сведения обычно легко.

Получив пакет от интересующей WAP, найдите поле BSS ID в заголовке 802.11. На этом адресе будет основан фильтр. Обнаружив MAC-адрес BSS ID, примените:

```
wlan.bssid == 00:11:22:33:44:55
```

После этого будет виден только трафик, проходящий через указанную WAP.

Фильтрация определённых типов беспроводных пакетов

Ранее в этой главе мы обсуждали разные типы беспроводных пакетов, встречающиеся в сети. Часто требуется фильтровать по их типам и подтипам. Для конкретных типов предназначен фильтр `wlan.fc.type`, а для определённых сочетаний типа и подтипа - `wc.fc.type_subtype`. Например, для фильтрации пакета данных NULL, пакета типа 2 и подтипа 4 в шестнадцатеричном виде, можно применить `wlan.fc.type_subtype == 0x24`. В таблице 13-1 приведена краткая справка по некоторым распространённым фильтрам типов и подтипов пакетов 802.11.

> **Примечание переводчика.** В исходном тексте один раз указано `wc.fc.type_subtype`; в примере и таблице используется `wlan.fc.type_subtype`.

Таблица 13-1. Типы и подтипы беспроводных кадров и соответствующий синтаксис фильтров

Тип/подтип кадра	Синтаксис фильтра
Кадр управления	`wlan.fc.type == 0`
Контрольный кадр	`wlan.fc.type == 1`
Кадр данных	`wlan.fc.type == 2`
Запрос связывания	`wlan.fc.type_subtype == 0x00`
Ответ связывания	`wlan.fc.type_subtype == 0x01`
Запрос повторного связывания	`wlan.fc.type_subtype == 0x02`
Ответ повторного связывания	`wlan.fc.type_subtype == 0x03`
Запрос зондирования	`wlan.fc.type_subtype == 0x04`
Ответ зондирования	`wlan.fc.type_subtype == 0x05`
Маяк	`wlan.fc.type_subtype == 0x08`
Разрыв связывания	`wlan.fc.type_subtype == 0x0A`
Аутентификация	`wlan.fc.type_subtype == 0x0B`
Деаутентификация	`wlan.fc.type_subtype == 0x0C`
Кадр действия	`wlan.fc.type_subtype == 0x0D`
Запросы блочного ACK	`wlan.fc.type_subtype == 0x18`
Блочный ACK	`wlan.fc.type_subtype == 0x19`
Опрос энергосбережения	`wlan.fc.type_subtype == 0x1A`
Запрос на отправку	`wlan.fc.type_subtype == 0x1B`
Разрешение отправки	`wlan.fc.type_subtype == 0x1C`
ACK	`wlan.fc.type_subtype == 0x1D`
Конец периода без конкуренции	`wlan.fc.type_subtype == 0x1E`
Данные NULL	`wlan.fc.type_subtype == 0x24`
Данные QoS	`wlan.fc.type_subtype == 0x28`
Данные QoS NULL	`wlan.fc.type_subtype == 0x2C`

Фильтрация определённой частоты

Если вы исследуете объединённый трафик с пакетами нескольких каналов, очень полезно фильтровать по каждому отдельному каналу. Например, если ожидается трафик только на каналах 1 и 6, можно ввести фильтр, который покажет весь трафик канала 11. Если что-либо обнаружится, значит, что-то не в порядке: возможно, допущена ошибка настройки или появилось несанкционированное устройство. Для фильтрации конкретного канала используйте синтаксис:

```
wlan_radio.channel == 11
```

Он покажет весь трафик канала 11. Значение 11 можно заменить номером нужного канала. Существуют сотни других полезных фильтров беспроводного сетевого трафика. Дополнительные фильтры беспроводного захвата представлены в вики Wireshark: http://wiki.wireshark.org/.

Сохранение беспроводного профиля

Настройка специальных столбцов и сохранение пользовательских фильтров для анализа беспроводных пакетов требуют немало труда. Чтобы не перенастраивать и не удалять столбцы и фильтры снова и снова, можно создать и сохранить пользовательский профиль, позволяющий быстро переключаться между конфигурациями проводного и беспроводного анализа.

Чтобы сохранить пользовательский профиль, сначала настройте столбцы и фильтры беспроводной сети по своему вкусу. Затем щёлкните правой кнопкой мыши имя активного профиля в правом нижнем углу экрана и выберите New. Назовите профиль `Wireless` и нажмите OK.

Безопасность беспроводной сети

Главная проблема при развёртывании и администрировании беспроводной сети - безопасность передаваемых в ней данных. Данные летят по воздуху и доступны любому, кто знает, как их получить, поэтому их критически важно шифровать. В противном случае увидеть их сможет каждый, у кого есть Wireshark и AirPcap.

> **Примечание.** Если применяется шифрование другого уровня, например SSL или SSH, трафик по-прежнему останется зашифрован на этом уровне, а обмен пользователя будет нечитаем для человека с анализатором пакетов.

Первоначально предпочтительным способом защиты передаваемых по беспроводным сетям данных был стандарт Wired Equivalent Privacy (WEP). Несколько лет WEP пользовался некоторым успехом, пока в управлении ключами шифрования не обнаружили ряд слабостей. Для повышения безопасности были созданы новые стандарты, включая Wi-Fi Protected Access (WPA) и более безопасный WPA2. Хотя WPA и WPA2 не лишены недостатков, они считаются более безопасными, чем WEP.

В этом разделе мы рассмотрим трафик WEP и WPA, а также примеры неудачных попыток аутентификации.

Успешная аутентификация WEP

Файл `3e80211_WEPauth.pcapng` содержит пример успешного подключения к беспроводной сети с WEP. Безопасность сети настроена с помощью ключа WEP. Этот ключ нужно предоставить WAP, то есть беспроводной точке доступа, чтобы пройти на ней аутентификацию и расшифровать отправляемые ею данные. Ключ WEP можно считать паролем беспроводной сети.

Как показано на рис. 13-11, файл захвата начинается с проверочного запроса, отправленного WAP (`28:c6:8e:ab:96:16`) беспроводному клиенту (`ac:cf:5c:78:6c:9c`) в пакете 3 (метка 1). Его назначение - определить, есть ли у клиента правильный ключ WEP. Запрос можно увидеть, развернув заголовок 802.11 и его параметры с тегами.

Беспроводной клиент отвечает, как показано на рис. 13-12: расшифровывает текст проверочного запроса (метка 1) ключом WEP и возвращает его WAP в пакете 4. Ключ WEP был предоставлен пользователем при попытке подключиться к беспроводной сети.

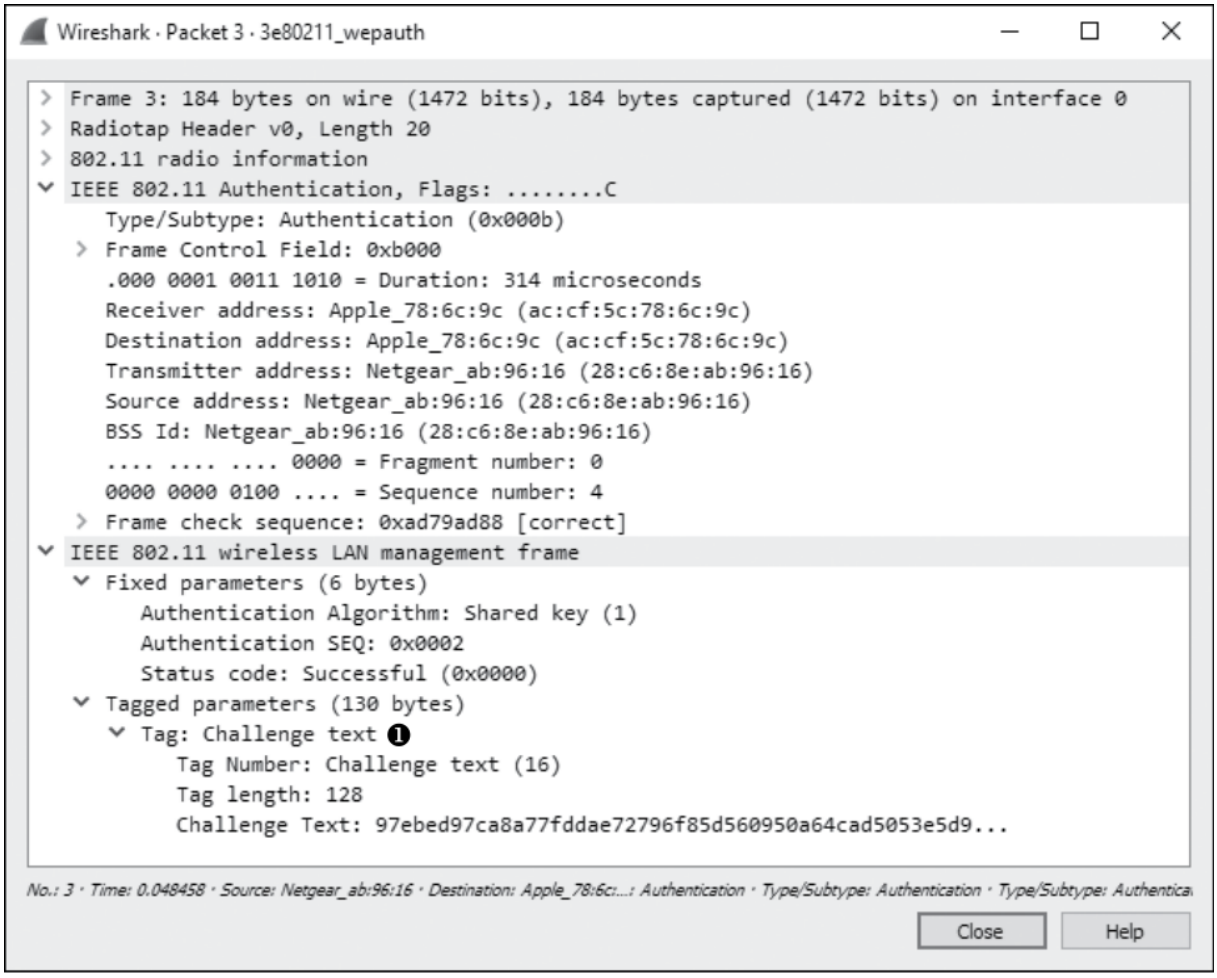


Рисунок 13-11. WAP выдаёт беспроводному клиенту проверочный текст

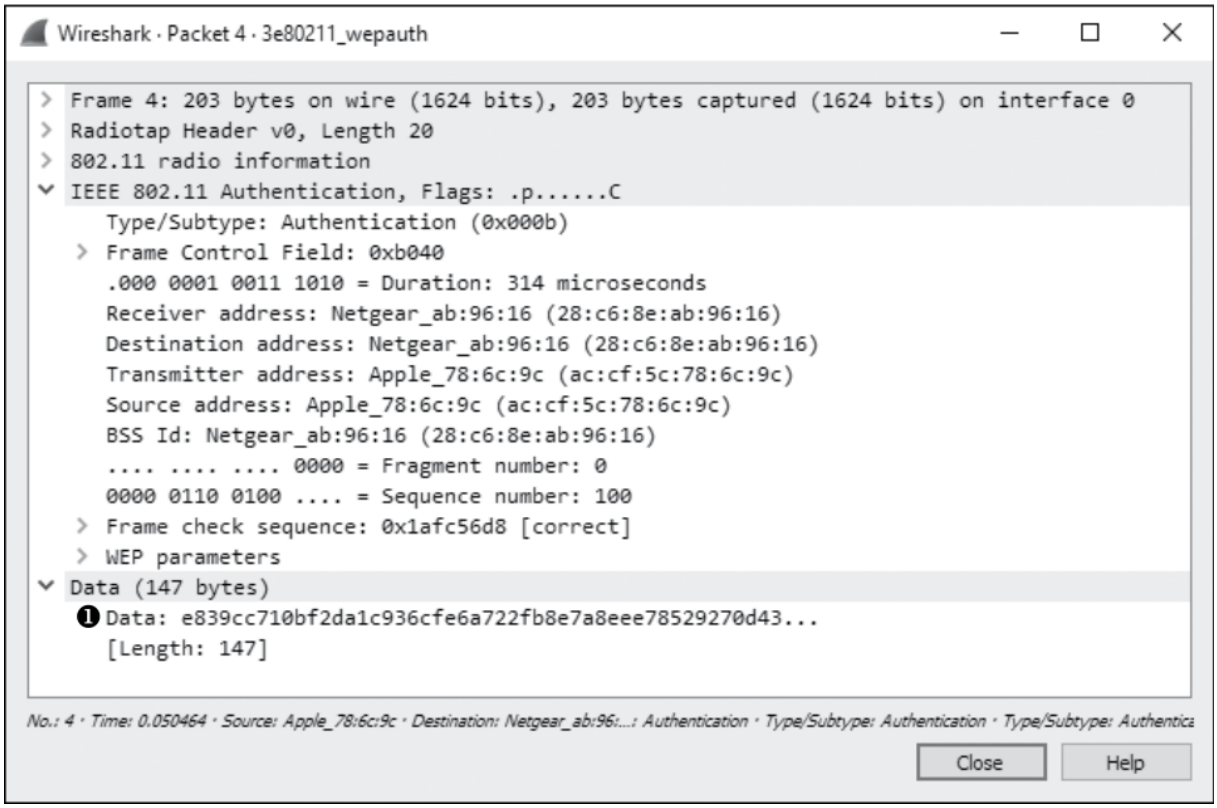


Рисунок 13-12. Беспроводной клиент отправляет незашифрованный проверочный текст обратно WAP

WAP отвечает беспроводному клиенту в пакете 5, как показано на рис. 13-13. Ответ содержит уведомление об успешном завершении аутентификации (метка 1).

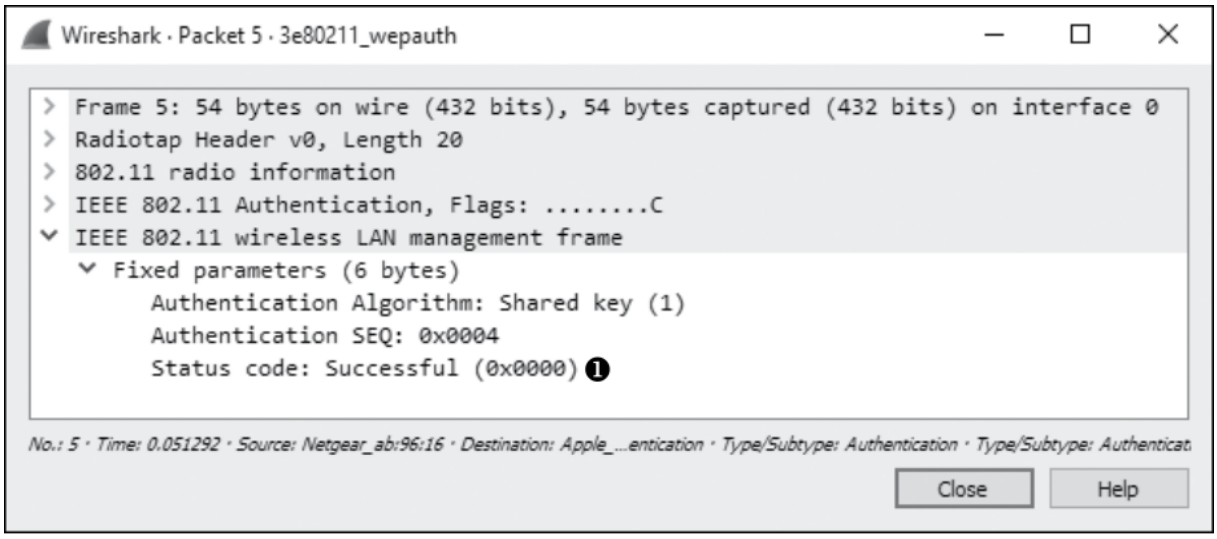


Рисунок 13-13. WAP уведомляет клиента об успешной аутентификации

Наконец, после успешной аутентификации клиент может передать запрос связывания, получить подтверждение и завершить процесс подключения, как показано на рис. 13-14.

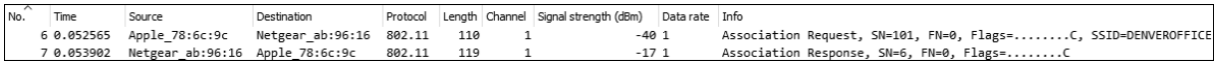


Рисунок 13-14. За процессом аутентификации следуют простые двухпакетные запрос и ответ связывания

Неудачная аутентификация WEP

В следующем примере пользователь вводит ключ WEP для подключения к WAP. Через несколько секунд клиентская беспроводная утилита сообщает, что не смогла подключиться к беспроводной сети, но не объясняет причину. Получившийся файл - `3e80211_WEPauthfail.pcapng`.

Как и при успешной попытке, обмен начинается с отправки WAP проверочного текста беспроводному клиенту в пакете 3. В пакете 4 клиент передаёт ответ, используя предоставленный пользователем ключ WEP.

Теперь мы ожидали бы увидеть уведомление об успешной аутентификации, но в пакете 5 появляется нечто иное, как показано на рис. 13-15 (метка 1).

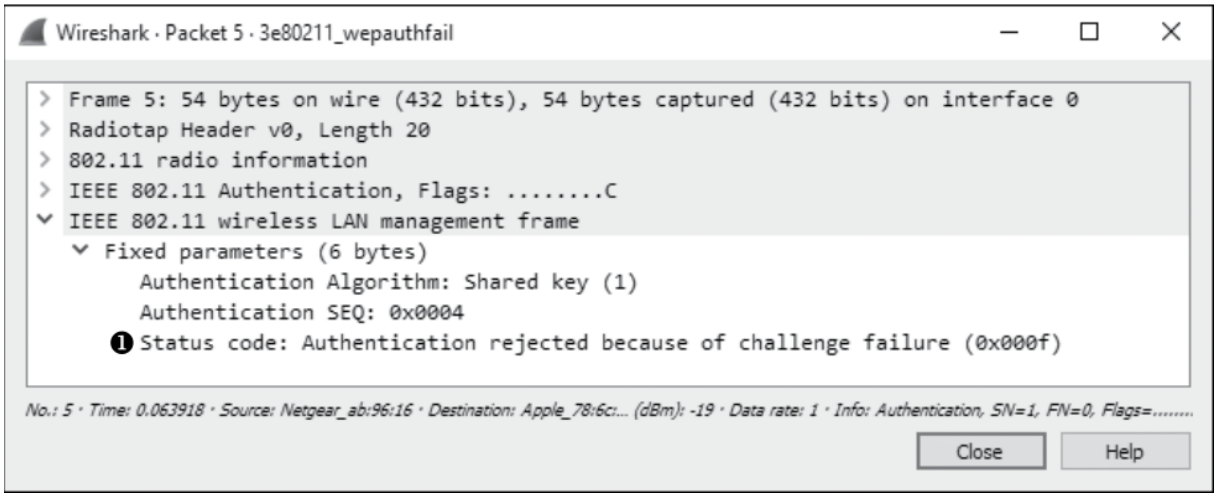


Рисунок 13-15. Это сообщение сообщает о неудаче аутентификации

Сообщение указывает, что ответ беспроводного клиента на проверочный текст был неправильным, а значит, скорее всего, был неверен и ключ WEP, с помощью которого клиент расшифровывал текст. В результате процесс подключения завершился неудачей. Его необходимо повторить с правильным ключом WEP.

Успешная аутентификация WPA

WPA использует совсем другой механизм аутентификации, чем WEP, но по-прежнему требует, чтобы для подключения к сети пользователь ввёл ключ в беспроводном клиенте. Пример успешной аутентификации WPA находится в файле `3e80211_WPAauth.pcapng`.

Первый пакет файла - широковещательный маяк от WAP. Разверните заголовок 802.11 этого пакета, найдите параметры с тегами и разверните заголовок Vendor Specific, как показано на рис. 13-16. Вы увидите раздел, посвящённый атрибутам WPA точки доступа (метка 1). Он сообщает, какую версию и реализацию WPA поддерживает WAP, если поддержка вообще есть.

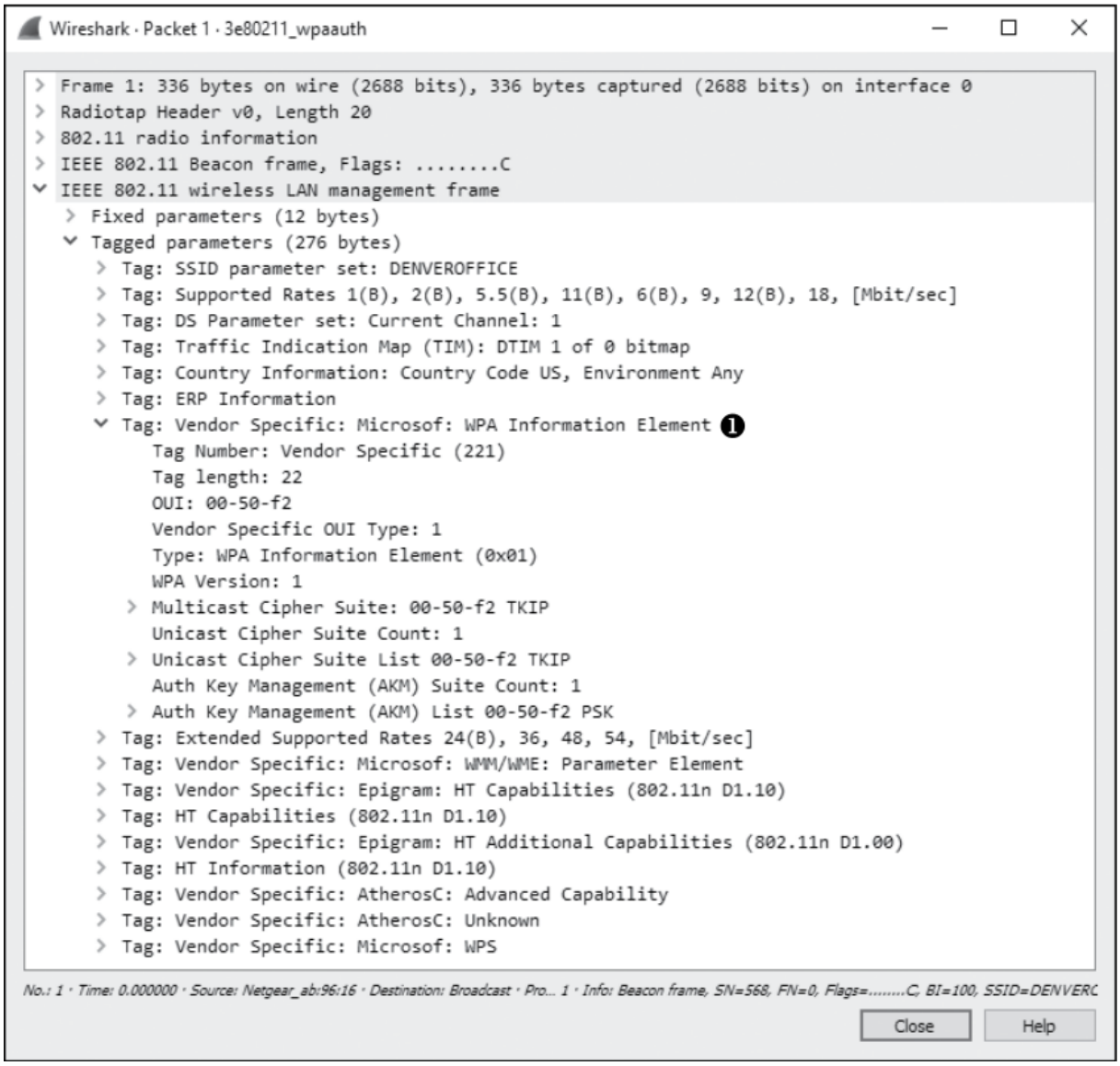


Рисунок 13-16. Этот маяк сообщает, что WAP поддерживает аутентификацию WPA

Получив маяк, беспроводной клиент (`ac:cf:5c:78:6c:9c`) широковещательно отправляет в пакете 2 запрос зондирования. Его принимает WAP (`28:c6:8e:ab:96:16`) и отвечает в пакете 3. После этого в пакетах с 4 по 7 между клиентом и WAP создаются запросы и ответы аутентификации и связывания. Они похожи на пакеты аутентификации и связывания из предыдущего примера WEP, но здесь проверочный запрос и ответ пока отсутствуют. Этот обмен происходит далее.

Самые активные события начинаются с пакета 8. Здесь запускается рукопожатие WPA, продолжающееся до пакета 11. Во время него происходит обмен проверочными запросами и ответами WPA, показанный на рис. 13-17.

No.	Time	Source	Destination	Protocol	Length	Channel	Signal strength (dBm)	Data rate	Info
8	0.…	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	157	1	-18	24	Key (Message 1 of 4)
9	0.…	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	183	1	-42	1	Key (Message 2 of 4)
10	0.…	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	181	1	-18	36	Key (Message 3 of 4)
11	0.…	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	157	1	-42	1	Key (Message 4 of 4)

Рисунок 13-17. Эти пакеты являются частью рукопожатия WPA

Присутствуют два проверочных запроса и два ответа. Каждый запрос можно сопоставить с ответом по полю Replay Counter под заголовком 802.1x Authentication, как показано на рис. 13-18. Обратите внимание: у первых двух пакетов рукопожатия значение Replay Counter равно 1 (метка 1), а у следующих двух - 2 (метка 2).

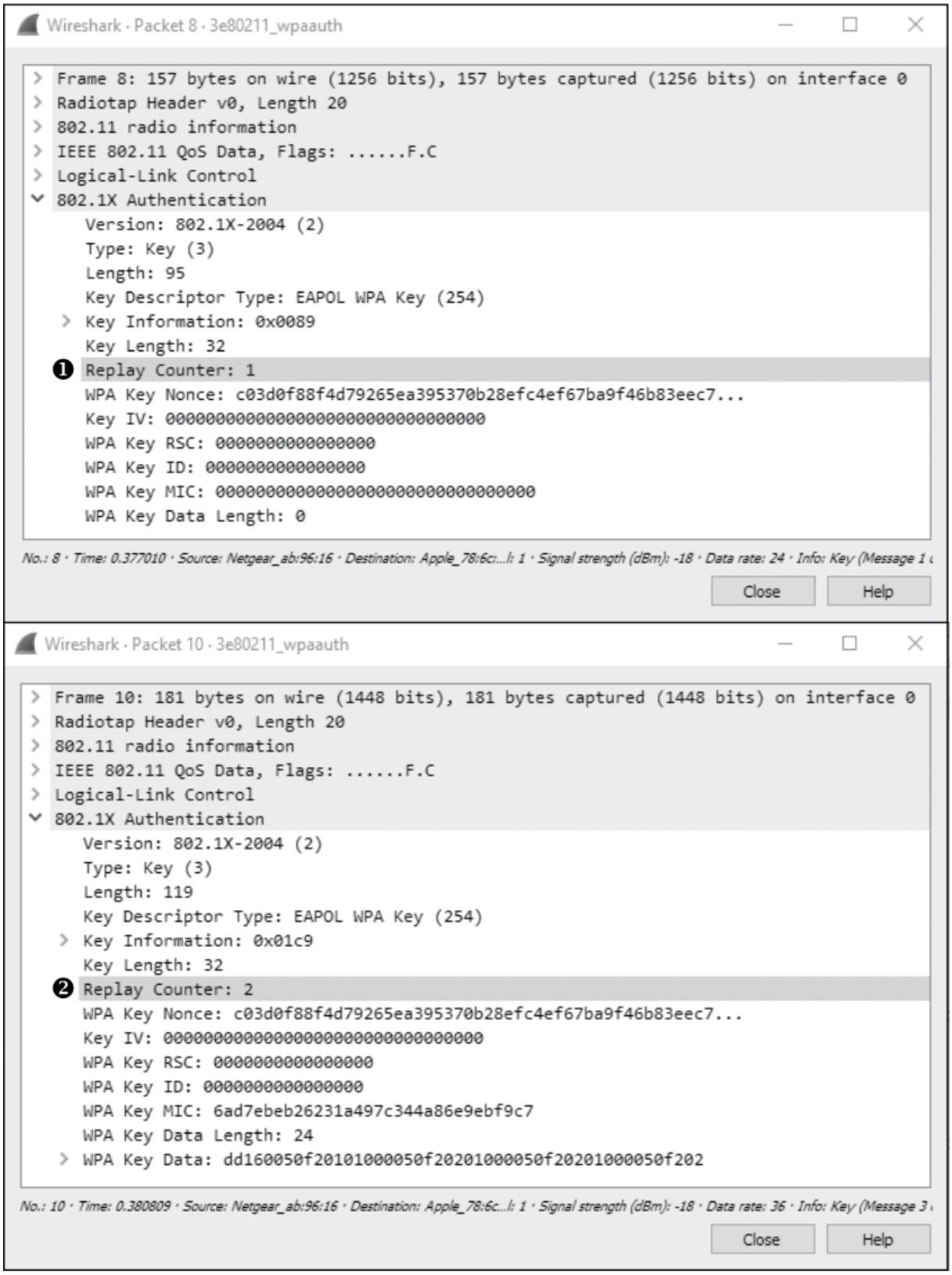


Рисунок 13-18. Поле Replay Counter помогает сопоставить проверочные запросы и ответы

После завершения рукопожатия WPA и успешной аутентификации между беспроводным клиентом и WAP начинается передача данных.

Примечание. В этом примере используется WAP с WPA и шифрованием TKIP. TKIP - лишь один из методов шифрования данных WLAN. Существует множество других видов шифрования, а разные точки доступа поддерживают разные методы. WAP с иным методом шифрования или другой версией WPA, вероятно, будет иметь на уровне пакетов другие характеристики. Чтобы лучше понять, как должна выглядеть последовательность подключения, можно прочитать документ RFC, относящийся к используемой технологии.

Неудачная аутентификация WPA

Как и в случае WEP, рассмотрим ситуацию, когда пользователь вводит ключ WPA, а клиентская беспроводная утилита сообщает, что не смогла подключиться к беспроводной сети, не указывая проблему. Получившийся файл - `3e80211_WPAauthfail.pcapng`.

Захват начинается точно так же, как файл с успешной аутентификацией WPA, и включает запросы зондирования, аутентификации и связывания. Рукопожатие WPA начинается в пакете 8, но теперь в нём восемь пакетов вместо четырёх, наблюдавшихся при успешной попытке.

Пакеты 8 и 9 - первые два пакета рукопожатия WPA. Однако на этот раз проверочный текст, возвращённый клиентом WAP, неправилен. Поэтому последовательность повторяется в пакетах 10 и 11, 12 и 13, а также 14 и 15, как показано на рис. 13-19. Каждый запрос можно сопоставить с ответом по значению Replay Counter.

No.	Time	Source	Destination	Protocol	Length	Channel	Signal strength (dBm)	Data rate	Info
8	0.073773	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	157	1	-18	24	Key (Message 1 of 4)
9	0.076510	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	183	1	-30	1	Key (Message 2 of 4)
10	1.074290	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	157	1	-19	24	Key (Message 1 of 4)
11	1.076573	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	183	1	-32	1	Key (Message 2 of 4)
12	2.075292	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	157	1	-18	36	Key (Message 1 of 4)
13	2.077610	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	183	1	-29	1	Key (Message 2 of 4)
14	3.077211	Netgear_ab:96:16	Apple_78:6c:9c	EAPOL	157	1	-18	48	Key (Message 1 of 4)
15	3.079537	Apple_78:6c:9c	Netgear_ab:96:16	EAPOL	183	1	-32	1	Key (Message 2 of 4)

Рисунок 13-19. Дополнительные пакеты EAPoL (Extensible Authentication Protocol over LAN) указывают на неудачную аутентификацию WPA

После четырёх неудачных попыток рукопожатия обмен прерывается. Как показано на рис. 13-20, в пакете 16 беспроводной клиент деаутентифицируется от WAP (метка 1).

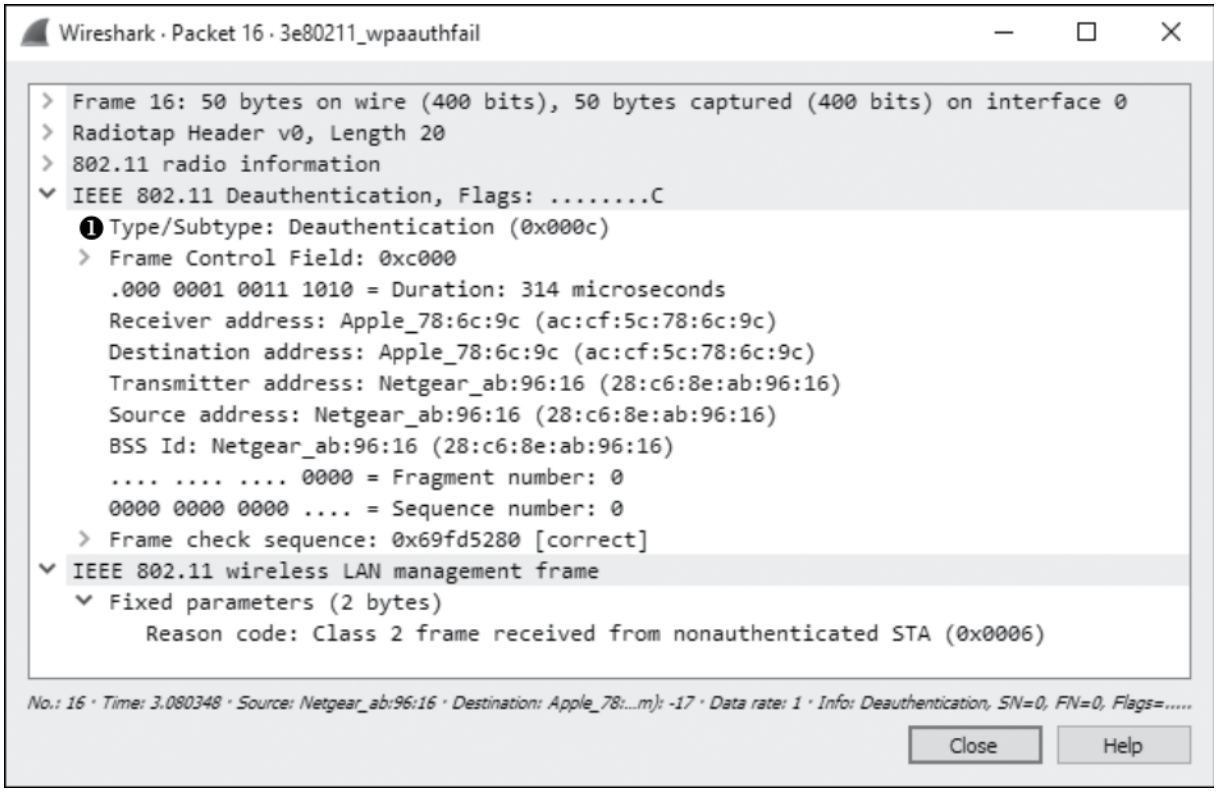


Рисунок 13-20. После неудачи рукопожатия WPA клиент деаутентифицируется

Заключение

Хотя беспроводные сети по-прежнему считаются до некоторой степени небезопасными, если не снабдить их множеством дополнительных механизмов защиты, это опасение не замедлило их развёртывание в различных организационных средах. Поскольку обмен без проводов стал новой нормой, критически важно уметь захватывать и анализировать данные не только проводных, но и беспроводных сетей. Представленные в этой главе навыки и понятия далеко не исчерпывающие, однако они должны помочь вам приступить к пониманию тонкостей устранения неполадок беспроводных сетей посредством анализа пакетов.

Приложение A. Дальнейшее чтение

Хотя главным инструментом в этой книге был Wireshark, при анализе пакетов вам пригодится множество дополнительных средств - как для общего устранения неполадок, так и для медленных сетей, проблем безопасности или беспроводных сетей. В этом приложении перечислены некоторые полезные инструменты анализа пакетов и другие учебные ресурсы.

Инструменты анализа пакетов

Рассмотрим несколько инструментов, которые я считаю полезными для анализа пакетов.

CloudShark

CloudShark, разработанный QA Cafe, - мой любимый инструмент хранения, индексирования и сортировки захватов пакетов. Это коммерческое веб-приложение, служащее хранилищем захватов. Оно позволяет снабжать захваты тегами для быстрого обращения и добавлять комментарии непосредственно в них. CloudShark даже предоставляет некоторые функции анализа, похожие на возможности Wireshark, как показано на рис. A-1.

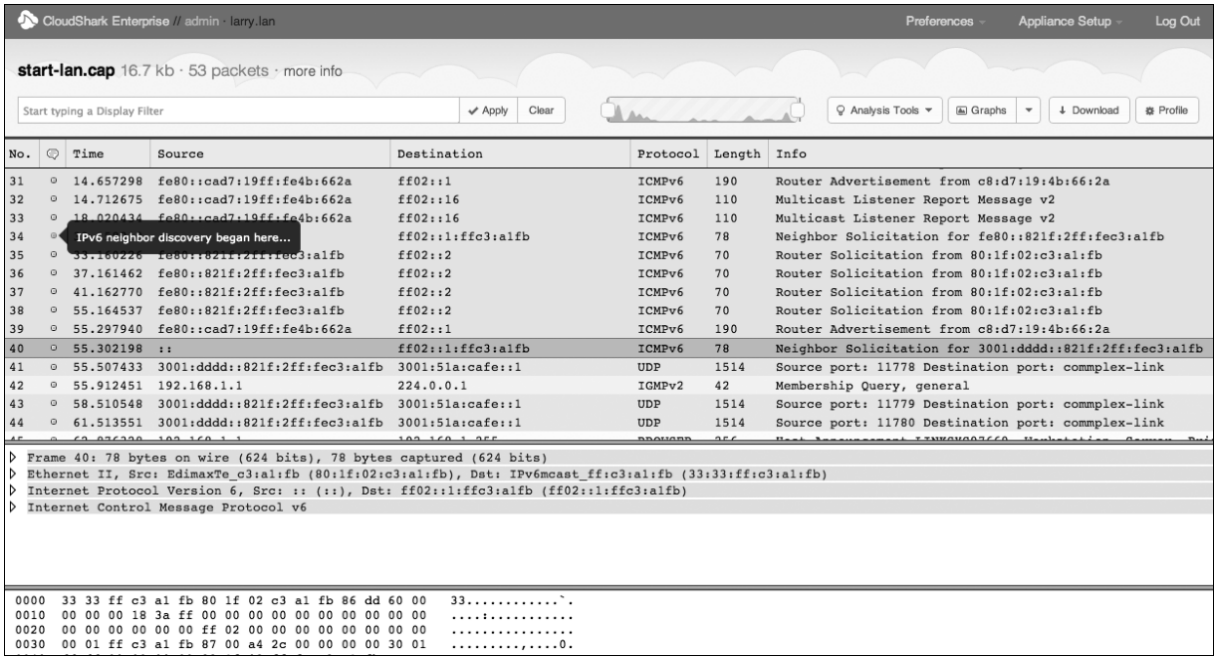


Рисунок A-1. Пример файла захвата, просматриваемого в CloudShark

Если вы или ваша организация храните большую библиотеку захватов пакетов либо, подобно мне, постоянно теряете свои файлы, CloudShark может помочь. Я развернул CloudShark в своей сети и использовал его для хранения и организации всех захватов этой книги. Подробнее о CloudShark можно узнать на сайте https://www.cloudshark.org/.

WireEdit

Для тестирования системы обнаружения вторжений, проверки на проникновение или разработки сетевого программного обеспечения может потребоваться создавать пакеты особого формата. Один вариант - воспроизвести в лаборатории сценарий, который создаст нужные пакеты, но это может занять много времени. Другой способ - найти похожий пакет и вручную изменить его в соответствии со своими потребностями. Мой любимый инструмент для этой задачи - WireEdit, графическое средство редактирования конкретных значений пакета. Его очень понятный пользовательский интерфейс похож на Wireshark. WireEdit даже пересчитывает контрольные суммы, чтобы при открытии в Wireshark пакеты не выглядели недействительными. Подробнее о WireEdit: https://wireedit.com/.

Cain & Abel

Рассмотренный в главе 2 Cain & Abel - один из лучших инструментов Windows для отравления кеша ARP. В действительности это очень мощный набор средств, которому вы наверняка найдёте и другие применения. Он доступен по адресу http://www.oxid.it/cain.html.

Scapy

Scapy - очень мощная библиотека Python, позволяющая создавать пакеты и манипулировать ими с помощью сценариев командной строки в её среде. Проще говоря, Scapy - самое мощное и гибкое доступное приложение для формирования пакетов. Подробнее о Scapy, её загрузке и примерах сценариев: http://www.secdev.org/projects/scapy/.

TraceWrangler

Захваты пакетов содержат много сведений о вашей сети. Если захват нужно передать поставщику или коллеге, возможно, вы не захотите раскрывать эти данные. TraceWrangler решает проблему, позволяя очищать захваты посредством анонимизации разных типов присутствующих адресов. В нём есть и другие функции, например редактирование и объединение файлов захвата, но я применяю его главным образом для очистки. Загрузить TraceWrangler можно с сайта https://www.tracewrangler.com/.

Tcpreplay

Когда мне нужно повторно передать в линию набор пакетов и посмотреть на реакцию устройства, я пользуюсь Tcpreplay. Этот инструмент специально предназначен для повторной передачи пакетов из файла захвата. Загрузить его можно по адресу http://tcpreplay.synfin.net/.

NetworkMiner

NetworkMiner главным образом применяется для сетевой криминалистики, но я считаю его полезным и во множестве других ситуаций. Хотя он умеет захватывать пакеты, настоящая его сила - разбор файлов захвата. NetworkMiner принимает PCAP и разбивает его по обнаруженным операционным системам и сеансам между узлами. Он даже позволяет извлекать переданные файлы прямо из захвата, как показано на рис. A-2. Все эти функции доступны в бесплатной версии; коммерческая предлагает несколько дополнительных полезных возможностей, например определение ОС по отпечатку, сравнение результатов с белым списком и повышение скорости обработки захвата. NetworkMiner можно бесплатно загрузить с сайта http://www.netresec.com/?page=NetworkMiner.

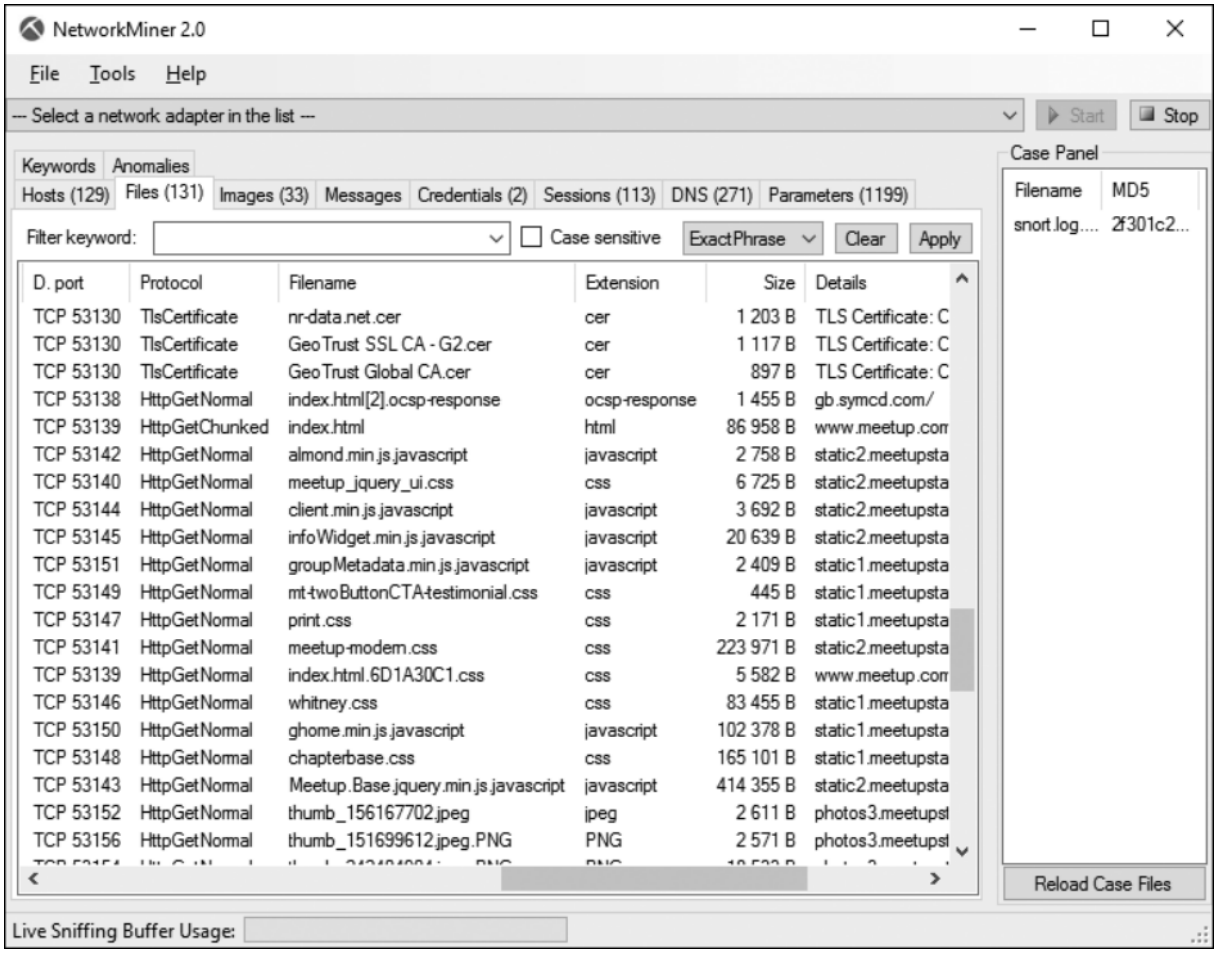


Рисунок A-2. Исследование файлов в захвате пакетов с помощью NetworkMiner

CapTipper

Надеюсь, из этой книги вы усвоили, что для поиска нужных ответов часто требуется взглянуть на те же данные по-другому. CapTipper - инструмент для специалистов по безопасности, которые анализируют вредоносный трафик HTTP, как показано на рис. A-3. Он предоставляет богатую функциями среду оболочки, где пользователь может интерактивно исследовать отдельные беседы, находить перенаправления, файловые объекты и вредоносное содержимое. В нём также есть несколько удобных функций для работы с найденными данными, включая извлечение данных gzip и отправку хешей файлов в VirusTotal. Загрузить CapTipper можно по адресу https://www.github.com/omriher/CapTipper/.

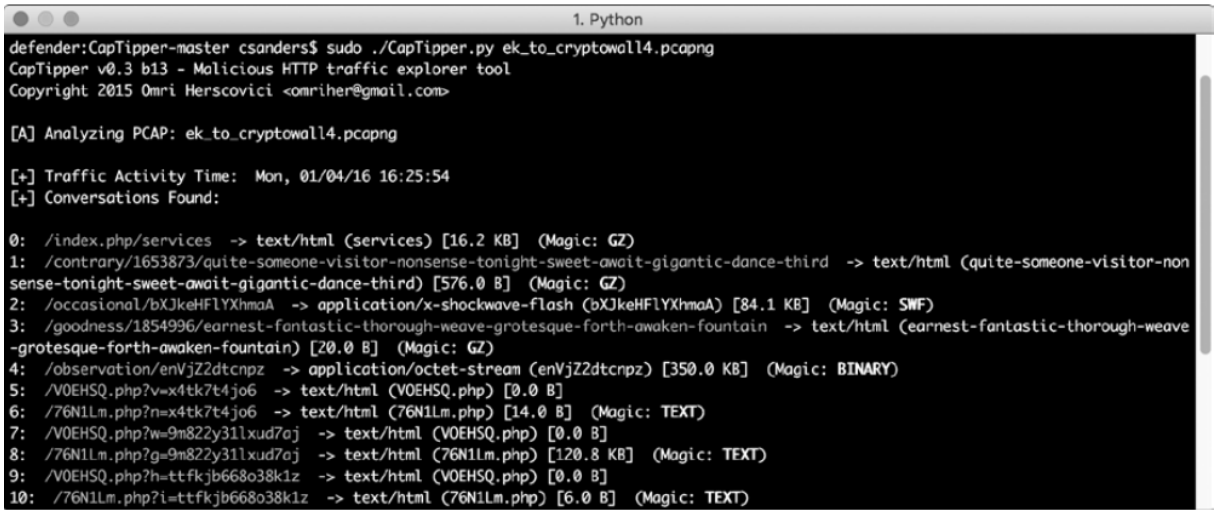


Рисунок А-3. Анализ доставки вредоносной программы через HTTP с помощью CapTipper

ngrep

Если вы знакомы с Linux, то наверняка использовали grep для поиска данных. ngrep похож на него и позволяет выполнять очень точные поисковые запросы по данным захвата пакетов. Я главным образом применяю ngrep, когда фильтры захвата и отображения не справляются с задачей или становятся чрезмерно сложными. Подробнее о ngrep: http://ngrep.sourceforge.net/.

libpcap

Если вы планируете выполнять сложный разбор пакетов или создавать работающие с ними приложения, библиотека libpcap станет вам очень хорошо знакома. Проще говоря, libpcap - переносимая библиотека C/C++ для захвата сетевого трафика. Wireshark, tcpdump и большинство других приложений анализа пакетов на том или ином уровне опираются на libpcap. Подробнее: http://www.tcpdump.org/.

Npcap

Npcap - основанная на WinPcap/libpcap библиотека проекта Nmap для перехвата пакетов в Windows. Сообщается, что она повышает производительность захвата и предоставляет дополнительные функции безопасности, связанные с ограничением захвата администраторами и применением контроля учётных записей Windows. Npcap можно установить вместо WinPcap и использовать с Wireshark. Подробнее: https://www.github.com/nmap/npcap/.

hping

hping - один из самых универсальных инструментов, которые можно иметь в своём арсенале. Это средство командной строки для формирования, редактирования и передачи пакетов. Оно поддерживает множество протоколов, работает очень быстро и интуитивно понятно. Загрузить hping можно по адресу http://www.hping.org/.

Python

Python - не инструмент, а язык сценариев, но он вполне заслуживает упоминания. По мере освоения анализа пакетов вам будут встречаться случаи, когда не существует автоматизированного средства для конкретной задачи. В таких ситуациях Python становится предпочтительным языком для создания инструментов, способных делать с пакетами интересные вещи. Кроме того, немного знать Python потребуется для работы с библиотекой Scapy. Мой любимый интернет-ресурс для изучения Python - популярная серия Learn Python the Hard Way: https://www.learnpythonthehardway.org/.

Ресурсы по анализу пакетов

Существует множество ресурсов по анализу пакетов, от главной страницы Wireshark до курсов и блогов. Перечислю несколько любимых.

Главная страница Wireshark

Главный ресурс обо всём, что связано с Wireshark, - его домашняя страница http://www.wireshark.org/. На ней есть ссылки на документацию, очень полезную вики с образцами файлов захвата и сведения о подписке на список рассылки Wireshark. На сайте https://ask.wireshark.org/ можно задавать вопросы о наблюдаемом в Wireshark и конкретных функциях. Это активное и очень отзывчивое сообщество.

Онлайн-курс Practical Packet Analysis

Если вам понравилась книга, может понравиться и дополняющий её учебный интернет-курс. В курсе Practical Packet Analysis вы сможете следовать за видеоматериалами, пока я разбираю все захваты книги и ещё несколько других. Я также предоставляю лабораторные работы с захватами для проверки навыков и форум, где по мере обучения можно перенимать опыт других студентов. Курс запускается в середине 2017 года. Подробнее о моих учебных предложениях: http://www.chrissanders.org/training/; подписаться на рассылку уведомлений о возможностях обучения можно здесь: http://www.chrissanders.org/list/.

Углублённый курс SANS по обнаружению вторжений

SANS SEC503: Intrusion Detection In-Depth сосредоточен на аспектах безопасности анализа пакетов. Даже если безопасность не является вашей специализацией, первые два дня курса дают прекрасное введение в анализ пакетов и tcpdump. Очные занятия проводятся несколько раз в год в разных странах мира.

Подробнее о SEC503 и других курсах SANS Institute: http://www.sans.org/.

Блог Криса Сандерса

Иногда я пишу статьи об анализе пакетов и публикую их в своём блоге http://www.chrissanders.org/. Блог также служит порталом со ссылками на другие мои статьи и книги и содержит контактные сведения. Там же находятся ссылки на захваты пакетов из этой и других книг.

Malware Traffic Analysis Брэда Дункана

Мой любимый источник связанных с безопасностью захватов пакетов - сайт Брэда Дункана Malware Traffic Analysis (MTA). Несколько раз в неделю Брэд публикует захваты с настоящими цепочками заражения. К ним прилагаются соответствующие двоичные файлы вредоносных программ и описание происходящего. Если вы хотите научиться разбирать заражения и познакомиться с современными методами вредоносных программ, начните с загрузки нескольких таких захватов и попытайтесь осмыслить их. Посетить MTA можно по адресу http://www.malware-traffic-analysis.net/, а получать уведомления об обновлениях - подписавшись на Брэда в Twitter: `@malware_traffic`.

Сайт IANA

Управление по присвоению номеров в Интернете (Internet Assigned Numbers Authority, IANA), сайт которого находится по адресу http://www.iana.org/, контролирует распределение IP-адресов и назначение номеров протоколов в Северной Америке. На сайте доступны ценные справочные инструменты: поиск номеров портов, сведения о доменных именах верхнего уровня и ссылки на сопутствующие сайты для поиска и просмотра RFC.

Серия TCP/IP Illustrated У. Ричарда Стивенса

Серия У. Ричарда Стивенса *TCP/IP Illustrated* (Addison-Wesley, 1994-1996), которую многие считают библией TCP/IP, обязательно стоит на полках большинства людей, живущих на уровне пакетов. Это мои любимые книги о TCP/IP, и при написании этой книги я часто обращался к этим томам. Второе издание тома 1, написанное совместно с доктором Кевеном Р. Фоллом, опубликовано в 2012 году.

The TCP/IP Guide

Книга Чарльза Козиерока *The TCP/IP Guide* (No Starch Press, 2005) - ещё один справочный ресурс о протоколах TCP/IP. В ней более 1600 страниц, она очень подробна и содержит множество прекрасных схем для людей с визуальным восприятием.

Приложение B. Навигация по пакетам

В этом приложении мы исследуем способы представления пакетов. Рассмотрим полностью интерпретированное и шестнадцатеричное представления пакетов, а также чтение значений пакета и обращение к ним с помощью схемы пакета.

Существует множество программ, способных интерпретировать данные пакетов, поэтому заниматься перехватом и анализом можно, даже не понимая сведений из этого приложения. Но если вы потратите время на изучение данных пакетов и их структуры, то будете гораздо лучше понимать, что показывают такие инструменты, как Wireshark. Чем меньше абстракций отделяет вас от анализируемых данных, тем лучше.

Представление пакета

Пакет можно представить для интерпретации множеством способов. Необработанные данные пакета могут быть представлены в двоичном виде, то есть сочетанием нулей и единиц в системе счисления с основанием 2:

```
01100000010100110101110000001010110000010000000000010000000000000010001100000010
11010101101110000000000000000000000000000000000000000000000001000000100000000001011
01101000000000000010000001100000000000000000000000010000000100000000010000000010
```

Двоичные числа представляют цифровые сведения на самом низком возможном уровне: 1 означает наличие электрического сигнала, а 0 - его отсутствие. Каждая цифра является битом, восемь битов составляют байт. Однако двоичные данные трудно читать и интерпретировать человеку, поэтому обычно их преобразуют в шестнадцатеричный вид - сочетание букв и цифр в системе счисления с основанием 16. Тот же пакет в шестнадцатеричном виде выглядит так:

```
4500 0034 40f2 4000 8006 535c ac10 1080
4a7d 5f68 0646 0050 7c23 5ab7 0000 0000
8002 2000 0b30 0000 0204 05b4 0103 0302
0101 0402
```

Шестнадцатеричная система, также называемая hex, использует цифры от 0 до 9 и буквы от A до F для представления значений. Это один из самых распространённых способов представления пакетов, поскольку он компактен и легко преобразуется в ещё более фундаментальный двоичный вид. В hex два символа представляют байт из восьми битов. Каждый символ внутри байта является полубайтом (nibble) из четырёх битов: левое значение - старший полубайт, правое - младший. В нашем примере первый байт равен 45, старший полубайт - 4, младший - 5.

Положение байтов в пакете представляется смещениями, начиная с нуля. Поэтому первый байт пакета, 45, находится в позиции `0x00`, второй, 00, - в `0x01`, третий, 00, - в `0x02` и так далее. Часть `0x` сообщает, что используется шестнадцатеричная запись. При обращении к позиции, занимающей несколько байтов, число дополнительных байтов указывается цифрой после двоеточия. Например, для обозначения первых четырёх байтов пакета `4500 0034` используется запись `0x00:4`. Это объяснение будет важно, когда в разделе "Навигация по загадочному пакету" на странице 330 книги мы применим схемы пакетов для разбора неизвестных протоколов.

> **Примечание.** Самая частая ошибка при попытке разобрать пакет - забыть начать отсчёт с нуля. К этому очень трудно привыкнуть, поскольку большинство людей учат считать с единицы. Я уже много лет режу пакеты на части, но всё равно совершаю эту ошибку. Лучший совет - не бойтесь считать на пальцах. Возможно, это выглядит глупо, но в этом совершенно нет ничего постыдного, особенно если способ помогает получить правильный ответ.

На гораздо более высоком уровне такой инструмент, как Wireshark, способен представить пакет в полностью интерпретированном виде с помощью анализатора протокола, который мы вскоре обсудим. Тот же только что рассмотренный пакет показан на рис. B-1 в полной интерпретации Wireshark.

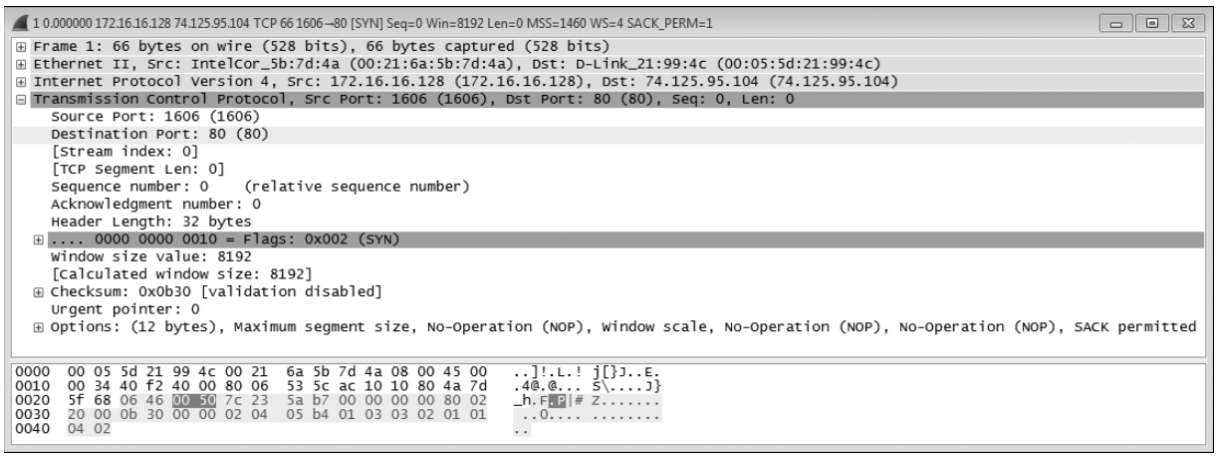


Рисунок B-1. Пакет, интерпретированный Wireshark

Wireshark показывает сведения пакета с описывающими их метками. Сами пакеты меток не содержат, но их данные соответствуют точному формату, заданному стандартом протокола. Полностью интерпретировать пакет означает прочитать данные согласно стандарту протокола и разобрать их на снабжённый метками, понятный человеку текст.

Wireshark и похожие инструменты способны полностью интерпретировать данные пакетов, потому что в них встроены анализаторы протоколов, определяющие положение, длину и значения каждого поля протокола. Например, пакет на рис. B-1 разбит на разделы на основании протокола управления передачей (Transmission Control Protocol, TCP). Внутри TCP находятся поля и значения с метками. Source Port - одна из меток, а 1606 - её десятичное значение. Благодаря этому при анализе легко найти нужные сведения. Когда такая возможность доступна, обычно это самый эффективный способ выполнить задачу.

Wireshark содержит тысячи анализаторов, но вам могут встретиться протоколы, которые он не умеет интерпретировать. Часто это специализированные протоколы производителей, не имеющие широкого распространения, и пользовательские протоколы вредоносных программ. В таком случае останутся лишь частично интерпретированные пакеты. Именно поэтому Wireshark по умолчанию показывает в нижней части экрана необработанные шестнадцатеричные данные пакета, как на рис. B-1.

Ещё чаще программы командной строки вроде tcpdump, показывающие необработанный hex, располагают гораздо меньшим числом анализаторов. Особенно это касается более сложных протоколов прикладного уровня, которые труднее разбирать. Поэтому при использовании такого инструмента частично интерпретированные пакеты являются нормой. Пример tcpdump показан на рис. B-2.

При работе с частично интерпретированными пакетами придётся полагаться на более фундаментальные знания их структуры. Wireshark, tcpdump и большинство других инструментов

помогают в этом, показывая необработанные данные пакета в формате hex.

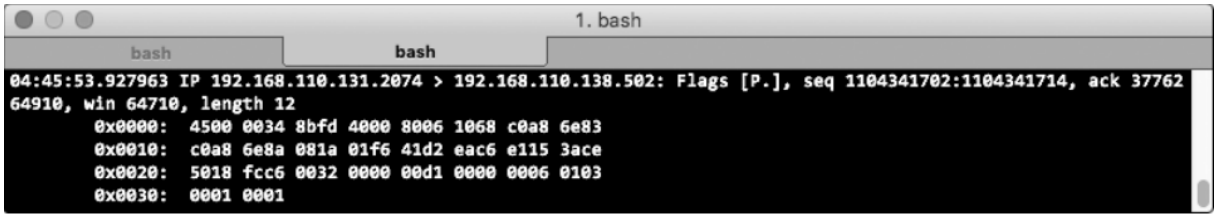


Рисунок B-2. Частично интерпретированные пакеты из tcpdump

Использование схем пакетов

Как мы узнали в главе 1, пакет представляет данные, формат которых основан на правилах протоколов. Распространённые протоколы форматируют данные пакетов определённым образом, чтобы аппаратное и программное обеспечение могло их интерпретировать, поэтому пакеты обязаны соблюдать явные правила формата. Выявить этот формат и использовать его для интерпретации данных позволяют схемы пакетов. Схема пакета - графическое представление, с помощью которого аналитик сопоставляет байты внутри пакета с полями определённого протокола. Она создаётся на основании документа спецификации RFC протокола и показывает присутствующие поля, их длину и порядок.

Ещё раз рассмотрим пример схемы пакета IPv4 из главы 7, повторно приведённый для удобства на рис. B-3.

Internet Protocol Version 4 (IPv4)							
Offsets	Octet	0		1	2		3
Octet	Bit	0–3	4–7	8–15	16–18	19–23	24–31
0	0	Version	Header Length	Type of Service	Total Length		
4	32	Identification			Flags	Fragment Offset	
8	64	Time to Live		Protocol	Header Checksum		
12	96	Source IP Address					
16	128	Destination IP Address					
20	160	Options					
24+	192+	Data					

Рисунок B-3. Схема пакета IPv4

На этой схеме горизонтальная ось представляет отдельные двоичные биты с номерами от 0 до 31. Биты объединены в восьмибитные байты с номерами от 0 до 3. Вертикальная ось также размечена по битам и байтам, а каждая строка разделена на 32-битные, или четырёхбайтовые, фрагменты. По осям мы подсчитываем позиции полей в записи смещений: сначала по вертикальной оси определяем четырёхбайтовый фрагмент, где находится поле, а затем по горизонтальной оси отсчитываем каждый байт внутри него. Первая строка состоит из первых четырёх байтов с 0 по 3, соответствующим образом обозначенных на горизонтальной оси. Вторая строка состоит из следующих четырёх байтов с 4 по 7, которые также можно отсчитать по горизонтальной оси. Здесь мы начинаем с байта 4, являющегося байтом 0 на горизонтальной оси, затем берём байт 5, соответствующий байту 1 на горизонтальной оси, и так далее.

Например, можно определить, что в IPv4 байт `0x01` представляет поле Type of Service, поскольку мы начинаем со смещения 0 и считаем до байта 1. На вертикальной оси первые четыре байта

находятся в первой строке, после чего по горизонтальной оси мы считаем от 0 до байта 1. Другой пример: байт `0x08` представляет поле Time to Live. По вертикальной оси определяем, что байт 8 находится в третьей сверху строке, содержащей байты с 8 по 11. Затем по горизонтальной оси считаем до байта 8, начиная с 0. Поскольку байт 8 является первым во фрагменте, столбец горизонтальной оси равен 0 - это и есть поле Time to Live.

Некоторые поля, например Source IP, занимают несколько байтов, как видно из `0x12:4`. Другие разделены на полубайты. Например, `0x00` содержит поле Version в старшем полубайте и IP Header Length в младшем. Байт `0x06` имеет ещё большую детализацию: для представления отдельных полей используются отдельные биты. Когда поле является единственным двоичным значением, его часто называют флагом. Примеры - поля Reserved, Don't Fragment и More Fragments заголовка IPv4. Флаг может иметь только двоичное значение 1, истина, или 0, ложь, поэтому при значении 1 говорят, что флаг установлен. Точное значение установленного флага зависит от протокола и поля.

> **Примечание переводчика.** В исходном тексте для поля Source IP указано смещение `0x12:4`; это значение сохранено дословно.

Рассмотрим ещё один пример на рис. B-4, схему которого вы можете узнать по главе 8.

Transmission Control Protocol (TCP)						
Offsets	Octet	0		1	2	3
Octet	Bit	0–3	4–7	8–15	16–23	24–31
0	0	Source Port			Destination Port	
4	32	Sequence Number				
8	64	Acknowledgment Number				
12	96	Data Offset	Reserved	Flags	Window Size	
16	128	Checksum			Urgent Pointer	
20+	160+	Options				

Рисунок B-4. Схема пакета TCP

На изображении показан заголовок TCP. По нему можно ответить на множество вопросов о пакете TCP, даже не зная точно, что делает TCP. Рассмотрим пример заголовка пакета TCP в шестнадцатеричном представлении:

```
0646 0050 7c23 5ab7 0000 0000 8002 2000
0b30 0000 0204 05b4 0103 0302 0101 0402
```

По схеме пакета можно найти и интерпретировать определённые поля. Например, мы способны установить следующее:

• Номер Source Port находится в `0x00:2` и имеет шестнадцатеричное значение `0646`, десятичное 1606.
• Номер Destination Port находится в `0x02:2` и имеет шестнадцатеричное значение `0050`, десятичное 80.
• Длина заголовка находится в поле Data Offset, в старшем полубайте `0x12`, и имеет шестнадцатеричное значение `8`.

Применим эти знания, разобрав загадочный пакет.

Навигация по загадочному пакету

На рис. B-2 я показал пакет, интерпретированный лишь частично. По интерпретированной части данных можно установить, что это пакет TCP/IP, переданный между двумя устройствами одной сети, но кроме этого о передаваемых данных почти ничего не известно. Вот полный шестнадцатеричный вывод пакета:

```
4500 0034 8bfd 4000 8006 1068 c0a8 6e83
c0a8 6e8a 081a 01f6 41d2 eac6 e115 3ace
5018 fcc6 0032 0000 00d1 0000 0006 0103
0001 0001
```

Быстрый подсчёт показывает, что в пакете 52 байта. Схема IP сообщает, что нормальный размер заголовка IP равен 20 байт; это подтверждается значением размера заголовка в младшем полубайте `0x00`. Схема TCP говорит, что его заголовок без дополнительных параметров также занимает 20 байт; здесь их нет, но подробнее параметры TCP обсуждаются в главе 8. Следовательно, первые 40 байт вывода относятся к уже интерпретированным данным TCP и IP, а оставшиеся 12 байт не интерпретированы:

```
00d1 0000 0006 0103 0001 0001
```

Без умения ориентироваться в пакетах это могло бы поставить вас в тупик, но теперь вы знаете, как применить схему пакета к неинтерпретированным байтам. Интерпретированные данные TCP сообщают, что порт назначения равен 502. Анализ используемых трафиком портов - не безошибочный метод идентификации неинтерпретированных байтов, но это хорошее начало. Быстрый поиск в Google показывает, что порт 502 чаще всего используется Modbus поверх TCP - протоколом сетей промышленных систем управления (Industrial Control System, ICS). Проверить это предположение и разобрать пакет можно, сравнив hex-вывод со схемой Modbus на рис. B-5.

Modbus over TCP					
Offsets	Octet	0	1	2	3
Octet	Bit	0–7	8–15	16–23	24–31
0	0	Transaction Identifier		Protocol Identifier	
4	32	Length		Unit Identifier	Function Code
8+	64+	Variable			

Рисунок B-5. Схема пакета Modbus поверх TCP

Эта схема создана на основании сведений из руководства по реализации Modbus: http://www.modbus.org/docs/Modbus_Messaging_Implementation_Guide_V1_0b.pdf. Оно сообщает, что должен присутствовать семибайтовый заголовок с полем Length в позиции `0x04:2` относительно начала заголовка. Отсчитав до неё, получаем шестнадцатеричное значение `0006`, или десятичное 6, указывающее, что за полем должны следовать шесть байтов. Именно это и наблюдается. Похоже, перед нами действительно данные Modbus поверх TCP.

Сравнение схемы пакета со всем шестнадцатеричным выводом даёт следующие сведения:

• Transaction Identifier находится в `0x00:2` и имеет шестнадцатеричное значение `00d1`. Это поле сопоставляет запрос с ответом.
• Protocol Identifier находится в `0x02:2` и имеет шестнадцатеричное значение `0000`. Оно определяет протокол как Modbus.
• Length находится в `0x04:2` и имеет шестнадцатеричное значение `0006`. Оно задаёт длину данных пакета.

• Unit Identifier находится в `0x06` и имеет шестнадцатеричное значение `01`. Это поле применяется для внутрисистемной маршрутизации.
• Function Code находится в `0x07` и имеет шестнадцатеричное значение `03`. Это функция Read Holding Registers, считывающая значение данных из системы.
• Исходя из значения кода функции 3, ожидаются ещё два поля данных. Reference Number и Word Count находятся в `0x08:4`, каждое имеет шестнадцатеричное значение `0001`.

Теперь загадочный пакет можно полностью объяснить в контексте протокола Modbus. Если бы вы устраняли неполадки отвечающей за него системы, этих сведений должно было бы хватить для продолжения работы. Даже если вы никогда не столкнётесь с Modbus, этот пример показывает подход к неизвестному протоколу и неинтерпретированному пакету с помощью схемы.

Всегда полезно осознавать уровень абстракции между вами и анализируемыми данными. Это помогает принимать более обоснованные и осведомлённые решения и позволяет работать с пакетами в самых разных ситуациях. Я оказывался во множестве сценариев, когда мог использовать для анализа пакетов только инструменты командной строки, такие как tcpdump. Поскольку в большинстве подобных инструментов нет анализаторов многих протоколов уровня 7, умение вручную разбирать определённые байты пакетов оказывалось критически важным.

> **Примечание.** Однажды моему коллеге пришлось участвовать в реагировании на инцидент в среде с высоким уровнем безопасности. У него был допуск к просмотру необходимых данных, но не к конкретной системе, где они хранились. За имевшееся время удалось лишь распечатать пакеты определённых бесед. Благодаря фундаментальному знанию строения пакетов и умению ориентироваться в них он смог найти нужные сведения в распечатанных данных. Разумеется, процесс был медленнее холодной патоки, стекающей по замёрзшей ветке. Это крайний случай, но он прекрасно показывает важность универсальных знаний, не зависящих от инструмента.

По всем этим причинам полезно тратить время на разбор пакетов, чтобы получить опыт просмотра разных представлений. Я делаю это так часто, что распечатал несколько распространённых схем пакетов, заламинировал их и держу рядом со столом. Кроме того, я храню цифровую версию на ноутбуке и планшете, чтобы быстро обращаться к ней в поездках. Для удобства я включил несколько распространённых схем пакетов в ZIP-файл с захватами к этой книге: https://www.nostarch.com/packetanalysis3/.

Заключение

В этом приложении мы узнали, как интерпретировать данные пакетов в разных форматах и применять схемы для навигации по неинтерпретированным данным. Обладая этими фундаментальными знаниями, вы без труда поймёте, как разбирать пакеты независимо от инструмента, используемого для просмотра их данных.